

Algoritma Trading Multi-strategi

User: Anonymous

Created: 8/17/2026 20:00:17

Updated: 8/18/2026 11:50:47

Exported: 8/18/2026 12:02:10

Link: <https://chatgpt.com/c/6a8305e1-25a4-83ec-bfaf-49a4769a569f>

Prompt:

8/17/2026, 8:00:03 PM

<https://github.com/tabassam823/TradeProject.git>

bisakah kau melihat isi dari repository ini? bisakah kau menjelaskan padaku algoritma utama yang bekerja pada kode yang ada pada repo tersebut?

Response:

8/17/2026, 8:00:17 PM

Searching the web

Searching github.com

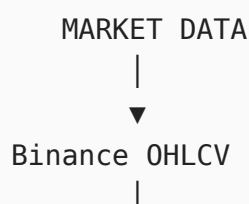
Worked for 10s

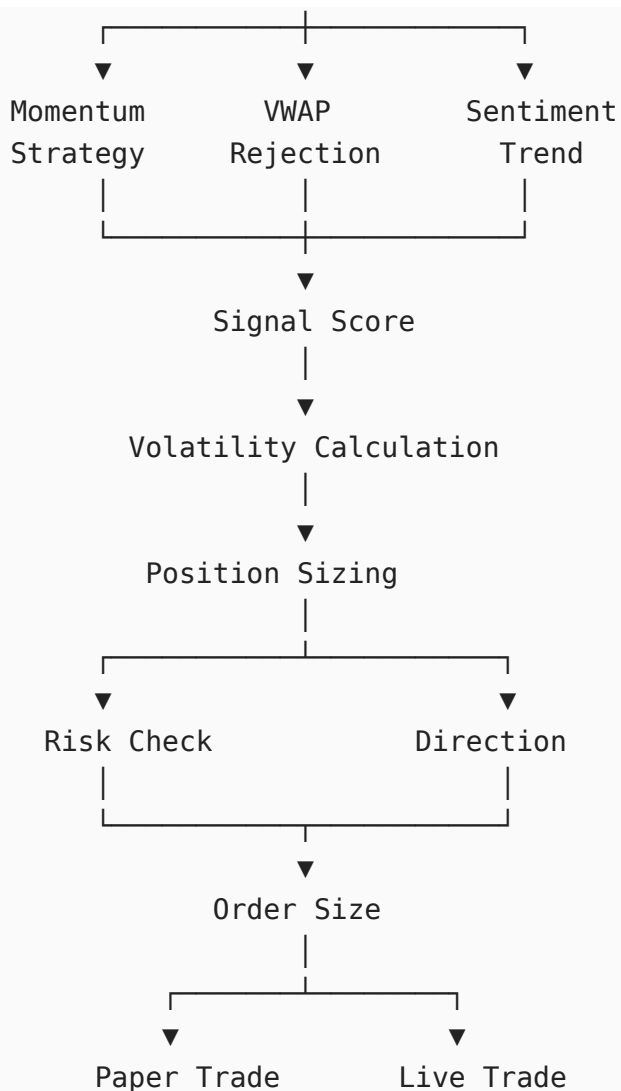
Ya. Aku bisa melihat repository tersebut, dan aku sudah menelusuri struktur utama serta kode strategi di dalamnya. Repository ini adalah [tabassam823/TradeProject](https://github.com/tabassam823/TradeProject), dan saat ini branch `main` berisi engine backtesting, paper trading, live trading, tiga strategi, position sizing berbasis volatilitas, serta integrasi Binance dan OpenBB. ([GitHub](https://github.com))

Yang menarik: **kode aktualnya tidak persis sama dengan narasi "AI self-improving trading agent" yang ada di `workflow.md`**. Implementasi trading yang benar-benar berjalan saat ini lebih tepat disebut **multi-strategy quantitative trading system dengan volatility-targeted risk management**. ([GitHub](https://github.com))

1. Gambaran besar algoritmanya

Secara konseptual, alurnya adalah:





Ada tiga lapisan yang sebaiknya kamu pisahkan secara mental:

1. **Signal generation** → "Apakah saya long/short?"
2. **Risk & position sizing** → "Seberapa besar posisi saya?"
3. **Execution** → "Bagaimana order dikirim ke Binance?"

Ini penting karena **algoritma trading-nya bukan sekadar satu indikator**.

2. Tiga strategi utama

`strategy_config.json` mengaktifkan tiga strategi:

Strategi	Ide utama	Signal
Multi-Horizon Momentum	Trend-following	-4 sampai +4
VWAP Rejection	Mean reversion	-2, 0, +2
Sentiment-Filtered Trend	EMA + news sentiment	-3, 0, +3

Konfigurasi saat ini menggunakan horizon momentum `[5, 10, 21, 42]` , VWAP window 24 candle dengan $\pm 2\sigma$, serta EMA 10/30 untuk sentiment trend. ([GitHub](#))

3. Strategi #1 - Multi-Horizon Momentum

Ini menurutku merupakan **strategi paling sederhana sekaligus paling penting** di repository.

Kode:

[src/strategies/momentum.py](#)

Algoritmanya melihat empat horizon:

$$\tau \in \{5, 10, 21, 42\}$$

Kemudian untuk masing-masing horizon dihitung:

$$\Delta P_{\tau} = P_t - P_{t-\tau}$$

Lalu:

$$\tau = 1, \quad P_t - P_{t-\tau} - 1, \quad P_t - P_{t-\tau} 0, \quad P_t = P_{t-\tau}$$

Kemudian semua signal dijumlahkan:

$$= \sum_{\tau \in \{5, 10, 21, 42\}} \tau$$

Sehingga:

$$\in -4, 4$$

Implementasinya memang persis seperti itu: setiap horizon memberikan `+1` , `-1` , atau `0` , kemudian dijumlahkan. ([GitHub](#))

Contoh

Misalnya:

```
5 candle → harga naik → +1
10 candle → harga naik → +1
21 candle → harga turun → -1
42 candle → harga naik → +1
```

Maka:

$$= 1 \ 1 - 1 \ 1 = 2$$

Artinya bias sistem adalah **long**.

Jadi ini bukan machine learning.

Ini adalah **multi-time-horizon trend consensus**.

4. Strategi #2 - VWAP Mean Reversion

Kode:

[src/strategies/vwap_rejection.py](#)

Ini jauh lebih menarik secara matematis.

Pertama dihitung typical price:

$$P_t = \frac{1}{3} (P_{t-1} + P_t + P_{t+1})$$

Kemudian VWAP:

$$P = \frac{\sum P_{tt}}{\sum t}$$

Kemudian deviasi standar berbobot volume:

$$= \sqrt{\frac{\sum (P_t - P)^2}{\sum t}}$$

Kemudian dibuat dua batas:

$$= P + 2\sigma$$

$$= P - 2\sigma$$

Karena konfigurasi menggunakan:

```
vwap_window = 24
num_std = 2
```

maka sistem melihat area $\pm 2\sigma$ di sekitar VWAP. ([GitHub](#))

Entry long

Jika:

$$P_{t-1} < P - 2\sigma$$

tetapi kemudian:

$$P_t > P - 2\sigma$$

maka:

$$= 2$$

Interpretasinya:

harga sebelumnya berada di bawah lower value area, lalu kembali masuk → dianggap rejection → long.

Sebaliknya:

$$P_{t-1}$$

dan:

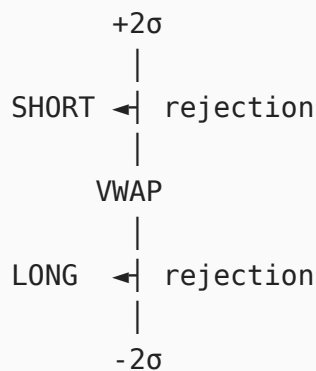
$$P_t$$

memberikan:

$$= -2$$

yaitu short.

Jadi secara konsep:



Ini **mean-reversion strategy**, bukan trend following.

5. Strategi #3 - Sentiment-Filtered Trend

Kode:

[src/strategies/sentiment_trend.py](#)

Di sini digunakan dua EMA:

$$t = 10$$

dan

$$= 0$$

Kemudian sistem mengambil sentiment berita melalui OpenBB.

Untuk long:

$$10 \ 0$$

dan:

$$tt \ -02$$

maka:

$$=$$

Untuk short:

$$10 \ 0$$

dan:

$$tt \ 02$$

maka:

$$= \ -$$

Jika tidak memenuhi kondisi:

$$= 0$$

([GitHub](#))

Jadi ide dasarnya:

Technical trend + fundamental/news confirmation.

6. Bagian yang menurutku paling penting: Position Sizing

Justru menurutku **bagian paling "quantitative"** dari repository ini bukan signal generation-nya, tetapi risk engine-nya.

Kode:

[src/core/position_sizer.py](#)

Volatilitas dihitung dari return:

$$r_t = P_t - P_{t-1}$$

kemudian:

$$\sigma_t = \sigma_{t-20}, \dots, \sigma_t$$

dengan lookback 21 candle. ([GitHub](#))

Kemudian volatilitas digunakan untuk menentukan ukuran posisi.

Ini berarti:

| signal menentukan arah, volatilitas menentukan seberapa agresif posisi.

7. Risk budget

Ada mekanisme tiered risk:

Jika modal:

$$100$$

maka:

$$= 1$$

Jika:

$$100$$

maka:

$$= 0.01$$

Jadi dengan modal \$10,000:

$$= 0.01 \times 10000 = 100$$

Artinya sistem mencoba membatasi kerugian per trade sekitar \$100. ([GitHub](#))

Ini jauh lebih masuk akal daripada:

| "Saya punya signal kuat → masuk 100% modal."

8. Stop loss dan take profit juga dihitung matematis

Ini bagian yang sangat menarik.

Repository menggunakan konsep expected value:

$$= -1 -$$

secara konseptual, dengan payoff ratio minimum:

$$= 1 -$$

Kode kemudian mengambil:

$$= 12, 15$$

Kemudian:

$$= 15,1$$

dan:

$$P =$$

([GitHub](#))

Jadi sistem tidak sekadar mengatakan:

| "Stop loss = 2%."

Tetapi secara prinsip:

$$\boxed{P}$$

9. Position size kemudian diturunkan dari risk

Misalkan:

- harga entry = P
- stop loss = P
- risiko yang diizinkan =

Jarak stop:

$$= P - P$$

Jumlah unit:

$$=$$

Itulah yang dilakukan kode:

```
units = risk_dollar / sl_distance
```

([GitHub](#))

Contoh sederhana:

$$P = 100$$

$$P =$$

sehingga:

$$= 2$$

Jika risiko:

$$= 100$$

maka:

$$= 1002 = 50$$

Jika stop loss tersentuh:

$$50 \cdot 2 = 100$$

Jadi **position sizing berasal dari risk budget, bukan dari confidence signal semata.**

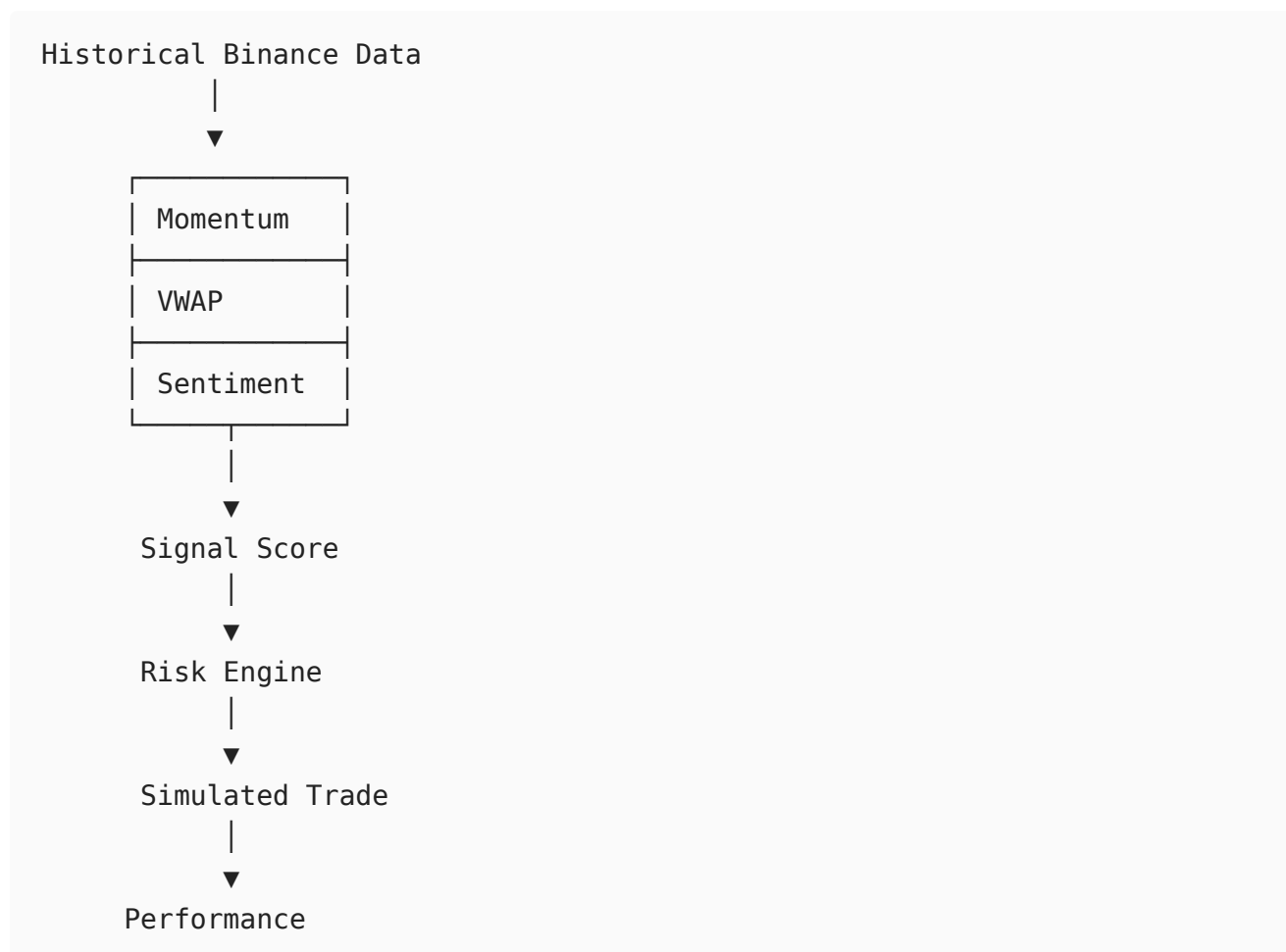
10. Backtesting

Backtester utamanya ada di:

[src/backtester.py](#)

Ia mengambil historical OHLCV dari Binance, kemudian menjalankan ketiga strategi tersebut. ([GitHub](#))

Strukturnya:



Yang perlu diperhatikan:

backtester mengevaluasi strategi secara terpisah, bukan sebuah neural network yang belajar dari hasil backtest.

11. Paper trading

Paper trader menggunakan data Binance secara real-time dan mensimulasikan eksekusi. Kode bahkan menjelaskan bahwa engine ini mengambil ticker real-time setiap jam dan menyimpan virtual ledger ke `logs/paper_ledger.json`. ([GitHub](#))

Jadi:

Tidak ada uang sungguhan.

12. Live trading

Live trader menghubungkan:

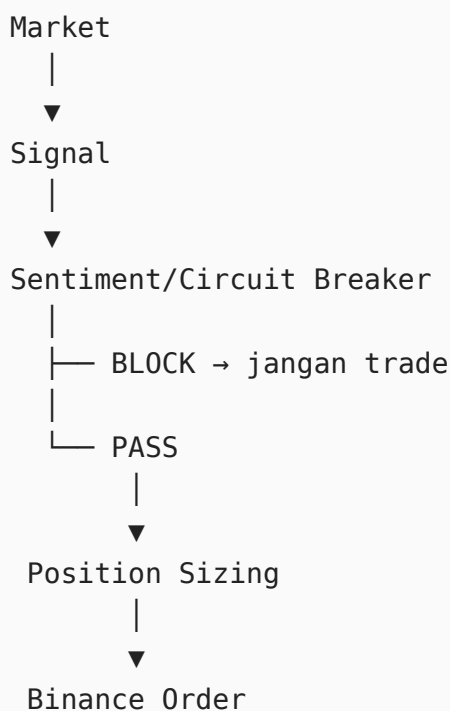
```

Binance API
+
OpenBB
+
Position Sizer
+
Momentum Strategy

```

dan kemudian dapat mengirim order ke Binance. ([GitHub](#))

Menariknya, ada **circuit breaker** berbasis OpenBB sentiment:



Tetapi ada satu hal yang sangat penting:

LiveTrader saat ini hanya memilih MultiHorizonMomentumStrategy

Di constructor-nya terlihat:

```
self.strategy = MultiHorizonMomentumStrategy(...)
```

Jadi walaupun konfigurasi memiliki tiga strategi, **live execution tidak menjalankan ketiganya sebagai ensemble.** ([GitHub](#))

Ini perbedaan penting antara:

"repository memiliki tiga strategi"

dan:

"live trading menjalankan tiga strategi."

Yang kedua **belum sepenuhnya benar** berdasarkan kode saat ini.

13. Lalu apakah ini "AI Trading Agent"?

Menurutku: belum dalam arti machine-learning/AI agent yang kuat.

Ini lebih tepat disebut:



dengan komponen:



Memang `workflow.md` berbicara tentang:

- self-improving agent
- feedback loop
- autonomous optimization
- perubahan parameter
- scientific method
- Sharpe ratio
- AI agent

([GitHub](#))

Tetapi **kode trading yang sekarang tidak menunjukkan adanya optimizer/ML loop yang benar-benar mengubah parameter strategi berdasarkan hasil trading.**

Dengan kata lain:

DOKUMENTASI

AI Agent

↓

Self Improvement

↓

Change Parameters

↓

Evaluate

↓

New Baseline

sedangkan implementasi aktual lebih dekat ke:

Fixed Parameters

↓

Rule-Based Strategy

↓

Signal

↓

Risk Management

↓

Trade

Itu perbedaan yang cukup besar.

14. Jadi algoritma utamanya sebenarnya apa?

Kalau aku harus merangkum repository ini dalam satu persamaan konseptual:

$$= , , , ,$$

Lebih detail:

dengan:

=

dan:

=

15. Ada satu hal yang menurutku sangat relevan dengan proyekmu

Kalau tujuanmu adalah **mengembangkan repository ini lebih lanjut**, terutama mengingat kita sebelumnya membahas algoritma trading otomatis dan QAOA/VQE, maka repository ini sebenarnya menyediakan tempat yang sangat jelas untuk memasukkan algoritma optimasi yang lebih canggih.

Saat ini:

$\square$

Tetapi bisa dikembangkan menjadi:

$\square$

Misalnya tiga strategi menghasilkan:

$=$

Kemudian kita mencari bobot:

$= 12$

sehingga:

$= 1 \ 2$

Nah, di sinilah sebenarnya QAOA/VQE atau metode optimasi lainnya bisa masuk **secara natural**-bukan menggantikan indikator, tetapi mengoptimalkan **allocation/weighting/portfolio decision** di atas signal yang sudah tersedia.

Dan itu menurutku jauh lebih menarik daripada sekadar mengganti salah satu indikator dengan "AI".

Kalau kamu mau, langkah berikutnya yang paling berguna adalah **aku bedah `backtester.py` baris demi baris dan turunkan seluruh persamaan matematis yang sebenarnya dipakai repository ini**, termasuk bagaimana `signal score` → `volatility` → `SL/TP` → `leverage` → `position size` → `PnL` bekerja. Itu akan memberi kita gambaran yang sangat jelas tentang **algoritma trading sebenarnya**, bukan hanya dokumentasinya. ([GitHub](#))

Sources:

Prompt:

8/17/2026, 8:00:47 PM

aku ingin bertanya lebih lanjut mengenai quantitative trading yang semestinya dilakukan oleh hedge fund dan prop firm tapi dengan matematika yang lebih lanjut. maksudku adalah optimasi yang kasusnya TSP, maxcut, portofolio optimization, dst. tapi digunakan untuk trading jangka pendek? mungkin bayanganku seperti ini

1. menggunakan data training harga histori aset yang ingin ditradingkan dan beberapa harga aset yang berhubungan seperti jika ingin membeli altcoin dengan marketcap menengah, maka harus melihat harga btc dan eth (bertujuan untuk menentukan money flow dari aset-aset yang berkaitan)
2. hasil data training digunakan untuk meletakkan order book (tidak langsung eksekusi) berikut TP dan SL sesuai dengan algoritma yang sudah ada agar tetap mengikuti resiko \$1
3. setiap jam atau periode tertentu, posisi dievaluasi apakah masih bisa dipertahankan jika order book belum terpanggil? atau jika order book sudah terpanggil, ada 2 kondisi : jika floating loss, biarkan saja tapi jika floating profit, apakah setup nya layak dipertahankan atau harus setting ulang stop loss nya
4. jika stop loss sudah diletakkan di BEP atau titik tertentu sesuai dengan algoritma dan ternyata masih floating profit tinggi, mungkin bisa dipertimbangan TP nya dinaikkan lagi

Response:

8/17/2026, 8:00:48 PM

Personalizing

Personalized quantitative trading plans around Binance automation and risk controls

Worked for 5s

Ya. Dan menurutku bayanganmu **sudah mengarah ke sesuatu yang jauh lebih menarik daripada sekadar "bot trading dengan indikator"**.

Yang kamu gambarkan sebenarnya bisa diformulasikan sebagai **short-horizon quantitative trading system dengan beberapa layer optimasi**:



Namun ada satu koreksi penting: **TSP, MaxCut, dan QUBO tidak harus dipaksakan langsung menjadi "algoritma untuk menentukan buy/sell"**. Justru penggunaan yang

lebih natural adalah mengubah bagian-bagian tertentu dari masalah trading menjadi masalah optimasi kombinatorial.

Dan menurutku, kalau kita ingin membangun ini di atas `TradeProject` , arsitektur yang bagus adalah seperti berikut.

1. Gambaran sistem yang kamu bayangkan

Misalnya kita ingin trading:

=

tetapi tidak hanya menggunakan data SOL.

Kita masukkan:

$$t = P, P, P, , , , , ,$$

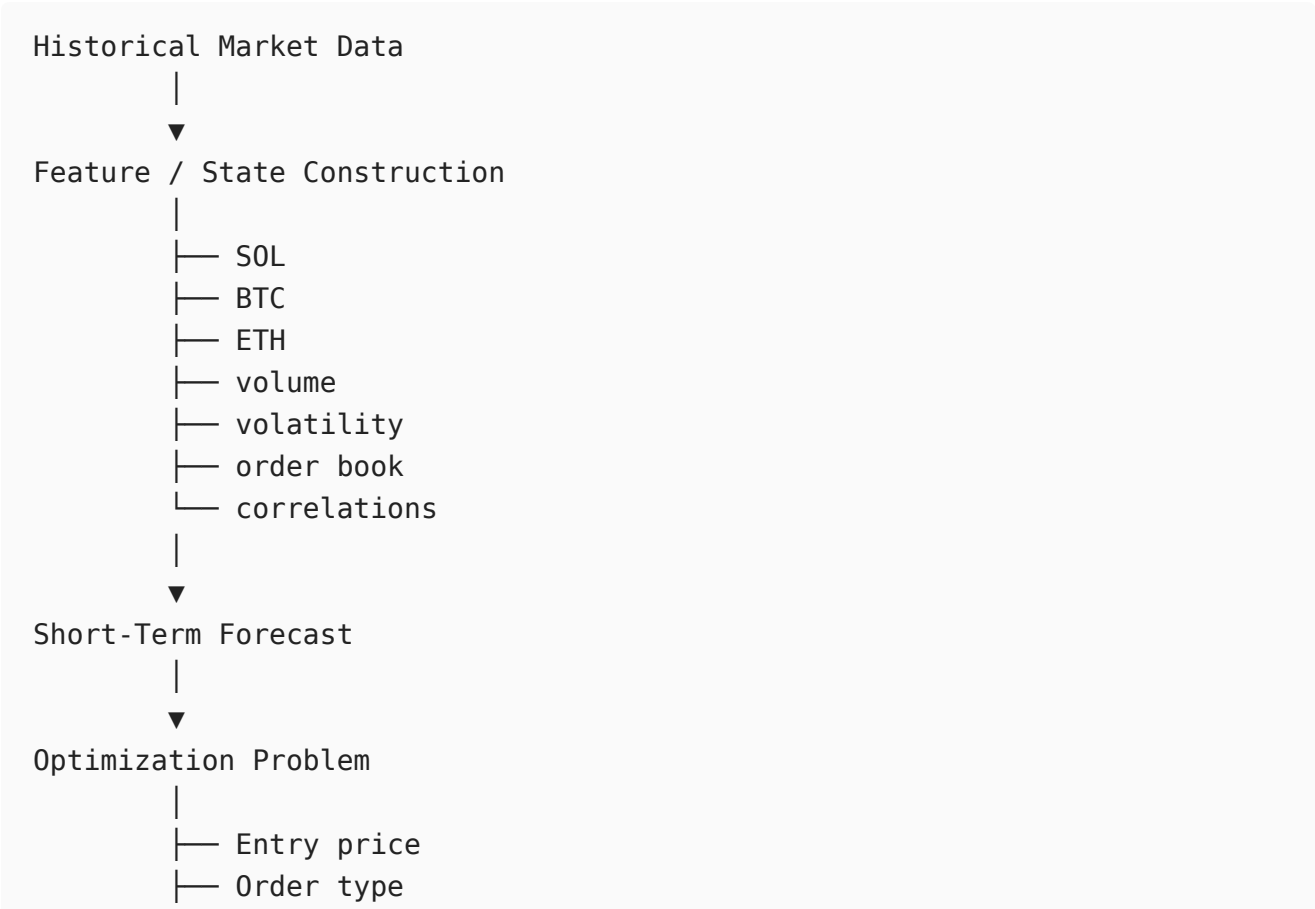
Jadi keadaan pasar bukan:

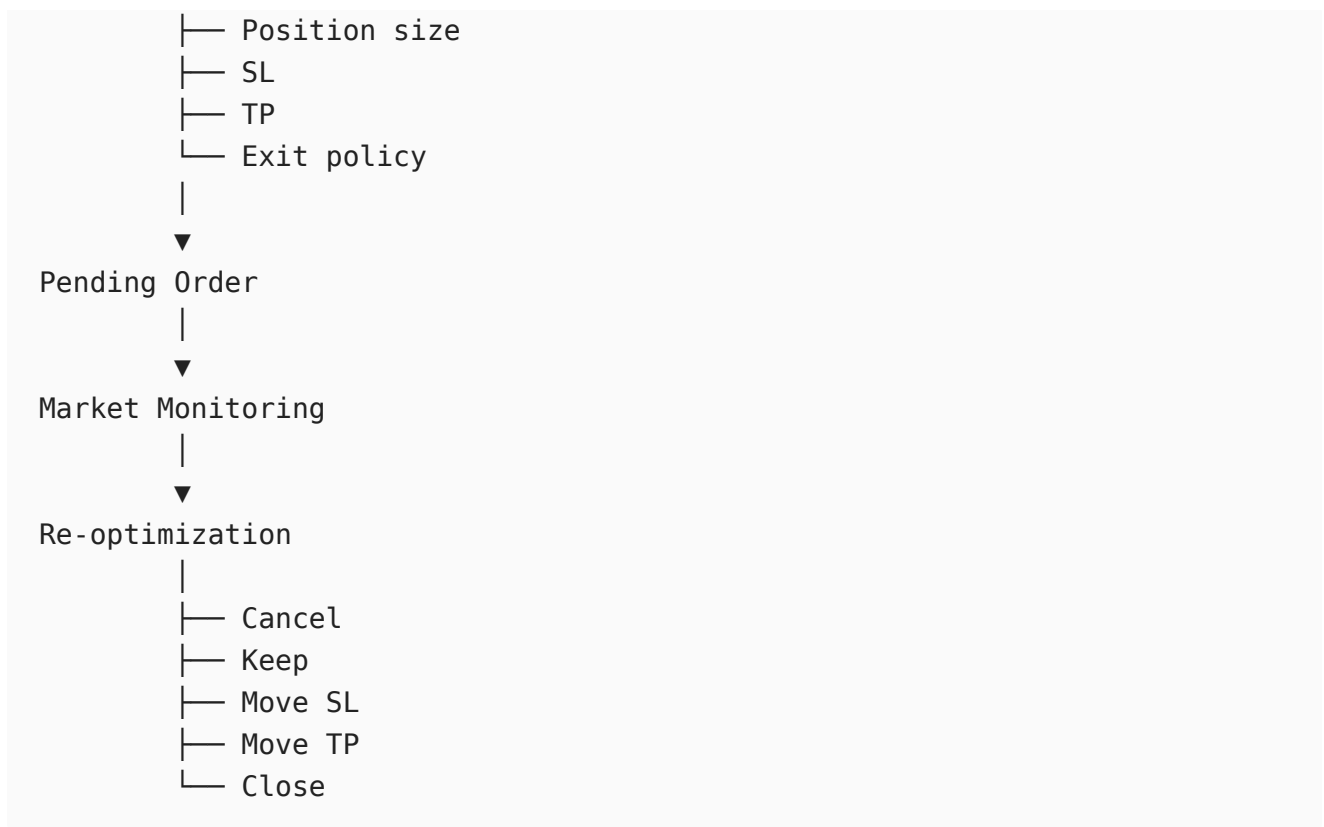
"SOL naik atau turun?"

melainkan:

"Bagaimana keadaan SOL relatif terhadap ekosistem aset yang memengaruhinya?"

Kemudian sistem melakukan:





Ini sudah jauh lebih dekat dengan cara berpikir systematic trading daripada sekadar strategi teknikal.

2. Bagian pertama yang sangat penting: jangan hanya menggunakan harga aset target

Contohmu tentang altcoin + BTC + ETH itu tepat.

Misalnya kita ingin memprediksi return SOL pada horizon 1 jam:

$$r_{SOL,t:t+1}$$

Daripada hanya:

$$r_{SOL,t}$$

kita bisa menggunakan:

$$r_{SOL,t:t+1} = r_{SOL,t:t+1,t}$$

Kemudian kita dapat membangun model:

$$r_{SOL,t:t+1} = f(r_{SOL,t:t+1,t})$$

Tetapi yang menarik adalah kita tidak harus berhenti pada korelasi Pearson.

Kita dapat membangun **conditional dependence**.

Misalnya:

,

tetapi juga:

,

atau bahkan mutual information:

$$= , ,$$

Ini mulai masuk ke wilayah **information-theoretic quantitative finance**.

3. Bahkan "money flow" yang kamu maksud dapat dibuat matematis

Misalnya BTC bergerak:

2

ETH:

sementara SOL:

02

Sistem dapat melihat:

| BTC dan ETH sudah bergerak, tetapi SOL belum mengikuti.

Namun jangan langsung menganggap:

| "SOL pasti naik."

Lebih tepat:

, , , ,

Kemudian kita mencari:

$$t = t \Delta t \quad t$$

dan:

$$2t = t \Delta t \quad t$$

Jadi setiap keputusan memiliki dua komponen:

4. Lalu masuk ke optimasi

Nah, di sinilah ide TSP/MaxCut/QUBO yang kamu sebut menjadi relevan.

Misalkan kita memiliki 10 kandidat aset:

$$= \{1, \dots, 10\}$$

dan hanya boleh memilih 3.

Kita dapat mendefinisikan binary variable:

$$\in \{0, 1\}$$

dengan:

$$= \begin{cases} 1 & 0 \end{cases}$$

Constraint:

$$=$$

Kemudian objective:

$$-$$

Ini sudah menjadi **binary portfolio optimization**.

Dan dapat ditulis sebagai QUBO:

$$\square$$

dengan penalty:

$$P - \sum_i x_i^2$$

untuk memastikan jumlah aset = 3.

Ini sangat natural untuk:

- QAOA
- simulated annealing
- classical branch-and-bound
- tabu search
- quantum annealing

dan sebagainya.

5. Tetapi untuk short-term trading, optimasinya bisa lebih menarik

Daripada hanya:

"aset mana yang dibeli?"

kita bisa membuat decision variable:

”

dengan:

- = aset
- = price level
- = side

Contohnya:

”

berarti:

pasang limit buy SOL pada harga .

Kemudian:

”

berarti:

pasang short pada harga .

Sekarang masalahnya berubah menjadi:

Di mana saya harus menaruh order?

Ini jauh lebih dekat dengan yang kamu bayangkan.

6. Order book sebenarnya dapat dijadikan optimization landscape

Misalnya order book:

```
ASK
101.0   20 SOL
100.9   15 SOL
100.8   50 SOL
-----
```

```

100.7   ← current
-----
100.6   10 SOL
100.5   35 SOL
100.4   80 SOL
BID

```

Kita dapat membuat candidate entry:

$$\in \{1004, 1005, 100\}$$

Untuk setiap , hitung:

$$P$$

dan:

serta:

$$P$$

Kemudian:

$$=$$

Ini sudah menjadi **optimal execution / order placement problem**.

Dan menurutku ini jauh lebih cocok untuk matematika lanjutan daripada sekadar membuat RSI yang lebih rumit.

7. Constraint \$1 yang kamu sebut sangat bagus

Kamu mengatakan:

tetap mengikuti risiko \$1.

Kita bisa menjadikannya hard constraint.

Misalnya:

$$= 1$$

Jika entry:

$$P$$

dan stop:

$$P$$

maka:

$$= 1P - P$$

sehingga:

$$P - P \cdot 1$$

Ini sangat penting.

Karena sekarang optimizer **tidak boleh mencari solusi yang bagus tetapi mempertaruhkan \$10 atau \$100.**

Risk constraint menjadi bagian dari mathematical optimization problem.

8. Bahkan TP dan SL dapat menjadi variable optimasi

Ini menurutku salah satu bagian terbaik dari ide kamu.

Jangan menetapkan:

$$= 2$$

dan:

$$P =$$

secara statis.

Anggap:

$$=$$

$$t =$$

Kita ingin mencari:

$$, t = , t, t$$

dengan constraint:

$$1$$

dan mungkin:

$$P$$

diperoleh dari historical distribution.

Misalnya:

SL	TP	P(TP)	P(SL)	Expected PnL
0.5%	0.8%	0.70	0.30	+...
0.5%	1.2%	0.52	0.48	+...

SL	TP	P(TP)	P(SL)	Expected PnL
1.0%	1.5%	0.48	0.52	+...
1.5%	3.0%	0.32	0.68	+...

Optimizer memilih kombinasi terbaik **berdasarkan kondisi pasar saat itu.**

9. Kemudian masuk ke ide kamu nomor 3

Ini menurutku sangat bagus.

Misalnya order:

$$_0 = P, P, P_P,$$

dipasang pada:

$$t_0$$

Tetapi pada:

$$t_1 = t_0 \ 1$$

order belum filled.

Kita jangan bertanya:

| "Apakah waktunya sudah 1 jam?"

Tetapi:

$$P_{t_1}$$

dan:

dibandingkan dengan:

Jadi keputusan:

$$\square$$

sendiri merupakan optimization problem.

10. Kalau sudah filled, problem-nya berubah

Ini penting.

Sebelum filled:

☐

Setelah filled:

☐

Misalnya posisi long.

Kondisi:

$$P_t > P$$

→ floating loss.

Sedangkan:

$$P_t < P$$

→ floating profit.

Tetapi **jangan otomatis mengatakan floating loss harus dibiarkan dan floating profit harus dikelola.**

Yang lebih matematis adalah:

t

dibandingkan:

Jadi:

$$\boxed{t}$$

11. Ini sebenarnya merupakan sequential decision problem

Dan ini menurutku adalah bagian paling penting dari seluruh ide kamu.

Trading bukan:

tetapi:

$$\boxed{t \quad t_1 \quad t_1 \quad t_2}$$

dengan:

$$t =$$

dan:

$$t =$$

Misalnya:

$$t \in \{, , , , , \}$$

Ini sudah mulai menyerupai:

□

atau bahkan:

□

12. Dan MPC menurutku sangat cocok dengan ide kamu

Bayangkan setiap jam sistem melakukan:

State estimation

$$t = P_{t,t}, t, , , t, t, t,$$

Kemudian optimizer menghitung:

$$t = t1 \ t,$$

Lalu hanya menjalankan keputusan pertama.

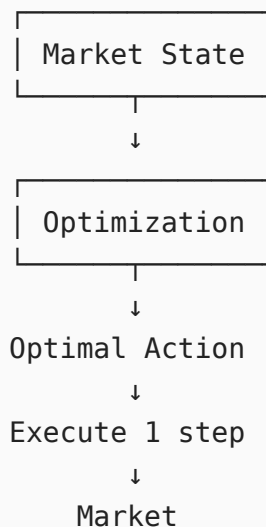
Satu jam kemudian:

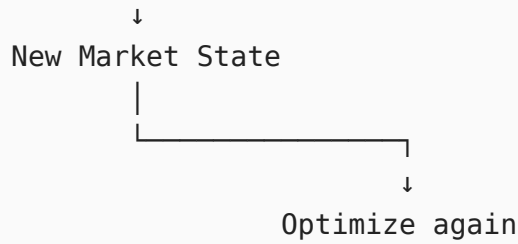
$t1$

berubah.

Optimizer dijalankan lagi.

Ini adalah prinsip **receding horizon control**.





Ini sangat dekat dengan apa yang kamu gambarkan sebagai **evaluasi setiap periode**.

13. Sekarang kasus floating profit yang kamu sebut

Misalnya:

$$P = 100$$

kemudian:

$$P_t = 105$$

dan SL awal:

$$=$$

Kita tidak sekadar berkata:

"Profit 5%, geser SL ke 100."

Kita dapat menghitung ulang distribusi return:

$${}_{t\Delta t} r_t$$

Misalnya ternyata:

$${}_{t\Delta t} r_t = 1$$

dengan volatilitas:

$$= 0$$

maka probabilitas continuation mungkin masih cukup tinggi.

Tetapi jika:

$${}_{t\Delta t} r_t = -0.2$$

maka momentum sudah melemah.

Jadi keputusan trailing stop dapat menjadi:

$$r_t = r_t$$

14. Bahkan TP dapat dinaikkan secara optimal

Ini persis ide nomor 4 milikmu.

Misalnya:

$$P = 100$$

$$P = 105$$

Harga sekarang:

$$P_t = 104$$

Kemudian model memperkirakan:

$$PP_{t1} \ 105 = 02$$

dan:

$$PP_{t1} \ 10 = 051$$

Maka kita dapat membandingkan:

$$P = 105$$

versus:

$$P = 10$$

dan menghitung:

$$P = 105$$

versus:

$$P = 10$$

Kalau:

$$10 \ 105$$

maka TP dinaikkan.

Jadi:

$$\boxed{P_t \ P_{t1} \ P_{t2}}$$

secara dinamis.

15. Nah, di sinilah TSP/MaxCut/QAOA mulai bisa masuk

Kita sekarang mempunyai banyak kandidat keputusan.

Misalnya:

Candidate actions

$$= \{1051010101102\}$$

Kita dapat mendefinisikan binary variables:

$$\in \{0, 1\}$$

dengan constraint:

$$= 1$$

dan objective:

$$-$$

Ini menjadi QUBO.

Kemudian:

$$\square$$

bisa menjadi salah satu solver.

16. Tapi jangan gunakan QAOA untuk semuanya

Ini sangat penting kalau kamu ingin membuat proyek yang benar-benar serius.

Misalnya:

"Saya menggunakan QAOA untuk menentukan apakah BTC akan naik."

Menurutku itu **bukan penggunaan terbaik**.

Lebih masuk akal:

kemudian:

Jadi:

Classical model

menjawab:

Apa yang kemungkinan terjadi?

Sedangkan:

Optimization model

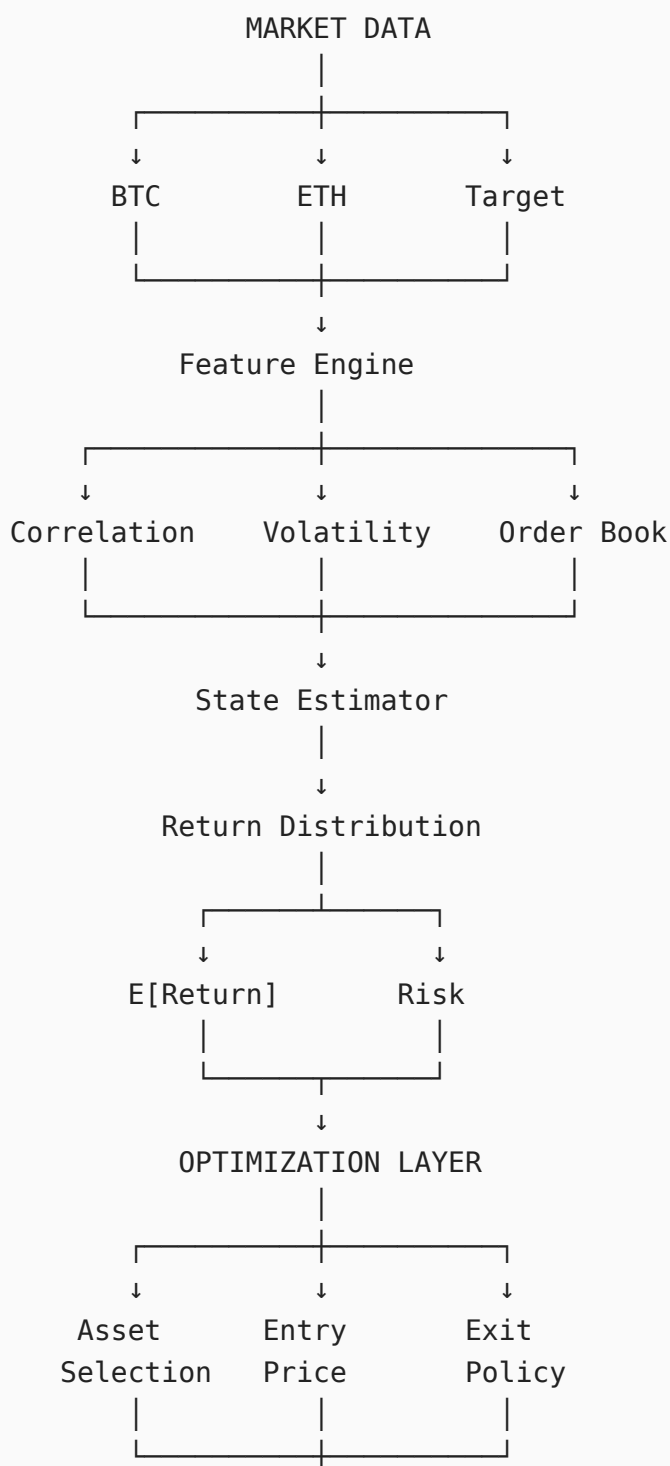
menjawab:

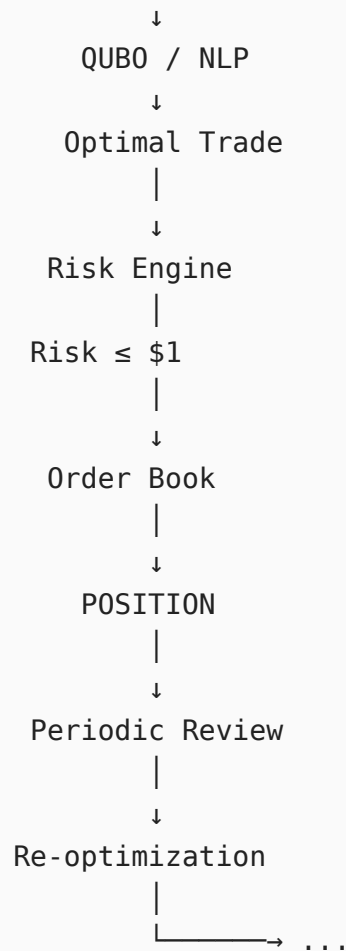
Dengan kemungkinan tersebut, keputusan terbaik apa yang dapat saya ambil?

Ini pemisahan yang sangat penting.

17. Bahkan arsitektur akhirnya bisa seperti ini

Menurutku ini jauh lebih kuat:





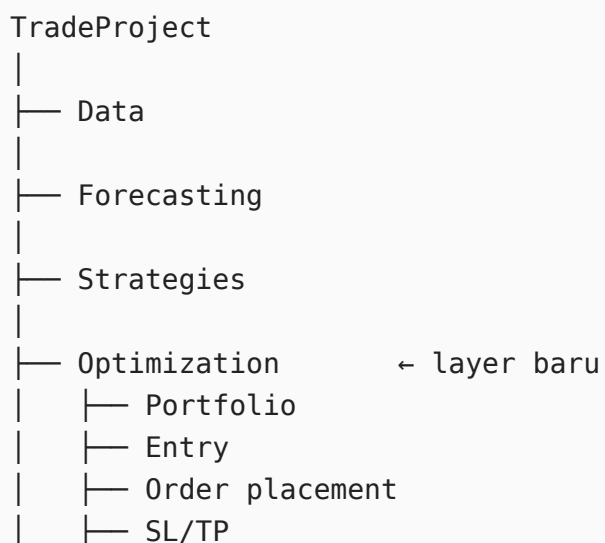
18. Kalau dikaitkan langsung dengan TradeProject

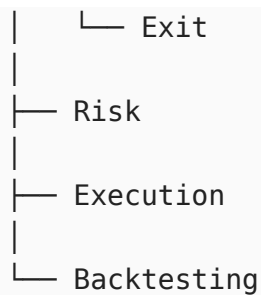
Repository yang sekarang sebenarnya sudah memiliki fondasi:

Yang kita perlukan adalah menambahkan satu layer besar:

□

sehingga:





Dan yang paling penting: **backtester harus bisa mensimulasikan seluruh siklus ini**, bukan hanya menghitung apakah signal menghasilkan profit.

19. Ada satu masalah besar yang harus kita selesaikan: look-ahead bias

Kalau kita melakukan ini dengan historical data, bahaya terbesar bukan QAOA, bukan VQE, dan bukan optimizer.

Tetapi:

□

Misalnya pada:

$$t = 10:00$$

sistem memutuskan order.

Ia hanya boleh mengetahui:

1000

Tidak boleh mengetahui:

$$P_{100}$$

walaupun data tersebut tersedia dalam dataset historis.

Jadi simulasi harus benar-benar:

□

bukan:

20. Masalah kedua: order book jauh lebih sulit daripada OHLCV

Kalau kita benar-benar ingin melakukan pendekatan yang kamu bayangkan, aku justru menyarankan **jangan langsung mulai dari OHLCV saja**.

Karena untuk short-term trading:

tidak cukup untuk mengetahui:

- bid/ask imbalance,
- queue position,
- spread,
- depth,
- liquidity,
- slippage,
- fill probability,
- market impact.

Kita perlu mempertimbangkan:

$$t = \{P, , P, \}$$

Misalnya order-book imbalance:

$$t = -$$

Kemudian:

$$t \ 0$$

memberikan indikasi tekanan beli.

Tetapi lagi-lagi, **indikasi $\neq$ prediksi pasti**.

21. Jadi kalau aku harus mendefinisikan proyek ini secara akademis

Aku tidak akan menyebutnya:

| "AI trading bot."

Aku akan mendefinisikannya sebagai:

□

dengan komponen:

Layer 1 — Market State

t

Layer 2 — Statistical Forecasting

$$t\Delta t \quad t$$

Layer 3 — Combinatorial Optimization

$$=$$

Layer 4 — Risk-Constrained Execution

$$1$$

Layer 5 — Sequential Re-optimization

$$t1 = \quad t1$$

Dan ini menghasilkan sesuatu yang menurutku sangat menarik

Kamu sebenarnya sedang menggabungkan **tiga bidang**:



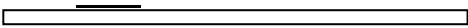
Bukan:

tetapi:



Dan ini juga membuat pengalamanmu dengan **Markowitz** → **QUBO** → **Ising** → **VQE/QAOA** menjadi sangat relevan. Portfolio optimization yang kamu kerjakan untuk skripsi sebenarnya bisa menjadi **versi long-horizon/static** dari problem yang sekarang ingin kamu bawa menjadi **short-horizon/dynamic optimization**.

Perbedaannya kira-kira:



Menurutku, justru bagian "repeated optimization" inilah yang membuat proyek ini jauh lebih menarik.

Langkah berikut yang paling masuk akal adalah kita **jangan langsung coding**. Kita desain dulu *mathematical formulation* untuk satu kasus konkret, misalnya **BTC sebagai market factor + ETH sebagai related asset + satu altcoin target**, lalu kita turunkan dari data historis → probabilitas return → optimal limit-entry → SL/TP → \$1 risk constraint → evaluasi 1 jam → re-optimization. Setelah formulasi klasiknya benar, baru kita tentukan **bagian mana yang benar-benar cocok dijadikan QUBO/Ising/QAOA** dan bagian mana yang lebih baik diselesaikan secara classical.

Prompt:

8/17/2026, 9:03:30 PM

aku suka semua hal yang baru saja kau sampaikan kepadaku. aku ingin semuanya dapat terwujud, terutama karena banyak hal yang sudah aku pelajari sebelumnya. sekarang aku ingin melanjutkan desaim matematisnya. aku tidak ingin berhenti hanya di kripto saja. aku ingin juga model kita dapat bekerja di pasar saham internasional dengan melihat market factor emas dan usd dominance

Response:

8/17/2026, 9:03:31 PM

Remembering

Extended quantitative trading framework across assets, factors, Markowitz, QUBO, gold, and USD dominance

Worked for 5s

Ya. Dan menurutku **kita sebaiknya mengubah desain ini dari "crypto trading model" menjadi sebuah kerangka general untuk systematic trading lintas aset**. Dengan begitu, kripto hanyalah salah satu *market universe*, sementara saham internasional, ETF, komoditas, FX, dan aset lain dapat masuk ke kerangka yang sama.

Ini juga sangat cocok dengan fondasi yang sudah kamu kuasai: **Markowitz** → **covariance** → **QUBO/Ising** → **VQE/QAOA**, tetapi sekarang kita menambahkan **time dependence, market factors, execution, dan dynamic re-optimization**.

1. Kita mulai dari prinsip paling dasar

Aku ingin memisahkan tiga hal:

$\square$

Contohnya:

Crypto

Target:

$=$

Market factors:

$= \{, , \}$

Saham AS

Target:

=

Market factors:

= {, , }

Saham Jepang

Target:

=

Market factors:

= {, , }

Saham emerging market

Target:

=

Market factors:

= {, , }

Jadi kita tidak ingin membuat:

"algoritma crypto"

dan

"algoritma saham".

Kita ingin membuat:

□

2. Market factor menjadi variabel fundamental dalam model

Misalkan terdapat aset.

Return aset :

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

Kita memiliki market factors:

$$t = 1, 2, \dots, T$$

Kemudian kita gunakan **factor model**:

$$r_{i,t} = \alpha_i + \beta_i' F_t + \epsilon_{i,t}$$

atau:

$$r_{i,t} = \alpha_i + \beta_i' F_t + \epsilon_{i,t}$$

Ini adalah salah satu perubahan paling penting dari desain sebelumnya.

Kita tidak lagi hanya bertanya:

"Bagaimana harga aset ini bergerak?"

tetapi:

"Berapa banyak pergerakan aset ini yang dapat dijelaskan oleh market factors?"

3. Untuk desain awal, kita gunakan tiga kelompok factor

Aku menyarankan kita membedakan:

A. Equity / Market factor

Misalnya:

$$F_t = \{r_{M,t}, r_{S&P,t}, \dots\}$$

B. Macro factor

Misalnya:

$$F_t = \{GDP_t, \dots\}$$

C. Cross-asset factor

Misalnya:

$$F_t = \{r_{oil,t}, \dots\}$$

Sehingga:

$$r_{i,t} = \alpha_i + \beta_i' F_t + \epsilon_{i,t}$$

4. Gold dan USD bukan sekadar "dua aset tambahan"

Ini penting.

Kalau kita memasukkan emas dan USD ke model, kita tidak sekadar berkata:

dan:

melainkan kita ingin menangkap **regime informasi**.

Misalnya secara konseptual:

$$\Delta_t \geq 0$$

dan:

$$\Delta_t \leq 0$$

dapat merepresentasikan kondisi tertentu pada global macro environment.

Sebaliknya:

$$\Delta_t \geq 0, \Delta_t \leq 0$$

merepresentasikan kondisi yang berbeda.

Jadi kita bisa membuat:

$$t = \text{,,,,}$$

Ini jauh lebih powerful daripada sekadar memasukkan harga emas sebagai feature.

5. Saya ingin kita mendefinisikan "market state"

Pada setiap waktu t , sistem memiliki:

$$t$$

yang merupakan seluruh informasi yang tersedia saat itu.

Misalnya:

$$t = \text{tttttt}$$

dengan:

- t : return aset,
- t : market factors,
- t : volatility,
- t : factor exposure,
- t : order book,
- t : volume,
- t : correlation/dependence,
- t : current positions.

Jadi:

$$t =$$

6. Dari state tersebut kita estimasi expected return

Kita ingin:

$$t\Delta = t\Delta \quad t$$

Ini adalah salah satu input utama optimizer.

Misalnya:

$$= 00020001 - 00005000$$

berarti model memperkirakan return jangka pendek empat aset tersebut.

Tetapi kita **tidak boleh hanya menggunakan** .

Kita juga memerlukan:

$$t = t\Delta \quad t$$

Ini langsung menghubungkan sistem baru dengan Markowitz yang sudah kamu kerjakan.

7. Tetapi covariance kita harus dynamic

Dalam skripsimu, covariance secara umum adalah:

Sekarang kita naikkan menjadi:

$$\Sigma_t$$

karena hubungan antar-aset berubah terhadap waktu.

Misalnya:

$$\Sigma_t$$

tidak harus sama sepanjang waktu.

Begitu juga:

$$\Sigma_t$$

bisa berubah ketika market memasuki regime berbeda.

Jadi model kita harus mengestimasi:

$$\Sigma_t$$

secara rolling atau conditional.

8. Factor model memberikan struktur covariance

Dari:

$$\Sigma_t = \Lambda_t \Lambda_t' + \Psi_t$$

kita memperoleh:

$$\Sigma_t = \Lambda_t \Lambda_t' + \Psi_t$$

dengan:

- Λ_t : factor loading,
- Ψ_t : covariance factor,
- Ψ_t : idiosyncratic covariance.

Ini sangat bagus untuk sistem kita.

Karena kita dapat mengatakan:

=

atau:

$$t = t, t, t$$

9. Sekarang kita punya "expected return surface"

Misalnya kita ingin trading NVDA.

Model tidak hanya mengatakan:

$$= 0.2$$

tetapi:

$$, t_1, \dots, t_n$$

Jadi secara konseptual:

$$= \dots$$

10. Sekarang kita masuk ke optimization problem

Kita mempunyai:

$$t$$

dan:

$$t$$

Kita ingin menentukan portfolio/action:

$$t$$

Versi continuous Markowitz:

$$—$$

dengan:

$$= 1$$

Tetapi trading kita memiliki keputusan diskrit.

Misalnya:

$$\in \{-1, 0, 1\}$$

yang berarti:

- -1 : short,

- 0: tidak memiliki posisi,
- 1: long.

Sekarang masalahnya menjadi jauh lebih menarik.

11. Kita dapat mengubahnya menjadi binary variables

Gunakan:

$$x_i \in \{0, 1\}$$

dan:

$$x_i = 1 \text{ if } \dots$$

Dengan constraint:

$$x_i \leq 1$$

Kemudian objective:

$$\max \sum x_i$$

Sekarang kita sudah sangat dekat dengan:

$$\square$$

12. Dan di sinilah MaxCut menjadi relevan

Misalnya kita ingin memilih kumpulan aset yang memiliki:

- expected return tinggi,
- correlation rendah,
- factor exposure tertentu.

Kita bisa membuat graph:

$$G = (V, E)$$

dengan:

$$V = \{1, 2, \dots, n\}$$

Kemudian memilih subset aset yang memenuhi objective tertentu.

Problem tersebut dapat memiliki struktur:

$$\square$$

Jadi MaxCut bukan sekadar "contoh quantum algorithm".

Ia menjadi cara untuk memecahkan **asset-selection / clustering / diversification problem**.

13. TSP juga bisa masuk, tetapi di tempat yang berbeda

TSP menurutku **bukan optimizer utama untuk portfolio allocation**.

Tetapi ada problem trading yang lebih natural.

Misalnya kita mempunyai beberapa execution venue:

$$= \{, , , \}$$

dan ingin mengoptimalkan sequence/order routing.

Atau jika sistem harus melakukan serangkaian transaksi:

$$1 \ 2$$

dengan biaya transaksi, latency, spread, dan market impact.

Maka kita bisa mendapatkan bentuk routing/scheduling problem yang lebih dekat ke:

$$\square$$

Jadi kita tidak memaksakan TSP ke semua masalah.

14. Saya ingin membangun empat optimizer berbeda

Ini menurutku akan menjadi arsitektur matematis yang sangat kuat:

Problem	Mathematical form	Solver potensial
Asset selection	QUBO	QAOA / SA
Portfolio allocation	QUBO / NLP	VQE / classical
Order placement	discrete optimization	QAOA / MILP
Execution routing	TSP / routing	QAOA / classical
Dynamic exit	sequential optimization	MPC / DP

Sehingga kita tidak memiliki:

| "satu algoritma quantum untuk trading."

Tetapi:

□

15. Sekarang kita masukkan \$1 risk constraint

Ini harus berada **di atas optimizer**, bukan setelah optimizer.

Kita definisikan:

$$= 1$$

Untuk posisi :

$$= P_t - P$$

Kemudian:

$$\square$$

Jika kita hanya mengizinkan satu trade:

$$1$$

Dengan beberapa posisi:

$$1$$

Ini membuat optimizer tidak bisa mengambil keputusan yang secara matematis menguntungkan tetapi secara risk management tidak valid.

16. Dan ini membawa kita ke konsep yang lebih penting: risk budget

Aku bahkan lebih suka jika kita tidak selalu menganggap \$1 sebagai angka absolut.

Kita definisikan:

$$_t =$$

Misalnya:

$$_t = 1$$

Jika ada tiga posisi:

$$_1 = 00, _2 = 040, = 020$$

maka:

$$= 00$$

Masih tersedia:

$$= 010$$

Sehingga optimizer harus memperlakukan risk sebagai resource.

Ini secara matematis sangat mirip **resource-constrained optimization**.

17. Sekarang kita masuk ke order

Misalnya optimizer menghasilkan:

$$= \{ \}$$

Itu belum cukup.

Kita perlu:

$$t = P, P, P_P, ,$$

Jadi action lengkap adalah:

$$t = , , , , ,$$

Dan:

$$t = \quad t$$

dengan constraint:

$$1$$

Nah, ini menurutku adalah inti matematis dari trading engine kita.

18. Pending order

Misalnya:

$$t = , 100, , 10,$$

Order belum filled.

Kita kemudian mengestimasi:

$$P_{t,t}$$

yaitu probabilitas order terisi.

Expected utility:

$$= P \cdot 1 - P$$

Sedangkan jika dibatalkan:

$$=$$

Keputusan:

$$\square$$

Ini sudah memberikan formulasi matematis untuk ide nomor 3 yang kamu berikan sebelumnya.

19. Setelah filled, kita masuk ke optimal stopping

Misalkan:

$$P_t, P$$

Posisi sedang profit.

Kita memiliki tiga pilihan:

$$= \{, , \}$$

Kemudian:

$$t = \in t$$

Jika modify:

$$= , P$$

maka:

$$, P = , P_t$$

20. Dengan demikian trailing TP milikmu menjadi masalah optimasi

Bukan:

| "Kalau profit 5%, TP naik."

Tetapi:

$$P_t = P_t$$

Dan:

$$t = t$$

Ini menurutku jauh lebih elegan.

21. Sekarang kita tambahkan regime

Aku ingin model kita memiliki:

$$t$$

sebagai **latent market regime**.

Misalnya:

$$t \in \{1, 2\}$$

Tetapi regime tidak kita tentukan secara manual.

Kita estimasi:

$$P_t = \pi_t$$

Contohnya:

$$P = 0.2$$

$$P = 0.20$$

$$P = 0.0$$

Kemudian:

$$t = P_t = \pi_t$$

dan:

$$t = P_t = \pi_t$$

plus komponen variance antar-regime jika kita ingin formulasi lengkap.

22. Gold dan USD kemudian menjadi bagian dari regime detector

Sekarang kita mendapatkan peran yang lebih tepat untuk:

$$t =$$

dan:

$$t =$$

Kita dapat membangun state:

$$t = 1, 2, 3, 4$$

Kemudian:

$$P_{t \ t}$$

Jadi model tidak berkata:

"Gold naik → short saham."

Melainkan:

"Perubahan gold + USD + equity + volatility memberikan probabilitas bahwa kita sedang berada pada regime tertentu."

Itu jauh lebih defensible secara kuantitatif.

23. Bahkan USD harus kita perlakukan dengan hati-hati

Untuk model global, "USD dominance" harus kita definisikan secara eksplisit.

Kita dapat menggunakan sebuah **USD factor** t , bukan menganggap satu ticker tertentu sebagai hukum universal.

Secara abstrak:

$$t = , ,$$

Kemudian:

$$,$$

menunjukkan sensitivitas aset terhadap faktor USD.

Misalnya:

$$, 0$$

berarti, dalam model kita, NVDA cenderung memiliki hubungan negatif dengan penguatan USD.

Tetapi nilai tersebut **harus diestimasi dari data**, bukan diasumsikan.

24. Ini menghasilkan factor exposure vector

Untuk setiap aset:

$$= , , , , ,$$

Contohnya secara ilustratif:

$$= 12015 - 0$$

artinya aset tersebut sensitif terhadap:

- equity factor,
- gold,
- USD.

Kemudian portfolio exposure:

$$\boxed{=}$$

Ini memungkinkan kita membuat constraint seperti:

atau:

25. Dan sekarang QUBO kita menjadi jauh lebih kaya

Objective:

$$-$$

ditambah factor exposure penalty:

$$- _ _ ^2$$

Ditambah transaction cost:

$$-$$

Ditambah risk constraint penalty:

$$- P - ^2$$

Secara keseluruhan:

$$\boxed{- _ _ - ^2 - _ _ P - ^2}$$

dengan:

$$\in \{0, 1\}$$

atau encoding long/short yang sesuai.

Setelah ekspansi kuadratik:

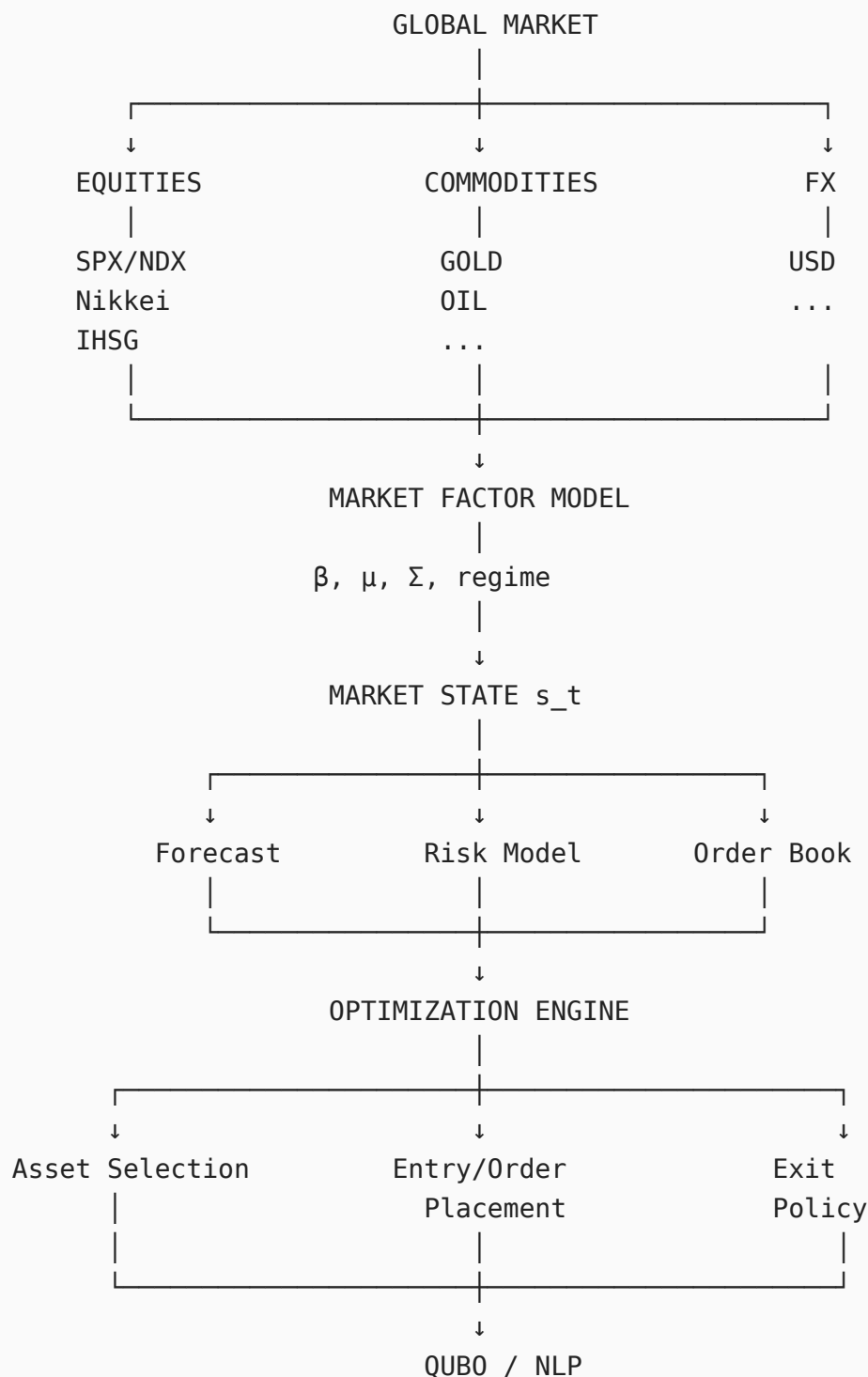


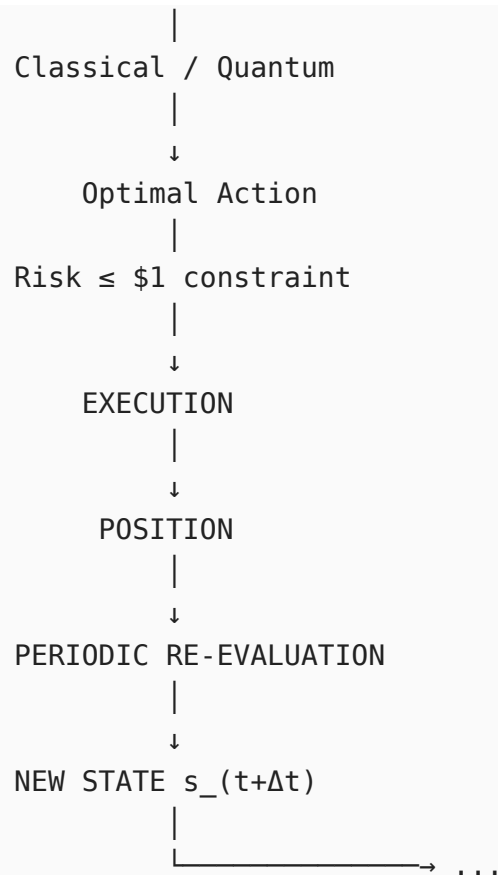
yang merupakan QUBO.

Ini sudah sangat dekat dengan sesuatu yang benar-benar bisa kita mapping ke Ising Hamiltonian.

26. Dan akhirnya kita dapat arsitektur matematis lengkap

Aku akan mendefinisikannya seperti ini:





27. Ada satu prinsip yang ingin aku tetapkan sejak awal

Jangan mulai dari QAOA.

Kita mulai dari:

☐

kemudian:

☐

baru:

☐

baru:

☐

baru:

☐

Dengan demikian, kalau nanti QAOA ternyata tidak mengalahkan classical solver, kita masih memiliki penelitian yang valid.

Kita bisa melakukan perbandingan yang fair:

pada **problem, encoding, dan objective yang sama**, lalu mengevaluasi:

Ini sejalan dengan pendekatan perbandingan terkontrol yang sebelumnya kita gunakan untuk pekerjaan VQE/QAOA-mu.

28. Tahap desain berikutnya

Menurutku sekarang **jangan langsung membangun seluruh sistem sekaligus**. Kita turunkan secara matematis dalam urutan berikut:

Tahap A - Market State

Kita definisikan secara rigor:

$$t$$

beserta data:

Tahap B - Dynamic Factor Model

Kita turunkan:

$$_{,t} = t_{,t}$$

hingga:

$$t = t_{,tt} t$$

Tahap C - Short-Horizon Forecast

Kita definisikan:

$$_{,t\Delta} = _{,t\Delta}t$$

dan:

$$_{2,t\Delta} = _{,t\Delta}t$$

Tahap D - Trading Optimization

Kita turunkan:

$$t = t$$

dengan:

$$1$$

Tahap E - QUBO

Kita ubah decision variables menjadi binary:

$$\in \{0, 1\}$$

dan turunkan seluruh objective menjadi:

$$\square$$

Tahap F - Ising

Kemudian:

$$= 1 - 2$$

sehingga:

$$=$$

dan baru kita bahas:

$$\square$$

Tahap G - Dynamic Re-optimization

Setelah posisi terbuka:

$$t \quad t \quad t\Delta$$

kita ulangi optimasi:

$$t\Delta = t\Delta$$

Di sinilah sistem berubah dari **static portfolio optimizer** menjadi **dynamic quantitative trading system**.

Dan aku kira **inilah desain yang seharusnya kita kejar**: bukan "bot crypto yang memakai QAOA", melainkan **kerangka mathematical systematic trading yang asset-agnostic**, di

mana crypto, saham AS, Jepang, Eropa, Indonesia, emas, FX, dan sebagainya hanyalah instansiasi dari universe yang berbeda.

Yang paling penting, desain ini memungkinkan seluruh pengetahuan yang sudah kamu bangun sebelumnya tetap hidup: **Markowitz memberikan objective risk-return, factor model memberikan market context, covariance memberikan risk coupling, QUBO memberikan discrete optimization, Ising memberikan Hamiltonian, dan QAOA/VQE menjadi solver- sementara MPC/sequential optimization mengubah semuanya menjadi sistem trading yang terus mengambil keputusan ulang.**

Prompt:

8/17/2026, 9:26:47 PM

hebat sekali. kita sudah merancang kerangka utama yang luar biasa. sekarang bisakah kita melanjutkan untuk membangun sistemnya tahap demi tahap? jika kau siap, mari kita mulai dari Tahap A dengan sangat detail. jika perlu, tambahkan penurunan matematis lengkap agar pendasarannya lebih dapat dibuktikan secara matematis namun tetap dengan pertimbangan paradigma finansial yang masuk akal

Response:

8/17/2026, 9:26:48 PM

Remembering

Reviewed Tahap A market-state design and cross-asset factor framework

Worked for 5s

Sangat siap. Dan untuk **Tahap A**, saya justru menyarankan kita bergerak pelan dan cukup rigor, karena seluruh tahap berikutnya—forecast, covariance, QUBO, Ising, sampai QAOA—akan bergantung pada definisi *market state* yang kita buat di sini.

Kita akan membangun Tahap A bukan sebagai sekadar kumpulan indikator, tetapi sebagai **representasi matematis keadaan pasar yang tersedia pada waktu t .**

Tahap A — Market State Representation

Tujuan akhirnya adalah memperoleh:

$$\boxed{t = t}$$

yang kemudian menjadi input untuk Tahap B:

$$t \quad \{t, t, t, t, \}$$

Ada prinsip fundamental yang harus kita pegang:

$$\boxed{t}$$

Ini akan menjadi fondasi untuk mencegah **look-ahead bias**.

A.1. Apa sebenarnya "state" dalam sistem kita?

Misalkan terdapat universe aset:

$$= \{1, 2, , \}$$

Contohnya:

$$= \{, , , \}$$

atau:

$$= \{, , \}$$

Kita juga memiliki sekumpulan market factor:

$$= \{1, 2, , \}$$

Misalnya:

$$= \{, , , , , \}$$

Untuk crypto:

$$= \{, , \}$$

Tetapi kita harus berhati-hati.

BTC sendiri dapat menjadi target asset maupun factor.

Jika kita trading SOL, BTC dapat menjadi factor.

Jika kita trading BTC, BTC tidak boleh sekaligus digunakan sebagai explanatory factor untuk dirinya sendiri.

A.2. Market state bukan hanya harga

Kesalahan yang sering terjadi pada sistem trading sederhana adalah mendefinisikan:

$$_t = P_t$$

Ini terlalu miskin.

Harga hanya memberi tahu:

"di mana pasar berada."

Padahal kita juga membutuhkan:

- bagaimana pasar bergerak,
- seberapa cepat bergerak,
- seberapa volatil,
- seberapa likuid,
- bagaimana aset lain bergerak,
- bagaimana factor bergerak,
- bagaimana hubungan antar-aset berubah,
- bagaimana order book tersusun.

Karena itu saya ingin mendefinisikan state sebagai:

$$t = t, t, t, t, t, t, t, t$$

dengan:

Komponen	Makna
t	price/return state
t	market-factor state
t	volatility state
t	cross-asset dependence
t	market/microstructure state
t	order-book state
t	liquidity state
t	current portfolio/risk state

Kita akan turunkan satu per satu.

A.3. Layer pertama: price state

Untuk aset , kita mempunyai:

$$P_{,t}$$

yang merupakan harga pada waktu t .

Namun **harga absolut** bukan representasi yang baik untuk membandingkan aset yang berbeda.

Misalnya:

$$P = 100000$$

dan:

$$P = 200$$

Angka tersebut tidak dapat dibandingkan secara langsung.

Karena itu kita bekerja dengan return.

A.3.1. Simple return

Definisikan:

$$r_{t,t-1} = \frac{P_t - P_{t-1}}{P_{t-1}}$$

atau:

$$r_{t,t-1} = \frac{P_t}{P_{t-1}} - 1$$

Misalnya:

$$P_{t-1} = 100$$

dan:

$$P_t = 102$$

Maka:

$$r_{t,t-1} = \frac{102 - 100}{100} = 0.02 = 2\%$$

Return adalah representasi yang lebih tepat karena bersifat relatif terhadap harga sebelumnya.

A.3.2. Log return

Untuk beberapa operasi matematis kita juga akan menggunakan:

$$r_{t,t-1} = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

Perhatikan:

$$1 + r_{t,t-1} = \frac{P_t}{P_{t-1}}$$

sehingga:

$$r_{t,t} = 1$$

Untuk return kecil:

$$1$$

Namun jangan menganggap keduanya identik.

Untuk holding multi-period:

$$PP_0 = \prod_{t=1}^T r_{t,t}$$

sehingga:

$$PP_0 = \prod_{t=1}^T r_{t,t}$$

Ini adalah salah satu alasan log-return sangat berguna untuk pemodelan temporal.

A.4. Multi-horizon state

Karena sistem kita adalah **short-horizon trading**, satu return saja tidak cukup.

Kita dapat membentuk:

$$r_{\Delta,t} = P_{t,t-\Delta} - 1$$

Misalnya:

$$\Delta \in \{1, 5, 15, 60, 240\}$$

menit, tergantung resolusi data.

Maka state harga dapat memuat:

$$r_{\Delta,t} = \{r_{1,t}, r_{5,t}, r_{15,t}, r_{60,t}, r_{240,t}\}$$

Interpretasinya:

- 1 menit → micro-momentum,
- 5 menit → very short-term momentum,
- 15 menit → intraday momentum,
- 1 jam → short-term trend,
- 4 jam → broader local regime.

Ini lebih baik daripada memilih satu timeframe secara arbitrer.

A.5. Layer kedua: market factor state

Sekarang kita masuk ke inti desain global.

Misalkan terdapat factors:

$$t = 1, t_2, t, t$$

Kita tidak menggunakan harga factor secara langsung.

Kita juga mengubahnya menjadi return:

$$t = 1, t_2, t, t$$

A.6. Contoh factor universe global

Untuk versi pertama sistem, saya menyarankan struktur konseptual seperti ini.

Equity factors

$$= \{, , , \}$$

Macro factors

$$= \{, \}$$

Risk factors

$$= \{, \}$$

Crypto factors

$$= \{, \}$$

Sehingga:

$$=$$

Tetapi **tidak semua factor digunakan untuk setiap aset**.

Itu penting.

A.7. Contoh: target NVDA

Untuk NVDA kita mungkin mempunyai:

$$t = ,t,t,t,t,t$$

Sedangkan untuk SOL:

$$t = ,t,t,t,t,t$$

Dengan demikian model menjadi **asset-conditional**.

A.8. Mengapa factor harus berupa return?

Misalkan:

$$P = 2500$$

Nilai:

$$2500$$

sendiri hampir tidak memberikan informasi yang berguna mengenai *shock* pasar.

Yang lebih informatif:

$$_{,t} = P_t - P_{t-1}P_{t-1}$$

Kemudian kita bisa membandingkan:

dengan:

$$,$$

tanpa masalah skala harga.

A.9. Gold sebagai macro factor

Kita definisikan:

$$t = P_t P_{t-1}$$

Kemudian gold state bukan hanya:

$$t$$

Kita bisa memiliki:

$$t = 1t, 5t, 15t, 0t, t$$

Dengan demikian sistem dapat mengetahui:

apakah gold sedang naik, seberapa cepat, dan dalam kondisi volatilitas seperti apa.

A.10. USD factor

Untuk USD kita perlu sedikit lebih hati-hati.

Daripada mengunci model pada satu instrumen tertentu, kita mendefinisikan abstraksi:

$$t =$$

Kemudian:

$$t = tt-1$$

State-nya:

$$ttt = 1t, 5t, 15t, 0t, t$$

Dengan demikian implementasi di kemudian hari dapat memilih instrumen representatif USD yang konsisten.

Ini penting untuk membuat sistem **asset-agnostic**.

A.11. Jangan langsung menyebutnya "money flow"

Ini adalah satu perubahan istilah yang menurutku penting.

Misalnya:

$$0$$

dan:

$$0$$

Kita tidak boleh menyimpulkan:

"uang mengalir dari BTC ke SOL."

Harga tidak secara langsung memberi kita observasi arus uang.

Lebih aman secara matematis menyebutnya:

$$\sigma_t$$

atau:

$$\sigma_t$$

Kemudian jika kita memiliki volume, order flow, atau flow data yang relevan, barulah kita dapat membangun konsep flow yang lebih eksplisit.

A.12. Layer ketiga: volatility state

Return:

$$r_{i,t}$$

belum memberi tahu kita seberapa tidak stabil aset tersebut.

Kita membutuhkan:

$$r_{i,t}$$

Estimator paling sederhana:

$$\sigma_{i,t}^2 = 1 - 1 - 1_{=0,t-} - r_{i,t}^2$$

dengan:

$$r_{i,t} = 1 - 1_{=0,t-}$$

A.13. Mengapa volatility harus conditional?

Karena:

$$r_{i,t} = r_{i,t-1}$$

Market memiliki volatility clustering.

Secara sederhana:

$$r_{i,t} = P_{t1}$$

Kita tidak perlu langsung menggunakan GARCH pada Tahap A.

Yang penting adalah kita mendefinisikan:

$$r_{i,t} =$$

yang nanti dapat digunakan untuk:

- position sizing,
- SL,
- TP,
- risk,
- forecast uncertainty.

A.14. Volatility relatif juga penting

Misalnya:

$$= 4$$

dan:

$$= 15$$

Kita dapat membentuk:

$$t = ,t,t$$

Jika:

$$t \ 1,$$

SOL sedang jauh lebih volatile daripada BTC.

Ini dapat menjadi informasi regime.

A.15. Layer keempat: cross-asset dependence

Sekarang kita kembali ke ide awalmu tentang BTC, ETH, gold, USD, dan sebagainya.

Untuk dua return:

$$, ,$$

covariance:

$$, = - -$$

dan correlation:

$$= ,$$

Namun kita tidak ingin:

yang statis.

Kita ingin:

$$r_{i,t}$$

menggunakan rolling window.

A.16. Rolling covariance matrix

Untuk aset:

$$r_t = [r_{1,t}, r_{2,t}]$$

Dengan window :

$$r_t = \frac{1}{N} \sum_{i=0}^{N-1} (r_{t-i} - \bar{r})(r_{t-i} - \bar{r})'$$

Perhatikan bentuknya:

menghasilkan matrix.

Karena itu:

$$r_t = \begin{bmatrix} r_{1,t} & r_{12,t} \\ r_{21,t} & r_{22,t} \end{bmatrix}$$

Ini adalah jembatan langsung menuju Markowitz yang sudah kamu kuasai.

A.17. Tetapi correlation saja belum cukup

Misalnya:

$$r_{i,j} = 0$$

Kita tahu hubungan linear kuat.

Tetapi kita belum tahu:

| Apakah SOL mengikuti BTC secara nonlinear?

Karena itu nantinya kita dapat memperluas dependency state menggunakan:

yaitu mutual information.

Secara definisi:

$$I(X; Y) = -\sum_{x,y} p(x,y) \log p(x,y)$$

Untuk continuous variables bentuknya:

$$I(X; Y) = -\int \int p(x,y) \log p(x,y) dx dy$$

Namun saya **tidak ingin memasukkan mutual information ke versi pertama model**.

Kita simpan sebagai extension setelah baseline covariance bekerja dengan benar.

A.18. Layer kelima: market microstructure

Sekarang kita masuk ke bagian yang membedakan model kita dari portfolio optimizer biasa.

Untuk short-term trading, OHLCV tidak cukup.

Kita membutuhkan order book.

Misalkan:

$$b_t$$

adalah harga bid level , dengan quantity:

$$q_t$$

Sedangkan:

$$a_t$$

adalah ask level , dengan:

$$q_t$$

Maka:

$$P_t = \{b_t, a_t\}_{t=1}^T$$

A.19. Best bid dan ask

Definisikan:

$$P_t = \{b_t, a_t\}_{t=1}^T$$

dan:

$$P_t = {}_1t$$

Spread:

$${}_t = P_t - P_t$$

Relative spread:

$${}_t = P_t - P_t P_t$$

dengan:

$$P_t = P_t P_t^2$$

Ini sangat penting untuk menentukan apakah limit order kita masuk akal.

A.20. Order-book imbalance

Kita dapat mendefinisikan:

$${}_t = {}_{=1}t$$

dan:

$${}_t = {}_{=1}t$$

Kemudian:

$${}_t = {}_t - {}_{tt} {}_t$$

Sehingga:

$$-1 {}_t 1$$

Interpretasi:

$${}_t 0$$

→ bid-side depth lebih besar.

$${}_t 0$$

→ ask-side depth lebih besar.

Sekali lagi:

$${}_t 0 P_{t1} P_t$$

Ia hanya menjadi **state information**.

A.21. Microprice

Kita dapat membangun microprice:

$$P_t = P_{tt} P_{ttt} \dots$$

Perhatikan jika:

,

maka microprice cenderung lebih dekat ke ask.

Selisih:

$$t = P_t - P_t$$

dapat menjadi feature microstructure.

Ini jauh lebih relevan untuk short-term execution daripada sekadar moving average.

A.22. Layer keenam: liquidity state

Sistem kita perlu mengetahui apakah sebuah posisi dapat dieksekusi tanpa terlalu banyak market impact.

Salah satu representasi sederhana:

$$t = \dots = 1 \dots = 1$$

Tetapi quantity saja tidak cukup.

Kita juga dapat menghitung depth pada jarak dari mid-price:

$$t = \dots_{P \in P-, P}$$

Jadi:

$$= \{, t, , , \}$$

A.23. Mengapa liquidity harus masuk ke state?

Misalnya dua aset mempunyai:

$$_1 = _2 = 02$$

dan:

$$_1 = _2 = 1$$

Secara Markowitz sederhana keduanya tampak identik.

Tetapi:

$$1 \quad 2$$

Maka expected execution cost:

$$1 \quad 2$$

Jadi secara trading:

$$1$$

lebih menarik.

Ini menunjukkan bahwa:

$$\square$$

dan sistem kita harus menggabungkan keduanya.

A.24. Layer ketujuh: current position state

Market state saja belum cukup.

Sistem juga harus tahu **apa yang sudah kita miliki**.

Misalnya:

$$,t =$$

Kemudian:

$$P_{t,t}$$

adalah average entry price.

Kita juga membutuhkan:

$$,t, P_{,t}$$

Maka position state:

$$_t = \{t, P_{tt}, t, P_{t,t}\}$$

dengan $_t$ dapat merepresentasikan unrealized PnL.

A.25. Unrealized PnL

Untuk long:

Untuk short:

Atau menggunakan signed position:

maka:

dengan:

untuk long dan:

untuk short.

A.26. Sekarang kita bisa membangun Market State Vector

Untuk sebuah target asset , misalnya SOL:

$$,t = ,t,t,t,t,t,t,t,t$$

Secara lebih eksplisit:

$$,t = 1515011111, {}^t P t P P$$

Ini **bukan berarti final feature vector kita harus persis seperti ini**. Ini adalah mathematical specification awal.

A.27. Tetapi ada persoalan yang sangat penting: timestamp

Ini mungkin terlihat sepele, tetapi sangat fundamental.

Misalnya:

- harga saham tersedia pada 10:00,
- harga BTC tersedia terus-menerus,
- gold tersedia pada waktu berbeda,
- USD factor tersedia pada interval berbeda.

Kita harus menentukan:

$$\boxed{t = t}$$

Setiap feature harus mempunyai timestamp.

Jika:

$$t \leq t$$

maka BTC boleh digunakan.

Jika:

$$t > t,$$

maka gold tersebut **tidak boleh digunakan**.

Walaupun secara database datanya sudah tersedia.

A.28. Ini adalah definisi formal anti-look-ahead

Kita definisikan filtration:

$$\mathcal{F}_t = \mathcal{F}_t$$

Kemudian keputusan:

$$t$$

harus memenuhi:

$$\mathcal{F}_t$$

Secara intuitif:

ketika sistem membuat keputusan pada t , ia hanya boleh menggunakan informasi yang diketahui pada t .

Ini akan menjadi salah satu aturan paling penting pada backtester kita nanti.

A.29. Sampling frequency

Sekarang kita harus memilih satu konsep penting:

$$\Delta t$$

Karena sistem kita short-term tetapi multi-asset, saya tidak ingin semua data dipaksa memiliki timeframe yang sama secara sembarangan.

Kita definisikan:

$$t_0, t_1, \dots, t$$

sebagai **decision times**.

Misalnya:

$$\Delta t = 1$$

Kemudian:

$$t$$

dibentuk pada setiap decision time.

Tetapi data order book dapat memiliki resolusi jauh lebih tinggi.

Jadi:

$$\square$$

Ini sangat penting.

A.30. Contoh

Misalkan decision time:

$$0, 00, 10, 00, 11, 00,$$

Tetapi order book:

$$0, 00, 01, 0, 00, 02,$$

Kita dapat menggunakan informasi microstructure sampai:

$$0, 5, 5$$

untuk membuat:

$$1000$$

Kemudian keputusan:

$$1000$$

Dengan demikian sistem tetap konsisten.

A.31. Market state sebenarnya adalah random variable

Ini penting secara probabilistik.

Kita dapat menulis:

$$t$$

sebagai random vector.

Future return:

$$t\Delta$$

bergantung secara conditional pada state:

$$P_{t\Delta \mid t}$$

Inilah yang nantinya menjadi fondasi Tahap B.

Kita ingin mendapatkan:

$$t\Delta \mid t$$

dan:

$$t\Delta \mid t$$

Jadi Tahap A sebenarnya membangun **conditioning information** untuk model forecast.

A.32. Ini juga menjelaskan mengapa kita tidak boleh mencampur state dan prediction

Tahap A:

$$t$$

adalah **observed/current information**.

Tahap B:

$$t, t$$

adalah **estimated future distribution**.

Tahap C/D:

$$t$$

adalah **decision**.

Jadi pipeline kita harus selalu:

$$t \mid P_{t\Delta \mid t} \mid t$$

bukan:

$$t \mid t$$

secara langsung.

A.33. Ini membuat sistem kita jauh lebih bersih

Kita sekarang memiliki tiga objek matematis:

State

$$t$$

Forecast distribution

$$t \Delta \quad t \quad t$$

Action

$$t = \quad t$$

Ini akan menjadi backbone seluruh sistem.

A.34. Struktur final Tahap A

Saya ingin menetapkan struktur formal berikut sebagai **spesifikasi Tahap A v1**:

$$t = t, tt, t, t, t, tt, tt$$

dengan:

$$\begin{aligned} t &= \{\Delta, t\}, \\ tt &= \{\Delta, t\}, \\ t &= \{, t, t\}, \\ t &= \{, t, t\}, \\ t &= \{t, t, P_t, P_t\}, \\ tt &= \{t, t_t, t\}, \end{aligned}$$

dan:

$$tt = \{t, P_{tt}, t, P_t, P_t\}$$

A.35. Satu keputusan desain penting: jangan memasukkan semuanya sekaligus

Walaupun kita sudah mendesain state yang sangat kaya, **implementasi pertama sebaiknya tidak langsung menggunakan seluruhnya**.

Saya sarankan kita membuat tiga tingkat:

State A0 — Baseline

$$o_t = \{t, tt, tt\}$$

State A1 — Cross-Asset

Tambahkan:

$$\{, t, t\}$$

State A2 — Execution-Aware

Tambahkan:

$$\{, , t, , t\}$$

Dengan begitu setiap peningkatan dapat diuji:

$$0 \quad 1 \quad 2$$

Kita bisa mengetahui apakah kompleksitas tambahan benar-benar meningkatkan hasil.

A.36. Dan ini penting untuk penelitian

Jangan sampai nanti kita memiliki sistem yang sangat kompleks dan menghasilkan:

$$= 1$$

tetapi kita tidak tahu apakah peningkatan tersebut berasal dari:

- market factor,
- order book,
- volatility,
- optimizer,
- atau overfitting.

Dengan baseline bertingkat:

$$\boxed{012}$$

kita dapat melakukan ablation study.

Misalnya:

$$\Delta_t = 1 - 0$$

dan:

$$\Delta = 2 - 1$$

Ini jauh lebih ilmiah.

A.37. Hubungan Tahap A dengan seluruh proyek

Sekarang kita sudah bisa melihat gambaran besarnya:

$$t \quad t, t, t$$

Kemudian:

Dengan demikian akhirnya kita memperoleh:

$$t \quad t \quad t \quad t\Delta$$

yang merupakan inti **adaptive systematic trading system** kita.

A.38. Kesimpulan Tahap A

Jadi, untuk saat ini fondasi matematis kita adalah:

$$t \quad t$$

di mana:

$$t = , , , , ,$$

dan kita menetapkan aturan fundamental:

$$t \quad t$$

serta:

$$\#$$

Kemudian Tahap B akan membangun:

$$P_{t\Delta} \quad t$$

dan dari distribusi tersebut kita mendapatkan:

$$tt$$

yang nantinya menjadi bahan bakar **optimization engine**.

Dan ada satu hal yang menurutku sangat penting untuk menjaga proyek ini tetap realistis: **gold, USD, BTC, ETH, indeks saham, dan aset target tidak kita anggap sebagai hubungan kausal yang sudah diketahui**. Mereka masuk sebagai observasi/factor candidate; kekuatan dan arah hubungannya harus **diestimasi dari data secara out-of-sample**. Dengan begitu kita tidak membangun cerita finansial terlebih dahulu lalu memaksa matematika membenarkannya.

Tahap A secara konseptual sudah selesai. **Tahap B nantinya akan menjadi jauh lebih matematis:** kita akan mengambil t ini dan menurunkan *conditional expected return*, *dynamic covariance*, *factor loading*, dan *regime-dependent distribution* sampai akhirnya memperoleh t dan t yang benar-benar layak diberikan kepada optimizer.

Prompt:

8/17/2026, 9:42:43 PM

lanjut tahap B

Response:

8/17/2026, 9:42:43 PM

Tentu. **Tahap B adalah bagian yang sangat penting**, karena di sinilah `market state` yang kita bangun pada Tahap A mulai diubah menjadi **distribusi probabilistik return masa depan**. Dengan kata lain, kita berpindah dari:

$\square$

menjadi:

$\square$

Dan saya ingin menjaga disiplin desain kita: **Tahap B belum melakukan optimasi trading**. Ia hanya menghasilkan informasi yang nanti dikonsumsi optimizer.

Tahap B — Conditional Forecast & Dynamic Risk Model

Secara keseluruhan kita ingin membangun:

$${}_t P_{t\Delta} \quad {}_t$$

kemudian mengambil:

$${}_t = {}_{t\Delta} \quad {}_t$$

dan:

$${}_t = {}_{t\Delta} \quad {}_t$$

serta:

$${}_t =$$

sehingga nantinya:

$$t = t, t t \quad t$$

Ini adalah objek yang akan masuk ke Tahap C/D.

B.1. Mengapa kita membutuhkan distribusi, bukan sekadar prediksi harga?

Misalkan sistem mengatakan:

$$, t_1 \quad t = 0.5$$

Sekilas ini terlihat bagus.

Tetapi ada dua kemungkinan:

Kondisi A

$$t_1 \quad 0.0005, 0.0002^2$$

Kondisi B

$$t_1 \quad 0.0005, 0.002^2$$

Expected return keduanya sama:

$$= 0.5$$

Tetapi risk-nya sangat berbeda.

Karena itu optimizer kita tidak boleh hanya mendapatkan:

Ia harus mendapatkan:

$$\boxed{?}$$

atau bahkan distribusi lengkap:

$$\boxed{P_{t\Delta} \quad t}$$

B.2. Definisi formal

Misalkan:

$$t\Delta = 1, t\Delta 2, t\Delta, t\Delta$$

adalah vector return masa depan.

Kita ingin memodelkan:

$$r_{t+\Delta t}$$

Secara umum:

$$r_{t+\Delta t} \sim N(\mu, \sigma^2)$$

dengan μ berisi parameter distribusi lain jika diperlukan.

Untuk versi pertama, kita dapat menggunakan:

$$r_{t+\Delta t} \sim N(r_t, \sigma^2)$$

tetapi ini adalah **working approximation**, bukan klaim bahwa return finansial benar-benar Gaussian.

B.3. Mengapa conditional?

Perhatikan perbedaan:

Unconditional

hanya menggunakan sejarah return.

Conditional

$$r_{t+\Delta t} | \mathcal{F}_t$$

mempertimbangkan keadaan pasar saat ini.

Ini jauh lebih sesuai dengan sistem kita.

Misalnya historical average return SOL:

$$\bar{r} = 0.1$$

Tetapi ketika:

$$r_t = -0.1$$

mungkin:

$$r_t = -0.2$$

Sedangkan ketika:

$$r_t = -0.3$$

mungkin:

$$t = 04$$

Jadi:

$$t$$

B.4. Kita mulai dari model factor

Sekarang kita gunakan struktur yang sudah kita tetapkan.

Untuk aset :

$$,t = t ,t$$

dengan:

$$t = 1,t,t$$

Dalam bentuk vector:

$$t = t t$$

dengan:

$$= 11 \quad 1 \quad 1$$

B.5. Apa arti ?

Misalnya:

$$= 12 \quad -04 \quad 010 \quad -02 \quad 015 \quad -0 \quad 005$$

dengan factor:

$$=$$

Maka untuk aset pertama:

$$_1 = _1 \quad 12 \quad -04 \quad 01 \quad _1$$

Jadi beta bukan sekadar indikator.

Ia merupakan **pemetaan matematis dari factor shock ke asset return**.

B.6. Penurunan expected return

Mulai dari:

$$r_{i,t+1} = \alpha_i + \beta_i r_{m,t+1} + \epsilon_{i,t+1}$$

Kita mengambil conditional expectation:

$$E_{t-1} r_{i,t+1}$$

Karena expectation linear:

$$= E_{t-1} \alpha_i + \beta_i E_{t-1} r_{m,t+1} + E_{t-1} \epsilon_{i,t+1}$$

Jika:

dan dianggap parameter:

$$\alpha_i = \alpha_i + \beta_i \alpha_m + \epsilon_{i,t}$$

Kita asumsikan:

$$E_{t-1} \epsilon_{i,t+1} = 0$$

Maka:

$$E_{t-1} r_{i,t+1} = \alpha_i + \beta_i E_{t-1} r_{m,t+1}$$

dengan:

$$E_{t-1} r_{m,t+1} = E_{t-1} r_{m,t} + \alpha_m$$

Ini adalah persamaan fundamental Tahap B.

B.7. Perhatikan sesuatu yang sangat penting

Expected return aset:

$$E_{t-1} r_{i,t+1}$$

sekarang berasal dari:

$$E_{t-1} r_{i,t+1}$$

Secara eksplisit:

$$E_{t-1} r_{i,t+1} = \alpha_i + \beta_i E_{t-1} r_{m,t+1}$$

Contohnya:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \epsilon_t$$

B.8. Tetapi ada masalah

Bagaimana kita memperoleh:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \epsilon_t$$

Kita membutuhkan **factor forecasting model**.

Ini berarti Tahap B sebenarnya mempunyai dua tingkat:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \epsilon_t$$

Bukan:

$$y_t = \phi_0 + \phi_1 y_{t-1} + \epsilon_t$$

secara black-box.

B.9. Factor forecast

Untuk setiap factor x_t , kita dapat membuat:

$$x_{t+1} = \phi_0 + \phi_1 x_t + \epsilon_t$$

Sehingga:

$$y_{t+1} = \phi_0 + \phi_1 y_t + \phi_2 x_{t+1} + \epsilon_t$$

Maka:

$$y_{t+1} = \phi_0 + \phi_1 y_t + \phi_2 (\phi_0 + \phi_1 x_t + \epsilon_t) + \epsilon_t$$

Ini membuat arsitektur model sangat jelas.

B.10. Tetapi jangan langsung memakai model machine learning yang rumit

Untuk baseline, saya sangat menyarankan model linear/dynamic terlebih dahulu.

Misalnya:

$$y_{t+1} = \phi_0 + \phi_1 y_t + \phi_2 x_{t+1} + \epsilon_t$$

Maka:

$$,t = t$$

Dengan demikian:

$$t = t$$

untuk representasi matrix yang sesuai.

B.11. Kenapa baseline linear penting?

Karena nanti kita ingin menjawab:

| Apakah peningkatan performa benar-benar berasal dari model yang lebih pintar?

Kita bisa membandingkan:

vs.

vs.

vs.

Tanpa baseline, kita tidak tahu apakah kompleksitas kita berguna.

B.12. Estimasi beta

Sekarang kita perlu memperoleh:

t

Untuk satu aset:

$=$

Kita memiliki observasi:

$t = 1, ,$

Dengan matrix design:

$$= 1 \quad 1,1 \quad ,11 \quad 1,2 \quad ,2 \quad 1 \quad 1, \quad ,$$

dan:

$$= ,1,2,$$

Model:

$$=$$

dengan:

$$= 1$$

OLS memberikan:

$$\boxed{=^{-1}}$$

jika invertible.

B.13. Tetapi beta tidak boleh dianggap konstan selamanya

Untuk short-term trading:

$$,t$$

lebih masuk akal daripada:

Karena hubungan:

atau:

dapat berubah.

Maka kita menggunakan rolling estimation:

$$\hat{t} = t$$

atau nantinya metode yang lebih canggih seperti exponentially weighted estimation atau state-space/Kalman filtering.

Untuk versi pertama:

□

sudah cukup.

B.14. Dynamic covariance

Sekarang bagian yang paling penting untuk optimizer.

Mulai:

$$t = \hat{t} \quad t$$

Covariance:

$$t = t \quad t$$

Karena:

$$t = \hat{t} \quad t$$

untuk bagian random:

$$\hat{t} \quad t$$

Dengan sifat covariance:

$$=$$

maka:

$$\hat{t} = t, t_t$$

Jika factor dan residual uncorrelated:

$$t, t = 0,$$

maka:

$$\hat{t} = t, t_t \quad t$$

dengan:

$$t = t$$

Ini adalah persamaan yang sangat penting untuk seluruh proyek.

B.15. Mengapa decomposition ini sangat berguna?

Kita dapat memisahkan:

$$=$$

Systematic:

$$_t = _{t,tt}$$

Idiosyncratic:

$$_t = _t$$

Jadi:

$$_t = _t \ t$$

B.16. Contoh sederhana

Misalkan:

$$= 120$$

dan factor covariance:

$$= 00004$$

Maka:

$$= 12000004 \ 12 \ 0$$

sehingga:

$$= 00005 \ 0000400004 \ 000025$$

Terlihat bahwa **satu common factor saja sudah menghasilkan covariance antar-aset.**

Ini adalah salah satu alasan factor model sangat natural untuk sistem kita.

B.17. Estimasi factor covariance

Kita perlu:

$$_{,t}$$

Versi sederhana:

$$_{,t} = _{t-1t}$$

menggunakan rolling window.

Tetapi karena kita memodelkan masa depan:

$$r_{t,t} = r_t \Delta t$$

secara ideal.

Ini berarti kita nantinya juga membutuhkan **conditional volatility model**.

B.18. Volatility clustering

Untuk satu factor:

$$r_t = \alpha + \beta r_{t-1}$$

dan:

$$r_t = \alpha + \beta r_{t-1}$$

dengan:

$$r_t$$

Kemudian:

$$r_{2t}$$

dapat dimodelkan secara dinamis.

Contoh klasik:

$$r_{2t} = \alpha + \beta r_{2t-1} + \gamma r_{2t-1}$$

yaitu GARCH(1,1).

Namun sekali lagi, saya **tidak akan menjadikan GARCH sebagai keharusan versi pertama**.

Kita dapat mulai dengan EWMA.

B.19. EWMA volatility

Definisikan:

$$r_{2t} = r_{2t-1} \left(1 - \frac{1}{2t-1} \right)$$

dengan:

$$0 \leq 1$$

Unroll:

$$2t = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

Teruskan:

$$\frac{1}{2t} = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

Jadi return terbaru mendapatkan bobot terbesar.

Ini sangat sesuai dengan short-term trading.

B.20. Dynamic correlation

Volatility saja belum cukup.

Kita membutuhkan:

$$t = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

dengan:

$$t = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

dan:

$$t = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

Ini adalah bentuk yang sangat berguna:

$$t = \frac{1}{2} \left(\frac{1}{2t-2} + \frac{1}{2t-1} \right)$$

B.21. Mengapa ini penting?

Misalnya:

$$1 = 2, 2 =$$

dan:

$$12 = 0$$

Maka covariance:

$$12 = 1212$$

Sehingga:

$$12 = 000200 = 00004$$

Risk portfolio kemudian bergantung bukan hanya pada individual volatility, tetapi juga hubungan:

$$\square$$

Ini nantinya menghasilkan coupling:

dalam QUBO/Ising kita.

B.22. Factor model vs direct covariance

Sekarang ada dua cara untuk mendapatkan covariance.

Cara 1

Direct:

$$tt$$

Cara 2

Factor:

$$tt = t, tt \ t$$

Saya menyarankan **keduanya kita hitung**.

Karena nanti kita dapat menguji:

$$tt$$

vs.

$$tt$$

Factor model biasanya jauh lebih menarik ketika:

B.23. Mengapa?

Misalnya:

$$= 100$$

aset.

Covariance matrix mempunyai:

parameter unik.

Untuk:

$$= 100$$
$$5050$$

parameter.

Sedangkan jika:

$$= 10$$

factors, kita hanya membutuhkan struktur factor:

$$\in^{10010}$$

plus:

$$\in^{1010}$$

dan residual variances.

Jauh lebih terstruktur.

B.24. Regime dependence

Sekarang kita tambahkan ide yang menurutku sangat penting dari desain awal kita:

$$t$$

sebagai market regime.

Misalnya:

$$t \in \{1, 2, \}$$

Kita dapat memiliki:

$$t$$

dan:

$$t$$

untuk setiap regime .

Jika:

$$_{,t} = P_t = \quad_t,$$

maka:

$t = \quad_{,t_t}$

B.25. Tetapi covariance mixture tidak sederhana weighted average

Ini penting secara matematis.

Jika:

$$= ,$$

dan:

$$P = = ,$$

maka:

$$=$$

Namun covariance:

$$=$$

Sehingga:

$$= - -$$

dengan:

$$=$$

Term kedua sering terlupakan.

Ia merepresentasikan **uncertainty akibat ketidakpastian regime**.

B.26. Ini sangat menarik untuk trading

Misalnya:

$$P = 05$$

dan:

$$P = 05$$

Meskipun masing-masing regime memiliki volatility kecil, perbedaan expected return antar-regime dapat membuat unconditional conditional-on-state variance menjadi besar.

Artinya:

$$\square$$

Ini sangat cocok untuk risk-aware optimizer kita.

B.27. Bagaimana kita menentukan regime?

Untuk baseline, kita dapat menggunakan fitur:

$$x_t = (y_t, y_{t-1}, \dots, y_{t-L+1})$$

Kemudian model:

$$P_t = \text{argmax}_r \pi_r(x_t)$$

Ada beberapa pendekatan:

Model 1 — threshold

Sederhana, tetapi kasar.

Model 2 — clustering

atau Gaussian Mixture Model.

Model 3 — Hidden Markov Model

□

lebih menarik karena regime memiliki persistence.

Untuk sistem kita, **HMM adalah kandidat yang sangat bagus**, tetapi saya akan menjadikannya versi B2, bukan baseline B0.

B.28. Dynamic factor model

Sekarang kita bisa menyusun keseluruhan model:

$$x_t = \sum_{i=1}^K \lambda_i f_{i,t}$$

dengan:

$$f_{i,t} = \mu_i + \sigma_i \epsilon_{i,t}$$

dan:

$$\epsilon_{i,t} = \rho \epsilon_{i,t-1} + \sqrt{1-\rho^2} \eta_{i,t}$$

Sehingga:

$$x_t = \sum_{i=1}^K \lambda_i (\mu_i + \sigma_i \epsilon_{i,t})$$

dan:

$$t = t, t t$$

Ini adalah **output utama Tahap B**.

B.29. Tetapi kita belum selesai

Karena target kita adalah **short-term trading**.

Expected return:

$$t$$

harus dibandingkan dengan biaya transaksi.

Misalnya expected gross return:

Expected cost:

$$t$$

Maka:

$$t, t = , t - t$$

Lebih lengkap:

$$t =$$

Ini sangat penting.

Jika:

$$= 00$$

tetapi:

$$= 010,$$

maka:

$$t = -002$$

Secara trading:

$$\square$$

meskipun forecast gross return positif.

B.30. Kita juga perlu probabilitas arah

Dari distribusi:

$$\sigma^2$$

kita dapat memperoleh:

$$P_0 =$$

Misalnya:

$$= 0.002$$

dan:

$$= 0.01$$

Maka:

$$P_0 = 0.205$$

Jadi expected return positif, tetapi probabilitas naik hanya sekitar 58%.

Ini jauh lebih informatif daripada:

"model memprediksi harga naik."

B.31. Bahkan kita dapat menghitung probability of hitting TP/SL

Ini akan sangat berguna di Tahap C/D.

Misalnya entry:

$$P_0$$

Stop:

$$P$$

dan target:

$$P_P$$

Kita ingin:

$$P_t$$

Untuk proses diffusion sederhana, kita dapat menggunakan model first-passage probability.

Jika log-price mengikuti:

$$x_t = x_0 + \sigma W_t$$

dengan:

$$0 = 0,$$

upper barrier:

$$0$$

dan lower barrier:

$$- 0,$$

maka probabilitas mencapai upper barrier sebelum lower barrier dapat diturunkan sebagai:

$$P = 1 - \frac{2^{2^2} - 2^2}{2^{2^2} - 2^2}$$

untuk 0 , dengan formulasi yang ekuivalen tergantung definisi barrier.

Untuk:

$$= 0,$$

hasilnya menyederhana menjadi:

$$P =$$

Ini belum kita gunakan sekarang, tetapi nantinya sangat berguna ketika menentukan TP/SL secara matematis.

B.32. Jadi Tahap B mulai terhubung langsung dengan ide awalmu

Kamu sebelumnya mengusulkan:

setiap trade memiliki risk \$1.

Sekarang kita dapat memisahkan:

Forecast

$$t, t$$

Risk

Reward

$$P$$

Probability

$$PP$$

sehingga expected value dapat ditulis:

$$= P_P - 1 - P_P -$$

dengan:

$$= 1$$

sebagai loss budget.

Ini nantinya menjadi salah satu objective yang sangat natural untuk trading engine.

B.33. Kita sekarang memiliki hierarki matematika

Perhatikan:



Tahap C/D:

□

Ini sangat bersih secara matematis.

B.34. Spesifikasi Tahap B versi pertama

Saya sarankan kita membuat **B0** terlebih dahulu:

$$t1 = \#1 \ t1$$

dengan:

Factor mean

$$\mu_t =$$

Factor covariance

$$\Sigma_t =$$

Factor loading

$$\Lambda_t =$$

Residual covariance

Versi pertama:

$$\Sigma_t = \Lambda_{t,1} \Lambda_{t,1}' + \Sigma_{t,2}$$

Output

$$y_t = \Lambda_t \mu_t + \epsilon_t$$

dan:

$$\epsilon_t = \Sigma_t \epsilon_t$$

B.35. Kemudian B1

Setelah B0 berhasil:

$$\square$$

sehingga:

$$\mu_t$$

dan:

$$\Sigma_t$$

menjadi time-varying.

B.36. B2

Kemudian:

$$\square$$

misalnya:

$$P_t = \dots$$

Kemudian:

$$t = \dots, t_t$$

dan:

$$t = \dots, t_t \dots, t_t - t_t - t$$

B.37. B3

Barulah kita dapat mencoba model nonlinear:

$$t = t$$

misalnya:

- Gradient Boosting,
- Random Forest,
- Neural Network,
- temporal model,
- atau model lain.

Tetapi model nonlinear tersebut **harus dibandingkan terhadap B0/B1/B2**, bukan langsung dianggap lebih baik.

B.38. B4 — Distributional forecast

Jika ingin lebih jauh:

$$t \Delta t$$

tidak lagi dianggap Gaussian.

Kita bisa menggunakan:

$$t$$

untuk heavy tails:

$$t,$$

Ini masuk akal secara finansial karena return memiliki tail risk yang sering lebih besar daripada Gaussian.

Kemudian:

$$P -$$

dapat dihitung secara lebih realistis.

B.39. Dan ini sangat relevan dengan risk \$1

Misalkan dua model menghasilkan:

$$= 0$$

Model Gaussian:

$$= 0.5$$

Model Student- t :

$$= 0.5, \quad = 4$$

Keduanya memiliki mean dan variance yang sama, tetapi tail probability berbeda.

Untuk trading dengan SL:

$$= 1,$$

perbedaan tail behavior tersebut dapat menghasilkan perbedaan besar pada:

$$P$$

Jadi pada akhirnya:

$$\square$$

B.40. Hal yang paling saya sukai dari desain ini

Sekarang sistem kita tidak lagi terlihat seperti:

"Saya punya AI yang memprediksi harga."

Ia menjadi:

$$\square$$

Itu jauh lebih dekat dengan cara kita seharusnya memformulasikan **systematic trading problem**.

B.41. Hubungan dengan quantum optimization

Dan sekarang kita bisa melihat dengan jelas mengapa QUBO/Ising akan muncul secara natural.

Tahap B menghasilkan:

$$t$$

dan:

$$t$$

Kemudian Tahap C/D akan membuat objective seperti:

$$t - t - - P$$

Perhatikan:

$$t$$

muncul pada quadratic term.

Artinya:

$$,t$$

akan menjadi coupling antara decision dan .

Secara konseptual:

$$,t ,t ,t$$

dan akhirnya:

$$,t$$

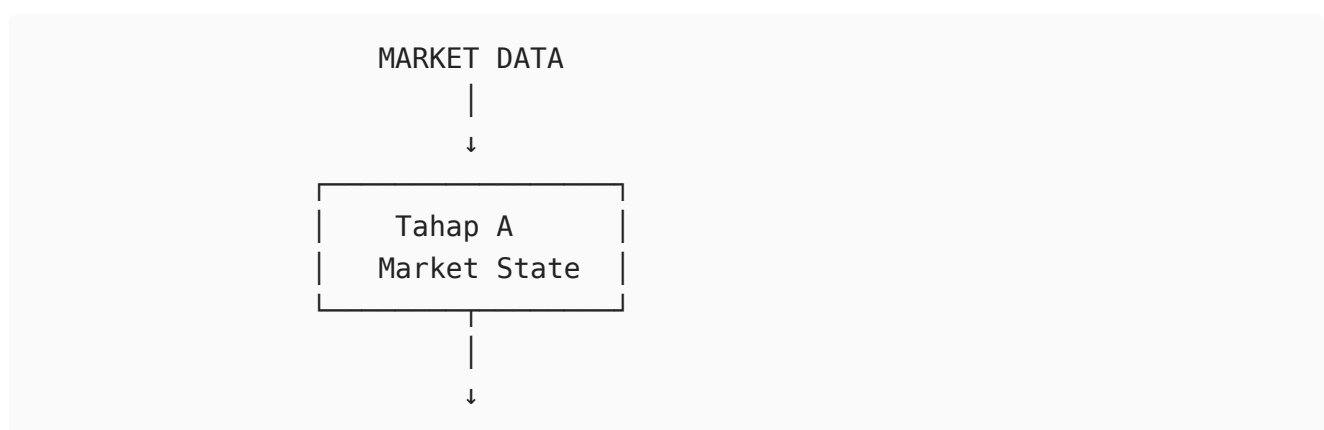
di Hamiltonian.

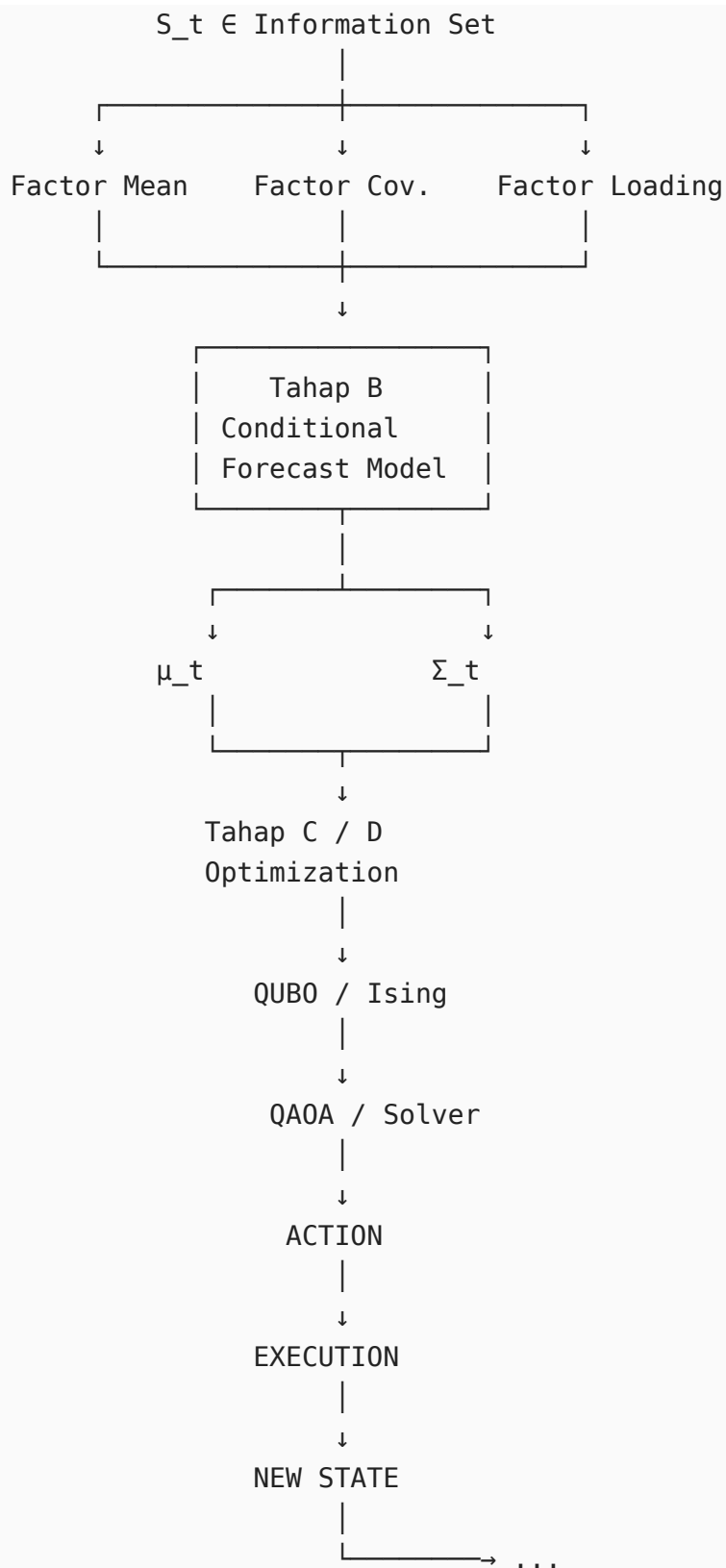
Jadi covariance yang kita bangun di Tahap B bukan sekadar statistik tambahan.

Ia pada akhirnya menjadi interaksi antar-qubit dalam Ising Hamiltonian kita.

Itulah hubungan yang sangat bagus antara finance dan quantum computing dalam arsitektur ini.

B.42. Pipeline yang sekarang sudah kita miliki





Posisi kita sekarang

Saya akan menetapkan **output resmi Tahap B** sebagai tiga objek:

$$t = \{t, t, t\}$$

dengan:

$$t = t_{\Delta} \quad t$$

$$t = t_{\Delta} \quad t$$

dan:

$$t =$$

Untuk **versi implementasi pertama**, saya sangat menyarankan kita **jangan langsung memasukkan HMM, GARCH, neural network, mutual information, dan lain-lain**. Kita bangun dulu B0:

□

dan pastikan ia dapat menghasilkan t dan t yang masuk akal secara *out-of-sample*. Setelah baseline itu kokoh, baru kita tambahkan regime dan model nonlinear satu per satu.

Dan satu hal yang sangat penting: **Tahap B belum boleh "melihat masa depan" ketika melakukan rolling estimation**. Untuk setiap decision time t , seluruh parameter t, t, t, t harus dibentuk hanya dari data t . Ini akan menjadi aturan keras ketika nanti kita masuk ke implementasi backtester.

Dengan fondasi ini, **Tahap C akan menjadi sangat menarik**, karena kita akan mengambil t, t, t dan secara matematis menurunkan **masalah trading optimization yang sebenarnya**: mulai dari menentukan *universe*, long/short/flat decision, entry, position size, \$1 risk budget, transaction cost, factor exposure constraint, hingga akhirnya membentuk **QUBO lengkap**. Dari sana barulah kita punya alasan matematis yang kuat untuk melakukan mapping:

□

Prompt:

8/17/2026, 9:44:08 PM

lanjut Tahap C

Response:

8/17/2026, 9:44:08 PM

Tentu. **Tahap C adalah titik di mana proyek kita mulai benar-benar berubah dari model forecasting menjadi sistem quantitative trading.**

Tahap A memberi kita **state pasar**:

t

Tahap B memberi kita **forecast distribusi return**:

$$t, t, t$$

Sekarang Tahap C harus menjawab:

Pada kondisi pasar t , aset mana yang sebaiknya diperdagangkan, dalam arah apa, dengan ukuran posisi berapa, pada harga entry/SL/TP seperti apa, sehingga expected utility maksimum dengan batas risiko yang telah ditentukan?

Dan di sinilah **TSP, Max-Cut, portfolio optimization, QUBO, Ising**, dan akhirnya **QAOA** mulai masuk secara natural.

C. Tujuan Tahap C

Kita ingin mengubah:

$$t, t, t$$

menjadi:

$$t$$

dengan t adalah keputusan trading optimal.

Secara abstrak:

$$t = \in t$$

dengan:

- : keputusan trading,
- : himpunan keputusan yang diperbolehkan,
- : objective/utility.

C.1. Jangan langsung berpikir "portfolio"

Untuk sistem kita, decision variable sebenarnya lebih kaya.

Untuk setiap aset , kita ingin mengetahui:

$$=$$

Tetapi:

tidak cukup.

Karena kita perlu membedakan:

Maka secara konseptual:

$$\in \{-1, 0, 1\}$$

Yaitu:

$$-1 =$$

$$0 =$$

$$1 =$$

Ini sudah merupakan **discrete optimization problem**.

Dan ini penting karena quantum optimization sangat cocok dengan decision variable diskrit.

C.2. Tetapi position size juga harus diputuskan

Misalkan:

$$= 1$$

Itu hanya mengatakan:

"Trade long."

Belum mengatakan berapa besar posisi.

Kita definisikan:

$$0$$

sebagai position size/notional.

Maka:

$$=$$

Jika:

$$= 1,$$

maka:

$$=$$

Jika:

$$= -1,$$

maka:

$$= -$$

Jika:

$$= 0,$$

maka:

$$= 0$$

C.3. Namun kita mempunyai constraint yang sangat spesifik

Sebelumnya kamu menentukan:

| setiap trade harus memiliki risiko \$1.

Ini sangat penting.

Misalkan:

$$P$$

adalah entry price.

Stop loss:

Risk per unit:

$$= P -$$

Jika posisi berukuran:

unit, maka dollar risk:

$$=$$

Kita ingin:

$$\square$$

dengan:

$$= 1$$

Jika ingin tepat \$1:

$$\boxed{= 1P -}$$

untuk aset dengan satuan harga yang sesuai.

C.4. Ini menghasilkan hubungan penting

Position size bukan dipilih sembarangan.

Ia berasal dari:

$$\frac{1}{P}$$

atau:

$$= P -$$

Contoh:

Entry:

$$P = 100$$

SL:

$$=$$

Maka:

$$= 1$$

Jika:

$$= 1,$$

maka:

$$= 1$$

Tetapi jika:

$$= ,$$

maka:

$$= 2$$

dan:

$$= 05$$

Jadi:

$$\frac{1}{P}$$

C.5. Mengapa ini berbeda dari portfolio optimization biasa?

Markowitz biasanya berbicara tentang:

sebagai bobot modal.

Sistem kita lebih dekat ke:

□

Kita tidak mengatakan:

"Saya mau mengalokasikan 10% modal."

Kita mengatakan:

"Jika trade ini gagal dan SL tersentuh, kerugiannya maksimal \$1."

Ini jauh lebih cocok untuk short-term systematic trading.

C.6. Sekarang kita masukkan forecast

Tahap B memberi:

$$= t$$

Expected PnL jika posisi :

$$P =$$

Untuk seluruh portfolio:

$$P =$$

Jika:

$$= 1, , ,$$

maka:

$$P =$$

C.7. Sekarang masuk covariance

Variance portfolio:

$$2P =$$

Ekspansi:

$$2P =$$

Perhatikan:

menciptakan interaksi antara trade dan .

Ini sangat penting.

Misalnya kita memiliki dua aset:

dan:

Jika:

$$, 1,$$

membuka long BTC dan long ETH bukan dua independent bets.

Mereka hampir merupakan:

$$\square$$

C.8. Maka objective dasar kita

Kita dapat menggunakan mean-variance utility:

$$= -$$

Dengan:

$$0$$

sebagai risk aversion.

Ini mirip Markowitz, tetapi sekarang:

adalah **trading positions**, bukan sekadar strategic portfolio weights.

C.9. Tetapi kita mempunyai factor exposure

Tahap B juga memberikan:

Portfolio factor exposure:

$$=$$

Misalnya factor:

$$= , , , P$$

Maka:

memberitahu kita berapa besar portfolio sebenarnya bertaruh terhadap faktor-faktor tersebut.

C.10. Ini sangat penting untuk contoh altcoin

Misalkan kita membuka:

dan:

Mungkin kita berpikir:

| "Saya punya dua trade."

Tetapi factor model mengatakan:

$$, = 1$$

dan:

$$, = 15$$

Maka:

$$= 1 \ 15$$

Jadi portfolio kita sebenarnya adalah:

$$\square$$

meskipun kita tidak membeli BTC.

C.11. Maka kita bisa memberikan constraint

Misalnya:

Secara matematis:

$$\square$$

Begitu juga:

dan:

C.12. Ini membuat sistem kita jauh lebih menarik

Karena sekarang optimizer tidak hanya bertanya:

"Trade mana yang menghasilkan profit terbesar?"

Tetapi:

"Trade mana yang memberikan profit terbesar **setelah memperhitungkan covariance dan factor exposure?**"

Itu jauh lebih dekat dengan quantitative portfolio construction.

C.13. Sekarang kita masuk ke entry

Forecast sebenarnya belum cukup.

Kita perlu menentukan:

$$P_t$$

Ada beberapa kemungkinan:

Market entry

$$P_t = P_t$$

Limit entry

$$P_t = P_t -$$

untuk long.

Atau:

$$P_t = P_t$$

untuk short.

Jadi entry distance:

□

sendiri menjadi decision variable.

C.14. Sekarang TP dan SL

Untuk long:

$$= P - \Delta$$

dan:

$$P = P \Delta_P$$

Untuk short:

$$= P \Delta$$

dan:

$$P = P - \Delta_P$$

Maka kita mempunyai dua parameter:

$$\square$$

dan:

$$\square P$$

C.15. Risk-reward ratio

Definisikan:

$$= P$$

Misalnya:

$$= 1$$

dan:

$$P = 2$$

Maka:

$$= 2$$

C.16. Tetapi RR tinggi tidak otomatis bagus

Misalnya:

$$=$$

Tetapi:

$$PP = 020$$

Expected value:

$$= 020 - 001$$

Sehingga:

$$= -020$$

Sebaliknya:

$$= 15$$

dengan:

$$PP = 00$$

memberikan:

$$\begin{aligned} &= 0015 - 0401 \\ &= 05 \end{aligned}$$

Jadi optimizer kita harus mempertimbangkan:

$$\boxed{PP, P,}$$

C.17. Expected trade value

Misalkan:

adalah keuntungan jika TP tercapai.

Dan:

adalah kerugian jika SL tercapai.

Maka:

$$\boxed{= P_P - P -}$$

Jika kita memaksa:

$$= 1,$$

maka:

$$\boxed{= P_P - P -}$$

Jika hanya ada TP/SL:

$$P = 1 - P_P,$$

maka:

$$\boxed{= P_P - 1 - P_P -}$$

C.18. Sekarang kita mempunyai dua level objective

Level individual trade

Level portfolio

$$= -$$

Kita tidak ingin memilih trade hanya berdasarkan:

Karena dua trade dengan EV tinggi dapat sangat berkorelasi.

Maka objective akhir harus menggabungkan keduanya.

C.19. Salah satu formulasi awal yang sangat bagus

Definisikan expected net return vector:

$$= \mu$$

Kemudian:

$$= -$$

Ini adalah:

$$-$$

C.20. Sekarang masalah kita sudah menjadi optimization problem

$$-$$

subject to:

Dan:

C.21. Tetapi kita belum mempunyai binary optimization

Karena:

masih continuous.

Untuk masuk ke QUBO/Ising, kita perlu membuat keputusan diskrit.

Saya ingin membangun ini **secara bertahap**, bukan memaksakan continuous optimization menjadi Ising.

C.22. Kita mulai dengan decision variable binary

Untuk setiap aset:

$$\in \{0, 1\}$$

dan:

$$\in \{0, 1\}$$

Interpretasi:

$$= 1$$

$$= 1$$

Kita harus mencegah:

$$= = 1$$

Constraint:

$$\boxed{1}$$

C.23. Posisi signed

Definisikan:

$$\boxed{= -}$$

Maka:

$$\in \{-1, 0, 1\}$$

0	0	0
1	0	+1
0	1	-1
1	1	forbidden

C.24. Ini sudah sangat dekat dengan QUBO

Kita punya:

$$= -$$

Expected value:

Risk:

dengan:

Jadi objective:

$$= - - - -$$

C.25. Constraint long-short

Kita tambahkan penalty:

$$P$$

Karena jika:

$$= = 1,$$

maka:

$$= 1$$

Sehingga objective menjadi lebih buruk.

Jadi:

$$= - P$$

Jika:

$$P,$$

solusi optimal tidak akan memilih keduanya.

C.26. Constraint jumlah trade

Misalkan maksimum:

trade sekaligus.

Kita ingin:

Untuk QUBO, constraint inequality dapat diubah menggunakan penalty.

Misalnya:

$$-x^2$$

Kemudian:

$$\boxed{P - x^2}$$

ditambahkan ke objective sebagai penalty.

C.27. Constraint factor exposure

Misalnya kita ingin:

$$= 0$$

untuk market-neutral terhadap factor .

Kita tambahkan:

$$\boxed{P^2}$$

Karena jika:

$$= 0,$$

penalty:

$$0$$

Jika exposure besar:

membesar.

C.28. Misalnya kita ingin membatasi BTC beta

Kita tidak harus selalu memaksa:

$$= 0$$

Kita bisa memberikan target:

$$=$$

Penalty:

$$\boxed{P - ^2}$$

Ini jauh lebih fleksibel.

C.29. Sekarang constraint gross exposure

Misalkan:

Untuk binary long-short:

$$=$$

jika long dan short tidak boleh simultan.

Maka:

$$\boxed{}$$

Sekali lagi dapat dipenalti:

$$P - ^2$$

C.30. Sekarang risk budget

Risk portfolio:

$$=$$

Jika kita ingin:

$$,$$

kita bisa menggunakan:

$$P_0 - \frac{1}{2}$$

Namun fungsi tidak langsung QUBO-friendly.

Untuk versi QUBO, kita lebih baik menggunakan:

quadratic equality dengan slack variables

$$- = 0$$

Kemudian:

$$P - \frac{1}{2}$$

Tetapi ini dapat meningkatkan jumlah qubit secara signifikan.

C.31. Karena itu saya menyarankan pendekatan berbeda untuk versi pertama

Jangan langsung memasukkan semua constraint.

Kita buat tiga tingkat.

C0

$$\square$$

C1

Tambahkan:

$$t - t$$

dan:

$$tt$$

C2

Tambahkan:

$$t$$

C3

Tambahkan:

$$t$$

dan constraint yang lebih kompleks.

Ini akan membuat debugging jauh lebih mudah.

C.32. Hubungan dengan Max-Cut

Sekarang muncul sesuatu yang menarik.

Risk term:

memiliki pairwise interaction:

Max-Cut juga memiliki objective pairwise:

$$1 -$$

untuk:

$$\in \{-1, 1\}$$

Artinya kedua masalah memiliki struktur:

$$\square$$

Untuk trading:

adalah coupling ekonomi.

Untuk Max-Cut:

adalah edge weight.

Jadi kita benar-benar sedang mengubah **asset correlation network menjadi optimization graph**.

C.33. Interpretasi graph

Anggap setiap aset:

sebagai node.

Edge:

memiliki weight:

Maka:

- positive covariance → posisi searah meningkatkan risk,
- negative covariance → posisi searah dapat mengurangi risk,
- covariance besar → interaction kuat.

Dengan demikian optimizer mencari kombinasi node yang memberikan:



Ini jauh lebih menarik daripada sekadar memilih top- predicted returns.

C.34. Sekarang TSP juga bisa masuk

TSP tidak cocok sebagai **core asset-selection problem** kita.

Tetapi TSP sangat masuk akal untuk **execution/order sequencing**.

Misalnya sistem ingin melakukan:

order.

Ada urutan:

1 2

Jika latency/slippage bergantung pada urutan eksekusi, kita dapat membuat:

sebagai cost berpindah dari order ke .

Kemudian:

Ini adalah TSP-like optimization.

Jadi:



sedangkan:



Kita tidak perlu memaksakan TSP ke masalah yang sebenarnya bukan TSP.

C.35. Dan Max-Cut?

Max-Cut bisa digunakan untuk:

□

Misalnya kita ingin membagi aset menjadi:

dan:

Dengan objective berdasarkan covariance/correlation graph.

Secara konseptual:

□

jika tujuan kita adalah hedging.

Sedangkan:

□

tergantung definisi objective.

C.36. Jadi kita tidak memiliki "satu quantum algorithm"

Arsitektur kita justru dapat memiliki beberapa optimizer.

Problem 1 — Asset selection

□

Problem 2 — Long/short allocation

□

Problem 3 — Position sizing

□

Problem 4 — Execution sequencing

□

Problem 5 — Rebalancing

□

Ini jauh lebih realistis.

C.37. Sekarang kita turunkan bentuk QUBO secara eksplisit

Definisikan vector binary:

$$= (x_1, x_2, x_3, x_4)$$

Kita ingin:

$$x_i \in \{0,1\}^2$$

Kenapa minimization?

Karena QUBO biasanya ditulis:

Sedangkan utility kita:

Maka:

$$-x_1 - x_2$$

C.38. Expected-return term

Utility:

$$= -x_1 - x_2$$

Maka minimization objective:

$$-x_1 - x_2$$

Jadi linear coefficient:

$$-x_1 - x_2$$

dan:

$$= -x_1 - x_2$$

Ini masuk akal.

Jika:

$$0,$$

long mendapatkan reward:

$$=$$

Short mendapatkan penalty:

C.39. Risk term

Dengan:

$$= - ,$$

maka:

$$= - -$$

Expand:

$$= - -$$

Karena:

$$= ,$$

maka:

$$= - 2$$

Maka risk term:

$$- 2$$

Ini secara langsung membentuk quadratic matrix .

C.40. Tambahkan exclusivity

$$P$$

Sehingga:

$$, , P$$

Dengan konvensi QUBO yang konsisten, faktor 2 pada off-diagonal harus ditangani hati-hati ketika mengimplementasikan . Ini nanti kita tetapkan sekali dalam kode agar tidak terjadi double-counting.

C.41. Tambahkan maximum number of trades

Constraint:

Untuk tahap awal, kita dapat menggunakan:

$$P - \frac{1}{2}$$

Expand:

$$= P - \frac{1}{2}$$

dengan:

$$=$$

Karena:

$$2$$

untuk binary variable memiliki sifat:

$$2 =$$

hanya jika sendiri binary; karena merupakan jumlah dua binary, kita sebaiknya expand langsung dalam , , bukan memperlakukan sebagai binary.

Ini contoh kecil mengapa **penurunan QUBO harus dilakukan secara hati-hati**, bukan sekadar "lempar constraint ke penalty."

C.42. Dan di sinilah kita mulai mendapatkan Hamiltonian Ising

Setelah QUBO:

$$=$$

kita gunakan transformasi:

$$= 1 - 2$$

dengan:

$$\in \{-1, 1\}$$

Maka:

$$= 1 - - 4$$

Substitusi menghasilkan:

$$=$$

Konstanta tidak mempengaruhi argmin.

Jadi:

$$=$$

Dan inilah Hamiltonian yang nantinya dapat diberikan kepada QAOA.

C.43. Ini merupakan hubungan paling penting sejauh ini

Kita sekarang mempunyai:

$$\square$$

$$t$$

$$t, t, t$$

$$\square$$

$$\square$$

$$\square$$

$$\square$$

$$\square$$

C.44. Tetapi ada satu masalah besar yang harus kita selesaikan

Kita tidak boleh membuat QUBO yang:

| secara matematis bagus tetapi secara finansial tidak bisa dieksekusi.

Contohnya optimizer mengatakan:

dengan:

$$= 001$$

dari entry.

Secara matematis risk \$1 dapat dicapai dengan posisi sangat besar.

Tetapi market impact dan spread dapat membuat strategi itu buruk.

Maka Tahap C harus memiliki:

C.45. Kita masukkan transaction cost

Misalkan:

adalah fixed cost.

Dan:

adalah proportional cost.

Maka:

=

Dalam binary approximation:

Sehingga:

$$= t$$

Ini hanya menambah linear term ke QUBO.

C.46. Slippage

Untuk short-term trading, kita juga dapat menggunakan:

=

Misalnya:

1

untuk model sederhana.

Atau:

1

untuk market impact nonlinear.

Jika position size binary-fixed:

= ,

maka:

yang kembali menjadi linear.

Ini sangat menguntungkan untuk QUBO.

C.47. Jadi objective yang lebih realistis

Sekarang:

$$= - - tt - - ttPt$$

Atau minimization:

$$= -$$

Ini adalah bentuk yang akan kita gunakan.

C.48. C0 — formulasi pertama yang saya sarankan

Untuk menghindari sistem terlalu kompleks, kita mulai dari:

$$-$$

subject to:

$$= -$$

dan:

$$1$$

Kemudian:

C.49. Apa yang belum masuk?

Belum:

- dynamic stop loss,
- trailing stop,
- TP adjustment,
- order-book placement,
- execution timing,
- portfolio turnover,

- regime transition,
- liquidity constraints,
- maximum drawdown,
- daily loss limit.

Dan itu memang sengaja.

Karena semua itu sebaiknya datang setelah kita mempunyai core optimizer yang benar.

C.50. Struktur Tahap C lengkap

Saya akan membaginya menjadi:

C1 — Trade candidate generation

Dari:

‘ ‘

tentukan candidate trade.

C2 — Discrete position selection

$$\in \{-1, 0, 1\}$$

C3 — Position sizing

$$= P -$$

C4 — Portfolio optimization

—

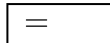
C5 — Factor constraints

C6 — Execution cost

C7 — QUBO



C8 — Ising



C.51. Dan satu hal yang sangat penting untuk skripsimu

Ada hubungan langsung dengan apa yang sudah kamu kerjakan sebelumnya.

Kamu sebelumnya telah mempelajari:

- ☐
- ☐
- ☐
- ☐
- ☐

dan:

- ☐

Sekarang semuanya dapat ditempatkan pada satu arsitektur yang jauh lebih besar:

- | | |
|--------------------------|--|
| ☐ | |
| ↓ | |
| ☐ | |
| ↓ | |
| ☐ | |
| ↓ | |
| ☐ | |
| ↓ | |
| ☐ | |
| ↓ | |
| ☐ | |

dan kalau kita ingin menggunakan penelitianmu sebelumnya:

☐

Jadi penelitian quantum portfolio optimization yang kamu kerjakan bukan sesuatu yang terpisah. Ia dapat menjadi **salah satu optimizer di dalam trading architecture yang lebih besar**.

C.52. Namun saya ingin kita membuat satu keputusan desain yang penting

Jangan membuat QAOA memutuskan semuanya.

Untuk sistem yang realistis, saya menyarankan:

☐

☐

☐

☐

Karena quantum optimization paling masuk akal ketika masalahnya memang **combinatorial**, bukan ketika kita memaksa seluruh sistem menjadi quantum.

C.53. Bentuk akhir Tahap C

Dengan demikian, pada waktu t :

Input

$$t = t, t, t$$

plus:

$$t = ,,,$$

Candidate generation

$$t = \{ \quad \}$$

Decision

$$, \in \{0, 1\}$$

Objective

$$= - P$$

QUBO

=

Ising

=

Solver

□

Output

{, , }

Dan menurutku inilah bagian yang paling penting

Kita sekarang telah berhasil membangun **rantai matematis yang utuh**:

$t \quad t, t, t$

dan yang paling menarik, t **dari Tahap B menjadi sumber coupling** . Jadi korelasi antar-aset bukan hanya statistik deskriptif; ia secara langsung menentukan interaksi antar-qubit dalam Hamiltonian trading.

Sementara itu, **TSP tidak perlu dipaksakan ke asset selection**. Ia lebih natural ditempatkan nanti pada *execution sequencing*. **Max-Cut** dapat muncul pada long/short partition atau hedging, dan **portfolio optimization** menjadi parent problem yang menghasilkan QUBO kita.

Dengan demikian kita tidak sedang membuat "bot trading yang kebetulan memakai QAOA". Kita sedang membangun:

□

yang kebetulan memiliki **quantum optimization layer**.

Dan setelah Tahap C ini, **Tahap D seharusnya bukan langsung coding QAOA**. Tahap D yang paling logis adalah **menurunkan QUBO trading kita secara lengkap dengan contoh numerik kecil**—misalnya 4 aset, masing-masing LONG/SHORT/FLAT, risk \$1, TP/SL, transaction cost, covariance, dan factor exposure—sampai kita benar-benar mendapatkan matriks J , lalu mengubahnya menjadi H dan C . Setelah itu baru kita bisa melihat dengan jelas **apa sebenarnya yang sedang dioptimalkan oleh QAOA** dan berapa qubit yang diperlukan.

Prompt:

8/17/2026, 9:44:47 PM

lanjut Tahap D

Response:

8/17/2026, 9:44:52 PM

Constructing example

I'm setting up a small four-asset, two-sided portfolio example with binary long/short variables. I'll derive covariance, returns, turnover costs, constraints, and the resulting numeric QUBO matrix.

Constructed an example

Computing a Four-Asset QUBO Portfolio Example

Computed and corrected a four-asset QUBO portfolio optimization example

Worked for 41s

Baik. **Tahap D** akan kita jadikan **tahap konstruksi matematis konkret**. Di sini kita berhenti berbicara pada level arsitektur dan benar-benar mengambil sebuah kasus kecil, lalu menurunkannya:

Saya akan menggunakan **4 aset** agar seluruh matriks masih bisa kita lihat secara eksplisit.

Catatan penting: angka di bawah adalah angka sintetis untuk demonstrasi matematis, bukan estimasi pasar aktual. Nanti ketika masuk implementasi, seluruh angka ini akan berasal dari data historis.

D.1. Kasus konkret

Misalkan pada waktu t , sistem sedang mengevaluasi empat kandidat:

$$= \{, , P, \}$$

Kita ingin memilih **tepat 2 trade**.

Setiap aset memiliki tiga kemungkinan:

Dan setiap trade yang dipilih memiliki risk budget:

$$= 1$$

D.2. Tahap pertama: menentukan stop loss dan position size

Misalkan sinyal menghasilkan:

Asset	Entry	SL distance
BTC	\$60,000	1.00%
ETH	\$3,000	1.20%
AAPL	\$200	1.00%
GLD	\$250	0.80%

Untuk setiap trade:

$$= P -$$

BTC

Stop distance:

$$0000001 = 00$$

Sehingga:

$$= 100 = 0001$$

ETH

$$= 0000012 =$$

Sehingga:

$$= 1 = 002$$

AAPL

$$p = 200001 = 2$$

Sehingga:

$$p = 05$$

GLD

$$= 250000 = 2$$

Sehingga:

$$= 0.5$$

Jadi **setiap trade yang dipilih mempunyai nominal posisi berbeda**, tetapi jika SL tersentuh:

$$1$$

Ini penting karena optimizer nantinya tidak boleh membandingkan BTC dan AAPL hanya berdasarkan nominal harga.

D.3. Expected trade value

Dari Tahap B, model forecasting kita menghasilkan expected net PnL untuk satu trade yang sudah dinormalisasi terhadap risk \$1.

Misalkan:

$$= 0.101501000$$

Artinya:

Asset	Expected net PnL
BTC	\$0.18
ETH	\$0.15
AAPL	\$0.10
GLD	\$0.08

Ini bukan berarti model berkata:

| BTC pasti menghasilkan \$0.18.

Melainkan:

$$t = 0.1$$

D.4. Covariance antar-trade

Sekarang bagian yang sangat penting.

Misalkan covariance matrix dari **risk-normalized PnL** adalah:

$$= \begin{pmatrix} 0.4 & 0.5 & 0.0 & -0.2 & 0.1 \\ 0.4 & 0.5 & 0.0 & -0.2 & 0.1 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ -0.2 & -0.2 & 0.0 & 0.5 & 0.4 \\ 0.1 & 0.1 & 0.0 & 0.4 & 0.5 \end{pmatrix}$$

Urutan:

$$P,$$

Perhatikan:

$$P, = 0.5$$

Sangat positif.

Sedangkan:

$$P, = -0.2$$

Negatif.

Ini akan menghasilkan konsekuensi yang menarik.

D.5. Mengapa covariance menjadi penting?

Bandingkan dua portfolio:

Portfolio A

dan:

Expected value:

$$0.1 \cdot 0.15 = 0$$

Tetapi variance:

$$^2 = , , 2,$$

Sehingga:

$$= 0.4 \cdot 0.1 \cdot 2 \cdot 0.5$$

$$= 25$$

Portfolio B

$$P$$

dan:

Expected value:

$$010 \ 00 = 01$$

Variance:

$$\begin{aligned} &= 02025 \ 00 \ 2 - 002025 \\ &= 0252 \end{aligned}$$

Jadi:

$$\boxed{= 0, = 25}$$

sedangkan:

$$\boxed{P = 01, = 0252}$$

Ini tepat alasan kita tidak boleh melakukan:

lalu mengambil dua teratas.

D.6. Risk aversion

Misalkan:

$$= 01$$

Objective dasar:

$$= -$$

Untuk BTC+ETH:

$$\begin{aligned} &= 0 - 0125 \\ &= 00 \end{aligned}$$

Untuk AAPL+GLD:

$$\begin{aligned} &= 01 - 010252 \\ &= 0154 \end{aligned}$$

Maka:

$$\boxed{P}$$

Walaupun:

$$P$$

Ini adalah perilaku yang kita inginkan dari optimizer.

D.7. Sekarang masukkan factor exposure

Sekarang kita ingin sistem tidak hanya melihat covariance.

Misalkan ada faktor:

Beta ilustratif:

$$= -0-0-002$$

Artinya secara ilustratif:

- BTC sensitif negatif terhadap DXY,
- ETH negatif,
- AAPL negatif,
- GLD sedikit positif.

Exposure portfolio:

$$=$$

Kita ingin menghindari exposure DXY terlalu besar.

Untuk demonstrasi pertama kita targetkan:

$$_{tt} = 0$$

Penalty:

$$P^2$$

Misalkan:

$$P = 05$$

D.8. Contoh factor exposure

Untuk:

$$,$$

kita mendapatkan:

$$= -0 - 0 = -14$$

Penalty:

$$05 - 14^2 = 0$$

Cukup besar.

Sedangkan AAPL+GLD:

$$= -0 \ 02 = -01$$

Penalty:

$$05 - 01^2 = 0005$$

Jadi optimizer semakin menyukai AAPL+GLD.

D.9. Transaction cost

Misalkan biaya efektif:

$$= 002002001001$$

Jadi:

- BTC: \$0.02,
- ETH: \$0.02,
- AAPL: \$0.01,
- GLD: \$0.01.

Biaya ini bisa berasal dari:

D.10. Sekarang kita buat decision variables

Untuk setiap aset kita membutuhkan dua binary variable:

dan:

Maka:

,

,

$$P^* P$$

,

Total:

$$\square$$

D.11. Kita susun vector

$$=_{PP}$$

Kemudian signed position:

$$= -$$

Sehingga:

$$= P$$

D.12. Constraint pertama: tidak boleh long dan short bersamaan

Kita ingin:

$$1$$

Daripada constraint inequality, untuk QUBO kita gunakan penalty:

$$\boxed{P}$$

Misalkan:

$$P = 5$$

Jika:

$$= = 1,$$

maka kita mendapatkan penalty:

$$5$$

Karena benefit trade normal jauh lebih kecil, solusi optimal akan menghindarinya.

D.13. Constraint kedua: tepat dua posisi

Untuk contoh ini kita pakai:

$$= 2$$

Kita inginkan:

$$= 2$$

Gunakan:

$$\boxed{P^2 - 2}$$

Misalkan:

$$P =$$

Perhatikan bahwa ini **memaksa tepat 2 trade**, bukan "maksimal 2 trade".

Untuk implementasi produksi nanti, kita bisa mengubahnya menjadi inequality dengan slack variable.

D.14. Objective lengkap

Sekarang:

$$\boxed{= - P^2 P P - 2}$$

Inilah **objective utama Tahap D**.

D.15. Mengapa expected return diberi tanda minus?

Karena QUBO biasanya kita tulis sebagai:

Sedangkan kita sebenarnya ingin:

Karena:

$$= -$$

Jadi:

$$= -$$

adalah reward yang diubah menjadi cost.

D.16. Kita sekarang ekspansi bagian return

Karena:

$$= - ,$$

maka:

$$- = -$$

Tambahkan transaction cost:

Maka linear coefficient awal:

Long

$$= -$$

Short

$$=$$

Sehingga:

Asset	Long coefficient	Short coefficient
BTC	-01	020
ETH	-01	01
AAPL	-00	011
GLD	-00	00

Sebelum penalty.

Ini sudah sangat intuitif.

Untuk BTC:

$$-01$$

membuat LONG menarik.

Sedangkan:

020

membuat SHORT tidak menarik.

D.17. Risk term

Kita memiliki:

Dengan:

= -

Maka:

= - -

Expand:

= - 2

Ini adalah bagian yang menghasilkan **coupling antar-qubit**.

D.18. Contoh coupling BTC–ETH

Dari:

, = 054

Untuk dua LONG:

2054

Dengan:

= 01,

maka:

010

Artinya memilih BTC dan ETH secara bersamaan menghasilkan additional risk cost.

Inilah analogi langsung:

□

D.19. Contoh AAPL–GLD

$$P, = -002025$$

Maka coupling:

$$\begin{aligned} 2P, &= 201-002025 \\ &= -000405 \end{aligned}$$

Jadi:

$$-000405_P$$

Negative coupling berarti memilih keduanya bersama-sama justru mengurangi risk term.

Ini adalah salah satu alasan kombinasi AAPL+GLD menjadi menarik.

D.20. Factor penalty juga menghasilkan coupling

Kita mempunyai:

$$P^2$$

Expand:

$$P$$

Jadi jika dua aset mempunyai beta factor yang sama arahnya, mereka memperoleh positive coupling.

Dengan kata lain:

$$\square$$

D.21. Sekarang kita dapat membentuk QUBO

Secara umum:

$$\square$$

Untuk contoh kita, dengan urutan 8 variable tadi, matrix yang dihasilkan adalah:

4 24 12 21 511 20 21 0224 21 0 2 20 52 202 0112 0 05 2 21 202 545

dan:

$$= -121 - 121 - 120 - 120 - 110 - 11 - 11 - 111$$

Konstanta:

$$= 12$$

D.22. Kenapa coefficient-nya terlihat besar?

Ini pertanyaan yang mungkin langsung muncul.

Karena:

$$P =$$

dan:

$$P = 5$$

menghasilkan coefficient penalty yang jauh lebih besar daripada expected return.

Itu memang disengaja.

Penalty harus cukup besar sehingga optimizer lebih memilih:

| solusi feasible dengan utility sedikit lebih rendah

daripada:

| solusi infeasible dengan utility tinggi.

Namun dalam sistem produksi, penalty tidak boleh dipilih asal besar. Kita akan membuat metode sistematis untuk menentukan lower bound penalty berdasarkan upper bound

objective benefit.

D.23. Mari kita lihat solusi optimal

Kita dapat melakukan brute-force karena hanya ada:

$$2 = 25$$

state.

Untuk contoh ini, ground-state solution adalah:

$$P = 1$$

dan:

$$= 1$$

Semua lainnya:

$$0$$

Jadi:

$$= 0, 0, 1, 1, 0, 0, 0, 0$$

Dan:

$$= 0, 0, 1, 1$$

Artinya:

$$P$$

$$\square$$

dan:

$$=$$

$$=$$

D.24. Mari kita verifikasi objective secara manual

Expected value:

$$= 010 \ 00 = 01$$

Risk:

$$= 0252$$

Risk penalty:

$$= 010252 = 00252$$

Transaction cost:

$$001 \ 001 = 002$$

DXY exposure:

$$-0 \ 02 = -01$$

Factor penalty:

$$05-01^2 = 0005$$

Exclusivity penalty:

$$0$$

Count penalty:

$$0$$

Maka:

$$= -01 \ 00252 \ 002 \ 0005$$

Sehingga:

$$\boxed{= -012}$$

Karena kita melakukan minimization, semakin negatif semakin baik.

D.25. Bandingkan dengan BTC + ETH

Untuk:

$$= 1, 1, 0, 0$$

Expected:

$$0$$

Risk:

$$25$$

DXY exposure:

$$-14$$

Maka:

$$= -0 \ 0125 \ 004 \ 0514^2$$

$$= -0 \ 025 \ 004 \ 0$$

Jauh lebih buruk.

D.26. Ini memberikan insight yang sangat penting

Kalau kita hanya memakai expected return:

akan menang.

Tetapi ketika kita mempertimbangkan:

$$t \quad ttt,$$

hasilnya:

$$P$$

Inilah alasan optimizer combinatorial kita diperlukan.

D.27. Sekarang kita ubah QUBO menjadi Ising

Kita gunakan:

$$= 1 - 2$$

Dengan:

$$\in \{-1, 1\}$$

Interpretasi:

$$= -1 \quad = 1$$

dan:

$$= 1 \quad = 0$$

D.28. Transformasi pasangan

Untuk:

kita memperoleh:

$$= 141 - -$$

Ini adalah formula yang mengubah QUBO menjadi Ising.

D.29. Bentuk Ising

Secara umum:

$$=$$

Konstanta:

boleh dibuang karena tidak mempengaruhi state minimum.

Sehingga:

$$=$$

D.30. Hubungan QUBO $\rightarrow$ Ising

Jika:

$$= ,$$

maka dengan konvensi $_2 =$, define:

$$=$$

Kemudian:

$$= -12 - 12$$

dan:

$$= 12$$

D.31. Hasil Ising untuk contoh kita

Coefficient linear :

$$_1 = -10 \quad _2 = -15 \quad _3 = -20 \quad _4 = -21 \quad _5 = -50 \quad _6 = -5 \quad _7 = -05 \quad _8 = -25$$

Dan pairwise coupling:

$$= 2$$

Misalnya:

$$_{12} = 14$$

karena:

$$12 = 24$$

Sedangkan:

$$1 = 1545$$

D.32. Hati-hati dengan konvensi Hamiltonian

Dalam banyak literatur Ising/QAOA kita menggunakan:

$$=$$

Tetapi dalam formulasi yang pernah kita gunakan untuk skripsimu:

$$= - -$$

Jadi:

$$= -$$

dan:

$$= -$$

Dengan demikian:

$$0$$

dan banyak menjadi negatif untuk contoh kita.

Ini **bukan perbedaan fisika**, hanya perbedaan konvensi tanda.

Kita harus menetapkan satu konvensi secara konsisten di implementasi.

D.33. Bentuk Hamiltonian akhir

Menggunakan konvensi skripsimu:

$$= - -$$

Inilah:

$$\square$$

Dan sekarang kita sudah benar-benar mempunyai Hamiltonian yang berasal dari masalah trading.

Bukan Hamiltonian yang dibuat sembarang.

D.34. Interpretasi fisiknya

Ini bagian yang menurutku sangat menarik untuk penelitianmu.

Dalam sistem ini:

$$\square$$

merepresentasikan **local preference** terhadap keputusan tertentu.

Sedangkan:

$$\square$$

merepresentasikan **interaction** antara dua keputusan.

Interaction tersebut dapat berasal dari:

Jadi:

$$\square$$

D.35. Ini persis alasan Ising sangat cocok

Bayangkan graph:

memiliki coupling kuat karena:

$$, 0$$

Sedangkan:

$$P$$

memiliki coupling negatif karena covariance negatif.

Maka sistem kita secara matematis berubah menjadi:

$$\square$$

D.36. Sekarang kita sampai ke QAOA

Setelah mendapatkan:

$$,$$

kita membutuhkan state yang mencari:

$$\square$$

QAOA menggunakan dua Hamiltonian.

Cost Hamiltonian

$$\square$$

yang baru saja kita turunkan.

Mixer Hamiltonian

Untuk standard QAOA:

$$\square$$

Atau sering ditulis:

$$= -$$

Tanda tidak terlalu penting selama konvensinya konsisten dengan unitary.

D.37. State awal

Standard QAOA:

$$=$$

dengan:

$$= 0 \ 12$$

Untuk:

$$= ,$$

kita memiliki:

$$=$$

D.38. Operator QAOA

Cost unitary:

$$= -$$

Mixer:

$$= -$$

Untuk $= 1$:

$$, =$$

Untuk layer:

$$= =1$$

D.39. Classical optimizer kemudian mencari

$$, =$$

Ini menghasilkan parameter:

$$1, ,$$

dan:

$$1, ,$$

Kemudian kita sampling quantum circuit.

D.40. Apa yang keluar dari QAOA?

Misalnya hasil sampling:

00110000

dengan probabilitas tinggi.

Dengan mapping:

$$= 1 - 2,$$

kita decode:

$$_P = 1$$

dan:

$$= 1$$

Sehingga:

$$P$$

D.41. Tetapi jangan langsung percaya bitstring paling sering

Ini sangat penting untuk trading.

Quantum optimizer dapat menghasilkan:

$$P$$

bukan hanya satu solusi.

Misalnya:

State	Probability
AAPL + GLD	0.41
ETH + GLD	0.17
BTC + GLD	0.12
BTC – ETH	0.08
lainnya	0.22

Maka kita sebenarnya mendapatkan **distribution over candidate portfolios**.

Ini bisa menjadi sangat berguna untuk sistem kita.

D.42. Kita bisa mendefinisikan confidence

Misalnya:

$$P = P$$

Jika:

$$P = 0.41,$$

confidence tidak terlalu tinggi.

Tetapi jika:

$$P = 0.05,$$

maka optimizer sangat konsisten terhadap solusi tersebut.

Kita dapat menggunakan:

$$\begin{bmatrix} P & P \end{bmatrix}$$

sebagai salah satu syarat eksekusi.

D.43. Bahkan entropy yang pernah kamu pelajari bisa masuk lagi

Kamu sebelumnya telah mempelajari Von Neumann entropy.

Di sini kita dapat menghitung entropy distribusi solusi:

$$= - \sum P_i \log P_i$$

Jika distribusi sangat terkonsentrasi:

$$P_1 \approx 1,$$

maka entropy rendah.

Jika distribusi tersebar:

$$P_i \approx 1/2,$$

entropy tinggi.

Jadi kita bisa mendefinisikan:

$$H(P) = - \sum P_i \log P_i$$

Ini akan menjadi hubungan yang menarik dengan penelitianmu mengenai entropy.

D.44. Dan sekarang kita bisa menghubungkan dengan EPG

Di penelitianmu sebelumnya, EPG menghasilkan:

Misalnya:

$$= (0, 0, 1, 1)$$

Ini dapat digunakan sebagai:

$$\begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix}$$

Daripada:

$$\begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}$$

kita dapat membangun initial state yang lebih dekat ke solusi EPG.

Secara konseptual:

$\square$

D.45. Dan sekarang kita mendapatkan arsitektur yang sangat kuat

Tahap A:

$\boxed{ttt}$

↓

Tahap B:

$\boxed{tt}$

↓

Tahap C:

$\boxed{tt}$

↓

Tahap D:

$\square$

↓

kemudian:

$\boxed{t}$

D.46. Tetapi ada satu koreksi penting terhadap desain awal kita

Kita **belum boleh** mengatakan:

"QAOA memutuskan TP dan SL."

Dalam formulasi yang baru kita bangun, QAOA memutuskan:

$\square$

sedangkan:

$, P, t$

sebaiknya berasal dari **trade construction layer**.

Untuk tahap berikutnya kita bisa membuatnya jauh lebih sophisticated.

Misalnya untuk setiap kandidat:

bukan hanya satu state:

Tetapi:

$\dots, 1$

untuk:

$$= 1, P = 2, \dots, 2$$

untuk:

$$= 15, P = ,$$

dan seterusnya.

Maka optimizer sebenarnya memilih:

$$\boxed{t, t, t, , P}$$

secara bersamaan.

Tetapi jumlah qubit akan meningkat.

D.47. Ini membawa kita ke formulasi yang jauh lebih menarik

Daripada:

$$\in \{-1, 0, 1\},$$

kita bisa membuat:

$$,,, \in \{0, 1\}$$

Contohnya:

$$,,, 1, 2$$

Dengan demikian QUBO dapat memilih:

BTC long, limit entry tertentu, SL 1 ATR, TP 2 ATR.

Ini jauh lebih dekat dengan sistem trading yang kamu bayangkan sejak awal.

Namun tentu saja:

akan meningkat sangat cepat.

D.48. Dan di sinilah kita harus berhati-hati

Misalnya:

$$t = 50$$

dan kita punya:

- 2 directions,
- 3 entry types,
- 3 SL choices,
- 4 TP choices.

Maka:

$$50 \cdot 2 \cdot 4 = 400$$

binary variables.

Artinya:

$$400$$

bahkan sebelum menambahkan auxiliary variables.

Ini tidak realistis untuk NISQ hardware saat ini.

Maka kita membutuhkan:

$$400$$

sebelum quantum optimization.

D.49. Ini membuat pipeline produksi kita menjadi

$$1000 \cdot t$$

↓

classical screening

↓

$$20 - 50t$$

↓

trade construction

↓

$$5 - 10t$$

↓

QUBO

↓

$$10 - 20$$

untuk prototype.

Baru kemudian kita menguji:

$$20 \quad 50 \quad 100$$

qubits pada simulator/quantum hardware yang tersedia.

D.50. Jadi Tahap D menghasilkan sesuatu yang sangat konkret

Kita sekarang mempunyai:

Decision space

$$, \in \{0, 1\}$$

Signed position

$$= -$$

Utility

$$= - - - P^2 - P_{tt}$$

QUBO

$$=$$

Ising

$$=$$

atau dengan konvensi skripsimu:

$$= - -$$

QAOA

$$, =_{=1} - -$$

Output

$$P$$

D.51. Satu insight besar yang saya ingin kamu pegang

Perhatikan rantai berikut:

↓

↓

↓

Jadi ada hubungan matematis langsung:

$$=$$

Dengan kata lain, **struktur korelasi pasar diterjemahkan menjadi struktur interaksi qubit.**

Dan factor model memberikan coupling tambahan:

$$=$$

Jadi nanti jika kita masukkan:

$$,,P,,$$

sebagai market factors, mereka bukan sekadar indikator yang kita tampilkan pada chart. Mereka benar-benar dapat memengaruhi **Hamiltonian trading**.

D.52. Tetapi Tahap D belum selesai secara sistem trading

Yang baru kita bangun adalah:

□

Kita belum membangun:

□

dan:

□

Padahal itu persis bagian yang kamu tanyakan di awal:

| kalau order belum terpanggil, apakah masih valid?

| kalau floating loss, apakah dipertahankan?

| kalau floating profit, apakah SL dinaikkan?

| kalau profit tinggi, apakah TP diperpanjang?

Itu tidak seharusnya ditangani oleh QUBO statis.

Kita membutuhkan **dynamic receding-horizon optimization**.

Dan menurutku itu adalah **Tahap E yang paling menarik**.

Kita akan mengubah sistem dari:

□

menjadi:

□

dengan state:

t

yang terus berubah.

Secara matematis:

$$t = t, t + \Delta t,$$

Jadi setiap satu jam, atau bahkan setiap beberapa menit:

$$t = t, t + \Delta t$$

Di situlah sistem kita mulai menjadi benar-benar "quantitative trading system", bukan sekadar portfolio optimizer.

Dan untuk Tahap E nanti kita perlu membangun secara matematis **state machine posisi: PENDING → OPEN → BEP → TRAILING → EXIT**, lalu menggabungkannya dengan **model probabilitas TP/SL, expected remaining value, trailing-stop optimization, order cancellation/replacement, dan receding-horizon/QUBO re-optimization**. Itu akan

langsung menjawab empat mekanisme trading yang kamu bayangkan pada awal percakapan.

Prompt:

8/17/2026, 9:46:27 PM

lanjut Tahap E

Response:

8/17/2026, 9:46:27 PM

Baik. **Tahap E** adalah bagian yang menurut saya paling penting untuk mengubah rancangan kita dari “optimizer yang memilih trade” menjadi sistem quantitative trading yang benar-benar dinamis.

Pada Tahap D kita memiliki:

Masalahnya, **trade yang optimal pada t_0 belum tentu optimal pada t_1 .**

Karena itu Tahap E akan memperkenalkan:

□

dengan prinsip:

Jangan menganggap keputusan entry sebagai keputusan final. Setiap posisi adalah keputusan yang terus dievaluasi berdasarkan informasi baru.

E.1. Masalah yang ingin kita selesaikan

Bayangkan sistem kita menghasilkan:

pada:

$$P_0 = 0,000$$

Dengan:

$$= 5,400$$

dan:

$$P = 1,200$$

Pada awalnya:

$$= 1$$

Tetapi satu jam kemudian ada beberapa kemungkinan.

Skenario A

Harga:

$$P_t = 5,500$$

Floating loss.

Apakah langsung ditutup?

Belum tentu.

Skenario B

Harga:

$$P_t = 0,00$$

Floating profit.

Apakah langsung menaikkan SL?

Belum tentu.

Skenario C

Harga:

$$P_t = 1,000$$

Profit besar, tetapi model sekarang mengatakan probabilitas continuation semakin kecil.

Mungkin lebih baik:

$$P$$

atau:

Skenario D

Harga:

$$P_t = 0,00$$

tetapi BTC dominance, ETH/BTC, DXY, gold, volatility regime, dan order flow berubah drastis.

Maka:

□

E.2. Kita membutuhkan state

Kita definisikan keadaan sistem pada waktu t :

t

sebagai:

$$t = t, P_{t,t,t,t,t,t}$$

Mari kita uraikan.

t

Market state:

$$_t = \{t, tt, t, t, t, t\}$$

P_t

Harga saat ini.

t

Volatility state.

Misalnya:

$$t, t, t$$

t

Factor state:

$$_t = \{, , P, , , \}$$

t

Order-book state:

$$_t = \{, t, , \}$$

t

Risk state:

$$_t = \{P, , , \}$$

t

Current positions:

$$t = \{s, P, t\}$$

E.3. Jadi sekarang keputusan bukan lagi

Pada Tahap D kita mempunyai:

$$t$$

Sekarang kita mempunyai:

$$\boxed{t}$$

sebagai **action**.

Contohnya:

$$t \in \{P, \dots, P, \}$$

Jadi:

$$\boxed{t = t}$$

Artinya:

keputusan trading merupakan fungsi dari keadaan pasar saat ini.

E.4. Kita bentuk Markov Decision Process

Secara matematis sistem dapat dipandang sebagai:

$$\boxed{t, t, t, t}$$

Dengan:

$$t \rightarrow t, t, t$$

Reward:

$$t$$

Tetapi saya **tidak menyarankan kita langsung menggunakan reinforcement learning**.

Kita bisa membangun policy secara model-based terlebih dahulu.

Ini lebih mudah dianalisis dan lebih sesuai dengan pendekatan matematis yang sedang kita bangun.

E.5. Expected remaining value

Misalkan kita sedang memegang posisi.

Harga sekarang:

$$P_t$$

Kita memiliki:

$$t$$

dan:

$$P_t$$

Definisikan:

$$pt = P_t$$

Maka jika:

$$t$$

adalah profit apabila TP tercapai, dan:

$$t$$

adalah loss apabila SL tercapai, expected value sederhananya:

$$t = pt_t - 1 - pt_t$$

E.6. Contoh

Misalkan:

$$t = 2$$

dan:

$$t = 1$$

Jika:

$$P = 0,$$

maka:

$$\begin{aligned} &= 02 - 041 \\ &= 0 \end{aligned}$$

Dengan:

$$= 1,$$

maka:

$$= 0$$

Masih sangat menarik.

E.7. Tetapi kita belum memperhitungkan opportunity cost

Misalkan posisi dipertahankan selama satu jam.

Ada kemungkinan muncul trade baru dengan expected value:

$$= 0.5$$

Jika posisi sekarang hanya memiliki:

$$= 0.1,$$

maka mempertahankan posisi lama mungkin tidak optimal.

Kita dapat mendefinisikan:

$$= -$$

Jika:

$$0,$$

kita memiliki alasan matematis untuk menutup posisi.

E.8. Ini menjawab pertanyaanmu tentang floating loss

Kamu sebelumnya mengatakan:

| jika floating loss, biarkan saja.

Saya akan mengubahnya menjadi aturan yang lebih rigor:

$$\square$$

Keputusan HOLD ditentukan oleh:

$$t \quad t$$

Misalnya posisi:

$$P = -0.4$$

Tetapi model memperkirakan:

$$PP_t = 0.5$$

Maka HOLD masih bisa rasional.

Sebaliknya:

$$P = -01$$

tetapi:

$$PP_t = 015$$

Maka exit mungkin lebih rasional.

E.9. Ini prinsip yang sangat penting

Kita tidak boleh menggunakan PnL saat ini sebagai satu-satunya dasar keputusan.

Karena:

$$P_t$$

adalah hasil masa lalu.

Yang kita butuhkan adalah:

$$\boxed{t}$$

Jadi sistem bersifat forward-looking.

E.10. Sekarang kasus floating profit

Misalkan:

$$P_t = 12$$

Pertanyaannya:

| Apakah kita harus menggeser SL?

Tidak otomatis.

Kita evaluasi tiga action:

$$1 =$$

$$2 =$$

$$=$$

Kemudian:

$$= t$$

E.11. Expected value HOLD

Misalnya:

$$P = 0.55$$

dan jika continuation:

$$= 20$$

Jika reversal:

$$= -0.2$$

karena SL sudah dinaikkan.

Maka:

$$= 0.55 \times 20 + 0.45 \times (-0.2)$$

$$= 10.1$$

E.12. Expected value EXIT

Jika kita tutup sekarang:

$$= 12$$

Maka:

$$12 \times 10.1$$

Secara expected-value murni:

$$\square$$

lebih baik.

E.13. Tetapi risk-adjusted EV lebih masuk akal

Expected return saja tidak cukup.

Kita bisa menggunakan:

$$= -$$

Atau lebih sophisticated:

$$= -1 - 2$$

Ini jauh lebih dekat dengan paradigma hedge fund.

E.14. CVaR

Misalkan:

$$= 0.5$$

Value at Risk:

$$0.5$$

Sedangkan:

$$0.5$$

mengukur rata-rata kerugian pada tail terburuk 5%.

Kita dapat memasukkan:

$$= -0.5$$

Dengan demikian sistem tidak hanya bertanya:

| "Berapa expected profit?"

tetapi:

| "Seberapa buruk tail risk jika saya mempertahankan trade ini?"

E.15. Sekarang kita masuk ke trailing stop

Misalkan:

$$P_t - P_0$$

untuk LONG.

Kita definisikan trailing distance:

$$t$$

Maka:

$$t = P_t - t$$

Tetapi:

$$t$$

tidak harus konstan.

Kita bisa membuat:

$$t = tt$$

E.16. Adaptive trailing stop

Misalkan:

$$t = t, t_t, t, tt, t$$

Contoh:

Jika:

$$tt = 0$$

dan trend kuat:

$$t = 25$$

Jika:

$$tt = 0,$$

maka:

$$t = 0$$

Jadi:

$$\boxed{tt \ t}$$

sedangkan:

$$\boxed{t \ ttt}$$

E.17. Mengapa ini lebih baik daripada aturan fixed?

Misalnya:

$$= P - 1$$

Dalam market yang sangat volatile, ini terlalu dekat.

Sedangkan:

$$= P -$$

dalam market yang tenang mungkin terlalu longgar.

Maka:

$$\boxed{t = P_t - tt}$$

lebih adaptif.

E.18. Optimasi t

Kita bahkan bisa menjadikan t sebagai optimization variable.

Misalkan:

$$t \in \{0.5, 1, 1.5, 2, 2.5, \dots\}$$

Untuk setiap kandidat:

,

kita hitung:

t

Kemudian:

$$= t$$

Ini jauh lebih menarik daripada:

"kalau profit 1%, SL digeser."

E.19. TP juga dapat dibuat dinamis

Misalkan:

$$P_t = P_{t+1}$$

Dengan:

$$t+1 = t$$

Jika continuation probability tinggi:

t

Jika continuation probability turun:

t

Sehingga:

$$P_t = P_{t+1}$$

E.20. Tetapi jangan memindahkan TP secara sembarangan

Misalkan awal:

$$P = 2$$

Harga sudah mencapai:

$$1$$

Model mengatakan momentum masih kuat.

Kita mungkin ingin:

$$P$$

Tetapi ada biaya opportunity:

$$P$$

meningkat.

Maka keputusan harus membandingkan:

$$P = 2$$

dengan:

$$P =$$

Jadi:

$$P = P_t$$

E.21. Sekarang kita punya tiga optimization problem

Untuk posisi terbuka:

Stop-loss

$$= t$$

Take-profit

$$P = P_t$$

Exit

$$= \in \{, , \} t$$

E.22. Sekarang kita gabungkan dengan QUBO

Ini bagian yang sangat penting.

Pada Tahap D:

$$t$$

dibentuk dari:

$$t, t, t$$

Tetapi pada waktu berikutnya:

$$t + \Delta t,$$

data berubah.

Maka:

$$t + \Delta t - t$$

$$t + \Delta t - t$$

$$t + \Delta t - t$$

Sehingga:

$$\boxed{t + \Delta t - t}$$

E.23. Receding horizon

Jadi kita menjalankan:

$$t - t$$

kemudian setelah:

$$\Delta t$$

$$t + \Delta t - t + \Delta t$$

Ini disebut:

$$\square$$

Kita tidak mengatakan:

"trade ini harus dipertahankan sampai TP."

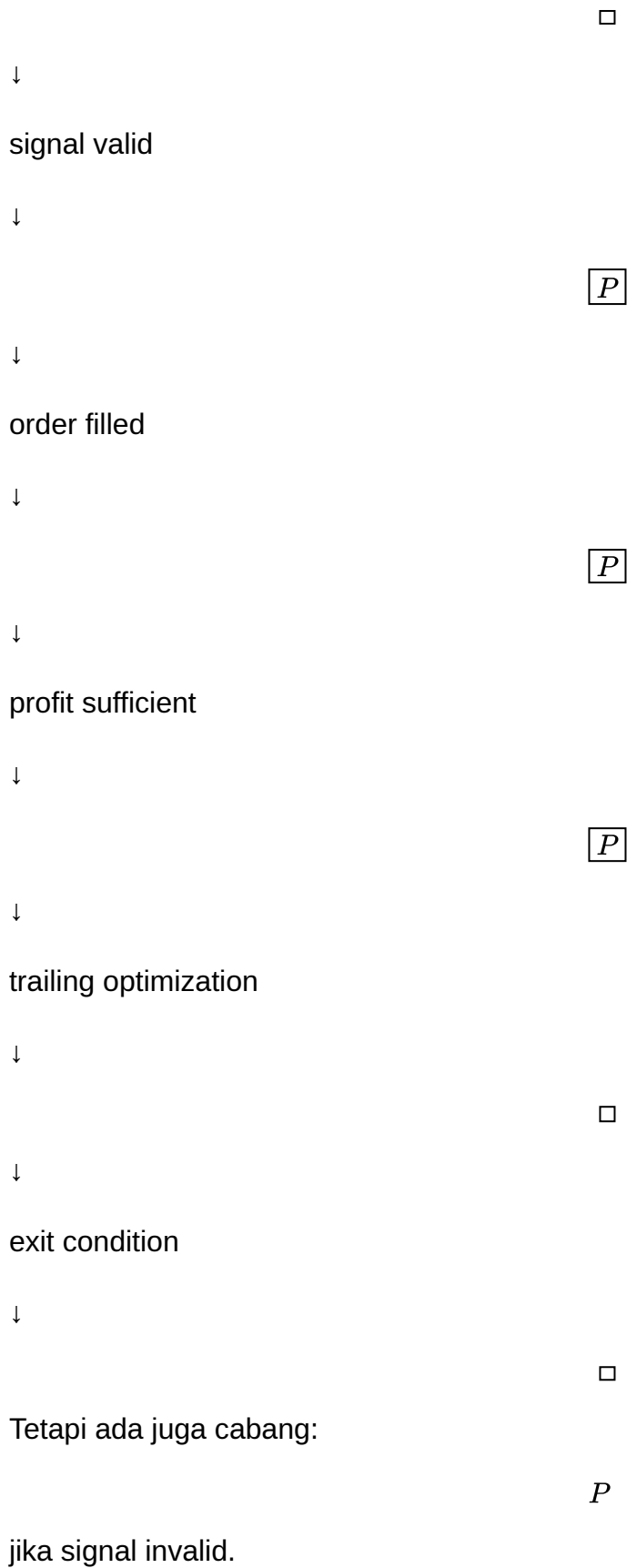
Kita mengatakan:

"berdasarkan informasi yang tersedia sekarang, keputusan terbaik adalah HOLD."

Lalu keputusan itu **ditinjau ulang**.

E.24. State machine sistem

Sekarang kita dapat membuat state machine:



E.25. PENDING state

Misalkan sistem memasang:

$$= P_0$$

Namun order belum filled.

Setiap interval:

$$\Delta t$$

kita hitung:

$$t$$

Jika:

$$,$$

maka:

$$\square$$

Ini menjawab pertanyaanmu:

"jika order book belum terpanggil, apakah masih bisa dipertahankan?"

Jawabannya:

$$\boxed{ttt}$$

E.26. Entry decay

Ini bahkan dapat dimodelkan sebagai:

$$t = 0^{-t} \Delta_t$$

Dengan:

$$0$$

Artinya semakin lama order tidak tersentuh, original signal semakin "basi".

Ini masuk akal karena model yang menghasilkan signal tersebut dibuat dari:

$$t_0$$

Bukan:

$$t$$

E.27. Kita dapat mendefinisikan signal half-life

Jika:

$$t = 0^{-t},$$

maka half-life:

$$t_{12} = 2$$

Ini merupakan parameter yang bisa kita estimasi dari data historis.

Misalnya:

$$t_{12} =$$

Maka setelah 3 jam tanpa fill, informasi signal secara statistik sudah jauh berkurang.

E.28. OPEN state

Setelah fill:

$$P_t = P$$

Kita hitung:

$$P_t = P_t - P$$

untuk LONG.

Untuk SHORT:

$$P_t = P - P_t$$

Kemudian:

$$t = P_{t0}$$

Dengan:

$$_0 = 1$$

Sehingga sistem selalu berbicara dalam unit:

□

E.29. Ini membuat semua aset comparable

Misalnya:

BTC:

$$P = 00$$

berarti:

0

AAPL:

$$P = 00$$

juga:

0

Tidak peduli harga aset berbeda jauh.

Ini sangat penting untuk portfolio multi-asset.

E.30. Risk budget portfolio

Kita tidak boleh hanya membatasi:

1

Kita juga membutuhkan:

Misalnya:

=

Maka maksimal tiga unit risk aktif.

Tetapi karena correlation:

bisa lebih besar daripada:

Jadi kita gunakan:

E.31. Risk budget dinamis

Bahkan:

dapat bergantung pada market regime.

Misalnya:

$$t = 0 \ t$$

Ketika volatility tinggi:

t

Ketika volatility rendah:

t

Jadi sistem secara otomatis menjadi defensive ketika market stress.

E.32. Sekarang kita masukkan DXY dan Gold

Kamu sebelumnya mengatakan ingin sistem tidak berhenti pada crypto.

Kita dapat mendefinisikan factor vector:

$$f_t = P_{tt} f_t$$

Untuk setiap asset:

$$f_{i,t} = P_{it} f_t$$

Jadi:

$$f_t = P_{tt} f_t$$

E.33. Factor surprise

Bukan hanya level factor.

Kita dapat menggunakan:

$$\Delta f_t = f_t - f_{t-\Delta t}$$

Lebih baik lagi:

$$f_{i,t} = P_{it} \Delta f_t$$

Sehingga sistem mengetahui:

□

Misalnya:

$$= 21$$

Artinya terjadi positive DXY shock.

Jika BTC memiliki:

$$, 0,$$

maka expected return BTC dapat dikoreksi turun.

E.34. Conditional expected return

Sekarang:

$$,t$$

tidak lagi sekadar historical average.

Kita buat:

$$_{,t} = \text{ ,t, t, t, t }$$

Misalnya:

$$_{,t} = t \text{ ,t ,t }$$

Ini adalah bagian yang nantinya dapat kita isi dengan:

- statistical model,
- gradient boosting,
- neural network,
- quantum model,
- atau kombinasi semuanya.

E.35. Tetapi optimizer tetap terpisah dari predictor

Ini arsitektur yang sangat penting:

$$P_t \text{ } t t$$

Model forecasting mengatakan:

$$P \text{ } t$$

Optimizer mengatakan:

$$t$$

Execution engine mengatakan:

τ, t

Jadi kita mempunyai tiga layer.

E.36. Arsitektur matematis lengkap

□

↓

$t\Delta t$

dan kembali lagi.

E.37. Inilah bentuk feedback loop

Secara matematis:

$t \quad t \quad t \quad t\Delta t \quad t\Delta t \quad t\Delta t$

Jadi sistem bukan:

Tetapi:

$\square$

E.38. Pseudocode matematis

Secara konseptual:

```
INITIALIZE capital
INITIALIZE risk budget

while market_is_open:

    S_t = collect_market_state()

    update:
        returns
        volatility
        correlations
        market factors
        order book
        current positions

     $\mu_t, \Sigma_t, \beta_t$  = forecast(S_t)

    Q_t = build_QUBO(
         $\mu_t$ ,
         $\Sigma_t$ ,
         $\beta_t$ ,
        constraints,
        transaction_cost
```

```

)

candidate_trades = solve(Q_t)

for pending_order:

    EV_entry = evaluate_entry(
        pending_order,
        S_t
    )

    if EV_entry < threshold:
        cancel_order()

for open_position:

    evaluate:
        P(TP | S_t)
        P(SL | S_t)
        expected continuation
        expected exit
        CVaR
        opportunity cost

    optimize:
        SL*
        TP*
        HOLD / EXIT

    execute optimal action

update risk budget

wait  $\Delta t$ 

```

E.39. Tetapi ada satu masalah besar: look-ahead bias

Ini **sangat penting** jika nanti kita melakukan backtest.

Misalnya kita menggunakan:

t

yang dihitung menggunakan data sampai:

$t - 1$

Maka sistem secara tidak sengaja melihat masa depan.

Backtest menjadi:

□

Semua feature harus memenuhi:

$$t = t$$

Tidak boleh:

$$t = t\Delta$$

E.40. Hal yang sama berlaku pada optimizer

QUBO pada:

t

harus dibangun dari informasi yang tersedia pada:

t

Bukan menggunakan covariance yang dihitung dari seluruh dataset.

Jadi:

$$t = t$$

Ini akan menjadi prinsip fundamental backtesting kita.

E.41. Kita juga membutuhkan walk-forward optimization

Tidak cukup:

$t \quad tt$

sekali.

Lebih realistis:

$$1 \quad t_1$$

kemudian:

$2 \quad t_2$

dan seterusnya.

Misalnya:

2021 – 01 2022 – 01

training,

2022 – 02 2022 – 04

testing.

Kemudian window bergeser.

E.42. Bahkan parameter trailing harus di-walk-forward

Jangan mencari:

$$= 2$$

menggunakan seluruh data 2021–2026 lalu mengatakan:

| strategy works.

Kita harus:

$$t =$$

menggunakan **training window**, lalu menguji pada window berikutnya.

E.43. Ini juga berlaku untuk QAOA

Parameter:

,

tidak boleh dioptimalkan menggunakan informasi masa depan.

Pada setiap periode:

t

dibangun.

Kemudian:

t, t

dicari.

Kemudian solusi diterapkan pada future interval.

E.44. Jadi kita memiliki dua jenis optimization

Ini perlu dibedakan.

Optimization level 1

Combinatorial optimization



menggunakan QAOA.

Tujuannya:

trade mana yang dipilih?

Optimization level 2

Continuous/control optimization



Tujuannya:

bagaimana trade dikelola?

Jadi QAOA tidak perlu dipaksa melakukan semuanya.

E.45. Ini justru membuat arsitektur lebih masuk akal

Kita menggunakan quantum optimization di tempat yang memang cocok:



Sedangkan continuous optimization dapat menggunakan:

- BFGS,
- SLSQP,
- SPSA,
- Bayesian optimization,
- grid search,
- atau metode lainnya.

Dengan demikian:



bukan kompetisi.

Mereka bekerja sebagai hybrid system.

E.46. Bentuk akhir Tahap E

Kita sekarang dapat mendefinisikan objective posisi:

$$t = P_{tt} \quad t, - P_{tt} - -$$

Kemudian:

$$t = \in t$$

Ini menjadi **otak Position Management Engine**.

E.47. Dan akhirnya, empat pertanyaan awalmu mendapatkan formulasi matematis

1. Order belum terpanggil

Hitung:

$$t$$

Jika:

$$,$$

maka:

$$\square$$

2. Floating loss

Jangan gunakan:

$$P \ 0$$

sebagai exit signal.

Gunakan:

$$\square$$

3. Floating profit

Evaluasi:

, ,

Kemudian:

$$=$$

4. Profit sangat tinggi

Optimalkan:

dan:

$$P$$

Jika:

$$P \ P,$$

maka:

$$P \ P$$

E.48. Hal yang paling menarik dari seluruh desain ini

Sistem kita sekarang mempunyai **tiga skala waktu**:

Fast

Order book:

$$t$$

Digunakan untuk:

$$t, , t$$

Medium

Trading signal:

$$t$$

Digunakan untuk:

Slow

Market regime:

Digunakan untuk:

t, t, ttt

Sehingga:

$\square$

bertemu dalam satu sistem.

E.49. Dan ini membawa kita ke Tahap F

Sampai sini kita sudah mempunyai:

ttt

t

t

$\square$

Ptt

Maka Tahap F seharusnya bukan sekadar "coding".

Kita perlu membangun **mesin backtesting yang benar-benar event-driven**.

Karena sistem kita memiliki:

- pending orders,
- partial fills,
- spread,
- slippage,
- TP,
- SL,
- trailing,
- cancellation,
- re-optimization,
- portfolio risk,
- multiple assets,
- factor data,
- dan QAOA/classical solver.

Jadi backtest sederhana seperti:

sudah tidak cukup.

Tahap F sebaiknya kita turunkan menjadi:

□

dengan **order book** → **order** → **fill** → **position** → **PnL** → **risk** → **reoptimization**, termasuk derivasi bagaimana mencegah **look-ahead bias**, **survivorship bias**, **data leakage**, **unrealistic fills**, dan **overfitting**.

Dan setelah mesin itu benar, barulah kita bisa menguji pertanyaan yang paling penting secara empiris:

□

Itulah eksperimen yang pada akhirnya akan membuat bagian quantum dari proyek ini **ilmiah**, bukan sekadar menempelkan QAOA pada strategi trading.

Prompt:

8/17/2026, 9:47:24 PM

lanjut Tahap F

Response:

8/17/2026, 9:47:25 PM

Tentu. **Tahap F adalah fondasi eksperimental seluruh sistem kita.** Kalau Tahap A–E adalah membangun "otak" strategi, maka Tahap F adalah membangun **dunia virtual yang cukup realistis untuk menguji otak tersebut.**

Dan saya ingin menekankan satu perubahan paradigma:

Kita tidak lagi membuat backtest sebagai fungsi sederhana `signal → return`. Kita membuat **event-driven market simulator** yang mereplikasi urutan kejadian: `market data → signal → order → fill → position → risk management → cancellation/modification → exit → PnL`.

Dengan begitu, nanti kita benar-benar bisa menguji apakah QAOA/quantum optimizer memberikan keuntungan dibandingkan metode klasik.

Tahap F — Event-Driven Backtesting & Simulation Engine

Arsitektur besar kita sekarang:

□

dan semuanya berjalan secara temporal:

$$t_0 \quad t_1 \quad t_2 \quad t$$

F.1. Mengapa backtest biasa tidak cukup?

Bayangkan kita mempunyai signal:

pada:

$$t = 10.00$$

Harga:

$$P_{1000} = 0.000$$

Strategi mengatakan:

$$t = 5.50$$

Kemudian harga bergerak:

$$5.0 \quad 5.0 \quad 5.50 \quad 5.00 \quad 0.100$$

Backtest sederhana mungkin melihat candle:

$$= 5.00,$$

lalu mengatakan:

| limit order pasti terisi.

Tetapi itu **belum tentu benar**.

Mengapa?

Karena kita tidak tahu:

- apakah order kita berada di depan atau belakang antrean,
- berapa banyak volume yang tersedia,
- apakah harga benar-benar menyentuh order,
- apakah fill hanya sebagian,
- bagaimana spread berubah,
- bagaimana latency memengaruhi order.

Jadi kita harus memodelkan:



F.2. Waktu menjadi variabel fundamental

Sebelumnya kita hanya memikirkan:

$$P_t$$

Sekarang kita harus memiliki:

$$\boxed{t}$$

sebagai informasi pasar yang tersedia **tepat pada waktu** t .

Misalnya:

$$t = \{P_{t,t}, t, t\}$$

Maka seluruh keputusan strategi harus memenuhi:

$$\boxed{t = t}$$

Tidak boleh:

$$t = t\Delta$$

Ini adalah prinsip anti-look-ahead bias.

F.3. Event-driven architecture

Kita definisikan event:

$$t$$

Contohnya:

$$t \in \{P, , , , , P, PP, PP, P\}$$

Backtester kemudian menjalankan:

$$\boxed{1 \ 2}$$

berdasarkan timestamp.

F.4. Market Event

Misalnya data datang:

$$tt$$

Informasinya:

$$t = \{t, t, t_t, t, t\}$$

Dari sini kita memperbarui:

t

F.5. State machine

Engine kita memiliki state global:

$t = ttt, Pttt, tt, tt$

MarketState

t

PortfolioState

t

OrderState

t

RiskState

t

F.6. Order state machine

Setiap order mempunyai state:

$\square$

↓

↓

P

↓

atau:

atau:

P

Ini penting karena order kita bukan hanya:

F.7. Contoh

Misalkan:

$$= 100$$

Kita ingin membeli:

$$100$$

Namun hanya 40 yang tersedia pada harga tersebut.

Maka:

$$= 40$$

Remaining:

$$= 0$$

Sehingga:

$$\boxed{=}$$

F.8. Fill model

Untuk market order sederhana:

$$P \ P$$

Untuk LONG:

$$P = P$$

Untuk SHORT:

$$P = P -$$

F.9. Slippage

Slippage dapat dimodelkan:

$$\boxed{t = t, t_t, tt_{t,t}}$$

Model sederhana:

$$_t = {}_t t_t$$

Sehingga:

$$t \ t$$

Ini sangat penting ketika nanti strategi kita mulai melakukan portfolio multi-asset.

F.10. Spread

Misalkan:

$$t = 100$$

dan:

$$t = 10002$$

Maka:

$$t = t - t = 002$$

Relative spread:

$$t = t - tt t^2$$

Strategi jangka pendek sangat sensitif terhadap:

$$t$$

F.11. Transaction cost

Total biaya kita definisikan:

$$t = t t_t t t t_t$$

Kemudian:

$$P = P -$$

Ini harus dimasukkan sejak awal.

Kalau tidak, strategi yang terlihat profitabel dapat berubah menjadi loss setelah biaya.

F.12. Market impact

Untuk ukuran order yang cukup besar:

$$\Delta P$$

Model sederhana:

$$t =$$

Dengan:

$$=$$

Biasanya:

$$0 \quad 1$$

F.13. Kita sekarang bisa menghitung execution price

Misalnya LONG:

$$P = {}_t P_t$$

Maka:

$$t = P - P$$

F.14. Position accounting

Setelah fill, kita memiliki:

$$t$$

sebagai posisi.

Untuk LONG:

$${}_t 0$$

Untuk SHORT:

$${}_t 0$$

Untuk FLAT:

$${}_t = 0$$

Nilai posisi:

$${}_t = {}_t P_t$$

F.15. Average entry price

Jika posisi masuk secara bertahap:

$${}_1, P_1, {}_2, P_2,$$

maka:

$$P = P$$

Ini penting ketika terjadi partial fill.

F.16. Unrealized PnL

Untuk LONG:

$$P_t = P_t - P$$

Untuk SHORT:

$$P_t = P - P_t,$$

dengan 0 sebagai absolute quantity.

Atau kita bisa menyatukannya menggunakan signed position:

$$P_t = {}_tP_t - P$$

F.17. Realized PnL

Ketika posisi ditutup:

$$P = P - P -$$

Kemudian:

$${}_tP_t = {}_{t_0}P_{\tau} P_t$$

F.18. Risk unit

Kita sebelumnya menetapkan:

$$= 1$$

Maka setiap trade harus mempunyai:

Misalnya entry:

$$P = 100$$

SL:

$$P =$$

Risk per unit:

$$= 1$$

Untuk:

$$= 1,$$

quantity:

$$\boxed{= P - P}$$

Sehingga:

$$= 1$$

F.19. Contoh lain

Entry:

$$P = 100$$

SL:

$$P =$$

Maka risk per unit:

$$2$$

Jika:

$$= 1,$$

maka:

$$= 12$$

Jika SL terkena:

$$1$$

sebelum biaya.

F.20. Tetapi kita harus memperhitungkan gap

Misalkan:

$$=$$

Namun harga melompat:

$$100 \ 5$$

Maka actual loss:

$$100 - 5 = 5$$

Bukan:

$$100 - = 2$$

Jadi:

$$\square$$

Untuk saham, gap risk sangat penting.

Untuk crypto 24/7, gap biasanya berbeda karakter tetapi tetap dapat muncul pada perubahan likuiditas ekstrem.

F.21. Maka kita membutuhkan risk multiplier

Misalnya:

$$=$$

Bukan sekadar:

$$- t$$

Jika expected execution loss:

$$= 125$$

maka:

$$= 1125 = 0$$

F.22. Order lifecycle

Sekarang mari kita ikuti satu trade.

t_0

Model menghasilkan:

$$= 1$$

Artinya LONG.

t_1

Trade constructor menghasilkan:

$$t = 100$$

=

$$P = 102$$

t₂

Order dikirim.

State:

P

t₃

Market:

=

$$= 10001$$

Order belum filled.

t₄

Market berubah.

Model baru:

Maka:

Trade tidak pernah terjadi.

Ini harus tercatat sebagai cancellation, bukan loss.

F.23. Contoh kedua

Order filled:

$$P = 100$$

SL:

TP:

102

Kemudian:

$$P_t = 100$$

Model menghitung:

Tetap HOLD.

F.24. Kemudian

Harga:

1012

Model menghitung:

$$= 100$$

Maka:

100

Sekarang maximum planned loss bukan lagi:

1

Tetapi posisi sudah memiliki locked profit:

0

F.25. Kemudian

Harga:

1025

Model mengatakan:

$$P = 104$$

Maka:

$$P \quad 102 \quad 104$$

F.26. Kemudian harga turun

SL terkena.

Realized PnL:

0

sebelum biaya.

Inilah contoh konkret dari **dynamic position management**.

F.27. Data yang diperlukan

Sekarang kita harus menentukan data minimum.

Untuk tahap awal:

Asset data

Market factors

Untuk saham internasional:

$,,P,,$

Untuk crypto:

$,,,$

Kemudian:

F.28. Untuk strategi short-term, timeframe harus multi-resolution

Saya menyarankan kita tidak hanya memiliki satu timeframe.

Misalnya:

Slow

1

untuk regime.

Medium

1

untuk signal.

Fast

5

untuk entry.

Jika data tersedia:

Microstructure

1

atau bahkan tick/order book.

Sehingga:

$$t = 1t, 1t, 5t, 1t$$

F.29. Mengapa multi-timeframe?

Misalnya:

Daily:

$$= P$$

Hourly:

$$t = P$$

5-minute:

$$P$$

1-minute:

$$0$$

Maka semua layer konsisten.

Sebaliknya:

Daily:

$$=$$

5-minute:

$$t = P$$

Mungkin LONG jangka pendek tetap bisa dilakukan, tetapi confidence harus lebih rendah.

F.30. Data leakage menjadi lebih rumit

Misalkan kita sedang berada di:

10 00

Candle 1H:

10 00 – 11 00

Kita **tidak boleh menggunakan close 1H** pada 10:15 karena candle belum selesai.

Kita hanya boleh menggunakan:

1000, 1015, 1015, 1015

Ini kesalahan yang sangat umum dalam backtest.

F.31. Point-in-time data

Prinsipnya:

$$t_t = ttt$$

Bahkan untuk fundamental data:

jangan menggunakan laporan yang baru dipublikasikan pada:

$t - 2$

seolah-olah sudah diketahui pada:

t

Untuk strategi kita, ini menjadi prinsip wajib.

F.32. Walk-forward framework

Sekarang kita masuk ke struktur validasi.

Misalnya:

1 =

1 =

1 =

Kemudian rolling:

2 2 2

Sehingga:

$\square$

berulang.

F.33. Mengapa tidak random train-test split?

Karena data finansial adalah time series.

Jika kita melakukan:

```
random train/test split
```

maka:

t_t

bisa masuk training sementara:

t_t

masuk test.

Ini merusak temporal causality.

Jadi:

$\square$

F.34. Parameter yang harus di-fit

Banyak sekali parameter kita:

$$= \{, , , P, , t, , t, \}$$

Masalahnya:

bisa sangat besar.

Jika kita terlalu agresif melakukan optimization:

$$= P$$

pada historical data, kita bisa mendapatkan:

$\boxed{tt}$

F.35. In-sample performance bukan bukti

Misalkan:

$$= 4$$

pada training data.

Itu belum berarti apa-apa.

Kita lebih tertarik pada:

Dan bahkan:

yang tinggi sekali juga harus dicurigai jika parameter terlalu banyak.

F.36. Parameter stability

Kita ingin melihat:

$$P$$

tidak hanya bagus di satu titik.

Misalnya:

$$= 1$$

menghasilkan Sharpe:

$$1$$

Tetapi:

$$= 1 \quad 1$$

juga menghasilkan:

$$15, 11$$

Ini bagus.

Sebaliknya:

$$= 10 \quad = 25$$

tetapi:

$$= 1 \quad 0$$

Itu tanda kuat overfitting.

F.37. Kita perlu baseline

Ini sangat penting untuk proyek quantum kita.

Kita tidak boleh hanya mengatakan:

| "QAOA menghasilkan profit."

Kita harus membandingkan:

t

Minimal:

Baseline 1

Equal weighting.

Baseline 2

Greedy selection.

Baseline 3

Classical QUBO solver.

Misalnya:

t

Baseline 4

MILP / exact solver untuk problem kecil.

Baseline 5

Heuristic portfolio selection.

F.38. Baru kemudian QAOA

Misalnya:

yang sama diberikan kepada:

tt

Semua menghasilkan:

Kemudian masing-masing diteruskan ke **execution simulator yang sama**.

Ini sangat penting.

Kalau execution model berbeda, perbandingan tidak fair.

F.39. Eksperimen yang benar

Kita definisikan:

$$= \{, , t, \}$$

Untuk setiap:

,

gunakan:

$$\boxed{t, t, , t}$$

yang sama.

Kemudian ukur:

P

F.40. Metrics

Jangan hanya menggunakan profit.

Minimal:

tt

tt

t

t

$$P_{tt}$$

$$t_t$$

F.41. Sharpe Ratio

Dengan return:

$$t,$$

maka:

$$= - -$$

Untuk intraday, harus dipilih sesuai frekuensi sampling secara konsisten.

F.42. Maximum Drawdown

Equity:

$$t$$

Running maximum:

$$t = \tau t^\tau$$

Drawdown:

$$t = t - t_t$$

Maka:

$$= t^t$$

F.43. Tetapi untuk strategi kita ada metric tambahan

Karena kita menggunakan optimizer kombinatorial:

Solution quality

$$= -$$

Jika exact solution tersedia.

F.44. Quantum-specific metrics

Kita bisa mengukur:

$$tt$$

Untuk maximization:

$$\boxed{=}$$

Kemudian:

F.45. Ini penting untuk tesis/proyekmu

Karena kalau QAOA menghasilkan:

$$t = 1$$

sementara classical:

$$t = 1,$$

itu menarik.

Tetapi kalau QAOA membutuhkan:

$$10$$

circuit evaluations dan classical solver hanya:

$$10,$$

kita tidak bisa hanya mengatakan:

| QAOA lebih baik.

Kita harus mempertimbangkan:

$$\boxed{P \quad ttt}$$

F.46. Kita dapat membuat objective eksperimental

Misalnya:

$$\boxed{= \quad - \quad - \quad - \quad ttt}$$

Atau mempertahankan metrics terpisah agar tidak mencampur interpretasi.

Untuk penelitian, saya lebih menyukai **menyimpan semua metrik secara terpisah**, lalu membandingkan Pareto frontier.

F.47. Pareto frontier

Misalnya kita punya:

$$- =$$

dan:

$$- = ttt$$

Kita ingin mencari solusi yang tidak didominasi:

P_{tt}

Ini sangat relevan untuk membandingkan:

dengan:

F.48. Stress testing

Backtest normal belum cukup.

Kita harus menguji:

Spread meningkat

$$2$$

Slippage meningkat

Latency meningkat

$$t \ 5$$

Volatility meningkat

$$2$$

Liquidity turun

$$t \ 05t$$

Jika strategi langsung runtuh, kita tahu strategi terlalu rapuh.

F.49. Monte Carlo trade reshuffling

Misalkan kita memiliki trade returns:

$$r_1, r_2, \dots$$

Kita dapat melakukan bootstrap:

$$r_1^*, \dots$$

Kemudian membentuk equity curve baru.

Ulangi:

$$= 10,000$$

kali misalnya.

Kita mendapatkan distribusi:

$$r_t$$

$$t,$$

Bukan hanya satu angka.

F.50. Probability of ruin

Jika risk per trade:

$$= 1,$$

dan capital:

$$0$$

Kita dapat mengestimasi probabilitas:

$$P_t$$

Ini jauh lebih informatif daripada hanya melihat CAGR.

F.51. Regime stress test

Karena kita memiliki:

$$r_t = \mu, P, \dots$$

kita dapat membagi data:

$$1 =$$

2 =

=

4 =

Kemudian:

P

Ini memungkinkan kita mengetahui:

kapan QAOA-based strategy bekerja dan kapan tidak.

F.52. Ini membawa kita pada konsep yang lebih menarik

Kita sebenarnya tidak hanya menguji:

Kita menguji:

$t \ t \ tt$

Misalnya:

Regime	QAOA	Classical
Low vol	1.4 Sharpe	1.3
High vol	0.8	0.7
Risk-on	1.8	1.5
Risk-off	0.5	0.6

Mungkin QAOA unggul pada regime tertentu tetapi tidak pada lainnya.

Itu justru hasil penelitian yang jauh lebih menarik daripada klaim "quantum selalu lebih baik".

F.53. Arsitektur software yang saya sarankan

Saya akan membagi project menjadi:

```
TradeProject/  
|  
└─ data/
```

```
|   ├── raw/
|   ├── processed/
|   └── features/
|
|   ├── market/
|   │   ├── market_state.py
|   │   ├── orderbook.py
|   │   └── factors.py
|   │
|   ├── forecasting/
|   │   ├── returns.py
|   │   ├── volatility.py
|   │   ├── correlation.py
|   │   └── regime.py
|   │
|   ├── optimization/
|   │   ├── qubo.py
|   │   ├── ising.py
|   │   ├── classical.py
|   │   └── qaoa.py
|   │
|   ├── execution/
|   │   ├── order.py
|   │   ├── fill.py
|   │   ├── slippage.py
|   │   └── transaction_cost.py
|   │
|   ├── portfolio/
|   │   ├── position.py
|   │   ├── pnl.py
|   │   └── risk.py
|   │
|   ├── strategy/
|   │   ├── entry.py
|   │   ├── position_manager.py
|   │   └── exit.py
|   │
|   ├── backtest/
|   │   ├── engine.py
|   │   ├── events.py
|   │   └── simulator.py
|   │
|   ├── validation/
|   │   ├── walk_forward.py
|   │   ├── stress_test.py
|   │   └── monte_carlo.py
|   │
|   └── experiments/
```

```
├─ qaoa_vs_classical.py
├─ regime_analysis.py
```

Ini jauh lebih baik daripada satu file besar yang berisi seluruh algoritma.

F.54. Event loop utama

Secara matematis:

$$t_1 = t, t, t$$

dengan:

sebagai transition function.

Secara konseptual:

```
initialize()

for event in chronological_events:

    update_market(event)

    update_orders(event)

    process_fills(event)

    update_positions(event)

    update_pnl(event)

    update_risk(event)

    if decision_time(event):

        state = build_state()

        forecast = model(state)

        optimization = optimizer(forecast)

        orders = strategy(optimization)

        submit_or_modify(orders)

record_state()
```

F.55. Ini adalah "digital twin" kecil dari trading system kita

Kita sebenarnya sedang membuat:

□

yang memungkinkan kita bertanya:

"Apa yang akan terjadi jika algoritma ini benar-benar beroperasi?"

Tanpa harus mempertaruhkan uang nyata.

F.56. Namun ada satu hal yang belum kita selesaikan

Data order book historis.

Jika kita ingin benar-benar melakukan:

$t \ t \ t,$

OHLCV saja tidak cukup.

Idealnya kita memiliki:

2

Misalnya:

10000 100 150 00

dan:

10001 12010002 2001000 50

Dari sini kita dapat menghitung:

= -

F.57. Order Book Imbalance

$$t = -1 \quad - = 1 \quad = 1 \quad = 1$$

Dengan:

$$-1 \quad 1$$

Jika:

$$1,$$

bid dominance.

Jika:

$$-1,$$

ask dominance.

Ini bisa menjadi feature untuk:

$$t$$

F.58. Tetapi kita jangan langsung melompat ke L2

Untuk **versi pertama sistem**, saya justru menyarankan tiga tingkat fidelity.

Level 1 — Research backtest

Gunakan:

Model:

$$,$$

secara parametrik.

Level 2 — Execution-aware backtest

Tambahkan:

$$,, tt$$

Level 3 — Microstructure simulator

Tambahkan:

$$2,, t, t$$

Dengan demikian kita tidak membuat sistem terlalu kompleks sebelum fondasinya tervalidasi.

F.59. Urutan implementasi yang saya rekomendasikan

Jangan langsung membuat semuanya.

Kita lakukan:

1

Historical data engine.

↓

2

Event engine.

↓

□

Order simulator.

↓

4

Position/PnL engine.

↓

5

Risk engine.

↓

□

Dynamic position management.

↓

□

Walk-forward validation.

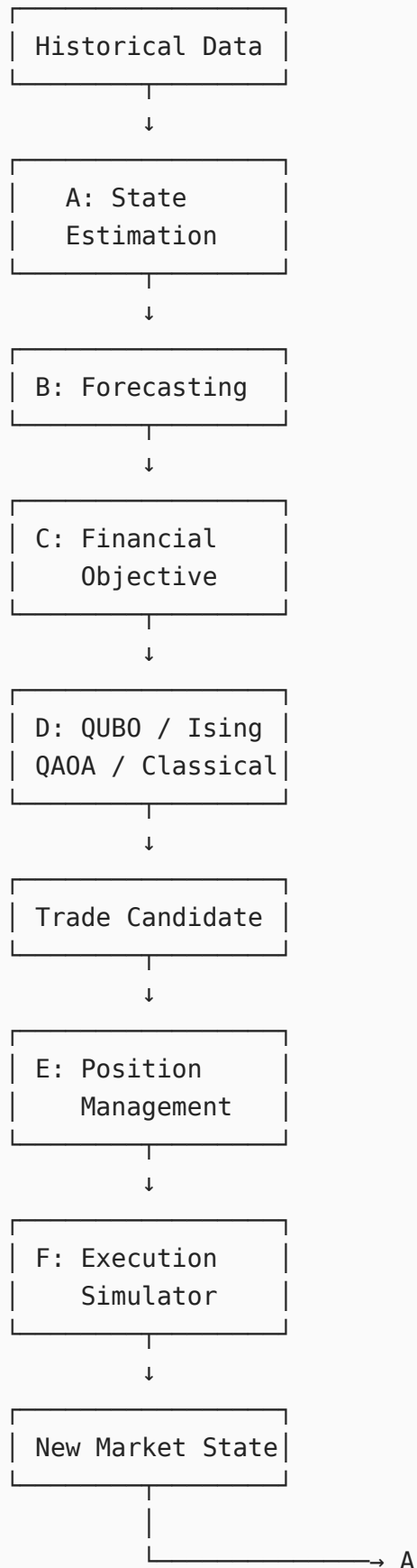
↓

□

QAOA vs classical experiment.

F.60. Hubungan Tahap A–F

Sekarang keseluruhan sistem kita mulai terlihat sangat jelas:



Jadi sistemnya **closed-loop**.

F.61. Satu prinsip yang saya ingin kita pegang

Jangan mengoptimalkan:

□

Yang harus kita optimalkan adalah:

□

Karena tujuan akhirnya bukan:

| menemukan parameter yang paling cocok dengan masa lalu.

Tetapi:

| menemukan algoritma yang tetap masuk akal ketika kondisi pasar berubah.

F.62. Dan di sinilah proyek ini mulai menjadi penelitian quantitative trading yang serius

Kita nantinya bisa menguji beberapa hipotesis:

H1

Apakah factor-aware forecasting lebih baik daripada price-only forecasting?

$$_1 P, P$$

H2

Apakah combinatorial optimization lebih baik daripada greedy selection?

2

H3

Apakah QAOA mendekati solusi classical/exact?

1

H4

Apakah dynamic position management meningkatkan risk-adjusted return?

4

H5

Apakah keunggulan QAOA tetap ada setelah transaction cost dan slippage?

5 t t

H5 justru merupakan salah satu eksperimen terpenting.

Karena sebuah metode bisa terlihat unggul sebelum biaya eksekusi, tetapi keunggulannya hilang setelah biaya dimasukkan.

F.63. Posisi kita sekarang

Dengan Tahap F selesai secara desain, pipeline kita sudah menjadi:

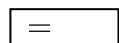


Dan ada satu hal yang menurut saya **sengaja harus kita tahan dulu**: jangan langsung menghubungkan ini ke Binance atau broker nyata.

Tahap berikutnya, **Tahap G**, sebaiknya kita gunakan untuk membangun **mathematical specification dari seluruh objective function dan QUBO final**—termasuk bagaimana **expected return, covariance, factor exposure (DXY/gold/SPX/BTC/ETH), transaction cost, turnover, position constraints, dan risk budget** semuanya masuk ke satu formulasi:

$\in \{0,1\}$

lalu diturunkan secara eksplisit sampai menjadi **Ising Hamiltonian**



yang benar-benar bisa kita masukkan ke **QAOA**.

Itu akan menjadi titik di mana seluruh pembahasan quantitative finance kita bertemu langsung dengan quantum computing yang sudah kamu pelajari.

Prompt:

8/17/2026, 9:49:12 PM

lanjut Tahap G

Response:

8/17/2026, 9:49:14 PM

Baik. **Tahap G** adalah titik di mana seluruh arsitektur yang kita bangun mulai dipadatkan menjadi satu formulasi matematis yang benar-benar bisa dieksekusi oleh **QAOA**.

Sampai Tahap F kita sudah punya banyak komponen:

Sekarang kita harus menjawab pertanyaan yang jauh lebih fundamental:

"Sebenarnya apa tepatnya yang sedang dioptimalkan oleh QAOA?"

Dan kita akan membangun jawabannya dari **objective finansial** → **binary decision** → **QUBO** → **Ising Hamiltonian** → **QAOA**.

Tahap G — Mathematical Formulation of the Trading Optimization Problem

Saya akan membaginya menjadi:

1. mendefinisikan universe aset,
 2. mendefinisikan keputusan trading,
 3. expected return,
 4. covariance/risk,
 5. market factors,
 6. transaction cost,
 7. turnover,
 8. exposure constraints,
 9. risk budget,
 10. mengubah semuanya menjadi QUBO,
 11. mapping QUBO → Ising,
 12. membentuk cost Hamiltonian,
 13. membentuk mixer,
 14. QAOA,
 15. interpretasi output QAOA sebagai trade.
-

G.1. Universe aset

Misalkan pada waktu t kita memiliki:

aset.

Kita definisikan:

$$= \{1, 2, \dots\}$$

Contoh untuk crypto:

$$= \{, , , P, \}$$

Untuk saham:

$$= \{P, , , \}$$

Tetapi sistem kita juga mempunyai **factor assets**.

Misalnya:

$$= \{, , P, \}$$

Untuk crypto:

$$= \{, , \}$$

G.2. Keputusan pertama: apakah aset dipilih?

Kita definisikan binary variable:

$$\in \{0, 1\}$$

dengan:

$$= \begin{cases} 1, & \text{dipilih} \\ 0, & \text{tidak dipilih} \end{cases}$$

Jadi:

$$= 1, 2, ,$$

Contoh:

$$= 1010$$

Artinya:

$$1,$$

dipilih.

G.3. Tetapi trading bukan sekadar "dipilih/tidak"

Kita membutuhkan arah.

Karena:

Ada dua pendekatan.

Pendekatan A — dua binary variables

Definisikan:

$$x \in \{0, 1\}$$

untuk BUY,

dan:

$$y \in \{0, 1\}$$

untuk SELL.

Dengan constraint:

$$x + y \leq 1$$

Pendekatan B — spin

Kita dapat menggunakan:

$$z \in \{-1, 0, 1\}$$

dengan:

$$z = 1, \quad 0, \quad -1,$$

Namun:

$$z \in \{-1, 0, 1\}$$

bukan binary.

Karena QUBO secara natural menggunakan:

$$x \in \{0, 1\},$$

saya menyarankan untuk **versi pertama menggunakan dua binary variables**.

G.4. Representasi BUY/SELL

Kita punya:

$$, \in \{0, 1\}$$

Net direction:

$$= -$$

Maka:

$$\in \{-1, 0, 1\}$$

Contoh:

0	0	0
1	0	+1
0	1	-1
1	1	0*

Tetapi kombinasi:

$$= = 1$$

harus dilarang.

G.5. Constraint arah

Kita tambahkan:

$$1$$

Dalam QUBO kita bisa menulis penalti:

$$P$$

Karena jika:

$$= = 1,$$

maka:

$$= 1$$

dan sistem menerima penalti.

G.6. Expected return

Sekarang kita membutuhkan:

$$t$$

Ini bukan sekadar historical mean.

Kita definisikan:

$$t = t_{\Delta} \quad t$$

Jadi:

$$t = 12$$

Nilainya dapat berasal dari model forecasting Tahap B.

Misalnya:

$$= t, t$$

G.7. Apa arti ?

Misalkan:

$$= 0002$$

Artinya model memperkirakan:

$$,t_{\Delta} = 02$$

Jika:

$$0,$$

maka LONG tidak menarik.

Tetapi SHORT mungkin menarik.

Karena expected directional return adalah:

$$\square$$

Jika:

$$= -1$$

dan:

$$= -0002,$$

maka:

$$= 0002$$

Jadi SHORT mendapatkan expected return positif.

G.8. Expected return objective

Kita ingin:

Karena QUBO biasanya ditulis sebagai minimization, kita ubah menjadi:

$$= - -$$

G.9. Risk

Sekarang kita memasukkan covariance.

$$= - -$$

Dengan:

$$= ,$$

G.10. Portfolio risk

Jika:

$$= 1, ,$$

maka:

$$_2 =$$

Ekspansi:

$$_2 =$$

Karena:

$$= - ,$$

maka:

$$_2 = - -$$

G.11. Ini adalah sumber coupling QUBO

Perhatikan:

Ada:

Artinya keputusan memilih aset bergantung pada apakah kita juga memilih aset .

Inilah struktur:

□

Dan ini alasan mengapa QUBO/Ising menjadi relevan.

G.12. Objective dasar Markowitz

Kita dapat menggunakan:

$$\begin{matrix} \square \\ = - \end{matrix}$$

Dengan:

0

Semakin besar:

,

semakin risk-averse.

G.13. Tetapi trading kita tidak boleh berhenti di Markowitz

Kita ingin:

$$,,P,, ,$$

masuk ke model.

Ada dua cara.

Cara pertama:

factor sudah masuk ke:

t

Tetapi itu belum cukup.

Kita juga ingin mengendalikan **factor exposure**.

G.14. Factor model

Kita definisikan:

$$=$$

Dengan:

$$= 1, ,$$

Untuk aset :

$$= 1, ,$$

Kita susun:

$$= \begin{matrix} 11 & 12 & 121 & 22 & 2 & 1 & 2 \end{matrix}$$

G.15. Portfolio factor exposure

$$=$$

Untuk factor :

$$=$$

G.16. Mengapa ini berguna?

Misalnya kita memiliki:

$$, = -0$$

Jika kita LONG BTC:

$$= 1,$$

maka:

$$0$$

Artinya portfolio kita memiliki negative DXY exposure.

Jika model macro mengatakan:

dengan kuat, kita mungkin ingin mengurangi exposure tersebut.

G.17. Factor exposure constraint

Misalnya kita tidak ingin:

Secara matematis:

$$-$$

Namun QUBO lebih mudah menggunakan squared penalty:

$$\boxed{P_{tt}^2}$$

G.18. Multi-factor penalty

Untuk semua factor:

$$\boxed{=_{=1} P_{=1}^2}$$

Di mana:

adalah target exposure.

Contoh:

$$= 0$$

Artinya kita ingin portfolio market-neutral terhadap DXY.

G.19. Ini menarik untuk strategi global

Misalnya kita trading:

$$P_{,,,}$$

Model menemukan semua expected return positif.

Naifnya:

$$P_{,,,}$$

Tetapi semuanya mungkin mempunyai:

$$P \cdot 0$$

Maka portfolio sebenarnya hanya satu taruhan:

Factor penalty mencegah optimizer menganggap keempatnya sebagai empat sumber alpha yang independen.

G.20. Risk decomposition

Covariance juga dapat ditulis melalui factor model:

$$\Sigma =$$

dengan:

$$=$$

dan:

$$=$$

Maka:

$$=$$

Sehingga:

$$\Sigma = t \cdot t'$$

Ini jauh lebih informatif daripada covariance matrix mentah.

G.21. Transaction cost

Sekarang kita masukkan biaya.

Untuk aset :

$$= t$$

Jika kita hanya mempertimbangkan apakah aset ditradingkan:

$$=$$

G.22. Tetapi kita juga ingin turnover

Misalkan posisi sebelumnya:

Kita tidak ingin optimizer setiap periode berubah total.

Definisikan:

$$\Delta = -$$

Transaction cost lebih tepat:

$$= \Delta$$

Masalahnya:

tidak quadratic.

Untuk versi binary sederhana kita bisa mengaproksimasi dengan penalti switching.

Misalnya jika posisi sebelumnya LONG:

$$= 1,$$

maka pindah ke FLAT/SHORT diberi biaya.

G.23. Binary switching penalty

Salah satu bentuk sederhana:

$$= P_t - 2 - 2$$

Karena:

$$^2 =$$

untuk binary variable, ekspansi menjadi quadratic.

G.24. Kenapa turnover penting?

Tanpa turnover penalty:

$$= 0,$$

optimizer mungkin melakukan:

setiap periode kecil.

Secara teori terlihat optimal.

Secara nyata:

Jadi:

□

G.25. Cardinality constraint

Kita mungkin tidak ingin 50 aset sekaligus.

Misalnya maksimal:

$$= 5$$

Maka:

$$\boxed{}$$

G.26. Mengubah cardinality menjadi penalty

Kita bisa menggunakan:

$$\boxed{= P - ^2}$$

Tetapi hati-hati.

Bentuk ini memberikan minimum ketika jumlah posisi tepat:

,

bukan:

Jika kita benar-benar ingin "maksimal", formulasi QUBO perlu auxiliary variables atau bentuk penalty yang sesuai.

Untuk versi awal, justru kita dapat menggunakan:

=

Misalnya:

| pilih tepat 5 kandidat trade.

G.27. Budget constraint

Misalkan setiap aset membutuhkan capital:

Kita ingin:

$$=$$

Penalty:

$$= P - \frac{1}{2}$$

G.28. Risk budget

Kita ingin:

Ini merupakan constraint quadratic yang agak lebih sulit.

Untuk binary QUBO, ada dua pendekatan.

Pendekatan pertama

Masukkan risk langsung ke objective:

Pendekatan kedua

Gunakan auxiliary/slack variables untuk mengubah inequality menjadi equality.

Untuk versi pertama, saya menyarankan pendekatan pertama.

G.29. Objective lengkap versi pertama

Sekarang kita sudah memiliki:

$$=$$

Mari kita tulis semuanya.

G.30. Full binary formulation

Dengan:

$$= - ,$$

kita definisikan:

$$C = \sum_{i,j} C_{ij} x_i x_j + \sum_i c_i x_i + c_0$$

dengan:

$$x_i \in \{0,1\}$$

Inilah inti mathematical specification sistem kita.

G.31. Kenapa fungsi ini adalah QUBO?

Karena semua komponennya maksimal quadratic.

Bentuk umum:

$$C_{ij} x_i x_j + c_i x_i + c_0$$

dengan:

$$c_i \in \mathbb{R}$$

Karena:

$$x_i \in \{0,1\},$$

maka ini adalah:

$$\square$$

G.32. Sekarang kita turunkan matrix

Ini bagian penting.

Misalkan:

$$C = \sum_{i,j} C_{ij} x_i x_j + \sum_i c_i x_i + c_0$$

Risk term:

Ekspansi:

$$C = \sum_{i,j} C_{ij} x_i x_j + \sum_i c_i x_i + c_0$$

Maka:

$$= - -$$

Karena:

$$= ,$$

maka:

$$=$$

Sehingga:

$$= - 2$$

G.33. Matrix block structure

Maka risk matrix memiliki struktur:

$$= \begin{matrix} & & \\ & & \\ & & \end{matrix}$$

Ini sangat elegan.

Bagian:

menghubungkan BUY-BUY.

Bagian:

$$-$$

menghubungkan BUY-SELL.

G.34. Return term

Kita memiliki:

$$- -$$

Sehingga:

$$-$$

Linear coefficient:

$$= -$$

dan:

$$=$$

G.35. Factor penalty

Untuk satu factor:

$$P - 2$$

Dengan:

$$= -$$

Definisikan:

$$= -$$

Maka:

$$=$$

Sehingga:

$$P - 2^2$$

Ekspansi:

$$P - 2^2$$

Jadi quadratic contribution:

$$\boxed{= P}$$

G.36. Ini berarti factor exposure menciptakan coupling

Jika:

$$0$$

dan:

$$0,$$

maka:

menghasilkan:

Jadi factor neutrality juga menghasilkan:

$$\square$$

G.37. Cardinality penalty

Misalkan:

$$=$$

Kemudian:

$$P - 2$$

Ekspansi:

$$P - 2 - 2 - 2 - 2$$

Karena:

$$2 =$$

jika binary hanya bila sendiri binary; tetapi:

$$=$$

dapat bernilai 2 jika constraint direction belum diberlakukan.

Lebih aman mengekspansi langsung , .

G.38. Direction constraint

$$P$$

Mudah:

$$= P$$

Jadi:

$$= = 1$$

menjadi mahal.

G.39. Switching penalty

Misalnya:

$$P - 2$$

Karena:

$$2 = ,$$

maka:

$$= P - 2^2$$

Karena:

$$\in \{0, 1\},$$

maka:

$$= P1 - 2$$

Jadi posisi sebelumnya mempengaruhi linear coefficient.

Ini berarti QUBO kita **dinamis**.

G.40. QUBO pada setiap waktu

Pada setiap:

$$t,$$

kita memiliki:

$$t = t, t, t, t, t-1$$

Jadi:

$$t_1 \quad t_2$$

secara umum.

Ini merupakan konsekuensi langsung dari dynamic optimization yang kita bahas di Tahap E.

G.41. Sekarang kita masuk ke Ising

QAOA bekerja secara natural dengan:

yang memiliki eigenvalue:

$$\in \{-1, 1\}$$

Sedangkan QUBO menggunakan:

$$\in \{0, 1\}$$

Gunakan mapping:

$$= 1 - 2$$

atau:

$$= 1 - 2$$

G.42. Mengubah linear term

Misalkan:

Dengan:

$$= 1 - 2,$$

maka:

$$= 2 - 2$$

Jadi:

$$2 - 2$$

G.43. Quadratic term

Misalkan:

Substitusi:

$$= 1 - 1 - 4$$

Maka:

$$= 141 - -$$

Jadi:

$$41 - -$$

G.44. Maka Hamiltonian

Secara umum:

$$= 0$$

menjadi:

$$= 0$$

Konstanta:

0

tidak memengaruhi argmin.

G.45. Koefisien Ising

Tergantung konvensi apakah menyimpan faktor dua pada off-diagonal, tetapi secara konseptual:

$\square$

Sedangkan:

$\square$

berasal dari kombinasi:

- linear QUBO coefficient,
- quadratic couplings yang melibatkan .

Untuk implementasi nanti kita harus memilih **satu konvensi matrix Q** dan tidak boleh mencampurnya.

Ini penting sekali karena kesalahan faktor 2 adalah salah satu bug paling umum ketika mengubah QUBO ke Ising.

G.46. Cost Hamiltonian

Kita akhirnya memperoleh:

$=$

Jika computational basis:

$= 12$,

maka:

$=$

Artinya:

$=$

Ini adalah koneksi langsung antara quantum mechanics dan optimization problem kita.

G.47. Interpretasi fisiknya

Misalkan:

$$0$$

Maka sistem cenderung memilih:

$$= 1$$

atau, tergantung mapping yang kita gunakan:

$$= 0$$

Sebaliknya jika:

$$0,$$

kecenderungannya berbeda.

Tetapi keputusan akhir tidak hanya ditentukan oleh:

Ada:

Jadi aset tidak berdiri sendiri.

G.48. Coupling

Misalnya:

$$0$$

Term:

cenderung meminimalkan energi ketika:

$$= -1$$

Artinya konfigurasi anti-aligned.

Sedangkan:

$$0$$

cenderung:

$$= 1$$

Ini memberikan interpretasi Ising.

Dalam konteks trading, coupling dapat berasal dari:

- covariance,
- factor exposure,
- cardinality,
- transaction cost,
- constraints.

Jadi:

□

Ia merupakan gabungan seluruh interaksi dalam objective.

G.49. Cost Hamiltonian bukan portfolio Hamiltonian murni

Ini penting.

Kita jangan mengatakan:

=

Lebih tepat:

$\{=, , , tt, t, \}$

G.50. QAOA

Sekarang kita mempunyai:

QAOA menggunakan mixer:

$\{= \}$

Untuk unconstrained binary optimization.

G.51. Initial state

Kita mulai dengan:

Dengan:

$= 0 \ 12$

Sehingga:

$$a_0 = \frac{1}{2} \sum_{i \in \{0,1\}} a_i$$

Artinya seluruh kandidat trading decision memiliki amplitudo awal yang sama.

G.52. QAOA layer

Untuk:

layers:

$$U_C = \exp(-i \sum_{i,j} C_{ij} \sigma_i^x \sigma_j^x)$$

Urutannya:

$$U_C = \exp(-i \sum_{i,j} C_{ij} \sigma_i^x \sigma_j^x)$$

kemudian:

$$U_C = \exp(-i \sum_{i,j} C_{ij} \sigma_i^x \sigma_j^x)$$

G.53. Mengapa cost Hamiltonian tidak mengubah probabilitas langsung?

Karena:

di computational basis hanya memberikan phase:

$$U_C = \exp(-i \sum_{i,j} C_{ij} \sigma_i^x \sigma_j^x)$$

Jadi:

$$U_C = \exp(-i \sum_{i,j} C_{ij} \sigma_i^x \sigma_j^x)$$

Kemudian mixer:

mengubah amplitudes antar state.

Interferensi antara keduanya menyebabkan state dengan cost yang lebih baik memperoleh probability lebih besar setelah parameter QAOA dioptimalkan.

G.54. Objective QAOA

Kita ingin:

$$\langle \psi | H | \psi \rangle$$

seminimal mungkin.

Jadi:

$$\min_{|\psi\rangle} \langle \psi | H | \psi \rangle$$

Perhatikan:

ini adalah optimization kedua.

G.55. Dua optimization yang sering tertukar

Optimization pertama:

$$\min_{\theta} \langle \psi(\theta) | H | \psi(\theta) \rangle$$

adalah **trading combinatorial optimization**.

Optimization kedua:

$$\min_{\theta} \langle \psi(\theta) | H | \psi(\theta) \rangle$$

adalah **variational parameter optimization**.

Jadi QAOA adalah nested optimization:

$$\min_{\theta} \left(\min_{|\psi\rangle} \langle \psi | H | \psi \rangle \right)$$

G.56. Setelah QAOA

Setelah parameter optimal:

$$|\psi(\theta^*)\rangle$$

kita memperoleh:

Kemudian kita lakukan measurement:

shots.

Contoh:

011001

muncul:

2, 40

kali.

State dengan frequency tertinggi menjadi kandidat:

G.57. Contoh interpretasi

Misalkan kita memiliki 4 aset:

$\{, , , \}$

Dengan 2 bits per asset:

1, 2, , 4

dan:

1, 2, , 4

Total:

=

qubits.

Misalnya hasil QAOA:

= 1, 0, 1, 0

= 0, 0, 0, 1

Maka:

= -

menjadi:

$\boxed{= 1, 0, 1, -1}$

Interpretasi:

G.58. Kemudian trade constructor

QAOA tidak langsung menghasilkan:

=

atau:

$P =$

Ia menghasilkan:

□

Kemudian Trade Construction Engine menggunakan:

t

untuk menentukan:

$t, P,$

G.59. Ini pembagian kerja yang sangat penting

QAOA

Menjawab:

| Apa yang harus diperdagangkan?

Classical risk/position optimizer

Menjawab:

| Seberapa besar?

Position manager

Menjawab:

| Kapan harus mempertahankan, memodifikasi, atau keluar?

Execution engine

Menjawab:

| Bagaimana order tersebut dieksekusi?

Dengan demikian kita tidak memaksa QAOA menjadi "super algorithm" yang melakukan segala sesuatu.

G.60. Position sizing

Misalkan QAOA memilih:

Trade constructor menghitung:

$$=$$

Jika:

$$= 1$$

dan:

$$= 0$$

maka:

$$= 125$$

G.61. Tetapi kita perlu portfolio risk adjustment

Jika ada tiga posisi:

$$,,$$

masing-masing:

$$= 1,$$

tidak berarti total risk:

Karena:

$$, 0$$

Jadi kita menghitung:

$$_2 =$$

Kemudian melakukan scaling:

$$\boxed{=}$$

dengan:

$$= 1,$$

G.62. Ini memberi dua lapis risk control

Lapisan 1

QUBO:

Lapisan 2

Position sizing:

Ini jauh lebih aman daripada hanya mengandalkan optimizer.

G.63. Sekarang masalah penting: discrete vs continuous

Kita sebenarnya mempunyai:

$$\in$$

untuk position size.

Sedangkan QAOA bekerja dengan:

$$\in \{0, 1\}$$

Jangan dipaksakan semuanya menjadi binary.

Saya menyarankan:

$$=$$

dan:

$$t =$$

Ini hybrid architecture yang lebih natural.

G.64. Maka final architecture kita

$$,,,tP$$

G.65. Tetapi ada masalah besar yang harus kita selesaikan sebelum coding

Jumlah qubit.

Jika:

$$= 100$$

dan kita menggunakan:

$$2$$

binary variables:

$$= 200$$

qubits.

QAOA simulasi statevector membutuhkan:

$$2^{200}$$

amplitudes.

Ini mustahil secara brute-force.

Maka kita harus menggunakan **candidate universe reduction**.

G.66. Candidate selection

Jangan berikan 5,000 saham kepada QAOA.

Gunakan classical pre-filter:

Misalnya:

$$5000 \quad 100 \quad 20$$

Baru:

$$20$$

masuk QUBO.

G.67. Bagaimana memilih candidate?

Misalnya:

$$= , t, tt, ttt$$

Ambil:

—

Misalnya:

$$= 12$$

Maka:

$$2 = 24$$

qubits untuk direction encoding.

Ini jauh lebih realistis untuk eksperimen awal.

G.68. Tetapi candidate pre-filter harus causal

Jangan menggunakan:

$$tt$$

untuk memilih candidate.

Harus:

$$t = t$$

Ini kembali ke prinsip Tahap F.

G.69. Alternative encoding

Kita juga bisa menggunakan satu binary variable:

$$\in \{0, 1\}$$

dan direction ditentukan oleh sign:

Misalnya:

$$0 ,$$

$$0$$

Kemudian QAOA hanya memilih:

Ini membutuhkan:

qubits, bukan:

$$2$$

Untuk versi awal saya sangat menyarankan **encoding ini sebagai baseline**.

G.70. Formulasi simplified QUBO

Kita definisikan:

$$=$$

dengan:

$$=$$

Maka:

$$\in \{-1, 0, 1\}$$

Tetapi karena sign sudah ditentukan:

$$=$$

Objective:

$$= -$$

Sekarang hanya:

qubits.

G.71. Apa kelemahannya?

Kita tidak bisa secara independen memilih:

atau:

Direction ditentukan oleh forecast.

Namun ini justru dapat menjadi **baseline QAOA pertama yang sangat bersih**.

Kemudian kita bandingkan dengan:

$$2$$

direction-aware formulation.

G.72. Saya menyarankan eksperimen bertahap

Model G1

Asset selection only.

Direction berasal dari forecast.

Model G2

Asset + direction.

2

Model G3

Asset + direction + cardinality.

Model G4

Asset + direction + factor exposure.

Model G5

Full formulation:

$t \quad t \quad t \quad t \quad tt$

Dengan demikian kita dapat melihat kontribusi setiap komponen.

G.73. Ini sangat penting untuk eksperimen ilmiah

Kalau kita langsung memasukkan:

20

constraint sekaligus dan performanya buruk, kita tidak tahu penyebabnya.

Dengan ablation study:

1 2 4 5,

kita dapat melihat:

ΔP

G.74. Contoh ablation

Misalnya hasil:

Model	Sharpe	MDD	Turnover
G1	1.12	-18%	4.2
G2	1.25	-16%	4.8
G3	1.39	-13%	3.1
G4	1.51	-11%	2.8
G5	1.57	-9%	1.9

Maka kita dapat mengatakan:

factor exposure dan transaction-cost-aware optimization memberikan kontribusi terhadap peningkatan risk-adjusted performance.

Itu jauh lebih kuat secara ilmiah.

G.75. Dan sekarang kita dapat melihat hubungan dengan skripsimu

Ada hubungan yang sangat langsung dengan apa yang telah kamu pelajari sebelumnya.

Kamu sudah bekerja dengan:

Dalam proyek ini kita memperluasnya menjadi:

kemudian:

Jadi ini bukan algoritma quantum yang ditempelkan ke trading.

Kita mulai dari **financial optimization problem**, lalu menunjukkan bahwa problem tersebut memiliki struktur combinatorial yang dapat dipetakan ke QUBO.

G.76. Bahkan ada hubungan langsung dengan Hamiltonian Ising yang sudah kamu pelajari

Kita sebelumnya menulis:

$$= - -$$

Sekarang kita dapat menginterpretasikan:

sebagai efek individual keputusan trading, sedangkan:

sebagai interaksi antar keputusan.

Interaksi tersebut dapat berasal dari:

$$\square$$

Jadi secara fisik:

$$\square$$

dengan energy landscape yang merepresentasikan objective finansial.

G.77. Tetapi jangan membuat klaim terlalu besar

Kita **tidak** boleh mengatakan:

"Karena pasar adalah sistem Ising."

Itu tidak benar.

Yang benar:

Kita memetakan **masalah optimasi diskrit yang berasal dari portfolio/trading problem** ke bentuk QUBO/Ising.

Jadi Ising adalah **representasi matematis optimization problem**, bukan klaim bahwa pasar secara literal merupakan material magnetik.

Ini penting secara akademik.

G.78. Tahap G dalam satu persamaan

Kalau seluruh pembahasan ini harus diringkas menjadi satu formula, saya akan menggunakan:

$$-t - t - P_{,t} - \frac{^2}{,t} ,t P - t-122 P - t-122 P P - , \in \{0,1\}$$

Kemudian:

$$\in \{0,1\}^{2^t}$$

kemudian:

$$= 1 - 2$$

kemudian:

$$t = t$$

Dan akhirnya:

$$t = \frac{1}{2^t}$$

Dengan:

$$, = , t$$

G.79. Namun ada satu hal yang sangat penting untuk Tahap H

Kita sekarang sudah mempunyai **mathematical formulation**.

Tetapi masih ada satu pertanyaan besar:

Bagaimana kita menentukan t , t , dan t secara benar dari data?

Karena seluruh QUBO bergantung pada:

$$t, t, t$$

Kalau estimasinya buruk, QAOA bisa memberikan **solusi optimal untuk QUBO yang salah**.

Dan ini merupakan jebakan yang sangat penting:

$$\square$$

QAOA bisa menemukan:

dengan sangat baik, tetapi jika:

$$t$$

salah, maka:

tetap bisa menjadi trade yang buruk.

G.80. Maka Tahap H seharusnya bukan langsung coding QAOA

Menurut saya, tahap berikutnya justru harus kita bangun sebagai:

Tahap H — Forecasting & State Estimation Engine

Di sana kita akan menurunkan secara matematis:

$$t$$

dari:

- historical returns,
- momentum,
- volatility,
- volume,
- BTC/ETH,
- DXY,
- gold,
- SPX/VIX,
- cross-asset correlation,
- order-book imbalance,

kemudian membangun:

$$t$$

secara **rolling / exponentially weighted / shrinkage**, dan:

$$t$$

untuk factor exposure.

Yang paling menarik, kita juga bisa membuat **regime-dependent estimator**, sehingga:

$$t, t, t = t$$

Dengan demikian QUBO kita benar-benar menjadi:

$$t = t$$

dan bukan sekadar QUBO statis yang dihitung sekali dari historical data.

Baru setelah Tahap H selesai, kita mempunyai seluruh bahan yang dibutuhkan untuk membuat implementasi QAOA yang serius: bukan hanya circuit-nya, tetapi juga **bagaimana angka-angka yang masuk ke circuit tersebut diperoleh secara causal dari pasar**.

Prompt:

8/17/2026, 9:50:26 PM

lanjut Tahap H

Response:

8/17/2026, 9:50:28 PM

Baik. **Tahap H adalah salah satu tahap paling penting dari seluruh sistem**, karena di sinilah kita menentukan *angka apa yang sebenarnya masuk ke QUBO*. QAOA hanya mengoptimalkan objective yang kita berikan. Jika estimasi expected return, covariance, dan factor exposure buruk, QAOA dapat menemukan solusi matematis yang sangat optimal tetapi secara finansial buruk.

Jadi Tahap H kita bangun sebagai:

Tahap H — Forecasting & State Estimation Engine

Arsitektur besarnya menjadi:



Saya ingin membuat tahap ini cukup matematis sehingga nantinya kita bisa langsung menerjemahkannya menjadi kode.

H.1. Masalah fundamental: apa yang sebenarnya kita prediksi?

Pada waktu:

$t,$

kita tidak mengetahui return masa depan:

$t\Delta$

Yang kita miliki hanyalah informasi:

$$t$$

yaitu seluruh informasi yang tersedia sampai waktu t .

Maka target forecasting yang benar adalah:

$$P_{t+\Delta t}$$

Perhatikan simbol:

$$t$$

Ini sangat penting.

Model tidak boleh menggunakan informasi:

$$P_{t+\Delta t}$$

ketika membuat keputusan pada:

$$t$$

Inilah prinsip **causality / no look-ahead bias**.

H.2. Return

Kita menggunakan log return:

$$r_{t,\Delta t} = \ln \frac{P_{t+\Delta t}}{P_t}$$

atau:

$$r_{t,\Delta t} = P_{t+\Delta t} - P_t$$

Untuk horizon:

$$\Delta t,$$

target dapat didefinisikan:

$$P_{t+\Delta t}$$

Misalnya:

- $\Delta t = 5$ menit,
 - $\Delta t = 1$ jam,
 - $\Delta t = 4$ jam,
 - $\Delta t = 1$ hari.
-

H.3. Kita sebaiknya mempunyai beberapa horizon

Trading jangka pendek tidak harus memiliki satu horizon.

Misalnya:

$$\Delta_1 = 15$$

$$\Delta_2 = 1$$

$$\Delta = 4$$

Maka:

$$15,t$$

$$1,t$$

$$4,t$$

Ini memungkinkan sistem mengetahui apakah signal hanya bersifat sangat pendek atau memiliki persistence.

H.4. State vector

Sekarang kita definisikan seluruh informasi pasar menjadi:

$$\boxed{t}$$

atau lebih eksplisit:

$$t = \text{ttttttttttt}$$

Kemudian:

$$\boxed{t = t}$$

Di sini:

adalah parameter model forecasting.

H.5. Jangan langsung menggunakan harga mentah

Harga:

$$P_t$$

biasanya bukan feature yang baik untuk cross-sectional model.

Lebih berguna menggunakan:

$$t,$$

volatility,

momentum,

relative strength,

volume,

factor return,

dan sebagainya.

H.6. Feature block 1 — momentum

Contoh:

$$r_{i,t} = P_{i,t} - P_{i,t-1}$$

Ini sama dengan:

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

Kita dapat memiliki:

$$r_{i,t}^{(5)}, r_{i,t}^{(20)}, r_{i,t}^{(60)}$$

Untuk data 1 jam misalnya:

- 5 jam,
- 20 jam,
- 60 jam.

H.7. Feature block 2 — volatility

Volatility historis sederhana:

$$\sigma_{t-1}^2 = 1 - \lambda - \lambda \sigma_{t-2}^2$$

Tetapi untuk trading yang lebih responsif, saya lebih menyukai EWMA.

H.8. EWMA volatility

Definisikan:

$$\sigma_{2,t}^2 = \lambda \sigma_{2,t-1}^2 + (1 - \lambda) r_{t-1}^2$$

dengan:

$$\lambda = \frac{2}{1 + \sqrt{2N}}$$

Jika:

λ besar, estimator lebih lambat.

Jika kecil, estimator lebih responsif.

H.9. Mengapa EWMA cocok untuk sistem kita?

Karena pasar bukan stationary process sempurna.

Misalnya:

$$r_{t-100} \neq r_t$$

Jika kita menggunakan rolling volatility panjang, model masih "mengingat" kondisi lama.

EWMA memberi:

H.10. Feature block 3 — volume

Volume dapat digunakan sebagai:

$$V_t$$

Tetapi volume absolut tidak comparable antar aset.

Lebih baik:

$$V_t = \frac{V_t}{V_{t-1}}$$

Jika:

$$t \geq 1,$$

volume lebih besar dari normal.

Misalnya:

$$t = 24$$

berarti volume sekitar:

$$24$$

rata-rata eksponensialnya.

H.11. Feature block 4 — volatility-adjusted momentum

Momentum mentah:

$$t$$

Tidak cukup.

Kita dapat menormalisasi:

$$\frac{t}{\sigma_t} = \frac{t}{\sigma_t}$$

Ini memberi semacam signal-to-noise ratio.

H.12. Feature block 5 — market factors

Sekarang bagian yang sangat penting dari desain kita.

Untuk saham internasional:

$$t = \frac{t}{\sigma_t} P_{t,t}$$

Untuk crypto:

$$t = \frac{t}{\sigma_t} \Delta_{t,t}$$

Jadi model tidak melihat sebuah aset secara terisolasi.

H.13. Contoh: altcoin

Misalkan:

$$r_{i,t} = \alpha + \beta_1 r_{BTC,t} + \beta_2 r_{SOL,t}$$

Jika:

$$\beta_2 = 1,$$

maka SOL sangat sensitif terhadap BTC.

Jika BTC mengalami:

$$1,$$

model secara kasar mengantisipasi kontribusi:

$$1$$

tentu dengan asumsi hubungan linear tersebut masih berlaku.

H.14. Factor model

Secara umum:

$$r_{i,t} = \alpha + \beta_1 r_{BTC,t} + \beta_2 r_{SOL,t}$$

Dengan:

$$r_{BTC,t} \in$$

$$r_{SOL,t} \in$$

$$r_{i,t} \in$$

H.15. Estimasi beta

Beta dapat diperoleh melalui rolling regression.

Untuk aset :

$$r_{i,\tau} = \alpha + \beta_1 r_{BTC,\tau} + \beta_2 r_{SOL,\tau}$$

Untuk window:

$$\tau = t - 1, \dots, t$$

Secara OLS:

$$=^{-1}$$

Namun ada masalah.

H.16. Masalah multicollinearity

Misalnya:

dan:

sangat berkorelasi.

Maka:

bisa ill-conditioned.

Solusinya:

$$=^{-1}$$

Ini adalah Ridge Regression.

H.17. Mengapa Ridge menarik untuk kita?

Karena kita tidak hanya ingin forecasting.

Kita juga ingin:

t

untuk factor exposure constraint.

Beta yang terlalu noisy akan membuat:

t

berubah-ubah secara tidak stabil.

Ridge memberikan:

H.18. Forecasting expected return

Sekarang kita dapat membuat model:

$$r_{t,t} = r_{t,t}$$

Tetapi kita bisa memperluasnya:

$$r_{t,t} = r_{t,t}, r_{t,t}, r_{t,t}, r_{t,t}$$

dengan:

$$t$$

sebagai market regime.

H.19. Saya menyarankan model forecasting tidak langsung memprediksi harga

Jangan:

$$P_{t\Delta}$$

Lebih baik:

$$t\Delta$$

Bahkan lebih baik lagi, model dapat memprediksi:

$$t\Delta, t\Delta,$$

sekaligus.

H.20. Probabilistic forecast

Misalkan:

$$r_{t\Delta},$$

Kita dapat menggunakan:

$$= P(0_t)$$

Kemudian expected return:

$$= r_t$$

Ini memberikan dua informasi:

dan:

H.21. Signal strength

Kita dapat membuat:

$$\frac{r_{i,t}}{\sigma_{i,t}} = \frac{r_{j,t}}{\sigma_{j,t}}$$

Ini secara konseptual mirip expected return per unit volatility.

Untuk LONG:

$$0$$

Untuk SHORT:

$$0$$

Semakin besar:

$$,$$

semakin menarik signal.

H.22. Tetapi jangan gunakan secara naif

Karena:

adalah forecast yang noisy.

Misalnya:

$$= 02$$

dan:

$$= 05$$

Maka:

$$= 04$$

Tetapi jika confidence model hanya:

$$20,$$

signal tersebut tidak boleh diperlakukan sama dengan signal:

$$= 04$$

dengan confidence:

$$0$$

H.23. Confidence-adjusted alpha

Kita dapat mendefinisikan:

$$\alpha_t =$$

dengan:

$$\alpha_t = \frac{1}{1 + t}$$

Contoh:

$$\alpha_t = \frac{1}{1 + t}$$

Maka QUBO menggunakan:

$$t$$

bukan:

$$t$$

mentah.

H.24. Ini sangat penting secara praktis

Bayangkan dua aset:

Asset A

$$A_t = 0$$

$$A_t = 0$$

sehingga:

$$A_t = 0.2$$

Asset B

$$B_t = 10$$

$$B_t = 0$$

Maka:

$$B_t = 0.0$$

Optimizer sekarang memilih A meskipun raw forecast B lebih besar.

H.25. Market regime

Sekarang kita masuk ke bagian yang lebih advanced.

Hubungan:

tidak selalu sama.

Begitu pula:

$$P$$

Maka kita membutuhkan:

$$t$$

Contoh:

$$t \in \{, , , \}$$

H.26. Regime sederhana

Kita dapat mendefinisikan:

$$t = t, t_t, , , t$$

Contoh:

$$t = \begin{cases} 1, & t = 0, \\ t \end{cases}$$

Ini versi paling sederhana.

H.27. Regime yang lebih formal

Kita bisa menggunakan Hidden Markov Model:

$$P_t = P_{t-1} = P$$

Dengan observation:

$$t = P_{t,t}, t_{t,t}, t_{t,t}$$

Maka:

$$P_{t-1,t}$$

memberikan probabilitas regime.

H.28. Jangan memilih regime secara hard

Daripada:

$$t = 1,$$

kita dapat mempertahankan:

$$1t' 2t' \text{ ' } t$$

Kemudian expected forecast:

$$\boxed{t = \sum_{=1} P_t = t_t}$$

Ini jauh lebih smooth.

H.29. Covariance estimation

Sekarang kita masuk ke komponen kedua yang sangat penting:

$$\boxed{t}$$

Estimator covariance sample:

$$= 1 - 1 \quad t - t -$$

Masalahnya:

Jika:

besar dibandingkan:

,

covariance matrix sangat noisy.

H.30. Shrinkage covariance

Kita gunakan:

$$\boxed{= 1 -}$$

dengan:

$$0 \quad 1$$

adalah target covariance yang lebih stabil.

Contoh:

=

H.31. Mengapa shrinkage penting untuk QAOA?

Karena covariance menjadi:

Jika covariance noise:

Akibatnya:

berubah secara tidak stabil.

QAOA kemudian mengoptimalkan landscape yang sebenarnya merupakan artefak estimation error.

Jadi:

□

H.32. Factor covariance

Kita sebelumnya mempunyai:

=

Ini sebenarnya sangat menarik.

Daripada mengestimasi:

covariance secara langsung, kita bisa mengestimasi:

dan:

Jika:

,

maka dimensionality reduction sangat besar.

Misalnya:

= 100,

tetapi:

= 5

H.33. Estimasi idiosyncratic covariance

Secara sederhana:

$$r_{i,t} = \alpha_i + \beta_i' F_t + \epsilon_{i,t}$$

Kemudian:

$$\epsilon_{i,t} = \epsilon_{i,t-1} + \eta_{i,t}$$

Dengan asumsi residual cross-correlation kecil.

Maka:

$$\text{Cov}(\epsilon_{i,t}, \epsilon_{j,t}) = 0$$

H.34. Keuntungan besar

Portfolio risk:

menjadi:

Definisikan:

$$\sigma_p^2 =$$

Maka:

$$\sigma_p^2 =$$

Interpretasinya:

$$\sigma_p^2 = \sigma_{\epsilon}^2$$

H.35. Ini sangat cocok dengan sistem kita

Kita ingin:

- alpha dari asset,
- factor exposure,
- covariance,
- risk.

Semuanya berasal dari satu model:

$$\Sigma_t = \Sigma_{t-1}$$

Ini membuat architecture jauh lebih konsisten.

H.36. Dynamic covariance

Pasar berubah.

Jadi kita gunakan:

$$\Sigma_t$$

Misalnya EWMA:

$$\Sigma_t = \lambda \Sigma_{t-1} + (1-\lambda) \Sigma_{t-1}$$

Namun untuk sistem yang lebih besar, saya lebih menyukai:

$$\Sigma_t = \lambda \Sigma_t + (1-\lambda) \Sigma_{t-1}$$

karena lebih interpretable.

H.37. Market factor covariance

Misalnya:

$$\Sigma_t = \lambda \Sigma_t + (1-\lambda) \Sigma_{t-1}$$

Ini juga dapat menggunakan EWMA/shrinkage.

H.38. Factor exposure target harus dynamic

Kita sebelumnya menulis:

Tetapi seharusnya:

$$\Sigma_{f,t}$$

karena target factor exposure bergantung pada regime.

Misalnya saat:

$$P_t$$

kita mungkin ingin:

$$P = 0$$

Sedangkan saat:

$$P = ,$$

kita dapat memberikan:

$$P = 0$$

H.39. Contoh regime-dependent factor target

Misalkan:

$$t =$$

Kita bisa menetapkan:

$$P = 0.5$$

Tetapi jika:

$$t = ,$$

maka:

$$P = -0.2$$

Sehingga:

$$= P - t^2$$

secara otomatis berubah.

H.40. DXY dan Gold

Untuk sistem global yang kamu inginkan, saya ingin membedakan tiga hal.

DXY

Proxy untuk:

$$t$$

Gold

Proxy untuk:

$$-t - t$$

Equity index

Proxy untuk:

$$t$$

Jadi state macro kita dapat berupa:

$$t = , , P ,$$

Untuk crypto ditambah:

$$, ,$$

H.41. Jangan langsung menggunakan level DXY

Kita lebih tertarik pada:

$$, t$$

Dan fitur turunannya:

$$, t'$$

$$, t''$$

$$, t'''$$

Misalnya:

$$, t = t - t, t$$

H.42. Cross-asset signal

Kita dapat membuat signal:

$$t, t = t$$

Kemudian residual:

$$, t = , t - t$$

Ini menarik.

Karena kita dapat bertanya:

| Apakah saham/crypto naik karena market factor, atau karena alpha idiosyncratic?

H.43. Idiosyncratic alpha

Misalnya:

naik:

$$2$$

Tetapi:

$$P$$

naik:

$$1$$

Maka sebagian besar gerakan mungkin berasal dari market.

Tetapi jika:

$$= 2$$

sementara:

$$P = -1,$$

maka residual return dapat sangat positif.

Ini jauh lebih menarik sebagai alpha candidate.

H.44. Residual momentum

Kita bahkan dapat mendefinisikan:

$$r_{i,t} - r_{i,t-1}$$

Ini mengukur momentum yang tidak dijelaskan oleh factor.

Dan:

dapat menjadi input forecasting.

H.45. Forecast model akhirnya

Kita dapat membangun:

$$,t = ,t,t,t,t,ttt$$

Kemudian:

$$,t = ,t$$

H.46. Model tidak harus deep learning

Ini penting.

Kita dapat memulai dengan:

$$t = t$$

Linear model.

Kemudian:

$$t = t$$

atau model lain.

Baru setelah baseline kuat, kita bisa mencoba:

???

dan sebagainya.

H.47. Mengapa baseline sederhana lebih baik?

Karena tujuan kita bukan:

"membuat model AI paling rumit."

Tujuannya:

$$=$$

Kalau model linear menghasilkan:

$$= 15$$

sedangkan neural network:

$$= 11,$$

maka model sederhana lebih baik.

H.48. Forecast harus dievaluasi out-of-sample

Jangan hanya melihat:

$$2$$

Kita membutuhkan:

$$= t, t\Delta$$

atau:

Kemudian:

$$t$$

Tetapi akhirnya yang terpenting:

$$\square$$

H.49. Economic value

Forecast yang bagus belum tentu trading strategy bagus.

Misalnya:

$$= 005$$

Terlihat kecil.

Tetapi jika:

rendah dan signal stabil, mungkin tetap profitable.

Sebaliknya:

$$= 015$$

tetapi turnover sangat tinggi, bisa menjadi:

$$tP$$

H.50. Maka forecasting objective dan trading objective berbeda

Forecasting:

$$\min_{\theta} \sum_{t=1}^T \ell(\theta; x_t, y_t)$$

Trading:

$$\max_{\theta} \sum_{t=1}^T P(\theta; x_t)$$

Ini dua optimization problem berbeda.

H.51. Ini mengarah ke konsep yang sangat penting: forecast-to-trade transformation

Kita memiliki:

$$x_t$$

Tidak langsung:

Melainkan:

$$\begin{bmatrix} t & t & t \end{bmatrix}$$

Jadi QAOA berfungsi sebagai **decision layer**.

H.52. Confidence-aware QUBO

Saya bahkan menyarankan objective menggunakan:

$$t = t \ t$$

Sehingga:

$$= -_t$$

Ini membuat optimizer secara otomatis mengurangi perhatian terhadap signal yang tidak dipercaya.

H.53. Prediction uncertainty yang lebih formal

Daripada sekadar , kita dapat menggunakan:

$$,t\Delta \ ,t, \tau_{2,t}$$

Maka signal-to-noise:

$$\frac{\sigma_t}{\tau_t} = \frac{\sigma_t}{\tau_t}$$

Kita kemudian dapat menggunakan:

$$= 1 \quad \tau$$

Semakin uncertainty besar:

$$\tau,$$

semakin kecil effective alpha.

H.54. Ini juga dapat mengendalikan position size

Setelah QAOA memilih aset, kita bisa menggunakan:

$$2$$

sebagai baseline.

Namun kita dapat memasukkan uncertainty:

$$2 \quad \tau_2$$

Jadi forecast uncertainty tidak hanya memengaruhi selection tetapi juga sizing.

H.55. State estimation pipeline

Pada setiap waktu t :

Step 1

Ambil data:

$$t$$

Step 2

Hitung:

$$t$$

Step 3

Hitung features:

$$t$$

Step 4

Estimasi regime:

$$P_{t \ t}$$

Step 5

Estimasi:

$$t$$

Step 6

Estimasi:

$$t$$

Step 7

Estimasi:

$$t$$

Step 8

Bangun:

$$t = t_t \ t$$

Step 9

Forecast:

$$t$$

Step 10

Forecast uncertainty:

$$\tau_t$$

H.56. Kemudian QUBO

Setelah itu:

$$t = t, t, t, t, t-1, t$$

Jadi QUBO bukan hard-coded.

Ia dihasilkan ulang dari market state.

H.57. Ini menghasilkan konsep yang sangat menarik

Sistem kita sebenarnya adalah:

□

karena:

$$t = t$$

Dengan demikian:

$$t_1 \quad t_2$$

ketika market state berubah.

H.58. Market berubah → Hamiltonian berubah

Misalnya:

Normal regime

$$t =$$

$$= t$$

$$P =$$

Hamiltonian:

Kemudian market crash:

$$t$$

$$P$$

Maka:

□

H.59. Ini berbeda dari QAOA portfolio optimization statis

Dalam portfolio optimization biasa:

dibuat sekali.

Kemudian:

dijalankan.

Dalam sistem kita:

$$t = t$$

dan:

$$t$$

dijalankan secara rolling.

Ini membuatnya jauh lebih dekat dengan **dynamic trading system**.

H.60. Tetapi kita tidak boleh menjalankan QAOA setiap tick

Itu tidak praktis.

Misalnya data:

$$1$$

Kita bisa melakukan:

setiap:

$$15$$

atau:

$$1$$

Di antara interval itu:

berjalan secara classical.

H.61. Arsitektur temporal

Misalnya:

$$= 1$$

sedangkan:

$$= 1$$

Jadi:

Setiap jam

$tt \quad t \quad t$

Setiap menit

$P \quad P \quad t$

H.62. Ini cocok dengan desain yang kita bahas sebelumnya

Kita memiliki:

☐

untuk QAOA.

Dan:

☐

untuk trade management.

Serta:

☐

untuk order.

H.63. Hierarchical architecture

Secara lengkap:

Dan feedback:

H.64. Satu masalah yang belum kita masukkan: order book

Untuk trading jangka pendek, OHLCV saja belum cukup.

Kita ingin:

t

(order book state).

Contoh feature:

$$t = -$$

Ini disebut order-book imbalance.

Jika:

$$t \gg 1,$$

bid jauh lebih dominan.

Jika:

$$t \ll -1,$$

ask dominan.

H.65. Microstructure feature

Kita bisa memiliki:

$$t = t_{ttt} P_t - P_t$$

Kemudian:

$$t = t, t, t$$

H.66. Ini membuat model dua skala

Slow information

$$, , P,$$

berubah relatif lambat.

Fast information

$$, ,$$

berubah cepat.

Maka state kita sebenarnya:

$$t = t, t$$

H.67. Saya bahkan menyarankan dua model forecast

Model 1 — medium horizon

$$t = , t, t$$

Model 2 — short horizon

$$tt = t, tt$$

Kemudian:

$$t = tt \cdot 1 - ttt$$

Bobot:

$$t$$

bergantung pada horizon trading dan kondisi market.

H.68. Contoh

Jika market sangat tenang:

$$t = 0$$

Medium-term information lebih dominan.

Jika terjadi sudden order-flow imbalance:

$$t = 0$$

Microstructure menjadi lebih penting.

H.69. Sekarang kita punya state estimator yang jauh lebih lengkap

$$t =$$

Kemudian:

$$t, \tau_t, t, t = t$$

dengan:

sebagai **state estimation engine**.

H.70. Dan akhirnya

$$t = t, \tau_t, t, t, t, t-1, t$$

kemudian:

$$t = t$$

dengan:

sebagai mapping QUBO $\rightarrow$ Ising.

Kemudian:

$$t = t$$

H.71. Satu hal yang menurut saya sangat penting untuk proyek kita

Jangan menganggap:

sebagai sumber alpha.

Alpha berasal dari:

□

QAOA berperan sebagai:

□

Dengan kata lain:

□

bukan:

Ini akan menjadi prinsip desain utama sistem kita.

H.72. Pipeline Tahap A–H sekarang

Kita akhirnya memiliki:

$$\boxed{\begin{aligned} A &: \text{Market \& trading problem formulation} \\ B &: \text{Market state / info} \end{aligned}}$$

Dan alur aktualnya:

$$\begin{matrix} t & t & \{t, t, t, \tau_t, t\} & t & t & t & t & P_{tt} \end{matrix}$$

H.73. Tahap berikutnya: Tahap I

Setelah Tahap H, menurut saya **kita belum boleh langsung coding QAOA**.

Kita perlu terlebih dahulu membangun:

Tahap I — Backtesting, Walk-Forward Validation, dan Statistical Evaluation

Ini sangat penting karena kita sekarang mempunyai sistem yang sangat mudah terkena:

- look-ahead bias,
- survivorship bias,
- data snooping,
- overfitting,
- regime overfitting,
- transaction-cost underestimation,
- slippage,
- QAOA parameter overfitting,

- optimizer overfitting.

Kita perlu membangun matematika untuk:

$$\square$$

serta menguji bukan hanya:

$$P,$$

tetapi:

$$,t, , , , , , tt, Ptt, , ,$$

dan yang menurut saya sangat menarik untuk proyek kita:

$$\square$$

pada **QUBO yang persis sama**.

Itulah yang nantinya memungkinkan kita menjawab pertanyaan ilmiah yang jauh lebih menarik daripada sekadar "*apakah bot ini menghasilkan uang?*":

Apakah QAOA memberikan kualitas solusi atau efisiensi optimasi yang berbeda dibandingkan classical solver pada dynamic short-horizon portfolio/trading problem yang sama, setelah mempertimbangkan estimation error dan biaya transaksi?

Itu sudah mulai menjadi **research-grade quantitative trading + quantum optimization framework**, bukan sekadar bot trading.

Prompt:

8/17/2026, 9:51:37 PM

lanjut Tahap I

Response:

8/17/2026, 9:51:38 PM

Baik. Kita masuk ke **Tahap I — Backtesting, Walk-Forward Validation, dan Statistical Evaluation**.

Menurut saya, ini justru tahap yang akan menentukan apakah seluruh arsitektur A–H kita benar-benar ilmiah atau hanya terlihat bagus di atas kertas. Kita sudah membangun forecasting, factor model, covariance, regime, QUBO, QAOA, execution, dan position management. Sekarang semuanya harus diuji **tanpa memberikan sistem informasi yang pada saat itu belum tersedia**.

Saya akan membangun Tahap I dari prinsip paling dasar sampai akhirnya menjadi protokol eksperimen yang bisa langsung kita implementasikan.

Tahap I — Backtesting dan Statistical Validation

Tujuan Tahap I:

$$=$$

Tetapi ada satu prinsip yang harus menjadi fondasi:

$$t, t$$

Tidak lebih.

I.1. Mengapa backtesting biasa berbahaya?

Bayangkan kita mempunyai data:

$$P_1, P_2, \dots, P$$

Kemudian kita melatih model menggunakan seluruh data:

$$P_1$$

Setelah itu kita menguji strategy pada data yang sama.

Misalnya diperoleh:

$$=$$

Ini hampir tidak berarti.

Sebab model sudah melihat masa depan.

Secara matematis kita ingin:

$$t = t$$

bukan:

$$=$$

I.2. Causal information set

Pada waktu:

$$t,$$

definisikan:

$$t = 1, 2, \dots, t$$

Artinya semua keputusan harus merupakan fungsi:

$$t = t$$

Bukan:

$$t = t_1$$

Ini kelihatannya sederhana, tetapi hampir seluruh sumber **look-ahead bias** berasal dari pelanggaran konsep ini.

I.3. Contoh look-ahead bias

Misalkan kita menggunakan:

$$t = P_t P_{t-1}$$

Jika pada akhir candle t kita membuat keputusan untuk:

$$t \geq 1,$$

maka menggunakan:

$$t$$

masih sah.

Tetapi jika kita menggunakan:

$$t_1,$$

maka kita sudah menggunakan informasi masa depan.

I.4. Bahkan preprocessing bisa menyebabkan leakage

Misalnya kita melakukan standardization:

$$t = t -$$

Jika:

$$= \frac{1}{\sum_{t=1}^T t},$$

maka menggunakan data masa depan.

Seharusnya:

$$t = \frac{1}{\sum_{\tau=1}^T t_{\tau}}$$

Demikian juga:

I.5. Jadi seluruh pipeline harus causal

Bukan hanya model forecasting.

Harus:

$$t_t = tt = {}_t t_t = t_{t, tt} = tt = {}_{t, t} t = tt = t$$

I.6. Train/validation/test

Kita tidak menggunakan random train-test split.

Untuk time series:

□

Misalnya data:

2021 – 2025

Kita bisa membaginya:

2021 – 202 _t 2024 _t 2025

Tetapi untuk sistem kita saya ingin sesuatu yang lebih kuat:

□

I.7. Walk-forward validation

Misalkan:

0

adalah awal training.

Kita mempunyai:

₁ = t_1, t_2

dan:

$t_1 = t, t_4$

Kemudian:

₂ = t_1, t_4

dan:

$$t_2 = t_5, t$$

Begitu seterusnya.

Secara umum:

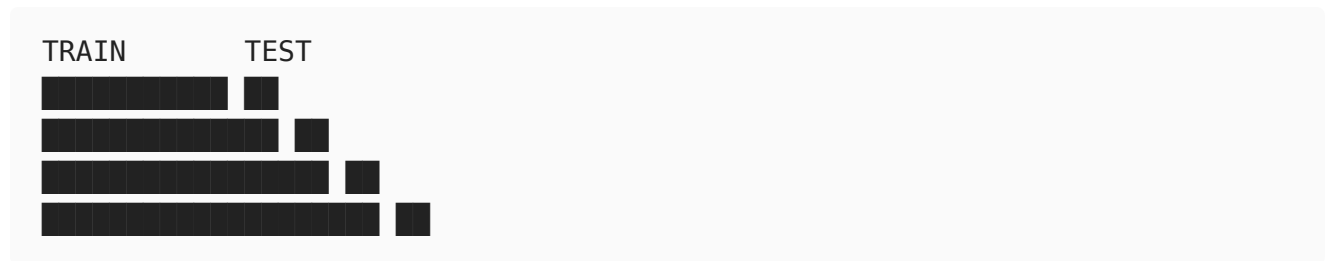
$$=$$

kemudian:

$$P = t, 1_1$$

I.8. Expanding window

Contohnya:



Training semakin panjang.

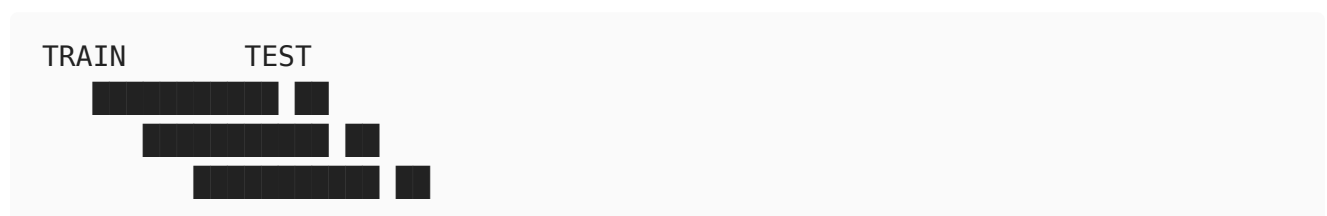
Secara matematis:

$$= t_0,$$

Ini cocok jika informasi historis lama masih relevan.

I.9. Rolling window

Alternatif:



Training window tetap:

Sehingga:

= - ,

Ini lebih cocok jika market mengalami structural change.

I.10. Untuk sistem kita: hybrid

Saya menyarankan:



Misalnya:

- covariance: rolling 60–120 hari,
- beta: rolling 60–120 hari,
- regime: rolling,
- forecasting model: retrain berkala,
- hyperparameter validation: expanding walk-forward.

Ini lebih masuk akal secara finansial.

I.11. Nested walk-forward

Kalau kita ingin benar-benar research-grade, bahkan pemilihan hyperparameter tidak boleh menggunakan test period.

Strukturnya:



dan di dalam training period:



Jadi:

OUTER TRAIN

OUTER TEST

| INNER TRAIN | INNER VAL |

| untouched |

Ini mencegah kita memilih model berdasarkan test performance.

I.12. Contoh konkret

Misalnya:

2021 – 01 – 01 2025 – 12 – 1

Outer test:

2025

Training:

2021 – 2024

Di dalam 2021–2024:

- 2021–2023 → inner train,
- 2024 → inner validation.

Pilih:

berdasarkan validation.

Kemudian **bekukan model**.

Baru jalankan:

2025

sebagai out-of-sample test.

I.13. Tetapi ada masalah yang lebih besar

Kita bukan hanya mempunyai satu model.

Kita mungkin mencoba:

- 5 feature sets,
- 4 forecasting models,
- 3 covariance estimators,
- 5 QAOA depths,
- 4 risk coefficients,
- 3 transaction cost assumptions.

Jumlah kombinasi:

$$5^4 \cdot 4^4 = 00$$

Kalau kita memilih yang menghasilkan Sharpe tertinggi, kita telah melakukan:

□

I.14. Backtest overfitting

Misalkan semua strategi sebenarnya tidak punya alpha:

$$= 0$$

Tetapi kita mencoba 1.000 strategi.

Kemungkinan besar ada beberapa yang memperoleh:

$$2$$

Bukan karena benar-benar bagus.

Tetapi karena:

$$\square$$

I.15. Inilah mengapa kita perlu experiment registry

Setiap eksperimen harus memiliki:

$$t$$

dan mencatat:

$$tttPtPttt$$

sehingga kita tahu berapa banyak eksperimen yang sebenarnya sudah kita lakukan.

I.16. Kita harus membedakan tiga jenis hasil

In-sample

Out-of-sample

Paper/live

Targetnya:

$$\boxed{P \ P \ P}$$

Tidak harus sama, tetapi divergence harus masuk akal.

I.17. Return portfolio

Misalkan keputusan portfolio:

$$t$$

Return:

$$_{t,t1} = _{t1}$$

Tetapi ini belum memasukkan transaction cost.

I.18. Transaction cost

Misalkan:

$$\Delta_t = t - t_{-1}$$

Cost sederhana:

$$t = \Delta_t$$

Maka:

$$_{t,t1} = _{t1} - \Delta_t$$

Ini minimal harus ada.

I.19. Spread dan slippage

Untuk trading jangka pendek, kita sebaiknya menggunakan:

$$t = t \ t \ t$$

Misalnya:

$$t = _t \Delta_t$$

Dengan:

$$t =$$

I.20. Market impact

Jika order besar:

$$t$$

dibandingkan liquidity:

$$t,$$

cost meningkat.

Model sederhana:

$$t_t = t$$

Misalnya:

$$12$$

Sehingga:

$$t$$

Ini jauh lebih realistis daripada fixed transaction cost.

I.21. Net PnL

Maka:

$$P_t = {}_{t-1}P_t - t - t_t$$

Semua metrik performance harus menggunakan:

$$P_t$$

Bukan gross PnL.

I.22. Equity curve

Mulai dari:

$$0$$

Kemudian:

$$t = {}_{t-1}1_{t,t}$$

Atau log wealth:

$$t = {}_{t-1}1_{t,t}$$

I.23. Cumulative return

$$= 0 - 1$$

Tetapi ini **tidak cukup**.

Strategi:

$$100$$

dengan:

$$= -0$$

sangat berbeda dari:

$$0$$

dengan:

$$= -10$$

I.24. Maximum Drawdown

Peak:

$$t = \tau t^\tau$$

Drawdown:

$$t = t - tt$$

Maximum drawdown:

$$= t^t$$

I.25. Sharpe ratio

Jika:

$$= 1_t t,$$

dan:

$$= t_t,$$

maka:

$$= -$$

adalah annualization factor.

Misalnya hourly:

$$\frac{252}{24}$$

untuk pasar yang berjalan 24/7, meskipun annualization harus disesuaikan dengan definisi return dan jam trading yang digunakan.

I.26. Tetapi Sharpe memiliki kelemahan

Sharpe menganggap:

sebagai risk measure yang cukup.

Padahal downside:

$$t = 0$$

lebih penting bagi trader.

Maka:

$$t = -$$

I.27. Calmar ratio

Untuk sistem trading yang mengalami drawdown:

$$=$$

Ini sangat berguna untuk membandingkan strategy dengan risk profile berbeda.

I.28. Tail risk

Kita juga perlu:

Misalnya:

$$5$$

Tetapi saya lebih tertarik pada:

$$\square$$

Expected Shortfall:

$$= -$$

Ini menjawab:

Jika kita masuk ke 5% kondisi terburuk, rata-rata kerugiannya berapa?

I.29. Trading-specific metrics

Kita juga perlu:

$$tt = ttt$$

Tetapi hit rate bisa menipu.

Misalnya:

$$tt = 0$$

namun satu loss besar menghapus seluruh profit.

Maka kita membutuhkan:

$$P_{tt} = P_t$$

I.30. Expectancy

Untuk setiap trade:

$$=$$

$$=$$

$$= P$$

Expected value:

$$= -1 -$$

Strategy profitable jika:

$$0$$

I.31. Tetapi untuk sistem kita, expectancy harus conditional

Kita memiliki signal strength:

$$t$$

Maka kita bisa menghitung:

$$P_t \in$$

Misalnya:

Signal	Expected PnL
weak	-0.01%
medium	+0.03%
strong	+0.12%
very strong	+0.18%

Jika hubungan seperti ini konsisten OOS, signal kita lebih meyakinkan.

I.32. Information Coefficient

Untuk cross-sectional trading, kita dapat menghitung:

$$t = t, t\Delta$$

Kemudian:

$$= 1_t^t$$

Ini mengukur apakah model dapat melakukan ranking asset.

I.33. Rank IC

Karena portfolio optimization sering hanya membutuhkan ranking:

$$t = t, t\Delta$$

Ini sering lebih robust terhadap outlier.

I.34. Forecast quality $\neq$ portfolio quality

Kita harus melakukan dua evaluasi terpisah.

Forecast layer

Portfolio layer

$$, P_t P$$

Ini sangat penting ketika nanti membandingkan QAOA.

I.35. Eksperimen utama kita

Kita ingin membandingkan:

□

tetapi **dengan problem yang identik**.

Misalnya:

t

Jangan membandingkan QAOA dengan classical optimizer pada objective berbeda.

I.36. Classical baseline

Kita bisa menggunakan:

Exact brute force

untuk:

20

misalnya.

Karena jumlah konfigurasi:

2

Ini memungkinkan kita mengetahui:

t

I.37. Mengukur approximation ratio

Misalkan objective minimization:

QAOA menghasilkan:

Exact:

Maka:

$$\frac{1}{n}$$

Semakin kecil:

$$0,$$

semakin dekat dengan optimum.

I.38. Probability of finding optimum

QAOA tidak selalu menghasilkan optimum.

Misalnya:

$$P = 0$$

Artinya jika kita melakukan sampling:

probabilitas menghasilkan konfigurasi optimal.

Kita dapat mencatat:

$$P_t$$

I.39. Expected objective dari QAOA

Jika output distribution:

$$P_t$$

maka:

$$\sum_i P_i E_i$$

Ini bahkan lebih informatif daripada hanya best sample.

I.40. Quantum advantage belum tentu PnL advantage

Ini sangat penting untuk skripsi/proyek kita.

Misalnya:

Tetapi setelah trading:

$$P \rightarrow P$$

Mengapa?

Karena:

dan:

mungkin mendominasi optimization quality.

I.41. Jadi kita harus punya tiga level benchmark

Level 1 — Mathematical

t

Mengukur:

t

Level 2 — Computational

t

Mengukur:

- runtime,
 - iterations,
 - objective,
 - scalability.
-

Level 3 — Financial

$PtPt$

Mengukur:

- PnL,
- Sharpe,
- MDD,
- turnover,
- CVaR,
- transaction cost.

I.42. Ini menghasilkan eksperimen yang sangat kuat

Kita bisa mempunyai:

$$t \quad | \quad Pt \quad | \quad t \quad | \quad PP \quad | \quad t \quad | \quad | \quad |$$

Kemudian:

$$\square$$

Barulah perbandingan adil.

I.43. Kita juga perlu baseline trading

Bukan hanya classical optimizer.

Bandingkan terhadap:

Buy & Hold

Equal Weight

$$= 1$$

Momentum

Mean-Variance

$$-$$

Classical combinatorial optimizer

QAOA

I.44. Dengan begitu kita dapat menjawab pertanyaan yang jauh lebih menarik

Bukan:

| "Apakah QAOA menghasilkan profit?"

Tetapi:

Apakah QAOA menghasilkan solusi yang lebih baik daripada classical optimization untuk masalah combinatorial portfolio/trading yang sama?

Dan:

Apakah improvement tersebut cukup besar untuk bertahan setelah estimation error, transaction cost, slippage, dan regime changes?

I.45. Stress testing

Backtest normal belum cukup.

Kita perlu stress:

□

Misalnya:

Bull

$t \rightarrow 0$,

Bear

$t \rightarrow 0$

Crisis

Sideways

0

Liquidity shock

, t

I.46. Factor stress

Karena sistem kita menggunakan:

$\mathbf{P}$,

kita dapat melakukan scenario:

2

atau:

$P -$

Kemudian hitung:

$$\Delta P_t$$

I.47. Stress test portfolio

Misalnya portfolio memiliki factor exposure:

$$=$$

Shock:

$$\Delta$$

Approximate PnL:

ΔP Δ

Ini sangat powerful.

Kita dapat mengetahui:

"Jika USD tiba-tiba menguat 2σ , apa yang terjadi pada portfolio?"

I.48. Scenario-based stress

Secara umum:

$$^t = {}_t \Delta$$

Kemudian:

$$^t = {}_t$$

dan:

$$^t = {}_t$$

Lalu kita dapat membangun:

t

Bahkan:

t

I.49. Ini berarti QAOA dapat menjadi regime-adaptive

Normal:

t

Stress:

tt

Crisis:

t

Dengan demikian sistem tidak sekadar:

Tetapi:

□

I.50. Monte Carlo

Kita juga perlu menguji apakah performance hanya terjadi karena satu trajectory historis.

Misalkan residual:

t

Kita dapat menghasilkan synthetic paths:

t

untuk:

$= 1, ,$

Kemudian menjalankan strategy pada:

simulasi.

I.51. Bootstrap

Kita dapat menggunakan block bootstrap karena return time-series tidak IID sempurna.

Ambil block:

$t, , t$

kemudian resample block.

Ini mempertahankan sebagian temporal dependence.

I.52. Distribusi Sharpe

Misalnya dari:

$$= 1000$$

simulasi kita memperoleh:

$$1, 2, 3, 4, 5$$

Kemudian kita bisa melihat:

$$P(0)$$

Bahkan:

$$5, 50, 5$$

quantile.

I.53. Probabilistic performance

Daripada mengatakan:

| Sharpe = 1.72

kita bisa mengatakan:

$$\boxed{= 12}$$

dengan:

$$5 = 0,241$$

Ini jauh lebih ilmiah.

I.54. Deflated Sharpe Ratio

Karena kita melakukan banyak eksperimen:

harus dikoreksi terhadap multiple testing.

Konsep yang relevan adalah:

$$\boxed{tt}$$

Intinya:

| semakin banyak strategi/model yang kita coba, semakin tinggi Sharpe yang dapat muncul secara kebetulan.

I.55. Probability of Backtest Overfitting

Kita juga dapat menggunakan pendekatan:

$$P$$

untuk mengukur probabilitas strategy yang terlihat terbaik di-sample akan gagal out-of-sample.

Ini akan sangat berguna ketika kita mulai bereksperimen dengan:

- feature,
- QUBO coefficient,
- QAOA depth,
- risk penalty,
- stop loss,
- take profit.

I.56. Parameter QAOA juga bisa overfit

Misalnya kita memilih:

$$= 1, 2, \dots, 10$$

dan:

,

berdasarkan profit OOS.

Hati-hati.

Kita telah melakukan hyperparameter optimization.

Jadi:

harus dipilih berdasarkan training/validation.

Test tidak boleh disentuh.

I.57. Quantum hyperparameter validation

Misalnya:

$$\in \{1, 2, \}$$

dan optimizer:

$$\{P, \}$$

Kita pilih berdasarkan:

$$tt$$

Kemudian:

$$t$$

hanya digunakan sekali.

I.58. Tetapi ada pertanyaan yang lebih penting

Apakah kita benar-benar membutuhkan QAOA?

Kita harus memberikan kesempatan kepada classical method untuk menang.

Ini penting secara ilmiah.

Jika classical optimizer ternyata:

$$10$$

lebih cepat dan menghasilkan:

$$tt,$$

maka itu adalah hasil yang valid.

Kita tidak boleh memaksakan:

I.59. Justru hasil negatif dapat menjadi penelitian yang bagus

Misalnya hasil:

$$t = t \ 05$$

Tetapi:

$$t \ t$$

Maka kesimpulan:

Pada ukuran masalah dan simulator/hardware yang digunakan, QAOA belum memberikan computational advantage.

Itu jauh lebih kredibel daripada memaksakan klaim quantum advantage.

I.60. Eksperimen ablation

Kita juga perlu menghapus komponen satu per satu.

Misalnya:

tanpa factor.

dengan factor.

dengan factor + regime.

dengan factor + regime + QAOA.

Kemudian:

P P P P

I.61. Contoh ablation table

Model	Factor	Regime	QAOA	Sharpe
Baseline	X	X	X	0.72
Factor	✓	X	X	0.91
Factor + Regime	✓	✓	X	1.08
Factor + Regime + Classical	✓	✓	Classical	1.21
Factor + Regime + QAOA	✓	✓	QAOA	1.24

Jika hasil seperti ini terjadi secara OOS, kita memiliki cerita penelitian yang sangat kuat.

I.62. Kita juga perlu turnover analysis

Turnover:

$$t = 12, t - , t - 1$$

Kenapa faktor:

12

Karena setiap pembelian biasanya mempunyai sisi penjualan counterpart dalam rebalancing portfolio.

I.63. Alpha vs turnover

Kita ingin mengetahui:

P

terhadap:

Misalnya:

$$P = 10$$

tetapi:

$$= 12$$

Strategy sebenarnya tidak viable.

Maka kita membutuhkan:

$$t = - tt$$

I.64. Capacity

Untuk hedge fund/prop firm, profit tidak cukup.

Kita juga bertanya:

□

Jika:

naik, market impact:

t

Maka:

P_t

akhirnya menurun.

I.65. Capacity curve

Kita dapat menghasilkan:

$$t$$

Misalnya:

$$, 10 - 21100 | 11 | 1410 | 0$$

Ini jauh lebih dekat dengan quantitative trading profesional.

I.66. Execution realism

Untuk setiap signal:

$$t$$

Kita tidak boleh mengasumsikan:

$$t = 100$$

Harus ada:

$$P_t, t$$

Misalnya limit order:

$$P = t, , tt$$

I.67. Limit order

Jika kita memasang:

$$P_t = P -$$

untuk long.

Maka fill hanya terjadi jika market menyentuh atau menembus level tersebut **dan** order kita mendapatkan execution.

Jadi backtest idealnya memiliki:

$$\square$$

I.68. Stop-loss juga harus realistis

Misalnya stop:

$$P$$

Jika market gap:

$$P_t \quad P,$$

kita tidak boleh menganggap fill tepat:

$$P$$

Harus menggunakan:

$$P$$

yang mungkin lebih buruk.

I.69. Ini sangat penting untuk crypto

Karena:

- volatility tinggi,
- spread berubah,
- liquidity berubah,
- liquidation cascade,
- market order impact.

Jadi:

bukan guaranteed price.

I.70. Evaluation hierarchy

Saya ingin sistem kita melewati:

$$\boxed{0 \quad tt}$$

kemudian:

$$\boxed{1 \quad ttt}$$

kemudian:

$$\boxed{2 \quad ttt}$$

kemudian:

$$-$$

kemudian:

$$4 \quad tt$$

kemudian:

$$5 \quad P$$

kemudian:

$$t$$

I.71. Unit test matematis

Contohnya kita harus memastikan:

$$=$$

dan:

$$0$$

Jika tidak:

bukan positive semidefinite.

Kemudian:

harus memiliki bentuk yang benar.

I.72. Test QUBO

Untuk:

kecil, lakukan brute force:

$$\in \{0, 1\}$$

Hitung:

untuk semua:

kombinasi.

Kemudian:

=

Bandingkan dengan QAOA.

I.73. Test portfolio

Pastikan:

= 1

jika fully invested.

Pastikan:

0

untuk long-only.

Jika cardinality:

Harus selalu dipenuhi.

I.74. Test causality

Ini salah satu unit test paling penting.

Ambil data sampai:

t

Jalankan model:

Pt_t

Kemudian tambahkan data:

$t + 1$

Prediction pada waktu t **tidak boleh berubah**.

Jika berubah:

□

I.75. Ini dapat kita jadikan automated test

Secara konsep:

$$t = t_1 t_t$$

harus menghasilkan:

$$t$$

Ini salah satu test yang sangat berharga untuk pipeline kita.

I.76. Test survivorship bias

Universe asset juga tidak boleh hanya berisi aset yang masih hidup sekarang.

Misalnya:

2021

ada 100 saham.

Pada:

202

hanya 70 yang masih ada.

Kalau backtest 2021 menggunakan hanya 70 saham yang masih hidup, kita sudah menggunakan future information.

I.77. Untuk crypto

Masalahnya sedikit berbeda karena banyak token:

- listing,
- delisting,
- liquidity collapse.

Universe harus berdasarkan:

t

bukan:

202

I.78. Corporate action

Untuk saham internasional kita harus memperhatikan:

- stock split,
- dividend,
- rights issue,
- merger,
- delisting.

Return yang digunakan harus konsisten.

I.79. Currency normalization

Ini penting karena kita ingin:

Portfolio base currency misalnya:

Jika asset dalam:

$$, P, P,$$

return harus memperhitungkan FX.

Jika:

$$P_t$$

dan:

$$,$$

maka USD price:

$$P = P$$

I.80. Ini berarti FX juga menjadi factor

Untuk international equities:

$$=$$

Dengan log return:

$$\boxed{=}$$

Ini salah satu alasan mengapa:

menjadi market factor penting.

I.81. Financial objective akhir

Kita akhirnya dapat mendefinisikan:

$$= t - t - -$$

dan QUBO kita merupakan pendekatan diskrit terhadap objective ini.

I.82. Tetapi evaluasi juga menggunakan objective tersebut

Jangan hanya:

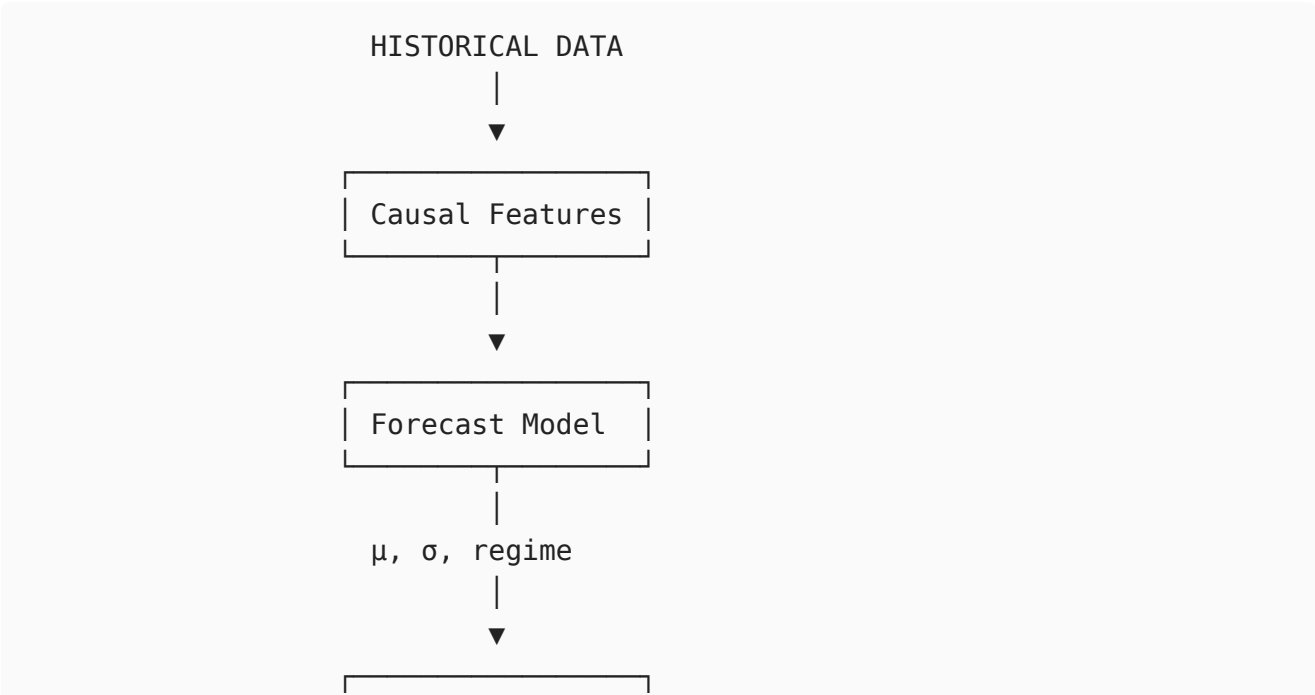
$$P$$

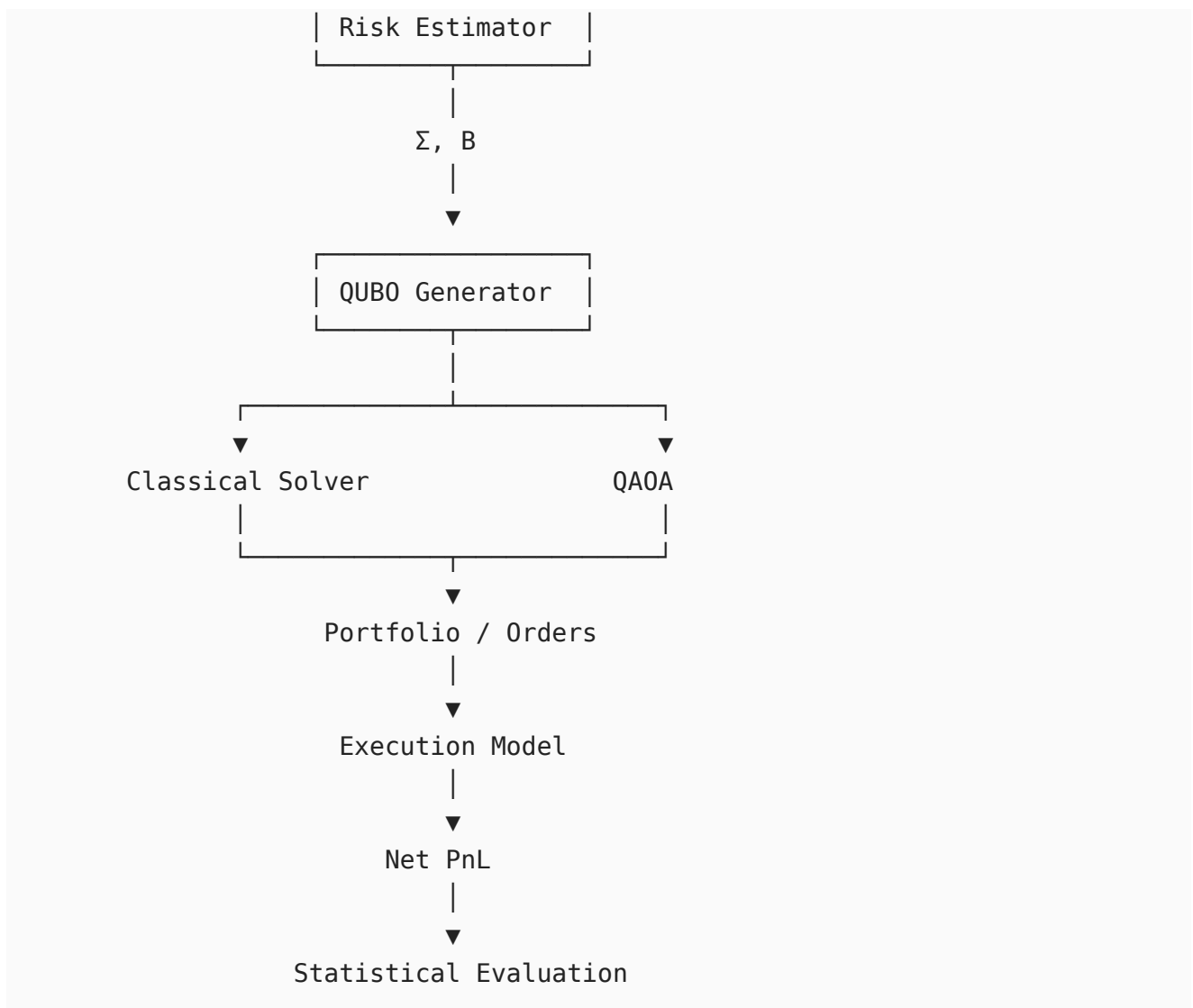
Kita evaluasi:

$$ttttt$$

I.83. Framework eksperimen final

Saya menyarankan eksperimen utama kita nantinya seperti ini:





I.84. Dan eksperimen kita harus memiliki "frozen test set"

Ini saya anggap **aturan keras**.

Misalnya:

2021 – 2024

digunakan untuk seluruh pengembangan.

Kemudian:

2025

adalah:

□

Kita tidak melihat performance 2025 untuk memilih:

- feature,

- model,
- QAOA depth,
- ,
- stop loss,
- take profit,
- rebalance interval.

Setelah semua desain selesai:

2025

I.85. Bahkan lebih baik: temporal holdout

Kalau data memungkinkan:

$$\begin{array}{c}
 2021 - 202 \quad t \\
 2024 \quad t \\
 2025 \quad tt \\
 202
 \end{array}$$

Dengan tanggal sekarang **17 Agustus 2026**, data sampai periode sekarang nantinya dapat diperlakukan sebagai forward period setelah seluruh parameter dikunci.

I.86. Apa yang sebenarnya kita ingin buktikan?

Setelah Tahap I, penelitian kita seharusnya bisa menjawab empat pertanyaan:

Pertanyaan 1

Apakah forecasting model memiliki predictive power?

0

Pertanyaan 2

Apakah portfolio optimization meningkatkan risk-adjusted return?

t

Pertanyaan 3

Apakah QAOA mampu menemukan solusi QUBO yang kompetitif?

t

Pertanyaan 4

Apakah keunggulan tersebut bertahan setelah:

$t \quad tt \quad ttt$

Pertanyaan keempat adalah yang paling penting.

I.87. Dan ada satu eksperimen yang menurut saya wajib kita lakukan

Kita buat **empat strategy**:

1 =

2 =

=

4 =

Kemudian:

1 2

mengukur manfaat optimization.

2

mengukur manfaat QAOA.

4

mengukur manfaat dynamic trade management.

Ini merupakan **ablation study** yang sangat bersih.

I.88. Bahkan kita dapat melakukan "QAOA replacement test"

Ambil QUBO:

t

Jangan ubah apa pun.

Jalankan:

$$t$$

dan:

$$t$$

Kemudian:

$$P$$

vs.

$$P$$

Dengan demikian semua perbedaan berasal dari **optimizer**, bukan dari data atau forecasting.

Ini adalah eksperimen yang menurut saya paling penting jika tujuan akhirnya adalah penelitian quantum optimization.

I.89. Struktur final penelitian kita

Dengan Tahap I, keseluruhan framework A–I sekarang menjadi:

Dan semuanya terhubung dalam satu persamaan konseptual:

$t \quad t \quad t, t, t, \tau_{t, t} \quad t \quad t \quad t \quad P_{t1}$

Kemudian:

$$P_{t1 \quad t1},$$

sehingga sistem menjadi **closed-loop dynamic system**.

Hal yang menurut saya paling penting setelah Tahap I

Kita **sudah tidak lagi sekadar mendesain strategi trading**. Kita sekarang memiliki kerangka eksperimen yang memungkinkan kita memisahkan tiga sumber keberhasilan:

dari:

dari:

Ini sangat penting untuk proyekmu. Kalau nanti QAOA memberikan hasil lebih baik, kita dapat mengatakan *di mana* peningkatannya terjadi. Kalau tidak lebih baik, kita juga dapat menunjukkan apakah masalahnya berasal dari QAOA, forecasting, covariance estimation, atau execution.

Dan menurut saya, **setelah Tahap I, tahap berikutnya sebaiknya bukan langsung "Tahap J = coding"**. Kita perlu terlebih dahulu membuat **Tahap J — Formal Mathematical Specification**, yaitu menyatukan A–I menjadi satu sistem matematis lengkap: mendefinisikan semua variabel, indeks waktu, asset universe, state vector, forecast, covariance, factor exposure, objective, constraint, QUBO, Ising Hamiltonian, QAOA circuit, position sizing, TP/SL, dan transition equation secara konsisten. Setelah itu barulah kita punya *blueprint* yang cukup ketat untuk diterjemahkan ke kode tanpa ada bagian algoritma yang ambigu.

Prompt:

8/17/2026, 9:53:11 PM

lanjut Tahap J

Response:

8/17/2026, 9:53:12 PM

Tentu. **Tahap J adalah tahap yang sangat penting**, karena di sini seluruh gagasan A–I kita satukan menjadi **satu spesifikasi matematis formal**. Setelah Tahap J selesai, kita seharusnya dapat mengambil dokumen ini dan menerjemahkannya hampir satu-ke-satu menjadi software.

Saya akan membuatnya cukup formal, tetapi tetap menjaga interpretasi finansialnya.

Tahap J — Formal Mathematical Specification

J.0. Tujuan Tahap J

Kita ingin mendefinisikan sistem:

$\mathcal{S}$

sebagai sebuah **dynamic stochastic control system**.

Secara abstrak:

$(\mathcal{S}, \mathcal{A})$

dengan:

- : state space,
- : action space,

- : transition dynamics,
- : reward/PnL function.

Namun kita **tidak perlu langsung memaksakan sistem menjadi reinforcement learning**. Kita dapat menggunakan formulasi ini sebagai kerangka matematis, sementara policy kita dibangun melalui forecasting + optimization.

J.1. Notasi utama

Kita definisikan:

$$t = 1, 2, \dots$$

sebagai indeks waktu.

Jumlah aset:

Harga asset :

$$P_{i,t}$$

Vektor harga:

$$\mathbf{P}_t = [P_{1,t}, P_{2,t}, \dots, P_{N,t}]$$

J.2. Return

Kita menggunakan log-return:

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}$$

Vektor return:

$$\mathbf{r}_t = [r_{1,t}, r_{2,t}, \dots, r_{N,t}]$$

Dengan demikian:

$$\mathbf{r}_t \in \mathbb{R}^N$$

J.3. Information set

Semua keputusan pada waktu t harus berdasarkan:

$$\mathcal{I}_t = \{\mathbf{P}_1, \mathbf{r}_1, \mathbf{P}_2, \mathbf{r}_2, \dots, \mathbf{P}_t, \mathbf{r}_t\}$$

dengan:

- t : market factors,
- t : order-book/execution information.

Keputusan:

$t = t$

Ini menjadi aturan kausal utama seluruh sistem.

J.4. Market factor vector

Kita tidak ingin model hanya melihat harga asset yang ditradingkan.

Kita definisikan:

$$t = ttttPt$$

Komposisinya dapat berubah berdasarkan asset class.

Misalnya untuk crypto:

$$tt = t, t, t, t, Pt$$

Untuk saham:

$$tt = Pt, t, t, t$$

J.5. Factor model

Kita menggunakan:

$$t \equiv tt \quad t$$

dengan:

$t \in$

Elemen:

$r_{i,t}$

adalah exposure asset terhadap factor .

J.6. Estimasi beta

Secara sederhana:

$$r_{i,t} = \alpha_i + \beta_i r_{m,t}$$

Tetapi kita ingin multivariate regression:

$$r_{i,t} = \alpha_i + \beta_{i1} r_{m1,t-1} + \beta_{i2} r_{m2,t-1}$$

adalah rolling estimation window.

J.7. Residual return

Setelah memperoleh $r_{i,t}$:

$$e_{i,t} = r_{i,t} - \hat{r}_{i,t}$$

Residual ini mencoba menangkap:

J.8. Expected return

Model forecasting kita menghasilkan:

$$\hat{r}_{i,t} = \alpha_i + \beta_{i1} r_{m1,t-1} + \beta_{i2} r_{m2,t-1}$$

Tetapi kita dapat memecahnya:

$$\hat{r}_{i,t} = \alpha_i + \beta_{i1} r_{m1,t-1} + \beta_{i2} r_{m2,t-1}$$

Ini sangat penting.

Ada dua sumber alpha:

$$\alpha_i$$

J.9. Forecast horizon

Kita harus eksplisit mengenai horizon.

Misalnya:

$$= 1$$

Maka:

$$1,t = ,t1 \ t$$

Untuk horizon lebih panjang:

$$= 4, 24,$$

model harus memiliki target yang berbeda.

J.10. Multi-horizon forecasting

Kita bahkan dapat mendefinisikan:

$$_{,t} = ,t \ t$$

Kemudian:

$$\in \{15, 1, 4, 24\}$$

Ini sangat berguna untuk menentukan apakah posisi:

- masuk,
- dipertahankan,
- ditutup,
- atau dire-optimasi.

J.11. Covariance

Kita memerlukan:

$$_t = t1 \ t$$

Untuk factor model:

$$_t = tt \ t$$

dengan:

$$_t = t$$

dan:

$$_t = t$$

J.12. Mengapa factor covariance penting?

Jika:

dan:

bergerak bersama, covariance mereka tidak boleh dianggap independen.

Demikian pula:

dan:

memiliki hubungan makro yang dapat mempengaruhi portfolio.

Dengan factor model:

mempunyai struktur yang lebih interpretable.

J.13. Portfolio weights

Definisikan:

$$w_t = (w_{1,t}, \dots, w_{N,t})$$

Jika long-only fully invested:

$$w_t \geq 0$$

dan:

$$w_t = 1$$

Tetapi untuk sistem trading kita, saya menyarankan **memisahkan asset selection dan position sizing**.

J.14. Asset selection

Definisikan:

$$x_{i,t} \in \{0, 1\}$$

Dengan:

$$x_{i,t} = 1$$

jika asset dipilih.

Cardinality constraint:

$$\sum_i x_{i,t} \leq 1$$

Ini adalah salah satu sumber naturally combinatorial optimization.

J.15. Position sizing

Kemudian:

$$x_{i,t} = \frac{w_i}{\sum_i w_i}$$

dengan:

$$w_i$$

sebagai size conditional on selection.

Contohnya:

$$w_i = 2$$

Atau:

J.16. Risk budget

Kita tidak ingin:

ditentukan hanya oleh expected return.

Definisikan portfolio variance:

$$\sigma_p^2 =$$

Kemudian risk constraint:

$$\sigma_p^2 \leq \sigma_{\text{target}}^2$$

J.17. Factor exposure constraint

Karena kita mengetahui:

,

kita dapat membatasi:

$$t = tt$$

Misalnya:

atau:

Ini mencegah portfolio diam-diam menjadi taruhan besar terhadap satu factor.

J.18. Portfolio objective

Objective continuous ideal kita:

$$- \quad - \quad , t-1 \quad -$$

Empat komponennya:

- - - -

J.19. Mengubah maximization menjadi minimization

Quantum optimization lebih mudah kita tulis sebagai:

$$-$$

Definisikan:

Maka:

$$=$$

J.20. Tetapi QAOA membutuhkan binary variables

Maka kita harus melakukan discretization.

Misalnya size:

$$= -1 \quad 2 \\ = 0$$

dengan:

$$\in \{0, 1\}$$

Ini adalah **binary encoding**.

J.21. Binary representation

Misalnya:

$$= 0 \cdot 2^1 + 4 \cdot 2^0$$

Maka kemungkinan:

$$0, 2, 4$$

Jika:

$$= 001,$$

maka:

$$\in \{0, 001, 010, 100\}$$

J.22. Selection + sizing

Kita dapat langsung menggunakan:

untuk selection dan:

untuk size.

Hubungkan keduanya:

Dalam binary formulation, constraint ini dapat dipenalti.

J.23. Constraint penalty

Misalkan:

$$= 1$$

Kita tambahkan:

$$P_t = -1$$

Sehingga objective:

$$= -1^2$$

J.24. Cardinality penalty

Untuk:

,

kita dapat menggunakan:

$$P = 0, -$$

Untuk QUBO murni, kita dapat menggunakan slack variable.

J.25. Slack variable

Ubah:

menjadi:

$$=$$

dengan:

$$0$$

Kemudian representasikan secara binary.

J.26. QUBO final

Secara umum:

$$= t \ t \ 0$$

dengan:

$$\in \{0, 1\}$$

Di sinilah seluruh financial problem kita akhirnya masuk ke bentuk combinatorial optimization.

J.27. Dari QUBO ke Ising

Kita menggunakan:

$$= 1 \ 2$$

dengan:

$$\in \{-1, 1\}$$

Substitusi ke:

Maka:

$$= 0$$

J.28. Hamiltonian

Kita kemudian mengganti:

Sehingga:

$$= 0$$

Inilah **cost Hamiltonian**.

J.29. Mengapa Hamiltonian ini sangat penting?

Karena:

$$=$$

Dengan kata lain setiap computational basis state:

merepresentasikan satu portfolio candidate.

Eigenvalue-nya:

Maka:

$$=$$

J.30. QAOA

Kita mulai dari:

Kemudian:

$$H = \sum_i Z_i$$

J.31. Mixer Hamiltonian

Untuk unconstrained QAOA:

$$H_M$$

Tetapi nanti untuk constraint yang lebih ketat kita dapat menggunakan:

$$H_M$$

agar evolution tidak keluar dari feasible subspace.

Ini akan menjadi pengembangan penting.

J.32. Parameter optimization

Kita mencari:

$$H = \sum_i Z_i$$

Objective:

$$\langle H \rangle$$

J.33. Measurement

Setelah optimal:

$$H$$

kita lakukan measurement.

Probability:

$$P = \langle H \rangle^2$$

Kita mengambil:

=

atau best feasible sample.

J.34. Dari binary solution ke portfolio

Setelah measurement:

,

decode:

Kemudian:

$$\boxed{t = t}$$

J.35. Risk-based capital allocation

Di sini kita perlu memisahkan **portfolio optimization** dari **trade risk management**.

Misalnya portfolio menentukan:

Tetapi trade-level risk menentukan:

Jika modal:

$t,$

dan risk budget:

$t,$

maka:

$$\boxed{,t = t}$$

Misalnya:

= 001

Maka:

= 1 t

J.36. Stop loss

Untuk posisi long:

$$P = P_t -$$

Risk per unit:

$$= P_t - P$$

Position size:

$$=$$

Dengan demikian:

$$=$$

Ini adalah mathematical foundation dari ide kita sebelumnya:

setiap trade tetap memiliki risk sekitar \$1.

J.37. Volatility-adjusted stop

Daripada fixed distance:

$$= tt,$$

kita gunakan:

$$=$$

atau:

$$=$$

Sehingga:

$$=$$

Volatility tinggi:

Volatility rendah:

Ini secara finansial jauh lebih masuk akal.

J.38. Take profit

Misalnya:

$$P_P = P_t - P$$

Risk-reward ratio:

$$= \frac{P_P - P_t}{P_t - P}$$

Tetapi kita tidak akan membuat:

$$= 2$$

secara arbitrer.

Idealnya:

$$P$$

ditentukan berdasarkan conditional return distribution.

J.39. TP berbasis distribution

Misalkan:

$$,t$$

Kita dapat menentukan TP sebagai quantile:

$$P = ,t$$

Misalnya:

$$= 05$$

J.40. Dynamic trade management

Setelah posisi terbuka:

$$tt_{tt} = P_t, P_t, P, P_P, P_{t,t}, t, t$$

Kita definisikan action:

$$tt \in \{, , , P\}$$

J.41. Floating loss

Jika:

$$P_t \geq 0,$$

kita tidak otomatis close.

Kita evaluasi:

$$P_t \geq 0$$

Jika probabilitas masih tinggi:

$$P_t > 0$$

position dapat dipertahankan.

J.42. Floating profit

Jika:

$$P_t \geq 0$$

kita dapat menghitung:

$$P_t \geq 0$$

Kemudian:

$$\square$$

dibandingkan dengan:

$$\square$$

J.43. Continuation value

Definisikan:

$$t = P_t$$

Sedangkan:

$$t = P_t$$

Maka:

$$\square$$

Ini mengubah trailing stop dari rule arbitrer menjadi decision problem.

J.44. Stop-loss adjustment

Jika:

$$P_t \geq P_t$$

cukup jauh, kita dapat memindahkan SL.

Definisikan:

$$P_t = P_t - t$$

Tetapi harus memenuhi:

$$P_t, P_t$$

Dengan demikian stop hanya bergerak ke arah profit protection.

J.45. Dynamic TP

Jika momentum masih kuat:

$$t \geq 0$$

dan:

$$P_t \geq 0$$

tinggi, TP dapat diperbesar.

Misalnya:

$$P_t = P_t + P_{t,t}$$

dengan:

$$P_{t,t} = t, t, t, t_t$$

J.46. Tetapi kita harus mencegah infinite TP expansion

Jika tidak:

$$P_t$$

Maka:

$$P_{t,t}$$

Selain itu:

$$P_t$$

harus dibatasi.

J.47. Portfolio reoptimization

Pada interval:

$$\tau$$

kita melakukan:

$$t_{\tau} = t_{t_{\tau}, t_{\tau}, t_{\tau}, t_{\tau}}$$

Jadi portfolio tidak statis.

J.48. Reoptimization trigger

Tidak harus berdasarkan waktu saja.

Kita dapat memiliki:

$$t = t, t-1$$

Jika market state berubah cukup besar:

$$t = 1$$

Maka:

$$t$$

J.49. Distance antar-regime

Misalnya:

$$t = t - t-12$$

Jika:

$$t ,$$

kita menganggap market state berubah signifikan.

J.50. Full state vector

Sekarang kita dapat mendefinisikan state:

$$t = t, t, t, t, P, t, P, t$$

Ini merupakan informasi yang digunakan oleh policy.

J.51. Action vector

Definisikan:

$$t = t, t, t, P, t, t$$

dengan:

- t : target portfolio,
- t : order size,
- P : stop-loss,
- P_P : take-profit,
- t : order type/status.

J.52. State transition

Market bergerak:

$$t \rightarrow t+1$$

Secara umum:

$$t+1 \rightarrow P, t, t$$

Ini adalah stochastic transition model.

J.53. Reward

Reward utama:

$$t = P_t - t - t_t - P t_t$$

Misalnya:

$$P t_t = 2, t$$

Maka:

$$t = t t 1 - t - t_t - t t t$$

J.54. Objective global

Jika seluruh periode:

$$t = 1, , ,$$

maka:

$$= t=1 t$$

Inilah objective global sistem.

Tetapi implementasi kita tidak harus memakai RL.

Policy:

dapat dibentuk secara modular dari:

$$t \quad t \quad t$$

J.55. Full mathematical pipeline

Sekarang kita dapat menulis sistem kita secara ringkas:

$$t \quad t$$

kemudian:

$$t \quad t, t, t, t$$

kemudian:

$$t, t, t \quad t$$

kemudian:

$$t \quad , t$$

kemudian:

$$,t \quad t$$

kemudian:

$$t \quad t$$

kemudian:

$$t \quad t$$

kemudian:

$$_t \quad P t_t$$

kemudian:

$$P t_t \quad P_{t1}$$

J.56. Closed-loop system

Dan yang paling penting:

$$P_{t1}$$

menjadi bagian dari:

$$t1$$

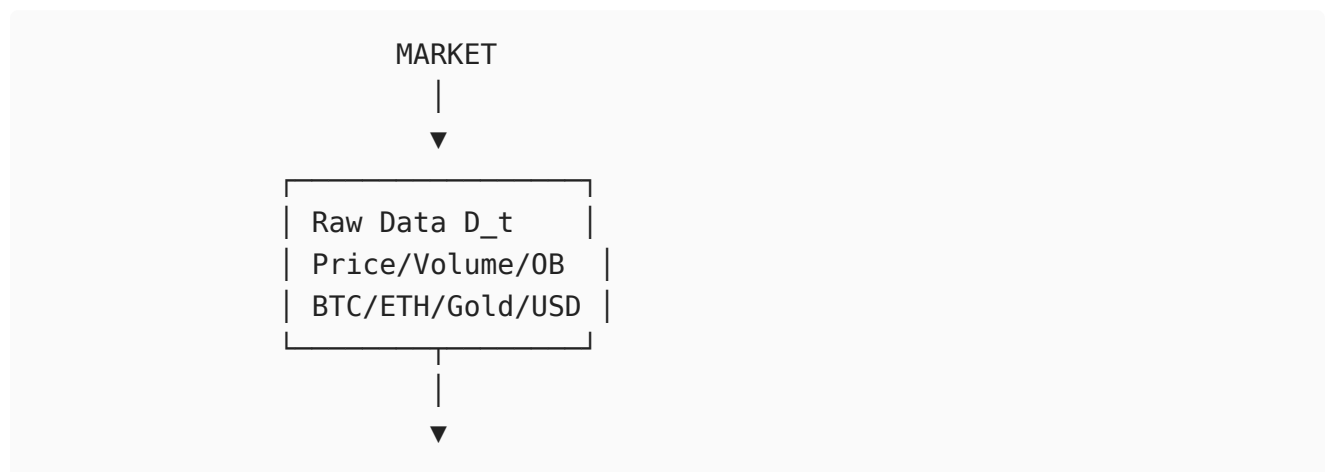
Sehingga:

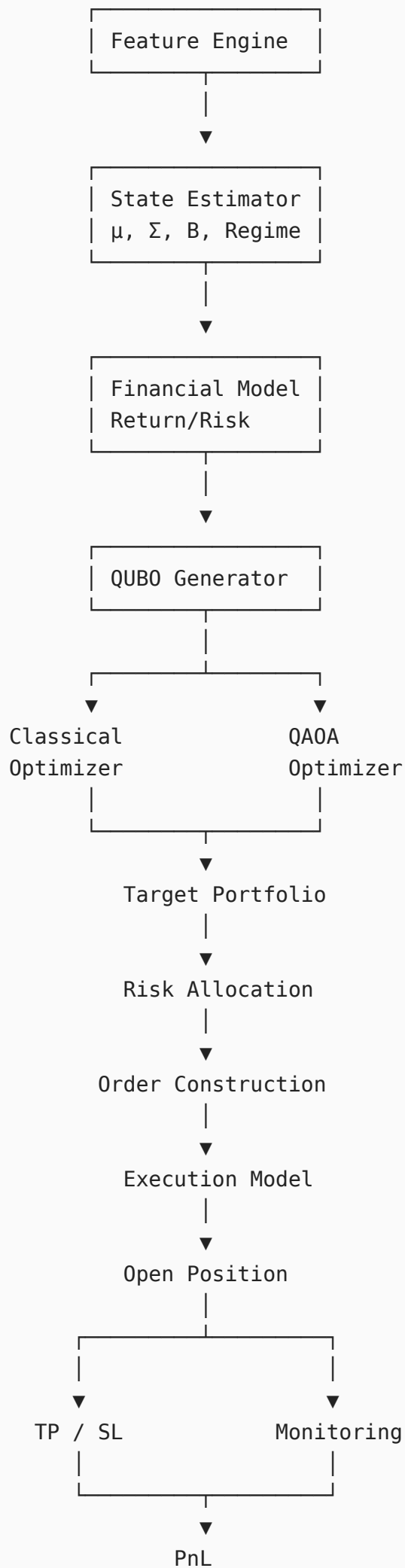
$$_t \quad t \quad P_{t1} \quad t1 \quad t1$$

Ini membuat sistem kita **closed-loop**.

J.57. Arsitektur matematis lengkap

Jika kita gambarkan:





↓
New State

J.58. Satu hal penting: QAOA bukan keseluruhan trading system

Ini harus kita pegang.

QAOA hanya berada di:

□

Bukan:

Bukan:

Bukan:

Sehingga:

t

Lebih tepat:

$t = t \quad tt \quad t \quad t$

QAOA adalah salah satu optimizer yang kita uji di dalamnya.

J.59. Ini juga membuat eksperimen kita jauh lebih bersih

Kita dapat mengganti:

t

tanpa mengubah komponen lain.

Misalnya:

$1 =$

dan:

$2 =$

Maka:

$$= t$$

sedangkan:

$$= t$$

Semua komponen lainnya identik.

J.60. Formulasi penelitian paling penting

Kalau seluruh proyek ini akhirnya kita jadikan paper/skripsi yang serius, saya akan merumuskan pertanyaan utamanya sebagai:

$ttttttttt,t,ttttttttt$

Bukan:

"Apakah quantum computing bisa menghasilkan profit?"

Itu pertanyaan yang terlalu sederhana.

J.61. Dan hipotesisnya

Kita dapat memiliki:

$$H_0$$

Tidak terdapat perbedaan signifikan antara QAOA dan classical optimization:

$$0 =$$

$$H_1$$

QAOA menghasilkan performa yang berbeda:

1

Atau jika kita ingin menguji keunggulan:

1

Tetapi kita harus membiarkan data menentukan hasilnya.

J.62. Yang menarik: kita sekarang memiliki tiga "ground truth"

Untuk ukuran kecil:

$$\boxed{ttt}$$

memberikan:

Kemudian:

$$\boxed{t}$$

memberikan:

Dan:

$$\square$$

memberikan:

Sehingga:

$$,$$

Untuk minimization.

Kemudian kita dapat mengukur:

$$= -$$

dan:

$$= -$$

Ini memberi kita **ground-truth mathematical benchmark**.

J.63. Baru setelah itu kita lihat uang

Kemudian:

$$P_t \ P$$

dan:

$$P_t \ P$$

Ini memungkinkan kita menjawab pertanyaan menarik:

$$\square$$

Dan justru **ketidaksetaraan tersebut merupakan sesuatu yang sangat menarik untuk diteliti.**

Jadi secara matematis sistem kita dapat diringkas menjadi:

Ini adalah mathematical backbone dari keseluruhan sistem kita.

Setelah menyatukan A–J, saya justru ingin sedikit mengubah filosofi sistem kita:

10

secara langsung.

$$\overline{t \quad tt \quad t \quad Pt \quad tt \quad t}$$

”

Sementara forecasting tetap menggunakan metode statistik/machine learning yang memang cocok untuk forecasting, dan execution tetap menggunakan model microstructure.

Itu membuat keseluruhan sistem jauh lebih masuk akal secara finansial **dan** jauh lebih kuat secara ilmiah.

Tetapi ada satu tahap yang menurut saya **sangat berharga sebelum coding**:

Tahap K — Concrete Market Specification

Di Tahap K kita berhenti berbicara secara abstrak dan menentukan **satu instansiasi nyata** dari seluruh framework.

Misalnya kita menetapkan:

$$= \{, , , \}$$

untuk crypto, lalu:

$$t = \{, , , P, \}$$

Kemudian menentukan secara eksplisit:

- timeframe,
- data source,
- lookback window,
- forecast horizon,
- model forecasting,
- covariance estimator,
- regime model,
- jumlah asset yang boleh dipilih,
- discretization weight,
- seluruh QUBO coefficient,
- penalty coefficient,
- encoding qubit,
- QAOA depth ,
- mixer,
- optimizer,
- risk budget,
- SL,
- TP,
- trailing logic,
- reoptimization interval,
- transaction cost,
- slippage,
- walk-forward scheme,
- dan **baseline classical optimizer**.

Dengan kata lain:

$$=$$

sedangkan:



Setelah K selesai, kita dapat membuat **versi 1.0 dari algoritma lengkap**, pseudocode end-to-end, struktur repository, database schema, dan kemudian mulai mengimplementasikannya secara modular tanpa lagi mengubah-ubah definisi matematis di tengah coding.

Prompt:

8/17/2026, 9:54:27 PM

lanjut Tahap K

Response:

8/17/2026, 9:54:28 PM

Remembering

Clarified Stage J/K trading system and QAOA portfolio design

Worked for 6s

Tahap K — Concrete Market & Numerical Specification

Sekarang kita masuk ke tahap yang menurut saya **lebih penting daripada sekadar mulai coding**.

Pada Tahap J kita sudah mempunyai:



Pada Tahap K kita akan menentukan **angka, variabel, data, horizon, constraint, encoding, dan prosedur eksperimen secara konkret** sehingga sistem tersebut dapat benar-benar diimplementasikan.

Saya akan membuat Tahap K sebagai **spesifikasi V1.0**. Artinya, kita jangan langsung membuat sistem terlalu kompleks. Kita buat sistem yang cukup kecil untuk divalidasi secara matematis, tetapi arsitekturnya sudah mampu berkembang dari crypto ke saham internasional.

K.1. Prinsip desain utama

Ada satu keputusan yang sangat penting:

Jangan langsung mencoba membuat "hedge-fund-grade system" dalam versi pertama.

Kita akan membuat:

$$1 =$$

Kemudian:

$$1 \quad 2$$

Dengan demikian setiap peningkatan dapat diuji secara independen.

K.2. Universe V1

Untuk tahap pertama saya memilih **6 aset crypto** sebagai universe utama:

$$= \{, , , , P, \}$$

Alasannya bukan karena enam aset ini pasti merupakan pilihan trading terbaik, tetapi karena jumlah ini cukup untuk menghasilkan masalah kombinatorial namun masih sangat mudah diverifikasi secara exact.

Jumlah aset:

$$=$$

Kemudian kita mempunyai beberapa **factor assets** yang tidak harus masuk ke portfolio:

$$= \{, , , , P, \}$$

BTC dan ETH dapat berfungsi ganda sebagai:

1. tradable asset,
2. market factor.

K.3. Mengapa kita tidak langsung menggunakan 50–500 aset?

Karena kita ingin membandingkan:

$$t$$

melawan:

melawan:

Untuk $n = 4$, jika setiap aset hanya mempunyai binary selection:

$$2^n = 16$$

portfolio.

Itu sangat kecil.

Kita dapat melakukan:

$$\min_t$$

untuk menemukan global optimum secara pasti.

Ini menjadi **ground truth**.

Kemudian kita dapat meningkatkan kompleksitas.

K.4. Tahap eskalasi ukuran

Saya menyarankan:

Experiment	Assets	Binary variables	Tujuan
E1	4	4–8	Validasi matematis
E2	6	6–12	Benchmark exact
E3	10	10–20	QUBO benchmark
E4	20	20–40	Scaling
E5	50+	50+	Realistic portfolio

Dengan demikian:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 5$$

dan bukan langsung melompat ke E5.

K.5. Timeframe utama

Untuk V1 kita menggunakan:

$$\Delta t = 1$$

sebagai timeframe trading.

Mengapa?

Karena kita ingin berada di antara:

- terlalu cepat seperti HFT,
- terlalu lambat seperti daily portfolio management.

Jadi sistem kita adalah:

$$t \ll T$$

bukan HFT.

K.6. Forecast horizon

Forecast utama:

$$h = 1$$

Sehingga:

$$r_t = r_{t-1} + \Delta r_t$$

Namun kita juga akan menghitung:

$$h \in \{1, 4, 24\}$$

untuk **trade management**, bukan sebagai tiga model terpisah pada tahap pertama.

K.7. Training window

Kita menggunakan rolling window:

$$h = 0$$

untuk V_1 .

Karena:

$$0.24 = 20$$

observasi hourly.

Jadi pada waktu t :

$$t = \{t-1, \dots, t\}$$

K.8. Mengapa rolling window?

Karena kita tidak ingin mengasumsikan:

$$P_{t1} = P_{t1}$$

selamanya sama.

Pasar memiliki:

$$- \frac{1}{t} \frac{dP}{dt}$$

Dengan rolling window:

$$t = t - \Delta t$$

sehingga parameter dapat berubah terhadap waktu.

K.9. Data yang dibutuhkan

Untuk setiap asset:

□

yaitu:

- Open,
- High,
- Low,
- Close,
- Volume.

Selain itu:

□

dan kemudian, pada versi berikutnya:

□

K.10. Kita jangan langsung memasukkan order book ke V1

Ini keputusan desain yang penting.

Order book memiliki dimensi yang jauh lebih besar:

1, ,

1, ,

beserta:

,

Kalau kita memasukkannya sekarang, kita akan sulit mengetahui apakah perubahan performa berasal dari:

- forecasting,
- factor model,
- QUBO,
- QAOA,
- atau microstructure.

Jadi:

$$\boxed{1 \quad t}$$

dan:

$$\boxed{2}$$

K.11. Return

Gunakan:

$$\boxed{_{,t} = P_{,t}P_{,t-1}}$$

seperti yang sudah kita tetapkan.

K.12. Feature vector

Untuk setiap asset , kita buat:

$$_{,t} = 1,_{,t} \ 4,_{,t} \ 24,_{,t} \ ,_{,t} \ ,_{,t} \ ,_{,t} \ ,_{,t} \ ,_{,t} \ ,_{,t}$$

Di sini:

$$= -1_{=0} \ t-$$

K.13. Volatility

Rolling volatility:

$$r_{t,\tau} = 1 - \frac{1}{t} \sum_{\tau=t-1}^{\tau} r_{\tau}^2$$

dengan:

$$r_{t,\tau} = 1 - \frac{1}{t} \sum_{\tau=t-1}^{\tau} r_{\tau}^2$$

K.14. Volume feature

Gunakan normalized volume:

$$v_{t,\tau} = \frac{r_{t,\tau}}{r_{t,\tau}}$$

Sehingga:

$$1$$

berarti volume relatif tinggi.

K.15. Factor exposure

Untuk setiap asset:

$$r_t = \alpha_0 + \alpha_1 r_{t-1} + \alpha_2 P_{t-1} + \epsilon_t$$

Kemudian:

$$r_t = \alpha_0 + \alpha_1 r_{t-1} + \alpha_2 P_{t-1} + \epsilon_t$$

K.16. Forecast model V1

Kita jangan langsung menggunakan neural network.

V1 menggunakan model yang sangat interpretable:

$$r_{t+1} = \alpha_0 + \alpha_1 r_t + \alpha_2 P_t + \epsilon_{t+1}$$

Secara vector:

$$r_{t+1} = \alpha_0 + \alpha_1 r_t + \alpha_2 P_t + \epsilon_{t+1}$$

Ini sengaja sederhana.

Karena kalau QAOA kemudian memberikan hasil buruk, kita dapat mengatakan:

masalahnya berasal dari optimization layer,

bukan:

mungkin neural network-nya buruk.

K.17. Regularisasi forecasting

Agar parameter tidak terlalu overfit:

$$- \frac{1}{22} \frac{1}{22}$$

Ini adalah ridge regression.

Solusinya:

$$= -1$$

K.18. Covariance estimator

Untuk V1:

$$t = t - 1t$$

Tetapi untuk implementasi yang lebih stabil, saya menyarankan shrinkage:

$$t = 1 - t \quad t$$

dengan:

$$t = t$$

K.19. Mengapa shrinkage?

Karena covariance matrix empiris dapat menjadi noisy.

Terutama ketika:

mendekati:

Shrinkage membuat:

lebih stabil dan positive definite.

K.20. Regime detection V1

Kita menggunakan tiga regime sederhana:

$$t \in \{, t, \}$$

Definisikan market return:

$$,t = 1 \quad ,t$$

Kemudian:

$$t = -$$

Aturan sederhana:

$$t = , \quad t \quad t, \quad t \quad , \quad t \quad -$$

K.21. Tetapi regime jangan langsung menjadi constraint

Ini penting.

Regime pada V1 digunakan sebagai:

$$t$$

dan bukan:

$$tt$$

Jadi model dapat belajar bahwa:

mungkin mengurangi expected return, tetapi tidak otomatis memaksa:

$$= 0$$

K.22. Universe selection

Setiap periode:

t

kita ranking asset berdasarkan:

$=$

Kemudian pilih:

$=$

Sehingga:

K.23. Mengapa hanya tiga?

Karena kita ingin membuat:

$t \ t$

menjadi masalah combinatorial yang nyata.

Jika semua asset boleh dipilih:

$= 1$

untuk semuanya, masalah selection menjadi kurang menarik.

K.24. Weight discretization

Gunakan step:

Maka weight:

$\in \{0, 005, 010, \dots, 050\}$

Maximum per asset:

$= 050$

K.25. Binary encoding

Karena:

$050005 = 10,$

kita membutuhkan representasi integer:

$$\in \{0, 1, 10\}$$

Binary encoding:

$$= 0 \cdot 2_1 + 4_2$$

Kemudian:

$$= 005$$

K.26. Ada masalah dengan angka 8

Karena:

dengan empat bit dapat mencapai:

$$15$$

Sedangkan maksimum kita hanya:

$$10$$

Maka konfigurasi:

$$10$$

harus dibuat infeasible.

Kita dapat menambahkan:

$$P$$

Ini contoh kecil tetapi penting dari desain QUBO.

K.27. Budget constraint

Kita ingin:

$$= 1$$

Penalty:

$$P = \frac{1}{2} - 1$$

K.28. Cardinality constraint

Penalty:

$$P = 0, -^2$$

Untuk QUBO murni nanti kita ubah menggunakan slack variable.

K.29. Selection-size coupling

Kita harus memastikan:

$$= 0 \quad = 0$$

Gunakan:

$$P = 1 -$$

Jika:

$$= 0$$

tetapi:

$$0,$$

maka ada penalty.

K.30. Financial objective V1

Kita gunakan:

$$= - - 1$$

Tiga komponen:

$$-$$

= reward expected return,

= risk,

dan:

$$- 1$$

= turnover penalty.

K.31. Mengapa turnover penting?

Tanpa turnover penalty:

$$t$$

dapat berubah drastis setiap jam.

Misalnya:

$$t = 05,0,02$$

kemudian satu jam kemudian:

$$t_1 = 0,01,0$$

Secara matematis mungkin optimal.

Secara trading:

$$\square$$

karena banyak transaksi.

K.32. Transaction cost

Kita gunakan model awal:

$$t = ,t -,t-1$$

dengan:

sebagai biaya efektif.

Kemudian pada versi order-book:

$$= tt$$

K.33. Objective lengkap

Jadi:

$$t, = - t t - t-11 - 1^2 P 1 -$$

K.34. Risk aversion

Parameter:

$\square$

mengontrol:

t

Jika:

$0,$

model semakin agresif.

Jika:

,

model semakin konservatif.

Kita jangan menentukan dengan melihat hasil backtest kemudian.

Itu akan menjadi:

t

K.35. Parameter calibration

Kita menggunakan training period untuk menentukan:

''''

Kemudian parameter dibekukan selama test period.

K.36. Penentuan penalty coefficient

Penalty harus lebih besar daripada keuntungan yang mungkin diperoleh dari melanggar constraint.

Secara konseptual:

$tt \Delta$

Sehingga:

tidak pernah lebih menguntungkan daripada solusi feasible.

K.37. QUBO

Setelah semua binary substitution:

$$=$$

dan:

$$= ,$$

kita akan mendapatkan:

$$\boxed{=}$$

Dengan:

$$\in \{0, 1\}$$

K.38. Jumlah qubit

Untuk enam asset:

Selection

Weight

4 qubits per asset:

$$4 = 24$$

Total:

$$\boxed{= 0}$$

sebelum slack variables.

Ini sudah cukup besar untuk menunjukkan mengapa masalah ini menarik secara quantum optimization.

K.39. Tetapi jangan langsung menjalankan QAOA 30 qubit

Ini sangat penting.

Untuk validasi:

$$= 4$$

terlebih dahulu.

Misalnya:

$$20$$

Kemudian:

$$=$$

dan seterusnya.

K.40. Classical exact solver

Untuk:

$$25$$

kita dapat menggunakan brute force jika memungkinkan.

Untuk setiap:

$$\in \{0, 1\}$$

hitung:

Kemudian:

$$=$$

Ini adalah ground truth.

K.41. Classical optimization baseline

Kita tidak cukup hanya membandingkan:

dengan brute force.

Kita perlu baseline realistis:

$$P$$

dan:

$$t$$

Untuk continuous version:

$$P$$

atau optimizer constrained classical yang setara.

K.42. Jadi kita mempunyai empat baseline

$$t$$

$$t$$

$$t$$

$$\square$$

Ini jauh lebih kuat daripada sekadar:

"Saya menjalankan QAOA."

K.43. QAOA V1

Cost Hamiltonian:

$$=$$

Mixer:

$$=$$

Depth awal:

$$= 1$$

Kemudian:

$$= 2,$$

sebagai eksperimen scaling.

K.44. Mengapa mulai dari $= 1$?

Karena:

berarti:

- circuit semakin dalam,
- parameter semakin banyak,

- optimization semakin sulit,
- noise semakin besar jika hardware digunakan.

Kita ingin mengetahui apakah:

$$= 1$$

sudah memberikan solusi yang berguna.

K.45. QAOA parameter optimizer

V1:

$$\square$$

atau optimizer derivative-free sejenis.

Kemudian dibandingkan dengan:

$$P$$

Karena kita memang sebelumnya sudah mempelajari SPSA dan parameter-shift.

K.46. Objective QAOA

$$, = , ,$$

Cari:

$$, = ,$$

K.47. Quality metric QAOA

Jangan hanya mengukur apakah QAOA menghasilkan profit.

Pertama:

$$tt$$

atau cost gap.

Untuk minimization:

$$= -$$

Semakin dekat:

0,

semakin baik.

K.48. Probability of sampling optimum

Karena QAOA menghasilkan distribusi:

,

kita dapat mengukur:

$$P_t =$$

Ini sangat penting.

QAOA tidak harus selalu menghasilkan optimum dalam setiap measurement.

Yang kita inginkan adalah:

$$P_t$$

cukup tinggi.

K.49. Feasibility probability

Kita juga ukur:

$$P = \epsilon$$

dengan:

adalah set solusi feasible.

Ini bahkan bisa menjadi lebih penting daripada:

$$P_t$$

K.50. Mengapa?

Bayangkan:

$$P_t = 0.005$$

tetapi:

$$P = 0.05$$

Ini masih cukup bagus.

Tetapi jika:

$$P_t = 020$$

dan:

$$P = 025,$$

maka sebagian besar distribution QAOA menghasilkan portfolio yang melanggar constraint.

K.51. Trade construction

Setelah portfolio:

,

kita menentukan entry signal.

Jangan langsung market order.

Kita definisikan:

$$= , , t$$

K.52. Entry threshold

Misalnya:

$$\square$$

Dengan kata lain:

expected return harus cukup besar dibanding uncertainty.

Jika:

,

tidak ada trade.

K.53. Limit order

Jika signal valid:

$$= {}^tP_t$$

Order **tidak langsung** dieksekusi.

Ini sesuai dengan filosofi yang kita bangun sebelumnya.

K.54. Order expiry

Limit order tidak boleh hidup selamanya.

Definisikan:

$$= t_t$$

Misalnya:

$$_t = 1$$

Jika tidak fill:

$$\square$$

K.55. Fill model V1

Karena kita belum menggunakan tick-level order book:

ditentukan berdasarkan OHLC.

Misalnya untuk long:

$$P_{,t} \ P_t$$

maka order dianggap fill.

Tetapi ini memiliki masalah:

| Apakah harga menyentuh entry sebelum SL/TP?

Karena itu kita memerlukan aturan konservatif.

K.56. Ambiguity rule

Jika dalam satu candle:

dan:

$$P,$$

kita **tidak boleh memilih outcome yang menguntungkan.**

Gunakan worst-case ordering:

□

Ini menghindari look-ahead bias.

K.57. Initial stop-loss

Untuk long:

$$= P_t -$$

V1:

$$= 2$$

Jadi:

$$= 2$$

K.58. Risk per trade

Misalnya:

$$_t = 1$$

Maka:

$$= 1P_t - P$$

Tetapi ini belum memperhitungkan fee/slippage.

Nanti:

$$= {}_t t_t$$

K.59. Portfolio-level risk

Jika ada tiga posisi:

$$_1 = {}_2 = {}_3 = 1,$$

maka bukan berarti portfolio risk otomatis:

Karena posisi dapat correlated.

Portfolio risk:

$$=$$

Ini alasan factor/covariance model tetap diperlukan pada trade-level.

K.60. Maximum aggregate risk

V1:

$$t, = t$$

Tetapi kemudian kita akan menggunakan correlation-adjusted risk.

K.61. Take profit V1

Gunakan:

$$P = P_t P$$

Dengan:

$$P =$$

Sehingga initial:

$$= 2 = 15$$

Ini **bukan angka sakral**. Nanti diuji melalui walk-forward training.

K.62. Dynamic SL

Jika:

$$P \ 0$$

dan:

$$P_t - P_t ,$$

kita dapat memindahkan:

$$P$$

Misalnya:

$$= 1$$

K.63. Trailing stop

Setelah BEP:

$$t = t_{-1}, P_t - tt$$

Ini menjamin:

$$t \quad t-1$$

K.64. Dynamic TP

Jika continuation value:

,

maka:

$$P$$

dapat diperbesar.

Misalnya:

$$P, = P, \Delta, P,$$

V1:

$$\Delta = 05$$

dan:

$$P, = 5$$

K.65. Exit decision

Setiap satu jam:

$$\boxed{tPt}$$

dengan:

$$t, t, t, t, t, P_t$$

K.66. Decision rule

Secara konseptual:

$$= , , , P \geq 0, P, , t$$

Ini akan kita buat lebih formal ketika masuk ke tahap implementasi trade-management.

K.67. Maximum holding period

Sebuah trade tidak boleh bertahan selamanya.

Definisikan:

$$\boxed{= 24}$$

Jika:

$$t - t_t \geq 24,$$

maka:

K.68. Ini penting untuk menghindari hidden risk

Tanpa:

,

sistem short-term dapat berubah menjadi:

Padahal model kita seharusnya:

$$t$$

K.69. Walk-forward backtesting

Sekarang bagian yang sangat penting.

Tidak boleh:

$$2021-2024 \quad t_{2021-2024}$$

Itu bukan backtest yang valid.

Kita gunakan:

K.70. Struktur walk-forward

Misalnya:



Setiap test period benar-benar out-of-sample.

K.71. Contoh konkret

Misalnya:

$$= 0$$

dan:

$$t =$$

Kemudian:

$$2 = 1$$

K.72. Rolling versus expanding

Untuk sistem kita:

lebih sesuai.

Jadi:

$$t = t - , t$$

Bukan:

$$t = 0, t$$

K.73. Kenapa?

Karena market regime berubah.

Data sepuluh tahun lalu mungkin mempunyai relevansi lebih rendah daripada:

$$0$$

Tetapi ini adalah hipotesis yang harus diuji, bukan asumsi final.

K.74. Backtest pipeline

Pada setiap waktu t :

$$1t2ttt4tt5ttttttt10tt11t12P$$

Kemudian:

$$t \quad t \quad 1$$

K.75. Yang harus disimpan setiap timestamp

Database/backtest record minimal:

$$t = \{tt, tt, t, , , t, , , t, , P, P\}$$

Ini akan sangat berguna untuk debugging.

K.76. Performance metrics

Kita tidak boleh hanya menggunakan:

$$tP$$

Minimal:

Return

$$r_t$$

Sharpe

$$S = \frac{r_t - r_f}{\sigma(r_t)}$$

Sortino

$$S_t = \frac{r_t - r_f}{\sigma_d(r_t)}$$

Maximum Drawdown

$$MDD = P_t - P_{t_{min}}$$

K.77. Trade metrics

Kita juga ukur:

$$R_t = \frac{P_t - P_{t_{min}}}{P_t}$$

Profit factor:

$$PF = \frac{P}{L}$$

Average win:

Average loss:

Expectancy:

$$E = (P \cdot W) - (L \cdot L)$$

K.78. QAOA-specific metrics

Selain financial metrics:

$$Q_t = \frac{P_t}{P_{t_{min}}}$$

K.79. Ini menghasilkan dua evaluation layers

Layer 1 — Mathematical

□

berdasarkan:

Layer 2 — Financial

tt

berdasarkan:

$P, \dots, P,$

Kita harus memisahkan kedua layer ini.

K.80. Eksperimen inti

Saya menyarankan eksperimen utama berikut.

Experiment A

t

vs

Tujuan:

Apakah QAOA mampu menemukan solusi dekat optimum?

Experiment B

vs

Tujuan:

Apakah QAOA kompetitif terhadap classical combinatorial optimizer?

Experiment C

Trading tanpa QAOA:

t

Bandingkan dengan:

t

K.81. Experiment D — no-factor model

Gunakan:

$$= t$$

tanpa:

''''

Kemudian bandingkan dengan:

$$t -$$

Ini menguji apakah market factors benar-benar menambah informasi.

K.82. Experiment E — no dynamic management

Bandingkan:

Static

$$, P$$

tidak berubah.

melawan:

Dynamic

$$t, P_t$$

berubah berdasarkan state.

Ini menguji ide yang kita diskusikan sejak awal.

K.83. Experiment F — transaction cost sensitivity

Jalankan:

$$= 0$$

kemudian:

$$=$$

kemudian:

=

kemudian:

=

Jika strategy hanya profitable ketika:

= 0,

maka secara praktis strategy tersebut tidak menarik.

K.84. Experiment G — regime robustness

Evaluasi terpisah:

, t ,

Kita ingin mengetahui apakah strategy hanya bekerja dalam satu regime.

K.85. Experiment H — quantum depth

Bandingkan:

= 1

= 2

=

Kemudian:

Kita ingin mengetahui:

Apakah depth tambahan benar-benar memberikan improvement?

K.86. Experiment I — scaling

Naikkan:

= 4, 10, 20

Catat:

dan:

t

Kemudian:

□

menjadi salah satu hasil penelitian yang sangat menarik.

K.87. Experiment J — stochastic robustness

Karena QAOA menghasilkan probabilitas:

,

jalankan measurement berulang:

$$t = \{100, 500, 1000, 5000\}$$

Kemudian ukur:

$$P_t$$

K.88. Data split

Kita harus mempunyai tiga bagian:

□

t

t

Test set **tidak boleh digunakan untuk memilih parameter**.

K.89. Parameter yang harus dibekukan

Setelah validation selesai:

$$= \{, , , , , P, , \}$$

dibekukan.

Baru kemudian:

t

K.90. Anti-lookahead rule

Ini akan menjadi salah satu aturan paling keras dalam implementasi.

Pada timestamp t :

$$\boxed{t}$$

Tidak boleh menggunakan:

$$P_{t1}$$

untuk menentukan:

$$t$$

K.91. Bahkan covariance harus lagged correctly

Jika:

$$t$$

digunakan untuk membuat portfolio t , maka:

$$t$$

harus dihitung dari:

$$t$$

Bukan:

$$t1$$

Ini terlihat sepele tetapi merupakan salah satu sumber terbesar backtest leakage.

K.92. Execution lag

Kita juga jangan mengasumsikan:

$$t \quad t$$

secara instan.

Minimal:

$$t \quad t \quad t\Delta$$

Untuk V1:

$$\boxed{\Delta = 1}$$

K.93. Mengapa?

Karena jika:

$$P_t$$

digunakan untuk menghasilkan signal dan kita langsung menganggap entry pada:

$$P_{t,}$$

kita berpotensi mendapatkan harga yang tidak benar-benar tersedia setelah signal dihitung.

K.94. Struktur final V1

Jadi sistem V1 kita:

$t = t = 1t = = 0 = = - tt = tt = t = 5tt = 50 = - t = t = t = = 1, 2, = 2P = t = 1 = 24tt =$

K.95. Arsitektur software

Sekarang spesifikasi matematis ini dapat diterjemahkan menjadi modul:

```
trade_system/
|
├─ data/
|   ├── downloader
|   ├── cleaner
|   ├── resampler
|   └─ factor_data
|
├─ features/
|   ├── returns
|   ├── volatility
|   ├── momentum
|   └─ factor_exposure
|
├─ forecasting/
|   ├── ridge
|   └─ prediction
|
├─ risk/
|   ├── covariance
|   ├── shrinkage
|   └─ portfolio_risk
|
├─ optimization/
|   ├── classical
|   └─ exact
```

```
|   |   | qubo
|   |   | ising
|   |   | qaoa
|
|   | execution/
|   |   | order_generator
|   |   | fill_model
|   |   | transaction_cost
|
|   | position/
|   |   | stop_loss
|   |   | take_profit
|   |   | trailing
|   |   | position_manager
|
|   | backtest/
|   |   | engine
|   |   | walk_forward
|   |   | metrics
|
|   | experiments/
|   |   | exact_vs_qaoa
|   |   | factor_ablation
|   |   | cost_sensitivity
|   |   | qaoa_depth
|   |   | scaling
```

K.96. Dan di sinilah proyek kita menjadi sangat menarik

Perhatikan bahwa **QAOA** hanya membutuhkan sebagian kecil dari keseluruhan sistem.

Strukturnya:



Jadi proyek ini bukan sekadar:

"Saya menggunakan quantum computer untuk trading."

Lebih tepat:

Saya membangun dynamic factor-aware portfolio/trading optimization problem yang kemudian diformulasikan sebagai QUBO/Ising dan diselesaikan menggunakan QAOA, dengan evaluasi terhadap classical optimization dan realistic execution constraints.

Itu jauh lebih kuat secara akademik.

K.97. Satu hal yang perlu kita luruskan sejak sekarang

Saya ingin memisahkan dua sistem:

Sistem penelitian

□

Tujuannya:

Sistem live trading

Pt

Tujuannya:

Jangan mencampurkan keduanya.

Production engine baru boleh dibuat setelah:

$tt \ P \ tt \ t$

K.98. Roadmap setelah Tahap K

Sekarang seluruh desain abstrak kita sudah cukup konkret.

Urutan selanjutnya seharusnya:

$\square$

dengan:

Tahap L — Data Engineering

Kita menentukan secara konkret:

$t \ t \ t \ t$

Termasuk masalah yang sangat penting seperti:

- timezone,
- missing candle,
- survivorship bias,
- delisted assets,

- corporate action untuk saham,
- perbedaan jam perdagangan saham vs crypto,
- DXY/Gold/SPX yang tidak trading 24/7,
- dan bagaimana factor data disejajarkan dengan hourly crypto data.

Tahap M — Forecasting & Factor Engine

Kita turunkan secara lengkap:

dan:

”

Tahap N — QUBO/Ising Engine

Ini akan menjadi salah satu bagian paling matematis:

$$t \quad t$$

Kita bahkan dapat mengambil **contoh numerik 4 aset** dan menurunkan seluruh elemen:

”

secara eksplisit sampai menjadi:

Tahap O — QAOA + Classical Benchmark

Baru kita implementasikan:

$$t$$

Dan kita ukur:

$$, P_t, P, t$$

Posisi kita sekarang

Dengan Tahap A–K, kita sudah berpindah dari sekadar ide:

"Bagaimana kalau kita menggunakan QAOA untuk trading?"

menjadi sebuah objek penelitian yang jauh lebih terdefinisi:

$$\boxed{tt1}$$

Dan yang menurut saya paling penting: **arsitektur ini tidak terkunci pada crypto**. Ketika kita pindah ke saham internasional, bagian optimization engine pada dasarnya tetap sama. Yang berubah terutama adalah universe, market-calendar handling, factor set, liquidity

model, dan execution model. Faktor seperti **Gold dan USD/DXY** dapat tetap menjadi latent/common drivers, sementara factor equity seperti SPX/VIX dapat ditambahkan.

Tahap berikutnya yang paling tepat adalah Tahap L — Data Engineering, karena sebelum kita menurunkan QUBO secara numerik, kita harus memastikan bahwa t , t , dan seluruh factor exposure yang masuk ke QUBO benar-benar berasal dari data yang secara temporal konsisten dan bebas look-ahead. Ini justru salah satu bagian yang paling sering membuat backtest kuantitatif terlihat bagus padahal salah.

Prompt:

8/17/2026, 9:55:53 PM

lanjut Tahap L

Response:

8/17/2026, 9:55:53 PM

Tahap L — Data Engineering: Dari Data Pasar Mentah Menjadi State Matematis yang Valid

Tahap L adalah fondasi seluruh sistem. Pada tahap ini kita belum berbicara tentang QAOA secara mendalam. Tujuannya adalah memastikan bahwa pada setiap waktu t , sistem memiliki **state pasar yang benar-benar hanya dibangun dari informasi yang tersedia pada saat t** .

Secara keseluruhan:

t

dengan t sebagai **information set** pada waktu t .

L.1. Prinsip paling penting: Information Set

Kita definisikan:

$t = t$

Forecast harus berbentuk:

$t1 = t1 \ t$

Bukan:

$t1 \ t1$

Perbedaan satu indeks ini sangat penting.

Model kita harus menjawab:

"Jika saya benar-benar berada pada pukul 10:00, informasi apa yang saya miliki?"

bukan:

"Setelah candle berikutnya selesai, apa yang sebenarnya terjadi?"

L.2. Masalah utama: pasar kita tidak memiliki kalender yang sama

Ini menjadi jauh lebih penting ketika kita menggabungkan crypto dengan:

- saham AS,
- emas,
- USD/DXY,
- indeks saham,
- VIX.

Crypto:

24

Saham AS:

Gold futures:

245

DXY:

Maka kita tidak boleh sekadar melakukan:

```
data = pd.concat(...)
```

dan menganggap timestamp-nya otomatis comparable.

L.3. Kita membutuhkan canonical clock

Untuk V1:

$$\Delta t = 1$$

dan kita gunakan:

$$t = t_0 + \Delta t$$

Sehingga semua data akhirnya dipetakan ke grid:

$$= \{t_0, t_1, \dots, t\}$$

L.4. Timezone

Saya sangat menyarankan seluruh internal engine menggunakan:

□

sebagai timezone canonical.

Jadi:

t t

Baru kemudian ketika menampilkan hasil:

t

atau timezone lain.

Jangan menyimpan sebagian data dalam WIB dan sebagian dalam UTC.

L.5. Mengapa UTC?

Karena jika kita menggunakan:

t

untuk crypto dan:

untuk saham, DST Amerika akan menciptakan pergeseran jam.

Dengan UTC:

□

L.6. Timestamp candle

Misalkan kita memiliki candle:

10:00, 11:00

Kita harus menentukan apakah timestamp candle merepresentasikan:

- opening time,
- closing time.

Saya menyarankan:

$$tt = t$$

Jadi candle:

10 00, 11 00

direpresentasikan sebagai:

$$t = 11\ 00$$

Dengan demikian seluruh OHLCV tersebut diketahui pada:

$$t = 11\ 00$$

L.7. Ini menghindari look-ahead

Misalkan:

$$t$$

adalah close candle pada pukul 11:00.

Kita boleh menggunakan:

$$t$$

untuk membuat signal pada:

$$t = 11\ 00$$

Tetapi kita **tidak boleh** menggunakan:

$$t_1$$

untuk membuat keputusan pada:

$$t$$

L.8. Data layer

Saya menyarankan struktur:

```
data/  
├─ raw/
```

```
|— normalized/  
|— features/  
|— factors/  
|— metadata/
```

Dengan:

```
raw/  
  BTCUSDT/  
  ETHUSDT/  
  SOLUSDT/  
  ...  
  
factors/  
  DXY/  
  GOLD/  
  SPX/  
  VIX/
```

L.9. Raw data harus immutable

Data mentah jangan pernah diedit.

Misalnya:

```
raw/BTCUSDT/2026-08-17.parquet
```

tidak boleh ditimpa setelah cleaning.

Pipeline:

bukan:

L.10. Mengapa?

Jika suatu hari hasil backtest berubah, kita harus dapat mengetahui:

□

Ini adalah prinsip reproducibility.

L.11. Schema OHLCV

Setiap bar:

$$t = t, , , ,$$

dengan:

- t : timestamp,
- : open,
- : high,
- : low,
- : close,
- : volume.

Untuk sistem kita tambahkan:

$$P$$

jika tersedia.

L.12. Validasi OHLC

Harus selalu memenuhi:

$$t \ t, t$$

dan:

$$t \ t, t$$

Selain itu:

$$t \ t$$

Jika tidak:

$$tt$$

L.13. Volume

Harus:

$$t \ 0$$

Jika:

$$t \ 0,$$

bar invalid.

Tetapi:

$$t = 0$$

tidak otomatis berarti corrupt.

Untuk instrumen tertentu, volume dapat memang tidak tersedia atau nol.

L.14. Duplicate timestamp

Untuk setiap asset:

$$\boxed{t \ t}$$

harus berlaku.

Jika terdapat duplicate:

$$t = t,$$

kita harus mempunyai aturan deterministic untuk memilih record yang benar.

Jangan menggunakan:

```
drop_duplicates()
```

tanpa memahami penyebab duplikat.

L.15. Missing candle

Misalkan canonical grid memiliki:

$$t_1, t_2, t, t_4, t_5$$

Tetapi data:

$$t_1, t_2, t_4, t_5$$

Maka:

$$t$$

missing.

L.16. Jangan sembarangan forward-fill harga

Untuk market data:

$$P_t \quad P_{t_2}$$

hanya karena data t hilang.

Forward filling dapat menciptakan:

$$\boxed{tt}$$

Ini berbahaya untuk:

L.17. Bedakan dua jenis missing data

Type A

Market memang tutup.

Contoh:

$$P$$

pada Sabtu.

Type B

Data seharusnya ada tetapi hilang.

Contoh:

pada satu jam tertentu.

Kedua kondisi ini **tidak boleh diperlakukan sama**.

L.18. Crypto

Untuk crypto:

$$\boxed{tt}$$

kecuali exchange memang tidak menghasilkan candle.

Kita perlu flag:

$$,t \in \{0, 1\}$$

L.19. Stock market

Untuk saham:

$$r_{i,t} = 0$$

pada periode ketika market memang tutup.

Tetapi kita tidak boleh menyimpulkan:

$$r_{i,t} = 0$$

Tidak ada return hourly jika market tidak trading.

L.20. Calendar-aware data model

Setiap asset harus memiliki:

$$\mathbb{I}_t$$

yang memberi tahu apakah instrumen tersebut aktif pada t .

Kemudian:

$$r_{i,t} = \begin{cases} 1, & \text{if } \mathbb{I}_t = 1 \\ 0, & \text{if } \mathbb{I}_t = 0 \end{cases}$$

L.21. Masalah penting pada factor data

Misalnya crypto trading:

$$r_{i,t} = 0$$

Tetapi:

$$P$$

tidak memiliki observasi baru.

Kita **boleh menggunakan observasi SPX terakhir yang diketahui** sebagai state factor, tetapi harus memahami maknanya.

Kita tidak boleh menganggap:

$$P_t$$

benar-benar mempunyai return nol.

L.22. Factor state versus factor return

Ini perbedaan penting.

Misalnya harga terakhir SPX:

$$P_{P,t_0}$$

Pada saat market tutup:

$$P_{P,t} = P_{P,t_0}$$

sebagai **last observed price**.

Tetapi:

$$P_{,t} = 0$$

tidak boleh otomatis dianggap sebagai actual market return.

Untuk forecasting crypto, kita dapat menggunakan:

$$\boxed{t_{t,t}}$$

dengan flag:

$$t,t$$

L.23. Factor age

Definisikan:

$$\boxed{_{,t} = t - t_t}$$

Misalnya:

$$P_{,t} = 1$$

Artinya informasi SPX terakhir sudah berumur 17 jam.

Ini dapat dimasukkan ke feature model.

L.24. Dengan demikian factor feature menjadi

$$\boxed{t,t = , , ,}$$

Jadi model mengetahui:

"Saya sedang menggunakan data SPX yang berumur beberapa jam."

L.25. DXY dan Gold

Kita akan perlakukan:

dan:

sebagai **macro factor**, bukan sekadar asset tambahan.

Secara konseptual:

$$t =$$

L.26. Tanda factor exposure

Perhatikan:

DXY

Jika:

$$0,$$

kenaikan USD berkorelasi dengan penurunan asset.

Gold

Jika:

$$0,$$

asset cenderung bergerak searah dengan gold.

Tetapi kita **tidak akan mengasumsikan tanda ini sejak awal**.

Model akan mengestimasi.

L.27. Factor return

Gunakan:

$$_{,t} = P_{,t}P_{,t-1}$$

Tetapi hanya jika kedua timestamp memang valid:

$$t - 1, t \in$$

L.28. Cross-sectional alignment

Setelah semua data tersedia:

$$t = ,t, ,t, ,t, , ,t, ,t, P,t, ,t$$

Namun bukan berarti semua nilai harus berasal dari candle yang identik.

Kita menggunakan:

$$t_t \ t$$

L.29. Last-known-value operator

Definisikan:

$$P_{,t} = P_{,\tau}$$

dengan:

$$\tau = \{ \tau \ t \ P_{,\tau} \}$$

Kemudian:

$$_{,t} = t - \tau$$

L.30. Batas stale data

Kita tidak boleh menggunakan last-known value selamanya.

Definisikan:

$$_{,t}$$

Jika:

$$_{,t} \ ,$$

factor dianggap:

$$t$$

Misalnya untuk V1:

$$= 4$$

L.31. Jadi setiap feature memiliki status

Misalnya:

BTC return	valid
ETH return	valid
DXY return	valid
Gold return	valid
SPX return	stale
VIX return	stale

Model tidak boleh kehilangan informasi mengenai status tersebut.

L.32. Asset universe versus factor universe

Kita pisahkan:

$$=$$

dan:

$$=$$

Contoh:

$$= \{, , , P, \}$$

Sedangkan:

$$= \{, , , P, \}$$

Overlap diperbolehkan.

L.33. Mengapa pemisahan ini penting?

Karena suatu factor tidak harus ditradingkan.

Misalnya:

dapat memberikan informasi:

tanpa pernah masuk ke portfolio.

L.34. Corporate actions pada saham

Ketika kita pindah ke saham internasional, ada masalah:

- stock split,
- reverse split,
- dividend,
- spin-off.

Kita harus menentukan apakah model menggunakan:

$$P$$

atau:

$$tP$$

L.35. Untuk return model

Saya lebih memilih:

$$t$$

untuk historical return estimation, karena kita ingin mengukur economic return yang comparable.

Tetapi untuk execution simulation:

$$tt$$

harus digunakan.

Jadi:

$$P \quad tP$$

dapat terjadi.

L.36. Ini menghasilkan dua price streams

Adjusted price

↓

return / covariance / factor model

Raw tradable price

↓
entry / SL / TP / execution

Ini sangat penting ketika nanti kita masuk ke saham.

L.37. Survivorship bias

Misalnya pada 2021 terdapat asset:

yang kemudian collapse/delist.

Jika kita hanya menggunakan asset yang masih hidup pada 2026:

hilang dari universe.

Akibatnya backtest menjadi terlalu optimistik.

L.38. Solusinya

Universe harus bersifat:

$$[t, t]$$

Artinya pada:

$$t$$

kita hanya menggunakan asset yang memang tersedia pada waktu t .

$$t$$

Bukan:

$$202$$

L.39. Crypto universe

Jangan langsung menggunakan:

| top 10 crypto saat ini.

Karena itu menciptakan survivorship bias.

Lebih baik:

$$t = t$$

L.40. Liquidity filter

Misalnya:

$$= 1_{t,t}$$

Kemudian:

Untuk crypto kita dapat menggunakan:

$$= P_t$$

L.41. Spread filter

Jika data tersedia:

$$= -$$

Kita dapat mengharuskan:

Untuk V1, spread dapat dimodelkan secara konservatif jika order book belum digunakan.

L.42. Data quality score

Saya menyarankan setiap asset memiliki:

$$t \in [0, 1]$$

Misalnya:

$$= 1 - 1 - 2t - t - 4t$$

Jika:

$$,$$

asset dikeluarkan dari universe.

L.43. Feature engineering

Setelah data bersih, kita mulai membangun:

$$t$$

Minimal:

$$_{,t} = \begin{matrix} 1 & 4 & 24 & 24 & 1 \\ , & , & , & , & , & , & , & , & , \end{matrix}$$

L.44. Momentum

$$_{,t} = P_{,t} P_{,t-}$$

Sehingga:

$$\begin{matrix} 1 & 4 & 24 \\ , & , & \end{matrix}$$

L.45. Volatility jangka pendek

$$_{24,t} = t_{,t-2}, , , t$$

L.46. Volatility jangka menengah

$$_{1,t} = t_{,t-1}, , , t$$

168 jam:

$$24$$

Ini memberikan proxy:

$$11$$

L.47. Volatility regime

Kita dapat membentuk:

$$_{,t} = 24_{,t} 1_{,t}$$

Jika:

$$1,$$

short-term volatility lebih tinggi daripada baseline.

L.48. Volume z-score

$$r_{i,t} = \alpha_i + \beta_i r_{m,t} + \epsilon_{i,t}$$

Lebih baik menggunakan log-volume:

$$r_{i,t} = \alpha_i + \beta_i \ln V_{i,t} + \epsilon_{i,t}$$

kemudian:

$$r_{i,t} = \alpha_i + \beta_i \ln V_{i,t} + \epsilon_{i,t}$$

L.49. Factor beta rolling

Untuk factor :

$$r_{i,t} = \alpha_i + \beta_i r_{f,t} + \epsilon_{i,t}$$

Estimasi dilakukan pada rolling window:

L.50. Residual return

Setelah factor model:

$$r_{i,t} = \alpha_i + \beta_i r_{f,t} + \epsilon_{i,t}$$

Kita dapat menghitung:

$$r_{i,t} = \alpha_i + \beta_i r_{f,t} + \epsilon_{i,t}$$

Residual ini sangat menarik.

Karena:

merepresentasikan pergerakan asset yang **tidak dijelaskan oleh common factors**.

L.51. Ini memberi kita dua jenis signal

Factor-driven

$$r_{i,t}$$

Idiosyncratic

Sehingga:

$$= t t$$

L.52. Ini jauh lebih kuat daripada hanya momentum

Karena model dapat mengatakan:

| BTC naik bukan berarti semua altcoin harus dibeli.

Ia dapat melihat:

,

dan:

Misalnya:

tetapi:

0

Maka SOL mungkin tetap tidak menarik.

L.53. Residual correlation

Kita juga dapat menghitung:

$$=$$

Kemudian portfolio risk dapat dipisahkan menjadi:

$$=$$

Ini adalah decomposition yang sangat berguna.

L.54. Interpretasi portfolio risk

$$Pt = t t$$

Secara lebih lengkap:

=

Ini nantinya dapat dimasukkan langsung ke:

L.55. Ini menghubungkan Tahap L dengan Tahap M

Tahap L menghasilkan:

$$t, t, t, t$$

Tahap M akan menggunakan semuanya untuk:

$$t$$

L.56. Data matrix

Pada waktu t , kita akhirnya mempunyai:

$$t \in$$

Misalnya:

$$=, = 20$$

Maka:

$$t \in^{20}$$

L.57. Factor matrix

Jika kita mempunyai = factor:

$$t \in^{20}$$

Kemudian:

$$t = t t$$

L.58. Estimasi

Secara OLS:

$$=^{-1}$$

jika matrix invertible.

Dengan intercept, kita augment:

$$= 1$$

Kemudian:

$$= -1$$

L.59. Tetapi kita sebaiknya menggunakan regularisasi

Karena factor dapat berkorelasi:

$$P$$

Maka:

dapat ill-conditioned.

Gunakan ridge:

$$= -1$$

L.60. Feature normalization

Untuk forecasting, feature harus dinormalisasi menggunakan **training window saja**.

$$,t = ,t - tt$$

Jangan menggunakan:

yang mencakup future data.

L.61. Pipeline anti-leakage

Secara tepat:

```
RAW DATA
  ↓
CLEAN
  ↓
TIME ALIGN
  ↓
ROLLING WINDOW ENDING AT t
  ↓
FIT SCALER USING [t-W,t]
  ↓
```

```
FIT MODEL USING [t-W,t]
↓
GENERATE  $\mu(t+1)$ 
↓
OPTIMIZE
↓
TRADE t+1
```

Bukan:

```
ALL DATA
↓
NORMALIZE
↓
TRAIN
↓
BACKTEST
```

L.62. Mengapa urutan ini sangat penting?

Karena misalnya:

202

mempengaruhi:

2021

Itu berarti model secara tidak langsung mengetahui masa depan.

Walaupun target tidak secara eksplisit diberikan, leakage tetap terjadi.

L.63. Feature store

Saya menyarankan menyimpan hasil feature:

```
features/
  2026-01.parquet
  2026-02.parquet
  ...
```

dengan schema:

```
timestamp
asset
```



```
return_1h
return_4h
return_24h
vol_24h
vol_168h
volume_z
beta_btc
beta_eth
beta_dxy
beta_gold
factor_age_dxy
factor_age_gold
regime
data_quality
```

L.64. State vector

Semua informasi yang diperlukan optimizer pada waktu t :

$$t = t, t, t, t, t_{t, t-1}, P t_t$$

Ini adalah **market state** kita.

L.65. Kemudian optimizer menerima

$$t_t \quad t$$

Dengan kata lain, QAOA **tidak membaca data mentah**.

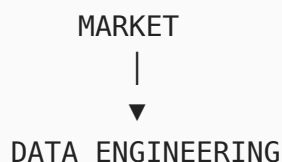
QAOA menerima:

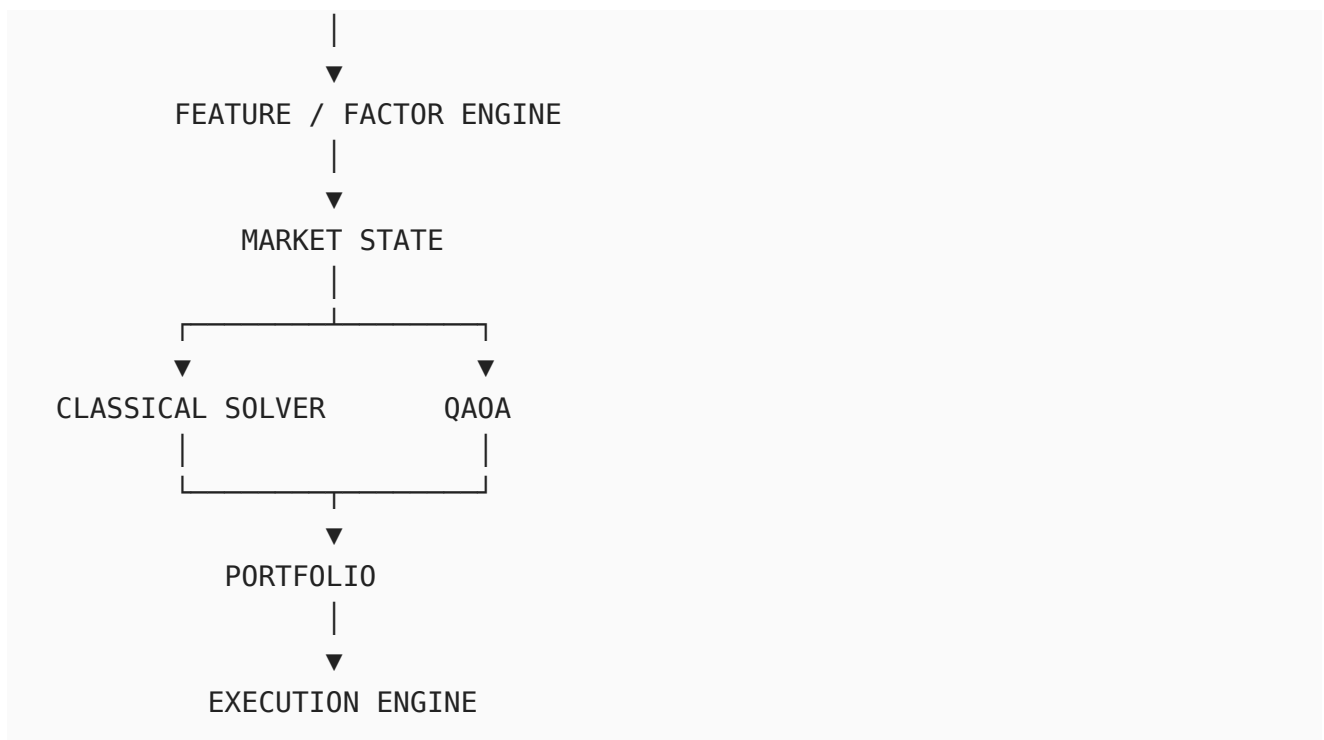
$$t t t t$$

dari kondisi pasar.

Ini arsitektur yang jauh lebih bersih.

L.66. Data layer dan quantum layer menjadi terpisah





Ini memungkinkan kita menguji:

dengan:

dan:

t

Itu sangat penting untuk penelitian yang fair.

L.67. Cross-market extension

Sekarang kita kembali ke tujuan besarmu: **tidak berhenti di crypto**.

Kita definisikan factor architecture:

$$t = t, t, , t, tt$$

Contohnya:

$$t = , , P, , ,$$

Kemudian universe dapat berubah menjadi:

$$= \{P, , , , ,\}$$

Model yang sama masih berlaku.

L.68. Yang berubah hanya data adapter

Crypto:

```
CryptoDataAdapter
```

Saham:

```
EquityDataAdapter
```

Gold:

```
CommodityDataAdapter
```

DXY:

```
MacroDataAdapter
```

Tetapi output semuanya harus sama:

<i>tt</i>

Ini prinsip **interface normalization**.

L.69. Contoh canonical schema

```
timestamp  
symbol  
asset_class  
open  
high  
low  
close  
volume  
currency  
market_open  
data_age  
data_quality
```

Kemudian downstream tidak perlu tahu apakah:

atau:

L.70. Multi-currency

Ini menjadi sangat penting ketika masuk saham internasional.

Misalnya:

$$P$$

dalam USD.

Tetapi jika nanti kita membeli saham Jepang:

$$20$$

dalam JPY.

Portfolio base currency kita misalnya:

Maka return harus mempertimbangkan FX.

L.71. Local return versus base-currency return

Jika:

$$P_{Pt}$$

dan:

$$_t = P,$$

maka:

$$P_t = P_{Pt}$$

Sehingga return investor USD menjadi:

$$\boxed{= -}$$

secara log-return approximation/exact sesuai definisi FX.

Jadi FX juga akhirnya menjadi factor.

L.72. Ini berarti arsitektur kita benar-benar global

Nanti:

$$t$$

dapat mencakup:

$$\{,,,P,,P,,\}$$

Tetapi jangan masukkan semuanya sekaligus.

Kita akan menambah factor secara bertahap melalui ablation test.

L.73. Data lineage

Setiap angka yang masuk ke QUBO idealnya dapat dilacak:

$$,t$$

harus dapat ditelusuri ke:

$$t$$

dan:

$$t,$$

yang dapat ditelusuri ke:

$$t-t,$$

yang dapat ditelusuri ke:

$$tt$$

Sehingga:

$$, \quad t \quad t \quad t$$

L.74. Ini sangat berguna ketika hasil aneh

Misalnya QAOA tiba-tiba memilih:

$$= 50$$

Kita dapat menelusuri:

$$, \quad t$$

Kita tidak mempunyai black box.

L.75. Checkpoint yang wajib ada

Sebelum masuk Tahap M, saya ingin pipeline kita dapat melewati **7 checkpoint**.

Checkpoint 1

Semua timestamp:

Checkpoint 2

Tidak ada duplicate candle.

Checkpoint 3

OHLC valid.

Checkpoint 4

Missing data teridentifikasi.

Checkpoint 5

Market calendar benar.

Checkpoint 6

Tidak ada feature menggunakan future information.

Checkpoint 7

Rolling features dapat direproduksi.

L.76. Unit test matematis

Kita bahkan dapat membuat test:

Return

Untuk:

$$P = 100,101$$

harus:

$$= 101100$$

Volatility

Bandingkan dengan manual calculation.

Beta

Untuk synthetic data dengan:

$$= 2,$$

model harus menghasilkan:

$$2$$

Factor decomposition

Jika:

$$= ,$$

maka residual harus memenuhi kira-kira:

$$, 0$$

L.77. Synthetic data test

Sebelum menggunakan data pasar nyata, buat:

$$=$$

dengan parameter yang kita ketahui.

Misalnya:

$$= 12-105$$

Kemudian lihat apakah engine dapat merecovery:

Ini jauh lebih mudah untuk debugging.

L.78. Setelah data engine benar

Barulah kita dapat mengatakan:

$$\boxed{t}$$

Dan dari sini kita masuk ke:

Tahap M — Forecasting & Factor Engine

Di Tahap M kita akan benar-benar menurunkan:

$$t$$

dari factor model + momentum + volatility + residual dynamics.

Kita juga akan membangun:

$$t = t, t_t, t$$

secara matematis.

Kemudian kita dapat menjawab pertanyaan yang lebih menarik:

Jika BTC naik, DXY turun, Gold naik, VIX naik, dan residual SOL negatif, secara matematis berapa expected return SOL untuk satu jam berikutnya, berapa covariance antar aset, dan bagaimana informasi tersebut akhirnya mengubah koefisien dalam QUBO?

Itulah jembatan langsung dari **data pasar** → **matematika finansial** → **QUBO** → **Hamiltonian Ising** → **QAOA** yang akan kita bangun pada tahap berikutnya.

Prompt:

8/17/2026, 9:57:44 PM

lanjut Tahap M

Response:

8/17/2026, 9:57:44 PM

Remembering

Continued Tahap M design by connecting forecasting inputs and clarifying QUBO transition

Worked for 7s

Tahap M — Forecasting & Factor Engine

Kita sekarang masuk ke salah satu bagian **paling penting secara matematis** dari sistem kita.

Pada Tahap L kita mengubah data mentah menjadi market state:

$$t = t, t, t, t, t, t, t-1$$

Tahap M menjawab pertanyaan:

Dari state pasar t , berapa expected return setiap aset pada horizon trading berikutnya, dan bagaimana ketidakpastian serta hubungan antar-aset berubah?

Output utama kita adalah:

$$t$$

dan

$$t$$

Keduanya nantinya menjadi bahan baku langsung untuk optimasi portfolio/QUBO.

M.1. Arsitektur Tahap M

Pipeline kita:

$$t \quad t$$

dan secara paralel:

$$t \quad t, t \quad t$$

sehingga:

$$t, t \quad P_{t,t}$$

M.2. Jangan langsung memprediksi harga

Ini keputusan desain penting.

Kita **tidak** ingin:

$$P_{t+1} = P_t$$

Lebih masuk akal menggunakan return:

$$r_{t+1} = P_{t+1}P_t$$

Kemudian:

$$r_{t+1} = r_{t+1} \quad t$$

Alasannya sederhana:

harga bersifat non-stationary, sedangkan return relatif lebih cocok untuk model statistik.

M.3. Horizon trading

Karena sistem kita ditujukan untuk short-term trading, kita perlu menentukan:

$$\Delta$$

Untuk V1 kita gunakan:

$$\Delta = 1$$

Maka:

$$P_{t+1} = P_t P_{t,t+1}$$

Kemudian:

$$P_{t,t+1} = P_{t,t+1}$$

M.4. Mengapa expected return saja tidak cukup?

Misalkan:

$$= 0.20$$

dan:

$$= 0.15$$

Secara naif kita memilih A.

Tetapi ternyata:

$$= 15$$

sedangkan:

$$= 0$$

Maka A memiliki return lebih tinggi tetapi risk yang jauh lebih besar.

Karena itu kita membutuhkan:

$$\Delta$$

bukan hanya:

M.5. Model factor dasar

Misalkan terdapat asset dan factor.

Kita definisikan:

$$t = t \text{ \# } t$$

Dengan:

$$t \in$$

dan:

$$t \in$$

M.6. Contoh factor kita

Misalnya:

$$t = ,t,t,t,tP,t,t$$

Untuk saham internasional nanti factor tersebut dapat diperluas.

Tetapi prinsipnya tetap sama.

M.7. Matrix exposure

Untuk asset :

$$= ,,,,P,$$

Seluruh asset:

$$=_{12}$$

Sehingga:

$$\in$$

M.8. Forecast factor

Sekarang kita tidak hanya ingin mengetahui:

$$t$$

Kita ingin:

$$t_1 = t_1 \quad t$$

Ini sangat penting.

Misalnya sistem mendeteksi:

$$,t_1 \quad 0,$$

tetapi:

$$,t_1 \quad 0$$

Keduanya dapat memberikan pengaruh berbeda terhadap asset yang sama.

M.9. Forecast asset dari factor

Dari:

$$,t_1 = t_1 \quad ,t_1,$$

ambil conditional expectation:

$$,t_1 \quad t$$

Maka:

$$,t = t_1 \quad t \quad ,t_1 \quad t$$

Sehingga:

$$,t = ,t \quad ,t \quad t_1 \quad ,t_1$$

Ini adalah persamaan inti Factor Engine kita.

M.10. Kita pecah expected return menjadi tiga bagian

$$=$$

Interpretasinya:

1. Baseline

2. Market/factor-driven component

3. Asset-specific component

M.11. Mengapa residual prediction penting?

Bayangkan:

Sebuah altcoin memiliki:

$$= 15$$

Secara factor model:

$$_t = 15$$

Tetapi ternyata asset tersebut sedang mengalami:

$$_t = 0$$

Maka forecast sebenarnya:

$$= 15$$

Ini jauh lebih informatif daripada sekadar:

| BTC naik → beli altcoin.

M.12. Residual bukan noise yang harus dibuang

Pada model sederhana:

$$= \epsilon_t$$

orang sering menganggap:

sebagai noise.

Tetapi untuk trading:

$$\boxed{\epsilon_t}$$

dapat mengandung alpha.

Misalnya residual mengalami mean reversion:

$$r_{t+1} = \alpha + \beta r_t$$

Jika:

$$\alpha = -1, \beta = 0,$$

residual memiliki kecenderungan reversal.

M.13. Model residual AR(1)

Kita dapat menggunakan:

$$r_{t+1} = \alpha + \beta r_t + \epsilon_{t+1}$$

Maka:

$$r_{t+1} - \alpha = \beta (r_t - \alpha) + \epsilon_{t+1}$$

Jadi:

$$r_{t+1} - \alpha = \beta (r_t - \alpha) + \epsilon_{t+1}$$

M.14. Jika $\alpha = 0$

Misalkan:

$$\beta = -0.4$$

dan:

$$\epsilon_t = -1$$

Maka:

$$r_{t+1} = -0.4 - 1 = -0.4$$

Artinya asset sedang mengalami underperformance terhadap factor model dan model memperkirakan sebagian reversal.

M.15. Jika $\alpha = 0$

Misalkan:

$$\beta = 0.4$$

Jika:

$$t = 1,$$

maka:

$$t_1 = 04$$

Ini lebih menyerupai momentum.

Jadi:

□

membantu menentukan karakteristik residual.

M.16. Tetapi jangan langsung percaya residual

Kita harus menguji apakah:

stabil.

Rolling estimation:

$$t = t - t$$

Jika:

t

berubah-ubah drastis, residual model tidak stabil.

Maka confidence terhadap signal harus turun.

M.17. Confidence-adjusted expected return

Ini ide penting untuk sistem kita.

Jangan hanya menggunakan:

Tambahkan:

$$\in 0,1$$

sebagai confidence.

Kemudian:

$$=$$

Jika model sangat yakin:

1

Jika tidak yakin:

0

M.18. Apa yang menentukan confidence?

Kita dapat membangun:

$$= {}^2, ttt, , tt, tt$$

Misalnya:

$$= t$$

M.19. Jangan membuat confidence secara arbitrer

Untuk V1, lebih baik kita membuatnya sebagai fungsi yang sederhana dan dapat diuji.

Misalnya:

$$= , 0, 1$$

dengan Information Coefficient:

$$= t, t1$$

M.20. Information coefficient

Jika:

$$0,$$

forecast cenderung benar secara arah.

Jika:

$$0,$$

forecast tidak berguna.

Jika:

$$0,$$

model bahkan berlawanan arah.

Namun:

harus dihitung **out-of-sample**.

M.21. Forecast factor dengan model sederhana terlebih dahulu

Untuk V1 kita tidak membutuhkan deep learning.

Kita dapat mulai dengan:

$$t_1 = t t$$

Ini adalah VAR(1).

M.22. VAR(1)

Dengan:

$$t_1 = t t_1$$

Maka:

$$t_1 = t$$

M.23. Mengapa VAR menarik?

Karena factor kita saling berhubungan.

Misalnya:

dan:

$$P$$

VAR menangkap cross-factor dynamics:

$$0$$

M.24. Contoh

Misalnya:

$$_{,t1} = 02_{,t} - 01_{,t}$$

dan:

$$r_{t,1} = 0.5r_{t-1} + 0.1r_{t-2}$$

Maka sistem tidak hanya bertanya:

| bagaimana BTC bergerak?

tetapi:

| bagaimana kondisi seluruh factor memengaruhi perkembangan factor berikutnya?

M.25. Namun VAR memiliki masalah

Jika:

$$=$$

dan kita menggunakan lag:

$$= 10,$$

jumlah parameter cepat meningkat.

Dengan:

$$2$$

parameter.

Karena itu kita dapat menggunakan ridge:

$$= \{t - t_2 - 2\}$$

M.26. Forecast factor uncertainty

Forecast tidak hanya:

Kita juga membutuhkan:

$$\square$$

Misalkan:

$$r_{t,1} = r_{t-1}$$

dengan:

$$=$$

Untuk one-step forecast:

$$=$$

M.27. Asset covariance

Kembali ke:

$$t = t t$$

Jika:

$$t =$$

dan:

$$t = ,$$

serta factor dan residual dianggap tidak berkorelasi:

$$, = 0,$$

maka:

$$=$$

Ini adalah persamaan inti risk engine.

M.28. Penurunan matematis

Mulai:

$$=$$

Covariance:

$$= - -$$

Karena:

$$- = - - ,$$

maka:

$$= , ,$$

Jika:

$$, = 0,$$

maka:

$$=$$

M.29. Mengapa ini lebih baik daripada covariance langsung?

Covariance sample biasa:

$$= 1 - 1$$

Masalahnya:

- noisy,
- unstable,
- sensitive terhadap regime,
- buruk jika σ besar dan σ relatif kecil.

Factor decomposition memberikan struktur:

$$\begin{bmatrix} \sigma & \sigma \\ \sigma & \sigma \end{bmatrix}$$

M.30. Factor risk

$$\begin{bmatrix} \sigma & \sigma \\ \sigma & \sigma \end{bmatrix}$$

Jika dua asset mempunyai:

$$\sigma, 0$$

dan:

$$\sigma, 0,$$

mereka dapat memiliki covariance tinggi meskipun tidak selalu terlihat dari short sample.

M.31. Idiosyncratic risk

$$\sigma$$

Pada model paling sederhana:

$$= \sigma_1, \sigma_2,$$

Tetapi ini terlalu sederhana.

M.32. Residual correlation

Kita dapat mengestimasi:

,

Jika signifikan, maka:

tidak diagonal.

Ini memungkinkan kita menemukan hubungan yang tidak dijelaskan factor global.

M.33. Shrinkage

Sample covariance residual dapat noisy.

Gunakan:

$$= 1 -$$

Misalnya:

=

M.34. Dynamic covariance

Pasar short-term berubah.

Maka:

$t \quad t-1$

Kita membutuhkan dynamic estimation.

Salah satu baseline yang sangat baik:

$$_t = {}_{t-1} 1 - {}_{t-1} {}_{t-1}$$

M.35. Factor covariance EWMA

Kita dapat menerapkan hal yang sama pada factor:

$$_{,t} = {}_{,t-1} 1 - {}_{t-1} {}_{t-1}$$

Dengan demikian covariance lebih responsif terhadap perubahan regime.

M.36. Volatility regime

Misalnya:

$$tt$$

dibandingkan:

$$t$$

Definisikan:

$$t = tt$$

Jika:

$$t \geq 1,$$

kita berada pada high-volatility regime.

M.37. Mengapa regime penting untuk ?

Signal momentum yang bekerja pada:

$$tt$$

belum tentu bekerja pada:

$$tt$$

Jadi:

$$\boxed{t = t}$$

bukan konstanta.

M.38. Regime-dependent forecast

Secara sederhana:

$$t = \begin{cases} t, & = t, & = \end{cases}$$

Tetapi kita tidak perlu langsung membuat dua model terpisah.

Kita dapat memasukkan regime sebagai feature:

$$t = t, tt, t,$$

M.39. Mean model

Sekarang kita dapat membangun model:

$$\hat{y}_t = \frac{1}{n} \sum_{i=1}^n y_i$$

Ini menjadi baseline forecasting engine kita.

M.40. Tetapi hati-hati: jangan terlalu banyak parameter

Kita tidak ingin membuat:

$$\hat{y}_t = 1000t$$

dengan dataset terbatas.

Karena:

$$t \neq t$$

akan menjadi musuh utama.

V1 harus **sederhana tetapi mathematically defensible**.

M.41. Maka arsitektur V1 saya tetapkan

Saya sarankan:

$$\hat{y}_t = \frac{1}{n} \sum_{i=1}^n y_i$$

dengan:

$$t = 1, 2, \dots, n$$

dan:

$$t = 1, 2, \dots, n$$

M.42. Secara eksplisit

Untuk asset :

$$\hat{y}_{t,i} = \frac{1}{n} \sum_{j=1}^n y_{t,i,j}$$

Ini sudah cukup kaya untuk V1.

M.43. Regularisasi

Parameter:

, ,

dapat diestimasi dengan ridge:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Solusinya:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

M.44. Mengapa ridge cocok?

Karena features kita dapat berkorelasi.

Contoh:

1

dan:

4

jelas tidak independen.

Begitu pula:

dan:

Ridge mengurangi coefficient explosion.

M.45. Standardisasi

Sebelum ridge:

harus dinormalisasi menggunakan training window.

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Kemudian model dilatih.

M.46. Forecast output

Pada setiap timestamp:

$$t$$

engine menghasilkan:

$$r_t = [r_{t,1}, r_{t,2}, \dots, r_{t,t}]$$

Contoh:

$$t = 00010000-000000021$$

Artinya:

$$01, 00, -00, 021$$

expected log-return satu jam ke depan.

M.47. Tetapi expected return harus dibandingkan dengan biaya

Ini sangat penting untuk trading.

Jika:

$$= 00$$

tetapi:

$$ttt = 010,$$

maka:

$$t \ 0$$

Karena itu kita definisikan:

$$t = -$$

M.48. Cost model

Minimal:

$$=$$

Sehingga:

$$= -$$

Ini lah expected return yang seharusnya masuk optimizer.

M.49. Market impact

Untuk order yang lebih besar:

$$t$$

meningkat terhadap ukuran order.

Model sederhana:

$$t =$$

Maka:

$$= t$$

Ini nantinya menjadi bagian penting dari optimasi position sizing.

M.50. Kita sekarang mempunyai problem optimasi yang lebih realistis

Bukan:

$$-$$

Tetapi:

$$- -$$

Di sinilah sistem kita mulai berbeda dari portfolio optimization akademik sederhana.

M.51. Hubungan dengan risk budget

Karena sebelumnya kita ingin setiap trade mengikuti:

1

risk budget, kita nantinya akan menghubungkan:

dengan:

Jika stop distance:

dan position size:

,

maka approximate dollar risk:

=

Constraint:

$\square$

Misalnya:

= 1

M.52. Expected return menjadi expected PnL

Jika:

unit asset dan:

expected return:

$P \quad P$

Sedangkan risk:

$P, \quad P$

Jadi:

$\square$

mulai terbentuk.

M.53. Sekarang covariance menjadi sangat penting

Untuk portfolio:

$$= 1,$$

expected return:

$$=$$

Risk:

$$=$$

M.54. Ini langsung menuju QUBO

Misalnya binary decision:

$$\in \{0, 1\}$$

Kita definisikan:

$$=$$

Objective:

$$- ,$$

Karena:

$$2 = ,$$

maka objective dapat ditulis sebagai:

$$-$$

Ini adalah QUBO.

M.55. Koefisien linear

$$= - 2$$

Jadi expected return forecast mempengaruhi:

$$\square$$

M.56. Koefisien interaksi

Untuk :

$$= 2$$

Jadi covariance forecast menentukan interaksi:

$$\square$$

Ini sangat penting:

Factor Engine bukan modul terpisah dari QAOA.

Outputnya secara langsung menentukan landscape energi yang akan dieksplorasi quantum optimizer.

M.57. Hubungan lengkap

Sekarang kita memiliki:

$$\square$$

$$\square$$

$$\square$$

$$\square$$

$$\square$$

$$\square$$

$$P_{tt}$$

M.58. Bahkan factor exposure dapat masuk ke constraint

Misalnya kita tidak ingin portfolio terlalu exposed terhadap BTC.

Portfolio beta:

$$=$$

Kita dapat memberikan constraint:

$$\square$$

Begitu juga:

$$, ,$$

Ini memungkinkan kita mengontrol **factor risk**, bukan hanya volatility.

M.59. Factor-neutral portfolio

Jika kita menginginkan:

$$0,$$

maka:

$$\boxed{0}$$

Ini menghasilkan portfolio yang relatif netral terhadap factor tertentu.

Contohnya:

$$, 0$$

Dengan demikian profit/loss lebih banyak berasal dari:

$$t$$

M.60. Tetapi untuk sistem trading kita

Kita tidak selalu ingin factor-neutral.

Karena jika model memperkirakan:

$$0,$$

kita mungkin **memang ingin mengambil BTC exposure**.

Jadi factor exposure sebaiknya menjadi:

$$\square$$

bukan selalu constraint nol.

M.61. Dynamic factor exposure

Kita dapat menentukan target:

berdasarkan regime.

Misalnya:

$$, = , ,$$

Kemudian optimizer mencari portfolio yang mendekati target tersebut.

Ini sudah jauh lebih advanced.

M.62. Tetapi belum kita masukkan ke V1

Untuk menjaga eksperimen tetap dapat dibuktikan, V1:

$$\boxed{,}$$

sedangkan:

$$t t t$$

kita masukkan setelah baseline tervalidasi.

Ini penting untuk menghindari sistem menjadi terlalu kompleks sebelum kita tahu apakah alpha dasarnya benar-benar ada.

M.63. Output final Tahap M

Pada setiap waktu t , Factor Engine harus menghasilkan object konseptual:

$$\boxed{t = \{t, t, t, t1, ,t, ,t, t, t\}}$$

Dengan:

Forecast

$$t$$

Risk

$$t$$

Factor exposure

$$t$$

Factor forecast

$$t1$$

Residual forecast

$$t$$

Trading cost

$$t$$

Confidence-adjusted alpha

t

M.64. Quality control Tahap M

Sebelum kita menggunakan output ini untuk trading, kita harus menguji:

1. Directional accuracy

$$P_t = t1$$

2. Information coefficient

$$= t, t1$$

3. Calibration

Apakah:

$$= 02$$

benar-benar menghasilkan realized return sekitar:

$$02$$

secara statistik?

4. Stability

Apakah parameter stabil terhadap waktu?

5. Turnover

Apakah signal berubah terlalu cepat?

M.65. Yang paling penting: forecast tidak dinilai dari satu trade

Misalnya model mengatakan:

$$= 015$$

Tetapi realized:

$$= -0$$

Itu tidak berarti model gagal.

Yang kita evaluasi adalah:

Jika secara agregat:

,

model memiliki predictive power.

M.66. Kita juga harus membandingkan dengan baseline

Model kita tidak boleh hanya dibandingkan dengan dirinya sendiri.

Minimal:

$$\boxed{1 = 0}$$

$$\boxed{2 \ t}$$

$$\boxed{t}$$

$$\boxed{4 \ t}$$

$$\boxed{5 \ t \ t}$$

Baru kita dapat mengetahui apakah kompleksitas tambahan memang menghasilkan alpha.

M.67. Ini juga penting untuk tesis quantum kita

Karena nanti kita ingin membandingkan:

t

vs.

Kita **tidak boleh** membiarkan perbedaan forecasting membuat perbandingan optimizer menjadi tidak fair.

Maka:

$$\boxed{tt}$$

harus diberikan kepada:

, t ,

Dengan demikian yang dibandingkan benar-benar:

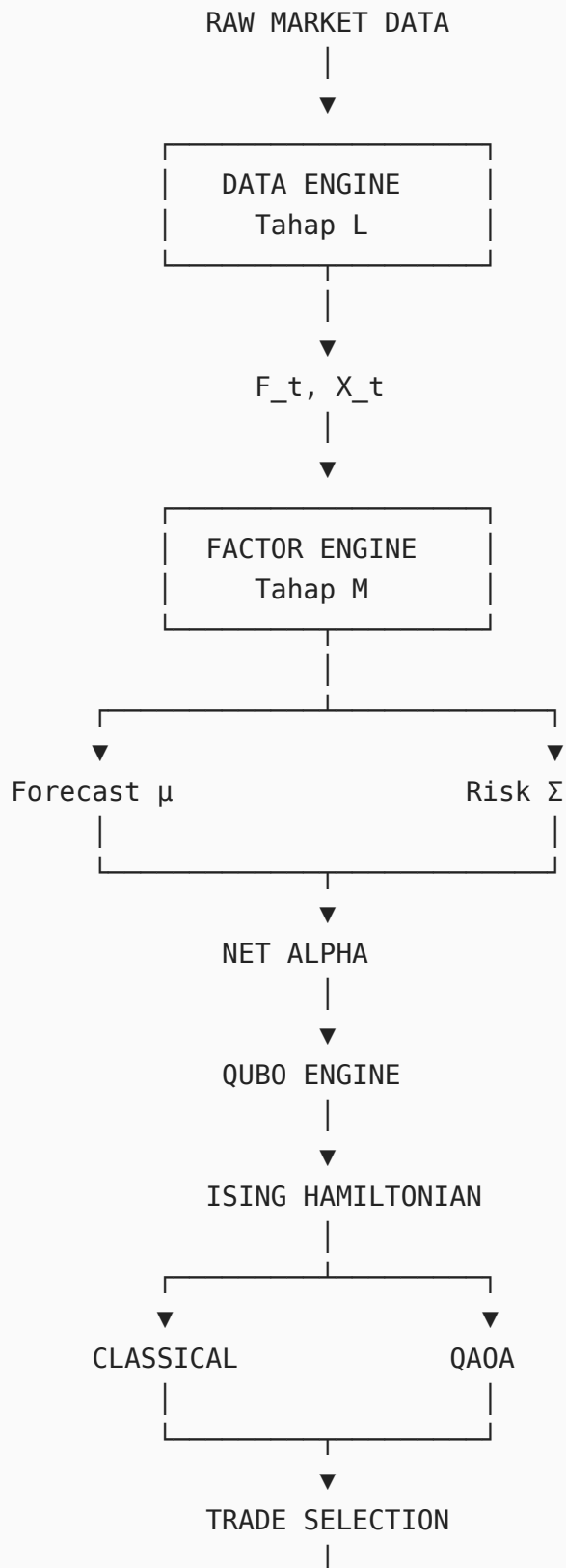
$$\boxed{ttt}$$

bukan:

ttt

M.68. Arsitektur lengkap sampai Tahap M

Sekarang sistem kita sudah menjadi:



M.69. Satu hal yang sangat penting

Sekarang kita dapat melihat bahwa **QAOA bukan predictor harga**.

QAOA tidak perlu mengetahui:

$$= 115000$$

Ia juga tidak perlu mengetahui:

$$=$$

Semua informasi tersebut sudah diproses menjadi:

$$\square$$

QAOA hanya menyelesaikan:

$$\square$$

kombinasi trade mana yang secara matematis paling optimal?

Ini adalah pemisahan yang sangat sehat antara:

$$P_t \quad t_t$$

M.70. Dan inilah inti sistem kita

Secara keseluruhan:

$$t \quad t \quad t, t \quad t \quad t$$

dengan:

$$t = t_1 \quad t$$

dan:

$$t = t, t_t, t$$

Kemudian:

$$t = t, t, t, t$$

dan akhirnya:

$$t = \begin{matrix} \in \{0,1\} \end{matrix} t$$

Itulah jembatan matematis yang kita cari sejak awal.

Apa yang berubah setelah Tahap M?

Sebelum Tahap M, kita hanya mempunyai:

Sekarang kita mempunyai **belief matematis tentang masa depan pasar**:

$$t$$

serta **geometri risiko pasar**:

$$t$$

Dan keduanya sekarang sudah siap diubah menjadi **masalah optimasi diskret**.

Tahap berikutnya yang paling natural adalah **Tahap N — Dynamic QUBO Construction & Position Sizing**. Di sana kita bisa menurunkan secara penuh, mulai dari

$$- ,$$

kemudian memasukkan **transaction cost, SL/TP, \$1 risk budget, liquidity, maximum concurrent positions, long/short, cardinality, dan rebalancing**, lalu secara sistematis mengubahnya menjadi

$$\square$$

dan akhirnya ke

$$=$$

Di titik itulah koneksi antara **quantitative trading yang realistis** dan **QAOA/Ising optimization** akan menjadi sangat konkret.

Prompt:

8/17/2026, 9:58:53 PM

lanjut Tahap N

Response:

8/17/2026, 9:58:54 PM

Remembering

Tahap N — Dynamic QUBO Construction & Position Sizing

Sekarang kita masuk ke tahap yang menurut saya **menjadi jantung matematis sistem trading kita**.

Pada Tahap M kita sudah memiliki:

$$t$$

sebagai expected return,

dan

$$t$$

sebagai covariance/risk structure.

Tetapi keduanya masih berada pada level **continuous portfolio theory**.

Trading nyata membutuhkan keputusan diskret:

aset mana yang dibeli, berapa besar posisinya, long atau short, pada harga berapa, dengan stop-loss berapa, dan apakah kombinasi seluruh posisi tersebut masih memenuhi risk budget?

Maka Tahap N bertugas melakukan transformasi:

$$t \rightarrow t$$

dan pada akhirnya menghasilkan:

$$t$$

yang dapat diberikan kepada execution engine.

N.1. Masalah sebenarnya yang ingin kita selesaikan

Misalkan terdapat aset.

Untuk setiap aset, kita mengetahui:

$$P$$

harga saat ini,

expected return,

2

variance,

dan:

covariance.

Kita ingin menentukan:

Secara klasik:



Tetapi ini belum cukup untuk trading kita.

N.2. Mengapa Markowitz biasa belum cukup?

Markowitz menganggap:

€

Padahal sistem kita memiliki keputusan seperti:

atau:

atau:

Selain itu kita memiliki:

- minimum order size,
- maximum position,
- \$1 risk budget,
- stop-loss,
- take-profit,
- transaction cost,
- slippage,
- liquidity,
- maximum number of simultaneous positions.

Jadi problem sebenarnya adalah:

N.3. Kita mulai dari satu trade

Sebelum membuat portfolio, mari kita pahami satu posisi.

Misalkan kita ingin long asset .

Harga:

$$P$$

Stop-loss:

Definisikan stop distance:

$$= P -$$

untuk long.

Jika kita membeli:

unit, maka approximate maximum loss:

$$=$$

Kita ingin:

Dengan risk budget:

$$= 1$$

Maka:

$$1$$

Sehingga:

$$1$$

Ini adalah dasar position sizing kita.

N.4. Position sizing bukan berdasarkan modal

Ini perbedaan penting.

Sistem kita tidak berkata:

Saya memiliki \$1000, maka saya membeli \$100 BTC.

Sebaliknya:

Berapa ukuran posisi yang menghasilkan maksimal \$1 loss jika SL tersentuh?

Secara sederhana:

$$=$$

Dengan:

$$= 1$$

N.5. Contoh

Misalkan:

$$P = 100$$

dan:

$$=$$

Maka:

$$= 2$$

Dengan:

$$= 1,$$

kita memperoleh:

$$= 12 = 05$$

Notional:

$$= P = 50$$

Jadi posisi \$50 memiliki stop-loss sekitar \$1.

N.6. Tetapi kita tidak ingin selalu kontinu

Quantum optimization paling natural ketika keputusan kita diskret.

Maka kita melakukan **discretization**.

Misalkan kita memiliki level position:

$$,1,2, \dots$$

Contohnya:

$$=$$

dengan:

$$,1 = 025,$$

$$,2 = 050,$$

$$, = 100$$

N.7. Binary encoding

Definisikan binary variables:

$$, \in \{0, 1\}$$

Dengan:

$$, = 1$$

berarti level dipilih.

Position:

$$\boxed{= , = 1''}$$

N.8. Tetapi kita harus mencegah memilih dua level sekaligus

Misalnya:

$$,1 = 1$$

dan:

$$, = 1$$

Maka kita mendapatkan:

$$= ,1 ,,$$

yang tidak kita inginkan jika level tersebut mutually exclusive.

Constraint:

$$\boxed{, 1}$$

N.9. Mengubah constraint menjadi penalty

Karena QUBO membutuhkan objective tanpa constraint eksplisit, kita masukkan penalty:

$$P, -1^2$$

Tetapi jika zero position diperbolehkan, kita sebenarnya membutuhkan:

$$, 1$$

Untuk binary variable, bentuk penalty yang lebih tepat dapat menggunakan:

$$\boxed{P,,}$$

Jika dua level dipilih sekaligus, penalty muncul.

N.10. Sekarang kita masukkan expected return

Untuk position:

$$,$$

expected PnL:

$$P$$

Definisikan:

$$, = P,$$

Dengan:

$$= -$$

Maka contribution:

$$\boxed{t = ,,,}$$

N.11. Mengapa kita menggunakan net alpha?

Karena expected return mentah:

belum mempertimbangkan:

$$t$$

Maka:

$$\boxed{= -}$$

Kalau:

$$0,$$

trade tersebut secara expected value sudah buruk bahkan sebelum risk diperhitungkan.

N.12. Sekarang masukkan portfolio risk

Position vector dalam notional:

$$= P$$

Maka:

adalah portfolio variance.

Karena:

$$= P \quad , ,$$

kita memperoleh:

$$\boxed{=}$$

N.13. Ekspansi binary

Substitusi:

$$= P \quad , ,$$

Maka:

$$= , PP \quad , ,$$

Sehingga:

$$\boxed{= , , PP \quad , ,}$$

Inilah yang membuat masalah kita menjadi QUBO.

N.14. Objective utama

Kita ingin:

$$tP - Pt - t - ttPt$$

Secara matematis:

$$- - P$$

N.15. Karena QUBO biasanya ditulis sebagai minimization

Kita ubah tanda:

$$- P$$

Definisikan:

$$= - P$$

Maka:

$$=$$

N.16. Dari sini QUBO muncul secara alami

Karena:

$$2 =$$

untuk:

$$\in \{0, 1\},$$

semua objective dapat ditulis:

$$=$$

atau:

$$=$$

N.17. Index sekarang berarti apa?

Kita gabungkan:

$$= ,$$

Artinya satu binary variable mewakili:

| memilih asset pada position level .

Misalnya:

,1

berarti:

| pilih BTC dengan position level 1.

N.18. Struktur QUBO kita

Secara konseptual:

$$= \begin{matrix} t & t & tt \end{matrix}$$

Ini sangat berguna karena kita dapat menganalisis setiap komponen secara terpisah.

N.19. Return term

Untuk variable ,:

$$t, = -P,$$

Semakin besar expected profit:

,

semakin negatif diagonal QUBO.

Karena optimizer ingin:

N.20. Risk diagonal

Variance asset:

Contribution:

$$P,$$

Sehingga:

$$= P,$$

N.21. Risk interaction

Untuk dua posisi:

,

dan:

, ,

interaction:

$PP_{,,}$

Interpretasinya sangat menarik.

Jika:

0,

memilih kedua posisi bersama menghasilkan penalty.

Jika:

0,

kombinasi tersebut dapat menurunkan portfolio risk.

N.22. Ini sebenarnya inti "quantitative trading"

Misalnya:

memiliki expected return tinggi.

Tetapi:

sangat berkorelasi dengan:

Optimizer dapat berkata:

| A bagus.

Tetapi:

| A + B terlalu redundant.

Sebaliknya:

mungkin lebih baik karena:

, 0

Jadi optimizer tidak hanya mencari:

Ia mencari:

□

N.23. Maximum number of positions

Kita mungkin tidak ingin membuka 20 trade sekaligus.

Definisikan:

= ,

Karena:

$\in \{0, 1\}$

jika level position dibuat mutually exclusive.

Constraint:

□

N.24. Penalty cardinality

Kita dapat menggunakan:

$P - tt^2$

Tetapi ini memaksa tepat tt .

Jika yang kita inginkan adalah **maksimal**, penalty dapat dibuat dengan auxiliary variables atau formulasi inequality-to-equality.

Untuk V1, saya menyarankan:

□

ditangani melalui slack variable.

N.25. Slack variable

Misalnya:

$$= ,$$

dengan:

$$0$$

Kemudian:

direpresentasikan dalam binary encoding:

$$= 2$$

Constraint menjadi:

$$P \quad 2 -$$

Ini kemudian masuk QUBO.

N.26. Risk budget portfolio

Sekarang masuk constraint yang lebih penting.

Kita ingin total worst-case stop-loss risk:

$$t$$

Dengan:

$$=$$

Maka:

$$\boxed{,, t}$$

N.27. Contoh

Jika:

$$t = 5,$$

dan:

$$= 1$$

untuk setiap posisi, maka maksimal lima posisi secara kasar dapat dibuka.

Tetapi optimizer juga melihat covariance.

Jadi mungkin ia memilih:

posisi saja jika ketiganya sangat correlated.

N.28. Ini penting

Risk budget dan variance risk **bukan hal yang sama.**

Risk budget:

Variance:

Kita membutuhkan keduanya.

Risk budget menjawab:

| Berapa banyak uang yang bisa hilang jika SL tersentuh?

Variance menjawab:

| Seberapa besar ketidakpastian portfolio secara statistik?

N.29. Position sizing yang lebih realistis

Sebelumnya:

$$= 1$$

Tetapi volatility juga harus dipertimbangkan.

Misalkan:

besar.

Maka position yang secara SL-risk aman belum tentu secara portfolio volatility aman.

Jadi kita memiliki dua filter:

$$t$$

dan:

$$ttt$$

N.30. Volatility-adjusted risk

Kita dapat mendefinisikan target risk:

$$tt =$$

Kemudian:

$$tt \quad tt,$$

Ini bisa menjadi constraint tambahan.

N.31. Tetapi kita harus hati-hati dengan double counting

Jika:

sudah masuk objective:

,

kita tidak perlu memberikan penalty volatility individual terlalu besar lagi.

Kalau tidak:

akan dihukum dua kali.

Karena itu:

SL risk digunakan sebagai hard/soft constraint, sedangkan covariance digunakan sebagai portfolio risk objective.

Ini desain yang lebih bersih.

N.32. Long dan short

Sekarang sistem kita harus mampu:

dan:

t

Kita tidak bisa menggunakan satu binary variable lagi.

Gunakan:

$$\in \{0, 1\},$$

$$\in \{0, 1\}$$

N.33. Mutually exclusive long-short

Tidak boleh:

$$= = 1$$

Constraint:

$$\boxed{1}$$

Penalty:

$$\boxed{P}$$

N.34. Signed position

Position:

$$= -$$

Dengan:

$$= , P ,$$

dan:

$$= , P ,$$

Maka:

$$\boxed{= -}$$

N.35. Expected return short

Jika:

$$0,$$

short dapat memiliki expected profit:

$$P_t = -$$

Secara umum:

$$P =$$

Karena 0 untuk short.

N.36. Covariance untuk long-short

Tidak perlu membuat formula baru.

Tetap:

$$=$$

Karena sekarang signed.

Menariknya, posisi short secara alami menghasilkan perubahan tanda pada covariance term.

N.37. Hedging muncul otomatis

Misalkan:

$$, = 0$$

Jika kita long A dan short B:

$$0, \\ 0$$

Maka:

$$0$$

Portfolio variance dapat turun.

Jadi optimizer dapat menemukan:

$$tt$$

secara matematis.

N.38. Take Profit

Sekarang kita masuk ke TP.

Misalkan entry:

$$P$$

TP:

$$P$$

Untuk long:

$$= P - P$$

Reward:

$$=$$

Stop loss:

$$=$$

Reward-risk ratio:

$$=$$

N.39. TP tidak harus masuk langsung ke objective

Ini keputusan desain penting.

Expected return:

sudah menggambarkan forecast return.

Jika kita sekaligus memasukkan TP sebagai expected return tanpa model probabilitas menyentuh TP, kita akan melakukan double counting.

Jadi untuk V1:

$$P_t$$

bukan langsung:

$$P_{tt}$$

N.40. Tetapi TP dapat digunakan sebagai feasibility constraint

Misalnya kita hanya mau trade jika:

cukup besar dibandingkan stop risk.

Definisikan:

$$t = P$$

Kita dapat meminta:

$$\boxed{t}$$

N.41. Expected return relatif terhadap stop

Fractional stop distance:

$$= P$$

Expected return:

Maka:

$$\boxed{=}$$

Jika:

terlalu kecil, trade mungkin tidak menarik.

N.42. Tetapi jangan menyamakan ini dengan probability of TP

Ini penting secara finansial.

adalah conditional expected return.

Sedangkan:

$$PP$$

adalah hitting probability.

Keduanya berbeda.

Jika nanti kita membangun **Barrier-Hitting Model**, barulah TP dan SL dapat masuk secara probabilistik.

Untuk Tahap N kita belum melakukan itu.

N.43. Liquidity constraint

Position tidak boleh lebih besar daripada liquidity.

Misalnya:

average daily volume.

Kita dapat menetapkan:

$$P$$

Dengan:

misalnya hanya sebagian kecil dari volume yang dianggap aman.

N.44. Untuk short-term trading, order book lebih baik

Karena sistem kita memang akan menggunakan order book, kita dapat mengganti ADV dengan:

depth yang tersedia di sekitar harga entry.

Maka:

$$\square$$

Ini jauh lebih relevan untuk execution.

N.45. Slippage

Jika depth tipis:

meningkat.

Jadi cost:

$$=$$

Karena:

bergantung pada , objective dapat menjadi nonlinear.

N.46. Untuk QUBO kita discretize cost

Misalnya setiap position level:

,

mempunyai:

,

Maka:

$$t = , ,$$

Dengan demikian cost tetap linear dalam binary variables.

Ini salah satu alasan mengapa discretization sangat berguna.

N.47. QUBO final V1

Sekarang kita dapat menulis:

$$= t \quad t \quad t \quad t$$

N.48. Return term

$$t = - , P , ,$$

N.49. Risk term

$$= , , PP , , , ,$$

N.50. Cost term

$$t = , ,$$

N.51. SL-risk constraint

Define:

$$, = ,$$

Constraint:

$$, ''$$

Dengan slack:

$$, '' =$$

Penalty:

$$\boxed{= , '' -^2}$$

N.52. Cardinality

Misalnya maksimal:

positions.

$$=$$

Penalty:

$$\boxed{t = -^2}$$

N.53. Long-short exclusivity

$$\boxed{=}$$

N.54. Liquidity

Jika position level sudah dibatasi sehingga:

$$, t, ''$$

maka liquidity constraint sebenarnya dapat diselesaikan **sebelum** QUBO.

Ini lebih baik daripada menambahkan penalty.

Kita dapat menghapus position levels yang tidak feasible.

N.55. Ini adalah prinsip desain penting

Tidak semua constraint harus menjadi QUBO penalty.

Ada tiga jenis:

Hard preprocessing

Contoh:

,

langsung dibuang.

QUBO penalty

Constraint yang memang perlu dioptimasi bersama.

Objective

Return dan risk.

Dengan demikian QUBO tidak menjadi terlalu besar.

N.56. Final QUBO

Setelah semua ekspansi:

$$=$$

Kemudian:

$$\begin{matrix} = \\ \in \{0,1\} \end{matrix}$$

Dengan:

$$=$$

untuk kombinasi:

- : assets,
 - : position levels,
 - : direction.
-

N.57. Contoh ukuran problem

Misalkan:

$$= 20$$

aset.

Position levels:

$$=$$

Direction:

$$= 2$$

Maka:

$$= 20 \cdot 2 = 120$$

binary variables.

QUBO:

$$\in {}^{120120}$$

Kemudian:

$$120$$

qubits diperlukan untuk direct QAOA encoding.

Ini mulai menjadi problem yang serius.

N.58. Dan di sinilah quantum advantage tidak boleh diasumsikan

Kita **tidak boleh** mengatakan:

| QAOA pasti lebih cepat.

Belum tentu.

Untuk ukuran kecil:

$$= 4, 12$$

classical solver mungkin jauh lebih cepat.

Nilai penelitian kita adalah menguji:

□

pada **problem yang sama**.

N.59. Exact solver sebagai ground truth

Untuk problem kecil, kita bisa melakukan exhaustive search:

$$2$$

kombinasi.

Kemudian:

$$=$$

Ini menjadi:

$$\boxed{tt}$$

QAOA kemudian dibandingkan terhadap ground truth.

N.60. Approximation ratio

Jika objective kita adalah minimization:

$$= -$$

Semakin:

$$0,$$

semakin dekat QAOA terhadap optimum.

N.61. Tetapi untuk trading kita membutuhkan metrik kedua

Quantum optimizer mungkin menemukan solusi dengan:

sangat bagus.

Tetapi belum tentu realized PnL bagus.

Karena itu kita membutuhkan dua level evaluasi:

Optimization quality

Financial quality

$$P, , , t, tt,$$

N.62. Ini menghasilkan dua eksperimen berbeda

Eksperimen A

Apakah QAOA dapat menemukan solusi QUBO yang dekat dengan optimum?

Eksperimen B

Apakah solusi tersebut menghasilkan trading performance yang baik out-of-sample?

Jangan mencampurkan keduanya.

N.63. Dynamic QUBO

Ini adalah alasan kita menyebut Tahap N **Dynamic**.

Setiap periode:

$$t$$

kita mendapatkan:

$$t, t, P_{t, t, t}, t_t$$

Maka:

$$t = t, t, P_{t, t, t}, t_t$$

Jadi:

$$t_1 \quad t$$

secara umum.

N.64. Pasar mengubah Hamiltonian

Ini menurut saya bagian paling menarik dari keseluruhan desain.

Karena:

$$t$$

berasal dari market state, maka:

$$t$$

juga berubah.

Secara konseptual:

$$\begin{bmatrix} t & t & t \end{bmatrix}$$

Jadi kita memiliki **time-dependent optimization landscape**.

N.65. Namun QAOA tidak "memprediksi" perubahan landscape

Forecasting layer:

$$t \quad t, t$$

Optimization layer:

$$t, t \quad t$$

Quantum layer:

$$t \quad t$$

Ini tiga layer yang harus tetap dipisahkan:

$$\begin{bmatrix} Pt & tt & t \end{bmatrix}$$

N.66. Menghubungkannya ke Ising

Setelah QUBO:

$$\in \{0, 1\},$$

kita gunakan:

$$\begin{bmatrix} = 1 - 2 \end{bmatrix}$$

Dengan:

$$\in \{-1, 1\}$$

Substitusi:

menghasilkan:

$$\begin{bmatrix} = \end{bmatrix}$$

Konstanta tidak memengaruhi argmin.

N.67. Interpretasi fisiknya

Sekarang setiap qubit merepresentasikan keputusan:

$$= -1$$

atau:

$$= 1$$

Yang secara ekonomi berarti:

$$\boxed{tt}$$

atau:

$$\boxed{t}$$

Kemudian:

merepresentasikan interaction antara dua keputusan.

N.68. Hubungan dengan covariance

Ini sangat menarik.

Karena QUBO memiliki:

$$,$$

maka Ising coupling akan mengandung:

$$\boxed{PP}$$

Jadi:

correlation structure pasar diterjemahkan menjadi coupling antar-qubit.

Asset yang highly correlated:

$$0$$

menghasilkan coupling tertentu yang membuat pemilihan keduanya menjadi kurang/lebih energetically favorable tergantung konvensi tanda Hamiltonian.

N.69. Factor model bahkan masuk secara tidak langsung ke coupling

Ingat:

$$=$$

Maka:

PP ,

Artinya:

□

sebenarnya membawa informasi:

- BTC exposure,
- ETH exposure,
- Gold exposure,
- USD exposure,
- equity exposure,
- volatility exposure,
- residual correlation.

Dengan kata lain:

market structure menjadi quantum interaction structure.

Ini adalah salah satu bagian paling kuat dari arsitektur penelitian kita.

N.70. Contoh sederhana

Misalkan terdapat:

, ,

Factor matrix:

$= \begin{bmatrix} 10 & 05 & 0 & 00 & 0 & 0 & 00 & 0 & 12 & -0 \end{bmatrix}$

Dengan factor:

, , ,

Maka covariance asset tidak lagi muncul sebagai angka arbitrer.

Ia berasal dari:

□

N.71. Ini membuat sistem kita jauh lebih interpretable

Kita dapat menjawab:

Mengapa optimizer tidak memilih asset A?

Bisa jadi:

rendah.

Atau:

tinggi.

Atau:

tinggi dengan posisi B yang sudah dipilih.

Atau:

terlalu dekat/jauh.

Atau:

t

terlalu besar.

Atau risk budget sudah habis.

Ini jauh lebih baik daripada black-box:

t

N.72. Position sizing dan SL sekarang terhubung

Kita dapat merumuskan:

$, = ,$

dengan:

$\in 0, 1$

Misalnya:

$= \{0.25, 0.05, 10\}$

Maka setiap asset secara otomatis memiliki position size yang menyesuaikan SL.

N.73. Contoh

Asset A:

$$= 05$$

Asset B:

$$= 2$$

Dengan:

$$= 1$$

Maka maximum position:

$$= 2,$$

sedangkan:

$$= 05$$

Jadi asset dengan SL lebih lebar secara otomatis mendapatkan position lebih kecil.

N.74. Ini menjawab salah satu masalah utama trading

Kita tidak boleh mengatakan:

| setiap trade membeli nominal yang sama.

Karena:

$$100$$

pada asset volatility rendah tidak sama risikonya dengan:

$$100$$

pada asset volatility tinggi.

Risk-based sizing memperbaiki hal tersebut.

N.75. Tetapi \$1 bukan berarti guaranteed loss = \$1

Ini harus kita definisikan dengan benar.

Secara model:

$$= 1$$

Tetapi realized loss dapat lebih besar karena:

$$, \\ , \\ tt,$$

atau:

$$t$$

Jadi \$1 adalah:

$$\boxed{t}$$

bukan jaminan matematis bahwa realized loss selalu tepat \$1.

N.76. Maka kita dapat menambahkan safety multiplier

Misalnya:

$$1$$

Effective risk budget:

$$\boxed{t =}$$

Jika:

$$= 12,$$

kita menganggap ada 20% buffer.

Namun nilai nantinya harus **diestimasi dari data execution**, bukan dipilih sembarangan.

N.77. Sekarang kita memiliki formulasi V1 yang cukup lengkap

Saya akan merangkum objective utama kita:

$$\boxed{- \sum_{i=1}^n P_i^2 - \sum_{i=1}^n PP_i^2 - \sum_{i=1}^n \dots - \sum_{i=1}^n \dots}$$

Dengan:

$$, = ,$$

N.78. Ini kemudian menjadi

$$\boxed{= t t}$$

Kemudian:

$$t = t$$

Dan:

$$P_t$$

N.79. Trade Plan

Output optimizer bukan langsung:

BUY BTC.

Tetapi sesuatu seperti:

Asset	Direction	Level	Position	SL	TP
BTC	LONG	2	0.004 BTC	...	...
ETH	LONG	1	0.010 ETH	...	...
SOL	NONE	-	0	-	-

Kemudian execution engine baru menentukan:

$t,$

$t,$

$P,$

N.80. Ini juga menjaga desain kita dari masalah besar

Optimizer **tidak boleh** mengatur execution microstructure secara langsung.

QUBO menentukan:

$\square$

sedangkan execution engine menentukan:

$\square$

Contoh:

Optimizer:

Long BTC, risk \$1, position level 2.

Execution engine:

Letakkan limit order pada bid/ask tertentu, tunggu fill, pasang SL/TP, monitor order book.

Pemisahan ini akan sangat penting ketika kita nanti benar-benar menghubungkannya ke Binance dan kemudian broker saham.

N.81. Dynamic re-optimization

Misalkan order belum filled.

Pada:

$$t_0$$

kita memiliki:

$$t_0$$

Satu jam kemudian:

$$t_1$$

market berubah:

$$t_1 - t_0$$

Maka sistem menghitung ulang:

$$t_1$$

Jika trade lama tidak lagi optimal, order dapat:

Kemudian:

Ini persis menjawab rancangan awalmu mengenai:

setiap periode mengevaluasi apakah pending order masih layak dipertahankan.

N.82. Untuk posisi yang sudah filled

Kita harus membedakan:

$$P$$

dengan:

$$Pt$$

Untuk pending:

$$t$$

dapat digunakan langsung untuk memutuskan:

Untuk open position, kita tidak langsung melakukan liquidation hanya karena:

t

berubah.

Kita membutuhkan **Position Management Layer**.

Itu akan menjadi bagian dari tahap berikutnya.

N.83. Mengapa?

Misalnya:

t_0

model berkata:

Trade filled.

Kemudian:

t_1

model berubah menjadi:

Tidak berarti:

secara otomatis.

Kita harus mempertimbangkan:

- unrealized PnL,
- time-in-trade,
- distance to SL,
- distance to TP,
- updated expected return,
- updated volatility,
- updated factor regime,
- opportunity cost.

Jadi keputusan:

dan:

adalah dua masalah optimasi yang berbeda.

N.84. Ini sangat cocok dengan desain awal kita

Kita sekarang mempunyai:

$$\boxed{ttP \quad tt}$$

dan nantinya:

$$\boxed{ttP \quad Ptt}$$

Problem B akan menjawab:

| Apakah posisi tetap dipertahankan?

| Apakah SL dinaikkan ke BEP?

| Apakah TP diperpanjang?

| Apakah sebagian posisi ditutup?

N.85. Bahkan itu bisa menjadi QUBO kedua

Misalnya sebuah posisi memiliki:

aksi:

$$1 = ,$$

$$2 = ,$$

$$=$$

Maka kita dapat membangun:

$$, \in \{0, 1\}$$

dan mengoptimasi expected future utility.

Jadi arsitektur akhirnya bukan hanya satu QUBO.

Kita dapat mempunyai:

$$\boxed{t}$$

dan:

t

Ini akan sangat menarik secara penelitian.

N.86. Posisi kita sekarang

Setelah Tahap N:

$t \quad t \quad t \quad Pt$

sudah tersambung secara matematis.

Dan yang paling penting, setiap layer memiliki fungsi yang jelas:

Layer	Pertanyaan
Data	Apa yang sedang terjadi?
Factor	Faktor apa yang menggerakkan pasar?
Forecast	Apa yang mungkin terjadi berikutnya?
Risk	Seberapa tidak pasti?
Position sizing	Berapa besar posisi yang aman?
QUBO	Kombinasi trade mana yang optimal?
Ising	Bagaimana merepresentasikan optimasi secara quantum?
QAOA	Bagaimana mencari state energi rendah?
Execution	Bagaimana mengeksekusi keputusan?
Management	Apakah posisi perlu dipertahankan/diubah?

Persamaan inti Tahap N

Kalau seluruh Tahap N harus dipadatkan menjadi beberapa persamaan utama, saya akan mempertahankan enam ini:

1. Position sizing

$, =$

2. Net expected return

$= -$

3. Portfolio risk

$$2 =$$

4. Risk budget

$$=$$

5. Dynamic QUBO

$$t = t, t, P_{t, t, t, t}$$

6. Optimization

$$t = \in_{\{0,1\}}^t$$

Kemudian:

$$= 1 - 2$$

memberikan:

$$=$$

Satu konsekuensi penting dari Tahap N

Kita sekarang dapat melihat dengan sangat jelas **di mana matematika quantum benar-benar masuk ke quantitative trading kita.**

Bukan pada forecasting:

$$P_{t1} = P_t,$$

dan bukan pada prediksi BTC.

Quantum masuk setelah seluruh informasi pasar telah direduksi menjadi:

$$t, t, t, t, t$$

Informasi tersebut membentuk **QUBO**, dan QUBO membentuk **Ising Hamiltonian**. Dengan demikian, apabila nanti kita membandingkan **Exact Solver vs Classical Optimizer vs VQE/QAOA**, ketiganya benar-benar menerima **problem matematis yang identik**. Ini sangat penting agar penelitian kita tidak berubah menjadi sekadar "menggunakan quantum karena quantum", tetapi menjadi eksperimen yang dapat dipertanggungjawabkan.

Tahap berikutnya secara logis adalah **Tahap O — Position Management & Dynamic Rebalancing**: kita akan memformalkan bagian yang sejak awal paling kamu inginkan, yaitu **apa yang dilakukan setelah order pending/filled**, termasuk floating loss, floating profit, break-even stop, trailing SL, dynamic TP, partial exit, time stop, dan kapan sistem harus

membangun ulang QUBO. Di sana kita bisa mulai membentuk **QUBO kedua untuk pengelolaan posisi yang sudah terbuka.**

Prompt:

8/17/2026, 10:00:00 PM

lanjut Tahap O

Response:

8/17/2026, 10:00:01 PM

Remembering

Clarified Tahap N/O design, dynamic QUBO, and position sizing memory

Worked for 4s

Tahap O - Position Management & Dynamic Rebalancing

Tahap O adalah kelanjutan yang sangat penting dari Tahap N.

Pada Tahap N kita telah menyelesaikan pertanyaan:

"Trade mana yang sebaiknya dibuka sekarang?"

Sekarang pertanyaannya berubah menjadi:

"Setelah trade dibuka, apakah trade tersebut masih layak dipertahankan, dan jika iya, bagaimana SL, TP, serta ukuran posisi harus dikelola?"

Ini bukan lagi masalah *entry optimization*. Kita sekarang membangun **dynamic position management**.

Arsitektur kita menjadi:



dan Position Management sendiri akan terus berjalan sepanjang umur posisi.

O.1. Mengapa position management harus dipisahkan dari entry?

Misalkan pada waktu:

t_0

model menghasilkan:

$$t_0 =$$

Order kemudian filled.

Pada:

$$t_1 = t_0 + 1,$$

kondisi pasar berubah.

Misalnya:

$$t_1, t_0$$

Tetapi BTC masih berada:

$$1$$

dari entry.

Apakah kita langsung menjual?

Tidak.

Karena informasi yang relevan sekarang bukan hanya:

$$t_1,$$

tetapi juga:

$$\begin{aligned} & , \\ & , \\ & , \\ & , \\ & , \end{aligned}$$

Jadi kita membutuhkan state tambahan.

O.2. State posisi

Untuk setiap posisi , kita definisikan:

$$t = P_t, P_t, t, P_t, t, \tau t, P_t, t, t, t$$

dengan:

- P_t : harga entry,
- P_t : harga sekarang,
- t : stop-loss sekarang,

- P_t : take-profit sekarang,
- t : quantity,
- τ : umur posisi,
- P : unrealized PnL,
- σ : volatility terkini,
- r : expected return terkini,
- ρ : remaining risk.

Maka sistem tidak lagi hanya melihat:

Ia melihat:

$$\square$$

O.3. Normalisasi PnL menggunakan

Karena sejak Tahap N kita menggunakan risk budget, ukuran yang sangat berguna adalah:

$$t = P t_t$$

Misalnya initial risk:

$$t = 1$$

Jika floating loss:

$$P = -05,$$

maka:

$$= -05$$

Jika profit:

$$P = 2,$$

maka:

$$= 2$$

Ini jauh lebih konsisten daripada menggunakan dollar PnL secara langsung.

O.4. Mengapa -multiple sangat penting?

Misalkan:

Trade A:

10

dengan initial risk:

20

Maka:

= 05

Trade B:

10

dengan initial risk:

2

Maka:

= 5

Secara nominal sama:

10,

tetapi secara risk-adjusted sangat berbeda.

Karena sistem kita memang dibangun berdasarkan:

1

maka seluruh position management sebaiknya menggunakan .

O.5. Tiga keadaan posisi

Kita dapat membagi posisi menjadi:

State I - Floating Loss

0

State II - Floating Profit

0

State III - Near Exit

Misalnya:

$P \ P$

atau:

P

Tetapi kita tidak boleh menggunakan PnL saja.

Kita juga perlu mengetahui apakah **thesis** trade masih valid.

O.6. Thesis score

Definisikan:

$$t$$

sebagai ukuran seberapa kuat alasan awal trade masih berlaku.

Misalnya:

$$= , t, t, tt, t, t$$

Secara sederhana:

$$0$$

berarti thesis masih mendukung posisi.

Sedangkan:

$$0$$

berarti thesis mulai invalid.

O.7. Kita tidak ingin threshold arbitrer

Daripada:

| Kalau profit 2%, geser SL.

lebih matematis jika menggunakan:

$$,$$

volatility,

dan expected future return.

Jadi keputusan tidak bergantung pada:

$$2$$

Melainkan pada keadaan relatif terhadap:

$$t$$

dan:

O.8. Stop-loss awal

Saat entry:

$$P_0$$

Untuk long:

$$_0 = P_0 -$$

Risk:

$$_0 =$$

Kita telah menetapkan:

$$_0 \ 1$$

Stop ini disebut:

$$\boxed{t}$$

O.9. Jangan langsung mempersempit SL ketika floating loss

Ini prinsip yang penting.

Misalnya:

$$= -05$$

Kemudian model memperbarui prediksi.

Jika thesis masih valid:

$$0,$$

maka kita tidak boleh otomatis melakukan:

$$P_t$$

Karena itu dapat mengubah trade yang secara statistik masih valid menjadi stop-out hanya karena noise.

O.10. Floating loss decision

Kita definisikan future expected utility:

$$t = {}_t t$$

Jika:

$$t \geq 0$$

dan:

$$t \geq 0,$$

maka posisi masih memiliki alasan untuk dipertahankan.

Sebaliknya jika:

$$t < 0,$$

thesis tidak lagi mendukung posisi.

O.11. Keputusan exit yang lebih matematis

Kita dapat mendefinisikan:

$$t = {}_t t - {}_t t$$

Jika:

$$0,$$

hold memiliki expected value positif.

Jika:

$$0,$$

exit menjadi lebih rasional.

O.12. Opportunity cost

Tetapi ada satu hal lagi.

Modal yang digunakan oleh posisi lama tidak dapat digunakan untuk trade baru.

Misalkan ada candidate trade baru dengan:

$$t$$

Maka kita dapat membandingkan:

dengan:

t

Definisikan:

$$= t -$$

Jika:

0,

trade baru memiliki expected value lebih tinggi.

Ini menghasilkan konsep:

tt

O.13. Jadi floating loss tidak otomatis berarti exit

Kita memperoleh aturan konseptual:

$\square$

Yang menentukan adalah:

$\square$

dan:

$\square$

O.14. Sekarang floating profit

Misalkan:

$= 15$

Pertanyaan kita:

| Apakah kita mengambil profit sekarang?

atau:

| Apakah kita membiarkan profit berkembang?

Secara matematis kita membandingkan:

dengan:

t

O.15. Immediate exit value

Jika kita keluar sekarang:

$$t = P_t - t$$

Sedangkan jika hold:

$$t = P_t - t$$

Maka:

$$\Delta = -t$$

O.16. Jika $\Delta \geq 0$

Lebih baik mempertahankan posisi.

Jika:

$$\Delta \geq 0,$$

lebih baik exit.

Ini memberikan dasar matematis untuk:

"floating profit tinggi, apakah TP harus dinaikkan?"

O.17. TP bukan angka statis

Awalnya:

$$P_0$$

Tetapi setelah market berubah:

$$P_t$$

dapat menjadi:

$$P_t - P_0$$

Namun kenaikan TP hanya dilakukan jika:

$$P_t$$

masih mendukung target tersebut.

O.18. Kita membutuhkan target distribution

Daripada hanya memprediksi:

$$t,$$

lebih ideal kita memiliki:

$$\Delta P_t$$

Dari distribusi tersebut kita dapat menghitung:

$$P P P$$

Dan:

$$P P$$

Ini jauh lebih berguna untuk position management.

O.19. Expected value dengan TP dan SL

Misalkan:

$$\begin{aligned} &= P P t, \\ &= P t, \end{aligned}$$

dan:

$$= 1 - -$$

untuk kondisi lainnya.

Expected PnL:

$$= -$$

Jika:

$$0,$$

target tersebut masih layak.

O.20. Dynamic TP

Misalkan kandidat TP:

$$P^1, P^2, P$$

Kita dapat menghitung:

$$1, 2,$$

Kemudian:

$$P = P$$

Jadi TP tidak lagi:

"naik 2% setiap kali profit."

Tetapi:

pilih target yang memiliki expected utility tertinggi berdasarkan state pasar sekarang.

O.21. Dynamic Stop-Loss

Sekarang masalah yang lebih menarik.

Kita ingin memilih:

$$t$$

Kandidatnya dapat berupa:

$$1 = t,$$

$$2 = P,$$

$$= tt,$$

$$4 = ttt$$

O.22. Break-even stop

Untuk long:

$$t = P_t$$

Namun sebenarnya BEP harus memperhitungkan biaya.

Jika:

$$t \ t$$

tidak nol, maka break-even sebenarnya:

$$P_P = P_{t \ tt}$$

Jadi:

$$= P_t$$

belum tentu benar-benar break-even.

O.23. Trailing stop berbasis volatility

Misalkan:

$$t$$

atau volatility scale:

$$t$$

Untuk long:

$$t = P_t - {}_tP_t$$

dengan:

$$0$$

Tetapi SL hanya boleh bergerak satu arah:

$$t \quad t-1$$

untuk long.

Untuk short:

$$t \quad t-1$$

O.24. Mengapa tidak hanya memakai trailing stop?

Karena trailing stop tidak mengetahui thesis.

Misalnya volatility meningkat.

Trailing stop dapat menjadi:

$$t = P_t - {}_tP_t$$

Jika t meningkat tajam, stop justru dapat bergerak **lebih jauh**.

Maka position management harus mempertimbangkan:

$$P \quad t \quad t$$

O.25. Kandidat action

Sekarang kita formalasikan decision space.

Untuk posisi yang sedang terbuka, kita memiliki:

$$\in$$

Misalnya:

$$= \{P, , P, , PP, P\}$$

O.26. Binary encoding

Definisikan:

$$, \in \{0, 1\}$$

Dengan constraint:

$$\boxed{\in, = 1}$$

Artinya satu action dipilih.

O.27. Utility setiap action

Untuk action :

$$, = , t^-, -, ,$$

Kemudian kita ingin:

$$\boxed{= ,}$$

O.28. Ini dapat menjadi QUBO kedua

Ubah menjadi minimization:

$$\boxed{= - \text{,,} P , - 1^2}$$

Dengan:

$$, \in \{0, 1\}$$

Ini adalah QUBO.

O.29. Tetapi kita dapat membuatnya jauh lebih kuat

Jika kita memiliki beberapa posisi:

$$= 1, \dots$$

maka setiap action memiliki interaksi.

Misalnya:

exit BTC memungkinkan capital dialihkan ke ETH.

Maka:

,

tidak lagi independen.

Kita membutuhkan:

$$= \dots$$

Dan kembali kita mendapatkan QUBO.

O.30. Position management QUBO

Secara umum:

$$= - P$$

atau sesuai konvensi:

$$= - \dots P$$

O.31. Apa yang dimaksud interaction antar-action?

Misalnya dua posisi:

dan:

Jika BTC ditutup:

,

capital tersedia.

Jika ETH dipertahankan:

$$P,$$

tidak ada masalah.

Tetapi jika:

$$P$$

dan:

$$,$$

total risk mungkin melebihi:

$$t$$

Maka interaction penalty muncul.

O.32. Dynamic portfolio risk

Pada waktu t :

$$t$$

adalah posisi aktual.

Jika kita mempertimbangkan action , kita memperoleh:

$$t$$

Risk:

$$\boxed{2 = t t t}$$

Maka action yang meningkatkan risk terlalu besar dapat dihukum.

O.33. Remaining risk

Ini sangat penting untuk sistem kita.

Jika:

$$P_t \ P_t$$

untuk long, dan SL telah dinaikkan:

$$_t \ P_t,$$

maka posisi sudah memiliki:

$$\boxed{t}$$

Define:

$$t = t - P_t$$

Untuk long.

Jika:

$$0,$$

minimum realized profit setelah SL tersentuh sudah positif, mengabaikan gap/slippage.

O.34. Locked profit

Dalam -multiple:

$$\boxed{= t}$$

Misalnya:

$$= 0$$

Artinya meskipun trade berbalik dan menyentuh SL, secara model kita masih mempertahankan:

$$0$$

O.35. Ini mengubah objective

Sebelum profit:

$$0$$

Setelah BEP:

$$0$$

Setelah trailing:

$$0$$

Maka position management harus memaksimalkan:

$$\boxed{t t t}$$

tanpa mengorbankan terlalu banyak:

$$\boxed{t}$$

O.36. Objective baru

Definisikan:

=

dan:

=

Kita dapat membuat:

= -

Parameter:

mengontrol seberapa penting capital protection.

O.37. Tetapi kita harus menghindari "profit fixation"

Jika:

terlalu besar, sistem akan terlalu cepat:

Jika:

terlalu kecil, sistem akan membiarkan profit menguap.

Jadi:

adalah parameter yang harus diuji melalui walk-forward validation.

O.38. Dynamic TP dan SL sebagai joint optimization

Jangan memilih:

P

dan:

secara terpisah.

Karena keduanya saling berhubungan.

Kita definisikan action:

$$= , P$$

Kemudian:

$$\in$$

Misalnya:

$$= \{_1, P_{1, 1}, P_{2, 2}, P_{2, }, P, \}$$

O.39. Expected utility

Untuk setiap kombinasi:

$$, P,$$

hitung:

$$= , P - t -$$

Kemudian:

$$, P = ,$$

O.40. Ini menjawab pertanyaan awalmu

Kamu sebelumnya membayangkan:

| jika floating profit tinggi, mungkin TP dinaikkan lagi.

Jawabannya sekarang menjadi:

Ya, tetapi bukan karena profit sudah tinggi.

TP dinaikkan jika:

$$P, P,$$

Jadi keputusan tersebut berasal dari **expected future utility**, bukan dari rule arbitrer.

O.41. Time stop

Ada kondisi lain:

Trade tidak loss, tetapi juga tidak bergerak.

Misalnya:

$$0$$

selama:

$$\tau =$$

Capital menjadi terkunci.

Kita definisikan:

$$t = t - \tau T$$

Jika:

$$t \geq 0,$$

exit dapat menjadi optimal.

O.42. Opportunity cost kembali masuk

Semakin lama posisi stagnan:

$$\tau,$$

semakin besar opportunity cost.

Jadi:

$$t$$

adalah variabel matematis, bukan sekadar informasi tambahan.

O.43. Regime change

Ini bahkan lebih penting.

Misalkan ketika entry:

$$t =$$

Kemudian:

$$t_1 = t$$

Expected return:

$$t$$

dapat berubah drastis.

Maka:

harus dihitung ulang.

O.44. Regime-conditioned management

Kita dapat menulis:

$$\boxed{t}$$

dengan:

$$t$$

adalah market regime.

Contoh:

$$t \in \{, , , , , \}$$

O.45. Gold dan DXY juga kembali masuk

Karena sebelumnya kita ingin sistem bekerja tidak hanya pada crypto tetapi juga saham internasional, factor layer kita dapat memberi:

$$t = , , , , P , ,$$

Jika misalnya:

dan:

$$,$$

model dapat memperbarui:

$$,$$

$$,$$

dan akhirnya:

Dengan demikian perubahan faktor global dapat memicu:

$$\boxed{tt}$$

O.46. Dynamic rebalancing

Sekarang kita dapat mendefinisikan proses periodik.

Pada waktu:

$$t$$

Kita memiliki:

$$t =$$

Update market:

$$t$$

Kemudian:

$$t, t$$

dihitung ulang.

O.47. Langkah pertama: valuasi posisi

Untuk setiap posisi:

hitung:

$$\begin{aligned} &P, \\ &, \\ &,' \\ &\tau, \\ &, \\ &, \end{aligned}$$

O.48. Langkah kedua: generate candidate actions

Untuk setiap posisi:

$$= \{P, , P, , P_1, P_2, P\}$$

O.49. Langkah ketiga: bangun management QUBO

$$\boxed{t = P_t, t, t, t, P_t, \tau_t, t_t, t_t}$$

O.50. Langkah keempat: solve

Kita dapat menggunakan:

$$t = t$$

Solver dapat berupa:

- exhaustive search untuk problem kecil,
- classical QUBO solver,
- simulated annealing,
- tabu search,
- MILP,
- QAOA.

Dengan demikian quantum bukan satu-satunya solver.

O.51. Langkah kelima: execution

Jika hasilnya:

$$P$$

tidak melakukan apa-apa.

Jika:

$$P$$

modify SL.

Jika:

$$PP$$

modify TP.

Jika:

$$P$$

reduce quantity.

Jika:

close position.

O.52. Setelah execution

Kita kembali ke:

$$tt$$

Sehingga terdapat loop:

$$\boxed{tt \ t \ t \ t \ - \ t}$$

Ini adalah inti sistem kita.

O.53. Full mathematical loop

Secara matematis:

$$t \ t \ t, t$$

kemudian:

$$t, t \ tt$$

kemudian:

$$tt \ t$$

kemudian execution menghasilkan:

$$t$$

Kemudian:

$$t, t \ t$$

dan:

$$t \ t$$

O.54. Kita sekarang memiliki dua Hamiltonian

Entry:

$$\boxed{{}_t t}$$

Management:

$$\boxed{t}$$

Keduanya berasal dari market state yang sama tetapi memiliki tujuan berbeda.

O.55. Ini memungkinkan arsitektur yang jauh lebih menarik

Secara keseluruhan:

$$t t t t t \boxed{t} P t \boxed{P t t} P t t$$

O.56. Namun ada satu hal yang harus kita cegah: excessive re-optimization

Jika kita menjalankan optimizer setiap:

$$1,$$

sistem mungkin:

$$P \quad P$$

hanya karena noise.

Ini disebut:

$$\square$$

O.57. Hysteresis

Kita dapat membuat threshold.

Misalnya keputusan baru hanya dilakukan jika:

$$\boxed{-t}$$

Dengan:

$$0$$

Jadi optimizer tidak mengubah posisi hanya karena improvement sangat kecil.

O.58. Transaction cost secara alami menghasilkan hysteresis

Sebenarnya kita dapat membuat:

$$t = -t - t$$

Switch hanya jika:

$$t \leq 0$$

Jadi biaya transaksi sendiri membantu mencegah overtrading.

O.59. Rebalancing bukan berarti selalu mengubah posisi

Ini penting.

Dynamic rebalancing berarti:

□

bukan:

□

Jika posisi saat ini masih optimal:

$$= P$$

Maka:

O.60. Position management sebagai optimal stopping

Ada perspektif matematika yang lebih dalam.

Pertanyaan:

| Kapan sebaiknya saya keluar?

sebenarnya adalah masalah:

$$t \leq \tau$$

Kita memiliki state:

$$t = P_t, t, t, t, \tau_t$$

Kemudian kita mencari stopping time:

$$\tau = \{t \mid t = t\}$$

dengan:

- t : value of continuing,

- t : value of exiting.

O.61. Ini sangat penting secara teoritis

Jadi position management kita sebenarnya dapat dipandang sebagai:

$$\boxed{P \quad tt \quad ttt}$$

Dan QUBO adalah representasi diskrit dari keputusan kombinatorial tersebut.

O.62. Bahkan kita bisa melihatnya sebagai MDP

State:

$$t$$

Action:

$$t$$

Reward:

$$t$$

Transition:

$$P_{t|t, t}$$

Maka kita memiliki:

$$\boxed{P}$$

Namun kita **tidak harus menggunakan reinforcement learning**.

Kita dapat menggunakan probabilistic forecast + optimization.

Ini lebih mudah dijelaskan dan divalidasi untuk penelitian kuantitatif.

O.63. Hubungan dengan QAOA

QAOA tidak menggantikan keseluruhan MDP.

QAOA menyelesaikan subproblem:

$$\boxed{t}$$

Jadi:

Ini framing yang jauh lebih tepat.

O.64. Contoh konkret

Misalkan:

$$t = 1$$

Posisi BTC:

$$= 24$$

Thesis score:

$$= 02$$

Expected future return:

$$0$$

Current SL:

$$0$$

locked.

Candidate actions:

Action	Locked profit	Expected future profit	Risk
EXIT	24	0	0
KEEP	0	11	tinggi
TRAIL	12	0	sedang
TP-UP	0	15	tinggi

Model menghitung:

Misalnya hasil:

$$= 25,$$

$$P = 10,$$

$$= 200,$$

$$P-P = 215$$

Maka:

□

menjadi optimal.

Bukan karena:

| profit sudah 2.4R.

Tetapi karena:

$$\frac{t}{t} \quad t$$

O.65. Contoh kedua

Misalkan:

$$= 24$$

tetapi:

$$= 01,$$

dan expected future return masih tinggi.

Mungkin:

$$P = 2$$

dan:

$$= 25$$

Maka:

$$\boxed{P}$$

Bahkan:

$$P$$

dapat dinaikkan jika:

$$P \quad P$$

O.66. Contoh floating loss

Misalkan:

$$= -0$$

Tetapi:

$$= 05,$$

dan:

$$= 0$$

Sedangkan:

$$t = -0$$

Maka:

$$\boxed{P}$$

Ini menjawab bagian awal desainmu:

floating loss tidak otomatis ditutup.

O.67. Tetapi jika thesis rusak

Misalnya:

$$= -0,$$

namun:

$$= -0,$$

dan:

$$= -12$$

Sedangkan:

$$t = -0$$

Maka:

$$\square$$

Jadi stop-loss bukan satu-satunya exit mechanism.

O.68. Kita sekarang bisa mendefinisikan empat alasan exit

1. Risk exit

$$P_t \quad t$$

2. Target exit

$$P_t \quad P_t$$

3. Thesis exit

t

4. Opportunity-cost exit

t

Ini jauh lebih kuat daripada hanya:

P

O.69. Tetapi SL/TP tetap menjadi safety layer

Ini penting untuk deployment.

Optimizer tidak boleh menjadi satu-satunya proteksi.

Jika:

hang,

atau:

t

putus,

atau:

menghasilkan output aneh,

SL yang sudah terpasang tetap harus bekerja.

Jadi:

t

harus berada di bawah:

tt

O.70. Hierarki kontrol

Saya menyarankan arsitektur:

$\square$

di paling bawah.

Di atasnya:

$$t$$

kemudian:

$$Ptt$$

kemudian:

$$ttt$$

dan paling atas:

$$t$$

Sehingga optimizer tidak pernah bisa berkata:

"Saya yakin sekali, jadi abaikan maximum loss."

Constraint risiko tetap menang.

O.71. Tahap O dalam bentuk algoritma

Secara konseptual:

```
INITIALIZE position state

WHILE market is open:

    observe market

    update:
        price
        volatility
        factors
        expected return
        covariance
        liquidity

    FOR every open position:

        calculate:
            PnL
            R-multiple
            locked profit
            time-in-trade
            thesis score
            future expected value
```



```
generate candidate actions:
```

```
KEEP
```

```
EXIT
```

```
BEP
```

```
TRAIL
```

```
TP-UP
```

```
PARTIAL EXIT
```

```
construct management QUBO
```

```
solve QUBO
```

```
IF new decision is sufficiently better:
```

```
execute action
```

```
update SL/TP/state
```

```
periodically:
```

```
rebuild entry QUBO
```

```
evaluate new opportunities
```

```
END
```

O.72. Arsitektur final setelah Tahap O

Sekarang sistem kita telah berkembang menjadi:

tt	t	t	t	t	Pt	$Pttt$	t	Pt
------	-----	-----	-----	-----	------	--------	-----	------

O.73. Dan sekarang ada sesuatu yang sangat menarik

Sebelum Tahap O, kita hanya mempunyai:

□

Setelah Tahap O, kita mulai memiliki:

□

Artinya keputusan pada waktu t mempengaruhi state pada:

$t + 1$

Misalnya:

membebaskan capital.

Capital tersebut dapat digunakan untuk:

Jadi keputusan kita memiliki **intertemporal effect**.

O.74. Persamaan konseptual terpenting Tahap O

Saya akan merangkum Tahap O dengan satu persamaan:

$$t = \{P_t, P_{t,t}, P_{t,t,t}, \tau_t, t_t, t_t\}$$

dengan:

$$t = \{P_t, P_{t,t}, P_{t,t,t}, \tau_t, t_t, t_t\}$$

Kemudian keputusan diskrit tersebut kita transformasikan menjadi:

$$t$$

dan akhirnya:

$$t$$

O.75. Posisi kita sekarang

Kita sudah memiliki dua problem optimasi yang berbeda:

$$t, t, tt$$

dan:

$$t, t, t, t$$

Keduanya dapat diuji dengan:

$$t$$

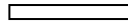
pada problem yang **identik**.

Dan menurut saya, ini adalah titik di mana proyek kita mulai menjadi jauh lebih serius.

Kita tidak lagi sekadar membuat:

"bot trading yang memakai QAOA."

Kita sedang membangun sebuah **dynamic quantitative decision system**, di mana:



dan setiap keputusan dapat ditulis secara matematis.

Yang juga sangat penting: **QAOA hanyalah solver**, bukan sumber alpha. Alpha berasal dari data, factor model, forecasting, dan risk model. QAOA menjawab pertanyaan kombinatorial: *dari seluruh keputusan diskrit yang feasible, mana yang paling baik menurut objective kita?*

Dengan Tahap O selesai, fondasi matematis untuk **entry + position management** sudah terbentuk. Tahap berikutnya secara alami adalah **Tahap P - Execution & Microstructure Optimization**, yaitu membawa keputusan abstrak seperti "*LONG BTC dengan risk \$1*" menjadi keputusan nyata: **harga limit order, kedalaman order book, probabilitas fill, expected slippage, queue position, cancel/replace, dan kapan sebuah order sebaiknya tidak diletakkan sama sekali**. Di situlah sistem kita mulai benar-benar bersentuhan dengan *short-horizon trading* dan market microstructure.

Prompt:

8/17/2026, 10:01:13 PM

lanjut Tahap P

Response:

8/17/2026, 10:01:14 PM

Tahap P — Execution & Market Microstructure Optimization

Tahap P adalah tahap yang menurut saya **sangat penting** jika tujuan akhirnya memang bukan sekadar membuat *portfolio optimizer*, tetapi membangun sistem **quantitative short-horizon trading** yang realistis.

Sampai Tahap O, kita sudah bisa mengatakan:

"Saya ingin **LONG BTC** dengan **risk budget \$1**, **target tertentu**, dan **stop-loss tertentu**."

Tetapi pasar tidak bekerja seperti itu.

Kita tidak bisa begitu saja menganggap:

$$P_t = P_t$$

Dalam kenyataan, kita harus menjawab:

- Apakah menggunakan **market order** atau **limit order**?
- Kalau limit order, di harga berapa?
- Berapa probabilitas order tersebut terisi?
- Berapa lama kita bersedia menunggu?
- Jika belum terisi, apakah order dipindahkan?
- Berapa *slippage*?
- Berapa *spread*?
- Berapa besar order yang sedang mengantre di depan kita?
- Apakah order book sedang menunjukkan tekanan beli/jual?
- Jika order kita tidak filled, apakah peluang pasar bergerak tanpa kita lebih buruk daripada menunggu?
- Bagaimana execution memengaruhi \$1 risk budget kita?

Dengan demikian Tahap P memperkenalkan:

□

ke dalam sistem.

P.1. Perubahan paradigma

Sebelumnya kita memiliki:

□

seolah-olah harga entry adalah:

$$P_t$$

Sekarang kita harus membedakan:

$$\boxed{P_t}$$

$$P_t$$

$$P_t$$

$$P_t$$

Dengan:

$$P_t = P_t P_t^2$$

Dan:

$$P_t - P_t = t$$

adalah bid-ask spread.

P.2. Entry signal $\neq$ execution

Ini merupakan prinsip utama Tahap P:

$$t \quad t$$

Model sebelumnya mungkin mengatakan:

Tetapi execution optimizer harus menentukan:

□

Misalnya:

$$,P, P-1t, P-2t,$$

P.3. State market microstructure

Sekarang state kita diperluas.

Sebelumnya:

$$t = \{P_t, t, t, t, \}$$

Sekarang:

$$t = \{P_t, P_{t, t, t, t, t, t, t, t, t}\}$$

Beberapa variabel penting:

- t : volume bid,
 - t : volume ask,
 - t : spread,
 - t : order-book imbalance,
 - t : depth,
 - t : queue position,
 - t : order arrival intensity,
 - t : short-term volatility.
-

P.4. Order-book imbalance

Salah satu feature paling dasar adalah:

$$t = t - t t \quad t$$

sehingga:

$$-1 \leq t \leq 1$$

Jika:

$$t \geq 1,$$

bid jauh lebih besar daripada ask.

Jika:

$$t \leq -1,$$

ask jauh lebih besar daripada bid.

P.5. Tetapi imbalance sederhana belum cukup

Misalnya:

$$V_1 = 100, v_1 = 20$$

Kita mendapatkan:

$$= 0$$

Tetapi mungkin volume tersebut berada sangat jauh dari mid-price.

Karena itu kita dapat menggunakan **depth-weighted imbalance**.

Misalkan terdapat N level:

$$= 1, \dots, N$$

Maka:

$$t = \frac{V_1}{V_1 + v_1} = 1, \quad \frac{V_1}{V_1 + v_1} = 1, \quad \frac{V_1}{V_1 + v_1} = 1,$$

dengan:

$$= -1$$

Level yang lebih dekat dengan mid mendapat bobot lebih besar.

P.6. Microprice

Mid-price:

$$t = \frac{V_1}{V_1 + v_1}$$

Tetapi mid-price tidak memperhitungkan imbalance.

Kita dapat menggunakan:

$$P_t = \frac{t_t}{t_t + t_{tt} + t_{ttt}}$$

dengan:

- t_t : best ask,
- t_{tt} : best bid.

Jika:

,

maka:

$$P$$

cenderung mendekati ask.

Ini memberi estimasi yang lebih informatif mengenai tekanan harga jangka sangat pendek.

P.7. Short-horizon expected price

Sekarang kita dapat membuat:

$$P_{t\Delta} = P_{t\Delta} + t$$

Atau expected return:

$$r_{t,\Delta} = \frac{P_{t\Delta} - P_t}{P_t}$$

Dan model dapat menggunakan:

$$r_{t,\Delta} = \frac{P_t - P_{t,t}, , ,}{P_t}$$

P.8. Sekarang masalah utama: fill probability

Misalkan kita ingin membeli.

Kita mempunyai dua pilihan:

Market order

Probability fill:

$$P \approx 1$$

Tetapi:

t_t

lebih besar.

Limit order

Cost lebih rendah.

Tetapi:

$$P \geq 1$$

Maka execution sebenarnya adalah trade-off:

$$t_t \quad P_t$$

P.9. Expected execution utility

Misalkan action:

adalah jenis order.

Kita definisikan:

$$= P$$

Jika filled:

Jika tidak filled:

Maka:

$$= 1 -$$

Ini merupakan fondasi execution optimizer kita.

P.10. Mengapa "tidak filled" bukan utility nol?

Ini sangat penting.

Misalkan model sangat yakin:

$$P_{t\Delta} \quad P_t$$

Kita menaruh limit buy:

$$P \quad P_t$$

Jika tidak filled dan harga naik, kita kehilangan kesempatan.

Opportunity cost tersebut adalah:

$$0$$

Jadi limit order murah belum tentu optimal.

P.11. Market order versus limit order

Misalkan:

Market

$$= 0$$

Expected execution price:

$$P = P_t$$

Limit

$$= 05$$

Execution price:

$$P \neq P_t$$

Tetapi jika tidak filled:

$$P_t \neq P_t$$

Maka:

$$t$$

bisa saja lebih rendah daripada:

$$t$$

P.12. Implementation shortfall

Kita memerlukan ukuran:

$$\square$$

atau implementation shortfall.

Untuk long secara sederhana:

$$= P - P$$

dalam satuan harga.

Jika:

$$P - P_t$$

maka execution lebih buruk.

Dalam return:

$$= P - P_t$$

P.13. Total execution cost

Kita dapat memecah:

$$= \int_t^T \text{cost} dt$$

Perhatikan:

$$dt$$

berasal dari order yang tidak terisi.

Jadi limit order tidak benar-benar "gratis".

P.14. Spread cost

Jika kita membeli menggunakan market order:

$$P_t$$

sedangkan reference price:

$$P$$

Maka:

$$= P_t - P = \text{spread}$$

Untuk sell:

$$P - P_t = \text{spread}$$

P.15. Slippage

Misalkan kita ingin membeli:

$$= 100$$

Tetapi best ask hanya memiliki:

$$40$$

Order kita harus mengambil:

$$40$$

pada level pertama,

sisanya:

$$0$$

pada level berikutnya.

Execution price menjadi weighted average:

$$P_P = P$$

Slippage:

$$= P_P - P_t$$

P.16. Market impact

Jika:

cukup besar relatif terhadap liquidity, order kita sendiri mengubah harga.

Secara sederhana:

$$t$$

dengan:

- : available liquidity,
- 0.

Model yang lebih realistis dapat menggunakan empirical impact function.

P.17. Risk budget sekarang harus memasukkan execution

Sebelumnya:

$$= P_t - P$$

Sekarang entry tidak diketahui secara pasti.

Kita harus menggunakan:

$$P$$

Maka:

$$= P - P_{t,t}$$

Dan constraint kita tetap:

$$1$$

Ini sangat penting.

P.18. Execution price uncertainty

Kita bahkan dapat menganggap:

$$P$$

sebagai random variable.

$$P, P_{t,t}$$

Maka risk menjadi:

$$= P - P$$

Tetapi untuk sistem risk-sensitive, expected value saja belum cukup.

Kita dapat menggunakan:

atau:

P.19. CVaR execution risk

Misalkan:

adalah execution loss.

Kita dapat membatasi:

$$\square$$

Dengan demikian order yang memiliki tail slippage besar dapat ditolak meskipun expected slippage kecil.

P.20. Queue position

Sekarang masuk bagian yang sangat menarik.

Misalkan kita menaruh:

di:

$$P = t$$

Di depan order kita terdapat:

$$t$$

unit.

Agar order kita filled, market harus mengonsumsi:

$$t$$

volume pada harga tersebut, kecuali terdapat cancelation.

Maka:

$$P = P_t$$

P.21. Queue depletion model

Misalkan order consumption mengikuti counting process:

$$t$$

Kita dapat memodelkannya sebagai Poisson:

$${}_t P t$$

Probability order terisi:

$$P_t = P_{t-1}$$

Secara eksplisit:

$$P_t = 1 - {}_{t-1} P_t = e^{-\lambda t}$$

Ini memberikan hubungan:

$$P_t$$

Model Poisson ini adalah **baseline**, bukan asumsi final; order flow nyata biasanya lebih kompleks dan dapat menunjukkan clustering serta self-excitation.

P.22. Cancellation rate

Queue tidak hanya berkurang karena transaksi.

Order di depan kita dapat dibatalkan.

Misalkan:

adalah cancellation intensity.

Maka effective queue depletion:

$$t = t$$

Sehingga:

$$P$$

meningkat jika cancellation rate meningkat.

P.23. Expected time to fill

Dengan intensity:

$$,$$

secara sederhana:

$$t$$

Tetapi dalam sistem nyata kita sebaiknya menggunakan distribusi:

$$t \sim t,$$

P.24. Time-to-fill menjadi decision variable

Sekarang kita dapat menentukan:

$$\tau$$

Misalnya:

$$\tau = 15$$

Jika:

$$\tau,$$

order dianggap tidak lagi menarik.

Tetapi τ sendiri seharusnya dioptimalkan.

P.25. Dynamic order placement

Kita sekarang punya kandidat:

$$\in$$

Misalnya:

$$= \{, 0, 1, 2, \}$$

Dengan:

$$= P - \Delta P$$

P.26. Objective execution

Untuk setiap action:

$$,$$

kita menghitung:

$$= \text{---}$$

Kemudian:

$$=$$

P.27. Tetapi execution dan alpha saling berinteraksi

Misalkan:

$$_5 = 010$$

Spread:

$$002$$

Limit order memberi expected price improvement:

$$0015$$

Tetapi probability fill hanya:

$$0$$

Jika harga diperkirakan naik cepat, waiting cost bisa lebih besar daripada price improvement.

Jadi execution optimizer harus mengetahui:

$$\square$$

P.28. Alpha decay

Misalkan expected return:

$$\tau$$

berkurang terhadap waktu:

$$\tau = 0^{-\tau}$$

Parameter:

adalah alpha decay rate.

Jika:

besar, kita harus cepat.

Jika:

kecil, kita dapat menunggu limit fill.

P.29. Ini menghasilkan trade-off yang sangat elegan

Limit order memberikan:

Tetapi menunggu memberikan:

Maka:

$$ttt = Pt - tt$$

dengan waiting cost bergantung pada:

P.30. Opportunity cost akibat non-fill

Misalkan jika tidak filled dalam waktu τ , kita kehilangan expected move:

$$\Delta P_\tau$$

Maka:

$$\tau = P, \tau \Delta P_\tau$$

Untuk long dan expected move positif, waiting bisa sangat mahal.

P.31. Limit order placement sebagai optimasi

Kita dapat memilih:

sebagai jarak limit dari mid:

$$P = P -$$

Kemudian:

$$=$$

Dengan:

$$= 1 -$$

Ini merupakan salah satu core problem Tahap P.

P.32. Sekarang kita bisa membuat execution QUBO

Diskretkan:

$$\in \{0, 1, 2, \dots\}$$

Definisikan binary:

$$\in \{0, 1\}$$

Dengan:

$$_{=0} = 1$$

Kemudian:

$$_{=0} =$$

P.33. Objective

Jika:

$$= ,$$

maka:

$$= - P^2 - 1$$

Ini sudah menjadi QUBO.

P.34. Jika kita juga mengoptimalkan order size

Sekarang:

juga menjadi decision variable.

Misalnya:

$$\in \{0,25, 0,5, 0,75, 1\}$$

Kita dapat menggunakan:

$$,$$

untuk kombinasi:

$$,$$

Kemudian:

$$, = ,$$

P.35. Tetapi quantity dibatasi risk

Constraint:

$$P \leq 1$$

Jika:

$$P$$

random, kita dapat menggunakan conservative estimate:

$$P_t \leq 1$$

Atau:

1

P.36. Multi-asset execution

Sekarang bayangkan:

$$,, , P$$

Kita ingin:

untuk setiap aset.

Total risk:

=

atau jika menggunakan dollar risk:

t

Execution optimizer sekarang harus menentukan:

$$\{, , \}_{=1}$$

P.37. Ini kembali menjadi combinatorial optimization

Karena:

bisa:

$$,$$

dan:

diskrit,

kita mendapatkan:

$\square$

Objective:

$$- - t$$

P.38. Cross-asset market impact

Ini menjadi menarik untuk crypto.

Misalnya kita ingin membeli:

Tetapi:

mengalami sell pressure besar.

Maka expected SOL return dapat berubah.

Dengan factor model:

$$= ,$$

Jika:

berubah, execution decision SOL juga berubah.

P.39. Untuk saham internasional

Kita dapat memperluas factor state:

$$t = , , P , , , ,$$

Misalnya untuk saham teknologi:

$$= , , PP ,$$

Execution optimizer kemudian mengetahui apakah:

cukup besar untuk membayar:

$$t$$

P.40. Ini menghasilkan konsep penting: alpha after execution

Alpha mentah:

Execution cost:

Maka:

$$t = -$$

Tetapi karena fill probability juga penting:

$$t = -1 -$$

Jadi signal yang terlihat sangat bagus dapat menjadi trade yang buruk setelah execution.

P.41. No-trade region

Ini menghasilkan konsep yang sangat penting.

Jika:

,

maka:

□

Dengan kata lain:

Model tidak wajib trading hanya karena ada signal.

Kita dapat mendefinisikan:

$$t \quad tt$$

sebagai syarat minimum untuk melakukan trade.

P.42. Hysteresis execution

Sama seperti Tahap O, jangan terus-menerus memindahkan limit order.

Jika:

— ,

maka:

$$P$$

Dengan demikian sistem tidak mengalami:

P.43. Order lifecycle

Sekarang setiap order mempunyai state machine.

$$\boxed{P \ P}$$

atau:

$$P$$

Atau:

$$P$$

P.44. Mengapa partial fill penting?

Misalkan target:

$$= 100$$

Tetapi hanya:

$$40$$

filled.

Maka kita memiliki:

$$= 40$$

Remaining:

$$= 0$$

Namun setelah 40 filled, market state dapat berubah.

Maka:

tidak boleh dianggap otomatis harus dikejar.

Kita harus melakukan re-optimization.

P.45. Partial fill sebagai decision problem

Setelah partial fill:

$$\boxed{\in \{P, , P, , \}}$$

Dan:

P.46. Adverse selection

Ini salah satu masalah paling penting dalam limit order.

Misalkan kita memasang:

Order kita filled.

Mengapa?

Mungkin karena:

$$P_t$$

justru akan turun.

Artinya:

$$\square$$

sendiri membawa informasi negatif.

Ini disebut **adverse selection**.

P.47. Fill probability tidak selalu positif

Kita sebelumnya menulis:

Tetapi sebenarnya:

$$P_t \geq 0$$

dapat lebih rendah daripada:

$$P_t \geq 0$$

Jadi kita harus memodelkan:

$$P \Delta P \geq 0, t$$

Bukan hanya:

$$P$$

P.48. Execution alpha

Kita dapat mendefinisikan:

$$= \Delta P - \Delta P$$

Jika:

$$0,$$

fill cenderung terjadi ketika market bergerak melawan kita.

Maka limit order tersebut mungkin memiliki adverse selection yang tinggi.

P.49. Objective execution yang lebih lengkap

Sekarang:

$$= , - 1 - , -$$

dan:

$$=$$

Ini jauh lebih realistis daripada:

"limit order selalu lebih baik karena fee lebih murah."

P.50. Hubungan dengan Tahap O

Tahap O menghasilkan:

$$P_{ttt}, \\ , \\ P,$$

Tahap P kemudian mengatakan:

Bagaimana kita mendapatkan tersebut dengan biaya dan risiko execution minimum?

Jadi:

$$P$$

bukan dua modul independen.

P.51. Execution-aware risk

Misalkan Tahap O mengatakan:

$$= 100$$

Tetapi Tahap P memperkirakan:

$$= 0$$

Maka actual risk:

$$t$$

Jika:

$$t \geq 1,$$

maka execution layer harus mengurangi:

Jadi quantity final:

$$= t, t$$

P.52. Risk budget hierarchy

Kita sekarang memiliki:

$$t \geq 1$$

Tetapi:

$$t \leq t$$

Sehingga secara konseptual:

$$t \leq t$$

Ini lebih realistis daripada mengalokasikan seluruh \$1 kepada price movement saja.

P.53. Dynamic execution budget

Misalnya:

$$t = 1$$

Market risk:

$$t = 0.5$$

Maka execution hanya memiliki:

Jika spread/slippage terlalu besar:

t 025,

trade harus:

□

P.54. Ini menghasilkan gate terakhir

Sebelum order dikirim:

t

memeriksa:

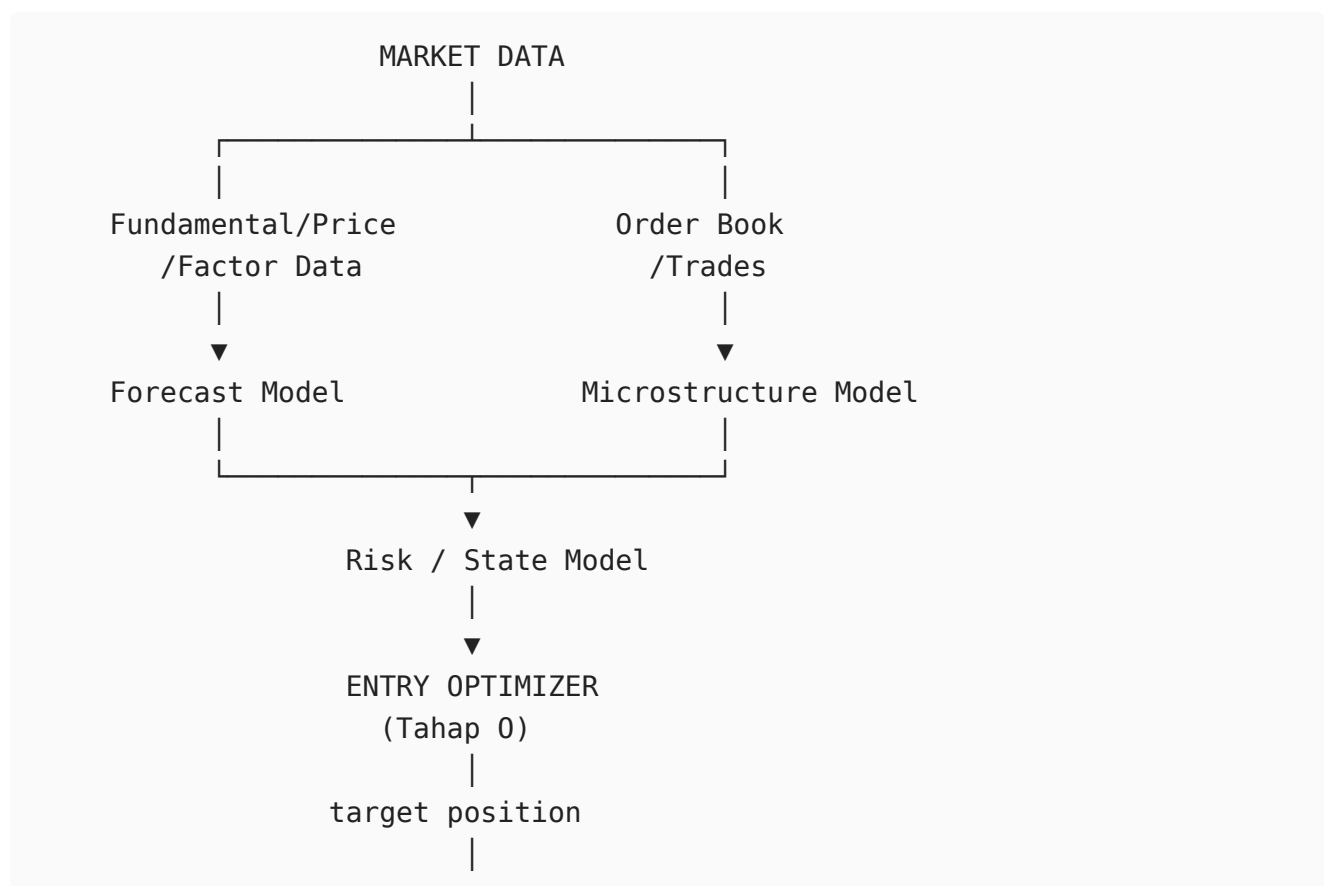
$0, t, t, tt, Pt,$

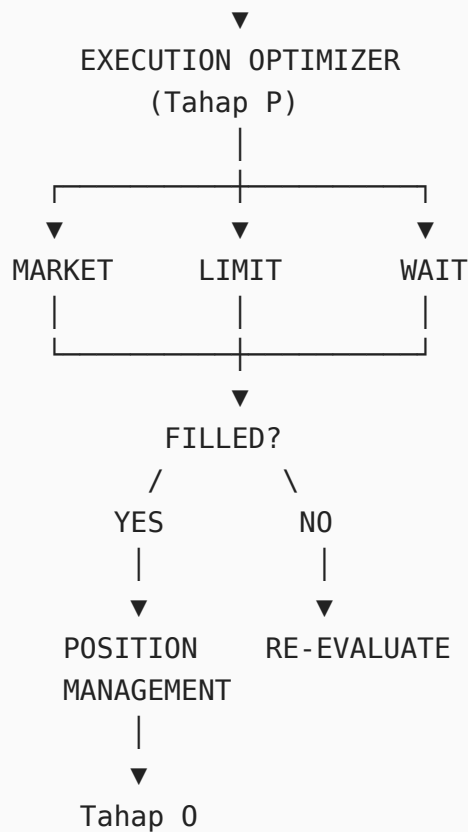
Jika salah satu constraint kritis gagal:

□

P.55. Arsitektur lengkap setelah Tahap P

Sekarang sistem kita menjadi:





P.56. Dan sekarang kita mempunyai tiga lapisan optimasi

Ini mulai menjadi sangat menarik.

Layer 1 — Portfolio / Entry

□

QUBO:

t

Layer 2 — Position Management

□

QUBO:

Layer 3 — Execution

□

QUBO:

P.57. Tiga QUBO

Secara ringkas:

$$t$$

Atau:

$$\square$$

Dan semuanya menggunakan market state yang diperbarui secara periodik.

P.58. Tetapi saya ingin membuat satu perubahan penting pada desain kita

Jangan membuat:

$$t, s$$

sepenuhnya terpisah.

Karena ada coupling.

Contohnya:

mempengaruhi:

Execution mempengaruhi:

$$P_t$$

Entry price mempengaruhi:

SL mempengaruhi:

Risk mempengaruhi:

$$tt$$

Jadi secara ideal:

$$\square$$

dengan feedback.

P.59. Mathematical feedback loop

Secara konseptual:

$$t \quad t \quad t \quad P_t \quad t \quad t$$

Kemudian:

$$t \quad t \quad t \quad t$$

Jadi sistem merupakan:

$$\boxed{- \quad t \quad t}$$

P.60. Ini juga membuat proyek kita cocok untuk eksperimen quantum

Karena kita dapat menguji:

Classical baseline

$$\boxed{t \quad t}$$

untuk problem kecil.

Classical heuristic

$$\boxed{t}$$

$$\square$$

$$\boxed{P}$$

Quantum

$$\square$$

Kemudian dibandingkan berdasarkan:

$$t \quad t, t \quad t, t \quad t, t, t \quad t$$

P.61. Metrik yang harus kita simpan

Untuk setiap order:

$$\boxed{P, P_t, P_{,,,}, t \quad t, t \quad t, t \quad t \quad t}$$

Ini akan menjadi **dataset execution** kita.

P.62. Ini membuka kemungkinan self-improving execution model

Setelah banyak trade, kita dapat mengestimasi:

$$P \quad tt,$$

Kemudian:

$$tt,$$

Kemudian:

$$\Delta P \quad , tt$$

Sehingga execution model semakin baik.

Jadi:

$$\boxed{t \quad tt \quad ttt \quad tttt}$$

P.63. Tetapi jangan langsung menggunakan reinforcement learning

Untuk tahap awal, saya justru menyarankan:

$$\boxed{ttt \quad tt}$$

daripada:

Karena kita dapat menjelaskan dengan jelas:

$$\begin{array}{c} P, \\ , \\ t, \\ tt, \end{array}$$

dan objective function.

RL dapat kita pertimbangkan jauh kemudian jika diperlukan.

P.64. Pipeline matematis Tahap P

Saya akan merangkumnya sebagai:

$$t$$

$$P_{,t}$$

$$,t$$

$$\Delta P_{,t}$$

$$\square$$

$$\square$$

$$\square$$

$$Pt$$

P.65. Satu persamaan yang merangkum Tahap P

Jika saya harus memilih satu persamaan inti:

$$t = \in \{ , -1 - , - \}$$

Ini adalah inti **execution optimization** kita.

P.66. Dan constraint paling penting tetap:

$$\square 1$$

Bahkan setelah memasukkan:

,

,

tt ,

,

dan:

tt

Jadi target awal kita:

"setiap trade maksimal berisiko \$1"

sekarang menjadi jauh lebih rigor.

Bukan lagi:

$$P_t - = 1,$$

tetapi kira-kira:

$$P - t t 1$$

P.67. Posisi sistem kita setelah Tahap P

Kita sekarang memiliki:

$$= t t P t t P t t$$

dan Tahap P menambahkan:

$$= t P t t t$$

Sehingga sistem akhirnya mulai menyerupai arsitektur **institutional systematic trading**:



dengan optimasi kombinatorial di beberapa lapisan.

Hal yang paling penting dari Tahap P

Ada satu perubahan filosofis yang menurut saya sangat penting untuk proyek kita:

Jangan menganggap order sebagai instruksi. Anggap order sebagai keputusan ekonomi yang memiliki probabilitas, biaya, dan konsekuensi.

Dengan begitu:



bukan lagi sebuah aksi sederhana.

Ia menjadi:

$$, , t t , t , t$$

yang seluruhnya merupakan **decision variables**.

Dan karena sebagian besar variabel tersebut dapat didiskretkan, kita kembali mendapatkan problem:



Posisi kita sekarang

Arsitektur besarnya sudah menjadi:

Tahap berikutnya, Tahap Q, sebaiknya kita gunakan untuk membangun bagian yang selama ini menjadi "jembatan" antara semua model tersebut: **Backtesting Engine & Event-Driven Simulation**. Kita perlu membuat dunia virtual yang benar-benar mensimulasikan urutan **data masuk** → **signal** → **QUBO** → **order** → **partial fill/non-fill** → **slippage** → **posisi** → **dynamic SL/TP** → **exit**, tanpa *look-ahead bias* dan tanpa *survivorship bias*. Ini krusial sebelum satu baris algoritma quantum pun dianggap memiliki performa trading yang valid.

Prompt:

8/17/2026, 10:02:18 PM

lanjut Tahap Q

Response:

8/17/2026, 10:02:18 PM

Tahap Q — Event-Driven Backtesting & Market Simulator

Ya. Tahap Q adalah titik yang sangat penting, karena seluruh matematika A–P yang kita bangun sekarang harus ditempatkan ke dalam lingkungan yang bisa menjawab pertanyaan paling fundamental:

"Jika sistem ini benar-benar berada di pasar pada waktu t , dengan hanya informasi yang tersedia pada saat itu, apa yang sebenarnya akan terjadi?"

Tanpa ini, model kita mudah menghasilkan backtest yang terlihat luar biasa tetapi sebenarnya menggunakan *look-ahead bias*, mengabaikan spread, menganggap semua limit order pasti filled, atau menghitung stop-loss dengan informasi masa depan.

Jadi Tahap Q bukan sekadar "membuat backtester".

Kita akan membangun:

□

yang menjadi **laboratorium virtual** untuk seluruh sistem.

Q.1. Paradigma utama

Backtester sederhana biasanya berpikir seperti:

$$P_t \quad t \quad t_{t1}$$

Itu terlalu sederhana untuk sistem kita.

Kita ingin:

$$\boxed{t \quad tt_t \quad tt_t \quad t \quad t_t \quad Pt_t \quad P_t}$$

dan kemudian:

$$\boxed{tt_{t1}}$$

bergantung pada apa yang benar-benar terjadi pada t .

Q.2. Event-driven, bukan candle-driven

Ini perbedaan fundamental.

Misalnya kita mempunyai candle:

$$= 100, = 105, = , = 10$$

Jika strategi memasang:

$$=$$

dan:

$$= ,$$

kita **tidak bisa langsung mengatakan:**

"Low = 97, jadi limit order filled lalu stop-loss terkena."

Karena kita tidak tahu urutan intrabar.

Bisa saja:

$$100 \quad 105 \quad 10$$

atau:

$$100 \quad 105 \quad 10$$

Konsekuensinya berbeda.

Q.3. Urutan kejadian adalah informasi

Misalkan:

$$P = 105,$$

=

Dalam satu candle:

= 105, =

Apakah TP atau SL terjadi terlebih dahulu?

Data OHLC saja tidak dapat menjawab.

Karena itu:

t, t

Q.4. Hierarki data

Untuk sistem kita, idealnya data dibagi menjadi:

Level 1 — Macro

$,, P, , , ,$

Level 2 — Market

Level 3 — Microstructure

$,, t, ,$

Level 4 — Event

$t, , , t, P$

Q.5. Informasi filtration

Ini bagian matematis yang sangat penting.

Misalkan:

t

adalah seluruh informasi yang tersedia sampai waktu t .

Strategi harus memenuhi:

$t \in t$

Artinya keputusan pada waktu t **tidak boleh bergantung pada informasi dari $t + \Delta t$.**

Ini disebut *adapted strategy*.

Q.6. Larangan utama

Tidak boleh:

$$t = P_{t1}$$

Tidak boleh pula:

$$t = tt1, tt1$$

Keputusan hanya boleh:

$$\boxed{t = t}$$

Q.7. Mengapa ini sangat penting untuk sistem kita?

Karena model kita memiliki:

$$\begin{aligned} &t, \\ &t, \\ &P, \\ &t, \\ &t, \\ &P_t \end{aligned}$$

Semua harus dihitung berdasarkan:

$$\boxed{t}$$

dan bukan berdasarkan seluruh dataset.

Q.8. Rolling information set

Misalnya kita melakukan backtest pada:

$$2021 - 01 - 01 \quad 202 - 12 - 1$$

Pada:

$$t = 2022 - 05 - 01,$$

model hanya boleh mengetahui:

$$2021 - 01 - 01 \quad 2022 - 05 - 01$$

Bukan:

$$2022 - 05 - 02 \quad 202 - 12 - 1$$

Q.9. Training → validation → test

Dataset harus dibagi secara temporal.

Misalnya:

$$= 2021 - 2022$$

$$t = 2021$$

$$t = 2022 - 4$$

Tetapi untuk sistem yang adaptif, kita sebaiknya melangkah lebih jauh:

$$t$$

Q.10. Walk-forward

Misalkan:

$$_1 \quad t_0, t_1 \quad t_1, t_2$$

Kemudian:

$$_2 \quad t_0, t_2 \quad t_2, t$$

Kemudian:

$$t_0, t \quad t, t_4$$

Sehingga:

$$\square$$

Q.11. Purgings

Jika target prediction menggunakan horizon:

dan sample training berdekatan dengan validation, dapat terjadi overlap.

Misalnya:

$$t \quad t_{10}$$

Kemudian validation dimulai pada:

$$t \quad 5$$

Maka informasi target overlap.

Karena itu kita dapat menggunakan:

$$P$$

Q.12. Embargo

Setelah training set berakhir, kita dapat memberikan gap:

$$t_t, t_{tt, tt}$$

untuk mengurangi leakage akibat dependensi temporal.

Q.13. Market simulator

Sekarang kita definisikan simulator:

$$\square$$

yang menerima:

$$tt_t, t_t, t_t$$

dan menghasilkan:

$$tt_{t1}, t_{t1}, t$$

Secara matematis:

$$t1 = t, t, t$$

Q.14. State

State sistem:

$$t = \{t, t, t, P_t, t, t\}$$

Dengan:

- t : market state,
- t : factor state,
- t : open orders,
- P_t : positions,
- t : risk state,
- t : capital.

Q.15. Event

Event:

$$t \in$$

Misalnya:

$$= \{tt, , t, , , t, Pt, \}$$

Q.16. Event queue

Simulator mempunyai:

$$\boxed{Pt}$$

berdasarkan timestamp.

Misalnya:

```
09:30:00.000 MarketUpdate
09:30:00.005 Signal
09:30:00.006 OrderSubmit
09:30:00.120 Trade
09:30:00.121 PartialFill
09:30:02.000 Trade
09:30:02.001 Fill
09:31:15.000 Timer
```

Simulator memproses satu per satu.

Q.17. Event loop

Secara konseptual:

```
while event_queue not empty:

    event = next_event()

    update_market_state(event)

    process_open_orders()

    process_positions()
```

```

if decision_event:
    generate_signal()

if optimization_event:
    solve_QUBO()

if execution_event:
    place_or_modify_orders()

update_PnL()

update_risk()

record_state()

```

Ini adalah jantung backtester.

Q.18. Timestamp discipline

Setiap data harus memiliki:

tt

Dan timestamp harus konsisten:

atau timezone yang jelas.

Tidak boleh mencampur:

0

dengan:

tanpa konversi.

Q.19. Clock model

Kita dapat menggunakan:

$t_0 \ t_1 \ t_2 \ t$

Setiap event:

$= t, t, t$

Kemudian:

$$t_1 \quad t$$

Q.20. Order model

Setiap order kita representasikan sebagai:

$$= , , , t , , tt , , tt , tt , , P$$

Misalnya:

$$= , , , 100000 , 0001 ,$$

Q.21. Order status

$$\boxed{P \quad P}$$

atau:

$$P$$

atau:

$$P$$

Q.22. Market order simulation

Market buy:

$$P = P$$

Market sell:

$$P = P$$

Jika hanya OHLCV tersedia, kita perlu menggunakan model slippage.

Misalnya:

$$P = P_t , ,$$

Q.23. Limit order simulation

Jika:

$$P \quad P_t$$

untuk buy, order berpotensi filled.

Tetapi **tidak otomatis filled**.

Kita harus mempertimbangkan:

Q.24. Queue simulator

Misalkan:

$$t = 500$$

Order kita:

$$= 100$$

Kemudian market sells consume:

$$200$$

Maka:

$$= 00$$

Order kita belum filled.

Jika kemudian:

$$400$$

unit dikonsumsi:

$$00$$

menghabiskan queue depan.

Maka:

$$100$$

dapat mengisi order kita.

Q.25. Partial fill

Jika hanya:

$$0$$

unit yang tersedia setelah queue habis:

$$= 0$$

Remaining:

$$= 0$$

Simulator harus mempertahankan state tersebut.

Q.26. Fill probability model

Jika order book tidak tersedia secara historis, kita dapat membuat:

$$= , t, , , tt, , t$$

Misalnya logistic model:

$$= 0$$

dengan:

$$= 11 -$$

Q.27. Tetapi ada masalah besar

Kita **tidak boleh** melatih:

dengan data masa depan lalu menggunakannya pada masa lalu.

Jadi:

harus diperoleh dari training window.

Kemudian digunakan pada test window.

Q.28. Stop-loss simulation

Misalkan posisi long:

$$P_t = 100$$

SL:

$$5$$

Jika market tick menyentuh:

5,

stop triggered.

Tetapi execution mungkin:

51

atau:

4

karena slippage.

Maka:

$$P_t - P$$

Q.29. Gap risk

Misalkan:

$$= 5$$

Tetapi harga berikutnya:

0

Maka:

$$P_t - 0,$$

bukan:

5

Ini penting untuk saham.

Untuk crypto yang berjalan 24/7, gap masih dapat terjadi secara efektif melalui lonjakan harga, meskipun mekanisme penutupan pasar berbeda.

Q.30. TP juga bukan guaranteed price

Jika:

$$P = 105$$

dan market melompat:

$$10 - 10,$$

execution dapat:

$$P_t = 105$$

Sebaliknya jika liquidity buruk:

$$P_t$$

bisa lebih buruk.

Q.31. PnL

Untuk long:

$$P_t = P_t - P_{t-1}$$

Untuk short:

$$P_t = P_t - P_{t-1}$$

Dengan:

$$t = t$$

Q.32. Realized dan unrealized PnL

$$P_t = P_t - P_{t-1}$$

Realized:

$$P$$

hanya berasal dari posisi yang sudah ditutup.

Q.33. Equity curve

Capital:

$$0$$

Maka:

$$t = 0 \quad P_{\tau}$$

Equity curve:

$$t = t$$

Ini yang akhirnya kita gunakan untuk evaluasi strategi.

Q.34. Drawdown

High-water mark:

$$t = \tau t^\tau$$

Drawdown:

$$t = t - tt$$

Maximum drawdown:

$$= t^t$$

Q.35. Tetapi Sharpe Ratio saja tidak cukup

Kita akan mengukur:

$$= -$$

Tetapi juga:

$$\begin{aligned} & t, \\ & , \\ & , \\ & Ptt, \\ & t, \\ & t, \end{aligned}$$

Q.36. Trade expectancy

Jika:

$$\begin{aligned} & = P, \\ & = , \\ & = , \end{aligned}$$

maka:

$$= -1 -$$

Namun untuk sistem kita, expectancy harus sudah memasukkan:

$$,$$

$$t$$

Q.37. Contoh penting

Strategi A:

$$t = 0$$

Strategi B:

$$t = 40$$

Belum tentu A lebih baik.

Misalnya:

A

$$= 1$$

$$= 2$$

Expectancy:

$$01 - 042 = -02$$

B

$$=$$

$$= 1$$

$$04 - 01 = 0$$

Jadi:

$$\boxed{t \quad ttt}$$

Q.38. Risk-adjusted performance

Karena kita menetapkan:

$$t \quad 1,$$

kita juga dapat mengukur:

$$\boxed{t}$$

Misalnya:

$$P$$

Ini lebih relevan terhadap filosofi sistem kita.

Q.39. Transaction Cost Analysis

Tahap Q juga harus memecah PnL:

□

menjadi:

— — — — t — ttt

Sehingga:

t

dapat dihitung.

Q.40. Ini akan sangat penting

Bayangkan:

$$t = 20$$

Tetapi:

$$tt = 1$$

Maka:

$$t = 2$$

Model tersebut secara praktis hampir tidak berguna.

Q.41. Sensitivity analysis

Kita tidak boleh hanya menjalankan satu konfigurasi.

Misalnya:

$\{05, 1, 2\}$

$\{05, 1, 2\}$

$\{05, 1, 2\}$

Kemudian lihat apakah strategi tetap profitable.

Q.42. Stress testing

Market regime:

$$= \{, , t, , \}$$

Backtester harus mengukur:

$$P$$

Q.43. Contoh

Jika:

$$= 21$$

tetapi:

$$= -40,$$

kita harus memahami mengapa.

Bukan sekadar mengatakan:

"Sharpe total = 1.7."

Q.44. Monte Carlo trade reshuffling

Setelah memperoleh sequence:

$$1, 2, , ,$$

kita dapat melakukan bootstrap.

Misalnya:

$$10,000$$

simulasi urutan trade.

Kemudian hitung:

Sehingga diperoleh distribusi:

$$P$$

Q.45. Ini membantu menjawab:

"Apakah drawdown terbesar yang saya lihat memang representatif?"

Misalnya backtest menunjukkan:

$$= 12$$

Tetapi Monte Carlo menunjukkan:

$$5 = 24$$

Maka strategi sebenarnya jauh lebih berisiko daripada yang tampak dari satu historical path.

Q.46. Parameter uncertainty

Kita juga harus mempertimbangkan:

bukan:

Misalnya:

, , ,

Semua memiliki uncertainty.

Maka:

□

harus masuk dalam evaluasi.

Q.47. Optimizer uncertainty

Ini bahkan lebih relevan untuk quantum.

Misalkan:

menghasilkan solusi:

Tetapi QAOA hanya memperoleh:

Kita harus mengukur:

$$= -$$

Q.48. Quantum trading backtest

Ini akan menjadi salah satu bagian paling menarik dari proyek kita.

Untuk setiap problem:

$$t$$

kita dapat menjalankan:

Exact

$$tt$$

Classical heuristic

$$t$$

QAOA

$$t$$

Kemudian:

$$P^t,$$

$$P,$$

$$P$$

Q.49. Tetapi jangan langsung membandingkan PnL

Misalnya QAOA menghasilkan PnL:

$$10$$

Classical:

$$12$$

Belum tentu QAOA buruk.

Kita harus mengetahui:

$$,$$

$$,$$

$$,$$

Q.50. Approximation ratio

Untuk minimization:

$$\boxed{=} t$$

dengan definisi yang harus disesuaikan dengan konvensi objective.

Untuk maximization:

$$\boxed{= t}$$

Jika:

$$1,$$

solusi semakin dekat optimum.

Q.51. Yang lebih penting: economic degradation

Kita dapat mendefinisikan:

$$\boxed{\Delta P = P_t - P}$$

Kemudian:

$$\boxed{tt}$$

adalah:

$$= \Delta P$$

Jadi kita tidak hanya mengatakan:

| "QAOA mempunyai approximation ratio 95%."

Tetapi:

| "Approximation error 5% tersebut menyebabkan penurunan expected PnL sebesar X."

Ini jauh lebih bermakna secara finansial.

Q.52. Event-driven backtest architecture

Saya menyarankan struktur:

```
TradeProject/  
|  
├─ data/  
|   ├── raw/  
|   ├── processed/  
|   └─ features/  
|  
├─ market/  
|   └─ market_state.py
```

```
|   ├── order_book.py
|   └── factor_state.py
|
|   ├── strategy/
|   │   ├── alpha.py
|   │   ├── signal.py
|   │   └── regime.py
|   |
|   ├── optimization/
|   │   ├── portfolio_qubo.py
|   │   ├── entry_qubo.py
|   │   ├── management_qubo.py
|   │   └── execution_qubo.py
|   |
|   ├── execution/
|   │   ├── order.py
|   │   ├── fill_model.py
|   │   ├── slippage.py
|   │   └── impact.py
|   |
|   ├── portfolio/
|   │   ├── position.py
|   │   ├── pnl.py
|   │   └── risk.py
|   |
|   ├── backtest/
|   │   ├── engine.py
|   │   ├── event.py
|   │   ├── clock.py
|   │   └── simulator.py
|   |
|   ├── evaluation/
|   │   ├── metrics.py
|   │   ├── drawdown.py
|   │   ├── monte_carlo.py
|   │   └── stress_test.py
|   |
|   └── experiments/
```

Q.53. Satu prinsip arsitektur yang sangat penting

Strategy tidak boleh mengetahui masa depan.

Jadi jangan membuat:

```
strategy(data)
```

dengan:

```
data = entire_dataset
```

lalu strategy mengambil apa saja.

Lebih aman:

```
strategy.on_event(event, state)
```

dan:

```
state
```

hanya berisi informasi sampai:

t

Q.54. Data leakage audit

Kita harus memiliki pemeriksaan:

$t_{tt} \ tt$

Untuk setiap feature:

$,t$

harus memenuhi:

$,t \in t$

Q.55. Contoh bug yang sangat berbahaya

Misalkan kita menghitung:

20

menggunakan:

```
rolling(20).mean()
```

itu relatif aman.

Tetapi jika kita menggunakan:

```
center=True
```

maka feature pada t dapat menggunakan:

$$P_{t1}, P_{t2},$$

dan backtest menjadi invalid.

Q.56. Bug lain: normalization

Jika kita melakukan:

$$t = t -$$

dengan:

dihitung menggunakan seluruh dataset, maka future information bocor.

Harus:

$$t = t - \text{tttt}$$

Q.57. Bug lain: covariance

Dalam portfolio optimizer kita menggunakan:

$$t$$

Jangan menghitung:

$$2021 - 202$$

untuk membuat keputusan pada:

$$2021$$

Harus:

$$t = t - t$$

Q.58. Bug lain: universe selection

Misalnya memilih saham berdasarkan:

"saham yang sekarang masih aktif dan performanya bagus."

Kemudian digunakan untuk backtest 2015.

Ini:

□

Universe harus direkonstruksi berdasarkan informasi yang tersedia pada saat itu.

Q.59. Bug lain: delisting

Jika saham hilang dari dataset karena bangkrut, lalu kita hanya menyimpan saham yang masih ada, hasil backtest terlalu optimistis.

Karena itu dataset historical harus mempertahankan:

tt

Q.60. Bug lain: corporate actions

Untuk saham:

$t, , t,$

harus diperhatikan.

Jika tidak, return dapat salah.

Q.61. Crypto memiliki masalah yang berbeda

Crypto:

- 24/7,
- exchange fragmentation,
- funding rate,
- liquidation,
- perpetual futures,
- maker/taker fee,
- spread yang berubah cepat.

Jadi simulator crypto perlu:

□

Q.62. Jika menggunakan perpetual futures

PnL tidak hanya:

$$P$$

Tetapi:

$$P = P_P - P - -$$

Funding:

$$P = P_t$$

dengan konvensi tanda yang sesuai dengan posisi long/short dan funding rate.

Q.63. Untuk saham internasional

Kita perlu:

Misalnya saham Jepang:

$$P$$

Tetapi account base:

Maka:

$$P = P_P - P$$

Ini membuat faktor:

dan FX exposure menjadi bagian dari risk model.

Q.64. Gold dan USD dominance

Karena kita sebelumnya ingin sistem lintas pasar, state global dapat:

$$t = , , P, , , ,$$

Kemudian:

$$= t$$

Q.65. Event-driven factor update

Jika:

berubah pada:

$$t,$$

maka factor state langsung berubah:

$$t \rightarrow t+1$$

Kemudian:

$$t$$

dihitung ulang.

Dengan demikian:

$$t \rightarrow t+1 \rightarrow t+2$$

secara dinamis.

Q.66. Re-optimization

Kita tidak perlu menjalankan QUBO pada setiap tick.

Kita dapat menetapkan:

$$\Delta t_t$$

Misalnya:

$$\Delta t_t = 1$$

Tetapi execution dapat berjalan pada resolusi lebih tinggi.

Jadi:

$$t \rightarrow t+1 \rightarrow t+2$$

sementara:

$$t \rightarrow t+1$$

tergantung data.

Q.67. Ini menghasilkan multi-timescale architecture

Misalnya:

1

$t \ 1$

5

$t \ 1$

atau bahkan:

Q.68. Ini sangat cocok dengan konsep sebelumnya

Kita sebenarnya memiliki:

tt

dan:

ttt

Slow:

’ ’ ’

Fast:

’ ’ ’

Kemudian:

$= tt, ttt$

Q.69. Akhirnya kita mendapatkan hierarchical control

Secara konseptual:

$\square$

menentukan:

$\square$

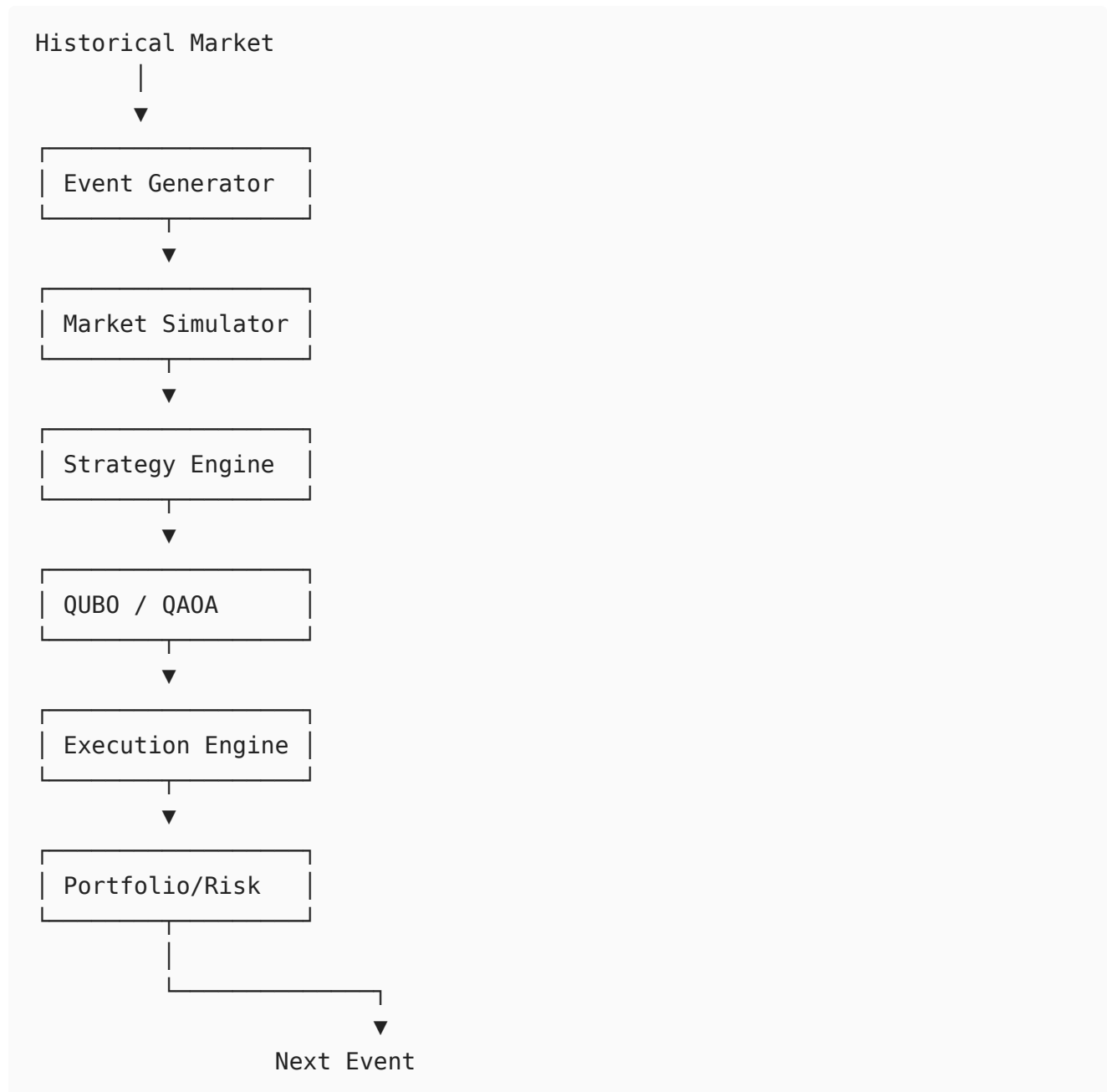
menentukan:

$\square$

menentukan:

Q.70. Dan semuanya diuji oleh simulator

Jadi:



Q.71. Kriteria bahwa backtester kita sudah "benar"

Saya tidak ingin kita mengatakan:

"Backtester selesai karena grafik equity curve sudah muncul."

Backtester baru layak dipercaya jika kita bisa membuktikan:

1. No look-ahead

$$t \in t$$

2. No survivorship bias

Universe historical valid.

3. Realistic execution

$$P \ P$$

4. Realistic limit fill

$$P \ 1$$

5. Transaction costs

$$t$$

6. Partial fills

Didukung.

7. Dynamic SL/TP

Didukung.

8. Multi-asset capital constraint

Didukung.

9. Walk-forward

Didukung.

10. Monte Carlo/stress test

Didukung.

Q.72. Unit test yang harus kita buat

Sebelum menjalankan strategi besar, kita buat market kecil yang kita **tahu jawabannya**.

Misalnya:

$$P_0 = 100,$$

kemudian:

$$100 \ 101 \ 102$$

Jika:

$$= 100,$$

$$= 102,$$

maka:

$$P = 2$$

Simulator harus menghasilkan tepat itu.

Kemudian tambahkan:

Kemudian:

Kemudian:

$$t$$

Dengan demikian setiap komponen dapat diverifikasi.

Q.73. Invariant risk

Simulator harus selalu memeriksa:

$$\square$$

Jika:

$$1$$

untuk sebuah trade, sistem harus menghasilkan error atau reject.

Bukan diam-diam melanjutkan.

Q.74. Capital invariant

Jika capital:

$$t$$

maka:

$$t \quad t$$

atau sesuai leverage/margin rules.

Tidak boleh:

$$P t \quad t$$

kecuali leverage memang dimodelkan.

Q.75. Position invariant

Misalnya strategi melarang short.

Maka:

$$t \geq 0$$

Jika futures:

$$t \in \mathbb{Z}$$

Semua constraint harus eksplisit.

Q.76. Audit trail

Setiap keputusan harus dapat dijelaskan.

Misalnya:

2026-08-17 10:00

Signal:
LONG SOL

Expected return:
0.31%

Risk:
\$0.73

QUBO solution:
BUY = 1

Execution:
LIMIT @ 182.40

Fill probability:
0.71

Queue:
1,243 SOL

Expected slippage:
0.004%

```
Decision:
PLACE LIMIT
```

Ini akan sangat berguna ketika kita melakukan debugging.

Q.77. Bahkan QAOA harus diaudit

Misalnya:

```
QAOA

n_qubits = 12
p = 2
shots = 4096

best bitstring:
101001101010

objective:
-17.42

exact optimum:
-17.91

approximation ratio:
0.973
```

Kemudian:

t

dari solusi tersebut dicatat.

Q.78. Backtest report

Setiap eksperimen sebaiknya menghasilkan:

```
Experiment ID
Data Version
Feature Version
Model Version
QUBO Version
Optimizer Version
Transaction Cost Model
```


Random Seed
Start Date
End Date

Sehingga eksperimen:

$$1$$

dapat direproduksi.

Q.79. Ini penting untuk quantum

Karena QAOA stochastic.

Misalnya:

$$= 1$$

menghasilkan:

$$P = 14$$

Sedangkan:

$$= 2$$

menghasilkan:

$$P =$$

Kita tidak boleh memilih seed yang paling bagus.

Harus:

$$\boxed{P}$$

dan:

$$P$$

Q.80. QAOA sebagai stochastic optimizer

Secara konseptual:

$$= t,$$

dengan:

adalah stochastic measurement noise.

Maka:

$$P$$

bukan deterministic.

Backtester harus mampu menangani hal tersebut.

Q.81. Ini membawa kita ke metrik yang sangat menarik

Bukan hanya:

$$P$$

Tetapi:

$$Pt$$

atau:

$$ttt$$

Misalnya:

$$_t = P - Pt$$

Ini jauh lebih dekat dengan pertanyaan:

| Apakah optimasi quantum benar-benar memberikan nilai ekonomi?

Q.82. Tahap Q mengubah proyek kita

Sebelum Q:

| "Kita mempunyai model trading yang kompleks."

Setelah Q:

| "Kita mempunyai eksperimen yang dapat diverifikasi terhadap market timeline."

Itu perbedaan besar.

Q.83. Hubungan seluruh tahapan

Sekarang pipeline kita secara konseptual:

$$P$$

dan Q bertindak sebagai **validation environment**.

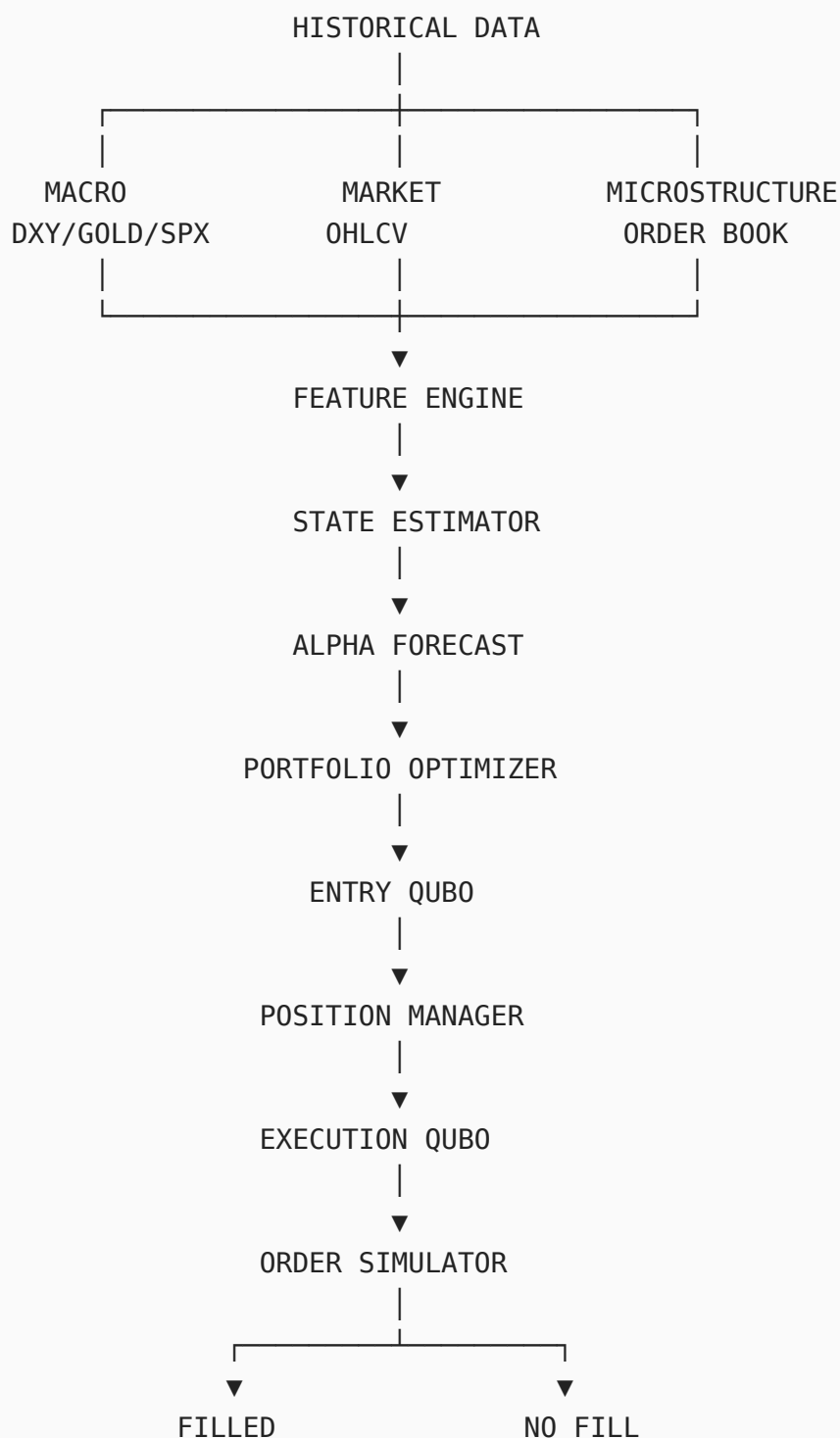
Lebih tepatnya:

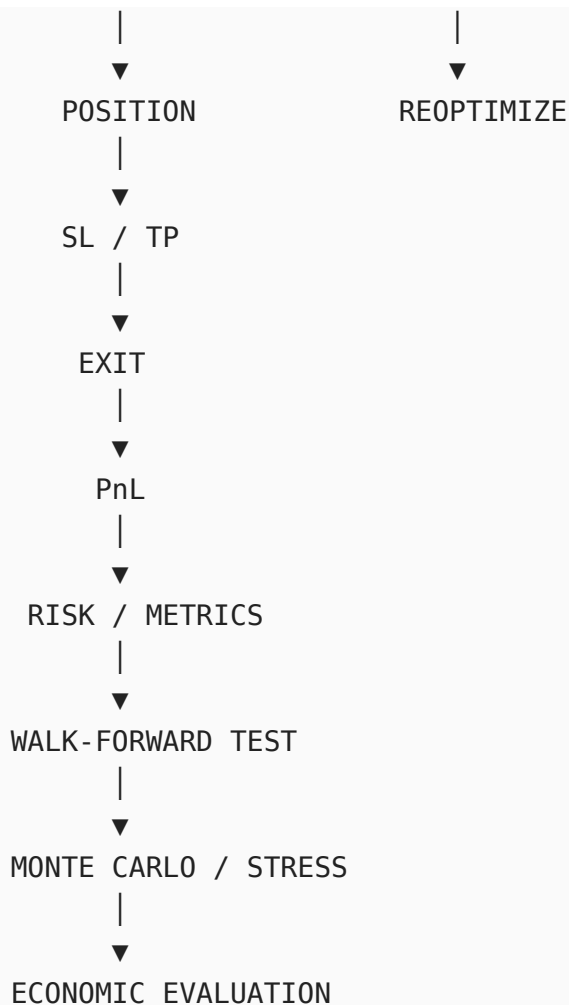
P

dalam arti simulator menjalankan seluruh komponen tersebut dalam waktu historis.

Q.84. Arsitektur final sementara

Saya akan menggambarannya seperti ini:





Q.85. Dan sekarang kita sampai pada satu titik yang sangat menarik

Kita sebenarnya telah mengubah masalah:

"Buat trading bot."

menjadi:

□

dengan state:

t ,

action:

t ,

constraint:

$t, t \geq 0$,

objective:

$$= \begin{matrix} t \\ t \end{matrix} t, t$$

Tetapi kita belum perlu menyebutnya reinforcement learning.

Kita dapat menyelesaikan setiap keputusan menggunakan optimizer yang sesuai.

Q.86. Formulasi matematis keseluruhan

Secara abstrak:

$$t_1 = t, t, t$$

dengan:

$$t = \in_t t,$$

Reward:

$$t = P_t - t - t$$

Kemudian objective:

$$\begin{matrix} t \\ t=0 \end{matrix} t$$

Ini adalah fondasi matematis yang nantinya memungkinkan kita, **jika memang diperlukan**, mengubah beberapa modul menjadi MDP/RL.

Q.87. Tetapi untuk sekarang saya akan mempertahankan prinsip

$$tt$$

Kita ingin setiap komponen memiliki interpretasi finansial dan matematis:

Dengan demikian apabila sistem gagal, kita dapat mengetahui:

di mana ia gagal.

Q.88. Tahap Q selesai jika kita mampu melakukan eksperimen ini

Misalnya:

202 – 01 – 01 202 – 12 – 1

dan simulator menghasilkan:

Capital	
Initial	\$1,000
Final	\$1,137
Trades	428
Win rate	47.4%
Profit factor	1.31
Sharpe	1.42
Sortino	1.96
Max DD	8.7%
Avg risk	\$0.73
Max risk	\$1.00
Avg slippage	0.031%
Avg fee	0.040%
Limit fills	62.1%
Partial fills	14.7%

Tetapi yang jauh lebih penting:

setiap angka tersebut harus dapat ditelusuri kembali ke event dan keputusan yang menghasilkan angka itu.

Q.89. Deliverable Tahap Q

Jadi implementasi nyata Tahap Q nantinya harus menghasilkan minimal:

Engine

ttt

Market simulator

tt

Order simulator

$$t$$

Fill model

$$\square$$

Cost model

$$tt$$

Portfolio

$$Pt$$

Risk

$$\square$$

Evaluation

$$P$$

Validation

$$\square$$

Robustness

$$t$$

Q.90. Dan satu aturan emas

Jangan optimasi QAOA terhadap historical PnL secara langsung terlebih dahulu.

Pertama:

$$Ptt \ t$$

kemudian:

$$t \ tt$$

kemudian:

$$tt$$

baru:

$$\square$$

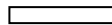
Dengan demikian jika QAOA menghasilkan hasil buruk, kita bisa mengetahui apakah masalahnya berasal dari:

- data,
 - signal,
 - formulasi QUBO,
 - optimizer,
 - execution,
 - atau memang QAOA.
-

Posisi kita sekarang

Tahap Q memberi kita "**physics engine**" untuk eksperimen trading.

Secara keseluruhan:



dan loop tersebut berjalan:

$$t = 0, 1, 2, \dots$$

Itulah yang membuat sistem kita berbeda dari sekadar script trading yang membaca candle lalu membeli/jual.

Tahap berikutnya, Tahap R, menurut saya sebaiknya menjadi *Robustness, Statistical Validation & Anti-Overfitting*. Setelah kita memiliki simulator event-driven, kita perlu menjawab pertanyaan yang jauh lebih keras: *apakah edge yang kita lihat benar-benar ada, atau hanya artefak dari parameter tuning?* Di sana kita bisa masuk ke **Deflated Sharpe Ratio, Probability of Backtest Overfitting, White's Reality Check, stationary/block bootstrap, purged walk-forward, regime-conditioned performance, multiple-hypothesis correction**, dan akhirnya menentukan apakah suatu hasil QAOA/classical optimizer layak disebut *genuine statistical edge*.

Prompt:

8/17/2026, 10:03:38 PM

lanjut Tahap R

Response:

8/17/2026, 10:03:39 PM

Tahap R — Robustness, Statistical Validation & Anti-Overfitting

Ya. Kalau Tahap Q adalah "**membangun dunia virtual tempat strategi diuji**", maka Tahap R adalah "**membuktikan bahwa hasil yang terlihat di dunia virtual itu bukan kebetulan.**"

Ini justru salah satu tahap paling penting untuk sistem kita.

Karena kita sekarang memiliki banyak komponen:

maka jumlah derajat kebebasan kita juga sangat besar.

Dan semakin banyak derajat kebebasan, semakin mudah kita secara tidak sengaja menemukan strategi yang **terlihat bagus hanya karena kita telah mencoba terlalu banyak hal.**

R.1. Masalah fundamental: backtest yang bagus belum tentu strategi yang bagus

Misalkan kita melakukan:

$$= 10000$$

eksperimen.

Kita menemukan satu strategi:

$$= 1$$

Apakah:

$$1$$

benar?

Belum tentu.

Karena jika kita mencoba cukup banyak strategi, sebagian akan terlihat bagus **secara kebetulan.**

Ini disebut:

$$\square$$

R.2. Contoh sederhana

Misalkan seluruh strategi sebenarnya tidak mempunyai alpha:

$$= 0$$

Kita mencoba 1.000 strategi.

Secara statistik, sangat mungkin beberapa memiliki:

2,

bahkan:

Kemudian kita mengambil strategi terbaik.

Kita akan mengatakan:

"Lihat! Sharpe = 3.2!"

Padahal:

□

R.3. Inilah musuh utama kita

Ada beberapa bentuk:

tt

t

t

tt

Pt

tt

dan:

$tttt$

R.4. Kita perlu membedakan dua konsep

In-sample performance

adalah performa pada data yang digunakan untuk membangun model.

Out-of-sample performance

adalah performa pada data yang **tidak digunakan** untuk memilih model.

Tujuan kita:

$$\boxed{P \ P}$$

bukan:

$$P \ 0$$

R.5. Tetapi kita bahkan tidak boleh hanya menggunakan satu OOS

Misalkan:

$$2021 - 202$$

digunakan sebagai training.

Lalu:

$$2024$$

sebagai test.

Jika kita terus mengubah model berdasarkan hasil 2024, maka:

$$2024$$

secara perlahan berubah menjadi training data.

Karena itu kita membutuhkan:

$$\boxed{tt}$$

R.6. Walk-forward validation

Misalkan:

Train 1	Test 1
_____	_____
2021	2022 Q1
Train 2	Test 2
_____	_____
2021-Q1 2022	2022 Q2
Train 3	Test 3

dan seterusnya.

Model selalu:

$P_{t:t}$

bukan:

$P_t P_t$

R.7. Purged walk-forward

Karena target kita dapat memiliki horizon:

Misalnya:

$= 1$

Jika sample training terakhir berada terlalu dekat dengan validation, targetnya bisa overlap.

Kita buang bagian tersebut.

Jika training berakhir:

$t_t,$

validation dimulai:

$t_{t:t},$

maka:

$t_{t:t} - t_t$

R.8. Embargo

Kita juga dapat menetapkan:

0

sebagai embargo.

Sehingga:

$t_t, t_{t:t,t}$

memiliki gap.

Secara sederhana:



R.9. Mengapa ini sangat relevan untuk sistem kita?

Karena kita mempunyai feature seperti:

$t,$
 $t,$
 $t,$
 $t,$
 $t,$
 $t,$

dan microstructure.

Beberapa di antaranya memiliki autocorrelation dan cross-correlation.

Jadi sample kita **tidak independent and identically distributed (IID)**.

R.10. Jangan menggunakan random train-test split

Misalnya:

```
train_test_split(X, y)
```

secara acak.

Ini berbahaya untuk time series.

Karena:

202

dapat berada di training sementara:

2022

berada di test.

Model secara tidak langsung melihat struktur masa depan.

Untuk sistem kita:



R.11. Distribution shift

Pasar tidak stationary.

Secara ideal kita ingin:

$$P_t = P_{tt}$$

Tetapi kenyataannya:

$$\boxed{P_t, P_{tt}}$$

Ini disebut:

$$\boxed{ttt}$$

R.12. Regime

Misalkan:

$$t \in \{, , , , \}$$

Maka:

$$P_{tt}$$

dapat berbeda drastis.

Karena itu performa harus dianalisis:

$$\begin{aligned} &= , \\ &= , \end{aligned}$$

dst.

R.13. Strategi yang hanya bekerja di satu regime

Misalnya:

$$= 5$$

tetapi:

$$= -20$$

Strategi tersebut bukan otomatis buruk.

Tetapi kita perlu memahami:

$$\boxed{t}$$

Mungkin strategi memang dirancang untuk:

R.14. Regime detector

Kita dapat mendefinisikan:

$$t = t$$

dengan faktor:

$$t = t, t, t, t, P_t,$$

Kemudian strategi:

$$t = t, t$$

R.15. Tetapi jangan overfit regime

Ini bahaya.

Misalnya kita menemukan:

"Strategi bekerja bagus jika VIX antara 17.23 dan 21.87."

Kemudian:

"DXY harus naik 0.17%."

Kemudian:

"Gold harus turun 0.08%."

Kita telah membuat filter yang sangat spesifik.

Itu kemungkinan besar:

$$\boxed{tt}$$

R.16. Parameter stability

Misalkan parameter strategi:

$$= , , ,$$

Kita tidak ingin hanya:

yang menghasilkan performa bagus.

Kita ingin neighborhood:

juga relatif bagus.

Secara visual:



R.17. Contoh

Buruk:

$$101 = 0$$

$$102 = 0$$

$$10 =$$

$$104 = 0$$

Ini mencurigakan.

Lebih sehat:

$$20, 21, 22, 21, 20$$

R.18. Robust optimum

Idealnya:

$$\boxed{\theta^*}$$

berada di area yang relatif datar.

Bukan puncak tajam.

Secara matematis:

$$0$$

dan:

$$2$$

tidak menghasilkan permukaan yang terlalu tajam.

Ini sebenarnya sangat mirip dengan konsep **robust optimization**.

R.19. Robust optimization

Daripada:

,

kita dapat mempertimbangkan:

$$P \in \mathcal{P}$$

dengan:

adalah uncertainty set distribusi pasar.

Artinya kita tidak mencari strategi yang terbaik pada satu estimasi distribusi.

Kita mencari strategi yang:

tetap cukup baik ketika asumsi kita salah.

R.20. Ini sangat cocok dengan proyek kita

Karena:

dan:

tidak pernah diketahui secara sempurna.

Misalnya:

=

=

Maka optimizer harus robust terhadap:

,

R.21. Stress test terhadap covariance

Misalkan:

Kita dapat membuat:

$$= \Delta$$

Dengan:

$$\Delta$$

Kemudian jalankan optimizer.

Jika strategi langsung rusak:

$$\square$$

R.22. Stress test terhadap expected return

Demikian juga:

$$= \Delta$$

Kemudian:

$$\Delta$$

Kita lihat apakah portfolio/order decision berubah drastis.

R.23. Sensitivity terhadap transaction cost

Misalkan base:

$$= 005$$

Kemudian:

$$\in \{0025, 005, 010, 020, 050\}$$

Kita hitung:

$$P$$

Jika:

$$P005 \ 0$$

tetapi:

$$P010 \ 0,$$

maka edge kita sangat tipis.

R.24. Slippage stress

Demikian juga:

$$\in \{05_{0,0}, 2_{0,0}\}$$

Kita ingin:

$$P$$

tetap masuk akal.

R.25. Liquidity stress

Kita dapat menurunkan:

$$t$$

secara artifisial.

Misalnya:

$$= 0.5$$

Kemudian:

$$P_t$$

akan meningkat.

Jika strategi langsung collapse:

$$P_{t+1} - P_t$$

R.26. Monte Carlo return bootstrap

Misalkan kita memiliki trade return:

$$r_1, r_2, \dots, r_T$$

Kita dapat melakukan bootstrap:

$$r_1^*, r_2^*, \dots, r_T^*$$

dengan sampling ulang.

Kemudian:

$$t_t$$

Ulangi:

$$= 10000$$

Kita memperoleh distribusi:

$$r_1^*, r_2^*, \dots, r_T^*$$

R.27. Tetapi IID bootstrap tidak selalu cocok

Karena return financial time series mempunyai:

- autocorrelation,
- volatility clustering,
- regime persistence.

Maka lebih baik:

$$\boxed{tt}$$

R.28. Block bootstrap

Kita membagi return:

$$1, \dots$$

menjadi blok:

$$1, 2, \dots$$

Kemudian sampling blok.

Dengan demikian struktur lokal tetap dipertahankan.

R.29. Stationary bootstrap

Alternatif yang lebih fleksibel:

$$t$$

untuk menentukan panjang blok.

Kemudian blok dimulai secara random.

Ini menghasilkan:

$$\boxed{tttt}$$

R.30. Yang kita cari bukan hanya confidence interval PnL

Kita ingin:

$$P \geq 0,$$

$$P \geq 0,$$

$$P \geq 0,$$

dan:

$$P$$

R.31. Probability of ruin

Misalkan capital:

$$0$$

Threshold:

Kita definisikan:

$$P = P_t$$

Ini jauh lebih berguna daripada sekadar melihat maximum drawdown historis.

R.32. Deflated Sharpe Ratio

Sekarang kita masuk ke alat yang sangat penting.

Sharpe ratio observasi:

Jika kita mencoba banyak strategi, maximum Sharpe yang kita lihat akan cenderung meningkat walaupun tidak ada skill.

Kita perlu memperhitungkan:

$$t$$

Inilah motivasi:

$$tt$$

yang secara konseptual menyesuaikan Sharpe berdasarkan:

- number of trials,
- non-normality,
- skewness,
- kurtosis,

- sample length,
- correlation antar strategi.

R.33. Mengapa sangat relevan bagi kita?

Karena kita mungkin mencoba:

$$1, 2, \dots$$

Kemudian:

$$1, \dots$$

Kemudian:

$$1, \dots$$

Jumlah kombinasi bisa menjadi:

Jika kita kemudian mengambil yang terbaik:

$$t$$

hampir pasti menjadi masalah.

R.34. Effective number of trials

Tidak selalu:

$$t =$$

Jika strategi sangat berkorelasi:

$$1,$$

maka informasi yang diperoleh dari eksperimen tersebut tidak independen.

Kita dapat mempertimbangkan:

$$\square$$

Ini lebih masuk akal daripada sekadar menghitung jumlah file eksperimen.

R.35. Probability of Backtest Overfitting

Salah satu konsep penting berikutnya:

$$P$$

atau:

$$Pttttt$$

Ide dasarnya:

Apakah strategi yang kita pilih sebagai pemenang in-sample juga cenderung menjadi pemenang out-of-sample?

Jika tidak:

$$P$$

tinggi.

R.36. Combinatorial Purged Cross Validation

Untuk time series, kita dapat menggunakan:

$$P$$

yaitu:

$$tPt$$

Data dibagi menjadi beberapa kelompok temporal.

Kemudian berbagai kombinasi train/test dibuat sambil menerapkan:

dan:

R.37. Mengapa ini lebih kuat?

Misalkan kita memiliki:

$$=$$

fold:

$$1, \dots$$

Kita tidak hanya mempunyai satu test period.

Kita dapat membuat banyak jalur:

$$1, \dots, 5$$

sebagai test pada kombinasi tertentu, misalnya dengan tetap menjaga urutan temporal dan aturan purging.

Akhirnya kita memperoleh distribusi performa OOS.

R.38. White's Reality Check

Sekarang masalah:

"Saya mencoba banyak strategi. Apakah strategi terbaik saya benar-benar memiliki predictive ability?"

Kita dapat menguji:

$$0 \quad 0$$

untuk seluruh strategi .

Alternative:

$$1 \quad 0$$

White's Reality Check dirancang untuk memperhitungkan data snooping dalam evaluasi banyak model.

R.39. Hansen's SPA

Alternatif yang juga sangat relevan:

$$P_{tt}$$

atau SPA test.

Pertanyaannya:

Apakah strategi terbaik benar-benar mengalahkan benchmark setelah memperhitungkan data snooping?

Ini lebih dekat dengan kebutuhan kita daripada hanya:

$$1$$

R.40. Multiple hypothesis testing

Misalkan kita menguji:

Jika masing-masing menggunakan:

$$= 0.05,$$

probabilitas setidaknya satu false positive meningkat seiring .

Jika test independen:

$$P = 1 - 1 -$$

Untuk:

$$= 100,$$

maka:

$$1 - 0.05^{100} \approx 4$$

Itulah sebabnya:

$$0.05$$

sendirian tidak cukup.

R.41. False Discovery Rate

Kita dapat menggunakan:

$$\square$$

misalnya prosedur:

$$-$$

Ini berguna ketika kita mengevaluasi banyak faktor atau banyak hipotesis alpha.

R.42. Tetapi ada satu masalah penting

Dalam trading, kita sering melakukan:

"Coba parameter."

Kemudian:

"Tidak bagus."

Coba lagi.

Kemudian:

"Masih tidak bagus."

Coba lagi.

Sampai:

"Wow, Sharpe 2.8!"

Secara statistik kita telah melakukan:

t

Walaupun kita tidak menulis 1.000 eksperimen secara eksplisit.

R.43. Karena itu experiment registry wajib

Setiap eksperimen harus dicatat:

t

t

t

Ptt

t

R.44. Jangan menghapus eksperimen buruk

Ini penting.

Jika:

$$_1 = -0$$

$$_2 = 02$$

$$= 21$$

kita tidak boleh hanya menyimpan:

Karena jumlah percobaan:

$$t =$$

bukan:

$$1$$

R.45. Ini juga berlaku untuk QAOA

Misalnya kita menguji:

$$\in \{1, 2, 4, 5\}$$

dan:

,

dengan banyak kombinasi.

Jika kita memilih kombinasi yang menghasilkan PnL terbesar:

, , ,

maka pencarian tersebut sendiri adalah:

$$\boxed{t}$$

Harus dihitung sebagai bagian dari trial history.

R.46. Bahkan ansatz adalah hyperparameter

Misalnya:

$$t^2$$

vs.

vs.

$$-t$$

Jika kita mencoba semuanya lalu memilih yang terbaik, itu juga:

$$t^t$$

R.47. Dan optimizer

Misalnya:

$$P,$$

,

,

—

Pemilihan optimizer berdasarkan hasil test juga merupakan:

$$t$$

R.48. Kita perlu membedakan tiga dataset

Ini sangat penting.

☐

digunakan untuk:

t

t

digunakan untuk:

t

t

digunakan hanya untuk:

t

R.49. Test set adalah "vault"

Idealnya:

☐

Jika kita melihat hasil test kemudian mengubah model:

t

R.50. Final evaluation

Setelah model benar-benar dibekukan:

☐

baru kita jalankan:

t

Dan tidak boleh melakukan tuning lagi berdasarkan hasil tersebut.

R.51. Statistical significance

Misalkan rata-rata trade return:

Sample standard deviation:

Standard error sederhana:

=

Test statistic:

$t =$

Tetapi financial returns sering:

- non-normal,
- autocorrelated,
- heteroskedastic.

Jadi asumsi sederhana ini harus diperlakukan hati-hati.

R.52. HAC / Newey-West

Jika terdapat autocorrelation, variance estimator dapat diperbaiki menggunakan:

$\hat{\sigma}^2_{HAC}$

atau HAC.

Ini memungkinkan standard error lebih realistis.

R.53. Sharpe juga memiliki masalah

Standard Sharpe:

=

Jika return non-normal, Sharpe tidak sepenuhnya menggambarkan risk.

Kita harus memperhatikan:

dan:

t

R.54. Tail risk

Kita dapat mengukur:

Misalnya:

5

Tetapi saya lebih menyukai:

□

atau Expected Shortfall:

=

R.55. Mengapa?

VaR menjawab:

"Pada quantile 95%, seberapa besar loss?"

Sedangkan ES menjawab:

"Jika kita sudah masuk 5% kejadian terburuk, rata-rata loss-nya berapa?"

Untuk tail-heavy financial distributions, ini sangat penting.

R.56. Drawdown distribution

Jangan hanya:

Kita juga dapat melihat:

t ,

,

,

t

Karena:

= 10

selama 2 hari berbeda dengan:

= 10

selama 8 bulan.

R.57. Recovery time

Misalkan equity mencapai peak:

Kemudian turun.

Recovery time:

$$= t - t$$

Ini sangat penting untuk strategi short-term yang mungkin memiliki banyak drawdown kecil.

R.58. Capacity analysis

Ini satu hal yang sangat sering dilupakan.

Jika strategy menghasilkan:

100

profit dengan:

1,000

capital, bagus.

Tetapi apakah bisa:

10,000,000

Belum tentu.

Karena:

tt

meningkat dengan:

R.59. Impact model

Secara sederhana:

$$t =$$

dengan:

- : order size,
- : market volume,
- : volatility,

- , : parameter.

R.60. Capacity

Kita dapat mencari:

sehingga:

$$t = tt$$

Di atas:

,

strategy tidak lagi economic.

Jadi:

$$\boxed{t}$$

adalah bagian dari robustness.

R.61. Latency stress

Untuk short-term trading, kita juga harus menguji:

$$\tau$$

sebagai latency.

Misalnya:

$$\tau \in \{0, 10, 50, 100, 500\}$$

Kemudian:

$$P\tau$$

Jika edge hilang pada:

$$\tau = 50,$$

strategi kita mungkin membutuhkan infrastructure yang sangat cepat.

R.62. Execution degradation curve

Kita dapat mendefinisikan:

$$\tau = P_0 - P_\tau$$

Ini memberikan gambaran:

seberapa cepat alpha kita decay.

R.63. Alpha half-life

Misalkan signal:

t

Autocorrelation:

Kita dapat mencari:

12

ketika:

$12 \ 05$

Itulah pendekatan sederhana untuk:

$-$

Jika half-life sangat pendek, execution menjadi kritikal.

R.64. Ini menghubungkan Tahap R dengan Tahap Q

Tahap Q:

Tahap R:

Jadi:

$\square$

secara alami.

R.65. Robustness terhadap universe

Misalnya awalnya:

$= \{, , , \}$

Kita harus menguji:

1, 2,

Jika strategi hanya bekerja pada satu subset yang dipilih setelah melihat hasil:

$$\boxed{t}$$

R.66. Cross-market validation

Karena tujuan kita bukan hanya crypto, kita dapat menguji:

t

t

t

t

jika data dan execution model mendukung.

R.67. Tetapi hati-hati

Kita tidak boleh mengatakan:

"Model bekerja di BTC, maka pasti bekerja di saham."

Justru kita harus menguji apakah struktur matematisnya transferable.

Misalnya:

mungkin transferable.

Tetapi:

jelas berbeda.

R.68. Hierarchical robustness

Saya ingin robustness kita dibagi menjadi empat tingkat:

Level 1 — Statistical

Level 2 — Temporal

Level 3 — Market

Level 4 — Operational

Jika model melewati keempatnya, tingkat kepercayaan jauh lebih tinggi.

R.69. Robustness score

Kita bahkan dapat membangun dashboard internal:

$$= 1t t \ 2t \ t \ 4t$$

Tetapi **jangan menggunakan skor ini sebagai pengganti statistical test.**

Ini hanya alat ringkasan.

R.70. Stress matrix

Saya menyarankan kita membuat:

Stress	Parameter
Cost	1 – 5
Slippage	1 – 5
Latency	0 – 500
Liquidity	01 – 10
Volatility	05 –
Factor error	
Covariance error	Δ
Fill probability	

Kemudian:

$$P = t, \, t, \, t,$$

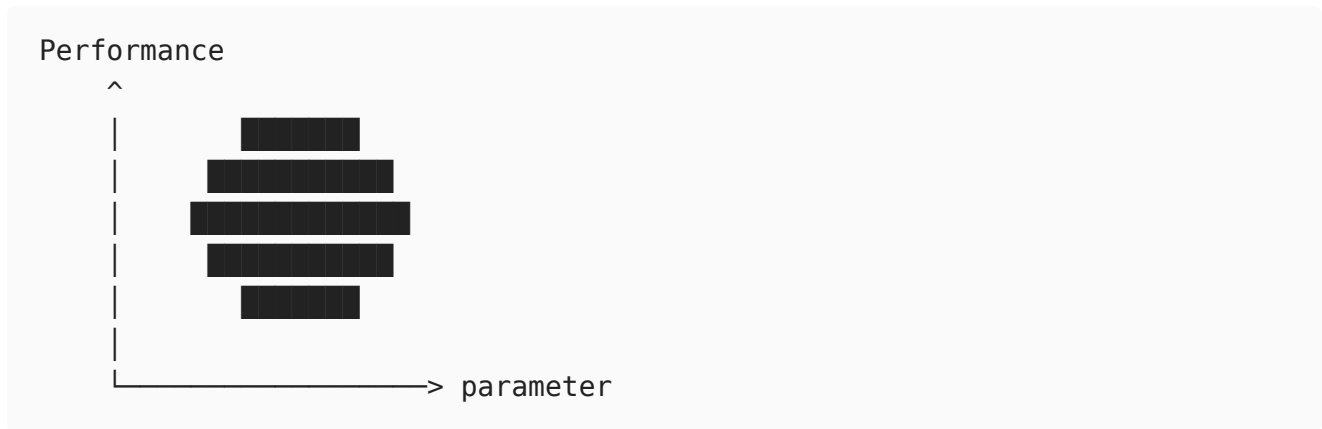
R.71. Yang kita cari bukan "profit maksimum"

Kita ingin:

□

dalam ruang parameter.

Secara visual:



Bukan:



R.72. Dari sini kita bisa mendefinisikan "robust alpha"

Saya mengusulkan definisi operasional:

=

Ini jauh lebih kuat daripada:

2

R.73. Dan sekarang bagian yang sangat penting untuk quantum

Kita memiliki tiga hipotesis berbeda.

H1 — Optimization hypothesis

Apakah QAOA mampu menyelesaikan formulasi optimasi dengan kualitas yang memadai?

H2 — Trading hypothesis

Apakah formulasi optimasi tersebut menghasilkan keputusan trading yang memiliki edge?

H3 — Quantum advantage hypothesis

Apakah QAOA memberikan nilai ekonomi atau komputasional yang tidak diperoleh classical optimizer?

Ketiganya:

□

R.74. Ini mencegah kesalahan besar

Misalkan:

menghasilkan solusi bagus.

Itu membuktikan:

1

Belum membuktikan:

2

Dan bahkan jika:

2

benar, belum membuktikan:

R.75. Quantum comparison matrix

Kita akhirnya akan membandingkan:

Model	Solution Quality	Runtime	PnL	Sharpe	MDD	Cost
Exact	100%	—	—	—	—	—

Model	Solution Quality	Runtime	PnL	Sharpe	MDD	Cost
Classical	—	—	—	—	—	—
QAOA	—	—	—	—	—	—

Dan:

□

harus menjadi salah satu kolom utama.

R.76. "Quantum advantage" yang realistis

Saya tidak ingin kita mendefinisikan:

$$tt = P \ P$$

Itu definisi yang buruk.

Karena optimizer yang berbeda bisa menghasilkan keputusan berbeda akibat stochasticity.

Lebih masuk akal:

$$tt = tt, t, t,$$

R.77. Scaling test

Misalnya jumlah aset:

$$= 4, , 12, 1, 20,$$

Kita ukur:

dan:

Juga:

Maka kita memperoleh:

□

R.78. Ini mungkin menjadi kontribusi ilmiah yang lebih kuat

Daripada:

"Saya menggunakan QAOA untuk trading dan menghasilkan profit."

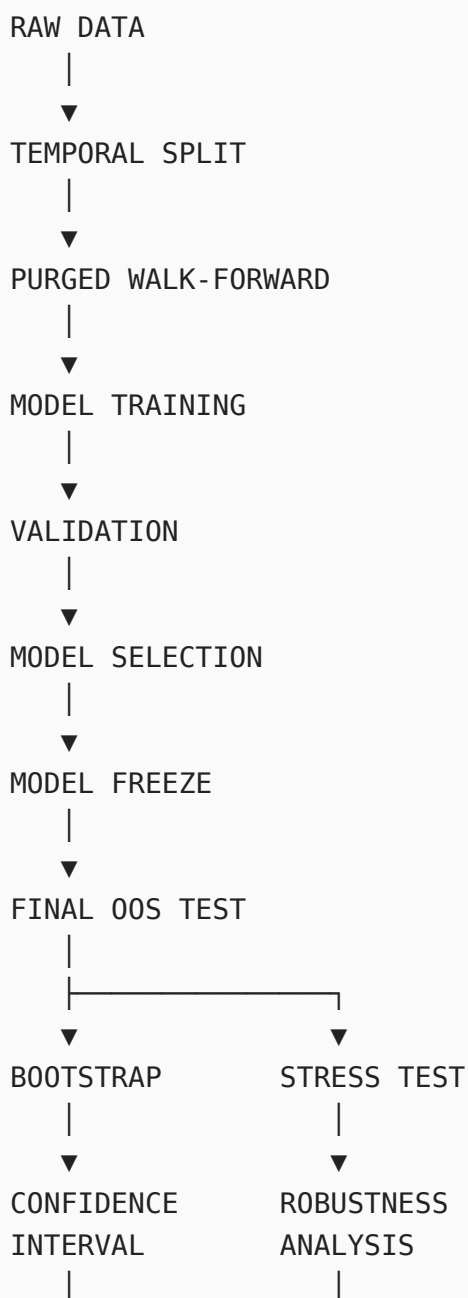
Lebih ilmiah jika:

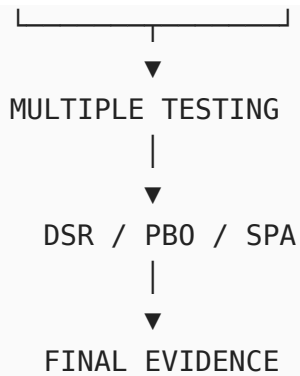
"Kami mengevaluasi hubungan antara approximation quality, computational cost, dan economic degradation pada dynamic financial optimization problem di bawah realistic transaction-cost and execution constraints."

Itu sudah menjadi penelitian yang jauh lebih serius.

R.79. Statistical validation pipeline final

Saya ingin Tahap R kita akhirnya berbentuk:





R.80. Decision gate

Setelah Tahap R, saya ingin sistem kita mempunyai **gate**.

Misalnya:

tt

untuk statistical evidence,

untuk out-of-sample,

t

untuk robustness,

dan:

t

untuk execution.

Kemudian:

$$= tt \quad t \quad t$$

Kalau salah satu gagal:

P

R.81. Saya bahkan menyarankan tiga status

RED

t

Strategi belum boleh dipakai.

YELLOW

$$Ptt$$

Lanjutkan penelitian/paper trading.

GREEN

$$t$$

Baru boleh masuk tahap paper/live dengan risk limit yang sangat kecil.

R.82. Dan ini sangat cocok dengan filosofi \$1 risk kita

Risk budget tetap:

$$t \quad 1$$

Tetapi sekarang kita menambahkan:

$$\square$$

dan:

$$t$$

Jadi:

$$t \quad t$$

saja.

Kita memiliki:

$$tt = t \quad t \quad t \quad t$$

R.83. Model risk

Misalnya:

salah.

Maka:

dapat menghasilkan keputusan yang salah.

Jadi:

$\frac{t}{t}$

Ini harus masuk dalam stress testing.

R.84. Decision uncertainty

Jika dua solusi memiliki objective yang hampir sama:

$$f_1 \approx f_2,$$

tetapi keputusan trading sangat berbeda:

$$x_1 \neq x_2,$$

maka sistem tidak boleh terlalu percaya diri.

Kita dapat mendefinisikan:

$$\Delta = f_1 - f_2$$

Jika:

$$\Delta \approx 0,$$

kita berada di daerah:

$$\frac{t}{t}$$

R.85. Ini membuka jalan untuk tahap berikutnya

Karena jika beberapa solusi hampir sama:

$$f_1, f_2, \dots, f_n \approx 0,$$

kita tidak harus selalu memilih:

Kita bisa membangun:

$$\frac{t}{t}$$

atau bahkan:

$$P$$

sebagai distribusi keputusan.

Ini akan sangat menarik jika dikombinasikan dengan output probabilistik QAOA.

R.86. Dan di sinilah proyek kita mulai menjadi jauh lebih dalam

Kita awalnya ingin:

Sekarang arsitektur kita telah berkembang menjadi:

□

dengan:

Dan keputusan:

t

harus tetap robust terhadap keempatnya.

R.87. Target akhir Tahap R

Pada akhir Tahap R, kita seharusnya tidak lagi bertanya:

"Apakah backtest menghasilkan profit?"

Pertanyaan kita berubah menjadi:

"Seberapa kuat bukti bahwa expected economic edge tetap positif ketika kita memperhitungkan temporal dependence, multiple testing, model uncertainty, regime changes, transaction costs, execution uncertainty, dan optimization error?"

Secara matematis:

$$P \geq 0, ,$$

harus cukup tinggi, dengan:

- : data,
- : model assumptions,
- : execution assumptions.

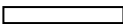
Dan bahkan itu belum berarti kita "membuktikan profit". Yang kita dapatkan adalah **evidence yang lebih kuat atau lebih lemah** terhadap adanya edge.

Posisi kita setelah Tahap R

Kita sekarang memiliki:

$$-P$$

Sehingga pipeline besarnya menjadi:



Dan menurut saya, **Tahap S berikutnya sebaiknya bukan langsung masuk live trading**. Kita perlu membangun **S — Portfolio/Strategy Ensemble & Capital Allocation**, yaitu bagaimana beberapa strategi/optimizer/factor model yang masing-masing telah lolos validasi R digabungkan secara matematis menjadi satu *meta-portfolio*, termasuk korelasi antar-strategi, risk budgeting, Kelly/fractional Kelly, hierarchical risk parity, regime-dependent allocation, dan—yang sangat cocok dengan arah penelitian kita—**formulasi capital allocation tersebut sebagai QUBO/Ising problem**.

Prompt:

8/17/2026, 10:05:08 PM

lanjut Tahap S

Response:

8/17/2026, 10:05:09 PM

Remembering

Reviewed Stage S design, strategy ensemble, capital allocation, and risk budgeting

Worked for 5s

Tahap S — Strategy Ensemble & Capital Allocation

Ya. Setelah Tahap R, kita **belum seharusnya memilih satu strategi pemenang lalu memasukkan seluruh modal ke sana**.

Tahap S menjawab pertanyaan yang lebih fundamental:

Jika kita memiliki beberapa strategi yang masing-masing memiliki edge tetapi memiliki karakteristik risiko yang berbeda, bagaimana modal dialokasikan di antara mereka secara matematis?

Ini menjadi jembatan dari:

□

menuju:

□

dan sangat cocok dengan latar belakangmu di **Markowitz, QUBO, Ising, VQE, dan QAOA**.

S.1. Dari satu strategi menjadi kumpulan strategi

Misalkan setelah Tahap R kita memiliki strategi:

$$= \{1, 2, \dots\}$$

Contohnya:

$$1 = ,$$

$$2 = ,$$

$$= ,$$

$$4 = ,$$

$$5 = ,$$

$$= ,$$

dan seterusnya.

Tetapi kita juga dapat memperlakukan **model yang sama pada universe berbeda** sebagai strategi berbeda.

S.2. Strategy return

Untuk strategi , definisikan return:

$$,t$$

Kemudian vector return:

$$t = 1,t2,t,t$$

Expected return:

$$= t$$

Covariance:

$$= t - t -$$

Perhatikan sesuatu yang penting:

Kita sekarang melakukan Markowitz terhadap strategi, bukan hanya terhadap aset.

S.3. Ini menghasilkan dua level portfolio

Sebelumnya:

Sekarang:

□

dan di dalam setiap strategi:

□

Jadi struktur kita menjadi:

□

S.4. Mengapa ini lebih kuat daripada memilih strategi terbaik?

Misalkan:

Strategy	Sharpe
1	2.0
2	1.8
	1.5

Kalau hanya melihat Sharpe:

1 2

Kita mungkin memilih:

100 1

Tetapi misalkan korelasi:

12 = 01

dan:

1 = 00

Maka kombinasi ketiganya dapat mempunyai:

$$t_1$$

S.5. Prinsip inti

Kita tidak mencari:

$$\square$$

tetapi:

$$\square$$

Ini merupakan perubahan paradigma yang sangat penting.

S.6. Portfolio return

Misalkan bobot strategi:

$$= 12$$

Dengan:

$$= 1,$$

maka return portfolio:

$$, t = t$$

Expected return:

$$=$$

Variance:

$$\boxed{2 =}$$

S.7. Markowitz objective

Kita dapat menggunakan:

$$\boxed{-}$$

dengan:

$$0$$

Ini sudah sangat familiar dengan penelitianmu.

Tetapi untuk sistem trading kita, ada satu masalah.

S.8. Bobot kontinu tidak selalu cukup

Dalam live trading, kita mungkin ingin keputusan:

$$\in \{0, 1\}$$

Artinya:

$$= \begin{cases} 1, & 0, \end{cases}$$

Atau:

$$\in \{0, 1, , \}$$

untuk discretized capital allocation.

Di sinilah:

$$\square$$

kembali masuk.

S.9. Binary strategy selection

Misalkan:

$$\in \{0, 1\}$$

Kita ingin memilih maksimal strategi:

Expected reward:

Risk:

$$,$$

Maka objective:

$$\boxed{-}$$

S.10. Mengubah menjadi minimisasi QUBO

QUBO biasanya ditulis:

Karena kita ingin memaksimalkan:

$$- ,$$

maka equivalent:

$$\boxed{-}$$

S.11. Constraint jumlah strategi

Misalkan:

$$=$$

Kita masukkan penalty:

$$P -$$

Maka:

$$\boxed{= - P -^2}$$

Ini sudah merupakan QUBO.

S.12. Tetapi kita ingin sesuatu yang lebih realistis

Kita sebelumnya memiliki risk budget:

$$_t 1$$

Sekarang kita memiliki:

$$_{tt}$$

Misalnya total capital:

Risk budget strategy :

Kita ingin:

$$\boxed{}$$

S.13. Risk contribution

Portfolio variance:

$$\sigma_p^2 =$$

Marginal contribution to risk:

$$=$$

Karena:

$$= ,$$

maka:

$$\boxed{=}$$

Risk contribution:

$$=$$

Sehingga:

$$\boxed{=}$$

S.14. Risk parity

Jika kita ingin setiap strategi memberikan kontribusi risiko yang sama:

$$=$$

Maka:

$$=$$

Ini menghasilkan:

$$\boxed{P_t}$$

S.15. Tetapi saya tidak menyarankan pure risk parity

Karena strategi yang memiliki:

$$0$$

tidak seharusnya mendapatkan capital besar hanya karena volatility-nya rendah.

Kita ingin:

$$\frac{t}{t}$$

dipertimbangkan sekaligus.

S.16. Expected return harus shrink

Ini sangat penting.

Estimasi:

untuk short-term strategy biasanya sangat noisy.

Misalnya:

$$= 0.0$$

Tetapi standard error:

$$= 0.10$$

Maka kita tidak boleh memperlakukan:

$$0.0$$

seolah-olah merupakan parameter yang pasti.

S.17. Shrinkage

Kita dapat menggunakan:

$$= 1 - \alpha$$

dengan:

$$\alpha = 1 - \alpha$$

α dapat berupa:

$$0$$

atau benchmark expected return.

Jika estimasi tidak pasti:

$$0$$

Jika sangat dipercaya:

$$1$$

S.18. Ini menghasilkan konsep yang sangat penting

$$t$$

Kita tidak hanya memiliki:

Tetapi:

$$=$$

Ini sangat cocok dengan hasil Tahap R.

S.19. Confidence berasal dari Tahap R

Misalnya kita memiliki:

$$\begin{aligned} & , \\ & , \\ & P, \\ & t \end{aligned}$$

Kita dapat membangun confidence score:

$$= , P, , t$$

Kemudian:

$$=$$

S.20. Ini membuat Tahap R dan S terhubung

Sekarang:

$$t \quad t$$

Strategi yang backtest-nya luar biasa tetapi rapuh:

Akibatnya:

Jadi model tidak sekadar:

"Strategi bagus → modal besar."

Tetapi:

S.21. Confidence-adjusted Markowitz

Kita dapat menggunakan:

$$\boxed{-}$$

dengan:

$$=$$

S.22. Masalah berikutnya: correlation

Misalkan:

$$1 =$$

dan:

$$2 =$$

Keduanya mungkin memiliki:

$$_{12} = 0.5$$

Memiliki dua strategi tidak berarti kita memiliki dua sumber diversifikasi.

S.23. Effective number of strategies

Kita dapat menggunakan:

$$\boxed{= 1/2}$$

untuk kasus bobot tertentu.

Jika:

$$= 1,$$

maka:

$$=$$

Jika:

$$1/2 = 1,$$

maka:

$$1$$

S.24. Tetapi korelasi perlu diperhitungkan

Kita dapat menggunakan eigenvalue decomposition:

$$=$$

Dengan:

$$= 1, ,$$

Jika satu eigenvalue mendominasi:

$$1 \gg 2, ,$$

maka strategi-strategi tersebut memiliki satu common risk factor besar.

S.25. Factor decomposition

Kita dapat menulis:

$$t = t \ t$$

Dengan:

$$= 11 \quad 1 \quad 1$$

Covariance:

$$\boxed{=}$$

S.26. Faktor kita

Untuk sistem global kita, faktor dapat mencakup:

$$\boxed{t = , , , P, , t, tt, t}$$

tentu disesuaikan dengan market yang diperdagangkan.

S.27. Mengapa DXY dan Gold penting?

Karena kita sebelumnya ingin model tidak berhenti di crypto.

Misalnya strategi saham memiliki exposure:

,

Maka risk portfolio tidak hanya berasal dari:

Tetapi juga:

□

S.28. Factor-neutral constraint

Misalnya kita tidak ingin portfolio memiliki exposure DXY terlalu besar:

,

Dengan:

, = ,

Begitu juga:

,

S.29. Ini menghasilkan portfolio yang lebih "institutional"

Kita bukan sekadar:

P

Kita mencari:

tt, t, t, t, t

S.30. Capital allocation dengan Kelly

Sekarang kita masuk ke Kelly Criterion.

Untuk satu taruhan dengan probabilitas menang , kalah $= 1 -$, dan payoff :

$= -$

Tetapi trading multi-strategy jauh lebih kompleks.

S.31. Continuous Kelly

Untuk return vector dengan expected return:

dan covariance:

,

approximation quadratic Kelly:

$$=^{-1}$$

Ini berasal dari memaksimalkan expected log wealth.

S.32. Derivasi

Untuk small returns:

$$1 - 12^2$$

Dengan:

$$= ,$$

maka:

$$1 - 12$$

Derivative:

$$= -$$

Set:

$$= 0$$

Maka:

$$=^{-1}$$

S.33. Masalah Kelly penuh

Kelly bisa sangat agresif.

Jika:

sedikit saja salah, maka:

$$-1$$

dapat menghasilkan leverage yang sangat besar.

Karena itu kita lebih realistis menggunakan:

$$t$$

S.34. Fractional Kelly

$$=$$

dengan:

$$0 \quad 1$$

Contohnya:

$$= 0.25$$

Ini mengurangi sensitivity terhadap estimation error.

S.35. Saya ingin kita menggunakan Kelly sebagai risk signal, bukan sebagai penguasa modal

Artinya:

memberikan informasi tentang:

"Seberapa agresif capital allocation secara teoritis."

Tetapi final allocation dibatasi oleh:

$$t,$$

$$,$$

$$t,$$

$$t,$$

dan:

$$t$$

S.36. Hierarchical Risk Parity

Karena covariance matrix bisa unstable, kita dapat menggunakan:

$$\boxed{P}$$

atau Hierarchical Risk Parity.

Pertama:

diubah menjadi distance:

$$\boxed{= 1 - 2}$$

S.37. Clustering

Kemudian strategi dikelompokkan:

$$\{1, 2, \}$$

misalnya menjadi:

$$t$$

Sedangkan:

$$\{4, 5\}$$

menjadi:

$$t$$

S.38. Recursive allocation

Capital kemudian dibagi secara hierarkis.

Misalnya:

$$= 0,$$

$$= 40$$

Kemudian di dalam :

$$_1 = 50,$$

$$_2 = 0,$$

$$= 20$$

Sehingga:

$$w_1 = 0$$

S.39. Mengapa HRP menarik?

Karena kita tidak harus memperkirakan:

secara agresif.

Ini berguna ketika expected return sangat noisy.

Jadi kita bisa memiliki beberapa allocator:

$$w_1 = ,$$

$$w_2 = ,$$

$$=$$

S.40. Lalu meta-allocation

Kita bahkan dapat mengalokasikan capital di antara allocator:

$$\boxed{w_t^P}$$

Ini terdengar recursive, tetapi secara konsep valid.

Kita kemudian dapat melakukan ensemble:

$$= \sum P$$

Dengan:

$$= 1$$

S.41. Tetapi jangan sampai menjadi overfitting

Jika kita memilih:

$$= 01,$$

$$= 042,$$

$$= 0201$$

hanya karena menghasilkan backtest terbaik, kita kembali ke masalah Tahap R.

Jadi parameter meta-allocation harus:

$$, t, tt$$

S.42. Sekarang kita sampai ke bagian quantum

Misalkan capital dibagi dalam unit:

$$\Delta$$

Jika total capital:

maka:

$$= \Delta$$

Kita memiliki binary variables:

$$, \in \{0, 1\}$$

yang berarti:

| unit modal dialokasikan ke strategy .

Constraint:

$$, = 1$$

S.43. Capital allocation menjadi QUBO

Expected reward:

$$, \Delta,$$

Risk interaction:

$$,, \Delta^2 ,,$$

Maka:

$$- , \Delta, ,, \Delta^2 ,,$$

dengan constraint capital allocation.

S.44. Constraint menjadi penalty

Untuk setiap unit:

$$^2, -1$$

Sehingga:

$$P^2, -1$$

Final:

$$= - P_t$$

S.45. Ini sangat menarik secara penelitian

Karena QAOA tidak lagi hanya digunakan untuk:

Tetapi:

□

Dan problem ini secara natural memiliki:

- binary decisions,
- combinatorial constraints,
- covariance interactions,
- factor constraints,
- cardinality constraints.

Ini adalah problem yang sangat cocok dengan paradigma QUBO.

S.46. Multi-level QUBO

Kita bahkan dapat membentuk:

”

dengan:

- : strategy,
- : asset,
- : capital unit.

Sehingga:

□

semuanya dapat diformulasikan dalam satu optimization layer.

S.47. Tetapi saya tidak menyarankan langsung membuat QUBO raksasa

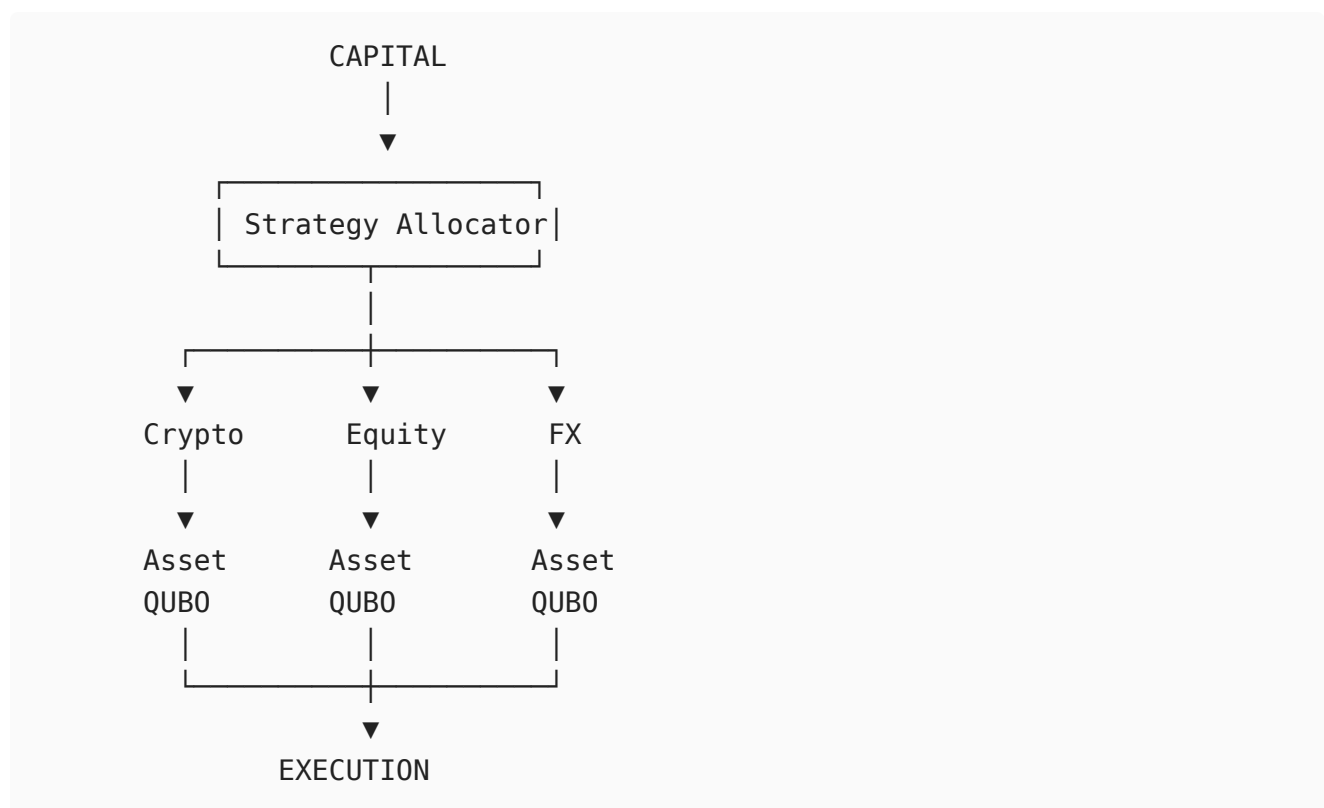
Karena jumlah qubit:

dapat tumbuh sangat cepat.

Kita harus memulai:

tt

S.48. Arsitektur hierarchical



S.49. Mengapa hierarchical lebih realistis?

Karena pasar memang hierarkis.

Kita punya:

t t t

Jadi struktur optimizer kita mengikuti struktur ekonomi.

S.50. Dynamic allocation

Capital allocation tidak boleh statis.

Kita definisikan:

$$t$$

Kemudian:

$$t = t, t, t, t, t, t$$

Dengan:

- t : expected return,
 - t : covariance,
 - t : macro factors,
 - t : regime,
 - t : liquidity,
 - t : current capital.
-

S.51. Turnover penalty

Kalau kita mengubah:

$$t-1 \quad t,$$

kita membayar biaya.

Maka tambahkan:

$$t - t-11$$

Objective:

$$- - - 1$$

S.52. Ini penting untuk sistem short-term

Tanpa turnover penalty:

$$t$$

dapat berubah setiap periode.

Misalnya:

10:00 → Strategy A
 11:00 → Strategy C
 12:00 → Strategy A
 13:00 → Strategy B
 14:00 → Strategy C

Ini menyebabkan:

$$t t$$

S.53. Dalam QUBO

Jika kita discretize weight:

$$w_{i,j,t}$$

perubahan posisi dapat direpresentasikan dengan:

$$w_{i,j,t} - w_{i,j,t-1}$$

Untuk binary:

$$-1 = 1 - 2$$

Ini sangat menarik karena tetap quadratic.

Jadi turnover penalty dapat dimasukkan secara natural ke QUBO:

$$w_{i,j,t} - w_{i,j,t-1} - 2 w_{i,j,t} w_{i,j,t-1}$$

S.54. Sekarang dynamic risk budget

Kita dapat mendefinisikan:

$$B_t$$

sebagai total risk budget.

Jika volatility regime meningkat:

$$B_t \uparrow$$

maka:

$$B_t$$

Misalnya:

$$B_t = 0 t t t$$

Kemudian dibatasi:

S.55. Ini adalah volatility targeting

Target:

$$\sigma_t$$

Current:

$$\sigma_t$$

Scale:

$$\sigma_t = \sigma_{t-1} \sqrt{\frac{\sigma_t^2}{\sigma_{t-1}^2}}$$

Sehingga:

$$\sigma_t = \sigma_{t-1}$$

S.56. Tetapi gunakan volatility yang robust

Volatility estimate:

$$\sigma_t$$

juga noisy.

Jadi kita dapat menggunakan:

- EWMA,
- realized volatility,
- robust volatility,
- factor volatility.

Contohnya EWMA:

$$\sigma_t^2 = \sigma_{t-1}^2 \alpha + (1 - \alpha) r_{t-1}^2$$

S.57. Sekarang hubungkan dengan market regime

Misalkan:

$$r_t \in \{1, -1\}$$

Kita dapat menetapkan:

$$t = , , ,$$

Namun jangan menjadikan regime sebagai hard switch tanpa validasi.

Lebih baik:

$$_t = P_t$$

S.58. Probabilistic regime allocation

Misalnya:

$$P_t = = 0,$$

$$P_t = = 0$$

Maka expected allocation:

$$_t = 0 \ 0$$

Ini menghindari:

S.59. Kita sekarang mempunyai meta-state

State sistem:

$$t = t, t, t, t, t, t, P_{t, t}$$

Kemudian policy:

$$t \ t$$

Ini mulai terlihat seperti **sequential decision problem**.

S.60. Dan ini membuka jalan ke tahap berikutnya

Tetapi sebelum melangkah lebih jauh, kita perlu menetapkan:

$$t \ t$$

Capital allocation menjawab:

Berapa modal diberikan ke strategy?

Position sizing menjawab:

Berapa besar posisi yang diambil oleh strategy tersebut pada trade tertentu?

S.61. Dua layer

Misalnya:

$$= 10,000$$

Allocator memberikan:

$$_{tt} = 20$$

Maka:

$$_{tt} = 2,000$$

Kemudian strategy memiliki trade dengan risk:

$$_t = 1$$

Maka position size ditentukan oleh:

$$= _t - t$$

Dengan penyesuaian:

$$= , t, t, tt$$

S.62. Ini sangat penting

Jangan mencampurkan:

$$\boxed{Pt}$$

dengan:

$$\square$$

Keduanya berbeda.

Kita bisa memiliki:

$$1$$

risk per trade tetapi 100 trade yang highly correlated.

Total portfolio risk bisa tetap besar.

S.63. Correlated trades

Misalnya:

$$1 =$$

$$2 =$$

$$=$$

dengan:

$$0$$

Masing-masing:

$$= 1$$

Bukan berarti:

$$t =$$

secara naif.

Covariance harus dihitung.

S.64. Portfolio-level risk

Jika trade exposure vector:

$$,$$

maka:

$$\boxed{2 =}$$

Jadi risk budget harus berlaku di:

$$\boxed{t \quad tt \quad t}$$

S.65. Risk hierarchy kita

Saya ingin struktur finalnya seperti ini:

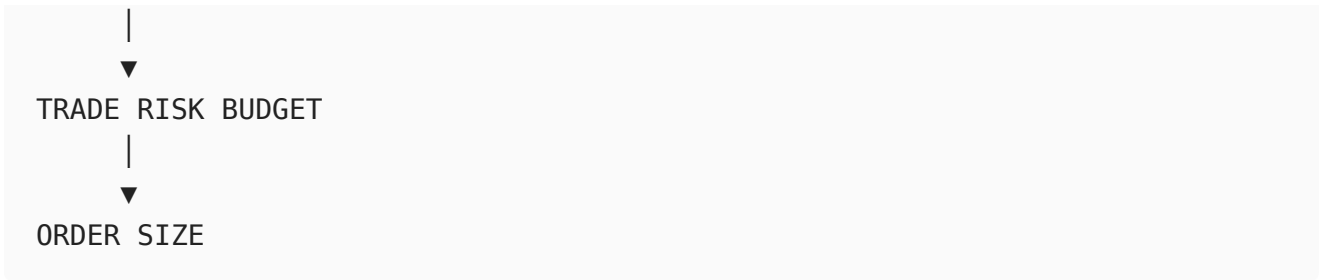
GLOBAL RISK



MARKET RISK BUDGET



STRATEGY RISK BUDGET



Dan:

$$\boxed{1}$$

yang sebelumnya kita tetapkan berada pada **trade-risk layer**, bukan seluruh sistem.

S.66. Capital allocator final

Secara konseptual:

$$\boxed{- \quad - \quad - \quad 1 - t - t}$$

subject to:

$$\begin{aligned} &= 1, \\ 0 &, \\ &t, \\ &t \end{aligned}$$

S.67. QUBO version

Setelah discretization:

$$= \Delta \quad ,$$

Kemudian objective menjadi:

$$\boxed{- \quad t \quad t \quad t \quad P_{tt}}$$

Dan inilah kandidat utama:

$$\boxed{tt}$$

S.68. Ising mapping

Karena:

$$= 1 - 2,$$

dengan:

$$\in \{-1, 1\},$$

maka QUBO dapat dipetakan menjadi:

$$=$$

Dengan:

merepresentasikan individual attractiveness,

sedangkan:

merepresentasikan interaction:

$$t \quad tt \quad t \quad t$$

S.69. Interpretasi fisiknya

Ini sangat menarik jika dikaitkan dengan pekerjaanmu.

Setiap strategy:

adalah spin.

State:

$$= 1$$

dapat berarti:

Sedangkan:

$$= -1$$

berarti:

Coupling:

menentukan apakah dua strategy:

atau:

S.70. Tetapi jangan mengatakan semua adalah correlation

Ini penting secara akademik.

Secara umum:

dapat berasal dari:

$$\begin{matrix} = & t & t & tt \end{matrix}$$

Correlation hanya salah satu sumber.

S.71. QAOA

Hamiltonian:

merepresentasikan capital allocation objective.

Mixer:

$$=$$

State:

$$, =_{=1}^{--}$$

Kemudian:

$$\square$$

diminimalkan.

S.72. Tetapi ada satu hal penting

Jika constraint sangat keras, standard mixer:

$$=$$

dapat menghasilkan state yang keluar dari feasible subspace.

Karena itu kita mungkin membutuhkan:

$$tt - P$$

Misalnya untuk constraint:

$$=$$

Kita dapat menggunakan mixer yang melakukan:

$$10 \rightarrow 01$$

sehingga jumlah bit 1 tetap:

S.73. Ini membuat QAOA kita lebih bermakna

Bukan:

sebagai sekadar demonstrasi quantum.

Tetapi:

$$= tt$$

S.74. Ensemble final

Setelah Tahap S, sistem kita memiliki:

$$tt$$

yang menghasilkan:

$$\{1, 0\}$$

Kemudian:

$$t$$

memberikan:

$$\{1, 0\}$$

Kemudian:

$$t$$

menghasilkan:

$$t$$

Kemudian:

$$Pt$$

menghasilkan:

t

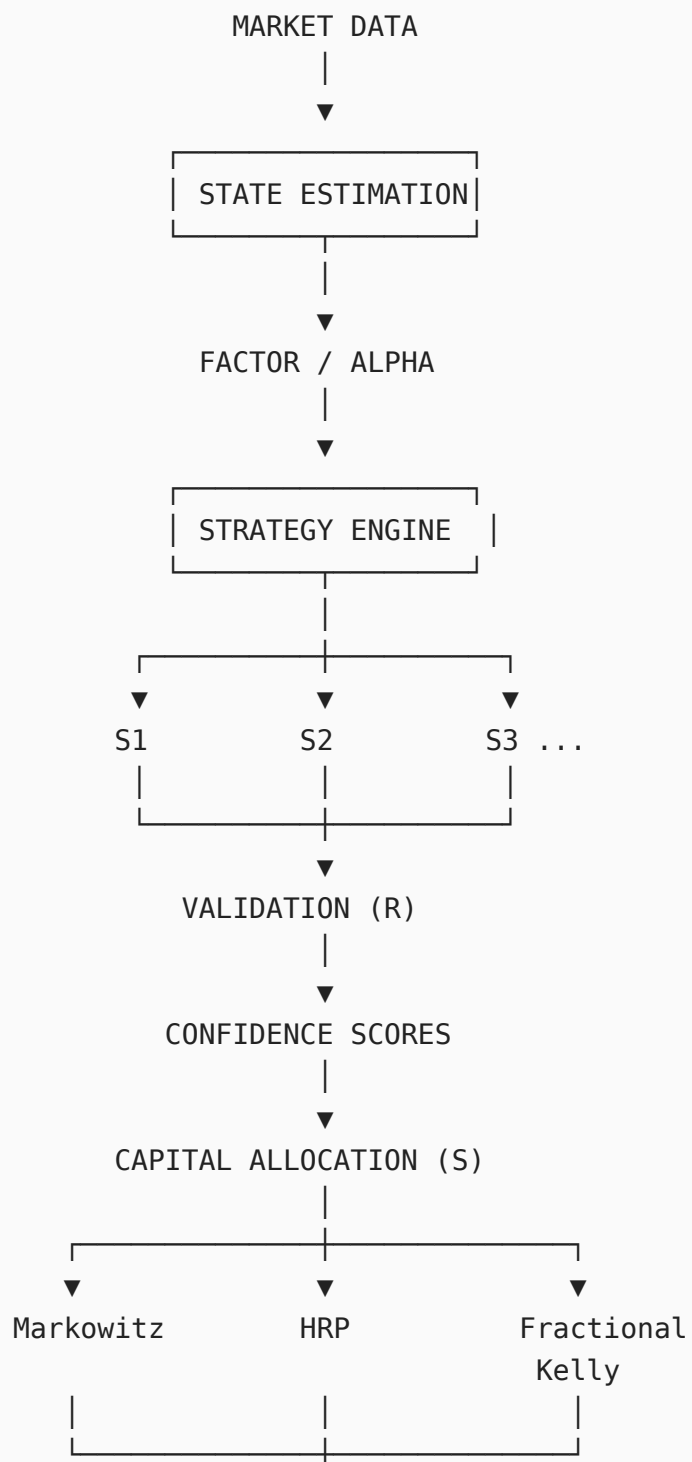
Kemudian:

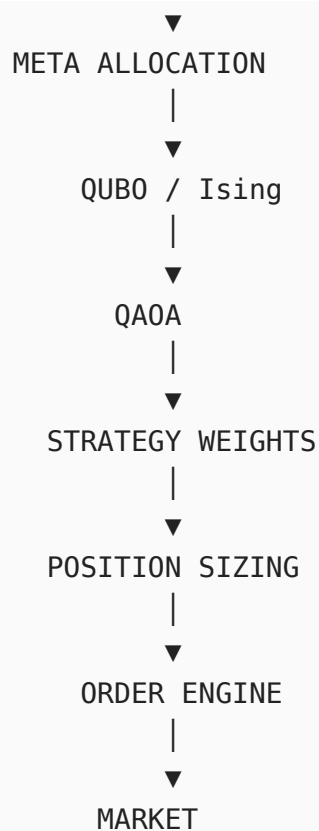
t

menghasilkan:

t

S.75. Pipeline lengkap kita sekarang





S.76. Hal yang paling saya sukai dari Tahap S

Di sinilah penelitianmu sebelumnya mulai **menyatu secara alami**.

Dulu kamu belajar:

secara relatif terpisah.

Sekarang semuanya dapat ditempatkan dalam satu arsitektur:



Dan yang lebih penting:

QAOA bukan dipaksakan ke trading.

Kita terlebih dahulu menemukan problem finansial yang memang memiliki struktur combinatorial, kemudian memetakannya ke QUBO.

S.77. Namun ada satu prinsip yang harus kita pegang

Jangan membuat:

$$\square$$

tetapi:

$$\square$$

Ini akan membuat penelitian kita jauh lebih defensible secara akademik.

S.78. Output Tahap S

Pada akhir tahap ini kita seharusnya memiliki empat objek utama:

1. Strategy return matrix

$$\in$$

2. Confidence-adjusted expected return

$$\square$$

3. Strategy covariance

$$\boxed{tt}$$

4. Capital allocation problem

$$\boxed{t}$$

yang dapat diselesaikan menggunakan:

$$\boxed{tP \ t}$$

sebagai tiga benchmark.

S.79. Dan saya ingin kita membuat satu aturan penelitian

Untuk setiap problem optimasi:

$$\square$$

Baseline harus tetap ada:

$$\boxed{t}$$

untuk problem kecil,

$$\boxed{t}$$

untuk problem praktis,
dan:

□

sebagai kandidat quantum solver.

Kemudian kita ukur:

,
,
,
,

dan akhirnya:

□

S.80. Posisi kita setelah Tahap S

Sekarang arsitektur kita telah berkembang:

—

kemudian:

□

Tahap berikutnya yang menurut saya paling logis adalah **Tahap T — Portfolio-to-Order Translation & Dynamic Position Sizing**.

Di sana kita akan menurunkan secara matematis bagaimana:

$$t \quad t \quad t \quad t \quad P_t \quad t$$

dengan tetap mempertahankan constraint yang sudah kita bangun: **risk \$1 per trade, portfolio-level risk, covariance antar posisi, liquidity, expected slippage, confidence score, volatility regime, dan dynamic adjustment ketika posisi sudah floating profit/loss**.

Dengan kata lain, Tahap T akan menjadi titik di mana seluruh matematika yang kita bangun mulai berubah menjadi **keputusan order konkret**.

Prompt:

8/17/2026, 10:06:15 PM

lanjut Tahap T

Response:

Tahap T — Portfolio-to-Order Translation & Dynamic Position Sizing

Ya. **Tahap T** adalah salah satu tahap paling penting, karena di sinilah seluruh lapisan matematika yang kita bangun sebelumnya akhirnya diterjemahkan menjadi keputusan trading konkret:

Dan saya ingin mempertahankan prinsip yang sudah kita bangun:

Optimizer tidak menentukan order secara langsung.

Optimizer menentukan *berapa banyak risk/capital yang layak diberikan kepada suatu opportunity*.

Execution engine kemudian menentukan bagaimana risk tersebut diwujudkan menjadi entry, SL, TP, dan order management.

Ini penting agar arsitektur kita tidak menjadi satu model besar yang sulit diuji.

T.1. Hirarki keputusan

Kita sekarang mempunyai lima level:

$1, 2, 3, 4, 5$

Misalnya:

$$t = 10,000$$

Capital allocator mengatakan:

$$tt = 20$$

Maka:

$$tt = 2,000$$

Tetapi itu **belum berarti membeli aset senilai \$2,000**.

Strategy kemudian menghasilkan sebuah trade opportunity dengan risk budget:

$$t = 1$$

Barulah position-sizing engine menentukan jumlah aset.

T.2. Risk budget sebagai mata uang universal

Saya menyarankan kita menjadikan **risk** sebagai bahasa penghubung seluruh sistem.

Bukan:

"Strategi A mendapatkan \$2,000."

Tetapi:

"Strategi A mendapatkan risk budget sebesar ."

Misalnya:

$$= 10$$

Sebuah trade kemudian mendapatkan:

$$_t = 1$$

Maka secara kasar:

$$_{t,} = _t = 10$$

Namun jumlah aktual tetap harus memperhitungkan correlation dan portfolio risk.

T.3. Trade risk

Untuk sebuah posisi long:

$$P = ,$$

$$P = ,$$

$$=$$

Gross loss jika SL terkena:

$$= P - P$$

Jika ada biaya:

$$, ,$$

dan expected slippage:

$$, ,$$

maka:

$$_t = P - P$$

Untuk short, tanda harga dibalik.

T.4. Position sizing

Kita ingin:

$$t \quad t$$

Sehingga:

$$t - - P - P$$

Jadi:

$$= t -$$

dengan:

$$= P - P$$

Ini merupakan **position sizing engine paling dasar**.

T.5. Tetapi kita tidak boleh berhenti di sini

Karena:

belum mempertimbangkan:

- liquidity,
- market impact,
- portfolio exposure,
- correlation,
- factor exposure,
- leverage,
- confidence,
- regime.

Maka quantity final harus:

$$= , t , t , t , t$$

T.6. Stop-loss tidak boleh menentukan risk budget

Ini prinsip penting.

Urutannya bukan:

Tetapi:

$$Pt$$

Mengapa?

Karena kita terlebih dahulu menentukan:

"Berapa kerugian maksimum yang bersedia diterima?"

Kemudian menentukan struktur trade yang masuk akal.

T.7. Menentukan Stop Loss

SL sebaiknya bukan angka arbitrer.

Kita dapat menggunakan kombinasi:

Volatility

=

Misalnya:

$$= 2$$

Market structure

$$P = P -$$

Statistical boundary

Misalnya:

$$P = P -$$

T.8. Model SL gabungan

Misalnya kita memiliki tiga estimasi:

,

tt ,

tt

Kita dapat menggunakan:

$$= , tt, tt$$

Tujuannya agar SL tidak terlalu dekat sehingga terkena noise.

T.9. Tetapi ada trade-off

SL semakin jauh:

maka:

Karena:

$$1$$

Jadi volatility tinggi secara otomatis menyebabkan:

$$t$$

Ini jauh lebih masuk akal daripada selalu membeli jumlah aset yang sama.

T.10. Take Profit

Sekarang kita menentukan TP.

Definisikan:

$$P = P_P - P$$

Risk distance:

$$= P - P$$

Risk-reward ratio:

$$= P$$

T.11. Tetapi TP seharusnya berasal dari expected distribution

Ini lebih menarik.

Misalkan model kita memperkirakan:

$$t$$

Kita ingin memilih TP:

$$P_P$$

berdasarkan distribusi return, bukan sekadar:

$$P = 2$$

T.12. Expected value

Misalkan:

$$= P,$$

$$= P,$$

dan:

$$= 1 - -$$

Dengan payoff:

$$= P - t,$$

$$= t -$$

Expected PnL:

$$\boxed{P = - Pt}$$

Jika neutral diabaikan:

$$P = -$$

T.13. Break-even probability

Jika:

$$P \ 0,$$

maka:

Jika hanya dua outcome:

$$= 1 -$$

Maka:

$$1 -$$

Sehingga:

Jadi:

$$\boxed{}$$

Untuk:

$$= ,$$

maka:

$$11$$

T.14. Contoh

Jika:

$$= 2,$$

maka:

$$1$$

Jadi win rate hanya:

sudah cukup secara teoritis sebelum biaya dan slippage.

Tetapi tentu model harus menghasilkan probability yang *out-of-sample calibrated*.

T.15. Expected utility

Kita bisa menjadi lebih sophisticated.

Daripada:

$$P,$$

gunakan utility:

$$= -2$$

Untuk trade:

$$= -2$$

Maka TP dipilih untuk:

$$P = {}_P P$$

T.16. Ini membuat TP menjadi optimization problem

Bukan:

$$TP = 2 \times SL$$

tetapi:

$$P = \frac{PP - 2PP - tP}{P}$$

Ini jauh lebih sesuai dengan sistem kita.

T.17. Sekarang masalah yang kamu tanyakan sebelumnya

Jika posisi sudah floating profit tinggi, apakah TP dinaikkan?

Jawabannya:

bisa, tetapi bukan karena profitnya tinggi.

Kita harus bertanya:

□

T.18. Conditional expected return

Misalkan posisi sudah menghasilkan:

$$P_t - P$$

State sekarang:

$$t$$

Model menghitung:

$$t|t$$

Jika:

$$t|t > 0,$$

maka posisi masih memiliki expected edge.

Jika:

$$t|t < 0,$$

maka tidak ada alasan kuat untuk mempertahankannya.

Jika:

$t \neq 0$,

maka exit mungkin lebih rasional.

T.19. Ini menghasilkan prinsip penting

Jangan menggunakan:

sebagai signal utama.

Gunakan:

□

Floating profit hanya merupakan bagian dari state.

T.20. State posisi

Kita definisikan:

$$t = P_t, P_{t,t}, P_{t,t,t}, \tau_{t,t,t,t}$$

Dengan:

- : maximum favorable excursion,
 - : maximum adverse excursion,
 - τ_t : holding time,
 - t : regime,
 - t : liquidity.
-

T.21. Dynamic Stop Loss

Kita definisikan:

$$t = t$$

Tetapi kita harus menghindari:

t

yang terus bergerak mengikuti harga tanpa alasan statistik.

T.22. Break-even transition

Misalnya:

$$t = P_t - PP - P_{,0}$$

Jika:

$$t \text{ ,}$$

maka SL dapat dipindahkan ke:

$$t = P$$

dengan memperhitungkan fees/slippage.

T.23. Contoh

Initial:

$$P = 100$$

SL:

$$5$$

Risk:

$$5$$

Jika harga:

$$P_t = 105,$$

maka:

$$t = 1$$

Kita dapat memindahkan:

$$= 100$$

atau sedikit di atas:

$$= 100$$

T.24. Tetapi BEP terlalu agresif

Masalah:

$$= t$$

membuat trade sangat mudah keluar akibat noise.

Jadi kita dapat menggunakan:

$$t = P_t$$

Misalnya:

$$= 02$$

Untuk long:

$$t = P_{02_t}$$

T.25. Trailing stop berbasis volatility

Alternatif:

$$t = P_t - t$$

untuk long.

Dan:

$$t = P_t t$$

untuk short.

Tetapi:

harus dikalibrasi menggunakan data OOS.

T.26. Bahkan lebih baik: trailing stop berdasarkan distribution

Misalkan model menghasilkan:

$$P_t - t$$

Kita ingin SL berada pada quantile tertentu:

$$P_t - t =$$

Sehingga:

$$= t - tt$$

Ini menghasilkan **probabilistic stop-loss**.

T.27. TP dinamis

Demikian pula:

$$P_t = P_{t,t,t,t,t}$$

Kita dapat menentukan:

$$P_t = P^t$$

T.28. Tetapi TP tidak boleh terus dinaikkan

Misalkan:

$$P_0 = 110$$

Harga:

$$P_t = 10$$

Model baru mengatakan target optimal:

$$P = 10$$

Maka kita **tidak** menaikkan TP menjadi 115 hanya karena harga sedang profit.

Kita justru menurunkannya.

Jadi:

$$P_t = ,$$

bukan:

T.29. Ini memberi kita tiga aksi

Pada setiap evaluation period:

$$t \in \{, , \}$$

HOLD

Jika:

$$tt \ t$$

MODIFY

Jika:

$$t \geq 0$$

tetapi optimal:

$$t \leq t-1$$

atau:

$$P_t \geq P_{t-1}$$

EXIT

Jika:

$$t \geq 0$$

T.30. Ini sebenarnya adalah optimal stopping problem

Secara matematis:

$$\tau = \{t \mid t \geq \tau\}$$

Dengan:

$$t$$

sebagai continuation value.

T.31. Continuation value

Kita dapat mendefinisikan:

$$t = t \Delta t,$$

Exit value:

$$t \geq t - t$$

Maka:

$$t \geq t \geq t$$

Ini jauh lebih fundamental daripada aturan:

"Kalau floating profit, geser SL."

T.32. Holding time juga penting

Misalkan trade sudah berjalan:

$$\tau =$$

Model awal memprediksi edge dalam:

$$= 2$$

Jika setelah 3 jam target belum tercapai:

$$PP_{\tau}$$

mungkin sudah turun.

Ini disebut **time decay of alpha**.

T.33. Hazard function

Kita dapat mendefinisikan:

$$p_t = \frac{P_t - P_{t-\Delta t}}{\Delta t}$$

Jika:

$$p_t$$

menurun secara signifikan, continuation probability juga turun.

T.34. Maximum holding time

Kita dapat menentukan:

$$\tau$$

dengan:

$$t, \tau, t$$

Maka:

$$\boxed{\tau, \tau}$$

T.35. Sekarang portfolio interaction

Ini sangat penting.

Misalkan kita memiliki:

=

dan:

=

Keduanya floating profit.

Trade A memiliki:

Trade B:

2

Tetapi market suddenly masuk regime:

Correlation:

,

Portfolio risk meningkat.

Maka mungkin kita tidak mempertahankan keduanya.

T.36. Portfolio-level position management

Kita harus menghitung:

$$\boxed{t,t = tt}$$

Jika:

$$t,t \quad t,$$

maka kita harus melakukan **risk reduction**.

T.37. Trade selection saat risk budget penuh

Misalkan terdapat tiga posisi:

, ,

Risk contributions:

$$= 40,$$

$$= 5,$$

$$= 25$$

Tetapi expected future alpha:

$$= 01,$$

$$= 05,$$

$$= 0$$

Jika perlu mengurangi risk, kita tidak otomatis menutup posisi yang paling profit.

Kita mencari:

$$\square$$

T.38. Marginal utility of risk

Definisikan:

$$= -2$$

Marginal contribution:

$$= -2$$

Jika:

$$0,$$

strategy tersebut sedang memberikan kontribusi marginal negatif terhadap objective.

T.39. Ini menghasilkan exit ranking

Kita dapat menghitung:

$$=$$

Kemudian posisi dengan:

paling rendah dikurangi terlebih dahulu.

T.40. Ini jauh lebih baik daripada FIFO

Bukan:

| posisi tertua ditutup.

Bukan:

| posisi yang profit paling kecil ditutup.

Bukan:

| posisi yang profit paling besar ditutup.

Tetapi:

$$-tt$$

yang dikurangi.

T.41. Liquidity constraint

Sekarang kita masuk ke masalah nyata.

Kita tidak boleh menganggap:

bisa berapa pun.

Misalkan:

=

Kita dapat membatasi:

$\square$

Dengan:

misalnya beberapa persen atau jauh lebih kecil tergantung market.

T.42. Market impact

Approximation sederhana:

$$t =$$

Dengan:

- : impact coefficient,
- : volatility,
- : volume,
- : scaling exponent.

Maka effective execution price menjadi:

$$P = P_t$$

T.43. Sekarang position size menjadi fixed point

Karena:

menentukan:

$$t,$$

dan impact mempengaruhi:

Maka:

$$= {}_t t$$

Jadi:

$$=$$

dengan impact diperhitungkan.

T.44. Untuk market sangat liquid

Jika:

$$t \rightarrow 0,$$

maka:

Tetapi untuk:

- small-cap stocks,
- mid-cap crypto,
- altcoins,
- thin order books,

impact dapat menjadi faktor dominan.

T.45. Order book

Kita sekarang masuk ke order execution.

Order book:

$$t = \{, \}$$

dan:

$$t = \{, \}$$

Bid:

$$t =$$

Ask:

$$t =$$

Spread:

$$t = t - t$$

T.46. Mid price

$$t = t \ t^2$$

Microprice dapat menggunakan imbalance.

Order book imbalance:

$$t = t - t \ t \ t$$

T.47. Order book sebagai short-term alpha

Misalnya:

$$t \ 0$$

Artinya bid-side depth relatif besar.

Model dapat menganggap:

$$P \Delta P \ 0_t$$

meningkat.

Tetapi:

$$t$$

tidak boleh dianggap sebagai signal independen; ia harus masuk ke model state kita.

T.48. Entry decision

Sekarang opportunity score:

$$t = t, t, t, t, \bar{t}, t$$

Trade hanya dibuat jika:

$$t$$

T.49. Expected edge setelah transaction cost

Ini sangat penting.

Model tidak boleh menggunakan:

Tetapi:

$$t = - - - - t$$

Trade hanya valid jika:

$$t \geq 0$$

T.50. Bahkan harus melewati hurdle rate

Karena estimation error:

$$t \neq t$$

Hurdle:

$$t \geq t$$

Sehingga:

$$t \geq \bar{t} \geq t$$

Ini membuat model lebih konservatif.

T.51. Pending order

Kamu sebelumnya menginginkan:

| order diletakkan tetapi tidak langsung dieksekusi.

Maka kita memiliki:

$$P_t$$

Order valid selama:

$$t - t$$

Jika belum terisi:

ketika:

$$P_t$$

turun terlalu rendah.

T.52. Fill probability

Kita dapat memodelkan:

$$P = t, , , tt,$$

Misalnya:

$$P = 0 \ 1 \ 2$$

Dengan sigmoid:

$$= 11 \ -$$

T.53. Expected value pending order

Trade limit order bukan hanya:

$$P$$

Tetapi:

$$t = P_t - t_t - ttt$$

Jadi order hanya diletakkan jika:

$$t \geq 0$$

T.54. Ini sangat penting

Karena limit order memiliki:

t

Jika order hanya filled ketika harga bergerak melawan kita, maka:

P

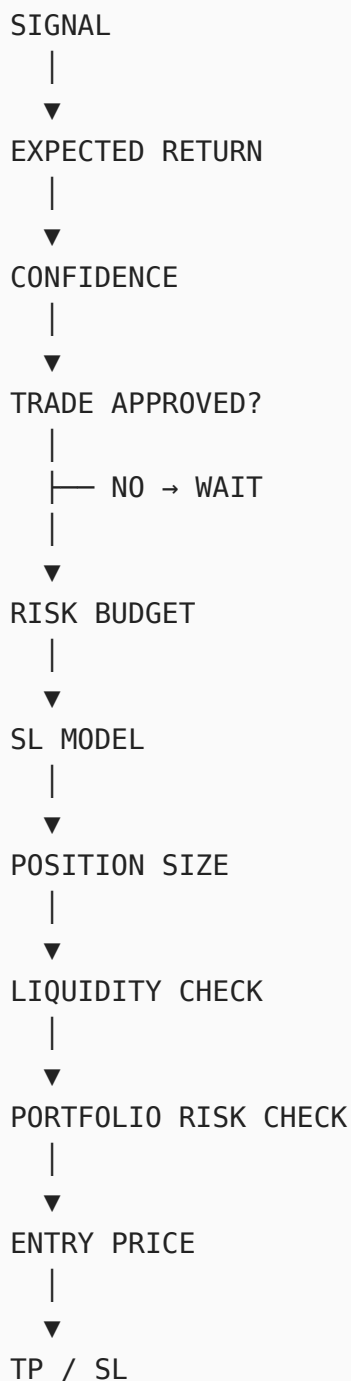
tidak independen terhadap future return.

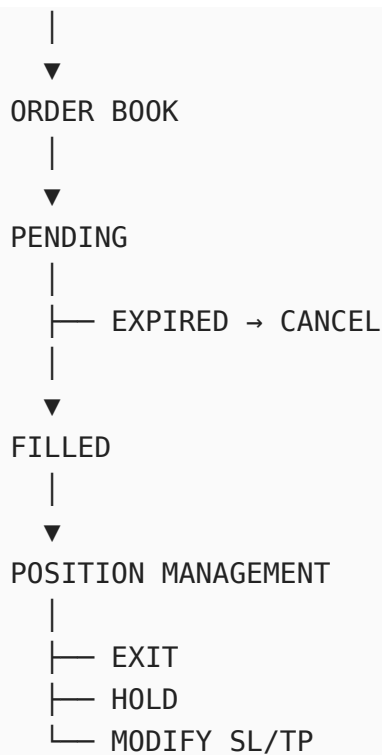
Sehingga:

$P_t \ P$

Ini harus dimodelkan.

T.55. Kita sekarang memiliki lifecycle lengkap





T.56. Dynamic management loop

Setiap:

$$\Delta t$$

kita menghitung:

$$t$$

Kemudian:

$$\boxed{t = t}$$

menghasilkan action:

$$t$$

Secara formal:

$$t = tt \, t,$$

T.57. Action space

Untuk posisi yang sudah aktif:

$$t \in \{,, , P, P, , \}$$

Saya justru menyarankan **ADD** diperlakukan sangat hati-hati.

Jangan melakukan averaging down hanya karena posisi loss.

T.58. Averaging down

Misalnya:

$$P_t > P$$

Kita tidak mengatakan:

"Harga turun, beli lagi supaya average price turun."

Sebaliknya:

$$P_t < P$$

ADD hanya dilakukan jika:

$$\Delta P < 0$$

dan:

$$P_t < P$$

T.59. Floating loss

Sekarang menjawab salah satu pertanyaan awalmu:

Jika floating loss, biarkan saja?

Tidak secara otomatis.

Floating loss bukan alasan untuk exit.

Tetapi juga bukan alasan untuk hold.

Kita evaluasi:

$$P_t > P$$

vs:

$$P_t < P$$

T.60. Decision matrix

Secara sederhana:

Kondisi	Action
Loss + alpha masih kuat	HOLD
Loss + alpha melemah	REDUCE
Loss + alpha negatif	EXIT
Profit + alpha kuat	HOLD / TRAIL
Profit + alpha melemah	Tighten / Partial
Profit + alpha negatif	EXIT
Portfolio risk terlalu tinggi	REDUCE
Liquidity memburuk	REDUCE / EXIT

Jadi:

$$P_t$$

T.61. Mark-to-market bias

Ada bahaya psikologis maupun matematis:

$$P_t - P$$

Informasi tersebut bisa membuat model terlalu terikat pada entry price.

Padahal setelah trade masuk, pertanyaan optimal adalah:

Jika **sekarang saya tidak memiliki posisi**, apakah saya masih ingin membuka posisi ini?

Ini disebut **decision from current state**, bukan sunk-cost reasoning.

T.62. Counterfactual decision

Kita dapat menghitung:

$$tt$$

seandainya kita belum memiliki posisi.

Bandingkan dengan:

$$t$$

Jika:

$$t \quad 0,$$

maka posisi seharusnya tidak dipertahankan kecuali ada alasan exit-cost yang membuat continuation masih optimal.

T.63. Ini memberi kita prinsip yang sangat kuat

$$t \quad tttt$$

Kemudian tanyakan:

$$tttt$$

Jika tidak:

menjadi kandidat kuat.

T.64. Namun kita masih harus memperhitungkan exit cost

Jika posisi ditutup sekarang:

$$t$$

Continuation value:

Maka:

$$t \quad t$$

Ini menghindari churn.

T.65. Dynamic risk budget

Risk budget juga harus berubah.

Misalkan:

$$t$$

adalah drawdown.

Kita dapat mendefinisikan:

$$t = 0t$$

Misalnya:

$$= 1, \quad {}_105, \quad {}_1 \quad {}_205, \quad {}_2 \quad 0,$$

Ini adalah **drawdown control layer**.

T.66. Tetapi kita dapat membuatnya continuous

Misalnya:

$$\boxed{t = 0 - t}$$

Ketika drawdown meningkat:

$$t$$

maka:

$$t$$

T.67. Ini membuat sistem self-regulating

Saat performa memburuk:

Saat kondisi membaik:

Tetapi tetap dibatasi:

$$t$$

T.68. Sekarang kita bisa menggabungkan semuanya

Trade risk:

$$t$$

Strategy risk:

$$tt$$

Portfolio risk:

$$t$$

Market risk:

$$t$$

Kita ingin:

$$t \quad tt \quad t \quad t$$

Ini bukan inequality matematis literal dalam setiap implementasi, tetapi merupakan **hierarki risk budget**.

T.69. Final position sizing equation

Secara konseptual saya ingin kita menggunakan:

$$t = , t, t, , t$$

dengan:

$$= t, t - t, t \quad t$$

Tetapi risk budget-nya sendiri dinamis:

$$t, t = 0 \quad t \quad t \quad t \quad t$$

dengan t sebagai confidence.

T.70. Ini menghubungkan hampir seluruh tahap sebelumnya

Perhatikan:

$$t, t = 0 \quad tt$$

Jadi:

Model confidence tinggi

$$t \quad t$$

Regime buruk

$$t \quad t$$

Drawdown tinggi

$$t \quad t$$

Volatility tinggi

T.71. Tetapi jangan double-count risk

Ini peringatan penting.

Jika:

t

sudah masuk ke:

t

dan kita juga memasukkan:

t

terlalu agresif, kita bisa melakukan:

□

Jadi setiap adjustment harus memiliki definisi yang jelas.

T.72. Architecture final Tahap T

Saya ingin engine kita nanti dipisahkan menjadi:

```
T1 Signal Validation
T2 Alpha Estimation
T3 Confidence
T4 Risk Budget
T5 SL Engine
T6 TP Engine
T7 Position Sizing
T8 Portfolio Constraint
T9 Liquidity / Impact
T10 Order Placement
T11 Fill Management
T12 Position Management
T13 Dynamic SL/TP
T14 Exit Engine
T15 Risk Shutdown
```

Masing-masing bisa diuji secara independen.

T.73. Pseudocode konseptual

```
for every evaluation_time t:

    market_state = estimate_state(t)

    alpha = estimate_alpha(market_state)

    confidence = estimate_confidence(alpha)

    strategy_weight = allocate_capital(
        alpha,
        covariance,
        factors,
        confidence
    )

    for each candidate trade:

        expected_return = estimate_expected_return()

        if expected_return <= transaction_cost + hurdle:
            continue

        risk_budget = compute_dynamic_risk_budget(
            strategy_weight,
            confidence,
            volatility,
            regime,
            drawdown
        )

        stop_loss = estimate_SL(
            volatility,
            market_structure,
            distribution
        )

        position_size = solve_position_size(
            risk_budget,
            stop_loss,
            slippage,
            liquidity
        )

        if portfolio_risk_after_trade > risk_limit:
            reduce_size()

        take_profit = optimize_TP(
```

```
        expected_return_distribution,  
        stop_loss,  
        utility  
    )  
  
    order = construct_order(  
        entry,  
        position_size,  
        stop_loss,  
        take_profit  
    )  
  
    execute_or_place(order)
```

T.74. Dan posisi aktif

```
for every active_position:  
  
    state = update_position_state()  
  
    continuation_value = estimate_hold_value(state)  
  
    exit_value = estimate_exit_value(state)  
  
    if portfolio_risk > limit:  
        reduce_position()  
  
    elif continuation_value < exit_value:  
        exit_position()  
  
    else:  
  
        new_SL = optimize_SL(state)  
        new_TP = optimize_TP(state)  
  
        if new_SL improves expected utility:  
            modify_SL(new_SL)  
  
        if new_TP improves expected utility:  
            modify_TP(new_TP)
```

T.75. Satu hal yang sengaja belum kita lakukan

Kita **belum** membuat machine learning model yang sangat kompleks.

Ini disengaja.

Kita ingin terlebih dahulu memastikan:

□

benar.

Barulah model seperti:

- XGBoost,
- LightGBM,
- neural network,
- transformer,
- temporal model,

digunakan untuk memperkirakan:

$$P_{tt}$$

atau:

tt

Model ML hanyalah **estimator**.

Optimizer dan risk engine tetap menjadi lapisan keputusan.

T.76. Ini membuat sistem kita berbeda dari bot trading biasa

Bot biasa:

Sistem kita:

$$\boxed{tt \quad tt \quad Pt \quad t \quad t - t}$$

Dan yang lebih penting:

□

T.77. Hubungan dengan TSP, MaxCut, dan portfolio optimization

Sekarang kita bisa melihat di mana problem combinatorial sebenarnya muncul.

Portfolio allocation

□

Strategy selection

□

Execution routing

Misalnya memilih venue:

□

Order scheduling

Memilih urutan eksekusi:

$P -$

Pair / asset selection

Memaksimalkan diversity:

$t -$

Capital allocation

□

Jadi kita tidak memaksakan satu algoritma quantum ke semua hal.

T.78. Contoh MaxCut dalam sistem kita

Misalkan kita memiliki:

10

strategi.

Kita ingin membagi strategi menjadi dua kelompok:

t

dan:

t

Kita ingin strategi yang highly correlated berada di kelompok berbeda atau setidaknya mendapatkan penalti.

Graph:

$$= ,$$

Edge weight:

$$=$$

MaxCut:

$$\boxed{1 - \epsilon}$$

Ini dapat digunakan untuk mencari diversification structure.

T.79. TSP-like execution

Misalkan kita perlu mengeksekusi:

order pada beberapa venue.

Kita ingin meminimalkan:

$$\boxed{tt \quad t \quad tt}$$

Jika urutan eksekusi penting, problem dapat memiliki struktur:

$$\boxed{Pt}$$

T.80. Jadi akhirnya kita memiliki "optimization toolbox"

Bukan:

Tetapi:

$$\square$$

dengan toolbox:

T.81. Tahap T dalam satu persamaan besar

Secara konseptual, seluruh keputusan order dapat dirangkum:

$$\boxed{t = \quad_t \quad tt \quad t, t}$$

subject to:

$$t \quad t, t, t \quad t, t, t \quad , \quad , t \quad , t \quad , P t \quad t, t t$$

Iniilah sebenarnya tujuan akhir dari seluruh sistem.

T.82. Dan sekarang kita memiliki loop tertutup

Sebelumnya sistem kita terlihat seperti:

$$t$$

Sekarang:

$$t \quad t t \quad t t \quad P t \quad t t \quad t t$$

Jadi:

$$\square$$

T.83. Posisi Tahap T dalam keseluruhan proyek

- A Market & Data
 - B Return / Factor Modeling
 - C Alpha Construction
 - D Market Regime
 - E Risk Modeling
 - F Strategy Construction
 - G Portfolio / QUBO
 - H Quantum Mapping
 - I QAOA / Optimization
 - J Backtesting
 - K Execution Modeling
 - L Transaction Cost
 - M Robustness
 - N Walk Forward
 - O Stress Testing
 - P Statistical Validation
 - Q Strategy Selection
 - R Strategy Validation
 - S Strategy Ensemble & Capital Allocation
 - T Portfolio → Order + Dynamic Position Management
- |
- ▼
- U

Dan menurut saya **Tahap U** nanti harus menjadi tahap yang sangat berbeda dari T: bukan lagi sekadar bagaimana menentukan order, tetapi bagaimana kita membangun

execution/routing engine yang realistis terhadap order book, queue position, latency, slippage, partial fills, maker/taker fee, cancellation, dan adverse selection.

Itu akan menjadi lapisan yang sangat penting jika tujuan akhirnya adalah membuat sistem yang benar-benar mendekati paradigma **quantitative trading infrastructure**, bukan sekadar backtest strategy.

Prompt:

8/17/2026, 10:07:53 PM

lanjut Tahap U

Response:

8/17/2026, 10:07:54 PM

Remembering

Reviewed prior Tahap structure and preserved mathematical trading conventions

Worked for 7s

Tahap U - Execution & Order-Routing Engine

Kita sekarang masuk ke lapisan yang membuat sistem kita berpindah dari **quantitative strategy** menjadi sesuatu yang lebih mendekati **trading infrastructure**.

Pada Tahap T kita sudah menentukan:

$,, P, t,$

Tetapi kita masih mempunyai satu masalah:

Bagaimana order tersebut benar-benar masuk ke pasar tanpa membuat asumsi backtest yang terlalu ideal?

Di dunia nyata, ketika model mengatakan:

P

belum tentu kita mendapatkan:

P

Kita bisa mendapatkan:

pada harga berbeda, sebagian order tidak terisi, harga bergerak sebelum order sampai ke exchange, spread berubah, atau order kita terkena adverse selection.

Maka Tahap U akan membangun:

□

yang berada di antara:

□

U.1. Prinsip utama

Kita harus memisahkan dua objective:

Portfolio/strategy objective

dengan tujuan mendapatkan alpha.

Execution objective

t

dengan tujuan mendapatkan exposure yang diinginkan **semurah dan seaman mungkin**.

Jadi:

□

sedangkan:

□

U.2. Mengapa harga model bukan harga eksekusi?

Misalkan model menghasilkan:

$$P_t = 100$$

Kita ingin:

$$= 10$$

Order book:

| 10001 | 210002 | 10005 | 510010 | 10

Jika kita langsung market buy 10 unit, maka kita memperoleh:

210001 10002 510005

Average execution:

$$P = 210001 \ 10002 \ 51000510$$

Hasilnya:

$$P = 1000$$

Jadi:

$$P - P = 00$$

Itulah salah satu bentuk **execution slippage**.

U.3. Decomposition transaction cost

Saya ingin kita mendefinisikan:

$$t = t \ t \ t t$$

Masing-masing mempunyai makna berbeda.

U.4. Spread cost

Mid-price:

$$t = t \ t^2$$

Jika kita membeli menggunakan ask:

$$= t - t = t - t^2$$

Jika menjual menggunakan bid:

$$= t - t$$

Jadi secara sederhana:

$$= 2$$

Untuk round trip:

$$,$$

U.5. Fee

Misalkan fee:

Untuk notional:

$$= P,$$

maka:

$$=$$

Untuk round trip:

$$, =$$

Kita tidak boleh hanya memasukkan fee entry.

U.6. Slippage

Definisikan implementation shortfall terhadap decision price:

$$P_0 = P$$

Untuk long:

$$= P_t - P_0$$

Untuk short:

$$= P_0 - P_t$$

Secara umum:

$$\boxed{I_t = P_t - P_0}$$

dengan:

$$I_t = \begin{cases} 1, & \text{long} \\ -1, & \text{short} \end{cases}$$

U.7. Implementation Shortfall

Untuk seluruh order:

$$\boxed{I = P_t - P_0}$$

untuk long.

Namun implementation shortfall yang lebih lengkap adalah:

$$\boxed{I = P_t - P_0 + \text{fee}}$$

Ini sangat berguna untuk mengevaluasi execution engine kita.

U.8. Arrival price

Kita membutuhkan sebuah referensi:

$$P$$

Biasanya:

$$P = t_0$$

Kemudian kita bertanya:

Berapa biaya yang sebenarnya kita keluarkan sejak keputusan dibuat?

Bukan sekadar:

Apakah order akhirnya profit?

U.9. Latency

Misalkan:

$$t = ,$$

$$t = ,$$

$$t =$$

Maka:

$$\boxed{= t - t}$$

Dalam market yang cepat, selama:

$$0,$$

harga dapat berubah.

Maka:

$$P_t t \quad P_t t$$

U.10. Expected latency cost

Misalkan instantaneous price drift:

$$t$$

Maka approximation:

$$\Delta P_t$$

Tetapi untuk short horizon, stochastic component lebih penting:

$$\Delta P =$$

Sehingga:

$$\Delta P = \sigma \sqrt{t}$$

Semakin besar latency:

$$t$$

U.11. Market impact

Ini bagian yang jauh lebih menarik.

Jika kita membeli:

unit, kita menghabiskan liquidity pada ask.

Harga rata-rata dapat dimodelkan:

$$P = P_0 + \lambda \sqrt{t}$$

Model sederhana:

$$t =$$

dengan:

- λ : impact coefficient,
- σ : volatility,
- V : volume,
- s : order size,
- α : impact exponent.

U.12. Mengapa volatility masuk?

Karena market yang volatile cenderung mempunyai:

- wider effective spread,
- lower certainty of liquidity,
- larger price response terhadap order.

Maka:

t

Ini berarti sistem kita secara otomatis menganggap execution menjadi lebih mahal ketika market menjadi lebih volatile.

U.13. Liquidity ratio

Kita dapat mendefinisikan:

$$P_{ttt} =$$

Misalnya:

$$= 1000$$

dan:

$$= 1,000,000$$

Maka:

$$P_{ttt} = 01$$

Semakin besar participation rate:

t

U.14. Order Book Depth

Untuk short-term trading, historical volume saja tidak cukup.

Kita ingin mengetahui:

$$=_{=1}$$

untuk level ask.

Begitu pula:

$$=_{=1}$$

U.15. Market order simulation

Misalkan:

$$= 100$$

Order book ask:

$$P = 10001 \mid 2010002 \mid 01000 \mid 50$$

Maka:

$$_1 = 20,$$

$$_2 = 0,$$

$$= 50$$

Execution price:

$$P_P = 2010001 \ 010002 \ 501000100$$

Inilah yang seharusnya dilakukan simulator kita.

Bukan:

$$P_t = 10001$$

U.16. VWAP

Secara umum:

$$P = P$$

Untuk market order:

$$P = P$$

U.17. Limit order

Sekarang lebih menarik.

Misalkan:

$$P_t = 5$$

Order tidak langsung dieksekusi.

Kita memiliki:

$$P$$

sebagai random variable.

Jadi:

Jika:

$$= 1,$$

order filled.

Jika:

$$= 0,$$

order tidak mendapat exposure.

U.18. Fill probability

Kita dapat membangun:

$$= , , , ,$$

dengan:

$$= P_t - P$$

dan:

$$=$$

U.19. Queue position

Misalkan:

$$= 500$$

Order kita:

$$= 100$$

Jika volume yang dieksekusi pada harga tersebut baru:

$$00,$$

maka kita belum filled.

Jika:

$$_t = 550,$$

maka secara ideal:

$$500$$

untuk queue di depan dan:

$$50$$

untuk order kita.

Sehingga:

$$= 50$$

U.20. Partial fill

Maka:

$$0 \quad tt$$

Kita definisikan:

$$\boxed{t = tt}$$

Misalnya:

$$t = 05$$

Artinya hanya 35% posisi yang berhasil diperoleh.

U.21. Partial fill adalah decision problem

Kita kemudian harus memutuskan:

$$\in \{, , P, \}$$

Misalnya:

$$= 50$$

Kita harus menghitung expected value dari masing-masing action.

U.22. Optimal cancellation

Misalkan:

$$t = t - t$$

Sedangkan:

$$t = t - t$$

Kita memilih:

$$\boxed{=}$$

Ini berarti cancellation bukan aturan arbitrer seperti:

"Cancel setelah 5 menit."

Tetapi:

$$t, t$$

U.23. Adverse selection

Ini salah satu konsep paling penting untuk limit order.

Misalkan kita memasang buy limit:

$$P_t = 100$$

Order kita filled.

Mengapa?

Bisa jadi karena:

$$P_t > 100$$

Dengan kata lain:

Kita mendapatkan fill justru ketika informasi pasar sedang buruk bagi kita.

Jadi:

$$P_t$$

dan:

$$t$$

tidak independen.

U.24. Conditional fill model

Kita harus memodelkan:

$$t, t$$

bukan:

$$t, t$$

Ini perbedaan yang sangat penting.

U.25. Expected value limit order

Maka:

$$t = P - P - t$$

Jika no-fill berarti kita kehilangan opportunity:

juga harus diperhitungkan.

U.26. Market vs limit order

Sekarang kita bisa membuat decision:

$$t$$

Jika:

$$t > t_s$$

gunakan marketable order.

Jika:

$$t \leq t_s$$

gunakan limit.

Ini lebih sophisticated daripada:

"Selalu gunakan limit order agar fee murah."

U.27. Order type sebagai optimization variable

Kita definisikan:

$$t \in \{, , P, , , P, P\}$$

Kemudian:

$$t = t$$

U.28. TWAP

Untuk order besar:

Daripada membeli semuanya:

sekaligus, kita bagi:

$$1, 2, \dots$$

dengan:

$$=$$

TWAP mengatur execution relatif merata terhadap waktu.

U.29. VWAP execution

Jika expected market volume:

$$1, 2, \dots$$

maka:

$$=$$

Sehingga:

$$\boxed{= t}$$

Kita mengikuti volume market.

U.30. Tetapi kita bisa membuat execution adaptif

Misalkan alpha kita:

$$t$$

Jika:

$$t > 0,$$

kita ingin memperoleh exposure lebih cepat.

Jika:

$$t < 0,$$

kita dapat lebih sabar.

Jadi execution schedule:

$$t = t_1, t_2, t_3, \dots, t_t$$

U.31. Urgency

Definisikan:

$$t \in 0, 1$$

Dengan:

$$t = 0$$

berarti:

Sedangkan:

$$t = 1$$

berarti:

Kemudian:

$$t = , t, tt, t$$

U.32. Time decay alpha

Misalkan expected alpha:

$$\tau = 0^{-\tau}$$

Semakin lama kita menunggu:

$$\tau$$

maka:

$$\tau$$

Maka urgency meningkat:

$$\tau$$

U.33. Ini menghasilkan execution policy

$$t = tt, t, t_t, t, t, t_t, t_t$$

Jadi execution sendiri menjadi sebuah **policy optimization problem**.

U.34. Market impact vs opportunity cost

Ini inti dari optimal execution.

Kalau kita eksekusi terlalu cepat:

$$t$$

Kalau terlalu lambat:

$$ttt$$

Maka kita mempunyai:

$$tt = tt \ ttt$$

Dan:

$$\tau = \frac{t\tau \ ttt\tau}{\tau}$$

U.35. Model sederhana

Misalkan:

$$tt\tau = \tau$$

dan:

$$ttt\tau = \tau$$

Maka:

$$\tau = \tau \ \tau$$

Turunan:

$$\tau = -\tau^{-1}$$

Set:

$$\tau = 0$$

Maka:

$$\tau^{-1} =$$

Sehingga:

$$\tau = \frac{11}{\tau}$$

Ini merupakan contoh sederhana bagaimana **execution horizon dapat ditentukan secara matematis**.

U.36. Hubungan dengan alpha

Jika:

,

maka:

τ

Artinya:

Alpha kuat → eksekusi lebih cepat.

Sebaliknya:

τ

Ini sangat masuk akal secara finansial.

U.37. Sekarang kita masukkan risk

Execution tidak boleh meningkatkan risk melebihi budget.

Misalkan target:

tt

Setiap delay memberikan exposure uncertainty:

$$= \tau^2$$

Maka:

$$\tau^2_{2tt,}$$

Ini menghasilkan upper bound terhadap execution horizon.

U.38. Execution constraint

Jadi:

$$\tau^2_{2tt,}$$

Market semakin volatile:

maka:

τ

Sekali lagi:

Volatility tinggi → kita tidak boleh terlalu santai saat eksekusi.

U.39. Exchange/venue selection

Untuk sistem multi-market, kita mungkin mempunyai:

$$= \{1, 2, \dots\}$$

Setiap venue memiliki:

$$= t \ t$$

Kita memilih:

$$=$$

U.40. Tetapi venue juga memiliki liquidity

Maka:

$$=$$

Sehingga:

$$=$$

Dan untuk order besar, kita bahkan dapat melakukan split:

$$1 \ 2 \ =$$

U.41. Venue allocation sebagai optimization problem

Misalkan:

adalah fraction order ke venue .

Maka:

$$= 1, \\ 0$$

Objective:

$$=$$

subject to:

$$= 1$$

Ini sudah merupakan **capital allocation problem**, hanya saja pada level execution.

U.42. Dan di sinilah QUBO dapat muncul lagi

Jika keputusan venue dibatasi menjadi:

$$\in \{0, 1\},$$

kita dapat membangun:

$$\square$$

Misalnya:

$$^2 \\ - 1$$

Penalty memastikan hanya satu venue dipilih.

U.43. Jika multi-venue splitting

Kita bisa menggunakan discrete quantity:

$$, \in \{0, 1\}$$

untuk memilih:

venue mendapatkan tranche .

Kemudian objective:

$$\boxed{t t t P t}$$

Ini menjadi candidate problem untuk QUBO.

U.44. Tetapi kita harus disiplin

Jangan menggunakan QAOA hanya karena kita bisa.

Jika:

dapat diselesaikan dengan:

- MILP,
- dynamic programming,
- quadratic programming,
- greedy,
- simulated annealing,

maka semuanya harus menjadi benchmark.

Prinsip proyek kita tetap:

$$\boxed{tt \quad ttt}$$

U.45. Event-driven execution simulator

Tahap U harus diwujudkan dalam simulator event-driven.

Event:

$$t \in \{P, \dots, P, \dots, P, PP\}$$

U.46. State machine

Order memiliki state:

$$\boxed{P}$$

atau:

atau:

Ini penting agar backtest tidak melakukan:

```
signal = buy
price = close
filled = true
```

secara instan.

U.47. Latency-aware simulator

Kita harus mempunyai:

t

t_t

t

t

Dengan:

$t \ t_t \ t \ t$

Kemudian harga yang digunakan untuk fill adalah:

$P_t,$

bukan:

P_t

U.48. Ini mencegah look-ahead

Misalnya candle 10:00-10:01.

Jika signal baru diketahui pada:

10 01 00,

maka simulator tidak boleh mengisi order menggunakan:

10 00 0

Ini merupakan:

$\boxed{t_t t}$

U.49. Untuk sistem kita, ini wajib

Karena kita menggunakan:

- crypto,
- saham internasional,
- gold,
- USD/DXY,
- market factors,

timestamp antar instrumen bisa berbeda.

Jadi kita membutuhkan:

$\boxed{t}$

bukan sekadar row number.

U.50. Multi-asset synchronization

Misalnya:

$$t$$

tersedia setiap detik.

Tetapi:

$$t$$

hanya tersedia ketika market FX buka.

Dan:

$$P_t$$

memiliki jam perdagangan tertentu.

Kita tidak boleh mengisi data masa depan secara sembarangan.

Untuk setiap asset :

$$t_t = \{\tau \quad t \quad t\tau\}$$

Kemudian:

$$t = tt_t$$

Tetapi kita harus menyimpan timestamp aslinya:

$$t = t - t_t$$

U.51. Staleness penalty

Jika:

$$t$$

terlalu besar:

$$t \quad ,$$

maka:

$$tt =$$

Ini penting untuk factor seperti:

U.52. Cross-asset latency

Misalkan:

bergerak sekarang.

Model menggunakan:

yang terakhir diperbarui 20 menit lalu.

Kita tidak boleh berpura-pura bahwa DXY merupakan informasi real-time.

Maka state kita harus menyimpan:

$$,,,P,$$

sebagai bagian dari market state.

U.53. Execution state

Saya menyarankan state lengkap menjadi:

$$_t = ttt, Pttt, tt, ttt$$

Dengan:

$$ttt = ,t, ,t, t, t,$$

U.54. Execution alpha

Ini konsep yang sangat menarik.

Kita memiliki:

$$tt$$

dan:

$$t$$

Strategy alpha mengatakan:

"Saya memperkirakan harga naik."

Execution alpha mengatakan:

"Saya memperkirakan sekarang adalah waktu/venue/order type terbaik untuk mendapatkan exposure tersebut."

Jadi:

$$t = tt - tt$$

U.55. Contoh

Strategy:

$$= 20$$

Execution A:

$$t = 5$$

Maka:

$$t = 15$$

Execution B:

$$t = 1$$

Maka:

$$t = 2$$

Keduanya menggunakan signal yang sama.

Tetapi execution engine menentukan apakah trade tersebut masih layak.

U.56. Trade admission

Maka sebelum order:

$$t = -t$$

dan trade hanya diterima jika:

$$t$$

Dengan sebagai hurdle rate.

U.57. Ini sangat penting untuk short-term trading

Semakin pendek holding period:

,

semakin besar kemungkinan:

t

mengonsumsi sebagian besar:

Maka:

$t \quad tt$

U.58. Backtest metric baru

Kita tidak cukup menggunakan:

- Sharpe ratio,
- CAGR,
- max drawdown.

Kita harus mengukur execution:

Implementation Shortfall

Fill Ratio

$$= \frac{tt}{t}$$

Slippage

$$= P - P$$

Market Impact

$$= P - P$$

Latency

$$= t - t$$

Cancellation Rate

$$= \frac{tt}{t}$$

U.59. Adverse Selection Metric

Untuk limit order:

$$\Delta = P_t - P$$

untuk long.

Jika:

$$0,$$

maka fill cenderung diikuti harga turun.

Itu berarti:

$$\boxed{tttt}$$

U.60. Implementation Shortfall decomposition

Kita dapat memecah:

$$\boxed{= t}$$

Dengan ini kita dapat mengetahui **mengapa strategy kehilangan performance.**

Misalnya:

$$= 0$$

Tetapi:

$$= 2$$

$$= 4$$

$$t =$$

$$=$$

Maka:

$$t = 15$$

U.61. Dan ini membuka optimasi yang sangat menarik

Jika kita melihat:

$$t = ,$$

kita bisa mengoptimalkan:

$$t$$

Jika:

$$= 2,$$

kita optimalkan:

$$t$$

Jika:

$$= 4,$$

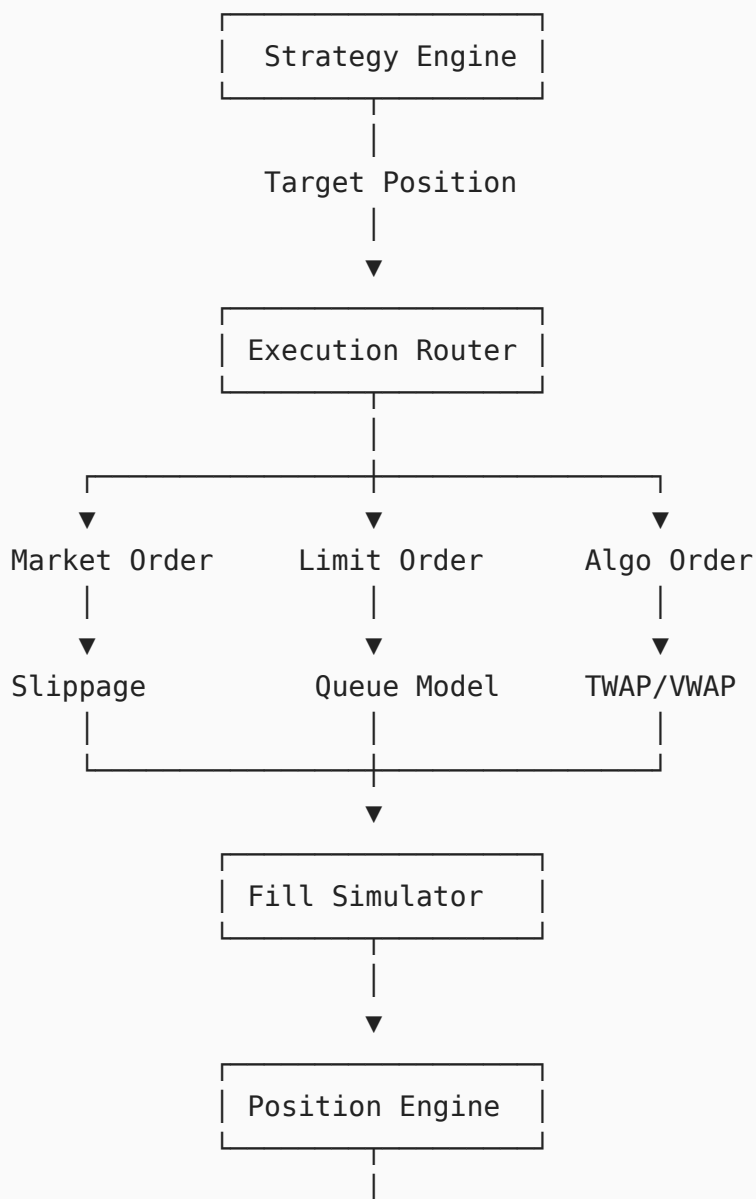
kita optimalkan:

$$t$$

Jadi diagnostic decomposition langsung memberikan roadmap improvement.

U.62. Arsitektur Tahap U

Saya sarankan:



U.63. Dan kita sekarang mempunyai dua optimizer

Ini menurut saya salah satu hasil terpenting dari Tahap U.

Optimizer I - Investment/Trading Optimizer

Menentukan:

$\square$

Contoh:

$, t, ,$

Optimizer II - Execution Optimizer

Menentukan:

$\square$

Contoh:

$, , t, tt, PP$

U.64. Unified objective

Akhirnya kita bisa menuliskan:

$\square$

dengan:

$=$

dan:

$=$

Subject to:

$, , t, t, , , tt$

Jadi bukan lagi:

Tetapi:

$\square$

U.65. Hubungan dengan sistem keseluruhan

Sekarang rantai kita menjadi:

$$\boxed{t \quad tt \quad t \quad Pt \quad tt \quad tPt \quad ttt \quad Pt \quad tt}$$

Dan karena:

$$tt$$

kembali masuk ke model, maka sistem benar-benar menjadi:

$$\square$$

U.66. Hal yang menurut saya paling penting dari Tahap U

Ada satu perubahan paradigma yang ingin saya tetapkan untuk implementasi kita nanti:

Backtest tidak boleh lagi menganggap bahwa "signal menghasilkan trade".

Yang benar adalah:

$$\boxed{tPt \quad t \quad tPt}$$

Dengan demikian kita dapat membedakan tiga hal:

$$\square$$

vs.

$$\square$$

vs.

$$\square$$

Ketiganya bisa berbeda secara signifikan.

U.67. Dan ini menjadi fondasi Tahap V

Setelah Tahap U, kita sudah memiliki hampir semua mekanisme yang dibutuhkan untuk membuat **simulator trading event-driven yang realistis**.

Maka Tahap berikutnya seharusnya bukan sekadar menambahkan indikator atau model ML baru. Yang jauh lebih penting adalah:

$\boxed{\text{Tahap V - Event-Driven Backtesting \& Market Replay Engine}}$

Di sana kita akan menyatukan:

$$\begin{matrix} t & t & t & P & t & t & t & P & t \end{matrix}$$

menjadi **satu simulator deterministik** yang dapat menjawab pertanyaan paling penting:

"Jika seluruh sistem kita benar-benar dijalankan pada informasi yang tersedia pada waktu t , dengan order yang benar-benar harus melewati execution process, berapa performance yang sebenarnya akan diperoleh?"

Dan itu juga akan menjadi tempat kita memastikan tidak terjadi **look-ahead bias**, **survivorship bias**, **data leakage**, **unrealistic fills**, atau **hindsight optimization**-sebelum hasil sistem kita mulai dibandingkan dengan benchmark klasik maupun QAOA.

Prompt:

8/17/2026, 10:09:11 PM

lanjut Tahap V

Response:

8/17/2026, 10:09:12 PM

Tahap V — Event-Driven Backtesting & Market Replay Engine

Sekarang kita masuk ke salah satu tahap **paling penting** dari seluruh arsitektur kita.

Kalau Tahap U menjawab:

"Bagaimana keputusan trading diterjemahkan menjadi order yang realistis?"

maka Tahap V menjawab:

"Bagaimana kita menghidupkan kembali pasar masa lalu sedemikian rupa sehingga strategi kita hanya mengetahui informasi yang memang tersedia pada saat itu?"

Ini yang akan menentukan apakah hasil backtest kita **layak dipercaya** atau hanya terlihat bagus karena simulator terlalu ideal.

V.1. Perubahan paradigma

Backtest sederhana biasanya seperti ini:

$$\begin{matrix} t_t & t & P & t_t & t_t \end{matrix}$$

Masalahnya, proses tersebut sering secara implisit mengasumsikan:

- order langsung terisi,
- harga eksekusi sama dengan harga signal,
- tidak ada latency,
- tidak ada spread,
- tidak ada partial fill,
- tidak ada queue,
- tidak ada rejection,
- tidak ada order cancellation,
- semua data tersedia secara simultan.

Untuk sistem kita, itu **tidak boleh**.

Kita ingin:

$$\boxed{t \quad t \quad tt \quad Pt \quad P}$$

V.2. Event-driven architecture

Pasar kita representasikan sebagai sequence:

$$= \{1, 2, \dots\}$$

dengan:

$$t_1 \quad t_2 \quad t$$

Setiap event:

$$= t, t, t$$

Contohnya:

$$= 10000102, P,$$

Atau:

$$= 100001051, , 1$$

V.3. Mengapa event lebih baik daripada candle?

Misalkan candle 1 menit:

$$100, 105, , 10$$

Kita hanya mengetahui:

$$= 100, = 105, = , = 10$$

Tetapi kita tidak mengetahui urutan sebenarnya.

Apakah:

$$100 \ 105 \ 10$$

atau:

$$100 \ 105 \ 10$$

Untuk strategi intraday dengan:

- SL,
- TP,
- limit order,

perbedaan tersebut sangat besar.

V.4. Ambiguity pada candle

Misalkan kita punya:

$$\begin{aligned} t &= 100, \\ &= , \\ P &= 105 \end{aligned}$$

Candle:

$$= 100, = 10, = , = 10$$

Dari OHLC saja kita tidak tahu apakah:

$$P$$

atau:

$$P$$

Tetapi PnL bisa sangat berbeda.

Karena itu:

$$\boxed{ttt}$$

V.5. Market replay

Kita ingin melakukan:

$$t_t$$

seolah-olah waktu berjalan dari masa lalu ke masa depan.

Pada setiap event:

$$t,$$

engine hanya boleh membaca:

$$t$$

yaitu information set yang tersedia sampai t .

V.6. Filtration

Dalam bahasa stochastic process, kita mendefinisikan filtration:

$$t = \tau \quad \tau \quad t$$

Strategi kita harus memenuhi:

$$t \in t$$

Artinya keputusan trading pada waktu t hanya boleh bergantung pada informasi yang tersedia hingga t .

Ini adalah fondasi matematis untuk menghindari:

$$\square$$

V.7. Contoh look-ahead bias

Misalkan kita ingin memprediksi return:

$$t1$$

Tetapi secara tidak sengaja feature menggunakan:

$$t1$$

Maka:

$$t = t, t1$$

Strategi memang akan terlihat sangat bagus.

Tetapi:

$$t1 \quad t$$

Jadi strategi tersebut **tidak kausal**.

V.8. Causality condition

Secara formal:

$$t = t$$

Bukan:

$$t = t$$

untuk:

$$0$$

V.9. Market state

Kita definisikan state:

$$t$$

Misalnya:

$$t = P_{tt} t_{tt} P_{ttt}$$

Kemudian:

$$t_1 = t, t_1$$

V.10. Strategy sebagai policy

Strategy kita sekarang dapat ditulis:

$$t = t$$

dengan:

$$t = \{ttt, t, tt, , , P\}$$

Parameter:

berasal dari seluruh model kita:

- factor model,
 - alpha,
 - regime,
 - risk,
 - optimizer,
 - execution policy.
-

V.11. Event loop

Secara konseptual:

```
while historical_data remains:

    event = next_event()

    update_market_state(event)

    update_orders(event)

    process_fills(event)

    update_positions()

    update_risk()

    generate_signal()

    optimize_target_position()

    generate_execution_plan()

    submit_orders()

    record_state()
```

Tetapi urutan ini harus sangat hati-hati.

Kita tidak boleh menggunakan informasi dari event sebelum event tersebut diproses.

V.12. Urutan kausal

Pada timestamp t :

$$\boxed{tt \ ttt}$$

Order tersebut baru dapat diproses pada:

$$t + \Delta t_t$$

Jadi:

$$\boxed{t \ t \ t}$$

V.13. Decision timestamp

Setiap keputusan harus mempunyai:

$$t$$

Contoh:

$$10 \ 15 \ 00000$$

Kemudian:

$$t_t = 10 \ 15 \ 0000,$$

$$t = 10 \ 15 \ 00010,$$

$$t = 10 \ 15 \ 00021$$

Dengan demikian kita dapat menghitung:

$$tt = t - t$$

V.14. Order object

Saya menyarankan setiap order memiliki struktur:

$$= \{, , , t, tt, t, t, tt, tt\}$$

Misalnya:

$$_1 = \{, , , 001, 0000, \}$$

V.15. Order lifecycle

Order:

kemudian:

kemudian:

Setelah itu:

$$P$$

atau:

Atau:

Atau:

V.16. Position state

Untuk setiap asset:

$$t$$

Cash:

$$t$$

Portfolio value:

$${}_t = {}_t {}_t P_t$$

Untuk multi-asset:

$$\boxed{{}_t = {}_t \quad , {}_t P_{,t}}$$

V.17. Realized dan unrealized PnL

Unrealized:

$$P_t = {}_t P_t - P_t$$

Realized:

$$P_t = {}_t \quad P$$

Total:

$$P_t = P_t P_t$$

V.18. Cash accounting

Untuk buy:

$$t1 = t - P -$$

Untuk sell:

$$t1 = t P -$$

Ini sederhana tetapi sangat penting.

Jangan pernah menghitung return hanya dari:

$$P$$

V.19. Short selling

Untuk short:

$$_t 0$$

Portfolio value tetap:

$$_t = _t _t P_t$$

Jika:

$$P_t ,$$

maka:

$$_t P_t - P_t 0$$

V.20. Leverage

Gross exposure:

$$_t = ,_t P_{,t}$$

Leverage:

$$_t = \#$$

Constraint:

$$_t$$

V.21. Margin

Jika short atau leveraged position digunakan, simulator harus mempunyai:

$$t_t$$

Jika:

$$t_t \quad t_{t,}$$

maka:

$$\boxed{t}$$

Ini harus disimulasikan.

V.22. Stop-loss bukan sekadar angka

Misalkan:

$$= 5$$

Kita tidak boleh otomatis mengatakan:

$$P \ 5 \ = 5$$

Realitanya:

$$P_t = 5$$

kemudian market order dikirim.

Harga fill dapat:

$$P \ 5$$

Jadi:

$$\boxed{t \quad t}$$

V.23. Stop-loss slippage

Untuk long:

$$P_{,} = P -$$

Dengan:

$$0$$

Pada crash:

dapat sangat besar.

Ini sangat penting untuk crypto dan small-cap stocks.

V.24. Take-profit juga tidak sempurna

Jika TP menggunakan limit:

$$P_P$$

Maka:

$$P = P_P$$

hanya jika liquidity tersedia.

Jika menggunakan market:

$$P \neq P_P$$

karena slippage.

V.25. OCO logic

Position biasanya mempunyai:

dan:

$$P$$

Keduanya harus dihubungkan sebagai:

$$\boxed{P = t}$$

Jika TP filled:

Jika SL filled:

$$P$$

V.26. Kalau tidak menggunakan OCO

Simulator dapat melakukan hal absurd:

$$P$$

kemudian beberapa event berikutnya:

Hasilnya posisi menjadi terbalik.

Ini adalah salah satu bug backtest yang sangat umum.

V.27. Trailing stop

Misalkan:

$$t = P_t -$$

Tetapi SL hanya boleh bergerak satu arah untuk long:

$$t1 \geq t$$

Jadi:

$$t1 = t, P_{t1} -$$

Untuk short:

$$t1 = t, P_{t1}$$

V.28. Break-even stop

Misalkan entry:

$$P$$

Setelah profit mencapai threshold:

$$t \geq$$

kita ubah:

$$= P$$

Namun sebenarnya kita harus mempertimbangkan biaya.

Jika fee + expected exit cost:

$$,$$

maka true break-even:

$$P = P$$

Untuk long.

V.29. Dynamic TP

Ini menghubungkan langsung dengan ide kita sebelumnya.

Misalkan:

$$P_t = t t, , t, t$$

Jadi TP tidak harus fixed.

Misalnya:

$$P_t = P_{tt}$$

Dengan:

$$t = t, t$$

V.30. Tetapi jangan mengubah TP karena hindsight

Misalnya harga sudah naik 5%.

Tidak boleh kemudian:

"Karena sekarang terlihat kuat, TP kita naikkan."

Kita hanya boleh menaikkan TP jika:

$$\boxed{t = t}$$

V.31. Market replay untuk cross-asset system

State kita harus mencakup:

$$t = , , t, , , P, , t,$$

Tetapi setiap asset memiliki timestamp sendiri.

Kita menggunakan:

$$\boxed{t}$$

yang menggabungkan semua event berdasarkan timestamp.

V.32. Priority queue

Misalkan:

$$= \{1, 2, , \}$$

Sort:

$$t_1 \ t_2$$

Engine selalu mengambil:

=

∈

t

Kemudian memprosesnya.

Ini merupakan inti event-driven simulator.

V.33. Data hierarchy

Saya menyarankan data kita dibagi menjadi:

Level 0

Raw market data.

Level 1

Order book:

$$12$$

Level 2

Trades:

$$Pt$$

Level 3

Bars:

$$1, 1, 5,$$

Level 4

Derived features:

$$tt, t, t,$$

Level 5

Model state:

V.34. Jangan mencampur level

Raw data:

harus immutable.

Kemudian:

t

Ini memungkinkan kita melakukan audit.

V.35. Point-in-time dataset

Untuk setiap feature:

$,t$

kita harus menyimpan:

tt_t

dan:

tt_t

Karena:

$tt_t \quad tt_t$

V.36. Contoh fundamental data

Misalnya laporan perusahaan:

1

Tanggal laporan:

10

Tetapi periode laporan:

—

Kita tidak boleh menggunakan data Q1 sebelum:

10

Maka:

$$tt$$

adalah yang menentukan kapan informasi boleh masuk ke model.

V.37. Ini disebut point-in-time data

Untuk setiap observation:

$$= , tt, tt$$

Backtest menggunakan:

$$tt$$

V.38. Survivorship bias

Misalkan pada 2026 kita melihat:

$$100t$$

Kemudian backtest 2021 menggunakan daftar tersebut.

Itu salah.

Karena pada 2021 kita belum tahu perusahaan mana yang akan menjadi top 100 pada 2026.

Universe harus:

$$t$$

dan berubah sesuai waktu.

V.39. Delisted assets

Jika perusahaan delisting pada:

$$2024,$$

kita tidak boleh menghapus seluruh history-nya dari database.

Justru:

$$t_{2021-2024}$$

harus tetap ada.

Jika tidak:

V.40. Corporate actions

Untuk saham:

- split,
- dividend,
- rights issue,
- merger,
- delisting.

Semua harus dipertimbangkan.

Misalnya:

2 1t

Harga:

100 50

Tetapi posisi:

1 2

Simulator harus mengubah quantity dan price secara konsisten.

V.41. Crypto memiliki masalah berbeda

Untuk crypto:

- exchange listing,
- delisting,
- token migration,
- redenomination,
- perpetual funding,
- liquidation,
- maker/taker fee,
- funding rate.

Jadi engine kita harus modular.

V.42. Funding rate

Untuk perpetual futures:

$$P = P_t$$

Secara periodik:

$$= t$$

Maka total PnL:

$$P = P_t - P_{t-1}$$

V.43. Karena itu strategi crypto dan saham tidak boleh hard-coded

Kita sebaiknya mempunyai interface:

$$t$$

dengan implementasi:

$$tt,$$

$$tt,$$

$$t,$$

$$tt$$

V.44. Tetapi strategy tetap sama

Strategy menerima:

$$tt_t$$

Kemudian:

$$t_t = tt_t$$

Market model yang menentukan bagaimana action tersebut dieksekusi.

Ini membuat arsitektur:

$$ttt$$

terhadap asset class.

V.45. Deterministic replay

Ini sangat penting untuk penelitian.

Jika kita menjalankan:

$$tt$$

dua kali pada dataset yang sama, seharusnya:

$$t_1 = t_2$$

Kecuali kita memang memasukkan stochastic process.

Untuk stochastic model kita gunakan:

Sehingga:

$$=$$

memberikan:

$$\boxed{t}$$

V.46. Monte Carlo execution

Setelah deterministic replay bekerja, baru kita masukkan uncertainty.

Misalnya slippage:

$$tt, t$$

Kemudian:

$$P = P$$

Kita lakukan:

simulasi.

V.47. Distribution of outcomes

Kita mendapatkan:

$$\{P_1, \dots, P\}$$

Kemudian:

$$P,$$

$$P,$$

$$,$$

Jadi bukan hanya:

$$tt^2$$

tetapi:

$$= 2,5 = ,$$

V.48. Stress testing

Kita juga mengubah:

$$,$$

$$,$$

$$t,$$

$$tt,$$

$$t$$

Misalnya:

$$2$$

Jika strategy masih profitable:

$$t$$

Jika langsung hancur:

$$tt$$

V.49. Execution stress matrix

Kita bisa membangun:

$$P = , , t, t$$

Contoh:

Scenario	Slippage	Spread	Latency	Sharpe
Optimistic	0.5×	0.5×	0.5×	...
Base	1×	1×	1×	...
Conservative	2×	2×	2×	...

Scenario	Slippage	Spread	Latency	Sharpe
Stress	5×	3×	5×	...

Ini jauh lebih informatif daripada satu angka backtest.

V.50. Walk-forward testing

Kita tidak boleh:

2021–2025

kemudian:

$t_{2021-2025}$

Sebaliknya:

$t \quad t$

berdasarkan waktu.

V.51. Rolling walk-forward

Misalnya:

2021 – 2022

$t \quad 2021$

$t \quad 2022$

Kemudian window bergeser:

20212 – 2021

$t \quad 2022$

$t \quad 202$

Dan seterusnya.

V.52. Parameter stability

Misalkan optimizer menghasilkan:

Kita tidak ingin:

berubah drastis setiap periode.

Definisikan:

$$\Delta_t = t - t_{-1}$$

Jika:

$$\Delta_t \rightarrow 0,$$

bisa jadi model overfit terhadap regime tertentu.

V.53. Hyperparameter optimization

Ini harus dilakukan **hanya pada training/validation**.

Bukan test.

Jika:

$$t$$

digunakan untuk memilih parameter terbaik, maka:

$$t$$

bukan lagi out-of-sample.

V.54. Nested optimization

Arsitektur kita akhirnya akan mempunyai:

$$\boxed{t \quad P_t \quad t}$$

dan masing-masing memiliki parameter.

Kita harus membedakan:

$$,$$

$$t,$$

$$t$$

V.55. Jangan optimalkan semuanya sekaligus

Jika kita langsung melakukan:

terhadap:

- alpha,
- risk,
- QUBO,
- execution,
- TP,
- SL,
- trailing,
- leverage,

kita hampir pasti memperoleh overfitting.

Lebih baik:

t

V.56. Validation hierarchy

Layer 1

Market data integrity.

Layer 2

Feature correctness.

Layer 3

Signal correctness.

Layer 4

Portfolio optimization.

Layer 5

Execution.

Layer 6

Full-system backtest.

Layer 7

Stress test.

Layer 8

Out-of-sample.

Layer 9

Paper trading.

Layer 10

Live trading dengan size sangat kecil.

V.57. Ini sangat penting untuk proyek kita

Karena kita menggunakan:

,
,
,
,

kita harus membuktikan bahwa improvement bukan berasal dari:

- look-ahead,
- data leakage,
- unrealistic execution,
- parameter tuning,
- random seed,
- simulator artifact.

V.58. Quantum benchmark

Misalkan classical optimizer menghasilkan:

Quantum optimizer menghasilkan:

Kita tidak boleh langsung menyimpulkan:

tt

Kita harus memastikan:

$t, tt, tt, t, , tt$

semuanya sama.

V.59. Paired experiment

Untuk setiap periode:

$$t,$$

jalankan:

$$ttt$$

dan:

$$ttt$$

dengan kondisi identik.

Kemudian:

$$\Delta_t = t - t$$

Kita bisa menguji:

$$_0 \Delta = 0$$

V.60. Dan bukan hanya return

Bandingkan:

,

$$t,$$

,

,

,

$$tt,$$

$$t,$$

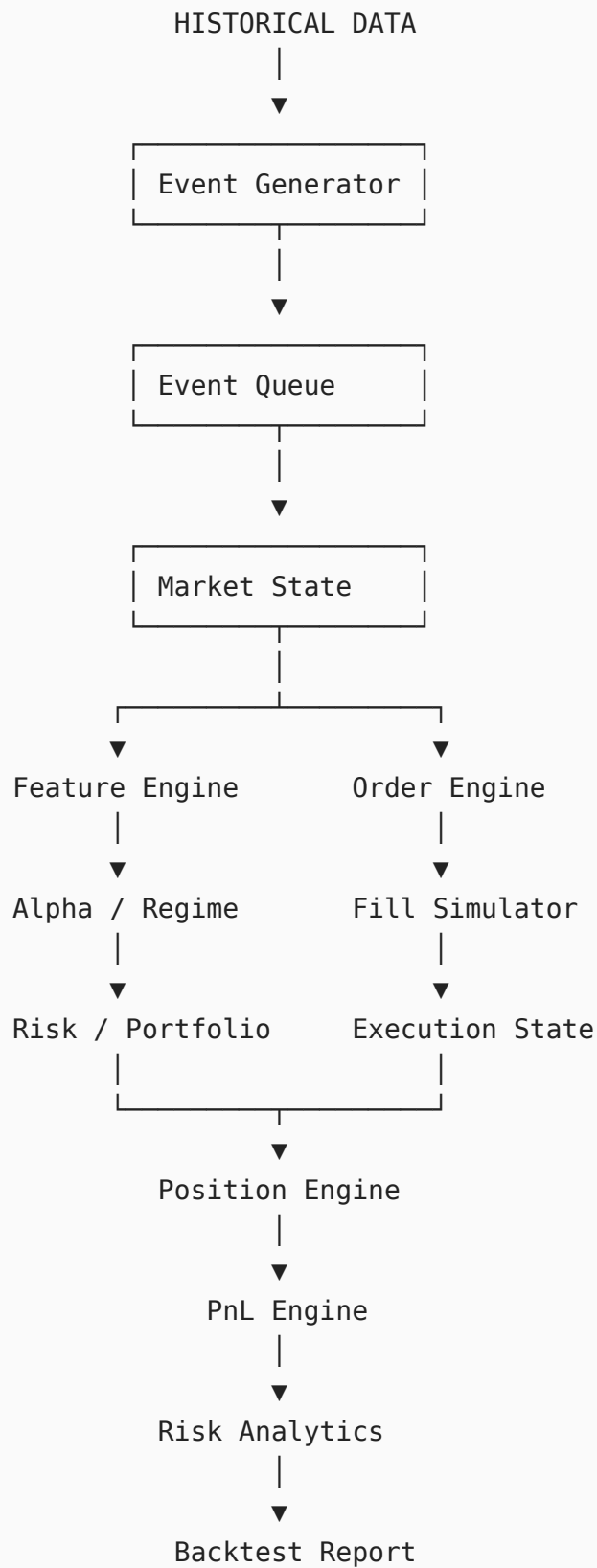
$$t,$$

$$P$$

Jadi benchmark menjadi multidimensional.

V.61. Architecture final Tahap V

Secara keseluruhan:



V.62. Output dari engine

Setiap event sebaiknya menghasilkan log:

$$t = tt, tt, t, , , t, P,$$

Dengan ini kita dapat melakukan **full forensic analysis** terhadap setiap trade.

V.63. Trade journal otomatis

Untuk setiap trade:

$$= \{t, t, t, P, \dots, t\}$$

Dua metrik yang sangat penting:

MAE

Maximum Adverse Excursion:

$$= \max_{t, t} t$$

MFE

Maximum Favorable Excursion:

$$= \max_{t, t} t$$

V.64. Mengapa MAE/MFE penting?

Misalnya kita menemukan:

sebagian besar trade hanya:

$$-0,$$

tetapi SL kita:

$$-1$$

Mungkin SL terlalu longgar.

Sebaliknya jika:

$$-0$$

sebelum trade yang akhirnya menang,

SL terlalu ketat dapat membunuh strategy.

Jadi:

$$\boxed{P}$$

V.65. Feedback loop

Sekarang sistem kita menjadi benar-benar adaptif:

$tt \ t \ t \ tt$

Tetapi hati-hati:

t

tidak boleh masuk ke feedback loop.

Kalau masuk:

$t \ tt$

maka:

t

sudah terkontaminasi.

V.66. Tiga dataset

Saya ingin kita menetapkan tiga wilayah:

☐

untuk membangun model.

☐

untuk memilih model/parameter.

☐

untuk evaluasi final.

Kemudian setelah test:

PP

dan baru:

☐

V.67. Dan ada satu lapisan lagi: Shadow Mode

Sebelum live execution:

$t \ tt$

tetapi:

t

Kita hanya mencatat:

$P,$

$P,$

Kemudian dibandingkan dengan market sebenarnya.

Ini disebut:

□

V.68. Hierarki deployment kita

Saya sarankan:

$tt \quad tt \quad t \quad P$

Bukan:

tt

V.69. Apa yang sebenarnya kita bangun?

Pada titik ini, proyek kita sudah jauh melampaui:

"bot trading."

Kita sedang membangun:

□

dengan struktur:

$ttttttPt\tttttttttPt$

V.70. Tahap V dalam satu persamaan

Kalau seluruh Tahap V harus diringkas dalam satu persamaan, saya akan menuliskannya sebagai:

$t1 = t, t1, t$

dengan constraint:

$t \in t$

dan PnL:

$$t_1 = t \quad P_{t, t_1, t - t}$$

Karena:

$$t \in t,$$

maka sistem bersifat **causal**.

Dan karena:

$$t_1$$

diproses satu per satu, kita memperoleh:

$$\square$$

V.71. Hubungan Tahap A $\rightarrow$ V

Sekarang kita dapat melihat mengapa tahap-tahap sebelumnya sebenarnya saling mengunci:

$$\square$$

bukan kumpulan modul terpisah.

Secara konseptual:

$$\square$$

Tahap V adalah **reality check** tersebut.

V.72. Hal yang harus kita implementasikan setelah ini

Sebelum masuk ke tahap berikutnya, saya justru menyarankan kita **tidak langsung melanjutkan menambah teori**.

Kita sudah mempunyai cukup banyak teori untuk mulai membangun engine pertama.

Urutan implementasi yang paling masuk akal:

V.1 — Data schema

Bangun format:

```
MarketEvent
TradeEvent
OrderBookEvent
OrderEvent
```

V.2 — Event queue

Implementasikan:

Pt

berdasarkan timestamp.

V.3 — Market state

Implementasikan:

t

V.4 — Order state machine

Implementasikan:

V.5 — Execution simulator

Mulai dari:

t

kemudian:

t

kemudian:

Pt

V.6 — Position & PnL

Implementasikan accounting yang benar.

V.7 — SL/TP/OCO

Kemudian:

PP

V.8 — Transaction cost

Masukkan:

t

V.9 — Walk-forward engine

Baru setelah simulator benar:

$t \ t$

V.10 — Monte Carlo & stress

Dan terakhir:

t

V.73. Satu keputusan desain yang sangat penting

Untuk proyek kita, saya menyarankan **jangan membuat backtest engine yang hanya menerima dataframe OHLC lalu menghasilkan equity curve.**

Kita sebaiknya membangun:

$\square$

yang menerima event:

$1, 2, \dots$

dan menghasilkan:

$1, 2, \dots$

Dengan begitu, engine yang sama nantinya dapat digunakan untuk:

$\boxed{ttt}$

dan kemudian, dengan interface yang sama:

$\boxed{P}$

dan bahkan:

$\square$

Artinya kita tidak perlu memiliki tiga sistem berbeda. Kita memiliki **satu trading engine**, hanya sumber event-nya yang berubah:

$\boxed{t \ P \ t \ t \ t}$

Menurut saya, ini adalah titik arsitektural yang sangat penting untuk proyek kita: **backtest bukan versi mainan dari live trading; backtest adalah live engine yang diberi historical event stream.**

Setelah fondasi ini selesai, **Tahap W** secara alami dapat kita gunakan untuk membangun **Performance Attribution, Risk Analytics, dan Robustness Framework**—yaitu menjawab *mengapa* sistem menghasilkan profit/loss, bukan hanya *berapa* profit/loss-nya.

Prompt:

8/17/2026, 10:10:02 PM

lanjut Tahap W

Response:

8/17/2026, 10:10:03 PM

Tahap W — Performance Attribution, Risk Analytics, dan Robustness Framework

Pada Tahap V kita sudah membangun fondasi bahwa sistem harus bereaksi terhadap **event pasar secara kausal dan realistis**.

Sekarang kita masuk ke pertanyaan yang lebih penting:

Jika strategi menghasilkan profit, sebenarnya profit itu datang dari mana? Dan jika gagal, apa yang menyebabkan kegagalannya?

Ini adalah perbedaan antara:

dan



Kita akan membangun tiga lapisan:



W.1. Jangan hanya melihat total return

Misalkan backtest menghasilkan:

$$tt = 5$$

Angka ini hampir tidak memberi tahu kita apa-apa.

Kita perlu mengetahui:

- berapa berasal dari alpha?
- berapa berasal dari market beta?
- berapa berasal dari BTC/ETH?
- berapa berasal dari gold?
- berapa berasal dari USD?
- berapa berasal dari leverage?
- berapa hilang karena fee?
- berapa hilang karena slippage?
- berapa berasal dari satu trade ekstrem?

Maka kita membutuhkan:

$$\begin{bmatrix} t & t \end{bmatrix}$$

W.2. Return decomposition

Untuk portfolio:

$$t$$

dan return:

$$t = t - t_{-1}$$

Kita ingin menulis:

$$t = \alpha + \beta_1 t_{-1} + \beta_2 t_{-2} + \dots + \beta_k t_{-k} + \epsilon_t$$

Ini bukan identitas tunggal yang selalu benar; ini adalah **model attribution** yang kita estimasikan.

W.3. Factor model

Kita dapat menggunakan model:

$$r_{i,t} = \alpha_i + \beta_{i1} r_{1,t} + \beta_{i2} r_{2,t} + \dots + \beta_{in} r_{n,t} + \epsilon_{it}$$

Untuk saham internasional kita dapat mengganti/menambah:

$$P, \dots, 10,$$

dan factor lain.

W.4. Mengapa ini sangat penting untuk sistem kita?

Bayangkan strategy menghasilkan:

$$= 2$$

Tetapi:

$$= 1$$

Artinya strategy mungkin sebenarnya hanya:

long BTC dengan cara yang sangat kompleks.

Jadi kita ingin alpha yang benar-benar residual:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

W.5. Factor exposure

Kita definisikan vector factor:

$$F_t = P$$

Portfolio exposure:

$$w_t$$

Maka:

$$r_{p,t} = w_t' F_t$$

W.6. Rolling factor exposure

Beta tidak boleh dianggap konstan.

Gunakan:

$$\beta_t = \frac{1}{n} \sum_{i=t-n+1}^t r_{i,t} r_{i,t-1}^{-1}$$

Dengan window:

Jadi kita memperoleh:

$$\beta_t$$

yang berubah terhadap regime pasar.

W.7. Ini menghubungkan Tahap A–V

Kita sebelumnya ingin:

"jika trading altcoin, lihat BTC dan ETH."

Sekarang kita bisa mengukurnya secara matematis.

Misalnya:

$$w_{BTC} = 0.2,$$

$$w_{ETH} = 0.1$$

Maka perubahan altcoin dapat didekomposisi menjadi kontribusi:

$$0.2 \cdot 0.1$$

W.8. Market-neutral alpha

Jika tujuan kita mencari alpha murni, kita dapat membuat portfolio:

dengan constraint:

$$w_{BTC} = 0$$

Contoh:

$$w_{BTC} = 0$$

Maka exposure terhadap BTC dihilangkan.

Untuk sistem multi-factor:

$$w_{BTC} = 0$$

dengan:

$$w_{BTC} = 0, w_{ETH} = 0, w_{LTC} = 0, w_{DOGE} = 0$$

W.9. Factor-neutral optimization

Kita dapat memasukkan constraint ini ke optimasi portfolio:

$$12 -$$

subject to:

$$w_{BTC} = 1,$$

$$w_{ETH} = 0,$$

Ini sangat relevan dengan seluruh pekerjaan optimasi kita sebelumnya.

W.10. Quantum formulation

Constraint factor-neutral tersebut dapat dimasukkan ke QUBO.

Misalnya objective:

$$= -$$

Factor neutrality:

$$= 0$$

Tambahkan penalty:

$$= 2$$

Kemudian:

Jadi optimasi quantum kita tidak berdiri sendiri.

Ia menjadi bagian dari **factor-aware trading system**.

W.11. Performance attribution per trade

Untuk trade :

$$P = P - - - t$$

Dengan:

$$P = P_t - P_t$$

Jadi:

$$P = P - tt$$

W.12. Transaction cost attribution

Total cost:

$$t = ,t ,t ,t ,t ,t$$

Maka:

$$t = -$$

Kita dapat mengetahui apakah strategy sebenarnya profitable **sebelum biaya tetapi tidak setelah biaya**.

W.13. Contoh yang sangat penting

Misalkan:

$$P = 20$$

Tetapi:

$$= -4,$$

$$= -,$$

$$= -$$

Maka:

$$tP =$$

Strateginya bukan strategy dengan:

$$20$$

profit.

Strateginya adalah:

$$\boxed{t}$$

W.14. Turnover

Define turnover:

$$_t = \Delta_{,t}2$$

Total:

$$tt = {}_t^t$$

Turnover sangat penting untuk short-term trading.

Strategy dengan:

$$= 2$$

dan turnover:

$$5000$$

sangat berbeda dari strategy:

$$= 2$$

dengan turnover:

W.15. Capacity

Profitability juga bergantung pada ukuran modal.

Jika order:

terlalu besar dibanding liquidity:

,

market impact meningkat.

W.16. Market impact

Model sederhana:

$$t =$$

Dengan:

$$0$$

Kemudian:

$$P = P t$$

W.17. Capacity curve

Kita dapat menguji:

$$= 1, 10, 100, 1, 10,$$

dan menghitung:

Kemudian:

$$t =$$

Ini sangat relevan dengan paradigma hedge fund/prop firm.

W.18. Risk analytics

Sekarang kita masuk ke risk.

Metric pertama:

Define running maximum:

$$t = \tau t^\tau$$

Drawdown:

$$t = t - tt$$

W.19. Maximum drawdown

$$= t - t$$

Tetapi MDD saja belum cukup.

Kita juga perlu:

$$t$$

W.20. Drawdown duration

Jika portfolio turun pada:

$$t_0$$

dan kembali ke previous high pada:

$$t_1,$$

maka:

$$= t_1 - t_0$$

Strategy dengan:

$$= 10$$

selama 2 hari berbeda dengan:

$$= 10$$

selama 8 bulan.

W.21. Sharpe ratio

$$\frac{1}{n} \sum_{t=1}^n (r_t - \bar{r})^2$$

Tetapi untuk intraday/short-term trading kita harus hati-hati terhadap:

- autocorrelation,
- non-normality,
- changing volatility.

W.22. Sortino ratio

Gunakan downside deviation:

$$= \sqrt{\frac{1}{n} \sum_{t=1}^n \max(0, -r_t)^2}$$

Kemudian:

$$t = \frac{\bar{r} - r_f}{\text{downside deviation}}$$

W.23. Calmar ratio

$$= \frac{\bar{r} - r_f}{\text{max drawdown}}$$

Sangat berguna untuk melihat return relatif terhadap drawdown ekstrem.

W.24. VaR

Value-at-Risk:

adalah loss threshold sehingga:

$$P(L > VaR) = 1 - P$$

Misalnya:

$$= 1000$$

Interpretasi:

Dalam model distribusi tersebut, kerugian harian diperkirakan melebihi \$1000 hanya sekitar 1% dari hari.

Tetapi VaR tidak mengatakan **seberapa besar loss setelah threshold tersebut terlampaui**.

W.25. CVaR

Conditional VaR:

$$\boxed{=}$$

Ini jauh lebih berguna untuk tail risk.

W.26. Expected Shortfall

CVaR juga sering disebut:

$$\boxed{tt}$$

Jika:

$$= 10,$$

tetapi:

$$= 5,$$

maka tail loss sebenarnya jauh lebih buruk daripada sekadar \$10.

W.27. Tail dependence

Ini sangat penting karena kita memiliki factor seperti:

$$, , ,$$

Korelasi Pearson:

$$,$$

tidak cukup.

Dua asset dapat memiliki:

$$0$$

tetapi crash bersama.

W.28. Copula intuition

Kita dapat memodelkan joint distribution:

$$, = ,$$

Dengan copula:

Ini memungkinkan kita mempelajari:

$$P$$

W.29. Tail correlation

Definisikan lower-tail dependence:

$$= {}_0P_{-1 \quad -1}$$

Ini jauh lebih relevan untuk stress scenario.

W.30. Regime-conditioned risk

Kita sebelumnya memiliki:

$$t$$

Misalnya:

$$t \in \{, , , \}$$

Maka risk:

$$t$$

dapat dihitung secara conditional.

W.31. Conditional Sharpe

$$= \quad = \quad =$$

Jadi kita bisa menemukan:

| Strategy bagus di bull market tetapi buruk di high-volatility regime.

Ini jauh lebih berguna daripada satu Sharpe global.

W.32. Risk decomposition

Portfolio variance:

$$2 =$$

Marginal contribution to risk:

$$=$$

Karena:

$$= ,$$

maka:

$$=$$

W.33. Component contribution

Risk contribution:

$$=$$

Dan:

$$=$$

Ini memungkinkan kita menjawab:

Asset mana yang sebenarnya menyebabkan portfolio risk?

W.34. Risk budget

Kita dapat menetapkan:

Misalnya:

$$20$$

Jadi bukan hanya:

$$20$$

Tetapi:

$$20$$

Ini jauh lebih sophisticated.

W.35. Dynamic risk budget

Karena volatility berubah:

$$= t$$

Contohnya saat:

$$= ,$$

risk budget turun:

W.36. Position sizing

Sekarang kita dapat menghubungkan risk ke position.

Misalkan target dollar risk:

$$= 1$$

Stop distance:

Maka quantity:

$$\boxed{=}$$

Ini adalah fondasi risk sizing yang sudah kita gunakan sebelumnya.

Tetapi sekarang kita dapat memperbaikinya.

W.37. Volatility-adjusted risk

Jika:

$$= t,$$

maka:

$$t = t$$

Sehingga:

$$\boxed{t \quad 1_t}$$

Volatility tinggi:

$$t$$

Volatility rendah:

$$t$$

W.38. Tetapi volatility saja belum cukup

Kita juga mempertimbangkan liquidity:

$$t$$

Misalnya:

$$t = t, t$$

Dengan:

sebagai maximum liquidity participation.

W.39. Robustness

Sekarang pertanyaan terpenting:

Apakah strategy benar-benar bekerja, atau parameter kita hanya kebetulan cocok dengan historical data?

Kita membutuhkan:

$$t$$

W.40. Parameter perturbation

Misalkan:

adalah parameter optimal.

Jangan hanya menguji:

Uji:

Misalnya:

$$\in -20, 20$$

W.41. Robust strategy

Jika:

$$P$$

tetap bagus pada region luas:

$$\in ,$$

maka strategy lebih robust.

Secara intuitif:

$$\square$$

W.42. Sharp peak = warning

Misalnya:

Sharpe

```
3.0      *
2.5      ***
2.0      *****
1.5      *****
1.0      *****
      0.8 1.0 1.2
      parameter
```

Jika hanya:

$$= 10$$

yang menghasilkan Sharpe tinggi, kemungkinan besar:

$$tt$$

W.43. Perturbasi transaction cost

Jika:

$$= 1,$$

kita uji:

$$\in \{0.5, 1, 2, , 5\}$$

Strategy yang hanya profitable pada:

$$= 0.5$$

tidak cukup robust.

W.44. Perturbasi latency

Uji:

$$\in \{1, 5, 10, 50, 100, 500\}$$

Kemudian:

Untuk short-term strategy, ini bisa menjadi faktor pembunuh.

W.45. Perturbasi slippage

Misalnya:

$$= t$$

Dengan:

$$\in 0.5, 5$$

Jika:

$$= 2$$

sudah menghancurkan strategy, kita tahu strategy terlalu execution-sensitive.

W.46. Randomized execution

Untuk setiap trade:

$$P = P$$

Dengan:

Kemudian:

$$= 10$$

atau:

$$10^4$$

simulation.

W.47. Probabilitas profitability

Kita dapat menghitung:

$$P_0 = \{0\}$$

Misalnya:

$$P_0 = 01$$

Ini jauh lebih informatif daripada satu backtest.

W.48. Confidence interval

Jika mean return:

dan standard error:

$$= ,$$

maka:

$$_5 = 1$$

Tetapi untuk financial returns yang dependent/non-normal, bootstrap lebih aman daripada asumsi IID sederhana.

W.49. Block bootstrap

Karena return memiliki autocorrelation:

$$t \neq 1,$$

kita dapat menggunakan block bootstrap.

Bagi data menjadi block:

$$1, 2, \dots$$

Kemudian sampling block, bukan individual observation.

Ini mempertahankan sebagian struktur temporal.

W.50. Deflated Sharpe Ratio

Masalah lain:

Jika kita mencoba:

$$= 10,000$$

strategies, beberapa pasti terlihat sangat bagus secara kebetulan.

Maka Sharpe tinggi saja tidak cukup.

Kita harus memperhitungkan:

$$\boxed{tt}$$

W.51. Probability of Backtest Overfitting

Secara konsep:

$$P = Pt$$

Semakin tinggi:

$$P,$$

semakin besar kemungkinan strategy kita merupakan hasil overfitting.

W.52. Data snooping

Jika kita mencoba:

- 100 factor,
- 50 parameter,
- 20 SL,
- 20 TP,
- 10 regime model,

jumlah kombinasi menjadi sangat besar.

Secara kasar:

$$100 \cdot 50 \cdot 20 \cdot 20 \cdot 10 = 20,000,000$$

kombinasi.

Tidak mengherankan jika satu kombinasi terlihat luar biasa.

W.53. Ini sangat relevan untuk proyek quantum

Karena kita dapat membandingkan:

dengan:

dan:

Tetapi jika kita mencoba banyak:

, , t , t , t , tt , t , t ,

maka kita juga melakukan multiple testing.

W.54. Quantum robustness matrix

Saya menyarankan kita nantinya mempunyai matrix:

Model	Optimizer	Depth	Initialization	Sharpe	MDD	CVaR
Classical	SLSQP	—	EPG			
Classical	BFGS	—	Random			
VQE	SPSA	2	EPG			
VQE	SPSA	4	EPG			
QAOA	COBYLA	2	EPG			
QAOA	SPSA	4	EPG			

Jadi kita tidak hanya menunjukkan:

| "QAOA menang."

Tetapi:

| "Dalam region parameter X, QAOA menunjukkan performa yang stabil."

W.55. Regime robustness

Kita juga melakukan:

$$P =$$

Misalnya:

Regime	Return	Sharpe	MDD
Bull			
Bear			
Sideways			
High Vol			
Crash			

W.56. Cross-market robustness

Ini sangat sesuai dengan tujuan awalmu.

Kita tidak ingin:

$$tt_t$$

saja.

Kita ingin:

$$tt$$

yang dapat diterapkan pada:

$$\boxed{t}$$

$$\boxed{t}$$

$$\boxed{t}$$

$$\square$$

$$\boxed{t}$$

dan sebagainya.

W.57. Factor universality

Tetapi jangan berharap parameter yang sama bekerja identik.

Yang kita cari adalah struktur:

$$\boxed{tt}$$

yang universal.

Sedangkan:

$$P_t$$

dapat asset-specific.

Jadi:

$$t \quad t$$

tidak masalah.

W.58. Market-specific execution model

Crypto:

$$24$$

Equity:

$$t$$

FX:

$$245$$

Futures:

Execution engine harus memahami perbedaan tersebut.

W.59. Gold dan USD dominance

Karena kita ingin menggunakan:

dan:

sebagai market factor, kita dapat mendefinisikan factor vector:

$$t = ,t,t,t,tP,t,t$$

Kemudian:

$t, t = t \neq t$

W.60. Bahkan factor exposure dapat menjadi signal

Misalnya:

$t, 0$

Kemudian:

Model mungkin memperkirakan:

$t \ 0$

Tetapi kita tidak langsung trading.

Kita gabungkan:

t

dengan:

dan:

W.61. Meta-alpha

Kita dapat membangun:

$t = t, t, , , tt,$

Kemudian model menghasilkan:

t

W.62. Signal-to-noise ratio

Signal:

t

Noise:

t

Maka:

$$t = tt$$

Jika:

$$t$$

rendah, strategy dapat memilih:

$$t \ 0$$

W.63. Abstention

Ini adalah fitur penting yang sering dilupakan.

Strategy tidak harus selalu trading.

Action space kita:

$$= \{, t, t\}$$

Tetapi kita dapat memperluasnya menjadi:

$$= \{, t, t, t\}$$

Secara praktis:

$$t = t$$

tetapi memiliki alasan:

$$t$$

W.64. Expected utility

Daripada hanya memaksimalkan expected return:

$$,$$

kita dapat menggunakan:

$$- - t$$

Misalnya:

$$t = t - 2t - t$$

W.65. Dynamic decision threshold

Trade hanya jika:

$$t \ 0$$

Atau:

$$tt$$

Ini secara otomatis mengurangi trading ketika edge kecil.

W.66. Hubungan dengan risk \$1

Sistem awal kita memiliki constraint:

$$1$$

Sekarang kita bisa menggeneralisasikannya:

$$\boxed{= \ t \ t,t}$$

Dengan:

$$t,t = tt, , t, t$$

W.67. Dynamic risk budget

Misalnya equity:

$$t$$

Risk fraction:

$$t$$

Maka:

$$t,t = tt$$

Jika drawdown meningkat:

$$t \ t$$

Jadi sistem otomatis **de-risk**.

W.68. Drawdown control

Misalnya:

$$t = 0 \ 1 - t$$

Ketika:

$$t \ ,$$

maka:

$$t \ 0$$

Ini dapat menjadi salah satu circuit breaker.

W.69. Kill switch

Kita juga membutuhkan hard constraints:

$$\boxed{t \quad tt}$$

Selain itu:

$$t \ t$$

Dan:

$$t \ t$$

W.70. Risk hierarchy

Akhirnya risk system kita menjadi:

$$\boxed{PtPt\text{t}t\text{t}}$$

W.71. System risk

Ini bahkan tidak berasal dari market.

Contohnya:

- exchange disconnect,
- stale data,
- clock drift,
- duplicated order,
- API error,
- corrupted state,
- model failure.

Maka:

$$\boxed{t = t \quad t \ t}$$

W.72. Model risk

Misalnya alpha model menghasilkan:

$$t$$

Tetapi prediction error:

$$t = t - t$$

Kita dapat memonitor:

$$t,$$

$$t,$$

$$t$$

Jika model mulai mengalami drift:

$$P_t$$

Maka sistem dapat mengurangi exposure.

W.73. Model decay

Define rolling performance:

$$t$$

Jika:

$$t \quad t$$

selama:

periode, maka:

$$tt =$$

Kemudian:

$$Pt$$

W.74. Jadi model kita sekarang mempunyai feedback control

Ini menarik secara matematis.

Sistem dapat dilihat sebagai:

$$\boxed{t \quad t \quad P \quad t}$$

Risk menjadi feedback controller.

W.75. State-space perspective

Kita bahkan dapat menulis:

$$t_1 = t, t, t$$

dan:

$$t = t, t$$

Dengan:

- t : latent market state,
- t : trading action,
- t : market noise,
- t : observed market data,
- t : observation noise.

Ini membawa kita sangat dekat ke:

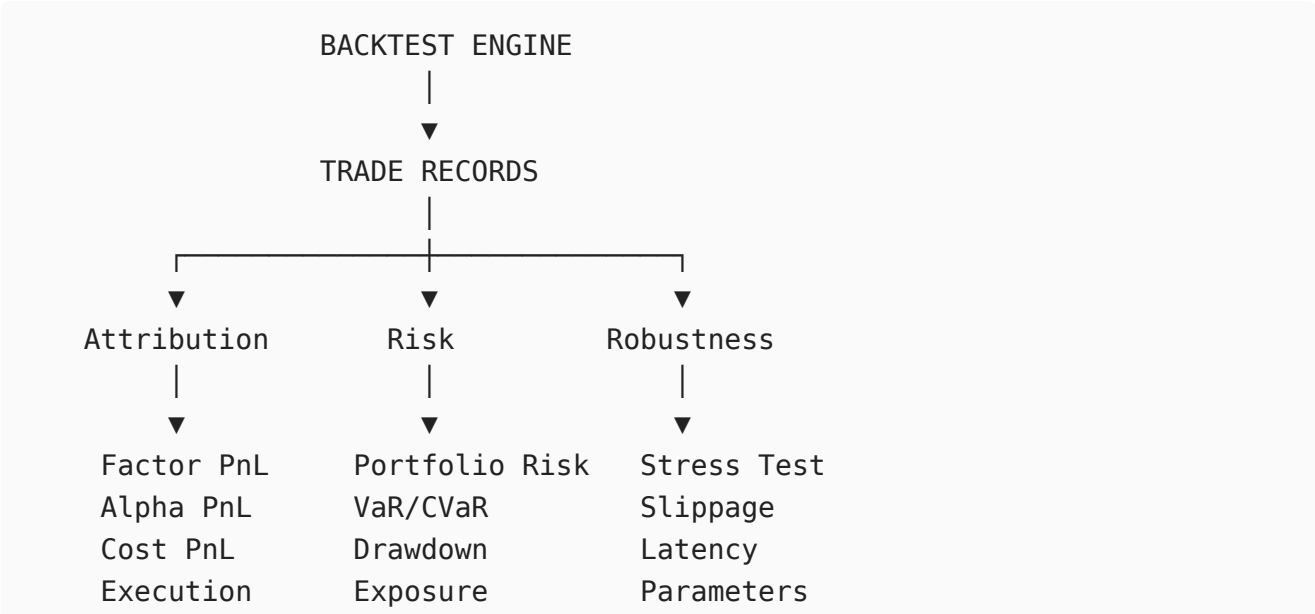
$$tt$$

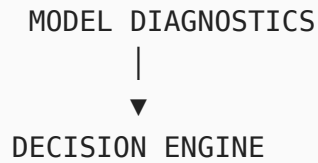
dan:

$$tttt$$

W.76. Arsitektur Tahap W

Sekarang sistem kita:





W.77. Output akhir Tahap W

Untuk setiap backtest kita tidak lagi menghasilkan satu file:

```
backtest_result.csv
```

tetapi sebuah **research report**:

```
run/
├─ config.yaml
├─ market_data/
├─ signals/
├─ orders/
├─ fills/
├─ positions/
├─ trades/
├─ pnl/
├─ attribution/
├─ risk/
├─ robustness/
├─ diagnostics/
└─ report/
```

W.78. config.yaml

Semua parameter eksperimen harus disimpan:

```
strategy:
  name: factor_quant

risk:
  risk_per_trade: 1.0
  max_drawdown: 0.15
```

```
execution:
  fee: ...
  slippage_model: ...
  latency_ms: ...

portfolio:
  max_leverage: ...

optimization:
  method: QA0A
  depth: 4
  optimizer: SPSA
```

Tujuannya agar:

dapat direproduksi.

W.79. Experiment ID

Setiap eksperimen:

$$t = t \ tt$$

Dengan begitu kita dapat mengetahui:

"Angka Sharpe 1.83 ini berasal dari konfigurasi yang mana?"

Ini adalah praktik yang sangat penting dalam quantitative research.

W.80. Performance report

Minimal:

$$tttttPttttt$$

W.81. Tetapi kita tambahkan sesuatu yang lebih penting

Attribution report

$$tP = \ t \ t \ t$$

Risk report

$$t = t \ t \ t \ t$$

Robustness report

$$P = P t, , t, t, , t$$

W.82. Dan inilah tujuan sebenarnya

Pada akhir pipeline, kita tidak ingin sistem mengatakan:

BUY BTC

atau:

BUY BBKA

saja.

Kita ingin sistem dapat menjelaskan:

Model mendeteksi positive expected return sebesar , dengan confidence , setelah mengontrol exposure terhadap BTC/ETH/Gold/DXY. Optimizer memilih posisi karena trade tersebut meningkatkan expected utility setelah transaction cost. Risiko maksimum diperkirakan , sementara portfolio factor exposure tetap berada dalam batas yang ditentukan. Order kemudian dieksekusi dengan expected slippage .

Itulah perbedaan antara **trading bot** dan **quantitative decision system**.

W.83. Dan ada konsekuensi besar terhadap proyek quantum kita

Sekarang kita memiliki tempat yang jelas bagi:

dan:

Mereka **bukan trading strategy secara keseluruhan**.

Mereka adalah:

$$tt$$

di dalam:

$$t$$

Strukturnya:

$$t \quad t \quad Pt \quad t \quad P$$

Dengan demikian penelitianmu tentang **Markowitz + VQE** dapat menjadi salah satu komponen dari sistem yang jauh lebih besar tanpa kehilangan identitas ilmiahnya.

W.84. Tahap W dalam satu formulasi matematis

Kita akhirnya dapat mendefinisikan objective sistem secara jauh lebih lengkap:

$$\left| \begin{matrix} 1 & 2t & -4 \end{matrix} \right|$$

subject to:

$$= 0$$

atau exposure constraints,

$$\begin{array}{c} 1 \\ , \\ , \\ t, \\ t \end{array}$$

dan:

$t \in t$

Constraint terakhir sangat penting:

strategi tidak boleh mengetahui masa depan.

W.85. Posisi kita sekarang

Dengan Tahap A sampai W, arsitektur konseptual kita telah berubah dari:

$$\boxed{t}$$

menjadi:

999

dengan:

Dan menurut saya, **setelah Tahap W kita sudah memiliki cukup banyak blok matematis sehingga tahap berikutnya sebaiknya mulai mengikat semuanya menjadi spesifikasi sistem yang benar-benar implementable**, bukan terus menambah teori secara terpisah.

Tahap berikutnya paling natural adalah **Tahap X — Research Protocol & Experimental Design**: bagaimana kita membuat eksperimen yang adil untuk membandingkan **classical optimization vs VQE vs QAOA**, termasuk dataset split, baseline, ablation study, statistical significance, multiple-testing control, dan desain eksperimen yang nantinya bisa langsung menjadi metodologi penelitian/paper.

Prompt:

8/17/2026, 10:10:58 PM

lanjut Tahap X

Response:

8/17/2026, 10:10:59 PM

Tahap X — Research Protocol & Experimental Design

Tahap X adalah titik di mana seluruh arsitektur yang kita bangun mulai berubah menjadi **protokol penelitian yang dapat diuji secara ilmiah**.

Sebelumnya kita telah membangun:

Sekarang pertanyaannya bukan lagi:

"Bagaimana membuat strategy yang profitable?"

tetapi:

"Bagaimana membuktikan secara objektif bahwa strategy tersebut bekerja, dan apakah optimizer quantum benar-benar memberikan sesuatu dibandingkan baseline classical?"

Ini sangat penting karena kalau kita tidak merancang eksperimennya dengan benar, kita bisa mendapatkan hasil seperti:

$$= 24$$

dan

$$= 1$$

tetapi sebenarnya QAOA hanya menang karena:

- mendapat data lebih banyak,
- mendapat parameter lebih banyak,
- menggunakan optimizer yang lebih baik,
- mendapatkan initialization berbeda,
- atau secara tidak sengaja melihat informasi masa depan.

Jadi Tahap X akan membangun **scientific protocol**.

X.1. Pertanyaan penelitian utama

Saya menyarankan kita mendefinisikan pertanyaan penelitian secara hierarkis.

RQ1 — Apakah trading framework memiliki predictive/optimization edge?

$$0 \quad tt$$

versus:

$$1 \quad tt$$

RQ2 — Apakah optimasi portfolio memberikan improvement?

Bandingkan:

dengan:

$$t$$

RQ3 — Apakah VQE memberikan improvement dibanding classical optimization?

$$0 \quad P \quad P$$

versus:

$$1 \quad P \quad P$$

RQ4 — Apakah QAOA memberikan improvement?

$$0 \quad P \quad P$$

RQ5 — Apakah improvement tersebut robust?

Kita ingin:

$$P$$

tetap baik terhadap:

- perubahan parameter,
- transaction cost,
- slippage,
- latency,
- market regime,
- asset universe.

X.2. Jangan menjadikan "profit" sebagai satu-satunya target

Kita membutuhkan objective vector:

$$= ttt$$

Karena bisa terjadi:

$$t \ t$$

tetapi:

Maka kita tidak dapat mengatakan A lebih baik secara sederhana.

X.3. Primary metric

Kita harus memilih **satu primary metric** sebelum eksperimen.

Saya menyarankan:

$$\boxed{P = tt}$$

dan secara konkret:

□

sebagai primary metric.

Sedangkan:

’, ’

menjadi secondary metrics.

Ini penting untuk mengurangi cherry-picking.

X.4. Dataset harus dipisahkan

Kita tidak boleh melakukan:

$t \ t \ t$

Kita membutuhkan:

$\boxed{t \ t}$

Misalnya:

02020

Tetapi untuk time series, random split **tidak boleh** digunakan.

X.5. Time-series split

Misalkan data:

201 202

Kita dapat membuat:

2018 — 2022

Train

2023 —

Validation

2024 —

Test

Tetapi untuk short-term trading kita membutuhkan sesuatu yang lebih baik.

X.6. Walk-forward validation

Gunakan:

$$1 \quad t_1$$

kemudian:

$$2 \quad t_2$$

dan seterusnya.

Contoh:

```
Window 1
Train: 2019–2021
Test : 2022 Q1

Window 2
Train: 2019–2022 Q1
Test : 2022 Q2

Window 3
Train: 2019–2022 Q2
Test : 2022 Q3
```

Sehingga model selalu hanya menggunakan informasi masa lalu.

X.7. Expanding vs rolling window

Ada dua pilihan.

Expanding

$$t = t_0, t$$

Semakin lama dataset semakin besar.

Rolling

$$t = t - , t$$

Window memiliki ukuran tetap.

Untuk financial market, rolling window sering lebih masuk akal karena:

$$P_t$$

dapat mengalami non-stationarity.

X.8. Purged validation

Karena trade dapat memiliki horizon:

1,

data training dan validation bisa overlap secara informasi.

Misalnya:

```
Train:
Jan 1  ————— Jan 10

Validation:
Jan 10 ————— Jan 20
```

Jika posisi Jan 10 masih terbuka, terdapat leakage.

Maka kita membutuhkan:

P

X.9. Embargo

Tambahkan gap:

antara training dan validation.

Misalnya:

```
TRAIN
_____

EMBARGO
_____

VALIDATION
_____
```

Ini mengurangi kemungkinan leakage temporal.

X.10. Data hierarchy

Dataset kita tidak hanya harga.

Kita membutuhkan:

$$t = \{P_{t,t}, t, t, t, t\}$$

dengan:

- P_t : price,
 - t : volume,
 - t : order book,
 - t : factor,
 - t : market state,
 - t : execution information.
-

X.11. Factor dataset

Untuk crypto:

$$t = \{, , t, t, , , , \}$$

Untuk equities:

$$t = \{P, , , 10, , t\}$$

Jadi architecture-nya tetap sama.

X.12. Asset universe

Kita harus menentukan universe sebelum melihat hasil.

Misalnya:

$$= \{1, , \}$$

Jangan memilih asset karena kita sudah tahu asset tersebut profitable.

Itu adalah:

□

X.13. Survivorship bias

Misalkan tahun 2020 terdapat:

1000

asset.

Tahun 2026 hanya:

500

yang masih eksis.

Jika kita hanya mengambil 500 asset tersebut, kita secara implisit membuang:

500

asset yang gagal.

Strategy terlihat lebih bagus daripada kondisi sebenarnya.

X.14. Universe harus point-in-time

Definisikan:

t

Asset yang tersedia pada waktu t hanya:

$\in t$

X.15. Corporate actions

Untuk equities kita juga harus memperhatikan:

- split,
- dividend,
- merger,
- delisting.

Price series harus menggunakan data yang benar secara point-in-time.

X.16. Data leakage taxonomy

Kita perlu secara eksplisit menguji:

$= \{, t, , , t\}$

X.17. Feature leakage

Misalnya kita membuat:

$$t$$

tetapi secara tidak sengaja menggunakan:

$$P_{t1}$$

Maka:

$$t$$

mengandung future information.

Ini dapat menghasilkan backtest yang luar biasa tetapi tidak dapat diperdagangkan.

X.18. Label leakage

Jika target:

$$t = t_1,$$

maka feature harus memenuhi:

$$t \in t$$

Secara formal:

$$\boxed{t}$$

X.19. Information filtration

Ini adalah fondasi matematis yang sangat bagus untuk sistem kita.

Definisikan:

$$t$$

sebagai seluruh informasi yang tersedia sampai waktu t .

Maka strategy harus memenuhi:

$$\boxed{t \in t}$$

Artinya keputusan pada t tidak boleh bergantung pada:

$$t\Delta$$

X.20. Backtest sebagai stochastic process

Portfolio:

$$t$$

Return:

$$t = t - t - 1$$

Strategy menghasilkan:

$$t = t$$

Maka:

$$t1 = t \quad t1 - t$$

Ini adalah formulasi yang jauh lebih bersih.

X.21. Baseline hierarchy

Kita tidak boleh langsung:

Kita membutuhkan hierarchy.

Baseline 0

Buy-and-hold.

Baseline 1

Equal weight.

Baseline 2

Momentum.

Baseline 3

Mean reversion.

Baseline 4

Markowitz classical.

Baseline 5

MILP / exact solver.

Baseline 6

Heuristic optimizer.

Baseline 7

VQE.

Baseline 8

QAOA.

X.22. Mengapa exact solver sangat penting?

Untuk masalah kecil:

$$20$$

kita dapat menghitung:

$$=$$

secara exact.

Misalnya:

$$t$$

Kemudian:

dibandingkan terhadap:

$$t$$

X.23. Approximation ratio

Definisikan:

$$= -$$

Atau sesuai orientasi minimization/maximization.

Dengan demikian kita dapat mengatakan:

| VQE memperoleh solusi dalam error dari optimum exact.

Ini jauh lebih ilmiah daripada sekadar:

| "VQE menghasilkan portfolio."

X.24. Optimization quality vs trading quality

Ini adalah dua hal berbeda.

Optimization quality

t

VS:

Trading quality

P

atau:

Bisa terjadi:

tetapi:

$P \ P$

Mengapa?

Karena market berubah.

X.25. Ini sangat penting untuk thesis-mu

Jika kita ingin mengklaim VQE bagus, jangan hanya mengukur:

Kita perlu:

$\boxed{tt \ PtP}$

X.26. QUBO benchmark

Misalnya:

$= -$

Kita generate:

$= 10, 20, 0,$

asset.

Kemudian:

t

vs:

vs:

vs:

X.27. Controlled experiment

Semua optimizer harus menerima:

□

Jangan:

X.28. Controlled initialization

Jika kita ingin menguji EPG warm-start:

P_0

Bandingkan:

vs:

P

Dengan:

- same data,
- same QUBO,
- same depth,
- same optimizer,
- same stopping criteria.

Hanya initialization yang berbeda.

Ini disebut:

X.29. Ablation study

Kita dapat membuat:

Component	Model A	Model B
Factor	✓	✓
Regime	✓	✓
EPG	✗	✓
VQE	✓	✓
Dynamic risk	✓	✓

Kemudian:

$$\Delta P$$

menunjukkan contribution masing-masing component.

X.30. Full factorial experiment

Misalnya kita punya:

$$t \quad \{, , \}$$

dan:

$$tt \quad \{, P\}$$

maka:

$$2 =$$

configuration.

Kita tidak hanya membandingkan dua model.

Kita dapat mempelajari interaction:

$$t \quad tt$$

X.31. Random seeds

Quantum optimizer sangat stochastic.

Maka:

$$1, \dots$$

Misalnya:

$$= 0$$

Untuk setiap konfigurasi kita memperoleh:

$$1, \dots, 0$$

X.32. Jangan melaporkan hanya mean

Laporkan:

dan:

Lebih baik:

dan:

Karena distribution bisa heavy-tailed.

X.33. Distribution of outcomes

Misalnya:

$$= \{12, 14, 1, 02, 15, \dots\}$$

Jangan mengatakan:

$$= 1$$

Lebih tepat:

$$= 15$$

dengan:

$$= 11, 14$$

X.34. Statistical significance

Misalkan kita memiliki paired observations:

$$= t_1 - t_2$$

Hipotesis:

$$\mu_0 = 0$$

Kita dapat menggunakan:

$$\square$$

untuk paired non-parametric comparison.

X.35. Mengapa paired?

Karena kedua model diuji pada:

$$t$$

Misalnya:

$$20241$$

dibandingkan:

$$20241$$

Jadi kita dapat menghilangkan sebagian noise akibat perbedaan market period.

X.36. Effect size

Significance saja tidak cukup.

Misalnya:

$$= 0.001$$

tetapi improvement:

$$0.001$$

Secara praktis mungkin tidak penting.

Maka kita membutuhkan:

$$\square t$$

X.37. Bootstrap confidence interval

Untuk performance difference:

$$\Delta = t - t$$

Kita bootstrap:

$$\Delta_1, \Delta$$

Kemudian:

$$_5 = 0025, 05$$

Jika interval tidak mencakup:

$$0,$$

maka terdapat evidence bahwa difference bukan sekadar noise sampling.

X.38. Multiple testing correction

Jika kita melakukan:

uji statistik, probabilitas false positive meningkat.

Kita dapat menggunakan:

$$\boxed{-}$$

untuk mengontrol:

X.39. Hyperparameter optimization harus berada di training

Misalnya parameter:

$$= \{, , t, t, \}$$

Kita boleh mencari:

menggunakan:

$$t$$

Tetapi:

$$\boxed{t}$$

tidak boleh disentuh.

X.40. Final test harus sekali

Idealnya:

```
TRAIN
  ↓
VALIDATION
  ↓
MODEL SELECTION
  ↓
LOCK MODEL
  ↓
FINAL TEST
```

Setelah final test dibuka:

□

Jika kita retune setelah melihat test, maka test bukan lagi test.

X.41. Nested walk-forward

Untuk eksperimen yang lebih ketat:

```
Outer loop
└──────────┘
    Test window

Inner loop
└──────────┘
    Train → Validation
```

Outer loop mengukur generalization.

Inner loop memilih hyperparameter.

X.42. Performance uncertainty

Kita harus mengakui:

dan:

P

memiliki uncertainty.

Jadi output:

$$= 12$$

sebaiknya menjadi:

$= 125 \quad 11 - 22$

X.43. Stress testing

Kita membuat scenario:

Scenario 1

Volatility:

2

Scenario 2

Spread:

2

Scenario 3

Slippage:

Scenario 4

Liquidity:

05

Scenario 5

Latency:

5

X.44. Crash scenario

Kita dapat membuat:

$$t = -10$$

dan:

Kemudian melihat:

$$, t$$

X.45. Monte Carlo strategy stress test

Kita juga dapat resample:

$$t$$

Buat:

$$= 10,000$$

possible equity paths:

$$1t', t$$

Kemudian hitung:

$$P$$

X.46. Probability of ruin

Jika capital:

$$0,$$

dan ruin threshold:

$$,$$

maka:

$$\boxed{P_t^t}$$

Untuk sistem leveraged ini merupakan metric yang sangat penting.

X.47. Capacity experiment

Kita juga menguji:

$$t = \{1, 10, 100, 1, 10\}$$

Kemudian:

tt

Jika:

t ,

maka strategy memiliki capacity constraint.

X.48. Latency experiment

Untuk short-term trading:

$$t = \{1, 5, 10, 50, 100, 500\}$$

Kemudian:

P

Kita ingin mengetahui:

t

Jika:

t

maka strategy tidak layak untuk infrastructure yang lebih lambat.

X.49. Execution realism ladder

Kita dapat membangun empat level:

Level 0

Mid-price.

Level 1

Fee.

Level 2

Spread + slippage.

Level 3

Order-book simulation.

Level 4

Latency + queue position + partial fills.

Kemudian:

$$P_0 \ P_4$$

Jika strategy hanya profitable pada Level 0, kita tahu strategy tidak realistic.

X.50. Ini menjadi eksperimen yang sangat kuat

Misalnya:

Execution	Sharpe
Mid-price	2.31
+ Fee	2.05
+ Spread	1.72
+ Slippage	1.34
+ Order book	1.12
+ Latency	0.98

Kesimpulannya bukan:

| strategy buruk.

Tetapi:

| edge sangat sensitif terhadap execution.

Itu merupakan **scientific finding**.

X.51. Quantum advantage harus didefinisikan dengan hati-hati

Ini sangat penting.

Jangan langsung mengatakan:

tt

Itu terlalu kuat.

Kita harus membedakan:

Optimization advantage

$$t$$

Computational advantage

$$t$$

Trading advantage

$$P_t \ P$$

Practical quantum advantage

Keunggulan yang tetap bertahan setelah:

- quantum overhead,
- noise,
- compilation,
- data loading,
- repeated measurements,
- classical optimizer cost.

X.52. NISQ reality

Untuk VQE/QAOA:

$$tt = t \ t \ t \ t$$

Maka meskipun:

$$t$$

kecil, belum tentu:

$$tt$$

kecil.

X.53. Quantum resource metrics

Kita harus mengukur:

$$\boxed{t^{ttttttt}}$$

X.54. Quantum solution quality

Selain energy:

$$= ,$$

kita hitung:

$$P$$

Jika optimum:

$$,$$

maka:

$$P = P =$$

X.55. Approximation probability

Jika exact optimum:

$$,$$

kita dapat menentukan tolerance:

$$-$$

Kemudian:

$$P = P -$$

X.56. Ini jauh lebih baik daripada hanya "energy rendah"

Karena dua circuit dapat memiliki:

tetapi:

$$P \ P$$

Untuk discrete optimization, probability distribution hasil measurement sangat penting.

X.57. End-to-end benchmark

Eksperimen final kita sebaiknya memiliki tiga level.

Level I — Mathematical

$$t$$

Level II — Portfolio

$$t \quad t$$

Level III — Trading

$$t \quad t \quad P$$

X.58. Dengan demikian kita bisa mendapatkan tabel seperti:

Model	Energy Error	Success Prob.	Sharpe	MDD	CVaR	Cost
Exact	0	1.00	...	...	...	...
MILP	0	1.00	...	...	...	...
Markowitz	...	...	...	...	...	...
VQE	...	...	...	...	...	...
QAOA	...	...	...	...	...	...

Ini adalah tabel utama yang sangat kuat.

X.59. Tetapi ada satu jebakan penting

Jangan menggunakan:

$$t$$

dan mengatakan:

"VQE memiliki Sharpe lebih tinggi."

Jika VQE hanya menemukan portfolio yang berbeda karena sampling, kita harus memastikan:

$$tt, \, t, \, tt$$

X.60. Experimental seed protocol

Untuk setiap:

jalankan:

$$= 1, , 0$$

Simpan:

```
experiment_id
seed
model
dataset_window
qubo
parameters
solution
energy
runtime
PnL
Sharpe
MDD
CVaR
```

X.61. Reproducibility

Setiap experiment harus dapat direproduksi dengan:

$$\begin{bmatrix} t & t \end{bmatrix}$$

Idealnya kita menyimpan:

```
Git commit
Python version
Qiskit/PennyLane version
CUDA version
CPU
GPU
OS
```

X.62. Research ledger

Saya bahkan menyarankan membuat:

```
experiments/
├─ EXP-0001/
├─ EXP-0002/
├─ EXP-0003/
...
```

Setiap folder immutable setelah eksperimen selesai.

X.63. Jangan overwrite hasil

Misalnya:

```
EXP-0042/  
  config.yaml  
  metrics.json  
  trades.parquet  
  optimizer.log  
  circuit/  
  report.html
```

Jangan mengubah:

```
EXP-0042
```

setelah melihat hasil.

Jika ingin mencoba sesuatu:

$P - 004$

X.64. Research DAG

Kita dapat melihat seluruh penelitian sebagai:

$t \ t \ \ tt \ Pt \ t \ P$

Setiap node memiliki version.

Ini membuat debugging jauh lebih mudah.

X.65. Final experimental protocol

Jadi untuk setiap experiment:

Step 1

Load point-in-time data.

Step 2

Generate factor.

Step 3

Generate signal.

Step 4

Determine regime.

Step 5

Construct expected return:

t

Step 6

Construct covariance:

t

Step 7

Construct optimization problem:

t

Step 8

Solve dengan:

$\{, , \}$

Step 9

Convert solution menjadi portfolio.

Step 10

Apply risk constraints.

Step 11

Generate orders.

Step 12

Simulate realistic execution.

Step 13

Update portfolio.

Step 14

Compute PnL.

Step 15

Store metrics.

Step 16

Repeat walk-forward.

X.66. Research matrix

Saya menyarankan eksperimen utama kita nantinya memiliki struktur:

t	t	t	tt	t
-----	-----	-----	------	-----

Misalnya:

4	4	5	2	0
---	---	---	---	---

Jumlah:

14,400

run.

Ini memang besar, tetapi sebagian besar dapat dilakukan pada simulator/classical backend secara paralel.

X.67. Tetapi jangan langsung menjalankan 14.400 eksperimen

Kita harus melakukan:

Pt	t
------	-----

X.68. Pilot experiment

Misalnya:

$= 4$

asset:

Window kecil.

Tujuan:

memastikan seluruh pipeline benar.

Bukan mencari profitability.

X.69. Benchmark experiment

Kemudian:

$$= , 1, 2$$

Ukur:

$$t, , Pt$$

Di sinilah kita mulai membandingkan:

$$t, P, ,$$

X.70. Trading experiment

Baru kemudian:

dan:

$$t$$

X.71. Final out-of-sample experiment

Dan terakhir:

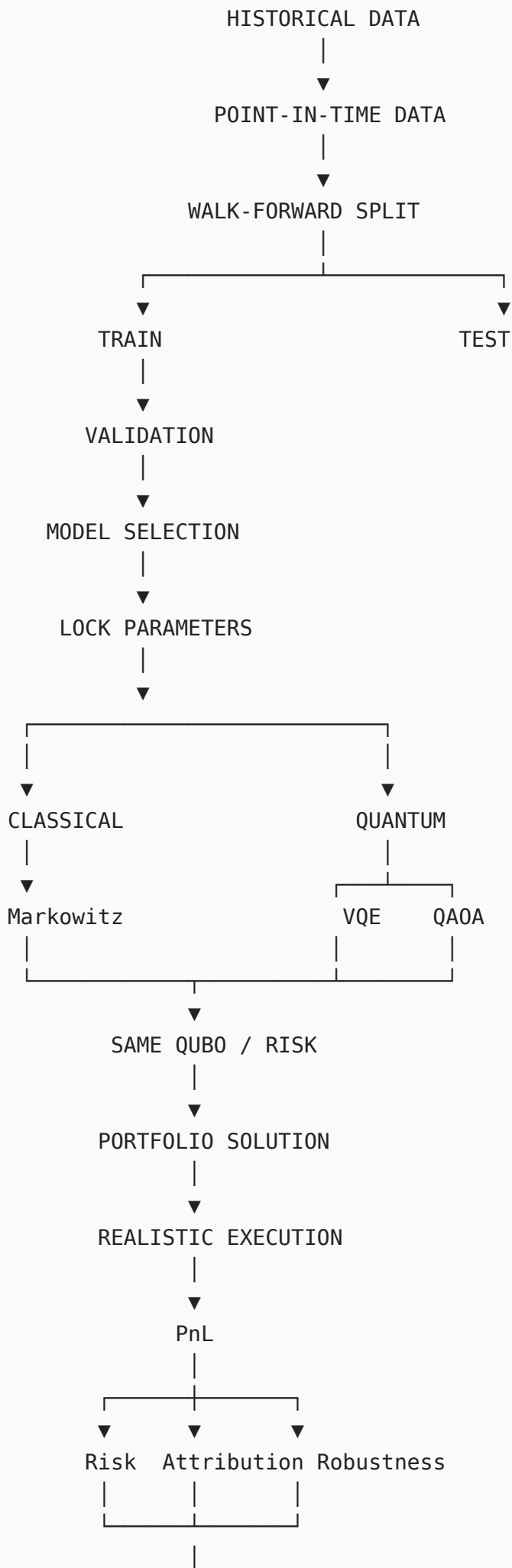
$$\square$$

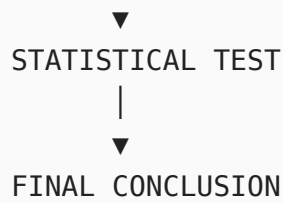
dijalankan pada data yang belum pernah disentuh.

Hasil inilah yang boleh kita jadikan:

$$\boxed{Pt}$$

X.72. Diagram lengkap Tahap X





X.73. Hipotesis utama yang nantinya bisa diuji

Saya sarankan kita tidak membuat klaim:

"Quantum lebih baik."

Tetapi membuat hipotesis yang jauh lebih spesifik.

H1

mampu memperoleh solusi QUBO dengan approximation error yang kompetitif terhadap classical optimizer.

H2

EPG warm-start menghasilkan:

$$P_P \ P$$

H3

EPG warm-start mengurangi:

$$tt$$

H4

Portfolio hasil optimizer menghasilkan performance OOS yang lebih baik daripada baseline.

H5

Improvement tetap signifikan setelah:

H6

Performance tetap robust terhadap:

$$ttt$$

X.74. Dan akhirnya kita dapat memisahkan tiga klaim ilmiah

Ini menurut saya **sangat penting untuk penelitianmu**.

Klaim A — Quantum optimization

Apakah VQE/QAOA mampu menyelesaikan problem optimasi?

Diukur dengan:

$$, P_{t,t}$$

Klaim B — Portfolio optimization

Apakah solusi tersebut menghasilkan portfolio yang lebih baik?

Diukur dengan:

$$t, , ,$$

Klaim C — Trading applicability

Apakah portfolio tersebut tetap menghasilkan edge setelah market dynamics dan execution diperhitungkan?

Diukur dengan:

$$tP, , t, , t, t$$

Jangan mencampur ketiganya.

Itu akan membuat penelitian kita jauh lebih kuat.

X.75. Posisi kita setelah Tahap X

Sekarang framework kita sudah memiliki:

□

dan:

□

Artinya kita sudah memiliki hampir semua yang dibutuhkan untuk membangun **research-grade quantitative trading laboratory**.

Tahap selanjutnya yang paling logis bukan sekadar menambah teori lagi. Kita perlu mulai **mengunci spesifikasi matematis final dari objective function dan state representation**.

Dengan kata lain, kita perlu menjawab secara konkret:

$$t, ,$$

$$,$$

$$,$$

dan akhirnya:

$$t$$

Itulah yang sebaiknya menjadi **Tahap Y — Unified Mathematical Formulation**, tempat seluruh komponen A–X kita satukan menjadi **satu formulasi matematis end-to-end**:

$$t \quad t \quad t, t \quad t \quad t \quad t \quad t \quad t_t \quad P_{t1}$$

Di tahap itu kita bisa mulai menurunkan **objective function final, constraint, state vector, action space, QUBO/Ising mapping, dynamic risk budget, dan feedback mechanism** secara matematis dari awal sampai akhir.

Prompt:

8/17/2026, 10:11:54 PM

lanjut Tahap Y

Response:

8/17/2026, 10:11:54 PM

Remembering

Reconstructed and checked the A-X framework, mappings, and factor design

Worked for 5s

Tahap Y — Unified Mathematical Formulation

Ya. Setelah Tahap X, sekarang kita sampai pada bagian yang menurut saya **paling penting secara matematis**: kita berhenti melihat sistem sebagai kumpulan modul terpisah, lalu menuliskannya sebagai **satu sistem keputusan stokastik yang utuh**.

Kerangka besarnya adalah:

$$t \quad t \quad t, t \quad t \quad t \quad t \quad t \quad t \quad t1 \quad t1 \quad t1$$

dengan feedback:

$$\quad \quad \quad$$

Ini pada dasarnya menjadikan sistem kita sebagai **closed-loop stochastic control system**.

Y.1. Titik awal: information filtration

Kita mulai dari konsep yang sangat fundamental.

Pada waktu t , trader tidak mengetahui masa depan.

Definisikan:

$$t = 0, 1, \dots, T$$

sebagai sigma-algebra seluruh informasi yang tersedia sampai waktu t .

Informasi tersebut dapat terdiri atas:

$$\mathcal{F}_t = \{P_t, V_t, O_t, M_t, R_t, S_t\}$$

Misalnya:

- P_t : harga,
- V_t : volume,
- O_t : order book,
- M_t : market factors,
- R_t : market regime,
- S_t : execution/position state.

Maka seluruh keputusan kita harus memenuhi:

$$a_t \in \mathcal{F}_t$$

Ini adalah constraint fundamental:

Tidak boleh ada future information dalam keputusan trading.

Y.2. State sistem

Sekarang kita bentuk state vector.

Kita definisikan:

$$x_t = [P_t, V_t, O_t, M_t, R_t, S_t]$$

Lebih eksplisit:

$$t = P_{t,t,t,t,t,t,t,t,t,t} \tau_t$$

Dengan:

t

adalah portfolio weight,

t

adalah posisi/quantity,

t

adalah cash,

t

adalah risk state,

dan

τ_t

adalah status order/position.

Y.3. Market factor vector

Karena kita ingin model berlaku baik untuk crypto maupun saham internasional, kita tidak boleh mendesain factor model khusus crypto.

Definisikan:

$t =$	$tttttttt$
-------	------------

Untuk crypto, misalnya:

$$t \equiv , , , , , , , ,$$

Untuk saham:

$$t = P, , t, , , 10, ,$$

Dengan demikian asset universe berubah, tetapi mathematical engine tetap sama.

Y.4. Cross-asset dependency

Misalkan return asset :

$$r_t$$

Kita modelkan:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

atau secara vector:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Ini sangat penting.

Kita tidak lagi mengatakan:

"Harga altcoin naik."

Kita mencoba memahami:

"Apakah pergerakan altcoin dapat dijelaskan oleh perubahan BTC, ETH, DXY, Gold, volatility, dan factor lainnya?"

Y.5. Expected return

Dari model tersebut:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Kita definisikan:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Jadi:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Ini menjadi input optimizer.

Y.6. Covariance

Kita juga membutuhkan:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Dari factor model:

$$r_t = \alpha + \beta r_{t-1} + \epsilon_t$$

Jika:

$$t =$$

dan:

$$t = ,$$

maka:

$$\boxed{=}$$

Ini memberi kita covariance matrix yang lebih terstruktur daripada sekadar sample covariance.

Y.7. Mengapa factor model penting?

Karena misalnya kita memiliki:

$$= 500$$

asset.

Sample covariance membutuhkan:

$$- 12$$

elemen covariance.

Untuk $= 500$:

$$12450$$

elemen.

Sedangkan factor model dengan:

$$= 10$$

factor hanya membutuhkan struktur:

$$\in {}^{500}_{10}$$

Ini jauh lebih manageable.

Y.8. Market regime

Namun:

$$t$$

dan:

t

tidak boleh dianggap stationary.

Kita definisikan latent regime:

$$t \in \{1, 2, \dots, K\}$$

Misalnya:

$$t = 1 \quad 2 \quad 4$$

Y.9. Conditional distribution

Maka:

$$p(t=1 | t=2) = \dots$$

Sehingga:

$$p(t=1 | t=2) = \dots$$

dan:

$$p(t=1 | t=2) = \dots$$

Atau versi lebih fleksibel:

$$p(t=1 | t=2) = \dots$$

$$p(t=1 | t=2) = \dots$$

Y.10. Sekarang masuk ke portfolio

Misalkan:

$$w_t \in \mathbb{R}^N$$

adalah vector portfolio weight.

Portfolio return:

$$r_{p,t} = w_t^T r_t$$

Expected portfolio return:

$$,t1t = tt$$

Variance:

$$,t1t = tt$$

Y.11. Risk-adjusted objective

Kita dapat memulai dari Markowitz:

$$= t - tt$$

dengan:

$$t \ 0$$

Namun untuk short-term trading, ini belum cukup.

Kita membutuhkan transaction cost.

Y.12. Transaction cost

Definisikan perubahan portfolio:

$$\Delta_t = t - t_{-1}$$

Biaya:

$$_t \Delta_t$$

Model sederhana:

$$_t = \Delta_t$$

Sehingga objective:

$$_t = t - tt - - t_{-1}$$

Sekarang optimizer tidak hanya mencari return tinggi.

Ia juga mempertimbangkan:

"Apakah perubahan posisi ini layak dibandingkan biaya melakukan perubahan?"

Y.13. Market impact

Untuk trading lebih realistis:

$${}_t\Delta = \Delta \Delta_{22}$$

Term pertama:

$$\Delta$$

merepresentasikan spread/fee.

Term kedua:

$$\Delta_{22}$$

merepresentasikan market impact.

Sehingga:

$${}_t = t - {}_{tt} - \Delta - \Delta^2$$

Ini jauh lebih dekat dengan problem institutional portfolio rebalancing.

Y.14. Risk budget

Sekarang kita masukkan filosofi yang sebelumnya kita bangun:

Setiap trade harus memiliki bounded risk.

Misalkan portfolio capital:

$${}_t$$

Risk budget:

$${}_t$$

Jika kita ingin maksimum loss:

$${}_t = 1$$

untuk sebuah trade, maka:

$$[{}_t \quad {}_t]$$

Untuk sistem general:

$${}_t = {}_{tt}$$

Y.15. Position sizing dari stop-loss

Misalkan:

t

adalah entry price.

Stop loss:

t

Risk per unit:

$$t = t - t$$

Jika quantity:

$t,$

maka:

$$t = tt$$

Maka constraint:

$$tt \ t$$

Sehingga:

$$\boxed{t \ tt}$$

Ini adalah jembatan langsung antara:

$$tt$$

dan:

$$t$$

Y.16. Expected return vs risk budget

Sekarang kita mempunyai dua layer:

Layer portfolio

$$t = \quad t$$

Layer execution

$$t = \quad tt, tt$$

Jadi optimizer tidak boleh secara langsung menentukan quantity tanpa risk constraint.

Y.17. Entry decision

Sekarang kita masuk ke short-term trading.

Untuk setiap asset , definisikan:

$$x_{i,t} \in \{-1, 0, 1\}$$

dengan:

$$-1 = \text{short}$$

$$0 = \text{hold}$$

$$1 = \text{long}$$

Maka action vector:

$$\mathbf{x}_t \in \{-1, 0, 1\}^N$$

Y.18. Mengapa discrete action?

Karena order placement sebenarnya discrete:

atau:

atau:

Iniilah yang membuat masalah ini cocok untuk:

dan:

Y.19. Binary representation

Untuk long-only:

$$x_{i,t} \in \{0, 1\}$$

Untuk long/short kita membutuhkan dua binary variable:

$$x_{i,t}^+, x_{i,t}^-$$

dengan constraint:

$$x_{i,t}^+ + x_{i,t}^- \leq 1$$

Kemudian:

$$x_{i,t} = x_{i,t}^+ - x_{i,t}^-$$

Y.20. Portfolio selection QUBO

Misalkan:

$$x_i \in \{0, 1\}$$

Objective:

$$= -$$

Untuk minimization:

$$\boxed{= -}$$

Inilah QUBO fundamental kita.

Y.21. Cardinality constraint

Misalnya maksimum:

asset.

Constraint:

$$=$$

Masukkan penalty:

$$P -$$
2

Maka:

$$\boxed{= - P -^2}$$

Y.22. Transaction-aware QUBO

Kita juga ingin memasukkan turnover.

Misalkan portfolio sebelumnya:

Turnover:

$$-$$

Maka:

$$= - \quad -$$

Karena:

$$, \in \{0, 1\},$$

kita memiliki:

$$- = - 2$$

Jadi seluruh objective tetap dapat ditulis sebagai quadratic polynomial.

Y.23. Sekarang faktor macro masuk ke QUBO secara tidak langsung

Ini sangat penting.

Kita **tidak perlu** membuat:

atau:

menjadi qubit tersendiri.

Mereka memengaruhi:

$$t$$

dan:

$$t$$

Sehingga:

$$t \quad t \quad t$$

Contoh:

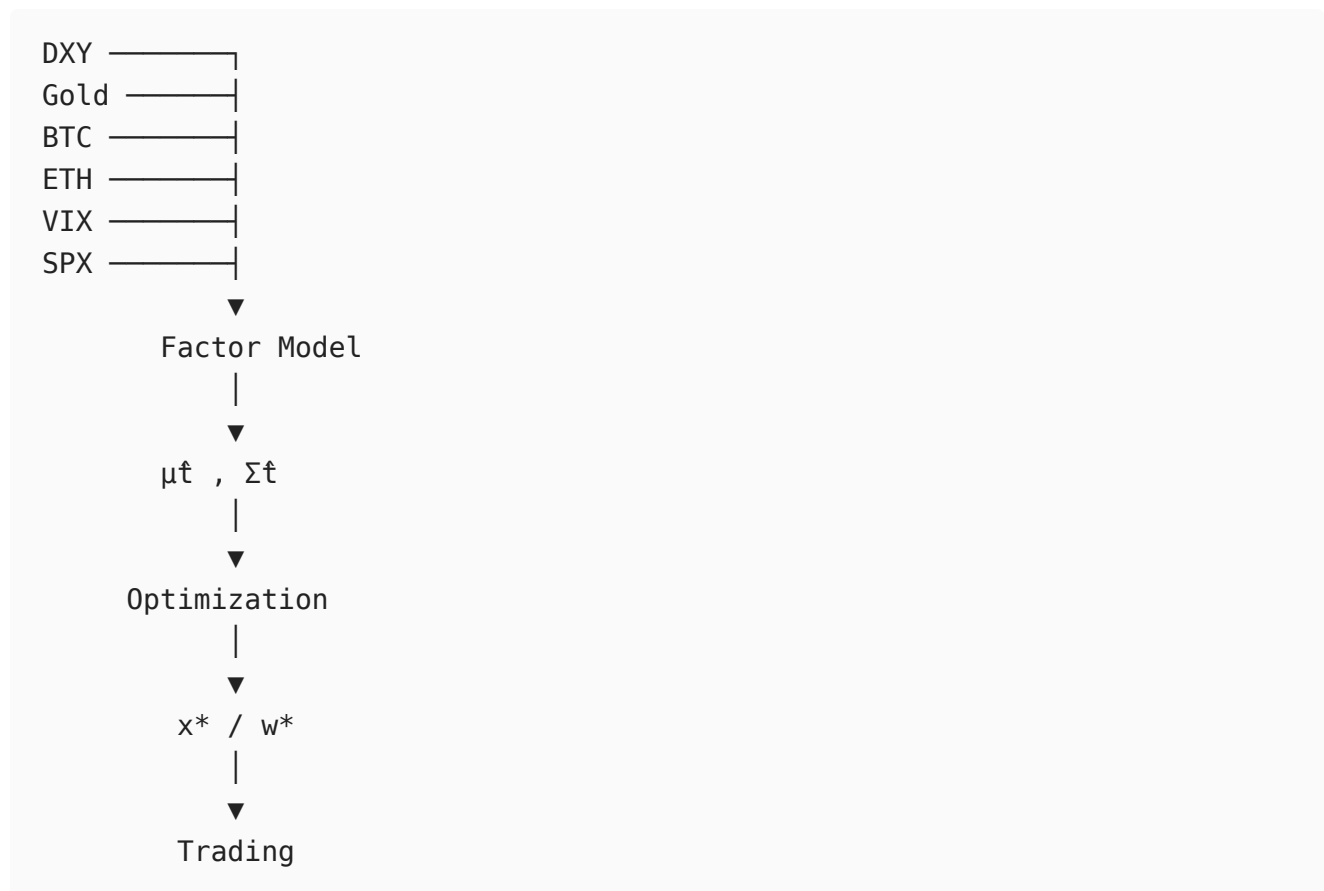
dapat mengubah:

$$t$$

menjadi lebih rendah.

Kemudian optimizer otomatis mengubah:

Y.24. Ini adalah arsitektur yang kita inginkan



Jadi macro factor tidak langsung menentukan order.

Ia mempengaruhi **conditional expectation**.

Y.25. Regime-dependent risk aversion

Sekarang kita memanfaatkan regime.

Kita definisikan:

$$\boxed{t = t}$$

Contoh:

Dalam crisis:

$$t$$

Sehingga penalti risk meningkat:

$$t$$

Akibatnya portfolio secara otomatis menjadi lebih defensif.

Y.26. Dynamic risk budget

Risk budget juga dapat bergantung pada regime:

$$t = t, t, t$$

Misalnya:

$$1$$

ketika:

$$t$$

tinggi.

Jadi:

$$t t \quad P t$$

Y.27. Entry signal

Kita sekarang membutuhkan threshold.

Misalkan expected excess return:

$$,t = ,t - ,t$$

Trade hanya jika:

$$,t$$

Tetapi threshold juga dapat mempertimbangkan uncertainty:

$$,t,t$$

Ini lebih masuk akal daripada hanya melihat return prediction.

Y.28. Signal-to-risk ratio

Definisikan:

$$,t = ,t, t$$

Kemudian:

$$,t = 1, \quad ,t = 0, \quad ,t = -1, \quad ,t = -$$

Namun dalam sistem kita, ini hanya **candidate signal**.

Final decision masih berasal dari optimizer.

Y.29. Candidate set

Kita tidak perlu mengoptimalkan seluruh dunia.

Definisikan:

$$t = \{ , t \}$$

Kemudian optimizer bekerja pada:

$$t = t$$

Ini sangat penting untuk scalability.

Y.30. Order generation

Setelah mendapatkan:

$$t,$$

kita menentukan:

$$t, t, P_{t, t}$$

Jadi:

$$\boxed{t \ P_t}$$

Kita definisikan:

$$P_t = t, t, , P, tt,$$

Y.31. Stop-loss mathematically

Untuk long:

$$t = P_t - t$$

Untuk short:

$$t = P_t \ t$$

Dengan:

$$t = ,t$$

atau volatility-adjusted distance.

Misalnya:

$$t = ,t \Delta t$$

Y.32. Take-profit

Jika reward-risk ratio:

$$= P - tt - ,$$

maka:

$$P = t t -$$

untuk long.

Sehingga:

$$P_t = P_t t$$

Y.33. Expected value trade

Jika:

adalah probability of winning,

dan:

$$= 1 - ,$$

dengan reward:

dan risk:

$$,$$

maka:

$$= -$$

Trade hanya layak jika:

$$0$$

Lebih realistis:

$$t = - - t$$

Maka:

$$t \ 0$$

Y.34. Ini menghubungkan optimizer dengan TP/SL

Sebelumnya portfolio optimizer menghasilkan:

Sekarang kita evaluasi apakah trade individual memenuhi:

$$t \ 0$$

Jadi:

$$P_{tt} \ t$$

Keduanya adalah layer berbeda.

Y.35. Order book

Misalkan best bid:

$$t$$

dan best ask:

$$t$$

Spread:

$$t = t - t$$

Mid-price:

$$t = t \ t^2$$

Entry order harus mempertimbangkan:

$$t, t, t_t$$

Y.36. Probability of fill

Untuk limit order:

$$P = , , t, t$$

dengan:

- : quantity,
 - : distance dari mid,
 - t : order book,
 - t : order-flow state.
-

Y.37. Expected value of placing order

Ini menjadi menarik.

Misalkan:

$$= P$$

Expected utility:

$$= t - 1 - tt$$

Jadi kita tidak hanya bertanya:

"Apakah trade profitable?"

Tetapi:

"Apakah menunggu limit order lebih baik daripada langsung market order?"

Y.38. Market order vs limit order

Misalkan:

$$t$$

dan:

$$t$$

Pilih:

$$t = \{t, t, t\}$$

Sekarang execution juga menjadi optimization problem.

Y.39. Posisi terbuka

Setelah order filled:

$$t_t \ 0$$

Kita memasuki state baru.

Definisikan:

$$t$$

sebagai entry price.

Floating PnL:

$$P_t = {}_tP_t - {}_t$$

untuk long.

Y.40. Position state

Kita dapat definisikan:

$${}_t = \{t, , P, P, , , \}$$

dengan:

- MFE = Maximum Favorable Excursion,
- MAE = Maximum Adverse Excursion.

Y.41. Inilah tempat feedback dimulai

Setiap periode:

$$t$$

dievaluasi.

Kita hitung:

$$t$$

Bandingkan dengan:

$$t$$

Kemudian:

$${}_{tt} = \{ , , t, \}$$

Y.42. Floating loss tidak otomatis berarti exit

Ini penting.

Jika:

$$P_t \geq 0,$$

bukan berarti:

$$t$$

Kita harus bertanya:

$$P_t$$

dan:

Jika thesis masih valid:

$$t;$$

maka posisi dapat dipertahankan.

Y.43. Floating profit juga tidak otomatis berarti TP

Jika:

$$P_t \geq 0,$$

kita mengevaluasi:

versus:

$$t$$

Jadi keputusan tidak berdasarkan:

"sudah profit, tutup."

Melainkan:

"Apakah expected future reward masih lebih besar daripada expected downside dan opportunity cost?"

Y.44. Dynamic stop-loss

Kita dapat mendefinisikan stop:

$$t = t, P, tt, ,$$

Contohnya:

$$t = t-1, P_t - t$$

untuk long.

Karena:

$$t \quad t-1,$$

stop tidak boleh mundur.

Y.45. Break-even

Jika profit mencapai threshold:

$$P_t ,$$

kita dapat memindahkan:

$$t \quad t$$

Secara matematis:

$$t = t-1, t$$

Y.46. Dynamic TP

Sekarang ide yang sebelumnya kamu berikan dapat diformalkan.

Jika:

$$t,$$

maka kita dapat meningkatkan target:

$$P_t \quad P_{t1}$$

Misalnya:

$$P_{t1} = P_t \quad tt$$

dengan:

$$t1 \quad t$$

jika momentum/regime masih mendukung.

Y.47. Tetapi jangan menaikkan TP hanya karena profit

Kita membutuhkan condition:

$$t_t \leq t_t$$

Jika:

$$t_t$$

melemah, TP justru dapat diturunkan atau posisi ditutup.

Y.48. State transition

Sekarang kita bisa menuliskan sistem sebagai:

$$t_{t+1} = t_t, t_t, t_{t+1}$$

dengan:

$$t_{t+1}$$

adalah random market shock.

Keputusan:

$$t_t = t_t$$

Jadi keseluruhan strategy adalah policy:

$$t_t$$

Y.49. Objective global

Sekarang kita sampai pada formulasi yang benar-benar menyatukan semuanya.

Kita tidak ingin hanya memaksimalkan satu-period return.

Kita ingin:

$$t_{t=0}^t, t_{t+1}, t_{t+1}$$

Misalnya:

$$t_t = t_{t+1} - t_t - t_t$$

Maka:

$$t_t^t, t_{t+1} - t_t - t_t$$

Ini adalah **global control objective**.

Y.50. Tetapi optimizer quantum tidak menyelesaikan **objective global secara langsung**

Ini distinction penting.

VQE/QAOA kita gunakan untuk menyelesaikan **local combinatorial optimization**:

$$t = t$$

Sedangkan:

adalah keseluruhan trading policy.

Jadi arsitektur kita adalah:

$$P \quad t \quad t \quad t$$

Y.51. Local-to-global hierarchy

Secara formal:

$$t \quad t$$

kemudian:

$$t \quad t$$

kemudian:

$$t \quad t$$

kemudian:

$$t \quad t$$

kemudian:

$$t \quad t$$

Y.52. Ising mapping

QUBO:

$$= 0$$

Gunakan:

$$\boxed{= 1 - 2}$$

dengan:

$$\in \{-1, 1\}$$

Substitusi menghasilkan:

$$=$$

Kemudian:

$$\boxed{=}$$

Ini adalah Hamiltonian yang masuk ke VQE/QAOA.

Y.53. VQE

VQE mencari:

$$=$$

Kemudian measurement menghasilkan:

Konversi:

Y.54. QAOA

QAOA menggunakan:

$$, = \text{---}$$

State:

$$, = ,$$

Kemudian:

dioptimalkan.

Y.55. Yang menarik: , bukan market variables

Mereka adalah:

□

Sedangkan:

t, t

adalah market-dependent quantities.

Jadi:

γ, t

dalam arti yang sama seperti market state.

Mereka merupakan parameter yang dipilih untuk menghasilkan solusi dari:

t

Y.56. Dynamic Hamiltonian

Ini merupakan bagian yang sangat penting dari sistem kita.

Hamiltonian sebenarnya berubah terhadap waktu:

$$H_t = H_t(t, \gamma_t, t_t, t_t t_t)$$

Jadi kita tidak mempunyai satu Hamiltonian permanen.

Kita memiliki:

$1, 2, \gamma$

Y.57. Dengan demikian trading loop menjadi:

$$H_t(t, \gamma_t, t_t)$$

kemudian market berubah:

$$H_{t+1}(t+1, \gamma_{t+1}, t_{t+1})$$

Kemudian:

t

tidak lagi dianggap optimal secara permanen.

Y.58. Ini menjawab masalah "mengapa harus re-optimize?"

Karena:

$$t_1 \quad t$$

Perubahan dapat berasal dari:

$$\Delta, \Delta, \Delta, \Delta t, \Delta t$$

Jadi:

$$t$$

dapat menjadi suboptimal.

Y.59. Re-optimization trigger

Kita tidak harus menjalankan optimizer setiap tick.

Definisikan:

$$\Delta_t = t - t_{-1}$$

Re-optimize jika:

$$\boxed{\Delta_t}$$

Atau jika:

$$\Delta = 1$$

Atau:

$$\Delta$$

Atau:

$$\Delta$$

Y.60. Jadi kita memiliki event-driven architecture

Optimizer aktif ketika:

$$\boxed{t_t = \{, , , Pt, t\}}$$

Ini jauh lebih efisien daripada:

$$t$$

setiap saat tanpa alasan.

Y.61. Full unified equation

Sekarang seluruh framework dapat diringkas:

$$t = 0t, t, t, t$$

kemudian:

$$t, t = t$$

kemudian:

$$t = -_t \, tt \, t, t-1 \, P_t$$

kemudian:

$$t = t$$

dengan:

$$\in \{t, P, , , \}$$

Kemudian:

$$t = t, t$$

dan:

$$t = tt, tt$$

Kemudian:

$$t = t, t, t, t, P_t$$

Execution menghasilkan:

$$t1 = t, t, t1$$

Y.62. Closed-loop formulation

Akhirnya:

$$t \, t \, t, t \, t \, t \, t \, t \, t \, t1 \, t1 \, t1$$

Dan:

$$t1$$

kembali masuk ke sistem.

Itulah feedback loop.

Y.63. Formulasi sebagai constrained stochastic control

Kalau kita ingin menuliskannya dalam bentuk yang lebih formal:

$$\boxed{_{t=0}^t t, tt1 = t, t, t1, t \in t, t \quad \tau_{t, t} \quad t, t_t \quad t}$$

Ini adalah **master formulation**.

Y.64. Tetapi ada dua level optimization

Ini menurut saya adalah desain paling penting dari seluruh framework kita:

□

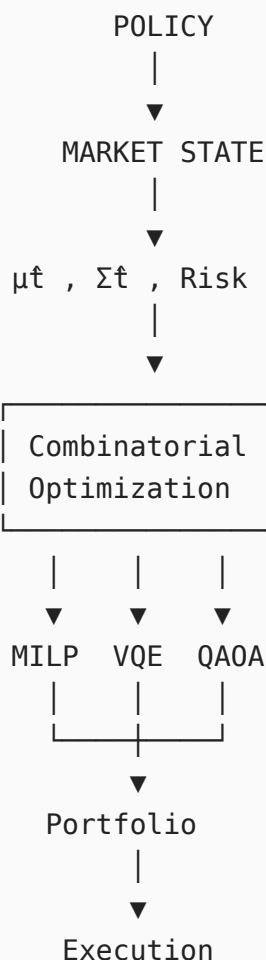
memilih bagaimana sistem mengambil keputusan.

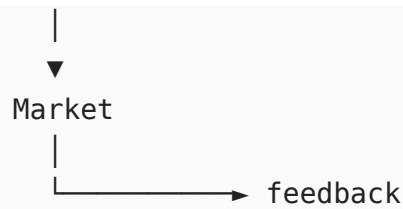
Sedangkan:

□

menentukan portfolio/order candidate pada state tertentu.

Secara diagram:





Y.65. Mengapa desain ini bagus untuk proyek kita?

Karena kita **tidak memaksakan quantum computing untuk menyelesaikan seluruh trading problem.**

Quantum computing hanya digunakan di tempat yang memang natural:

□

Sementara:

- factor estimation,
- covariance estimation,
- regime detection,
- risk management,
- execution,

tetap dapat menggunakan classical methods yang memang sesuai.

Ini jauh lebih defensible secara ilmiah daripada mencoba membuat "semua bagian trading menjadi quantum".

Y.66. Dan ini juga membuka banyak jenis problem

Framework yang sama dapat menghasilkan:

Portfolio selection

MaxCut

untuk clustering/exposure structure.

TSP

Untuk routing order execution atau allocation scheduling tertentu:

Knapsack

Untuk capital allocation:

subject to:

Assignment

Untuk allocation capital/order capacity.

Dengan demikian:

□

bukan terikat pada portfolio saja.

Y.67. Dan akhirnya kita bisa melihat visi besar sistem

Sistem kita bukan:

"bot yang membeli crypto."

Melainkan:

□

yang mengambil:

dan menghasilkan:

Y.68. Formulasi final yang saya sarankan kita "kunci"

Saya akan menjadikan persamaan berikut sebagai **central equation** dari seluruh proyek:

$$P_t = \sum_{t=0}^{t-1} P_{t,t-1}$$

dengan:

$$P_{t,t} = P_{0t,0t,0t,t}$$

dan:

$$P_{t,t} = P_{t,t,t,t}$$

Kemudian keputusan dieksekusi melalui:

$$t = t, t, t, P_t$$

Dan seluruh sistem berevolusi:

$$t1 = t, t, t1$$

Y.69. Satu hal yang perlu kita luruskan

Ada perbedaan antara **model penelitian** dan **bot trading production**.

Untuk penelitian, kita ingin:

□

Kita harus memastikan:

- no leakage,
- valid mathematical formulation,
- correct QUBO,
- correct Ising mapping,
- correct execution simulation,
- robust OOS result.

Baru setelah itu:

$$t \ t$$

Y.70. Posisi kita sekarang

Dengan Tahap Y, arsitektur kita sudah berubah dari sekadar:

menjadi satu persamaan besar:

$$t \ t \ t, t \ t \ t \ t \ t \ t \ P \ t1$$

Dan menurut saya, **Tahap Y** adalah titik di mana proyek ini akhirnya memiliki **"mathematical spine"** yang jelas.

Tahap berikutnya yang paling natural adalah **Tahap Z — Concrete Model Specification & Implementation Blueprint**: kita berhenti pada formulasi abstrak dan menentukan **persis** variabel apa yang digunakan, timeframe, asset universe, feature/factor matrix, estimasi t , estimasi t , regime model, bentuk QUBO final, encoding qubit, constraint, penalty coefficient,

VQE/QAOA circuit, serta format data yang harus masuk dan keluar dari setiap modul. Dengan itu kita bisa mulai menerjemahkan formulasi Y menjadi arsitektur kode yang benar-benar dapat dibangun dan diuji.

Prompt:

8/17/2026, 10:12:59 PM

lanjut Tahap Z

Response:

8/17/2026, 10:13:01 PM

Tahap Z — Concrete Model Specification & Implementation Blueprint

Baik. Tahap Z adalah transisi dari desain matematis ke sistem yang benar-benar dapat diimplementasikan.

Sampai Tahap Y kita sudah mempunyai:

t t t t t t t t t t

Sekarang kita harus menjawab pertanyaan yang jauh lebih konkret:

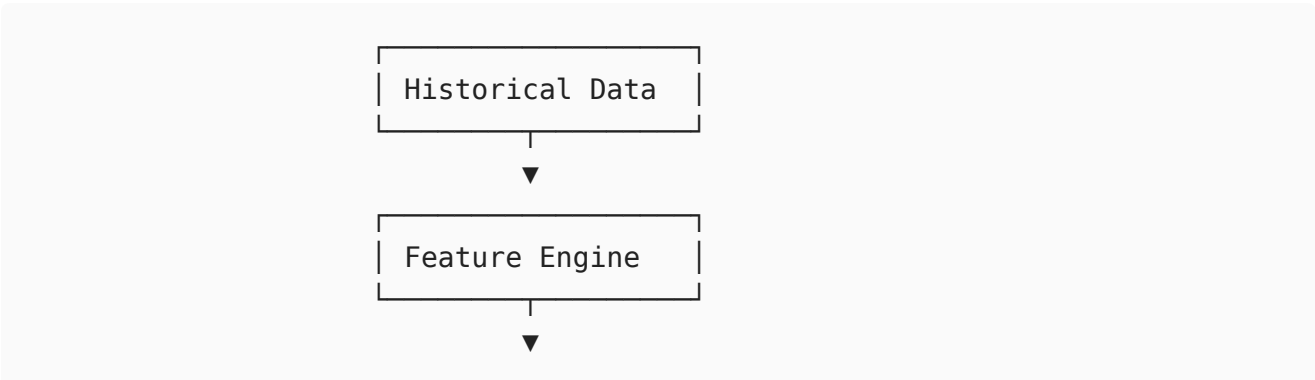
"Data apa yang masuk? Diproses bagaimana? Persamaan persisnya apa? Output tiap modul apa? Dan di mana QUBO/Ising/VQE/QAOA benar-benar digunakan?"

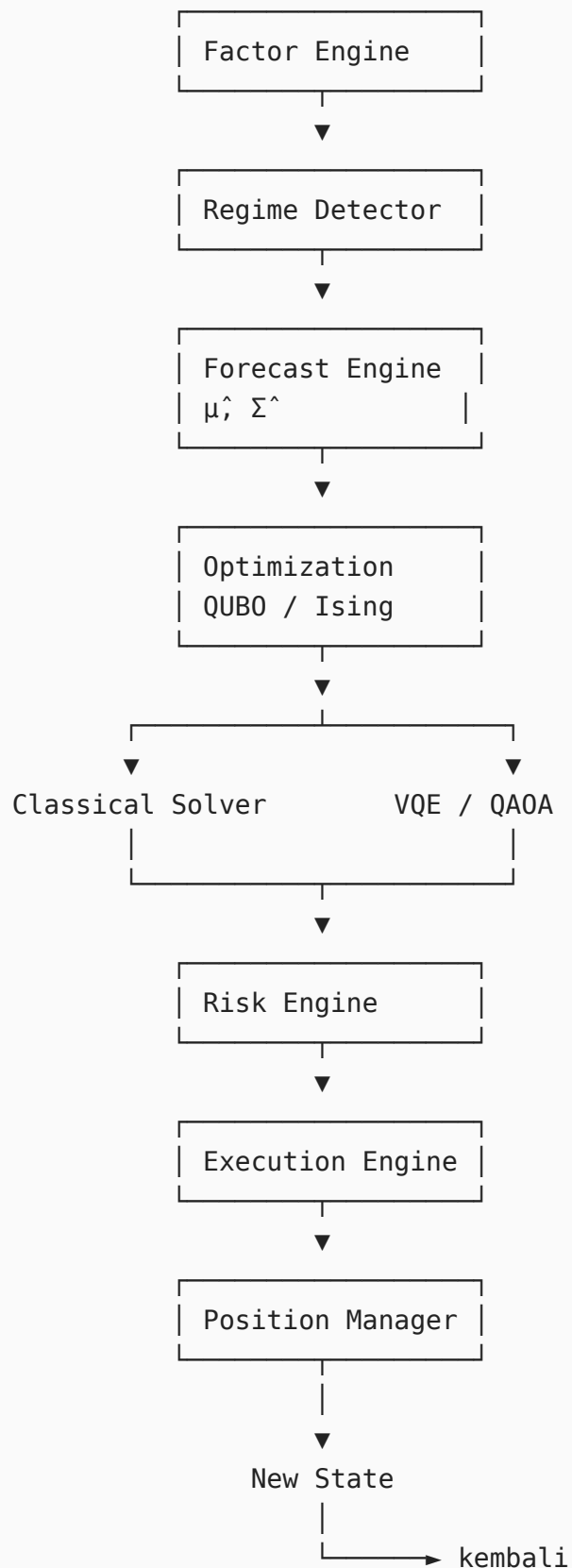
Saya akan membuat Tahap Z sebagai **blueprint teknis versi 1.0**. Kita jangan langsung coding sebelum struktur ini konsisten.

Z.1. Prinsip desain utama

Saya ingin kita memisahkan sistem menjadi **10 layer**:

Secara arsitektur:





Z.2. Jangan langsung mulai dengan 500 aset

Ini keputusan desain yang penting.

Versi pertama sebaiknya menggunakan **small controlled universe**.

Misalnya untuk crypto:

$$t = \{, , , P, \}$$

sekitar:

$$= -20$$

Untuk saham:

$$t = \{P, , , \}$$

Dan factor:

$$= \{P, , , , 10\}$$

Tujuannya bukan membuat sistem terbesar.

Tujuannya:

□

Z.3. Multi-asset universe

Saya menyarankan kita mendefinisikan tiga kategori.

Tier 1 — Target assets

Asset yang benar-benar boleh ditradingkan:

Contoh:

$$= \{, P, , \}$$

Tier 2 — Market leaders

Asset yang digunakan sebagai explanatory factors:

$$= \{, \}$$

Tier 3 — Macro factors

$$= \{, , , P, , 10\}$$

Dengan demikian:

$$\boxed{=}$$

Z.4. Timeframe

Kita perlu membedakan:

Data resolution

Misalnya:

$$\Delta t = 1$$

Decision interval

Misalnya:

$$\Delta t = 1$$

Execution resolution

Misalnya:

$$\Delta t = 1$$

Ini sangat penting.

Kita tidak boleh melakukan:

| prediksi 1 jam → eksekusi menggunakan harga close 1 jam

tanpa memodelkan bagaimana order benar-benar terisi.

Jadi:

$$\boxed{1}$$

sedangkan:

$$\boxed{1 \quad t}$$

Z.5. Dataset utama

Untuk setiap asset , kita simpan:

$$,t = , , , ,$$

Dengan:

- : open,
- : high,
- : low,
- : close,
- : volume.

Untuk order book:

$$t = 1, 1, 1, 1,$$

Z.6. Return

Kita gunakan log return:

$$r_{i,t} = \ln \frac{P_{i,t}}{P_{i,t-1}}$$

Untuk multi-asset:

$$r_t = (r_{1,t}, r_{2,t}, \dots, r_{N,t})$$

Z.7. Feature vector

Untuk setiap asset:

$$f_{i,t} = (r_{i,t-1}, r_{i,t-2}, \dots, r_{i,t-L}, \dots)$$

Tetapi saya menyarankan **jangan memasukkan puluhan indikator teknikal sejak awal**.

Kita ingin mengetahui kontribusi setiap kelompok informasi.

Maka feature groups:

$$f_t = (f_{1,t}, f_{2,t}, \dots, f_{N,t})$$

Z.8. Factor matrix

Misalkan kita mempunyai:

factor.

Maka:

$$f_t = (f_{1,t}, f_{2,t}, \dots, f_{K,t})$$

Contoh:

$$f_t = (r_{1,t}, r_{2,t}, \dots, r_{N,t}, P_t, \dots)$$

Z.9. Standardisasi

Karena skala masing-masing factor berbeda:

$$x_{i,t} = \frac{x_{i,t} - \min_t x_{i,t}}{\max_t x_{i,t} - \min_t x_{i,t}}$$

Penting:

$$x_{i,t}$$

dan:

$$x_{i,t}$$

harus dihitung **hanya menggunakan data yang tersedia sampai waktu training tersebut.**

Tidak boleh:

Itu data leakage.

Z.10. Factor model

Kita gunakan:

$$x_{i,t} = \mu_i + \beta_i' F_t + \epsilon_{i,t}$$

Secara matrix:

$$X = \mu + \beta F + \epsilon$$

Z.11. Estimasi beta

Untuk versi awal:

$$\beta = (X'X)^{-1}X'y$$

Namun untuk rolling system:

$$x_{i,t} = \mu_i + \beta_i' F_t + \epsilon_{i,t}$$

Jadi beta berubah terhadap waktu.

Z.12. Expected return

Kita memperoleh:

$$t = t + \epsilon$$

Tetapi saya menyarankan kita tidak berhenti di sini.

Kita dapat menambahkan signal component:

$$t = \mu + \epsilon$$

Misalnya:

$$t = \mu + \epsilon$$

Dengan demikian:

- factor model memberikan economic structure,
- signal model menangkap nonlinear residual behavior.

Z.13. Versi pertama tidak perlu deep learning

Ini keputusan penting.

Untuk baseline:

bisa berupa:

- linear regression,
- ridge,
- elastic net,
- random forest,
- gradient boosting.

Baru setelah baseline valid, kita bandingkan dengan model neural.

Karena kalau langsung menggunakan deep learning dan hasil bagus:

kita tidak tahu apakah keuntungan berasal dari model atau leakage/overfitting.

Z.14. Covariance matrix

Residual:

$$e = t - \hat{t}$$

Covariance factor:

$$=$$

Residual covariance:

$$=$$

Kemudian:

$$\Sigma_t = \Sigma_{t-1} - \frac{\Sigma_{t-1} \mathbf{y}_t \mathbf{y}_t^T \Sigma_{t-1}}{1 + \mathbf{y}_t^T \Sigma_{t-1} \mathbf{y}_t}$$

Z.15. Shrinkage

Covariance matrix short-term biasanya noisy.

Maka kita dapat menggunakan:

$$\Sigma_t = (1 - \alpha) \Sigma_{t-1} + \alpha \Sigma_t^{\text{target}}$$

dengan:

- Σ_t^{target} : target covariance,
- α : shrinkage intensity.

Ini penting karena optimizer sangat sensitif terhadap covariance.

Z.16. Regime engine

Kita definisikan:

$$r_t \in \{1, 2, \dots, K\}$$

Feature regime:

$$\mathbf{x}_t = \mathbf{x}_{t, r_t} = \mathbf{x}_{r_t, t}$$

Versi pertama dapat menggunakan:

$$\Sigma_t = \Sigma_{r_t}$$

Misalnya:

$$=$$

Z.17. Tiga regime awal

Saya menyarankan:

$$t = 1 \quad 2$$

Kemudian:

$$t = t$$

Misalnya secara konseptual:

$$t$$

Z.18. Regime juga mengubah expected return

Bukan hanya risk aversion.

Kita dapat menggunakan:

$$t = \begin{matrix} t & t \\ t & t \end{matrix}$$

Dan:

$$t = \begin{matrix} t & t & t \\ & t & \end{matrix}$$

Ini menjadikan model **regime conditional**.

Z.19. Sekarang masuk ke optimizer

Kita mendefinisikan binary variable:

$$= \begin{cases} 1, & t \neq 0, \\ 0, & t \end{cases}$$

Portfolio:

$$\in \{0, 1\}$$

Z.20. Objective

Kita mulai dengan:

$$= -$$

Interpretasi:

$$-$$

berusaha memaksimalkan expected return.

Sedangkan:

menghukum risk.

Z.21. Capital constraint

Misalkan setiap asset mempunyai allocation weight .

Constraint:

$$1$$

Penalty:

$$P - 1^2$$

Z.22. Cardinality

Jika maksimum asset:

$$,$$

maka:

Penalty:

$$P - 2^2$$

Jika kita ingin **tepat** :

$$\boxed{P - 2^2}$$

Z.23. Turnover

Misalkan:

Biaya:

$$t = -$$

Karena binary:

$$- = - 2$$

Sehingga:

$$t = - 2$$

Ini quadratic.

Sangat cocok untuk QUBO.

Z.24. Final QUBO

Maka versi pertama:

$$t = - t^2 - P - P -$$

Ini adalah **QUBO utama sistem kita**.

Z.25. Constraint tambahan

Nanti kita dapat menambahkan:

Maximum sector exposure

$$\in t$$

Maximum correlated assets

$$1$$

Minimum liquidity

$$= 0$$

Maximum leverage

Semua ini dapat kita ubah menjadi binary constraints/penalties jika diperlukan.

Z.26. Penalty coefficient

Ini bagian yang sangat sering salah.

Tidak boleh asal:

$$P = 1$$

Secara prinsip:

$$P$$

Misalnya:

$$P$$

Namun kita harus hati-hati karena penalty terlalu besar juga membuat landscape numeriknya buruk.

Maka nanti kita lakukan:

$$P \in \{P_1, P_2, \dots, P\}$$

dan melakukan sensitivity analysis.

Z.27. QUBO $\rightarrow$ Ising

Gunakan:

$$= 1 - 2$$

Secara umum:

Kemudian:

$$=$$

Quantum Hamiltonian:

$$=$$

Z.28. Jumlah qubit

Jika:

$$= 10$$

asset binary:

$$= 10$$

Jika kita menggunakan long/short encoding:

$$, - ,$$

maka:

$$= 2$$

Untuk tahap awal saya sangat menyarankan:

$$= 12$$

supaya kita dapat membandingkan:

$$t$$

Z.29. Exact solver adalah wajib

Jangan langsung membandingkan:

| VQE vs market.

Kita harus memiliki ground truth.

Untuk:

$$= 10,$$

jumlah kemungkinan:

$$2^{10} = 1024$$

Kita dapat brute force seluruh state.

Sehingga:

$$t = \in \{0,1\}^{10}$$

Ini memberi:

$$t$$

Z.30. Quantum approximation ratio

Jika QAOA/VQE menghasilkan:

$$,$$

sedangkan optimum:

$$,$$

kita dapat mengukur:

$$= -$$

Untuk minimization:

$$0$$

Relative gap:

$$\frac{\text{gap}}{\text{max}}$$

Ini jauh lebih ilmiah daripada sekadar:

"QAOA menghasilkan profit lebih tinggi."

Z.31. VQE baseline

VQE:

$$= 0$$

Energy:

$$= t$$

Optimasi:

$$=$$

Z.32. Ansatz

Untuk baseline:

$$= t$$

Misalnya:

dan entanglement:

$$,1$$

Kita bisa menggunakan EfficientSU2 untuk baseline.

Namun jangan lupa:

ansatz bukan bagian dari market model.

Ia adalah computational mechanism untuk mendekati ground state.

Z.33. QAOA baseline

Untuk QAOA:

$$, = P_{=1}^{--}$$

Objective:

$$, =$$

Z.34. Classical benchmark

Minimal kita punya:

$$\boxed{t}$$

$$\boxed{Ptt}$$

$$\boxed{P}$$

$$\square$$

$$\square$$

Tidak semua harus digunakan pada versi pertama, tetapi exact solver **harus** ada.

Z.35. Dari binary selection ke actual portfolio

Ini juga penting.

Misalkan QUBO menghasilkan:

$$= 1, 0, 1, 0, 1$$

Itu **belum** berarti:

$$= 0, 0, 0, 0, 0$$

Kita perlu allocation layer.

Untuk selected assets:

$$= \{ = 1 \}$$

Kemudian solve:

$$\boxed{= -}$$

subject to:

$$= 0$$

Z.36. Jadi QUBO bukan necessarily optimizer seluruh weight

QUBO kita dapat digunakan sebagai:

$\square$

Sedangkan continuous optimizer digunakan untuk:

$\square$

Ini menurut saya desain yang jauh lebih kuat.

Z.37. Dari weight ke risk budget

Setelah:

didapat, kita hitung:

$$t =$$

Kemudian scaling:

$$\boxed{= t t t}$$

Dengan batas:

$$1$$

Z.38. Dari portfolio weight ke quantity

Jika capital:

$$t,$$

maka dollar allocation:

$$= t$$

Quantity:

$$\boxed{= P}$$

Kemudian risk constraint:

$$P -$$

Sehingga:

$$\boxed{= {}_t P, P -}$$

Ini adalah quantity final.

Z.39. Stop-loss engine

Untuk versi awal:

$$= P -$$

untuk long.

Untuk short:

$$= P$$

Lebih tepat jika:

dikonversi menjadi volatility pada horizon trade.

Misalnya:

$$\Delta t = \Delta t_1$$

Z.40. TP engine

Gunakan:

$$P = P \quad P -$$

untuk long.

$$P = P - \quad - P$$

untuk short.

Dengan:

$$= , , t$$

Z.41. Expected trade value

Kita hitung:

$$= -1 - - t$$

Trade hanya boleh diteruskan jika:

$$\boxed{0}$$

Ini menjadi **final gate** setelah optimizer.

Z.42. Execution engine

Execution engine menerima:

```
TradePlan
├─ symbol
├─ side
├─ quantity
├─ entry
├─ stop_loss
├─ take_profit
├─ order_type
└─ expiry
```

Kemudian menentukan:

t

atau:

t

Z.43. Limit order model

Kita harus mensimulasikan:

P

Jangan menggunakan:

“kalau candle menyentuh entry maka order filled.”

Itu terlalu optimistik.

Minimal kita perlu mempertimbangkan:

$t, , , tt$

Z.44. Execution cost

Actual return:

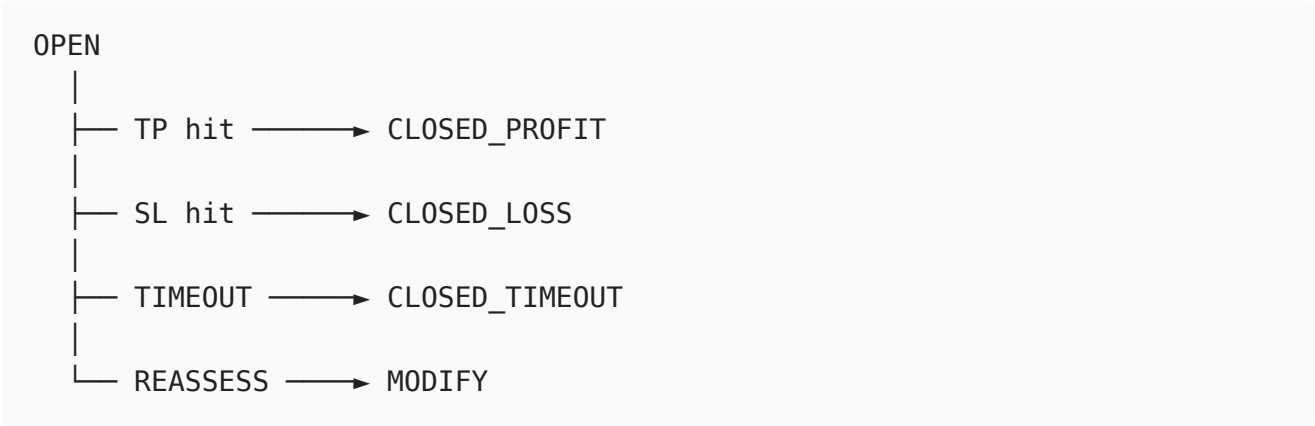
$t = - - - - t$

Sehingga:

$t = -t$

Z.45. Position manager

Setelah filled:



Z.46. Reassessment

Setiap:

$$\Delta t$$

kita hitung:

Kemudian:

dibandingkan dengan:

$$t$$

Jika:

$$t,$$

hold.

Jika:

$$t ,$$

exit.

Z.47. Break-even rule

Jika:

$$P_t \geq P_{t-1}$$

maka:

$$t = t$$

Namun kita dapat menggunakan:

$$t = t$$

untuk long.

Z.48. Trailing stop

Jika momentum tetap kuat:

$$t = t-1, P_t - t$$

Ini memastikan:

$$t$$

hanya bergerak ke arah yang mengurangi downside.

Z.49. Dynamic TP

Misalkan:

$$t = t$$

Jika:

$$t \geq t$$

kita dapat mempertahankan/menaikkan TP.

Jika:

$$t < t$$

kita kurangi target atau exit.

Jadi:

$$P_{t1} = P_{t, t, t, P_t}$$

Z.50. Full state object

Secara implementasi, saya menyarankan satu objek konseptual:

```
State
├── timestamp
├── market
│   ├── prices
│   ├── volume
│   └── order_book
├── factors
│   ├── BTC
│   ├── ETH
│   ├── DXY
│   ├── Gold
│   ├── SPX
│   └── VIX
├── regime
│   ├── state
│   └── probability
├── forecast
│   ├── mu
│   └── Sigma
├── portfolio
│   ├── weights
│   ├── positions
│   └── cash
├── risk
│   ├── budget
│   ├── volatility
│   └── exposure
└── execution
    ├── open_orders
    └── fills
```

Z.51. Pipeline data

Secara computational:

t	tt	tt	t	tt	t	P	t
-----	------	------	-----	------	-----	-----	-----

Ini akan sangat membantu ketika nanti kita mulai membuat repository.

Z.52. Struktur repository yang saya sarankan

Saya akan mengubah proyek kita menjadi kira-kira:

```
quant-system/
|
├── data/
|   ├── raw/
|   ├── processed/
|   └── features/
|
├── config/
|   ├── assets.yaml
|   ├── factors.yaml
|   └── strategy.yaml
|
├── src/
|   ├── data/
|   |   ├── loader.py
|   |   └── cleaner.py
|   |
|   ├── features/
|   |   └── feature_engine.py
|   |
|   ├── factors/
|   |   └── factor_model.py
|   |
|   ├── regime/
|   |   └── regime_model.py
|   |
|   ├── forecast/
|   |   └── return_model.py
|   |
|   ├── optimization/
|   |   ├── qubo.py
|   |   ├── ising.py
|   |   ├── exact.py
|   |   ├── classical.py
|   |   ├── vqe.py
|   |   └── qaoa.py
|   |
|   └── risk/
|       ├── sizing.py
|       ├── stop_loss.py
|       └── risk_manager.py
```



```
|
|
|   └─ execution/
|       └─ order.py
|       └─ fill_model.py
|       └─ slippage.py
|
|   └─ portfolio/
|       └─ allocator.py
|
|   └─ backtest/
|       └─ engine.py
|
|   └─ evaluation/
|       └─ metrics.py
|       └─ attribution.py
|
└─ notebooks/
└─ experiments/
└─ tests/
└─ README.md
```

Z.53. Sangat penting: jangan membuat `quantum.py` sebagai pusat sistem

Quantum hanyalah:

`tt`

Struktur:

```
OptimizationProblem
|
|   └─ ExactSolver
|   └─ ClassicalSolver
|   └─ VQE
|   └─ QAOA
```

Semua menerima **problem yang sama**.

Ini memungkinkan kita melakukan eksperimen yang benar-benar fair.

Z.54. Interface solver

Secara konseptual:

```

solve(problem)
├── exact
├── classical
├── vqe
└── qaoa

```

Output:

```

Solution
├── bitstring
├── objective_value
├── feasible
├── runtime
└── metadata

```

Dengan begitu:

t

dan:

t

memiliki format yang sama.

Z.55. Backtest engine

Backtest harus berjalan secara chronological:

$t_0 \ t_1 \ t_2 \ t$

Pada setiap t :

1. update data,
2. update feature,
3. estimate factor,
4. detect regime,
5. estimate , ,
6. build QUBO,
7. solve,
8. generate order,
9. simulate execution,
10. update position,

11. update equity.

Z.56. Tidak boleh ada future leakage

Pada waktu:

t ,

model hanya boleh menggunakan:

t

Bukan:

t

Dengan kata lain:

$$t = t_t$$

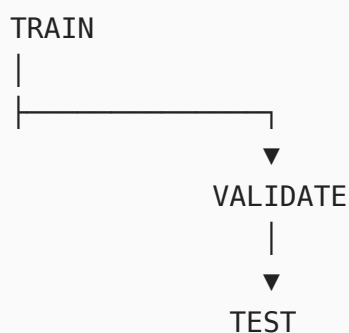
Bukan:

t_0

Ini harus menjadi **aturan absolut**.

Z.57. Walk-forward training

Saya menyarankan:



Kemudian window bergerak:

```
Train — Validation — Test
      Train — Validation — Test
            Train — Validation — Test
```

Ini disebut:

Z.58. Contoh

Misalnya:

2021 – 01 2022 – 0

training.

2022 – 0 2022 – 0

validation.

2022 – 10 2022 – 12

test.

Kemudian window bergeser.

Z.59. Benchmark yang wajib

Kita jangan hanya membandingkan:

vs:

Kita harus membandingkan:

☐

t

t

☐

☐

☐

Dengan **identical data dan execution assumptions**.

Z.60. Metrics

Minimal:

Return

Annualized return

Volatility

Sharpe

$$= -$$

Maximum drawdown

$$= {}_t P_t - {}_{t_t} P_t$$

Calmar

$$=$$

Sortino

$$t = -$$

Z.61. Tetapi untuk sistem kita ada metrics tambahan

Karena tujuan kita bukan sekadar profit.

Kita harus mengukur:

$$tttt$$

misalnya:

Kemudian:

$$tt$$

berapa persen solusi memenuhi seluruh constraint.

Dan:

$$t$$

Misalnya:

$$ttt$$

Z.62. Kita bahkan bisa membuat decomposition

Total performance:

$$P_{tt}$$

dapat diuraikan menjadi:

$$P_{tt} = P \ P_t \ P_t \ P - P_t$$

Ini sangat berguna untuk penelitian.

Jika hasil buruk:

| Apakah prediction buruk?

atau:

| QUBO buruk?

atau:

| Execution buruk?

atau:

| Risk management buruk?

Z.63. Quantum advantage tidak boleh didefinisikan sebagai "profit lebih tinggi"

Ini salah satu prinsip penelitian terpenting.

Quantum algorithm dapat memberikan:

tetapi belum tentu:

$$P_t \ P_t$$

Karena market outcome bersifat stochastic.

Maka kita pisahkan:

Computational performance

$$, t, t, t$$

Financial performance

$$t, , , t$$

Keduanya harus dilaporkan.

Z.64. Experimental matrix

Saya menyarankan eksperimen seperti:

Model	Optimizer	Data	Regime	Execution
Baseline	Equal Weight	OHLCV	No	Ideal
Markowitz	Classical	OHLCV	No	Realistic
Factor	Classical	OHLCV+Factors	Yes	Realistic
QUBO	Exact	Same	Yes	Realistic
QUBO	VQE	Same	Yes	Realistic
QUBO	QAOA	Same	Yes	Realistic

Ini jauh lebih kuat daripada sekadar:

"Saya menjalankan VQE pada crypto."

Z.65. Tahap Z menghasilkan satu master data flow

Pada akhirnya:

$$0t \ t \ t \ t \ t, \ t \ t \ t \ t \ t \ P_t \ t_t \ P_{t_t} \ P_t \ t_t$$

Z.66. Versi minimum viable system

Saya justru **tidak** ingin kita langsung membangun semua bagian.

Versi pertama seharusnya:

$$=$$

assets.

$$\Delta t = 1$$

decision.

Factors:

$$,,,P,$$

Forecast:

$$t$$

Regime:

$$-tt$$

Optimization:

$$\square$$

Solver:

$$t$$

Risk:

$$t$$

Execution:

$$tt$$

Backtest:

$$-$$

Z.67. Dan versi pertama sebaiknya belum menggunakan live trading

Urutannya:

$$tt \ P \ tt \ t \ Pt$$

Bukan:

$$tt$$

Karena bahkan jika mathematical model benar, execution layer masih dapat mempunyai bug.

Z.68. Apa yang akan menjadi "core research question"?

Dengan blueprint ini, kita sekarang bisa mengajukan pertanyaan penelitian yang jauh lebih tajam:

Apakah quantum approximate optimization dapat menghasilkan solusi asset-selection yang kompetitif terhadap classical optimization pada short-horizon multi-asset trading dengan factor-conditioned risk, transaction cost, dan realistic execution constraints?

Kemudian subquestion:

Q1

Apakah QUBO menghasilkan solusi optimal/near-optimal?

Q2

Apakah VQE/QAOA mendekati solusi exact?

t

Q3

Apakah solusi tersebut menghasilkan performa finansial yang lebih baik?

$, , t$

Q4

Apakah keuntungan bertahan setelah:

Q5

Apakah model tetap robust ketika:

$t \quad tt$

Z.69. Ini juga membuat proyekmu jauh lebih dekat dengan quantitative research sebenarnya

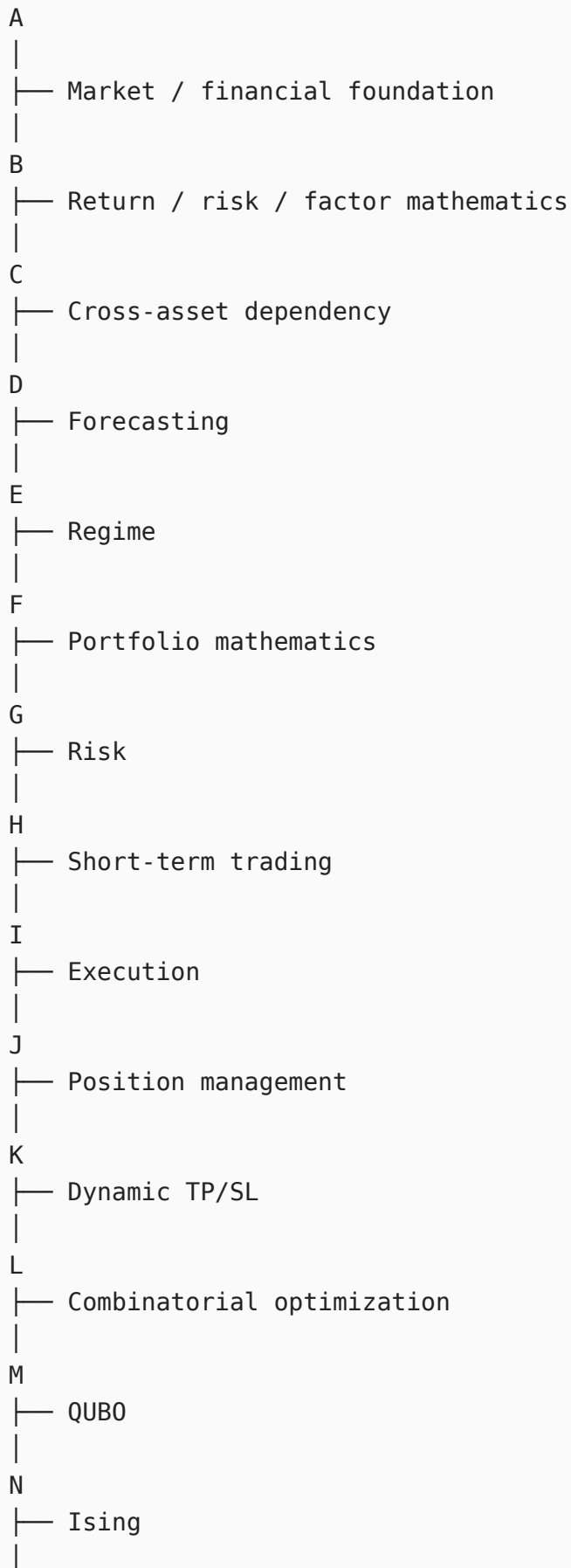
Karena kita tidak lagi memiliki:

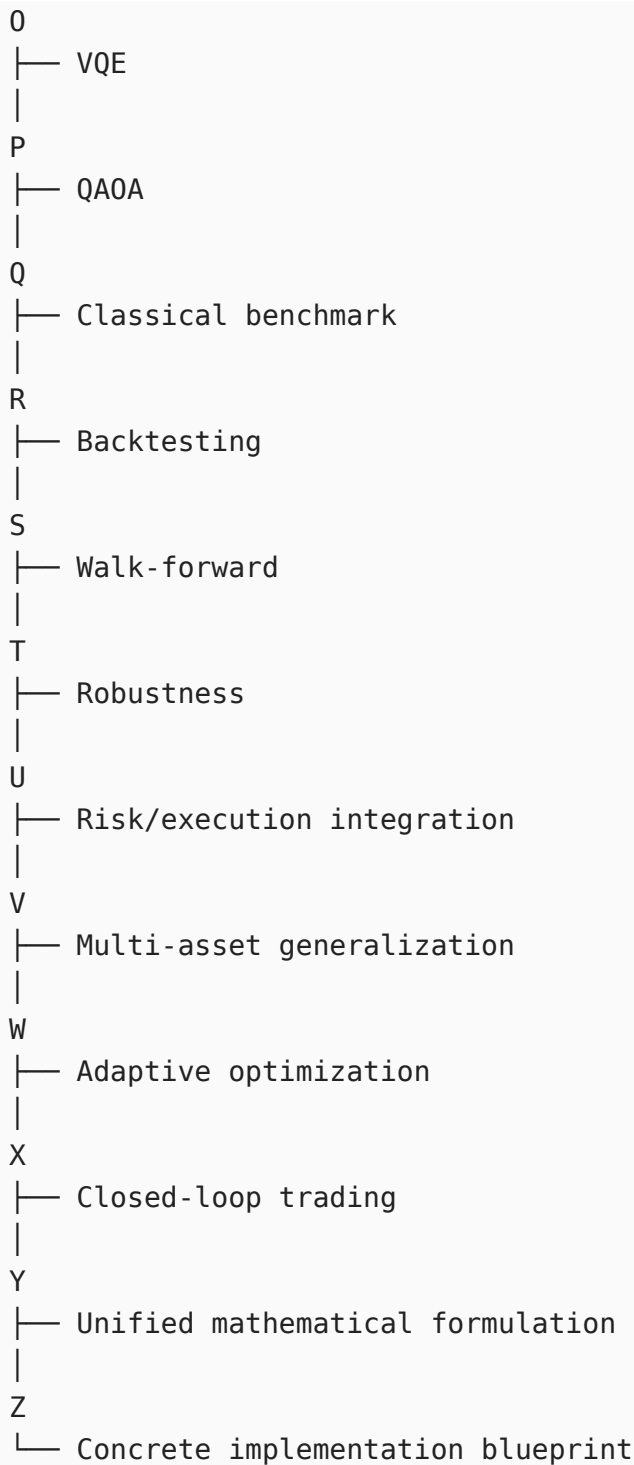
Melainkan:

Dan **itu** adalah perbedaan besar antara trading bot sederhana dengan quantitative decision system.

Z.70. Arsitektur final Tahap A–Z

Kalau seluruh perjalanan kita diringkas:





Dan sekarang kita mempunyai satu bentuk yang sangat jelas:

$$\boxed{t \quad t, t, t \quad t \quad t \quad t, t \quad t \quad t1}$$

Ini lah mathematical spine dari sistem yang kita bangun.

Langkah setelah Z

Menurut saya, setelah Z kita **jangan langsung membuat Tahap AA yang semakin abstrak**. Kita sudah cukup lama berada di level desain. Sekarang waktunya mulai mengubah blueprint menjadi **spesifikasi eksperimen yang benar-benar konkret**.

Jadi Tahap berikutnya yang paling masuk akal adalah:

AA — Definisi Dataset, Asset Universe, Timeframe, dan Walk-Forward Experiment

Di sana kita akan menentukan secara eksplisit, misalnya:

$$= \{, , , \}$$

$$= \{P, , , \}$$

$$= \{, , P, , 10, , , \}$$

lalu menentukan **tepat bagaimana data disinkronkan antara pasar crypto 24/7 dan saham yang hanya buka pada jam tertentu**, bagaimana missing data ditangani, bagaimana factor lag dibuat agar tidak terjadi look-ahead bias, bagaimana rolling window dihitung, serta bagaimana satu baris data pada t berubah menjadi satu QUBO t .

Dan menurut saya bagian sinkronisasi **crypto** ↔ **saham** ↔ **DXY** ↔ **Gold** ini justru sangat penting, karena di situlah desain kita akan mulai berhadapan dengan masalah nyata quantitative research, bukan sekadar matematika di atas kertas.

Prompt:

8/17/2026, 10:14:34 PM

lanjut Tahap AA

Response:

8/17/2026, 10:14:35 PM

Tahap AA — Dataset, Asset Universe, Timeframe, dan Walk-Forward Experiment

Kita sekarang masuk ke bagian yang menurutku **sangat penting**: mengubah mathematical spine pada Tahap Z menjadi **eksperimen yang bisa direproduksi**.

Kalau Tahap Z menjawab:

"Apa sistem matematis kita?"

maka Tahap AA menjawab:

"Data apa yang boleh dilihat model, kapan model boleh melihatnya, bagaimana data antar-pasar diselaraskan, dan bagaimana kita menguji model tanpa kebocoran informasi?"

Saya akan cukup ketat di sini, karena kesalahan pada tahap ini dapat membuat seluruh hasil VQE/QAOA terlihat bagus padahal sebenarnya terkena **look-ahead bias**.

AA.1. Prinsip dasar: waktu adalah dimensi utama

Misalkan pada waktu t , sistem membuat keputusan.

Kita definisikan informasi yang tersedia:

$$I_t = I_\tau \cup I_t$$

di mana:

- I_τ = seluruh data yang benar-benar tersedia pada waktu τ ,
- I_t = information set pada waktu t .

Model hanya boleh menghasilkan:

$$I_t = I_t$$

dan **tidak boleh** menggunakan:

$$I_{t+\Delta t}$$

Jadi:

$$I_t = I_t$$

Ini akan menjadi hukum utama backtest kita.

AA.2. Kita tidak membuat satu dataset

Sistem kita sebenarnya membutuhkan **empat jenis data**.

1. Market data

$$I_t$$

berisi OHLCV dan order book.

2. Factor data

$$I_t$$

berisi BTC, ETH, DXY, Gold, SPX, VIX, dan sebagainya.

3. Execution data

$$I_t$$

berisi spread, bid/ask, depth, fill, slippage.

4. Portfolio state

$$t$$

berisi posisi yang sedang terbuka, cash, exposure, entry, SL, TP.

Jadi:

$t = \{t, t, t, t\}$

AA.3. Asset universe

Saya ingin membagi universe menjadi tiga kelompok.

A. Target assets

Ini adalah asset yang boleh dipilih optimizer.

Misalnya crypto:

$$t = \{P, , , , \}$$

Untuk saham:

$$t = \{P, , , , \}$$

B. Market factors

Crypto:

$$t = \{, \}$$

Saham:

$$t = \{P, \}$$

C. Macro factors

$= \{, , , 10\}$

Sehingga sistem dapat mengetahui misalnya:

,

dan:

AA.4. Jangan mencampur crypto dan saham terlebih dahulu

Walaupun tujuan akhirnya adalah satu framework universal, eksperimen pertama sebaiknya:

tt

dan:

tt

secara terpisah.

Alasannya sederhana.

Crypto:

24

Saham:

t

Kalau langsung dicampur, kita akan memiliki masalah:

- timezone,
- market holiday,
- overnight gap,
- asynchronous information,
- different liquidity,
- different trading hours.

Jadi arsitektur sama, dataset berbeda.

AA.5. Timeframe utama

Saya menyarankan:

$t = 1$

Kenapa 1 jam?

Karena kita ingin short-term trading tetapi tidak terlalu mikro.

Jika menggunakan:

1

atau:

5,

market microstructure menjadi jauh lebih dominan.

Sementara:

1

masih cukup cepat untuk:

- intraday,
- swing pendek,
- dynamic TP/SL,
- regime adaptation.

AA.6. Multi-resolution architecture

Namun bukan berarti seluruh sistem hanya menggunakan 1h.

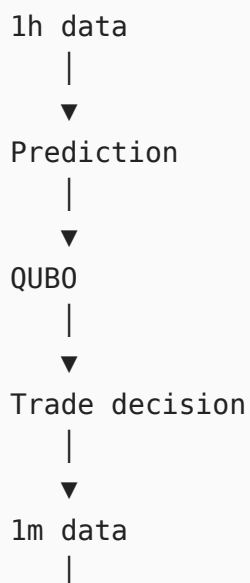
Kita gunakan:

$1 =$

dan:

$15 = t$

Misalnya:



AA.7. Mengapa ini penting?

Misalkan model menghasilkan:

$$P_t = 100$$

dan ingin membeli pada:

$$P_t = 5$$

Jika candle 1h memiliki:

$$t =$$

maka kita tidak boleh langsung mengatakan:

order pasti filled.

Karena harga bisa saja:

$$100 \quad 101$$

tanpa pernah tersedia cukup liquidity pada 5.

Karena itu:

$$\boxed{t \quad tt}$$

AA.8. Timestamp

Semua data harus mempunyai timestamp yang konsisten:

$$t_0, t_1, \dots, t$$

Saya menyarankan menggunakan:

□

sebagai internal canonical timezone.

Jangan menyimpan dataset internal dengan:

WIB
EST
UTC

secara campur-aduk.

Semua dikonversi ke UTC.

AA.9. Tetapi UTC bukan berarti semua pasar memiliki informasi simultan

Ini jebakan yang sangat penting.

Misalnya:

$$t = 14:00$$

Data:

- crypto tersedia,
- DXY tersedia,
- Gold tersedia,
- saham AS mungkin baru saja membuka,
- data saham sebelumnya mungkin berasal dari sesi sebelumnya.

Maka kita tidak boleh hanya melakukan:

$$tt$$

Kita harus melakukan **information-time alignment**.

AA.10. Market clock

Untuk setiap asset, kita definisikan:

sebagai himpunan timestamp ketika data asset tersebut tersedia.

Misalnya crypto:

$$= \{ \}$$

Saham:

$$P = \{ \}$$

AA.11. Last Observation Carried Forward

Untuk factor yang tidak update setiap jam, kita mungkin menggunakan:

$$t = \tau$$

dengan:

$$\tau = \{\tau \mid t \in \tau\}$$

Namun **LOCF tidak boleh digunakan secara buta.**

Kita juga harus menyimpan:

$$t = t - \tau$$

Jadi model mengetahui:

$$t, t$$

AA.12. Contoh DXY

Misalnya:

$$_{1000} = 1005$$

Kemudian tidak ada update sampai:

$$_{1100}$$

Untuk crypto pada:

$$_{10} \ 0,$$

kita boleh menggunakan:

$$_{1000}$$

jika memang data tersebut sudah tersedia.

Tetapi untuk:

$$_0 \ 0,$$

kita tidak boleh menggunakan:

$$_{1000}$$

AA.13. Publication lag

Ini lebih penting lagi untuk data makro.

Misalnya suatu economic indicator mempunyai:

$$_{tt} = t$$

tetapi baru dipublikasikan:

$$t \Delta$$

Model hanya boleh menggunakan setelah:

$$\boxed{t \Delta}$$

Secara formal:

$$t = t_{\tau} \tau \Delta t$$

AA.14. Ini menghasilkan konsep "as-of timestamp"

Setiap observasi sebaiknya mempunyai:

$$\boxed{tt}$$

dan:

$$\boxed{t}$$

Misalnya:

Data	Observation	Available
Gold	10:00	10:01
DXY	10:00	10:01
CPI	08:00	08:30
AAPL close	16:00	16:00+

Model menggunakan:

$$t$$

Bukan sekadar observation time.

AA.15. Data structure

Saya menyarankan setiap dataset memiliki kolom:

```
timestamp
available_at
symbol
open
```

```
high
low
close
volume
bid
ask
```

Untuk factor:

```
timestamp
available_at
factor
value
```

AA.16. Return target

Misalnya kita memprediksi return satu jam ke depan:

$$r_{t+1} = P_{t+1}P_t$$

Untuk horizon :

$$r_t = P_tP_t$$

Jadi:

$$= 1$$

berarti 1 jam.

AA.17. Jangan menggunakan target sebagai feature

Feature:

t

hanya menggunakan:

t

Target:

t

menggunakan:

t

Secara eksplisit:

$$t = t$$

dan:

$$t = t$$

AA.18. Feature matrix

Untuk asset :

$$,t = ,t,t-1,t-,t,t,t,t$$

Kemudian:

$$t = 1,t, ,t$$

AA.19. Rolling window

Misalnya:

$$= 1$$

Karena:

$$1 =$$

Maka covariance:

$$t = t-1t$$

Expected return:

$$t = t-1t$$

AA.20. Jangan selalu memakai satu window

Nanti kita uji:

$$\in \{4, 2, 1, , 20\}$$

Yaitu kira-kira:

- 2 hari,
- 3 hari,
- 1 minggu,
- 2 minggu,
- 1 bulan.

Ini menjadi hyperparameter.

AA.21. Tetapi hyperparameter tidak boleh dipilih dari test set

Misalnya kita mencoba:

$$= 4, 2, 1,$$

Kita pilih:

$$= 1$$

berdasarkan validation.

Kemudian:

$$= 1$$

dikunci sebelum test.

Kalau kita melihat test lalu memilih yang terbaik:

□

AA.22. Walk-forward experiment

Sekarang kita masuk ke inti AA.

Misalkan data:

2021 – 01 – 01 2025 – 12 – 1

Kita buat:

t t

AA.23. Expanding window

Contoh:

```
Train:      2021 ——— 2022
Validation:           2023 Q1
Test:              2023 Q2

Train:      2021 ——— 2023 Q1
Validation:           2023 Q2
Test:              2023 Q3

Train:      2021 ——— 2023 Q2
Validation:           2023 Q3
Test:              2023 Q4
```

Training semakin besar.

AA.24. Rolling window

Alternatif:

```
Train:      [———]
Validation:  [—]
Test:       [—]

Train:      [———]
Validation:  [—]
Test:       [—]
```

Training window memiliki panjang tetap.

AA.25. Mana yang kita gunakan?

Untuk sistem kita saya menyarankan:

Forecast model

Rolling window.

Karena market berubah.

Factor beta

Rolling/Exponentially weighted.

Regime

Rolling.

Long-term baseline

Expanding window sebagai pembanding.

Dengan demikian kita dapat membandingkan:

$tttt$

AA.26. Contoh satu fold

Misalkan:

$$t = 20$$

$$t = 1$$

$$tt = 1$$

Maka:

```
<----- 720h ----->
TRAIN
                        <---168h--->
                        VALIDATION
                                <---168h--->
                                TEST
```

Kemudian window bergeser.

AA.27. Decision timestamp

Pada:

t

kita menjalankan:

$t \ t \ tt_t \ t$

Order baru boleh dianggap aktif:

$t_t \ t$

AA.28. Order expiration

Order limit tidak boleh hidup selamanya.

Kita definisikan:

Jika:

$$t$$

dan order belum filled:

$$\square$$

Misalnya:

$$=$$

AA.29. Mengapa ini cocok dengan desain kita sebelumnya?

Karena Tahap Z mengatakan:

"setiap jam evaluasi apakah order masih layak."

Maka:

$$t_0$$

Jika belum filled:

$$t_1 - t$$

Jika:

$$0,$$

cancel.

AA.30. State machine

Order:

$$P$$

atau:

$$P$$

Position:

Kemudian:

$$P$$

AA.31. Data universe untuk crypto

Untuk eksperimen pertama, saya menyarankan:

Core factors

$$,$$

Macro

$$,$$

Risk sentiment

$$\square$$

Equity market

$$P$$

Jadi:

$$t = , , , , , P$$

AA.32. Target crypto

Jangan langsung mengambil semua altcoin.

Pilih misalnya:

$$, P , , , ,$$

sebagai contoh universe eksperimen.

BTC dan ETH menjadi factors, bukan target.

AA.33. Kenapa BTC dan ETH tidak sekaligus menjadi target?

Untuk eksperimen awal:

$$,$$

digunakan sebagai market leadership.

Kita ingin mengetahui:

□

Baru nanti kita dapat mengizinkan BTC/ETH menjadi selectable assets.

AA.34. Universe saham

Untuk eksperimen kedua:

$$P_{,,,,}$$

dan factors:

$$P_{,,,,,10}$$

Kita ingin melihat apakah:

dapat dimodelkan melalui:

AA.35. Sector factor

Untuk saham, kita bahkan bisa menambahkan:

,,,,

sebagai sector factors.

Sehingga:

$$t = P_{,,,,,10},,$$

Ini akan membuat factor model jauh lebih ekonomis.

AA.36. Cross-market lag

Ini bagian yang menarik.

Kita tidak hanya menggunakan:

t

Kita gunakan:

$t, t-1, t-2,$

Misalnya:

$$r_{t,t} = r_{t,t-1} + r_{t-2,t}$$

Karena pengaruh factor mungkin tidak instantaneous.

AA.37. Distributed lag model

Misalnya:

$$r_{t,t} = \alpha_0 + \alpha_1 r_{t-1} + \alpha_2 r_{t-2} + \dots$$

Ini sangat menarik secara finansial.

Karena kita dapat mengetahui apakah:

$$r_{t,t}$$

mempengaruhi altcoin:

$$r_{t-1}, r_{t-2}, \dots$$

bukan hanya correlation contemporaneous.

AA.38. Ini bisa menjadi sumber signal

Misalnya hasil estimasi:

$$\alpha_1 = 0$$

Artinya secara statistik:

$$r_{t,t}$$

mempunyai hubungan dengan:

$$r_{t-1}$$

Tetapi kita harus hati-hati membedakan:

$$r_{t,t}$$

dengan:

$$r_{t,t}$$

Kita tidak perlu menyebutnya causal tanpa pengujian yang tepat.

AA.39. Factor momentum

Kita juga dapat mendefinisikan:

$$r_{t,t-1} = \frac{r_t - r_{t-1}}{r_{t-1}}$$

Demikian:

$$r_{t,t-1}$$

Untuk DXY:

$$r_{t,t-1}$$

Gold:

$$r_{t,t-1}$$

AA.40. Relative factor flow

Kita bisa membangun:

$$r_{t,t-1} = \frac{r_t - r_{t-1}}{r_{t-1}}$$

Namun saya tidak ingin kita menentukan secara manual.

Kita biarkan:

$$r_{t,t-1}$$

di training set.

AA.41. Bahkan kita dapat menggunakan PCA

Misalkan:

$$r_t \in \mathbb{R}^n$$

Covariance:

Eigenproblem:

$$\lambda$$

Kemudian:

$$P = \frac{1}{\sqrt{n}} \begin{bmatrix} \sqrt{r_1} & \sqrt{r_2} & \dots & \sqrt{r_n} \end{bmatrix}$$

Ini dapat mengurangi dimensionality.

AA.42. Namun untuk versi pertama jangan gunakan PCA sebagai default

Kenapa?

Karena interpretasi:

lebih jelas daripada:

$$P_1$$

Kita ingin menjaga interpretability terlebih dahulu.

AA.43. Feature groups

Saya ingin setiap feature diberi label:

PRICE
VOLUME
VOLATILITY
FACTOR
MOMENTUM
REGIME
ORDERFLOW

Dengan demikian nanti kita dapat melakukan:

$$tt$$

AA.44. Ablation study

Misalnya:

Model A

$$P$$

Model B

$$P \ t$$

Model C

$$P \ t$$

Model D

$$P_t$$

Kemudian lihat:

$$\Delta$$

Ini sangat penting untuk mengetahui apakah kompleksitas benar-benar memberikan nilai.

AA.45. Dataset final

Secara konseptual satu baris:

$$t = t, t, t, P_{t,t1}$$

Tetapi ketika training:

$$P_{t_t}$$

harus berasal dari state sebelumnya.

Dan:

$$t_1$$

tidak boleh tersedia sebagai input.

AA.46. Train/validation/test harus benar-benar temporal

Bukan:

```
train_test_split(..., shuffle=True)
```

karena itu menghancurkan struktur waktu.

Kita harus:

$$t_t \quad t_t \quad t_{tt}$$

AA.47. Purging

Karena target kita menggunakan horizon:

Misalnya:

$$_t = P_{t1}P_t$$

Observasi di dekat boundary dapat overlap.

Maka kita perlu:



Misalnya:

TRAIN | purge | VALIDATION

untuk menghindari overlapping labels.

AA.48. Embargo

Selain purge, kita bisa menggunakan:



Artinya setelah validation/test boundary, kita memberikan gap tertentu sebelum data berikutnya digunakan.

Ini berguna terutama jika label overlap dan data sangat autocorrelated.

AA.49. Data leakage checklist

Sebelum model dianggap valid:

Leakage 1

Scaler menggunakan seluruh data.



Leakage 2

Covariance dihitung dari masa depan.



Leakage 3

Hyperparameter dipilih berdasarkan test.



Leakage 4

Order fill menggunakan future low/high tanpa execution model.



Leakage 5

Survivorship bias.



Leakage 6

Delisted stock dihilangkan.



Leakage 7

Corporate action tidak disesuaikan.



Leakage 8

Macro data menggunakan revision terakhir.



AA.50. Revision bias

Untuk data ekonomi, ada masalah:

$$t_t \quad t$$

Jika kita menggunakan dataset hari ini:

$$P_{2021}$$

yang telah mengalami revisi berkali-kali, kita mungkin memberikan model informasi yang pada 2021 belum diketahui.

Idealnya:

$$\boxed{ttt}$$

AA.51. Survivorship bias

Misalkan tahun 2021 kita memilih hanya saham yang masih ada pada 2026.

Maka:

2021

sudah mengandung informasi masa depan.

Salah.

Universe harus dibentuk berdasarkan:

t

AA.52. Delisting

Jika asset delisting:

$P_t = 0$

atau memiliki liquidation event, data tersebut tidak boleh sekadar dihapus.

Karena:

akan membuat performance terlihat terlalu bagus.

AA.53. Crypto memiliki masalah berbeda

Crypto tidak terlalu terkena:

t

Tetapi memiliki:

- listing/delisting,
- exchange-specific liquidity,
- perpetual funding,
- different trading pairs,
- symbol changes.

Jadi universe juga harus memiliki:

t_t

dan:

t_t

AA.54. Exchange-specific data

Ini sangat penting untuk sistem live nanti.

Harga:

$$P \ P$$

secara tepat pada setiap waktu.

Maka jangan menggabungkan:

```
price dari exchange A
order book exchange B
volume exchange C
```

tanpa model yang jelas.

AA.55. Untuk backtest awal

Saya menyarankan:

□

Misalnya satu exchange untuk crypto.

Kemudian:

$$P_{t, t, t}$$

berasal dari venue yang sama.

AA.56. Untuk saham

Gunakan satu sumber consolidated market data untuk baseline.

Kemudian baru nanti kita tambahkan:

ttt

AA.57. Data frequency

Untuk decision:

Untuk execution:

$$1$$

Untuk order book:

$$tt$$

jika tersedia.

Jadi hierarchy:

1

1

AA.58. Aggregation

Misalnya data 1m:

$$\{P_{t,1}, P_{t,2}, \dots, P_{t,0}\}$$

menjadi:

$$t = P_{t,1}$$

$$t = P_{t,2}$$

$$t = P_{t,3}$$

$$t = P_{t,0}$$

Volume:

$$t = t_{t,0}$$

AA.59. Order book features

Jika tersedia:

Bid:

$$t$$

Ask:

$$t$$

Spread:

$t = t - t$

Mid-price:

$$t = t^2$$

AA.60. Order book imbalance

Dengan depth :

$$t = -1, -1, -1, -1,$$

Sehingga:

$$-1 \quad t \quad 1$$

Ini dapat menjadi feature:

$$t$$

AA.61. Tetapi jangan memasukkan order book ke model 1h secara mentah

Order book sangat noisy.

Lebih baik agregasikan:

$$1t = t, \tau$$

atau:

$$t, \tau$$

Bisa juga menggunakan:

$$t$$

AA.62. Liquidity feature

Misalnya:

$$t = 1 \quad t_{\tau=t-\tau}$$

Kemudian:

$$t = , , t$$

AA.63. Optimizer harus melihat liquidity

Kita dapat menambahkan:

$$= 0$$

jika:

$$t$$

Atau penalty:

$$P \ 1 -$$

AA.64. Slippage model

Versi pertama:

$$= 2 \ t$$

Kemudian:

$$t =$$

Nanti kita dapat mengganti dengan model empiris.

AA.65. Ini masuk ke QUBO

Jika asset memiliki expected net return:

$$t = - t$$

Maka QUBO menggunakan:

$$\boxed{-t}$$

Jadi optimizer tidak mengoptimalkan:

$$t$$

tetapi:

$$\boxed{ttt}$$

AA.66. Bahkan covariance dapat disesuaikan

Jika execution uncertainty besar:

$$t = t \ t$$

Kemudian:

$$= -t$$

Ini lebih konservatif.

AA.67. Struktur satu decision cycle

Sekarang semuanya bisa kita satukan.

Pada:

$$t$$

Step 1

Ambil:

$$t$$

Step 2

Bangun:

$$t$$

Step 3

Update factor:

$$t$$

Step 4

Estimate regime:

$$t$$

Step 5

Estimate:

$$t, t$$

Step 6

Build:

$$t$$

Step 7

Solve:

$$t$$

Step 8

Allocation:

t

Step 9

Risk sizing:

t

Step 10

Generate:

t

Step 11

Execute.

Step 12

Update:

tt_{t1}

AA.68. Formal recursive system

Kita bisa menuliskannya:

$$t1 = t, t, t$$

dengan action:

$$t = t$$

Policy kita sebenarnya adalah:

$$= t \quad t \quad tt \quad t$$

Ini mulai terlihat seperti **closed-loop stochastic control system**.

AA.69. Dan di sinilah desain kita menjadi menarik

Sistem kita bukan:

Pt

Tetapi:

$$\begin{bmatrix} P_t & t t & t & t t & P_t \end{bmatrix}$$

Jadi keputusan hari ini mengubah:

$$t1,$$

yang kemudian mempengaruhi keputusan berikutnya.

AA.70. Eksperimen pertama yang saya rekomendasikan

Kita jangan langsung membuat model besar.

Saya ingin **Experiment AA-1** menjadi sangat sederhana tetapi scientifically clean.

Universe

$$\begin{bmatrix} ,P, , , , \end{bmatrix}$$

Factors

$$\begin{bmatrix} , , , , P, \end{bmatrix}$$

Timeframe

$$\begin{bmatrix} 1 \end{bmatrix}$$

Forecast horizon

$$\begin{bmatrix} = 1 \end{bmatrix}$$

Rolling window

$$\begin{bmatrix} = 1 \end{bmatrix}$$

Regime

3-state.

Optimizer

Exact QUBO.

Execution

1m historical simulation.

Risk

Fixed:

$$t = 1$$

AA.71. Mengapa mulai dari Exact QUBO?

Karena kita ingin memisahkan dua pertanyaan.

Pertanyaan pertama:

Apakah mathematical trading problem kita sendiri menghasilkan keputusan yang bagus?

Kita jawab menggunakan:

$$t$$

Pertanyaan kedua:

Apakah VQE/QAOA mampu menyelesaikan problem tersebut?

Baru kemudian:

$$t$$

Ini sangat penting untuk penelitian quantum.

AA.72. Experiment AA-2

Setelah AA-1:

$$t$$

kita lakukan:

Misalnya:

$$t$$

atau solver classical lainnya.

AA.73. Experiment AA-3

Kemudian:

Ukur:

terhadap:

AA.74. Experiment AA-4

Kemudian:

Ukur:

AA.75. Experiment AA-5

Baru kita masukkan:

tt

Bandingkan:

vs:

t

AA.76. Experiment AA-6

Kemudian:

t

vs:

t

Framework yang sama:

$\square$

AA.77. Experiment AA-7

Kemudian stress test:

tt

t

t

Kita ingin melihat:

P

AA.78. Experiment AA-8

Terakhir:

t

Bandingkan:

P

VS:

$P \ t$

VS:

$P \ t$

VS:

$P \ t$

AA.79. Output yang harus kita simpan

Setiap decision timestamp harus menghasilkan log:

```
timestamp
regime
mu
covariance
qubo_matrix
ising_h
ising_J
solver
solution
objective
feasibility
weights
quantity
entry
stop_loss
```

```
take_profit
expected_return
expected_risk
spread
slippage
fill_status
realized_pnl
```

Ini penting.

Karena nanti kita bisa menjawab:

"Mengapa trade pada 2026-05-13 pukul 10:00 dilakukan?"

Secara matematis.

AA.80. Kita bahkan bisa merekonstruksi setiap keputusan

Misalnya:

$$t = 10\ 00$$

Model melihat:

$$= -0$$

$$= -05$$

$$= 0$$

$$= 02$$

dan:

$$= t$$

Kemudian:

$$= -01$$

$$p = 04$$

Optimizer kemudian memilih:

$$= 0, 1, 0, 1, 0, 0$$

Kita dapat mengetahui **seluruh rantai sebab-akibat komputasionalnya**.

AA.81. Dan inilah yang nantinya sangat berguna untuk skripsi/riset

Kita bisa menunjukkan:

$$tt \quad tt \quad tt$$

Bukan hanya:

"VQE menghasilkan portfolio."

AA.82. Spesifikasi AA final

Jadi untuk saat ini saya ingin kita **mengunci** konfigurasi baseline:

$$1 \quad 1 \quad , P, , , , \quad , \quad , , P, \quad 1 \quad 1 \quad 1 \quad t \quad - \quad t$$

Ini **bukan berarti angka-angka tersebut final selamanya**. Mereka adalah baseline yang kita gunakan agar eksperimen pertama memiliki definisi yang jelas.

AA.83. Hal yang paling penting dari Tahap AA

Ada satu perubahan konseptual yang ingin saya tekankan.

Sebelumnya kita berpikir:

$$t$$

Sekarang kita harus berpikir:

$$t \quad tt \quad t \quad t$$

Dengan kata lain, **timestamp adalah bagian dari matematika model**, bukan sekadar metadata dataset.

Dan untuk sistem quantitative trading yang ingin kita bangun, itu mungkin sama pentingnya dengan QUBO, VQE, atau QAOA itu sendiri.

Tahap berikutnya: AB

Setelah AA, saya menyarankan kita masuk ke:

Tahap AB — Mathematical Feature & Factor Engine

Di sana kita mulai benar-benar **menurunkan setiap feature dari data mentah**, bukan sekedar menyebut nama indikator.

Kita akan membangun secara matematis:

$$t \quad tt \quad t \quad t \quad t \quad tt$$

lalu menurunkan **factor model untuk crypto dan saham secara terpadu**, termasuk:

$$,t1 = t_{=1} \neq ,t1,$$

estimasi rolling, covariance factor model, shrinkage, cross-asset lag, dan akhirnya bagaimana semua itu masuk ke:

$$t, t$$

yang kemudian menjadi input langsung ke **QUBO/Ising** kita.

Prompt:

8/17/2026, 10:15:55 PM

lanjut Tahap AB

Response:

8/17/2026, 10:15:56 PM

Remembering

Recalled trading architecture, decisions, risk rules, and Stage AB focus

Worked for 6s

Tahap AB — Mathematical Feature & Factor Engine

Baik. Pada **Tahap AA** kita sudah mengunci arsitektur data:

$$t \quad t, t$$

Maka Tahap AB adalah tempat kita membangun bagian yang selama ini masih berupa kotak hitam:

$$t, t$$

Dan saya ingin kita membuatnya **secara matematis**, bukan sekedar memasukkan indikator teknikal seperti RSI, MACD, dan sebagainya.

AB.1. Tujuan Mathematical Feature Engine

Pada setiap waktu keputusan t , sistem memiliki informasi:

$$t$$

Dari informasi tersebut kita ingin membentuk feature vector:

$$,t = t$$

untuk setiap asset .

Kemudian model forecast menghasilkan:

$$,t = ,t1 \ t$$

dan covariance:

$$t = t1 \ t$$

Jadi sebenarnya ada **dua mesin berbeda**:

$$\square$$

dan:

$$\square$$

Keduanya akhirnya bertemu di optimizer.

AB.2. Return sebagai variabel fundamental

Kita mulai dari harga:

$$P_{,t}$$

Saya ingin menggunakan log-return:

$$,t = P_{,t} P_{,t-1}$$

karena memiliki sifat:

$$,t = P_{,t} P_{,t-} = -1_{=0},t-$$

Ini sangat berguna untuk multi-horizon model.

AB.3. Return vector

Untuk asset:

$$t = 1, 2, \dots, t$$

Untuk baseline kita:

$$=$$

dengan:

$$= \{P, \dots\}$$

Maka:

$$t \in$$

AB.4. Multi-horizon return

Satu return saja terlalu miskin.

Kita definisikan:

$$r_{t,t-1} = \frac{P_t}{P_{t-1}} - 1$$

Kita dapat mengambil:

$$h \in \{1, 3, 6, 12, 24, 2, 1\}$$

Karena decision timeframe kita adalah 1 jam, maka:

- $h = 1$: 1 jam,
- $h = 3$: 3 jam,
- $h = 6$: 6 jam,
- $h = 12$: 12 jam,
- $h = 24$: 1 hari,
- $h = 2$: 3 hari,
- $h = 1$: 1 minggu.

AB.5. Momentum

Momentum sederhana:

$$r_{t,t-1} = \frac{P_t}{P_{t-1}} - 1$$

Secara matematis sebenarnya sama dengan cumulative log-return.

Tetapi nanti kita dapat menggunakan beberapa transformasi:

$$r_{i,t} = P_{i,t} P_{i,t-1}$$

AB.6. Normalized momentum

Momentum absolut tidak dapat dibandingkan dengan baik antar-asset.

Maka kita normalisasi dengan volatility:

$$\tilde{r}_{i,t} = \frac{r_{i,t}}{\sigma_{i,t}}$$

Dengan:

$$\sigma_{i,t} = \sqrt{\frac{1}{n} \sum_{j=1}^n r_{i,t-j}^2}$$

untuk mencegah pembagian nol.

Sekarang $\tilde{r}_{i,t}$ dapat dibandingkan antar asset.

AB.7. Volatility

Volatility rolling:

$$\sigma_{i,t}^2 = \frac{1}{n} \sum_{j=1}^n r_{i,t-j}^2$$

Baseline kita:

$$\sigma_{i,t}^2 = 1$$

Jadi sistem mengetahui volatility satu minggu terakhir.

AB.8. EWMA volatility

Volatility rolling memiliki kelemahan: semua observasi mendapat bobot sama.

Kita dapat menggunakan EWMA:

$$\sigma_{i,t}^2 = \lambda r_{i,t}^2 + (1 - \lambda) \sigma_{i,t-1}^2$$

Dengan:

$$0 \quad 1$$

Semakin besar , semakin lambat model bereaksi terhadap perubahan volatility.

AB.9. Mengapa volatility penting?

Karena:

$$= 1$$

tidak berarti sama jika:

$$= 0.5$$

dibanding:

$$= 5$$

Maka signal harus dipandang sebagai:

□

AB.10. Z-score return

Kita dapat membuat:

$$r_{i,t} = r_{i,t} - r_{m,t}$$

Ini memberi informasi apakah return saat ini abnormal relatif terhadap regime lokal.

AB.11. Volume

Volume mentah juga tidak comparable.

Gunakan:

$$r_{i,t} = r_{i,t} - r_{m,t}$$

Ini memberi:

| Apakah pergerakan harga sekarang terjadi dengan volume abnormal?

AB.12. Dollar volume

Untuk likuiditas:

$$_{,t} = P_{,tt}$$

Kemudian:

$$_{,t} = 1 - 1_{=0,t-}$$

Ini akan digunakan juga pada execution/risk engine.

AB.13. High-low range

Kita dapat mendefinisikan:

$$,t \stackrel{=}{=} ,t \stackrel{-}{=} ,t,t$$

Kemudian rolling average:

$$,t = 1 - 1_{=0},t -$$

Ini merupakan proxy volatility intrabar.

AB.14. Sekarang masuk ke bagian yang jauh lebih penting: cross-asset factors

Misalkan target kita adalah SOL.

Kita tidak hanya melihat:

$, t$

Kita melihat:

$$t = ,t,t,t,tP,t,t$$

Dengan:

AB.15. Factor model

Model dasar:

$$t = t t$$

dengan:

$$\in$$

Elemen:

mengukur sensitivitas asset terhadap factor .

AB.16. Contoh interpretasi

Misalkan:

$$, = 14$$

Secara sederhana:

$$\Delta 14\Delta$$

untuk komponen pergerakan yang dijelaskan oleh factor tersebut.

Sedangkan:

$$, = -0$$

Maka kenaikan DXY secara statistik berkaitan dengan tekanan terhadap SOL.

Ini bukan klaim kausalitas.

Kita hanya mengatakan:

AB.17. Estimasi beta

Untuk setiap asset:

$$=$$

OLS:

$$=^{-1}$$

Namun kita tidak akan menghitungnya menggunakan seluruh history.

AB.18. Rolling beta

Pada waktu t :

$$\beta_t = \frac{\sum_{\tau=t-1}^t (r_{t-\tau} - \bar{r})^2}{\sum_{\tau=t-1}^t (r_{t-\tau} - \bar{r})^2}$$

Jadi beta berubah:

$$\beta_t \neq \beta_{t-1}$$

Ini jauh lebih masuk akal untuk short-term trading.

AB.19. Mengapa beta harus berubah?

Karena hubungan pasar dapat berubah.

Misalnya:

Regime 1

$$\beta_t = 12$$

Regime 2

$$\beta_t = 20$$

Jika kita menggunakan beta jangka panjang:

$$\beta_t = 12$$

maka model mungkin underestimate risiko SOL ketika BTC menjadi dominant driver.

AB.20. Weighted regression

Kita dapat memberikan bobot lebih besar pada data terbaru:

$$\beta_t = \frac{\sum_{\tau=t-1}^t (r_{t-\tau} - \bar{r})^2}{\sum_{\tau=t-1}^t (r_{t-\tau} - \bar{r})^2}$$

Misalnya:

$$=$$

Ini menghasilkan **exponentially weighted factor model**.

AB.21. Residual return

Setelah factor exposure diketahui:

$$r_{i,t} = \alpha_i + \beta_1 r_{m,t} + \beta_2 r_{f,t} + \epsilon_{i,t}$$

Ini sangat penting.

Karena kita sekarang dapat memisahkan:

$$r_{i,t} = \alpha_i + \beta_1 r_{m,t} + \beta_2 r_{f,t} + \epsilon_{i,t}$$

AB.22. Residual momentum

Kita bahkan dapat menghitung momentum residual:

$$r_{i,t} - \alpha_i = -1 \cdot r_{i,t-1} + r_{i,t}$$

Ini jauh lebih menarik daripada momentum harga biasa.

Misalnya:

naik karena:

naik.

Itu belum tentu merupakan alpha.

Tetapi jika:

mengungguli apa yang dijelaskan oleh BTC/ETH/macro factors:

$$0,$$

maka terdapat **relative strength** yang lebih spesifik terhadap SOL.

AB.23. Ini menghasilkan dua jenis momentum

Market momentum

$$r_{m,t}$$

dan:

Residual momentum

$$r_{i,t}$$

Kemudian feature vector dapat memiliki:

$$x_t = [x_t, x_{t-1}, x_{t-2}, \dots]$$

AB.24. Cross-asset lag

Sekarang kita masuk ke ide yang sebelumnya sudah kita tetapkan.

Model:

$$x_{t+1} = \alpha_0 + \alpha_1 x_t + \alpha_2 x_{t-1} + \dots$$

Misalnya:

$$x_{t+1} = \alpha_0 + \alpha_1 x_t + \alpha_2 x_{t-1} + \alpha_3 x_{t-2} + \dots$$

AB.25. Mengapa ini penting?

Bayangkan:

bergerak lebih dahulu.

Kemudian:

bereaksi beberapa jam kemudian.

Jika pola tersebut stabil, maka:

t

bisa menjadi **leading factor**.

Model kita tidak lagi sekadar bertanya:

"Apa hubungan BTC dan SOL?"

tetapi:

"Apakah informasi BTC sekarang mengandung informasi tentang return SOL berikutnya?"

AB.26. Distributed-lag factor matrix

Kita dapat mendefinisikan:

$$t = t - \lambda t$$

Kemudian:

$$t_1 = t, t_1$$

AB.27. Jangan membuat terlalu besar

Jika:

$$= 24$$

dan factor:

,

maka:

$$25 = 150$$

coefficient per asset.

Dengan hanya:

$$1$$

observasi, kita mengalami masalah:

$$\square$$

Maka estimasi menjadi tidak stabil.

AB.28. Regularisasi ridge

Kita dapat menggunakan:

$$= - \frac{2}{22}$$

Solusinya:

$$= -1$$

Ini sangat cocok untuk factor model kita.

AB.29. Mengapa ridge lebih cocok daripada langsung memperbanyak feature?

Karena kita ingin:

bukan:

Dalam trading:

$$2t$$

yang tinggi tidak berarti:

$$P$$

tinggi.

AB.30. Factor interaction

Kita bahkan dapat memiliki:

Misalnya:

$$t = ,t,t$$

Kemudian:

$$,t1 = 1t \times t$$

Ini memungkinkan hubungan non-additive.

AB.31. Tetapi jangan terlalu cepat memasukkan interaction

Karena jumlah parameter bertambah.

Kita gunakan prinsip:

$$\boxed{t}$$

Setiap feature tambahan harus melewati:

—

dan:

$$t$$

AB.32. Factor covariance

Sekarang bagian kedua yang sangat penting:

$$t$$

Dari:

$$t = t_t, t,$$

kita mendapatkan:

$$t = t_t, t_t, t$$

Ini adalah **factor covariance model**.

AB.33. Mengapa tidak langsung menghitung covariance asset?

Kita bisa:

$$=$$

Tetapi jika:

besar dan:

relatif kecil, covariance sample menjadi noisy.

Factor model memberikan struktur:

$$\square$$

AB.34. Factor covariance

Definisikan:

$$,t = t$$

Rolling estimate:

$$,t = t-1t$$

Bisa juga EWMA.

AB.35. Residual covariance

Residual:

$$t = t - tt$$

Jika kita mengasumsikan residual relatif independent antar asset:

$$\begin{bmatrix} t & 2,1 & 2, \\ t & 2,1 & 2, \end{bmatrix}$$

Ini adalah **diagonal factor model**.

AB.36. Mengapa asumsi diagonal menarik?

Karena:

$$- 12$$

off-diagonal covariance harus diestimasi jika menggunakan covariance penuh.

Untuk:

$$= 100,$$

itu:

$$450$$

parameter.

Factor model dapat mengurangi kompleksitas secara drastis.

AB.37. Tetapi residual sebenarnya bisa berkorelasi

Misalnya:

dan:

masih berkorelasi.

Maka nanti kita dapat menggunakan:

$$=$$

Tetapi baseline kita gunakan:

$$\begin{bmatrix} = \\ = \end{bmatrix}$$

AB.38. Shrinkage covariance

Kita juga dapat melakukan shrinkage:

$$= 1 -$$

dengan target covariance.

Contoh:

$$=$$

Ini mengurangi estimation error.

AB.39. EWMA covariance

Alternatif:

$$_t = {}_{t-1} 1 - {}_{t-1} {}_{t-1}$$

Secara online:

$$_t$$

selalu diperbarui tanpa menghitung ulang seluruh history.

AB.40. Kita sebenarnya akan menggunakan kombinasi

Untuk baseline:

$$_t = {}_t, {}_t, {}_t$$

Kemudian:

$$, {}_t$$

dapat menggunakan EWMA.

Dan:

$$, {}_t$$

juga dapat menggunakan EWMA residual volatility.

AB.41. Sekarang expected return

Ini bagian yang akan menentukan:

$$t$$

Model paling sederhana:

$$t = t + t t_1$$

Masalahnya:

kita tidak mengetahui t_1 .

Maka kita perlu memprediksi factor.

AB.42. Factor forecast

Misalnya:

$$t_1 = t_1 + t$$

Kita dapat menggunakan model sederhana:

$$t_1 = t$$

atau:

$$t_1 = \sum_{t=0}^{\infty} t$$

AB.43. Namun ada pendekatan yang lebih konservatif

Untuk short-term trading, kita tidak harus mencoba memprediksi seluruh market.

Kita dapat memisahkan:

$$= t$$

Ini lebih fleksibel.

AB.44. Baseline component

Misalnya:

$$=$$

Tetapi intercept saja biasanya terlalu lemah.

Maka kita dapat memasukkan momentum.

AB.45. Momentum component

$$= 11,t, t \ 2424,t$$

AB.46. Factor component

$$_t = ,t \ t1$$

AB.47. Residual component

Residual return dapat memiliki autocorrelation:

$$,t = ,t-1 ,t$$

Maka:

$$,t1 = ,t$$

AB.48. Gabungan

Kita memperoleh:

$$,t = ,t \ t1 ,t ,t$$

Ini menjadi baseline forecast engine kita.

AB.49. Tapi ada masalah penting

Kalau kita memasukkan terlalu banyak model:

$$= t \ t \ tt$$

maka kita dapat mengalami:

$$tt$$

Karena itu semua coefficient harus dipelajari **hanya dari training window**.

AB.50. Regime conditioning

Kita sebelumnya telah menetapkan 3-state regime.

Misalkan:

$$t \in \{1, 2, \}$$

Contoh:

$$1 =$$

$$2 =$$

$$=$$

Maka:

$$,t = t$$

Dan:

$$t = t$$

AB.51. Mengapa regime mempengaruhi covariance?

Karena:

$$t \ t = 1 \ t \ t =$$

Saat stress:

dan sering:

Jadi diversification dapat menjadi lebih lemah saat crash.

Ini sangat penting bagi optimizer.

AB.52. Regime-conditioned factor beta

Kita bahkan dapat mendefinisikan:

$$\boxed{\gamma}$$

sehingga:

$$,t =_t t^t ,t$$

Tetapi untuk baseline kita **tidak langsung melakukan ini**, karena parameter akan terlalu banyak.

Kita mulai dengan:

$$t$$

rolling dan:

$$t$$

sebagai feature.

AB.53. Feature vector final

Untuk asset , baseline feature vector:

$$,t = 1\ 241124,,,,,P,,tt$$

Ini sudah cukup kaya.

AB.54. Tetapi optimizer tidak menerima feature vector

Ini perbedaan penting.

Feature vector:

$$,t$$

masuk ke:

$$t$$

Outputnya:

$$, t$$

Sementara covariance engine menghasilkan:

$$t$$

Barulah:

$$t, t$$

dikirim ke optimizer.

AB.55. Pipeline matematis lengkap

Sekarang:

$$P, , ,$$

menjadi:

$$, , , ,$$

menjadi:

$$t, t, ,$$

menjadi:

$$t, t$$

menjadi:

$$t$$

menjadi:

$$t$$

AB.56. Dari , ke QUBO

Sekarang kita hubungkan langsung dengan penelitian quantum kita.

Misalkan binary:

$$\in \{0, 1\}$$

Objective:

$$-t \ t$$

Jika:

berisi constraint/penalty.

AB.57. Ekspansi quadratic

Karena:

$$= 2 \ ,$$

maka:

$$=$$

Dengan koefisien berasal dari:

$$t \ t$$

Jadi feature engine secara langsung menentukan Hamiltonian.

AB.58. Konsekuensi yang sangat penting

Artinya quantum optimizer **tidak perlu mengetahui harga**.

VQE/QAOA hanya menerima:

Semua financial intelligence terjadi sebelumnya:

$$t \ ttt \ t$$

Kemudian quantum algorithm melakukan:

$$t \ t$$

Ini adalah pemisahan arsitektur yang sangat bersih.

AB.59. Ini juga membuat perbandingan quantum menjadi fair

Exact solver:

Classical heuristic:

VQE:

QAOA:

Semuanya menerima:

$$t, t$$

Jadi kalau performanya berbeda, kita dapat mengatribusikan perbedaan kepada **optimizer**, bukan data pipeline.

AB.60. Sekarang kita dapat memperkenalkan "alpha"

Kita tidak boleh menganggap:

sebagai return mentah.

Lebih baik:

$$,t = ,t - ,t$$

dengan:

$$,t =$$

untuk crypto/perpetual bila relevan.

Sehingga optimizer bekerja pada:

$$ttt$$

AB.61. Risk-adjusted alpha

Kita dapat menambahkan:

$$\hat{\mu}_{t,t} = \hat{\mu}_{t,t}$$

Tetapi saya tidak menyarankan mengganti dengan secara langsung.

Lebih baik optimizer tetap menggunakan:

$$\hat{\mu}_{t,t}$$

karena covariance sudah menangani risk secara eksplisit.

AB.62. Signal confidence

Kita juga dapat mengukur confidence:

$$\hat{\mu}_{t,t} = \hat{\mu}_{t,t}$$

Ini dapat digunakan sebagai filter:

$$\hat{\mu}_{t,t} = 0$$

Namun ini sebaiknya menjadi eksperimen terpisah.

AB.63. Prediction uncertainty

Lebih formal:

$$2$$

Jika interval:

mencakup nol, kita dapat menganggap signal lemah.

Ini akan sangat berguna untuk **trade selection**.

AB.64. Jangan membuang signal negatif secara otomatis

Ini penting.

Misalkan:

$$= -0.5$$

Dalam long-only portfolio, asset tersebut mungkin tidak dipilih.

Tetapi dalam sistem long-short:

dapat berarti:

$$= 1$$

untuk posisi short.

Jadi architecture kita sebaiknya dari awal tidak terlalu mengunci:

—

AB.65. Binary encoding untuk long-short

Kita dapat menggunakan dua binary variable:

dan:

—

Dengan constraint:

$$\boxed{\begin{matrix} - & 1 \end{matrix}}$$

Exposure:

$$\boxed{= \quad - \quad -}$$

Sehingga:

$$\in \{-1, 0, 1\}$$

Ini jauh lebih cocok untuk trading.

AB.66. Ini akan menjadi penting pada tahap QUBO berikutnya

Karena:

$$= \quad - \quad -,$$

maka portfolio objective:

—

Setelah substitusi:

$$= \quad - \quad -,$$

kita memperoleh QUBO binary.

Ini akan membuat sistem kita mampu memilih:

atau:

$$t$$

atau:

$$t$$

AB.67. Feature engine untuk saham

Untuk saham internasional, kita tinggal memperluas factor vector:

$$_t = P10$$

Kemudian:

$$_{,t} = t_{,t}$$

Dengan tambahan sector factor jika diperlukan.

AB.68. Mengapa Gold dan DXY sangat menarik?

Karena kita tidak ingin hanya melihat:

$$P$$

Kita ingin melihat kondisi cross-market.

Misalnya:

dapat merepresentasikan kondisi USD yang menguat.

Sementara:

dapat membawa informasi mengenai demand terhadap safe-haven/real-rate/inflation-related dynamics.

Tetapi kita **tidak akan hard-code** interpretasi seperti:

$$t$$

Model harus belajar:

$$r_t$$

AB.69. Factor exposure sebagai feature dinamis

Maka untuk setiap saham:

$$r_{i,t} = \alpha_i + \beta_{i,1}r_{1,t} + \beta_{i,2}r_{2,t} + \beta_{i,3}r_{3,t} + \beta_{i,4}r_{4,t} + \beta_{i,5}r_{5,t} + \beta_{i,6}r_{6,t} + \beta_{i,7}r_{7,t} + \beta_{i,8}r_{8,t} + \beta_{i,9}r_{9,t} + \beta_{i,10}r_{10,t} + \epsilon_{i,t}$$

Ini sangat menarik karena optimizer nantinya dapat memilih asset berdasarkan:

AB.70. Portfolio-level factor risk

Setelah optimizer menghasilkan:

$$w$$

factor exposure portfolio:

$$E_t$$

Misalnya kita ingin membatasi:

$$w$$

Maka constraint dapat dimasukkan ke QUBO.

Ini berarti optimizer bukan hanya memilih:

"asset mana yang profit-nya paling besar?"

tetapi:

"kombinasi asset mana yang menghasilkan expected return tinggi dengan exposure faktor yang masih dapat diterima?"

AB.71. Ini mulai mendekati problem institutional portfolio construction

Karena kita memiliki:

$$r_t$$

dengan:

dan:

t

dan:

tt

dan:

t

dan:

Pttt

Kemudian seluruhnya dipetakan menjadi:

AB.72. Namun jangan memasukkan semuanya sekaligus

Kita perlu menjaga eksperimen bertingkat.

Saya ingin baseline AB menjadi:

Kemudian:

AB-1

Price only.

AB-2

Price + momentum.

AB-3

Price + momentum + volatility.

AB-4

- BTC/ETH.

AB-5

- DXY/Gold/SPX/VIX.

AB-6

- residual momentum.

AB-7

- order flow.

Dengan demikian kita bisa mengukur:

$$\Delta P$$

dari setiap kelompok feature.

AB.73. Feature leakage test

Setiap feature harus memenuhi:

$$P_{t,t} = P_{t,t}$$

Contoh yang benar:

$$P_{24,t} = P_{t,t-24}$$

Contoh yang salah:

$$P_{24,t} = P_{t,t-2}$$

Yang kedua sudah melihat masa depan.

AB.74. Feature normalization

Jika model machine learning digunakan, normalisasi juga harus rolling/training-only.

Misalnya:

$$P_{t,t} = P_{t,t} -$$

Tetapi:

$$,$$

harus dihitung hanya dari training data.

Bukan:

$$2021 - 202$$

secara keseluruhan.

AB.75. Feature stationarity

Kita harus membedakan:

P

dengan:

t

Harga biasanya non-stationary.

Return lebih dekat ke stationary process.

Karena itu model utama sebaiknya bekerja dengan:

t, t, t

daripada harga mentah.

AB.76. Feature covariance

Feature sendiri juga dapat berkorelasi.

Misalnya:

t

dan:

t

sangat correlated.

Jika terlalu banyak feature serupa, model dapat menjadi unstable.

Ridge regularization akan membantu.

AB.77. Feature selection

Kita dapat memilih feature berdasarkan:

$=, t, t1$

Ini disebut Information Coefficient.

Namun kita harus menghitungnya **out-of-sample** atau dalam training window.

Jangan memilih feature berdasarkan seluruh history.

AB.78. Information coefficient

Untuk feature :

$$t = ,t, t1$$

Jika:

$$0,$$

feature cenderung memiliki hubungan positif dengan future cross-sectional return.

Tetapi satu tidak cukup.

Kita ingin:

$$t$$

stabil sepanjang waktu.

AB.79. IC stability

Kita dapat mengukur:

dan:

Kemudian:

$$=$$

Feature dengan:

lebih tinggi cenderung lebih konsisten.

Tetapi tetap harus diuji out-of-sample.

AB.80. Sekarang hubungan dengan \$1 risk

Semua estimasi:

$$,$$

akan masuk ke position sizing.

Jika stop distance:

$$= P_t - P,$$

maka quantity:

$$\boxed{= 1}$$

untuk target risk:

$$= 1$$

Namun quantity ini **belum final**, karena nanti kita juga harus mempertimbangkan:

$$t$$

AB.81. Individual risk vs portfolio risk

Ini sangat penting.

Dua posisi:

dan:

masing-masing mempunyai:

$$1$$

stop risk.

Bukan berarti portfolio risk:

$$2$$

secara efektif, karena mereka dapat sangat correlated.

Jika:

$$, 1,$$

maka keduanya sebenarnya satu common risk cluster.

Karena itu covariance matrix:

$$\boxed{t}$$

menjadi sangat penting.

AB.82. Portfolio variance

Jika:

adalah position vector:

$$2 = t$$

Ini akan menjadi bagian utama objective QUBO.

AB.83. Jadi seluruh AB dapat diringkas menjadi satu persamaan

Saya ingin kita menyimpan persamaan ini sebagai **persamaan inti Stage AB**:

$$t1 = t \#1 \ t1$$

dengan:

$$t = t \#1 \ t \ t$$

dan:

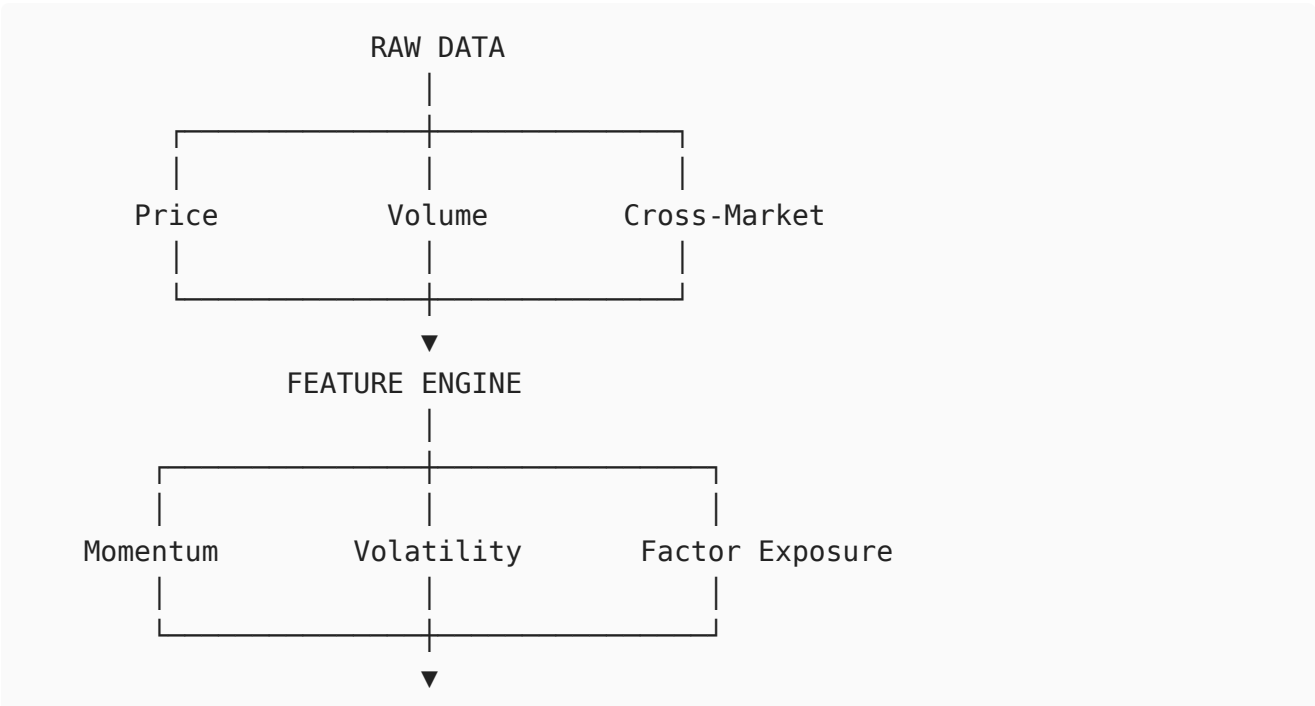
$$t = t,t \ t \ ,t$$

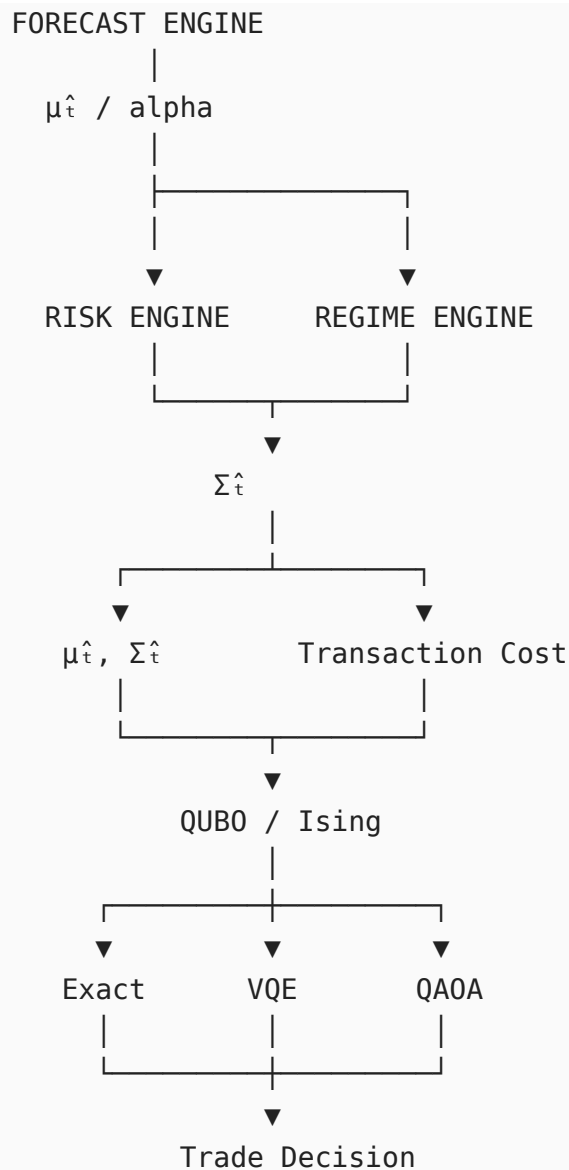
Kemudian:

$$t, \ t \ t$$

AB.84. Arsitektur final setelah Tahap AB

Sekarang sistem kita memiliki struktur:





AB.85. Hal yang sekarang sudah kita miliki

Setelah Tahap AB, kita tidak lagi mempunyai sekadar:

"model melihat harga BTC, ETH, Gold, dan DXY."

Kita mempunyai konstruksi matematis:

μ_t, Σ_t

Dan yang paling penting:

γ

Ini menurutku adalah posisi yang jauh lebih kuat secara metodologis.

AB.86. Apa yang belum kita selesaikan?

Ada satu bagian besar yang sengaja **belum** kita masukkan secara penuh:

□

Kita baru memiliki:

t t

Tahap berikutnya akan membawa:

t t

ke dalam sistem.

Dan ini akan sangat penting karena tujuan akhir kita bukan sekadar:

,

tetapi:

□

Dengan kata lain, **Tahap AC seharusnya menjadi Market Microstructure & Order-Flow Engine**. Di sana kita bisa mulai menurunkan secara matematis bagaimana:

,

yang kemudian akan mengubah t dari **expected market return** menjadi **expected executable return**. Itu merupakan jembatan penting sebelum kita benar-benar memasukkan order ke sistem QUBO.

Prompt:

8/17/2026, 10:17:05 PM

lanjut Tahap AC

Response:

8/17/2026, 10:17:05 PM

Remembering

Reviewed Tahap AC, market microstructure, order flow, and QUBO trading workflows

Worked for 4s

Tahap AC — Market Microstructure & Order-Flow Engine

Baik. Kita lanjutkan secara konsisten dari Tahap AB.

Pada Tahap AB kita sudah membangun:

$$t, t, t, t$$

Tetapi ada satu masalah fundamental:

Expected return belum tentu sama dengan expected executable return.

Misalnya model memperkirakan:

$$t_1 = 00$$

Tetapi untuk mendapatkan posisi tersebut kita harus memasang limit order. Bisa saja:

- order tidak pernah filled,
- hanya terisi sebagian,
- harga bergerak sebelum fill,
- spread terlalu besar,
- market bergerak melawan kita,
- liquidity menghilang,
- atau setelah memperhitungkan fee + slippage, alpha menjadi negatif.

Maka Tahap AC akan menambahkan:

□

di antara forecast dan execution.

AC.1. Perubahan paradigma

Sebelumnya kita memiliki:

$$,t = ,t1 \ t$$

Tetapi trader tidak mendapatkan:

$$,t1$$

secara gratis.

Kita membutuhkan execution decision:

$$,t = , , ,$$

Maka quantity yang benar-benar terealisasi menjadi random variable:

$$,t$$

Sehingga return aktual dari sebuah order lebih tepat ditulis:

$$,t = ,t \ ,t - ,t$$

AC.2. Order book sebagai keadaan pasar

Misalkan pada waktu t kita mempunyai order book:

$$t = \left\{ \begin{matrix} b_1, b_2, \dots, b_n \\ a_1, a_2, \dots, a_m \end{matrix} \right\}$$

dengan:

- b_k : bid price level ke- k ,
- q_k : bid quantity,
- a_k : ask price level ke- k ,
- q_k : ask quantity.

Best bid:

$$b_1$$

Best ask:

$$a_1$$

Mid-price:

$$\frac{b_1 + a_1}{2}$$

AC.3. Bid-ask spread

Spread absolut:

$$a_1 - b_1$$

Spread relatif:

$$\frac{a_1 - b_1}{\frac{b_1 + a_1}{2}}$$

Ini penting karena sebuah strategi dapat mempunyai:

$$0$$

tetapi:

$$t$$

sehingga tidak ekonomis untuk dieksekusi secara agresif.

AC.4. Microstructure-adjusted alpha

Maka expected alpha harus diubah menjadi:

$$t, t = , t - , t - , t - , t - , t$$

Untuk saham:

biasanya tidak relevan seperti pada perpetual crypto.

Untuk crypto perpetual, funding dapat menjadi signifikan.

AC.5. Order Book Imbalance

Salah satu feature paling fundamental:

$$t = t - tt \ t$$

dengan:

$$t = \quad = 1$$

dan:

$$t = \quad = 1$$

Range:

$$-1 \ t \ 1$$

Interpretasi:

$$t \ 0$$

berarti depth bid lebih besar.

Sedangkan:

$$t \ 0$$

berarti depth ask lebih besar.

AC.6. Tetapi depth level 1 saja terlalu sederhana

Kita dapat memberikan bobot terhadap setiap level:

$$= - - 1$$

Kemudian:

$$t = \quad = 1 ,$$

dan:

$$t = 1$$

Sehingga:

$$t = -$$

Level terdekat mendapatkan bobot terbesar.

AC.7. Menghubungkan OBI dengan return

Sekarang kita dapat memperluas model AB:

$$r_{t+1} = \alpha r_t + (1 - \alpha) r_{t+1}$$

Maka:

$$r_{t+1} = \alpha r_t + (1 - \alpha) r_{t+1}$$

Tetapi kita harus hati-hati.

OBI bukan otomatis:

$$0$$

Hubungannya harus dipelajari dari data.

AC.8. Order Flow Imbalance

Order book adalah **state**.

Kita juga ingin mengetahui **perubahan state**.

Misalkan:

$$\Delta_t = r_t - r_{t-1},$$

dan:

$$\Delta_t = r_t - r_{t-1}$$

Maka sebuah bentuk sederhana:

$$r_t = \Delta_t - \Delta_t$$

OFI menangkap tekanan order flow.

AC.9. Trade imbalance

Jika tersedia data transaksi:

$$t$$

dan:

$$t'$$

kita dapat mendefinisikan:

$$t = t - t t t$$

Ini berbeda dari OBI.

OBI:

TI:

AC.10. Menggabungkan tiga dimensi

Sekarang kita mempunyai:

$$t = t, t, t, t, t,$$

Kemudian:

$$,t = t t t t t$$

Namun saya tetap ingin model baseline dibuat sederhana terlebih dahulu.

AC.11. Market impact

Sekarang kita masuk ke bagian yang sangat penting.

Misalkan kita ingin membeli:

unit.

Jika:

$$1,$$

kita mungkin dapat membeli semuanya pada:

$$t$$

Tetapi jika:

1,

kita harus memakan level berikutnya:

2,

AC.12. VWAP execution price

Jika quantity yang dieksekusi pada level adalah:

,

maka:

$$P =$$

Slippage terhadap mid:

$$= P - t$$

untuk buy.

Untuk sell:

$$= t - P$$

AC.13. Market impact function

Untuk ukuran order yang lebih besar, kita dapat memodelkan:

$$=$$

dengan:

- : order size,
- : average daily volume,
- : impact scale,
- : impact exponent.

Sering kali:

$$0 \quad 1$$

Jadi market impact tidak selalu linear terhadap ukuran order.

AC.14. Implementation shortfall

Jika kita memiliki target price:

$$P,$$

dan execution price:

$$P,$$

maka:

$$= P - P$$

untuk long.

Secara lengkap:

$$= tt$$

AC.15. Sekarang masuk ke inti strategi kita: limit order

Kita tidak selalu ingin langsung membeli.

Misalnya:

$$t = 100$$

Kita memperkirakan:

$$P_t = 102$$

Kita dapat:

Market order

beli sekarang di:

$$10005$$

atau:

Limit order

letakkan:

$$0$$

Masalahnya:

$$0$$

mungkin tidak pernah tersentuh.

AC.16. Maka fill probability menjadi variabel utama

Definisikan:

$$= P_{t, t, t}$$

Ini adalah probabilitas order terisi.

AC.17. Expected executable alpha

Misalkan:

- profit jika filled = ,
- loss/opportunity cost jika tidak filled = .

Maka:

$$= -1 -$$

Namun kita dapat membuatnya lebih konkret.

AC.18. Limit order expected value

Misalkan limit buy pada harga:

$$P$$

Jika filled, expected future price:

$$P_t$$

Profit per unit:

$$= P_t - P$$

Jika order tidak filled, kita dapat mengalami opportunity cost:

$$= P_t - P$$

Maka:

$$_t = -1 - -$$

AC.19. Ini jauh lebih baik daripada sekadar signal

Sistem sebelumnya mungkin berkata:

0

Sekarang:

0

Kita memerlukan:

t 0

AC.20. Market order versus limit order

Kita dapat menghitung:

t

dan:

t

Kemudian:

$t = \in \{t, t, \}$

Ini merupakan **execution optimization problem**.

AC.21. Bahkan dapat diperluas

Action space:

$t = , , tt, t,$

Contoh:

$t = , 0, 05, , 0$

Jadi kita tidak hanya mengoptimalkan:

Kita juga mengoptimalkan:

□

AC.22. Time-to-live

Misalkan limit order memiliki TTL:

$$\tau$$

Kita definisikan:

$$\tau$$

Biasanya:

$$\tau_2 \tau_1$$

jika:

$$\tau_2 \tau_1,$$

karena order memiliki lebih banyak waktu untuk terkena.

Tetapi opportunity cost juga meningkat.

AC.23. Fill probability sederhana

Sebagai baseline:

$$= 0 \ 1 \ 2 \ 4 \tau \ 5$$

dengan sigmoid:

$$= 11 \ -$$

Di sini:

$$= P_{-t}$$

AC.24. Queue position

Ini sangat penting.

Misalkan kita memasang limit buy pada:

$$P$$

Di depan kita terdapat:

Jika:

$$= 1000$$

unit, maka order kita tidak akan filled hanya karena harga menyentuh level tersebut.

Kita juga membutuhkan:

$$\square$$

AC.25. Queue depletion model

Misalkan market sell volume yang masuk:

$$t$$

Queue position setelah periode τ :

$$\tau = 0 - \tau,$$

dengan:

$$\tau$$

adalah cumulative queue depletion.

Order filled ketika:

$$\tau \leq 0$$

AC.26. Probabilistic queue model

Karena kita tidak tahu future order flow secara pasti:

$$\tau$$

adalah random variable.

Maka:

$$\tau = P\tau_t$$

Ini secara matematis jauh lebih masuk akal daripada:

"Harga menyentuh limit berarti order filled."

AC.27. Partial fill

Misalkan:

$$= 100$$

Tetapi hanya:

$$0$$

unit yang filled.

Maka:

$$= 0$$

Kita dapat mendefinisikan fill ratio:

$$=$$

dengan:

$$0 \quad 1$$

AC.28. Expected fill quantity

Daripada hanya:

,

kita dapat menghitung:

$$=$$

Ini lebih tepat untuk position sizing.

AC.29. Execution-adjusted expected return

Sekarang kita gabungkan semuanya.

Misalkan:

adalah expected price return.

Kemudian:

adalah fill probability.

Expected executable alpha:

$$= -$$

Untuk partial fill:

$$= -$$

AC.30. Tetapi ini masih belum cukup

Karena:

dan:

tidak independent.

Misalnya kita memasang limit buy jauh di bawah market.

Jika market turun sampai limit tersebut:

Tetapi kondisi tersebut mungkin berarti:

$$P_t$$

juga berubah.

Jadi:

$$\boxed{P(\text{fill})}$$

dan:

$$P_t$$

harus dimodelkan bersama.

AC.31. Selection effect

Secara probabilistik:

$$= P \quad P$$

Ini sangat penting.

Karena order yang filled bukan sampel random.

$$\boxed{\text{Fill}}$$

sendiri membawa informasi tentang market path.}

AC.32. Contoh intuitif

Misalkan kita memasang:

ketika market:

Jika harga langsung naik:

100 101,

order tidak filled.

Jika harga:

100 ,

order filled.

Tetapi kondisi kedua berarti market terlebih dahulu bergerak turun.

Jadi:

P_{tt}

bisa berbeda jauh dari:

P_{tt}

AC.33. Ini menghasilkan konsep penting

Kita membutuhkan:

t

Yaitu:

$= t, t$

Bukan hanya:

$t \ t$

AC.34. Execution-aware alpha

Maka:

$= -$

Tetapi kita juga dapat memasukkan opportunity cost:

$t = -1 - -$

AC.35. Ini mengubah QUBO kita

Sebelumnya:

Sekarang:

$$-$$

Dengan:

$$= , , , t , , ,$$

Jadi QUBO sekarang memilih asset berdasarkan **expected executable alpha**, bukan theoretical alpha.

AC.36. Order price juga dapat menjadi optimization variable

Ini bahkan lebih menarik.

Misalkan tersedia kandidat price:

$$1, 2, ,$$

Untuk setiap:

kita hitung:

$$,$$

Kemudian:

$$= ,$$

Jadi kita tidak lagi mengatakan:

"Beli SOL."

Tetapi:

"Jika membeli SOL, harga limit optimal adalah ."

AC.37. Discretization

Kita dapat membentuk:

$$= t - \Delta$$

Misalnya:

$$= 0, 1, \dots, 10$$

Kemudian setiap candidate memiliki:

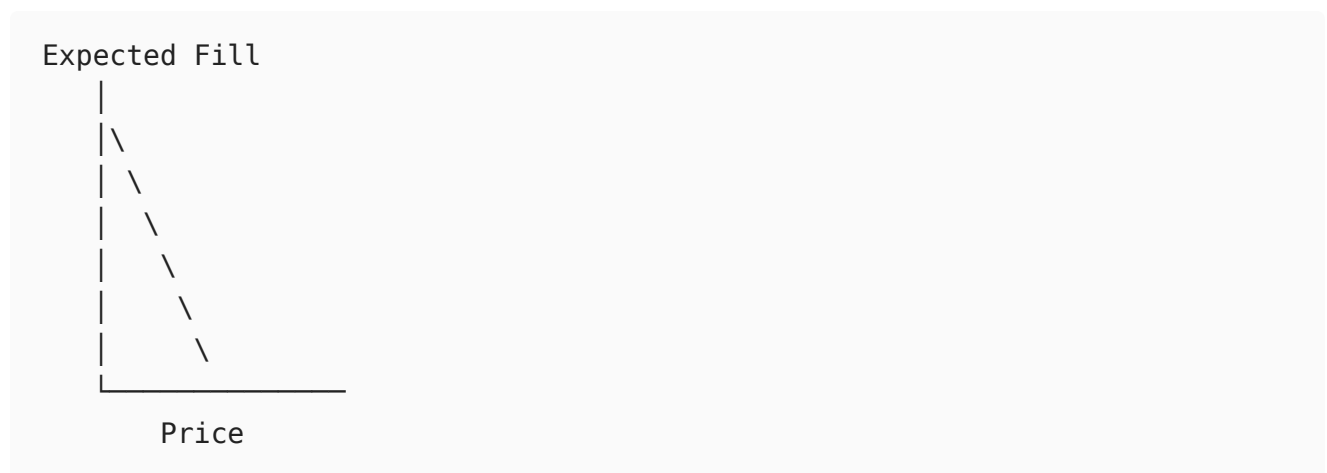
,’
,

AC.38. Execution frontier

Kita akan mendapatkan trade-off:



Secara konseptual:



Semakin agresif order:

tetapi:

P

untuk buy.

Semakin pasif:

P

tetapi:

AC.39. Optimal execution

Maka:



Ini sudah merupakan optimization problem tersendiri.

Dan menariknya, problem ini juga dapat diformulasikan sebagai:

□

jika kita ingin menggunakan pendekatan quantum untuk pemilihan discrete execution actions.

AC.40. Contoh QUBO execution

Misalkan kita memiliki tiga kandidat:

$$\begin{aligned}x_1 &= , \\x_2 &= -1t, \\x_3 &= -2t\end{aligned}$$

Definisikan:

$$x_i \in \{0, 1\}$$

Constraint:

$$x_1 + x_2 = 1$$

Objective:

$$- \sum_{i=1}^2 P_i - 1$$

Ini sudah menjadi QUBO.

AC.41. Tetapi saya tidak ingin langsung melakukan ini

Karena kita sekarang mempunyai **dua optimization layer**:

Layer 1 — Portfolio optimization

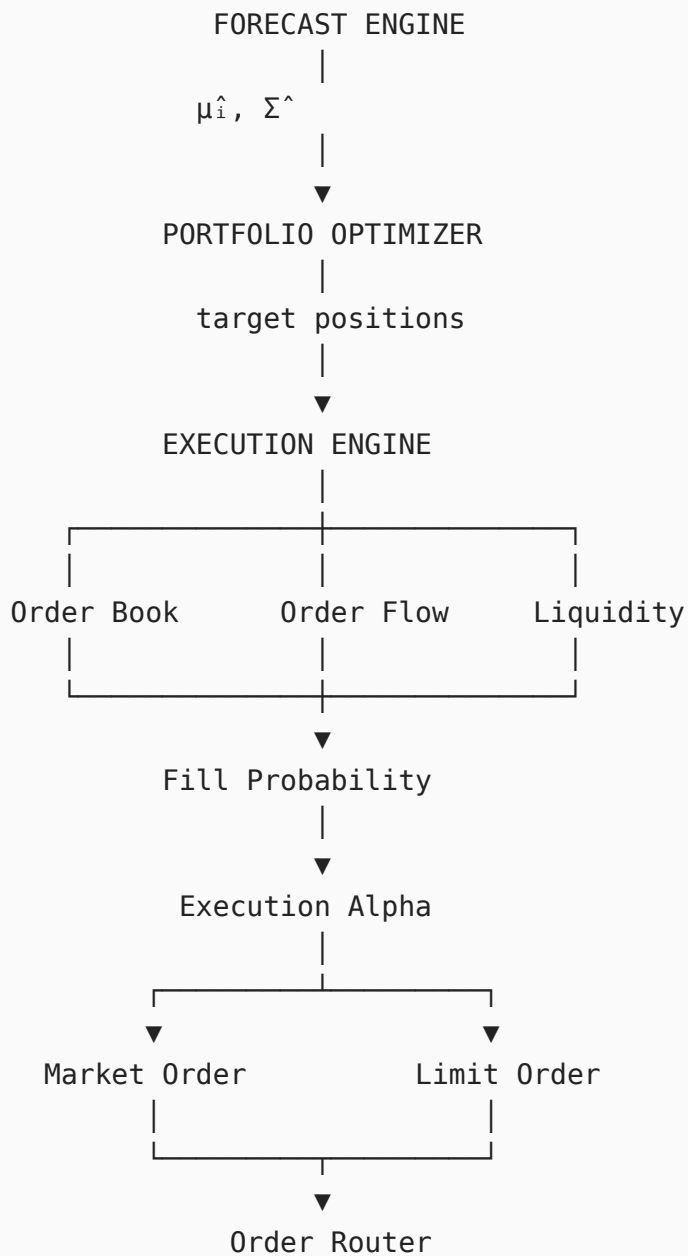
□

Layer 2 — Execution optimization

□

Sebaiknya kita pisahkan dahulu.

AC.42. Arsitektur AC yang saya sarankan



AC.43. Ini cocok dengan desain awal kita

Karena sebelumnya kita ingin:

data → menentukan order book → TP/SL → evaluasi periodik.

Sekarang bagian "menentukan order book" mempunyai dasar matematis:

$$Pt = tt$$

bukan sekadar rule:

"pasang limit 0.5% di bawah harga."

AC.44. Stop Loss juga masuk ke microstructure

Misalkan entry:

$$P$$

Stop:

$$P$$

Gross risk:

$$= P - P$$

Tetapi actual loss:

$$t =$$

Jadi target:

$$= 1$$

harus menggunakan:

$$t \quad 1$$

Bukan hanya:

$$= 1$$

AC.45. Stop-loss slippage

Ini sangat penting pada kondisi cepat.

Jika:

$$P =$$

tetapi market bergerak:

$$2 \quad 5,$$

stop execution mungkin terjadi di:

atau lebih buruk.

Maka:

$$P_{tt} \quad P_t$$

Kita perlu:

□

dan:

□

untuk sizing.

AC.46. Expected stop loss

Misalkan slippage stop:

adalah random variable.

Maka:

$$= P - P$$

Target risk:

$$\boxed{1}$$

untuk baseline.

Tetapi untuk risk control yang lebih konservatif, kita nantinya dapat menggunakan quantile:

$$\boxed{1}$$

AC.47. Ini akan sangat berguna untuk crypto

Karena pada market yang volatile:

dapat berubah cepat.

Maka:

$$t$$

harus dinamis.

Misalnya:

$$t = 1$$

AC.48. Take Profit juga bukan sekadar fixed percentage

Misalkan:

$$P = 2$$

Itu hanya static rule.

Kita dapat memperkirakan:

$$PP_t P_t$$

Definisikan:

$$P = P_t P_t$$

AC.49. Expected value TP/SL

Misalkan:

$$= P - P$$

dan:

$$= P -$$

Maka:

$$= P - 1 - P -$$

Trade hanya layak jika:

$$0$$

AC.50. Ini membawa kita ke konsep path probability

Bukan hanya:

$$PP_t P$$

Kita memerlukan:

$$P\tau_P \tau_t$$

dengan:

$$\tau_P = \{ P P \}$$

dan:

$$\tau = \{ P \}$$

Ini adalah **first-passage probability**.

Dan ini jauh lebih sesuai untuk sistem trading kita.

AC.51. Sekarang hubungan dengan tahap berikutnya mulai terlihat

Kita nantinya dapat menghitung:

$$PP$$

menggunakan:

- historical path,
- stochastic process,
- empirical distribution,
- regime,
- volatility,
- order flow.

Kemudian:

$$P$$

dan:

tidak lagi harus fixed.

AC.52. Dynamic TP

Misalkan posisi sedang profit:

$$P_t \ P$$

Kita dapat menghitung ulang:

$$P,$$

untuk berbagai candidate:

$$P$$

Kemudian:

$$P_t = P_{t-1}$$

Inilah fondasi matematis dari ide yang kamu sampaikan sebelumnya:

jika floating profit tinggi, apakah TP layak dinaikkan?

Jawabannya nanti:

$$P_t = P_{t-1}$$

AC.53. Dynamic Stop Loss

Demikian pula:

$$P_t = P_{t-1}$$

dengan constraint:

Ini dapat menghasilkan:

$$P_t = P_{t-1}$$

Tetapi **bukan berdasarkan fixed percentage**.

Melainkan:

$$P_t = P_{t-1}$$

AC.54. Tahap AC akhirnya menghasilkan empat output

Market microstructure engine kita akan menghasilkan:

$$P_t = \{P_t, t\}$$

Kemudian:

$$P_t = P_t, t$$

Dan:

$$P_t = P_t, t$$

AC.55. Pipeline sekarang menjadi jauh lebih lengkap

tt

tt

t, t

$Pttt$

tPt

tt

$,, t, t$

ttt

$\square$

AC.56. Dan sekarang kita memiliki prinsip yang sangat penting

Sistem kita tidak lagi bertanya:

"Apakah harga akan naik?"

Melainkan secara bertingkat:

Level 1 — Forecast

$t1 \ t$

Level 2 — Portfolio

ttt

Level 3 — Execution

ttt

Level 4 — Risk

tt

Level 5 — Management

t

Ini adalah struktur yang jauh lebih dekat dengan **systematic trading architecture** daripada sekadar model prediksi harga.

AC.57. Persamaan inti Tahap AC

Kalau Tahap AB mempunyai:

$$t, t$$

sebagai output utama, maka Tahap AC dapat diringkas oleh:

$$_{,t} = P_{t, \text{ , } t, -}$$

dengan action:

$$= , , tt, t, ,$$

Kemudian:

$$= , t$$

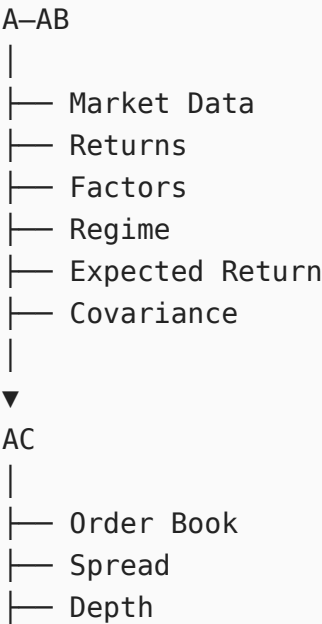
Dan portfolio-level system menggunakan:

$$t$$

sebagai pengganti t mentah.

AC.58. Posisi AC dalam keseluruhan proyek

Sekarang arsitektur kita sudah menjadi:



```

├─ OBI
├─ OFI
├─ Trade Flow
├─ Queue Position
├─ Fill Probability
├─ Partial Fill
├─ Slippage
├─ Market Impact
├─ Execution Alpha
├─
▼
Portfolio / Execution Optimization
├─
▼
QUBO / Ising
├─
├─ Exact Solver
├─ Classical Optimizer
├─ VQE
├─ QAOA
├─
▼
Order
├─
▼
Position Management
├─
├─ TP
├─ SL
├─ BEP
├─ Trailing
├─ Re-optimization

```

AC.59. Satu keputusan desain yang menurut saya sangat penting

Jangan langsung memasukkan order book ke QUBO.

Lebih baik kita mempertahankan hierarki:



Dengan demikian QUBO tidak menjadi model yang mencoba memahami seluruh market microstructure.

QUBO hanya menyelesaikan problem kombinatorial:

Ini membuat eksperimen quantum kita jauh lebih terkontrol.

AC.60. Dan ada satu hal yang menjadi sangat menarik

Setelah AC, kita sebenarnya telah memiliki **dua problem optimasi yang berbeda**:

□

—

dan:

□

—

Keduanya dapat berupa QUBO.

Artinya, pada akhirnya kita bahkan dapat membangun arsitektur:

□

Tetapi **jangan digabung dahulu**. Kita harus membuktikan masing-masing layer secara terpisah agar ketika performa sistem berubah, kita tahu penyebabnya.

Dan menurut urutan desain yang telah kita bangun, **Tahap AD** paling tepat digunakan untuk membangun **Trade Opportunity & Entry/Exit Path Engine**: mengubah , volatility, regime, dan probabilitas fill menjadi distribusi kemungkinan perjalanan harga sampai **TP/SL**, sehingga kita dapat menurunkan secara matematis **probabilitas TP sebelum SL, expected R-multiple, optimal entry, optimal SL, dan optimal TP**. Ini akan langsung menghubungkan AC dengan mekanisme posisi yang sejak awal kamu inginkan.

Prompt:

8/17/2026, 10:18:47 PM

lanjut Tahap AD

Response:

8/17/2026, 10:18:48 PM

Tahap AD — Trade Opportunity, First-Passage & Optimal TP/SL Engine

Baik. Tahap AD adalah salah satu bagian **paling penting** dari seluruh arsitektur kita, karena di sinilah sistem mulai berubah dari:

| "model memprediksi return"

menjadi:

"model menentukan apakah sebuah trade secara matematis layak dilakukan, di mana entry-nya, di mana SL/TP-nya, dan kapan struktur trade tersebut tidak lagi optimal."

Pada Tahap AC kita sudah memperoleh informasi microstructure:

$$t = \{t, t, t, t, t, t, t\}$$

Sekarang kita gabungkan dengan:

$$t, t, t, t$$

untuk membangun **distribusi perjalanan harga setelah entry**.

AD.1. Masalah yang sebenarnya ingin kita selesaikan

Misalkan sistem menemukan:

$$t \ 0$$

Secara sederhana kita mungkin berkata:

Buy.

Tetapi itu belum menjawab:

1. Berapa harga entry optimal?
2. Berapa jauh SL?
3. Berapa jauh TP?
4. Berapa probabilitas TP tercapai?
5. Berapa probabilitas SL tercapai?
6. Mana yang tercapai lebih dahulu?
7. Berapa lama trade diperkirakan bertahan?
8. Apakah expected value positif?
9. Apakah trade masih layak setelah memperhitungkan fee/slippage?
10. Jika posisi sudah berjalan, apakah TP/SL perlu diubah?

Maka objek matematis kita bukan lagi sekadar:

$$t1$$

Kita membutuhkan **price path**:

$$\{P_t\}_0$$

AD.2. Dari return menjadi stochastic process

Kita mulai dari log-price:

$$r_t = \ln \frac{P_t}{P_{t-1}}$$

Return kontinu:

$$r_t = \lim_{\Delta t \rightarrow 0} \ln \frac{P_t}{P_{t-\Delta t}}$$

Jika kondisi pasar relatif stationary untuk horizon pendek, kita dapat menggunakan pendekatan:

$$r_t = \sigma \epsilon_t$$

Ini adalah geometric Brownian motion pada harga:

$$P_t = P_0 e^{\int_0^t r_s ds}$$

Tetapi untuk sistem kita, GBM **bukan model final**.

Ia hanya baseline matematis.

AD.3. Mengapa GBM saja tidak cukup?

Pasar nyata memiliki:

- volatility clustering,
- fat tails,
- jumps,
- autocorrelation pada order flow,
- regime changes,
- asymmetric response,
- liquidity shocks.

Karena itu nantinya kita dapat mengganti:

,

dengan:

$$r_t = \mu + \sigma \epsilon_t$$

$$r_t = \mu + \sigma \epsilon_t$$

Namun untuk menurunkan fondasi first-passage secara analitik, kita mulai dari proses sederhana terlebih dahulu.

AD.4. Return conditional terhadap informasi saat ini

Pada waktu t :

$$r_t = \frac{P_t}{P_{t-1}}$$

dan:

$$r_{2t} = \frac{P_{2t}}{P_{2t-1}}$$

Jadi r_t dan r_{2t} bukan konstanta universal.

Mereka merupakan output dari model kita.

AD.5. Definisi entry

Misalkan entry price:

$$P_t$$

Dalam log-space:

$$\ln P_t$$

Kita kemudian menentukan:

$$P_t - P_{t-1}$$

untuk posisi long.

Dalam log-space:

$$\ln P_t$$

Definisikan jarak:

$$P_t - P_{t-1}$$

dan:

$$P_t - P_{t-1}$$

AD.6. Normalisasi posisi

Agar lebih mudah, kita definisikan:

$$r_t = \frac{P_t}{P_{t-1}}$$

Maka:

$$r_0 = 0$$

Batasnya:

$$-\tau$$

Dengan:

$$- = PP$$

dan:

$$= P_P P$$

AD.7. Trade sekarang menjadi masalah first-passage

Kita definisikan waktu mencapai TP:

$$\tau_P = \{\tau \mid 0 \leq \tau \leq \tau_P\}$$

Dan waktu mencapai SL:

$$\tau = \{\tau \mid 0 \leq \tau \leq \tau_P\}$$

Kemudian:

$$\tau_t = \tau_P, \tau$$

AD.8. Ini adalah inti Tahap AD

Kita tidak lagi bertanya:

$$PP \ P$$

Tetapi:

$$P_{\tau_P \ \tau}$$

Artinya:

berapa probabilitas TP tercapai sebelum SL?

Ini jauh lebih relevan terhadap trading.

AD.9. Menurunkan probabilitas TP sebelum SL

Misalkan:

$$t = t_t$$

Definisikan:

$$= P\tau \tau_-$$

Dengan boundary condition:

$$- = 0$$

dan:

$$= 1$$

Untuk diffusion process, fungsi memenuhi backward Kolmogorov equation:

$$\boxed{12^2 = 0}$$

AD.10. Menyelesaikan persamaan

Kita punya:

$$12^2 = 0$$

Misalkan:

$$=$$

Maka:

$$12^2 = 0$$

Sehingga:

$$= -2^2$$

Solusinya:

$$= -2^2$$

Integrasi:

$$= -2^2$$

Dengan boundary condition kita memperoleh:

$$\boxed{= 1 - {}^{-2^2}1 - {}^{-2^2}}$$

Karena entry berada pada:

$$= 0,$$

maka:

$$\boxed{P = 1 - {}^{-2^2}1 - {}^{-2^2}}$$

Ini adalah probabilitas TP tercapai **sebelum** SL.

AD.11. Kasus tanpa drift

Jika:

$$0,$$

kita tidak boleh langsung memasukkan $= 0$ ke bentuk tersebut karena menghasilkan 00.

Gunakan limit:

$$-1 -$$

Maka:

$$1 - -2^2 \ 2^2$$

Dan:

$$1 - -2^2 \ 2 \ 2$$

Sehingga:

$P =$

untuk:

$$= 0$$

Menariknya, tanpa drift, probabilitas ditentukan oleh posisi relatif terhadap kedua boundary.

AD.12. Interpretasi finansial

Misalkan:

$$= 1$$

dan:

$$= 2$$

Jika:

$$= 0,$$

maka:

$$P = 1$$

Sedangkan:

$$= 2$$

Jadi trade dengan reward:risk:

$$2 \text{ } 1$$

tidak otomatis memiliki probabilitas profit besar.

AD.13. Expected value trade

Sekarang kita dapat membangun expected value.

Misalkan:

$$P =$$

dan:

$$=$$

Untuk long:

$$= P P - 1 - P -$$

dengan:

$$= t \text{ } t$$

AD.14. Breakeven probability

Abaikan biaya terlebih dahulu.

Trade profitable jika:

$$P - 1 - 0$$

Maka:

$$P -$$

Sehingga:

$$P$$

Jadi:

$$= P$$

AD.15. Contoh

Jika:

$$= 1$$

dan:

$$P = 2,$$

maka:

$$= 1$$

Jadi kita tidak membutuhkan win rate 50%.

Tetapi setelah biaya:

AD.16. Dengan biaya

Misalkan biaya dalam unit risk:

Maka:

$$P - 1 - - 0$$

Sehingga:

$$P$$

Maka:

$$\boxed{= P}$$

AD.17. Ini menghasilkan trade filter

Sistem kita dapat mengatakan:

$$\boxed{P}$$

sebagai salah satu syarat minimum.

Tetapi saya menyarankan jangan berhenti di sana.

Kita juga perlu:

$$\boxed{0}$$

dan:

$$\boxed{tt}$$

serta:

$$t_{tt}$$

AD.18. Sekarang kita dapat mengoptimalkan TP

Misalkan entry dan SL sudah ditentukan.

Kita punya kandidat:

$$P_1, P_2, \dots, P$$

Untuk masing-masing:

$$= P P$$

Hitung:

$$= P \tau_P \tau$$

Kemudian:

$$= -1 - -$$

Maka:

$$P = P$$

AD.19. Jadi TP bukan fixed 2%

Ini perubahan besar.

Sistem tidak berkata:

| TP = 2%.

Melainkan:

$$P = t, t, t, t_t, t t_t,$$

AD.20. Optimal SL juga

Sekarang misalkan TP tetap.

Kita punya kandidat:

$$1, 2, \dots,$$

Untuk setiap:

,

kita hitung:

$$= P$$

Kemudian:

$$= P \tau_P \tau$$

Dan:

$$= P - 1 - -$$

Maka:

=

AD.21. Tetapi ada constraint risk

Misalkan modal risk maksimum:

$$= 1$$

Maka tidak semua SL boleh dipilih.

Position size:

$$= ,$$

dengan:

$$= P - P$$

Lebih realistis:

=

AD.22. Ini sangat penting

Kita tidak memilih:

terlebih dahulu kemudian memaksakan:

Kita melakukan:

$, P,$

sebagai satu paket keputusan.

AD.23. Trade candidate menjadi vector

Kita definisikan:

$$= P, P, P_P, \tau$$

Kemudian objective:

$$= - - t$$

AD.24. Time cost

Trade yang membutuhkan 3 hari berbeda dari trade yang membutuhkan 30 menit.

Misalkan:

$$\tau_t$$

adalah expected holding time.

Kita dapat memberikan penalty:

$$t = \tau_t$$

Sehingga:

$$= - \tau_t$$

Ini penting untuk strategi short-term.

AD.25. Mengapa holding time penting?

Misalkan dua trade:

Trade A

$$= 010$$

dengan:

$$= 10$$

Trade B

$$= 020$$

dengan:

$$= 24$$

Trade B memiliki EV absolut lebih besar.

Tetapi capital efficiency bisa jauh lebih buruk.

AD.26. Opportunity cost modal

Kita dapat mendefinisikan:

$$t =$$

atau secara annualized:

Namun hati-hati: annualization agresif dapat menyesatkan jika trade tidak benar-benar dapat direplikasi.

AD.27. Expected holding time

Untuk proses diffusion dengan dua absorbing boundaries, expected exit time:

$$= \tau_t$$

memenuhi:

$$12^2 = -1$$

Boundary:

$$- = = 0$$

Pada:

$$= 0$$

kita mendapatkan expected holding time.

Untuk:

$$= 0,$$

hasil sederhananya:

$$\tau_t = 2$$

Ini sangat menarik.

AD.28. Interpretasi

Jika:

dan:

semakin jauh, holding time meningkat.

Jika:

semakin tinggi:

Artinya volatility tinggi membuat boundary lebih cepat tersentuh.

Tetapi ini **tidak berarti volatility tinggi selalu bagus**.

Karena:

juga lebih mudah terkena.

AD.29. Maximum holding time

Di dunia nyata kita tidak ingin trade menggantung selamanya.

Definisikan:

Jika:

$$\tau_t,$$

maka:

$$t$$

Ini menghasilkan boundary ketiga:

$$P, t$$

AD.30. Maka trade menjadi three-barrier problem

Kita memiliki:

$P_P,$

$P,$

dan:

Trade berakhir ketika salah satu tercapai.

Secara matematis:

$$\tau = \tau_P, \tau,$$

AD.31. Probability vector

Kita sekarang ingin:

$$= P P P P t$$

Dengan:

$$P P P P t = 1$$

AD.32. Mengapa ini lebih baik daripada win rate?

Karena kita tidak lagi hanya menyimpan:

Kita menyimpan:

$$P P, P, P t$$

Dan juga:

Sehingga setiap trade memiliki probabilistic profile.

AD.33. Trade quality score

Kita dapat mendefinisikan:

$$t = t - 1t - 2t$$

Tetapi saya lebih menyukai mempertahankan komponen-komponen ini terpisah.

Jangan terlalu cepat membuat satu "magic score".

AD.34. Entry optimization

Sekarang kita kembali ke entry.

Misalkan current mid-price:

$$t$$

Kita memiliki candidate entry:

$$P_{,1}, P_{,2}, \dots, P_{,}$$

Untuk setiap entry:

$$P_{,}$$

kita harus menghitung ulang:

$$\begin{aligned} & , \\ & P_{,} \end{aligned}$$

Maka:

$$\boxed{P = P_{,}}$$

AD.35. Ini menggabungkan AC + AD

Pada AC:

$$P_{,} ,$$

Pada AD:

$$P_{,} P_{,}$$

Sehingga:

$$\boxed{= , P_{,} P_{,} - 1 - P_{,} - }$$

Ini jauh lebih realistis.

AD.36. Conditional probability chain

Kita dapat menulis:

$$PP = PPP$$

Jadi:

$$PP = P$$

Demikian pula:

$$P =$$

AD.37. Tetapi ada satu kasus lagi

Jika order **tidak filled**, tidak ada trade.

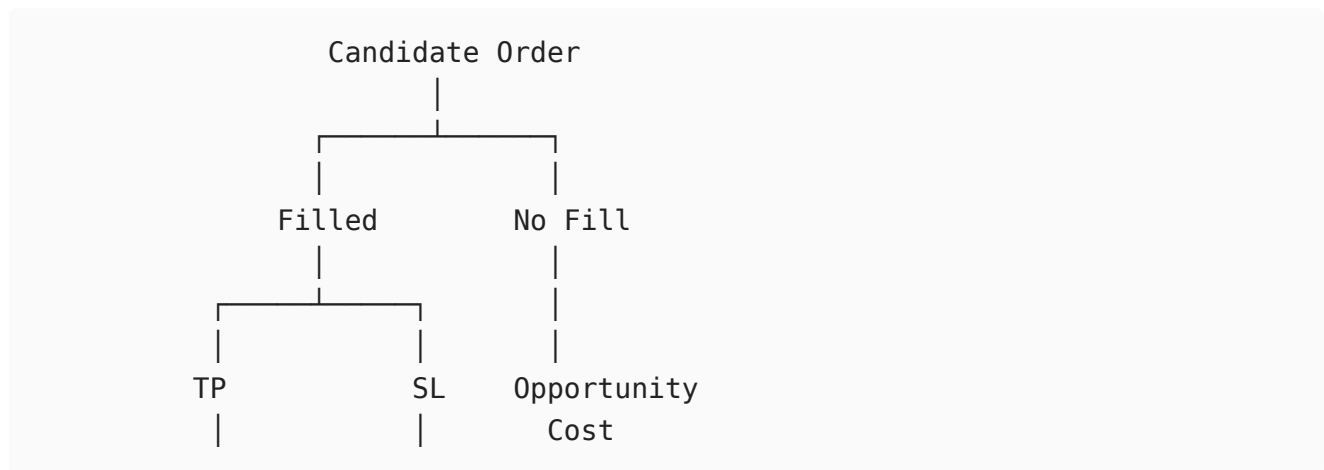
Maka:

$$Pt = 1 -$$

Jadi expected value keseluruhan:

$$= t - tt$$

AD.38. Ini menghasilkan decision tree



Ini secara konseptual sangat penting untuk backtest kita.

AD.39. Market order

Untuk market order:

$$1$$

tetapi:

$$t$$

lebih besar.

Untuk limit order:

1

tetapi:

t

lebih kecil.

Jadi sistem dapat membandingkan:

t

AD.40. Dynamic entry

Misalkan sekarang:

$t = 00$

sedangkan:

$t = 012$

Maka:

t

lebih baik.

Tetapi 20 menit kemudian:

$t = 015$

dan:

$t = 010$

Sistem seharusnya beralih:

$t \ t$

Ini adalah **execution adaptation**.

AD.41. Sekarang kita masuk ke posisi yang sudah terbuka

Ini langsung menjawab ide awalmu tentang:

"jika floating profit tinggi, apakah TP dinaikkan?"

Misalkan entry:

$$P$$

Harga sekarang:

$$P_t \quad P$$

Kita tidak boleh mengatakan:

"Sudah profit 2%, naikkan TP."

Sebaliknya kita menghitung ulang:

$$\boxed{P, \quad t}$$

AD.42. Re-optimization

Pada setiap evaluation interval:

$$t = t_0, t_1, t_2,$$

kita hitung:

$$t = \quad t$$

Dengan:

$$= , P, P_t, t$$

AD.43. Dynamic stop loss

Misalkan:

$$= P$$

Kita menemukan:

$$P$$

Maka kita dapat melakukan:

$$\boxed{=}$$

Ini berarti kita mengunci profit.

Tetapi:

tidak boleh hanya berdasarkan floating profit.

Harus memenuhi:

$$, P, P$$

AD.44. Dynamic TP

Jika:

$$P \ P,$$

kita naikkan TP hanya jika:

$$\boxed{P \ P}$$

Jadi trailing TP menjadi hasil optimasi.

AD.45. Tetapi ada masalah penting: sunk cost

Setelah posisi terbuka, kita tidak boleh lagi menggunakan:

$$P$$

sebagai satu-satunya dasar keputusan.

Keputusan optimal harus berdasarkan:

$$\square$$

Artinya:

"Jika saya **sudah memiliki posisi ini sekarang**, apa tindakan terbaik dari titik ini?"

Ini adalah masalah **stochastic control**.

AD.46. State posisi

Kita dapat mendefinisikan state:

$$\boxed{t = P_t, P, t, P_t, t, t, t, t, t}$$

Kemudian action:

$$\boxed{t = , , , P}$$

AD.47. Objective continuation value

Untuk action :

$$t_s = tP - t_{t_s}$$

Kemudian:

$$t = t_s$$

Ini adalah awal dari **dynamic programming**.

AD.48. Bellman equation

Jika kita ingin benar-benar membangun model matematis lanjut:

$$t = \max_t \{t, t_{t1}, t_s\}$$

Dengan discount factor :

$$t = \gamma t_{t1} + t_s$$

Ini nantinya dapat menjadi jembatan ke:

- stochastic control,
- dynamic programming,
- reinforcement learning.

Tetapi **belum kita gunakan sebagai core engine**.

AD.49. Mengapa saya tidak menyarankan RL sekarang?

Karena kita sudah memiliki struktur matematis yang lebih interpretable:

$$, , PP, P, , , t$$

Kalau langsung menggunakan RL, kita berisiko mendapatkan:

| agent menghasilkan profit.

tetapi tidak mengetahui:

| mengapa agent melakukan tindakan tersebut.

Untuk penelitian yang ingin memiliki fondasi matematis kuat, lebih baik:

AD.50. Market regime masuk ke AD

Misalkan regime:

$$t \in \{, , , , \}$$

Maka:

$$= t,$$

$$= t$$

Akibatnya:

$$P = P, , t, t$$

Jadi TP/SL optimal berbeda untuk setiap regime.

AD.51. Contoh konseptual

Regime A

$$0,$$

Model mungkin menghasilkan:

$$P =$$

Regime B

$$0,$$

Mungkin:

$$P = 15$$

Karena walaupun upside besar, probabilitas SL juga meningkat.

AD.52. Market factor juga masuk

Kita sebelumnya ingin sistem tidak hanya crypto.

Untuk saham internasional, state kita dapat memasukkan:

$$t = P$$

Kemudian:

$$,t = t$$

Sehingga:

$$P, = ,t, ,t, t, tt_t$$

AD.53. Contoh BTC

Misalnya kita ingin trading altcoin .

State:

$$t = , , , , , , , ,$$

Maka:

$$= tt$$

AD.54. Untuk saham internasional

Misalkan kita ingin trading:

$$P$$

State dapat berupa:

$$t = P, P, , , , , , , tt$$

Model:

$$P = PP$$

Jadi **engine AD** tidak perlu tahu apakah underlying-nya crypto atau saham.

AD.55. Asset-agnostic architecture

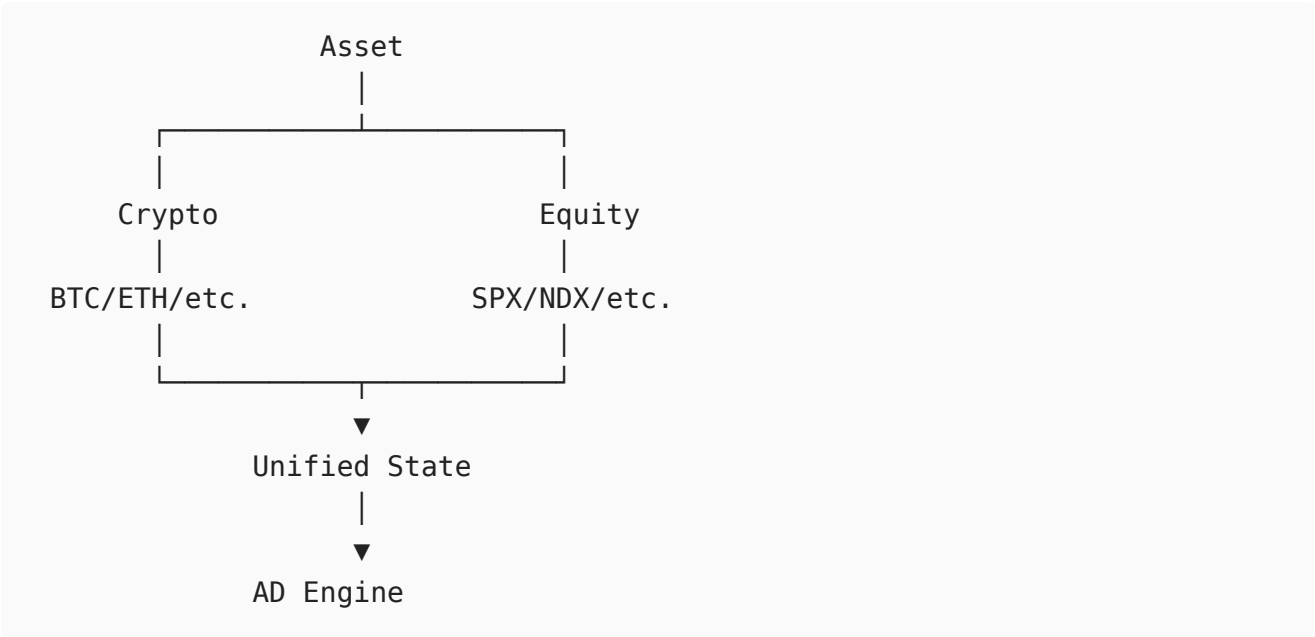
Kita dapat mendefinisikan:

$$ttt_t = t, t, tt, t, tt,$$

Kemudian seluruh engine menggunakan:

$$ttt_t$$

Dengan demikian:



AD.56. Sekarang kita dapat mendefinisikan Trade Opportunity

Sebuah trade candidate:

$$= t, , t, , P, ,$$

Trade candidate diterima hanya jika:

$$0$$

Dengan constraint:

$$, \\ t ,$$

AD.57. Opportunity set

Misalkan kita memiliki aset dan konfigurasi trade per aset.

Maka:

$$t = \{1, 1, 2, 3\}$$

Kita kemudian menghitung:

,

AD.58. Ini mengarah kembali ke optimasi kombinatorial

Sekarang kita bisa memiliki:

$$, \in \{0, 1\}$$

dengan:

$$, = 1$$

jika trade candidate pada asset dipilih.

Constraint:

,

Objective:

$$,, -$$

Ini sudah menjadi kandidat kuat untuk:

□

AD.59. Jadi sekarang QUBO kita semakin menarik

Sebelumnya:

=

Sekarang:

, =

Misalnya:

BTC
 └─ Long, TP=1R, SL=1R
 └─ Long, TP=2R, SL=1R
 └─ Long, TP=3R, SL=1R
 └─ Short, TP=2R, SL=1R

ETH

└─ Long, TP=1R, SL=1R
└─ Long, TP=2R, SL=1R
...

Quantum optimizer memilih kombinasi terbaik.

AD.60. Ini merupakan perkembangan besar

Awalnya kita memiliki:

□

Sekarang menjadi:

□

Decision variable bukan lagi hanya:

Tetapi:

$$t, t, P, t$$

AD.61. Namun jangan langsung mengoptimalkan semuanya

Ini penting secara engineering.

Kita sebaiknya memiliki hierarchy:

Level 1

Level 2

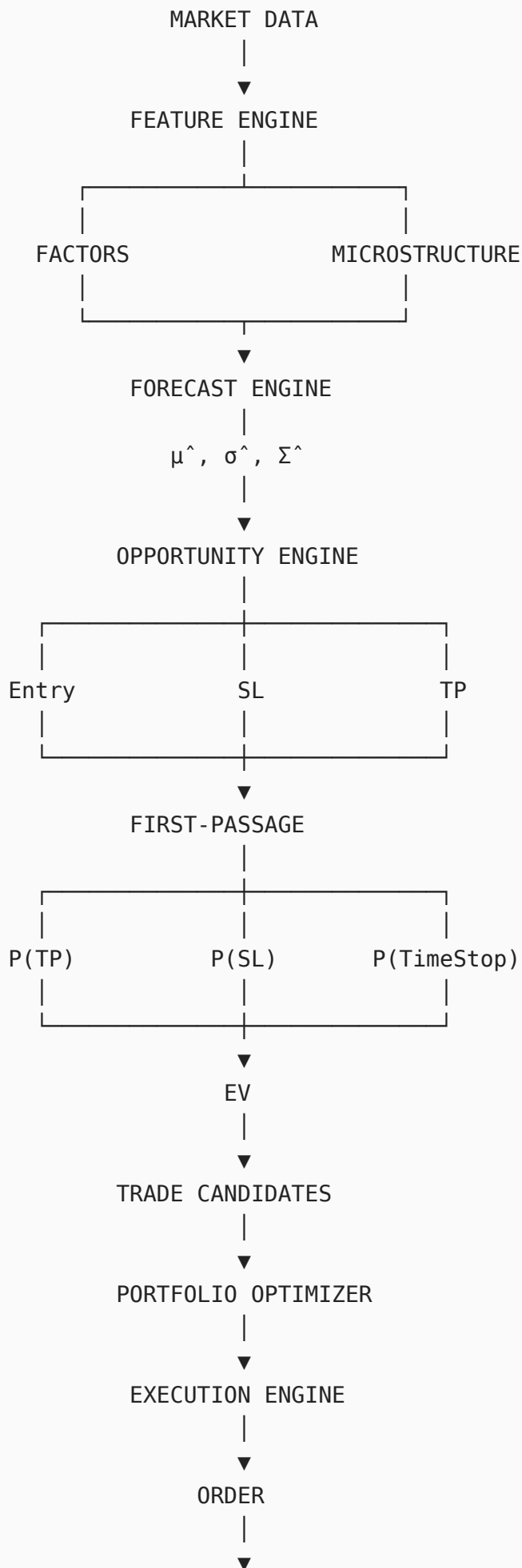
Level 3

Level 4

Level 5

Karena jika semua variabel dilempar ke satu optimizer, dimensionality akan meledak.

AD.62. Pipeline final setelah AD



POSITION MANAGEMENT



RE-OPTIMIZATION

AD.63. Formula inti Tahap AD

Kalau harus diringkas menjadi satu rangkaian matematis:

1. Conditional process

$$t = t_t$$

2. Barrier

$$-t-$$

3. First-passage probability

$$P = 1 - e^{-2^2} 1 - e^{-2^2}$$

4. Expected value

$$= P - 1 - P -$$

5. Optimal configuration

$$=$$

dengan:

$$= t, P, ,$$

AD.64. Dan inilah koneksi langsung dengan target \$1 risk

Kita ingin:

$$= 1$$

Untuk setiap kandidat:

,

hitung:

$$= 1, ,$$

Kemudian:

$$P_1 = P_2$$

Dan:

$$= P_1 P_2 - 1 - P_1 - t$$

Maka sistem memilih:

$$=$$

Dengan kata lain:

\$1 bukan target profit. \$1 adalah budget risiko maksimum per trade.

Ini perbedaan yang sangat penting.

AD.65. Bahkan kita dapat mendefinisikan "quality of opportunity"

Misalkan:

$$0$$

Tetapi kita dapat membandingkan terhadap risk:

$$ttt =$$

Misalnya:

$$= 0.15$$

Berarti:

$$= 0.15$$

Semakin besar semakin menarik, dengan catatan estimasi probabilitasnya reliable.

AD.66. Tetapi ada satu hal yang harus kita cegah

Jangan sampai sistem memilih:

$$P = 100$$

hanya karena model memberikan sedikit probabilitas untuk mencapainya.

Misalnya:

$$= 0.01, P = 100$$

Secara naïf:

$$= 001100 - 01 = 001$$

Terlihat positif.

Tetapi estimasi tail probability seperti itu sangat rentan estimation error.

Maka kita membutuhkan:

$$P_{tt}$$

dan:

$$t_{tt}$$

Ini akan menjadi bagian penting tahap berikutnya.

AD.67. AD menghasilkan fondasi untuk sesuatu yang jauh lebih besar

Sekarang kita telah memiliki tiga lapisan probabilistik:

$$P$$

$$PP$$

$$P$$

sehingga:

$$PP = PPP$$

Dan akhirnya:

$$= PP - P -$$

Ini adalah **trade-level expected value engine**.

AD.68. Hubungan AD dengan seluruh visi awalmu

Ide awalmu adalah:

"gunakan data historis, lihat aset terkait, pasang order book, tentukan TP/SL, lalu evaluasi posisi secara berkala."

Sekarang masing-masing sudah mempunyai fondasi matematis:

Ide awal	Engine matematis
Data historis	Forecast engine
BTC/ETH/Gold/DXY	Factor engine
Order book	Microstructure engine
Limit order	Fill probability
Harga entry	Entry optimization
SL	Barrier optimization
TP	Barrier optimization
\$1 risk	Position sizing
Floating profit	State update
Naikkan SL	Dynamic optimization
Naikkan TP	Dynamic optimization
Evaluasi berkala	Receding-horizon optimization
Banyak aset	Portfolio optimization
Kombinasi trade	QUBO
Short-term	First-passage/time horizon

Jadi kita sekarang **bukan sekadar membuat bot trading**.

Kita sedang membangun:

□

yang kemudian dapat mempunyai **quantum optimization layer** sebagai salah satu solver-nya.

AD.69. Langkah berikutnya

Setelah AD, ada satu masalah yang harus kita selesaikan sebelum sistem boleh dipercaya:

□

Misalnya model mengatakan:

$$PP = 0.72$$

Apakah 72% benar-benar berarti sekitar 72 dari 100 trade serupa akan mencapai TP sebelum SL?

Kalau tidak, seluruh:

$$P,$$

bahkan QUBO kita bisa menjadi **garbage-in-garbage-out** yang sangat matematis.

Karena itu **Tahap AE** seharusnya kita dedikasikan untuk:



yang mencakup **walk-forward validation, purged/embargoed time-series cross-validation, calibration curve, Brier score, log-loss, confidence interval, bootstrap, multiple-testing bias, data leakage, survivorship bias, look-ahead bias, dan Deflated/Probabilistic Sharpe Ratio.**

Ini menurut saya justru salah satu tahap paling krusial dalam keseluruhan proyek, karena setelah AE kita baru bisa mengatakan bahwa optimasi yang kita lakukan **memiliki bukti statistik**, bukan sekadar menghasilkan backtest yang terlihat bagus.

Prompt:

8/17/2026, 10:19:51 PM

lanjut Tahap AE

Response:

8/17/2026, 10:19:51 PM

Remembering

Reviewed and preserved your Tahap AE trading architecture and validation integrity

Worked for 4s

Tahap AE — Probability Calibration, Statistical Validation & Backtest Integrity

Baik. Setelah Tahap AD, kita sudah memiliki **trade engine** yang dapat menghasilkan keputusan seperti

$$t = t, P,$$

berdasarkan estimasi:

$$t, PP, P, Pt,$$

dan expected value.

Tetapi sekarang muncul pertanyaan yang jauh lebih fundamental:

Seberapa besar kita boleh mempercayai probabilitas dan expected value yang dihasilkan model?

Kalau model mengatakan

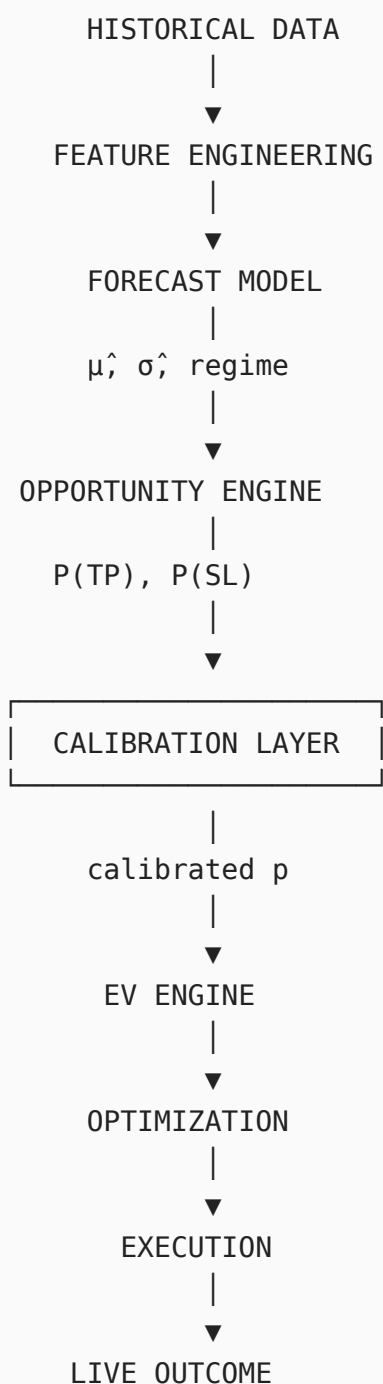
$$PP = 00,$$

tetapi pada kenyataannya hanya 55% trade yang berhasil, maka optimasi matematis kita akan mengoptimalkan **dunia yang salah**.

Jadi Tahap AE kita jadikan sebagai **lapisan validasi statistik yang berdiri di antara model prediksi dan optimizer**.

AE.1. Prinsip utama

Arsitektur kita sekarang seharusnya menjadi:



VALIDATION / UPDATE

Dengan demikian:

Ketiganya harus dipisahkan.

AE.2. Prediction versus probability

Misalkan model memberikan:

$$t = 0.2$$

Artinya secara ideal:

Pada kumpulan kondisi yang mempunyai probabilitas prediksi sekitar 72%, sekitar 72% outcome seharusnya benar-benar terjadi.

Ini adalah konsep **calibration**.

Jika model menghasilkan:

$$= 0$$

untuk 1.000 trade, kita mengharapkan kira-kira:

$$0.0$$

trade berhasil.

Bukan berarti setiap kelompok kecil harus tepat 70%, tetapi secara statistik harus mendekati.

AE.3. Perfect calibration

Secara formal, model dikatakan calibrated apabila:

$$P = 1 \quad = \quad =$$

Dengan:

$$= \begin{cases} 1, & P_0, \end{cases}$$

Jadi:

$$PP = =$$

Ini sangat penting.

AE.4. Contoh model yang tidak calibrated

Misalkan kita mengelompokkan prediction:

Prediksi	Jumlah trade	TP aktual
0.50	100	48
0.60	100	51
0.70	100	55
0.80	100	61
0.90	100	68

Model jelas **overconfident**.

Model mengatakan:

0

tetapi kenyataannya:

Jika optimizer menggunakan 90%, EV akan sangat overestimated.

AE.5. Reliability diagram

Cara visual paling sederhana:

=

dan:

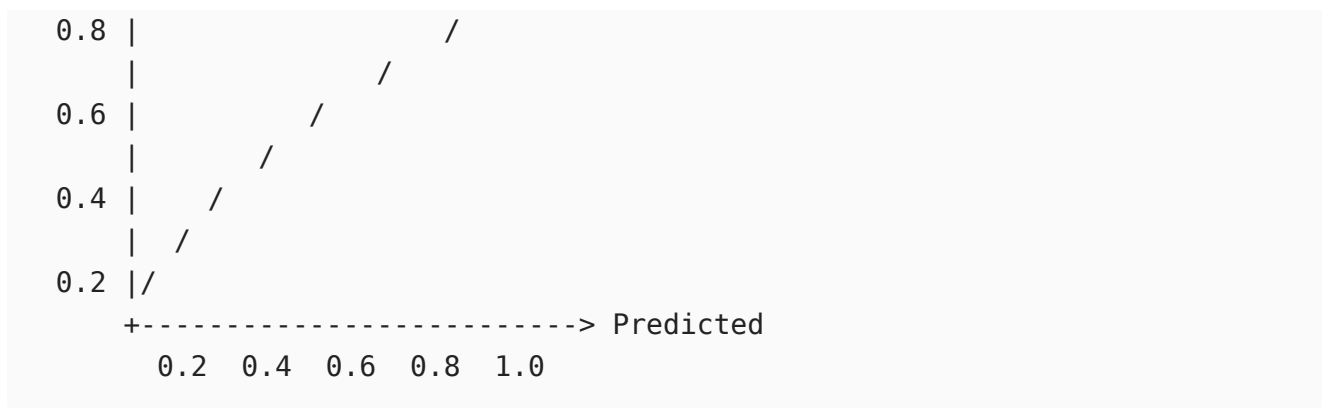
=

Perfect calibration adalah:

=

Misalnya:

Actual
1.0 | /
| /



Semakin dekat terhadap diagonal, semakin calibrated.

AE.6. Brier Score

Untuk binary outcome, kita dapat menggunakan:

$$= 1 - \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Dengan:

$$y_i \in \{0, 1\}$$

Semakin kecil:

$$\square$$

AE.7. Mengapa Brier Score sangat cocok untuk sistem kita?

Karena kita tidak hanya peduli:

| apakah trade benar?

Kita peduli:

| apakah probabilitas yang diberikan model masuk akal?

Misalkan:

Model A

$$= 0.51$$

ketika:

$$= 1$$

Error:

$$0.51 - 1^2 = 0.2401$$

Model B

$$= 0.5$$

ketika:

$$= 0$$

Error:

$$0.5 - 0^2 = 0.25$$

Model B mendapat hukuman jauh lebih besar karena **sangat percaya diri tetapi salah**.

Ini persis jenis kesalahan yang berbahaya bagi optimizer kita.

AE.8. Log-loss

Kita juga menggunakan:

$$= -1 \ln(1 - 1) = -1 \ln(0)$$

Jika:

$$= 1$$

dan:

$$0,$$

maka:

$$-$$

Artinya prediksi yang **sangat yakin tetapi salah** mendapat hukuman sangat besar.

AE.9. Brier versus Log-loss

Keduanya mengukur kualitas probabilitas, tetapi sifatnya berbeda.

Metric	Fokus
Accuracy	klasifikasi
Brier Score	calibration + probabilistic accuracy
Log-loss	penalti confidence yang salah

Metric	Fokus
Sharpe	risk-adjusted return
PnL	profitability

Kita tidak boleh menggunakan Sharpe untuk menggantikan calibration.

AE.10. Calibration tidak sama dengan profitability

Ini sangat penting.

Model dapat memiliki:

yang sangat bagus tetapi tidak profitable setelah:

- fee,
- slippage,
- market impact,
- spread.

Sebaliknya model dapat profitable tetapi poorly calibrated.

Jadi kita membutuhkan **dua dimensi evaluasi**:

$$tttt$$

dan:

$$t$$

AE.11. Economic validity

Untuk setiap trade:

$$= -1 - -$$

Tetapi outcome aktual:

Kita ingin:

$$0 \ 0$$

Ini adalah pengujian apakah probabilistic forecast benar-benar menghasilkan **economic value**.

AE.12. Jangan melakukan random train-test split

Ini adalah salah satu aturan terpenting.

Misalnya data:

2021 ————— 2025

Jangan:

Random 80% → Train
Random 20% → Test

Karena data finansial adalah **time dependent**.

Random split dapat menyebabkan informasi masa depan masuk ke training.

AE.13. Time-series split

Kita gunakan:

Train	Test
2021 — 2022	2023

kemudian:

Train	Test
2021 — 2023	2024

kemudian:

Train	Test
2021 — 2024	2025

Ini disebut:

t

AE.14. Mengapa walk-forward?

Karena situasi live sebenarnya adalah:

$$\boxed{Pt \quad t}$$

Bukan:

$$t \quad Pt$$

Walk-forward mempertahankan arah waktu.

AE.15. Expanding window

Salah satu bentuk:

$$_1 = t_0, t_1$$

$$t_1 = t_1, t_2$$

Kemudian:

$$_2 = t_0, t_2$$

$$t_2 = t_2, t$$

Dan:

$$= t_0, t$$

$$t = t, t_4$$

Ini disebut:

$$\square$$

AE.16. Rolling window

Alternatif:

$$_1 = t_0, t_1$$

$$t_1 = t_1, t_2$$

Kemudian:

$$_2 = t_1, t_2$$

$$t_2 = t_2, t$$

Dan seterusnya.

Ini:

□

Untuk pasar yang mengalami structural regime changes, rolling window sering sangat menarik karena model tidak terlalu bergantung pada sejarah yang terlalu lama.

AE.17. Purged cross-validation

Ada masalah yang lebih halus.

Misalkan kita membuat label berdasarkan:

$$t = t_0$$

Training sample pada:

$$t = t_0 + 1000$$

menggunakan informasi sampai:

$$t = t_0 + 1100$$

Sementara test dimulai:

$$t = t_0 + 1000$$

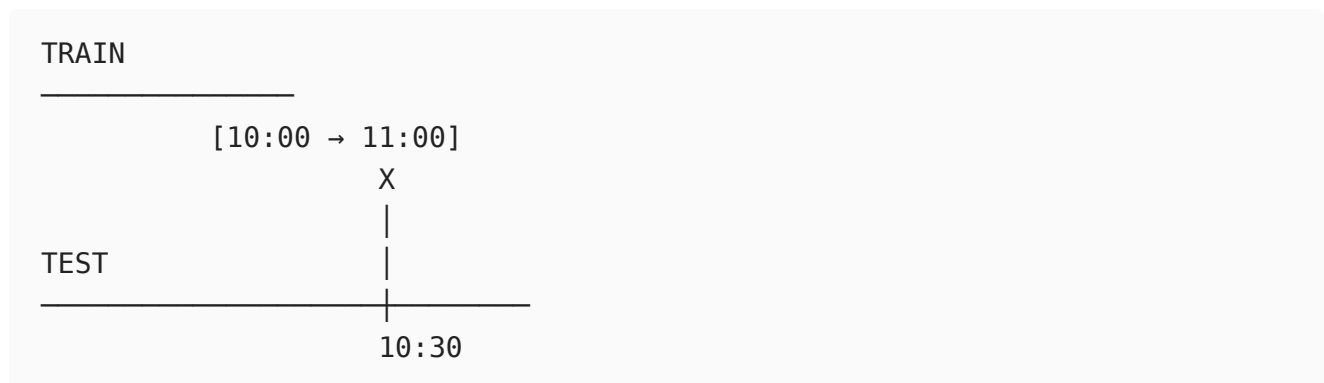
Maka dua sample tersebut overlap.

Itu adalah leakage.

AE.18. Purging

Kita menghapus training observations yang labelnya overlap dengan test interval.

Misalnya:



Sample training tersebut harus dibuang.

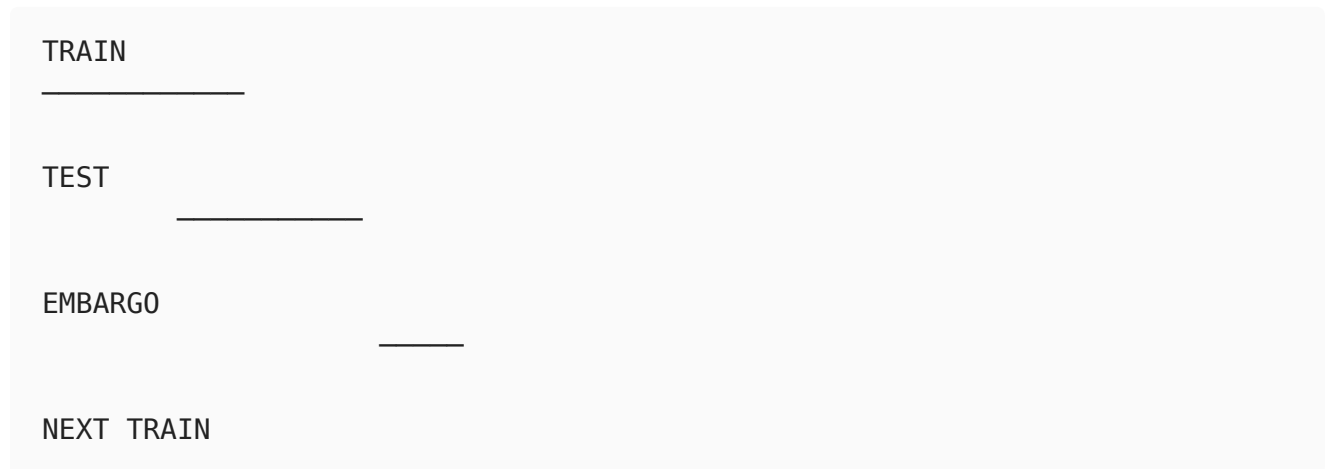
Ini disebut:

□
P

AE.19. Embargo

Masih ada masalah lain.

Setelah test period, kita dapat memberikan gap:



Gap ini mencegah ketergantungan temporal merembes antar-fold.

Disebut:

□

AE.20. Purged + Embargoed CV

Jadi validasi kita idealnya:

□

dengan:

□

Ini jauh lebih tepat untuk strategi yang memiliki:

- overlapping labels,
- holding period,
- multiple horizons,
- triple-barrier outcomes.

AE.21. Triple-barrier labeling

Ini sangat cocok dengan Tahap AD.

Untuk setiap entry:

$$t,$$

kita menentukan:

$$P_{t,},$$

Kemudian label:

$$= 1, \quad P-1, \quad 0, \quad t$$

Jadi kita tidak menggunakan label:

$$P_{t1} \quad P_t$$

Melainkan outcome yang benar-benar berkaitan dengan trade.

AE.22. Ini membuat training dan validation konsisten

Tahap AD:

$$PP, P, Pt$$

Tahap AE:

$$\in \{P, , t\}$$

Dengan demikian probabilistic model memprediksi **event yang sama dengan event yang digunakan optimizer**.

Ini sangat penting.

AE.23. Multiclass calibration

Sekarang probabilitas kita bukan hanya binary.

Model menghasilkan:

$$= P$$

dengan:

$$P = 1$$

Maka kita dapat mengevaluasi calibration masing-masing outcome.

Misalnya:

$$PP = 00$$

Di antara semua trade dengan prediksi sekitar 70%, actual TP harus mendekati 70%.

AE.24. Calibration map

Jika model ternyata overconfident, kita dapat membangun:

$$\boxed{=}$$

Misalnya:

$$= 00$$

tetapi calibration menunjukkan:

$$= 0$$

Optimizer **harus menggunakan**:

$$0,$$

bukan:

$$00$$

AE.25. Platt scaling

Untuk binary model, salah satu metode:

$$\boxed{= 11}$$

dengan adalah raw model score.

Parameter:

,

dipelajari menggunakan validation data.

AE.26. Isotonic regression

Alternatif yang lebih fleksibel:



dengan constraint:

$$1 \ 2 \ 1 \ 2$$

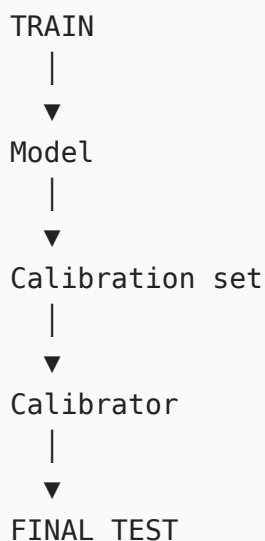
Artinya calibration function monotonic.

Ini berguna ketika hubungan antara raw probability dan actual frequency tidak linear.

AE.27. Jangan calibration menggunakan test set

Ini sangat penting.

Urutannya:



Kalau test digunakan untuk fitting calibration:



AE.28. Confidence interval

Misalkan:

$$= 00$$

dari:

$$= 100$$

Jangan langsung berkata:

$$= 0$$

Kita perlu uncertainty.

Untuk binary proportion:

$$=$$

Standard error:

$$= 1 -$$

AE.29. Contoh

Jika:

$$= 00, = 100,$$

maka:

$$= 004100$$
$$= 004$$

Jadi uncertainty sekitar:

$$4$$

Kita tidak boleh memperlakukan 60% sebagai angka eksak.

AE.30. Wilson interval

Untuk probability yang lebih robust daripada Wald interval, kita dapat menggunakan Wilson interval:

$$^2_2 \ 1 - \ ^2_4 2^1 \ ^2_2$$

Untuk confidence level 95%:

$$1$$

AE.31. Mengapa uncertainty penting untuk EV?

Misalkan:

$$= 055$$

Dengan:

$$P = 2, = 1$$

Maka:

$$= 0552 - 0451 = 05$$

Terlihat bagus.

Tetapi jika actual probability uncertainty sangat besar, misalnya:

$$\in 045, 05,$$

maka:

$$= 0452 - 055 = 05$$

Masih positif.

Tetapi untuk setup lain mungkin:

berubah dari positif menjadi negatif.

AE.32. Conservative EV

Kita dapat menggunakan lower confidence bound:

$$\square$$

daripada:

Kemudian:

$$t = -1 - -$$

Trade hanya dipertimbangkan jika:

$$t \geq 0$$

Ini membuat sistem lebih defensif.

AE.33. Bootstrap

Kita juga perlu mengukur uncertainty terhadap:

- mean return,
- Sharpe,
- max drawdown,
- win rate,
- EV.

Bootstrap dilakukan dengan resampling data.

Misalnya:

```
Original trades  
1 2 3 4 5 ... N
```

```
Bootstrap 1  
3 3 8 1 7 ... N
```

```
Bootstrap 2  
2 9 9 4 1 ... N
```

```
...
```

Kemudian hitung metric untuk setiap sample.

AE.34. Distribusi Sharpe

Misalkan:

$$1, 2, \dots$$

dari bootstrap samples.

Kita memperoleh distribusi:

$$\square$$

Kemudian kita dapat mengambil:

$$5 = 25, 5$$

AE.35. Tetapi bootstrap biasa punya masalah

Return finansial memiliki:

- autocorrelation,
- volatility clustering,
- regime dependence.

Random resampling individual trade dapat menghancurkan struktur temporal.

Karena itu untuk beberapa analisis kita dapat menggunakan:

$$\boxed{tt}$$

Kita melakukan resampling blok waktu, bukan observasi individual secara independen.

AE.36. Sharpe Ratio

Untuk return:

$$1, \dots$$

Sharpe:

$$= -$$

Untuk short-term trading, sering relatif kecil pada horizon sangat pendek, tetapi jangan mengabaikannya secara konseptual.

AE.37. Masalah Sharpe klasik

Sharpe tinggi tidak selalu berarti strategi bagus.

Misalnya:

$$=$$

Tetapi diperoleh dari:

- 1000 konfigurasi strategy,
- 100 parameter combinations,
- berbagai TP,
- berbagai SL,
- berbagai assets.

Kalau kita memilih yang terbaik setelah melihat semua hasil, kita mengalami:

$$tt$$

AE.38. Ini sangat berbahaya

Bayangkan kita mencoba:

$$10,000$$

strategi random.

Walaupun semua sebenarnya tidak punya edge, sebagian hampir pasti akan terlihat luar biasa.

Ini adalah:

$$t$$

atau:

$$t$$

AE.39. Contoh

Kita mencoba:

$$1000$$

kombinasi:

$$P, , t, t$$

Strategi terbaik memiliki:

$$= 2$$

Pertanyaannya bukan:

"Apakah Sharpe 3.2?"

Tetapi:

"Berapa banyak strategi yang dicoba sebelum mendapatkan 3.2?"

Ini adalah pertanyaan yang jauh lebih penting.

AE.40. Deflated Sharpe Ratio

Deflated Sharpe Ratio mencoba mengoreksi interpretasi Sharpe terhadap:

- multiple testing,
- non-normality,
- selection bias.

Secara konseptual:

$$= P$$

Dengan benchmark yang mempertimbangkan jumlah trial dan statistik return.

Kita tidak boleh menganggap:

$$= 2$$

sebagai bukti kuat jika kita telah mencoba ribuan model.

AE.41. Probabilistic Sharpe Ratio

PSR menjawab:

Seberapa besar probabilitas bahwa true Sharpe lebih besar dari threshold tertentu?

Secara sederhana:

$$P = P_t$$

Ini lebih informatif daripada sekadar:

$$= 1$$

AE.42. Sharpe saja juga tidak cukup

Kita akan menyimpan minimal:

$$\{t, , t, , , t, , , , t\}$$

AE.43. Max Drawdown

Equity:

$$t$$

Running maximum:

$$t = t$$

Drawdown:

$$t = t - t$$

Maximum drawdown:

$$= t^t$$

AE.44. Mengapa drawdown penting untuk \$1 risk?

Walaupun setiap trade:

$$= 1,$$

kita masih dapat mengalami:

$$10, 20, 0,$$

loss berturut-turut.

Jadi fixed-dollar risk tidak berarti fixed portfolio risk.

AE.45. Risk of ruin

Jika probabilitas loss:

$$= 1 - ,$$

dan kita memiliki capital:

Dengan risk per trade:

Maka jumlah unit risk:

$$=$$

Misalnya:

$$= 1000, = 1$$

Kita memiliki:

$$1000$$

Ini memberikan ruang drawdown yang jauh lebih besar daripada jika:

$$= 20$$

AE.46. Tetapi \$1 risk tetap harus adaptif

Risk budget dapat ditulis:

$$t = t, tt_t, t, t$$

Misalnya ketika drawdown tinggi:

$$1$$

Jadi risk diturunkan.

AE.47. Model confidence

Kita dapat mendefinisikan confidence berdasarkan:

,

uncertainty:

,

dan calibration error.

Misalnya:

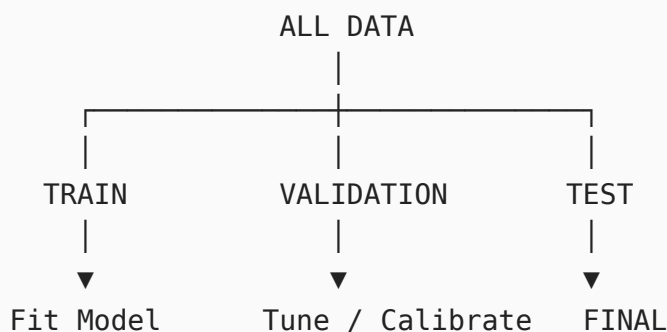
$$t = t, t, t_t$$

Ini lebih baik daripada:

$$= -05$$

AE.48. Out-of-sample hierarchy

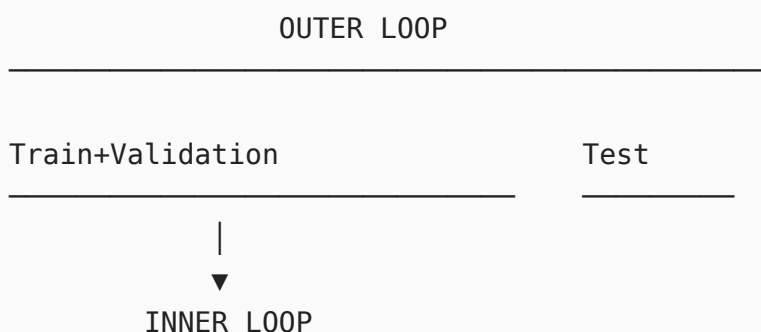
Saya menyarankan struktur dataset kita menjadi:



Tetapi untuk time series, pembagian ini dilakukan **secara temporal**, bukan random.

AE.49. Bahkan lebih baik: nested walk-forward

Misalnya:



model selection
hyperparameter
calibration

Dengan demikian final test benar-benar untouched.

AE.50. Contoh konkret

Misalkan data:

2021 – 01 – 01 2025 – 12 – 1

Kita dapat menggunakan:

Fold 1

Train:

2021

Validation:

2022

Test:

2021

Fold 2

Train:

2021 – 2022

Validation:

2021 – 2

Test:

202

Dan seterusnya.

Dalam implementasi sebenarnya, panjang window akan disesuaikan dengan horizon dan jumlah data.

AE.51. Feature leakage

Ini salah satu jebakan terbesar.

Misalkan kita menghitung:

$$0$$

menggunakan seluruh bulan.

Jika trade terjadi pada:

$$15,$$

tetapi rolling mean menggunakan data sampai:

$$1,$$

maka kita telah memasukkan masa depan.

AE.52. Rule sederhana

Untuk setiap timestamp t :

$$t_t = t_t$$

Bukan:

$$t_{t\Delta}$$

AE.53. Normalization juga dapat menyebabkan leakage

Misalkan:

$$t = t -$$

dengan , dihitung menggunakan **seluruh dataset**.

Maka informasi future masuk ke normalization.

Seharusnya:

$$t, t$$

dihitung hanya dari training history yang tersedia pada waktu tersebut.

AE.54. Bahkan PCA bisa leakage

Misalkan kita melakukan PCA terhadap covariance matrix:

menggunakan seluruh data 2021–2025.

Kemudian menggunakan principal components untuk memprediksi 2022.

Itu leakage.

PCA harus di-fit hanya pada:

$$t$$

Kemudian transform:

$$t_t, t_t$$

tanpa melakukan fitting ulang.

AE.55. Factor model juga harus temporal

Misalkan:

$$=$$

Parameter:

harus diperoleh dari data yang tersedia sebelum timestamp prediction.

Bukan dari seluruh periode.

AE.56. Survivorship bias

Ini khususnya penting untuk saham internasional.

Misalkan kita mengambil:

"100 saham terbesar saat ini."

Lalu backtest ke tahun 2021.

Masalahnya:

perusahaan yang sekarang besar sudah diketahui **ex post**.

Perusahaan yang bangkrut/delisted mungkin hilang dari dataset.

Ini menghasilkan:



AE.57. Universe harus point-in-time

Idealnya universe pada waktu t :

$$t$$

harus mencerminkan aset yang benar-benar dapat dipilih investor pada saat itu.

Jadi:

2021 202

Ini sangat penting ketika nanti sistem kita berpindah dari crypto ke:

- US equities,
- European equities,
- Asian equities.

AE.58. Corporate actions

Untuk saham:

- stock split,
- dividend,
- merger,
- delisting.

harus diperhitungkan.

Kalau tidak:

$$t_t$$

bisa salah.

AE.59. Crypto juga punya masalah tersendiri

Untuk crypto:

- token listing,
- delisting,
- perubahan market structure,
- exchange migration,
- perpetual funding,

- liquidity fragmentation.

Karena itu:

$$t$$

juga harus point-in-time.

AE.60. Backtest tidak boleh menggunakan candle close sebagai fill sempurna

Misalkan candle:

$$= 100$$

$$= 105$$

$$= 5$$

$$= 10$$

Jika kita memiliki:

$$P = 104$$

dan:

$$= ,$$

kita tahu keduanya berada dalam candle.

Tetapi kita tidak tahu:

$$P$$

atau:

$$P$$

Intrabar ordering harus diketahui atau dimodelkan secara konservatif.

AE.61. Ini sangat penting untuk first-passage model

Karena kita justru mendefinisikan:

$$\tau = \tau_P, \tau$$

Jika data hanya OHLC, kita mungkin tidak mengetahui:

$$\tau_P \tau$$

Maka backtest harus:

- menggunakan data intrabar yang cukup granular, atau
- menggunakan asumsi konservatif.

AE.62. Execution realism

P&L backtest:

$$P$$

harus diganti menjadi:

$$P_t = P - \dots - t -$$

AE.63. Slippage model

Salah satu model sederhana:

$$= t$$

Atau:

$$P_t = P$$

Dengan:

- : order size,
- : available depth,
- : market impact coefficient.

AE.64. Menguji robustness terhadap biaya

Kita tidak hanya menggunakan:

$$t = 0$$

Kita harus melakukan stress test:

$$, 15, 2,$$

Jika strategy hanya profitable pada:

$$= 01$$

tetapi langsung hancur pada:

$$= 1,$$

maka edge-nya sangat rapuh.

AE.65. Sensitivity analysis

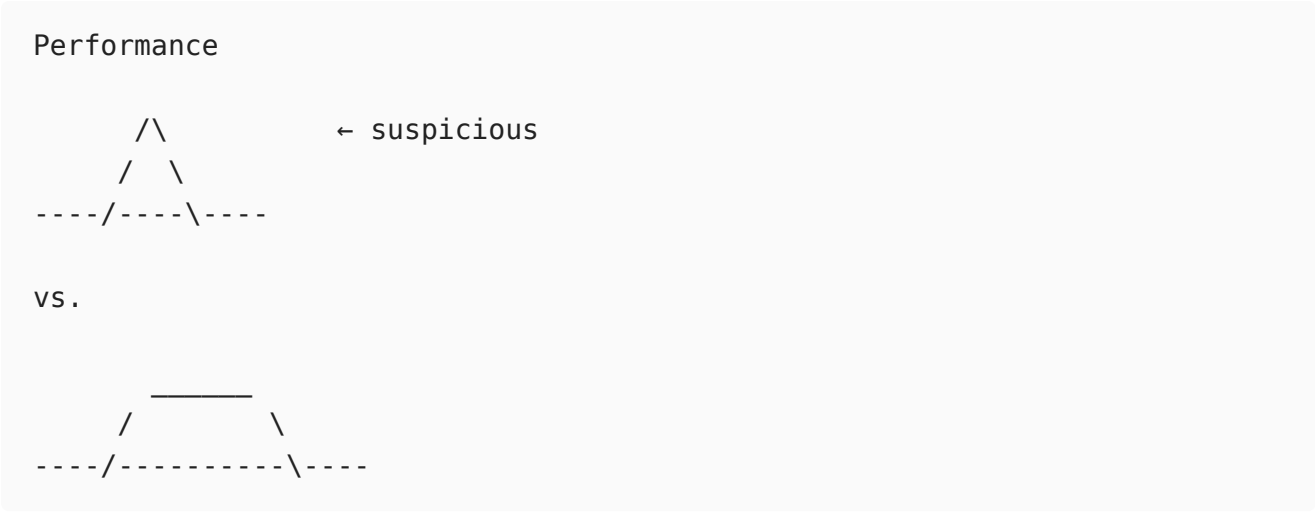
Untuk parameter:

$$= , , P, t, ,$$

kita hitung:

$$P$$

Strategi yang sehat biasanya memiliki **performance basin**, bukan satu titik tajam.



Puncak yang sangat tajam dapat menunjukkan overfitting.

AE.66. Parameter stability

Misalnya:

$$P = 20$$

menghasilkan Sharpe:

$$25$$

Tetapi:

$$P = 202$$

menghasilkan:

$$0$$

Ini mencurigakan.

Lebih baik jika:

$$P \in 1, 24$$

semuanya masih reasonably profitable.

AE.67. Regime robustness

Kita juga harus membagi hasil berdasarkan regime:

''''

Misalnya:

Regime	Sharpe	EV
Bull	1.8	+
Bear	0.9	+
Sideways	0.2	~0
High Vol	1.4	+
Low Vol	1.1	+

Ini jauh lebih informatif daripada:

$$= 14$$

AE.68. Cross-asset robustness

Karena tujuan kita bukan hanya crypto, kita harus menguji:

t

vs.

t

vs.

vs.

dan seterusnya.

Tetapi **jangan memaksa satu model memiliki parameter identik untuk semuanya.**

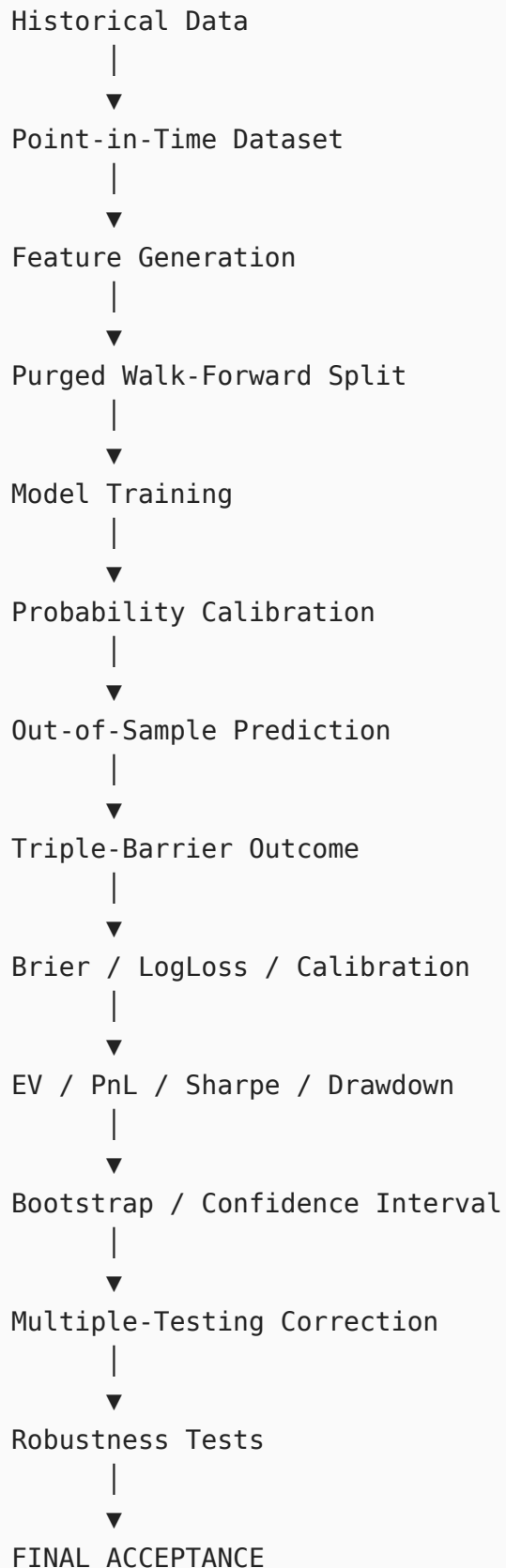
Yang sebaiknya universal adalah:

tt

Bukan selalu parameter.

AE.69. Statistical validation pipeline

Maka AE akhirnya memiliki pipeline:



AE.70. Final acceptance criteria

Saya menyarankan kita **tidak** mendefinisikan:

"Sharpe > 1 berarti strategi diterima."

Terlalu sederhana.

Sebaliknya, strategy candidate harus melewati beberapa layer.

Statistical

t

t

t

Economic

0

tP 0

t

Robustness

ttt

$Pttt$

tt

Statistical significance

P

dan confidence interval yang masuk akal.

AE.71. Decision gate

Kita akhirnya dapat membuat:

$$t = t \quad 0_t \quad t_t$$

Tetapi bahkan ini belum cukup.

Untuk live deployment kita dapat menambahkan:

$$t_t = 1$$

Sehingga:

$$t = t \quad t \quad t$$

AE.72. Model health

Model dapat mengalami degradation.

Misalnya:

Training:

$$= 012$$

Live:

$$= 025$$

Maka:

$$t$$

Kita tidak boleh terus meningkatkan position size hanya karena historical backtest bagus.

AE.73. Population Stability

Kita juga harus memeriksa apakah distribusi feature berubah.

Misalnya:

$$t$$

memiliki distribusi:

$$P_t$$

Sedangkan live:

$$P$$

Jika:

$$P \neq P_t,$$

maka terjadi:

$$ttt$$

AE.74. Concept drift

Lebih buruk lagi:

P

sendiri dapat berubah.

Misalnya dulu:

0

berarti bullish.

Tetapi setelah market structure berubah:

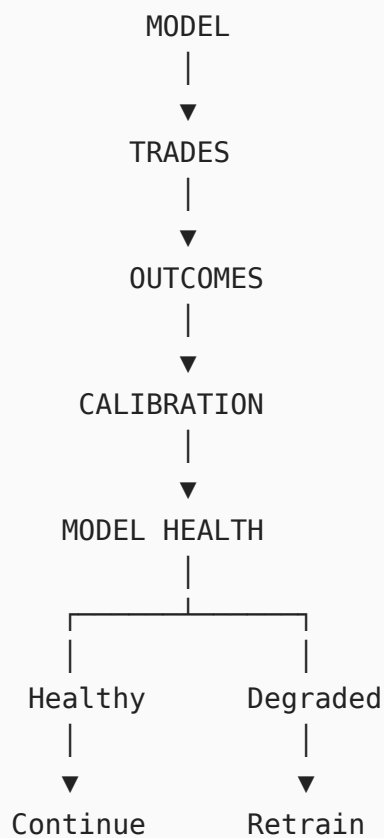
0

tidak lagi memiliki predictive power yang sama.

Ini:

tt

AE.75. Maka sistem kita harus memiliki feedback loop



Ini membuat sistem **adaptif tetapi tetap terkontrol**.

AE.76. Dan ini mengubah definisi backtest

Backtest kita bukan:

"Jalankan strategy pada data lalu lihat profit."

Tetapi:

$$tt = t \quad tttt \quad t \quad tttt$$

AE.77. Hierarki bukti

Saya ingin kita menggunakan hierarki berikut:

$$tt \quad t \quad P \quad t$$

Semakin ke kanan, semakin kuat bukti bahwa sistem benar-benar bekerja di dunia nyata.

Tidak ada backtest yang bisa membuktikan strategy akan selalu profitable.

AE.78. Hubungan dengan quantum optimization kita

Ini juga penting untuk tujuan besar kita.

Quantum optimizer:

tidak boleh diberi tugas:

"cari parameter yang membuat backtest paling profitable."

Itu akan sangat mudah menghasilkan overfitting.

Sebaliknya:

$$tttt$$

harus berada **di luar optimizer**.

Quantum/classical optimizer hanya menyelesaikan:

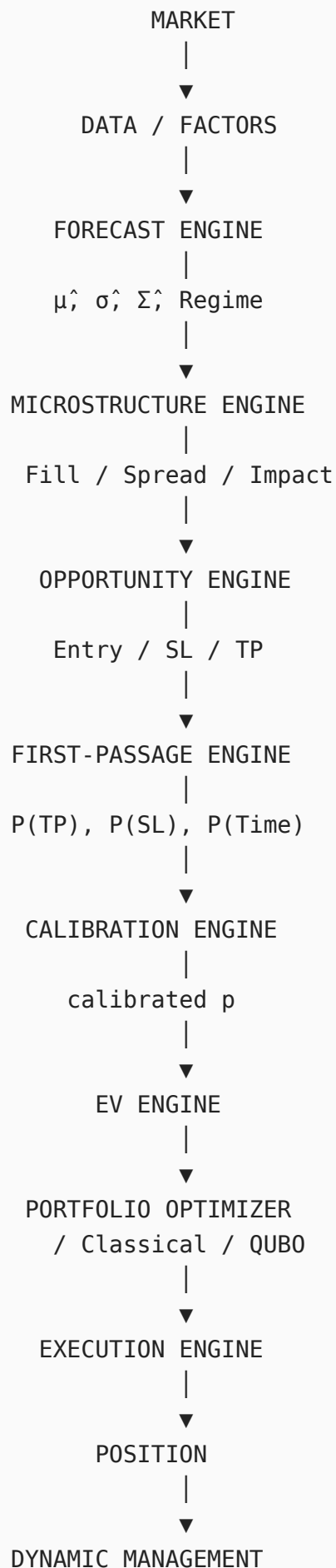
$$\square$$

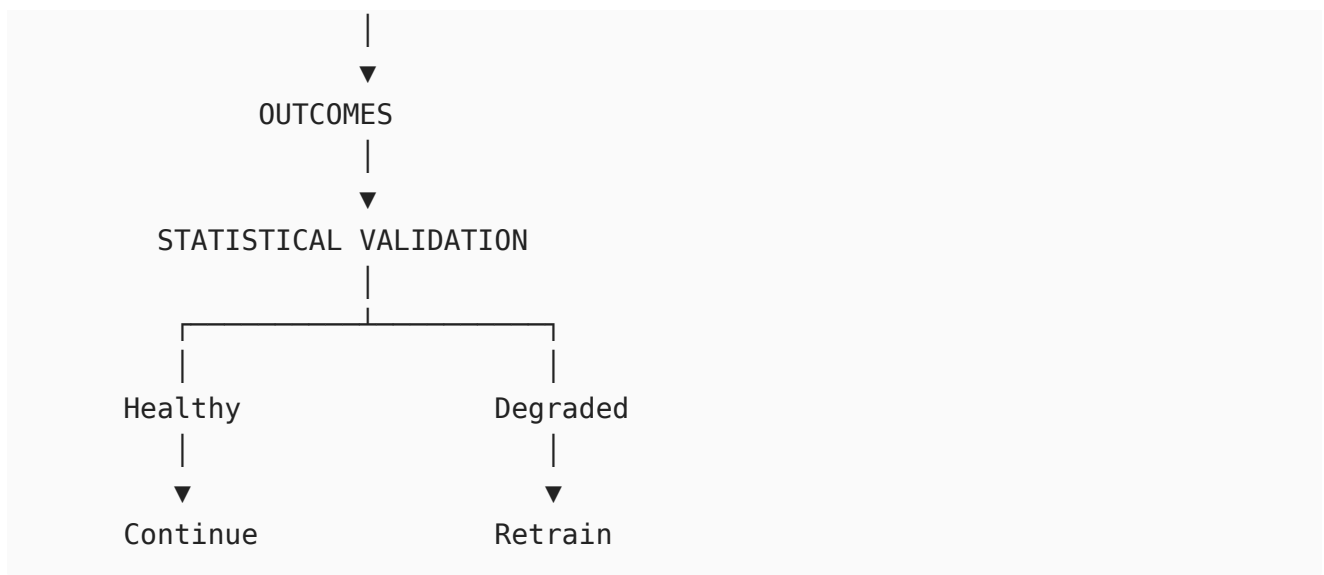
Validasi menentukan apakah:

layak dipercaya.

AE.79. Arsitektur final sampai Tahap AE

Kita sekarang memiliki:





AE.80. Persamaan "kontrak" antar-engine

Ini menurut saya merupakan salah satu hasil paling penting dari Tahap AE.

Forecast engine memberikan:

$$y, y$$

Opportunity engine mengubahnya menjadi:

$$= P, y$$

Calibration engine menghasilkan:

$$= t$$

EV engine:

$$= P - -$$

Optimizer:

$$=$$

dan validator kemudian menguji apakah:

$$\square$$

benar-benar terealisasi secara out-of-sample.

AE.81. Prinsip yang harus kita pegang

Ada satu prinsip yang menurut saya harus menjadi **aturan fundamental** proyek ini:

□

Dan bahkan lebih spesifik:

□

Bukan:

Perbedaan tersebut sangat besar.

AE.82. Posisi kita sekarang

Sampai Tahap AD kita sudah menjawab:

"Bagaimana menentukan trade terbaik?"

Tahap AE menjawab:

"Bagaimana memastikan bahwa probabilitas dan edge yang digunakan untuk menentukan trade tersebut memang memiliki dasar statistik yang dapat dipercaya?"

Dengan demikian kita sudah memiliki dua komponen:

□

dan:

□

Dan **Evidence Engine** inilah yang mencegah seluruh sistem berubah menjadi overfit backtest yang sangat kompleks.

AE.83. Tahap berikutnya: AF

Setelah ini kita sudah bisa masuk ke bagian yang jauh lebih menarik: **AF — Portfolio-Level Trade Selection & Cross-Asset Risk Optimization**.

Di sana masalahnya berubah dari:

menjadi:

500,

Karena misalnya pada pukul 10:00 sistem menemukan:

20 100 0

Masing-masing mungkin memiliki:

0

Tetapi **kita tidak boleh mengambil semuanya**, karena exposure-nya bisa sangat berkorelasi.

Di sinilah:

,

factor exposure,

,

drawdown constraint,

dan akhirnya **QUBO/Ising/QAOA** yang sejak awal menjadi tujuan kita mulai benar-benar bertemu dengan **short-term quantitative trading** secara natural.

Prompt:

8/17/2026, 10:21:13 PM

lanjut Tahap AF

Response:

8/17/2026, 10:21:14 PM

Remembering

Continued quantitative trading workflow with Stage AF objectives and risk constraints

Worked for 4s

Tahap AF - Portfolio-Level Trade Selection & Cross-Asset Risk Optimization

Kita sekarang masuk ke salah satu bagian **paling penting** dari arsitektur yang kita bangun.

Sampai Tahap AE, kita sudah dapat mengatakan apakah **satu trade secara individual** mempunyai edge yang layak:

0

Tetapi dalam trading nyata, masalahnya bukan memilih satu trade.

Misalkan pada satu waktu sistem menemukan:

- 15 peluang crypto,
- 80 saham AS,

- 20 ETF,
- beberapa aset Eropa/Asia,
- gold,
- USD-related instruments,

dan 40 di antaranya memiliki:

$$0$$

Kita **tidak otomatis mengambil 40 trade tersebut**.

Karena bisa saja 20 trade sebenarnya merupakan taruhan yang hampir sama:

$$, , , t_1, t_2,$$

semuanya akan jatuh ketika satu faktor market mengalami shock.

Maka pertanyaan Tahap AF adalah:

$$\square$$

Dan di sinilah **portfolio optimization, QUBO, Ising, Max-Cut, dan QAOA** mulai mempunyai fungsi yang sangat natural dalam sistem kita.

AF.1. Perubahan paradigma

Sebelumnya:

$$\square$$

sekarang:

$$\square$$

Misalkan terdapat opportunity:

$$= \{1, 2, \dots\}$$

Untuk setiap trade , kita mempunyai:

$$,$$

risk:

$$,$$

dan exposure terhadap trade lain:

Maka kita mencari:

$$\square$$

dengan:

$$= \begin{cases} 1, & 0, \end{cases}$$

Jadi secara langsung kita telah mendapatkan **binary optimization problem**.

AF.2. Mengapa binary variable?

Karena pada tahap awal kita tidak perlu menentukan berapa besar posisi.

Kita terlebih dahulu menjawab:

| Apakah trade ini diambil atau tidak?

Dengan:

$$\in \{0, 1\}$$

Kemudian tahap berikutnya menentukan:

atau position size.

Jadi kita memisahkan:

$$t$$

dari:

$$Pt$$

Ini akan sangat membantu ketika kita ingin mengubah masalah menjadi QUBO.

AF.3. Expected return portfolio

Jika trade mempunyai expected return:

$$,$$

maka expected portfolio return:

$$= \sum_{i=1}^n$$

Dalam bentuk vektor:

$$=$$

AF.4. Portfolio variance

Sekarang masalah menjadi lebih menarik.

Jika:

$$= \sigma^2$$

maka:

$$= \sigma^2$$

Ekspansinya:

$$= \sigma^2$$

Karena:

$$= \sigma^2$$

untuk binary variable, maka:

$$= \sigma^2$$

Dan di sinilah coupling antar-trade muncul.

AF.5. Interpretasi fisiknya

Perhatikan:

Jika:

$$= \sigma^2$$

maka mengambil σ^2 dan sekaligus meningkatkan variance.

Jika:

$$= \sigma^2$$

maka mengambil keduanya dapat memberikan diversification benefit.

Jadi:

$$= \sigma^2$$

secara matematis berperan sebagai **interaction energy** antar opportunity.

Ini akan sangat berguna untuk pemetaan ke Ising.

AF.6. Tetapi covariance saja belum cukup

Untuk sistem kita, covariance antar aset bukan satu-satunya sumber risiko.

Misalkan:

$$=$$

Dengan:

$$= t$$

Maka:

$$\boxed{=}$$

Ini adalah **factor model**.

AF.7. Mengapa factor model penting?

Misalkan:

$$, = 05$$

Itu sudah menunjukkan korelasi.

Tetapi kita ingin mengetahui **mengapa**.

Mungkin keduanya mempunyai exposure besar terhadap:

$$t$$

Demikian pula:

$$P, ,$$

mungkin semuanya memiliki exposure terhadap:

Sementara:

dan:

mempunyai exposure terhadap faktor makro yang berbeda.

AF.8. Factor exposure portfolio

Jika trade memiliki:

,

maka exposure portfolio terhadap factor :

$$=$$

Dalam bentuk matriks:

$$=$$

Kita kemudian dapat memberikan constraint:

AF.9. Contoh

Misalkan:

$$= 12$$

terhadap crypto factor.

Dan:

$$= 15$$

Jika:

$$= = 1,$$

maka:

$$_t = 2$$

Kalau batas kita:

$$_t = 2,$$

maka kombinasi tersebut tidak diperbolehkan.

AF.10. Ini jauh lebih baik daripada sekadar membatasi jumlah trade

Misalnya kita menetapkan:

Ini tidak menjamin diversification.

Sepuluh trade bisa saja:

$$1, 1, 1, 1, 1, 1, 1, 1, 1, 1$$

yang semuanya sangat terpapar faktor crypto yang sama.

Sebaliknya:

$$5t \quad t \quad 1 \quad 1$$

mungkin mempunyai diversification jauh lebih baik.

AF.11. Objective dasar

Kita dapat menggunakan objective:

$$-$$

Dengan:

$$0$$

sebagai risk-aversion parameter.

Ini merupakan bentuk discrete Markowitz optimization.

AF.12. Hubungan dengan Markowitz

Markowitz klasik:

$$-$$

Tetapi sekarang:

$$\in \{0, 1\}$$

Maka:

$$t \quad t \quad P \quad t \quad t$$

Dan masalah ini dapat menjadi combinatorial optimization.

AF.13. Mengapa ini menarik untuk QUBO?

Objective:

—

sudah merupakan quadratic function terhadap binary variables.

QUBO didefinisikan sebagai:

$$\in \{0,1\}$$

Jadi secara struktural:

$$P_{tt}$$

AF.14. Sign convention

Karena kita ingin memaksimalkan:

$$-$$

kita dapat mengubahnya menjadi minimisasi:

$$-$$

Maka:

dibangun dari:

$$-$$

Dengan perhatian khusus terhadap konvensi diagonal/off-diagonal QUBO yang digunakan implementasi kita.

AF.15. Constraint jumlah trade

Katakanlah maksimal:

trade boleh dibuka.

Constraint:

$$=$$

Untuk mengubahnya menjadi unconstrained QUBO, kita tambahkan penalty:

$$P -$$

Sehingga:

$$= - \frac{P}{2} - \frac{1}{2} x^2$$

AF.16. Ekspansi penalty

Perhatikan:

$$\frac{1}{2} x^2$$

Kita punya:

$$= \frac{1}{2} x^2 - \frac{1}{2} x^2$$

Karena:

$$\frac{1}{2} x^2 = \frac{1}{2} x^2$$

maka:

$$= \frac{1}{2} x^2 - \frac{1}{2} x^2$$

Jadi penalty tetap quadratic.

AF.17. Constraint factor exposure

Misalkan:

Kita dapat menggunakan slack variables atau penalty.

Untuk constraint equality sederhana:

$$= 0,$$

gunakan:

$$P - \frac{1}{2} x^2$$

Untuk inequality:

$$,$$

kita dapat memperkenalkan slack:

$$=$$

AF.18. Risk budget

Kita dapat membatasi:

$$\boxed{2}$$

Secara langsung ini merupakan quadratic constraint.

Dengan penalty:

$$P - \frac{1}{2}$$

Namun ini menghasilkan polynomial degree 4 jika diekspansi.

Jadi untuk QUBO murni kita harus berhati-hati.

AF.19. Solusi yang lebih baik

Kita tidak harus memaksakan semua constraint menjadi QUBO sekaligus.

Kita dapat menggunakan:

$$\boxed{tt}$$

Contohnya:

Layer 1

Filter opportunity:

Layer 2

Risk screening:

Layer 3

Factor exposure screening.

Layer 4

QUBO:

—

Ini membuat problem jauh lebih tractable.

AF.20. Hard constraint versus soft constraint

Kita harus membedakan:

Hard constraint

Tidak boleh dilanggar.

Contoh:

Soft constraint

Boleh dilanggar dengan penalty.

Contoh:

$$t$$

Maka objective:

$$= - - Pt$$

AF.21. Cross-asset factor matrix

Sekarang kita mulai membangun struktur yang lebih besar.

Misalkan terdapat:

$$= 100$$

trade opportunities.

Dan:

$$=$$

factor.

Kita memiliki:

$$\in^{100}$$

Baris:

adalah exposure trade .

Kolom:

adalah factor.

AF.22. Faktor yang kita inginkan

Untuk desain awal sistem universal, saya akan menggunakan kelompok faktor:

$$\{t, t, t, , , t, tt, t\}$$

Kemudian universe tertentu dapat menambahkan faktor spesifik.

AF.23. Contoh

Untuk BTC:

$$= 11014-02010151$$

Untuk gold:

$$= 0100-0512-00405$$

Angka ini hanya ilustrasi; dalam sistem nyata semuanya harus diestimasi secara point-in-time.

AF.24. Faktor USD

Ini sangat relevan dengan tujuanmu.

Misalkan:

merepresentasikan kekuatan USD.

Kita dapat memasukkan:

,

Jika portfolio terlalu long terhadap aset yang sensitif terhadap pelemahan USD, maka kita dapat membatasi:

$\square$

AF.25. Gold sebagai macro hedge

Demikian pula:

Gold bukan hanya sebuah aset untuk ditrade.

Ia juga dapat berfungsi sebagai:

t

atau:

tt

Ini merupakan perbedaan penting antara:

| trading gold

dan:

| menggunakan gold sebagai komponen risk model.

AF.26. Covariance decomposition

Factor model memberi:

$=$

Jika:

$=$

dan:

$= ,$

maka:

$=$

Ini sangat penting.

Kita tidak harus mengestimasi covariance antar 500 aset secara langsung.

Kita dapat memecahnya menjadi:

$$\begin{bmatrix} t & t \end{bmatrix}$$

AF.27. Mengapa ini sangat cocok untuk pasar besar?

Misalkan:

$$= 1000$$

Covariance matrix:

$$1000 \quad 1000$$

Jumlah elemen:

$$10$$

Estimasi langsung dapat menjadi noisy.

Tetapi jika:

$$= 20$$

factor, kita dapat menggunakan:

$$100020$$

Jauh lebih manageable.

AF.28. Risk portfolio factor

Variance:

$$=$$

Pisahkan:

$$=$$

Karena:

adalah portfolio factor exposure, maka:

$$\begin{bmatrix} = \end{bmatrix}$$

Ini memberikan interpretasi yang sangat bagus:

$$Pt = t \quad t$$

AF.29. Trade clustering

Kita dapat mendefinisikan similarity:

$$=$$

Jika:

$$0,$$

maka trade , memiliki exposure serupa.

Kita dapat memberikan penalty:

$$\square$$

Ini akan mendorong diversification.

AF.30. Hubungan dengan Max-Cut

Menariknya, jika kita memiliki graph:

$$= , ,$$

dengan:

sebagai similarity/exposure, kita dapat menginginkan agar node-node yang sangat berkorelasi tidak dipilih bersama.

Ini dapat diformulasikan sebagai graph optimization.

Max-Cut klasik:

$$\square 1 -$$

atau equivalent formulations.

Jadi:

$$\square tt$$

AF.31. Mengapa kita tidak langsung menggunakan Max-Cut?

Karena tujuan utama kita bukan:

memotong graph.

Tujuan kita:

$$t t t t$$

Max-Cut hanya menjadi salah satu substructure yang dapat muncul.

AF.32. QUBO general form

Kita dapat menulis:

$$= - P$$

Interpretasinya:

adalah individual attractiveness.

Sedangkan:

adalah interaction.

AF.33. Dalam sistem kita

Kita dapat membuat:

$$= -_2$$

dan:

$$= 2$$

Sehingga:

$$= - -_2 P t$$

AF.34. Ini adalah titik penting

Perhatikan struktur:

berasal dari:

probabilistic trading model.

Sedangkan:

berasal dari:

cross-trade interaction.

Jadi optimizer tidak lagi hanya bertanya:

"Trade mana paling profitable?"

Ia bertanya:

"Trade mana yang paling menguntungkan setelah memperhitungkan apa yang sudah kita pilih?"

Itulah perbedaan fundamental antara:

tt

dan:

P_{ttt}

AF.35. Position sizing setelah selection

Setelah QUBO memilih:

,

kita belum selesai.

Kita kemudian menyelesaikan continuous optimization:

$-$

subject to:

$= 0 = 0$

Dengan:

0

untuk long-only.

Atau:

$\in$

untuk long-short.

AF.36. Ini menghasilkan hybrid architecture

Dan ini menurut saya jauh lebih baik daripada mencoba memaksa seluruh masalah menjadi QAOA.

Kita gunakan:

$$\boxed{tttt}$$

untuk:

dan:

$$\boxed{ttt}$$

untuk:

AF.37. Secara keseluruhan

$$\boxed{t \ P_t \ t \ tt \ t \ t \ t}$$

AF.38. Constraint risk \$1

Sekarang kita hubungkan dengan desain awal kita.

Jika setiap trade harus memiliki maximum loss:

$$= 1,$$

maka stop-loss distance:

$$= P_t - P$$

Position size:

$$\boxed{= 1}$$

dengan conversion factor sesuai instrument.

Dengan demikian:

AF.39. Tetapi portfolio risk tidak otomatis \$1

Misalnya kita mengambil 10 trade:

$$= 1$$

Worst-case nominal loss:

$$10$$

Tetapi jika semuanya sangat correlated, probabilitas mengalami 10 loss sekaligus bisa jauh lebih besar daripada portfolio yang benar-benar diversified.

Jadi kita membutuhkan:

$$P_t$$

di atas:

$$\square$$

AF.40. Portfolio risk budget

Kita dapat menentukan:

$$t$$

misalnya dalam unit .

Kemudian:

$$t$$

Ini lebih bermakna daripada hanya:

$$= 1$$

AF.41. Conditional Value-at-Risk

VaR menjawab:

"Pada confidence level , berapa threshold loss?"

Misalnya:

$$\alpha = 10$$

Tetapi VaR tidak mengatakan seberapa buruk kondisi setelah threshold tersebut dilewati.

CVaR menjawab:

$$=$$

Jadi:

$$\square$$

lebih informatif untuk tail risk.

AF.42. Constraint CVaR

Kita dapat menentukan:

$$5$$

Ini menjadi salah satu constraint portfolio-level.

Untuk implementasi optimization yang lebih kompleks, CVaR dapat diformulasikan menggunakan auxiliary variable.

AF.43. Scenario-based risk

Daripada hanya menggunakan covariance matrix, kita dapat menghasilkan:

scenario.

Misalnya:

$$= 1, ,$$

Untuk setiap scenario:

Portfolio loss:

$$= -$$

Kemudian:

$$,$$

dihitung dari distribusi:

$$\{^1, , \}$$

AF.44. Ini sangat penting untuk market shock

Misalnya scenario:

Scenario A

BTC:

—

Scenario B

USD:

2

Scenario C

Gold:

4

Scenario D

Equity:

−5

Kita ingin mengetahui:

untuk portfolio yang dipilih.

AF.45. Stress testing

Kita dapat secara eksplisit membuat:

$$\boxed{t}$$

seperti:

$$= -10$$

$$P = -5$$

$$= 2$$

$$=$$

dan melihat apakah portfolio masih berada dalam risk budget.

AF.46. Ini menghubungkan sistem kita dengan macro regime

Sebelumnya kita memasukkan:

'''

Sekarang mereka tidak hanya menjadi feature.

Mereka dapat menjadi:

P_t

Misalnya:

mengindikasikan crypto risk appetite.

mengindikasikan dollar liquidity.

mengindikasikan defensive/macro demand.

Jadi macro factors memengaruhi:

P_t

dan sekaligus:

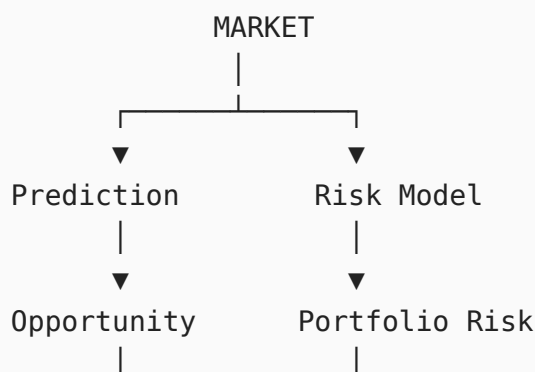
□

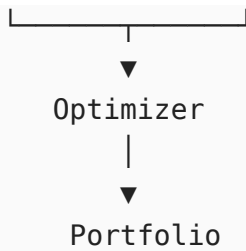
AF.47. Ini membuat arsitektur dua arah

Bukan:

Market → Prediction → Trade

tetapi:





AF.48. Dynamic portfolio optimization

Masalah ini harus diselesaikan setiap:

$$t$$

Jadi:

$$t = t$$

Karena:

$$t, t, t$$

berubah seiring waktu.

Dengan demikian kita sebenarnya memiliki:

$$\square$$

AF.49. Warm-start

Ini sangat cocok dengan pengalamanmu pada EPG dan VQE.

Jika pada:

$$t - 1$$

kita mempunyai:

$$t-1,$$

maka pada:

$$t$$

solusi biasanya tidak berubah secara ekstrem.

Kita dapat menggunakan:

$$t-1$$

sebagai warm-start.

AF.50. Ini dapat mengurangi computational burden

Daripada menyelesaikan:

$$t$$

dari nol setiap saat, kita memulai dari:

$$t-1$$

Kemudian hanya melakukan adjustment terhadap opportunity yang berubah.

Ini sangat cocok untuk short-term trading.

AF.51. Hysteresis

Ada masalah lain.

Jika optimizer terlalu sensitif:

$$t$$

bisa berubah:

$$1010101010$$

setiap periode.

Ini menghasilkan:

- excessive turnover,
- fee,
- slippage.

Maka kita tambahkan switching penalty:

$$t, t-1$$

AF.52. Binary switching penalty

Karena:

$$t, t-1 \in \{0, 1\},$$

kita dapat menulis:

$$t - t-1$$

sebagai:

$$t \quad t-1 - 2tt-1$$

Jadi tetap quadratic.

Ini sangat menarik karena:

$$t$$

juga dapat masuk ke QUBO.

AF.53. Objective lengkap sementara

Kita sekarang dapat menulis:

$$t = - \quad , t \quad t \quad , t-1 \quad P \quad - \quad ^2$$

Dengan:

- : expected edge,
- : covariance,
- : factor-risk penalty,
- : turnover penalty,
- : maximum number of positions.

AF.54. Dan sekarang kita benar-benar memiliki QUBO trading

Secara konseptual:

$$t = t \quad t \quad tt$$

Ini bukan lagi QUBO contoh akademik.

Ia berasal langsung dari:

$$tt$$

AF.55. Mapping ke Ising

Sekarang gunakan transformasi standar:

$$= 1 \ 2$$

dengan:

$$\in \{-1, 1\}$$

Kemudian:

$$= 1 \ 4$$

Maka:

dapat ditulis:

$$=$$

AF.56. Hamiltonian kuantum

Ganti:

dengan Pauli :

$$=$$

Dan ini persis bentuk Ising Hamiltonian yang sebelumnya sudah kita gunakan dalam penelitianmu.

AF.57. Sekarang QAOA memiliki makna trading yang jelas

QAOA menggunakan:

$$= -$$

dan:

$$= -$$

Dengan:

$$=$$

State:

$$, = 11$$

AF.58. Interpretasi state QAOA

State akhir:

$$=$$

Probabilitas:

$$P = 2$$

Artinya:

QAOA tidak sekadar menghasilkan satu portfolio; ia menghasilkan distribusi kandidat portfolio.

Ini sangat menarik untuk sistem kita.

AF.59. Candidate portfolio distribution

Misalnya QAOA menghasilkan:

$$P_1 = 0.42$$

$$P_2 = 0.1$$

$$P = 0.11$$

dan seterusnya.

Kita dapat mengambil:

$$= P$$

Tetapi kita juga dapat mengambil beberapa kandidat:

$$\{^1, ^2, , \}$$

AF.60. Ini membuka pendekatan yang lebih menarik

Kita tidak harus berkata:

QAOA menghasilkan portfolio $\rightarrow$ langsung trading.

Kita dapat:

$$\boxed{tPt \ t \ t}$$

Jadi quantum optimizer menjadi **candidate generator**.

AF.61. Ini juga menyelesaikan masalah NISQ

Untuk sistem nyata, QAOA tidak harus menghasilkan solusi exact.

Kita cukup mencari:

yang:

cukup rendah.

Kemudian classical validator melakukan:

Dengan demikian quantum hardware tidak menjadi single point of failure.

AF.62. Objective final yang lebih aman

Saya menyarankan:

- ☐
- ☐

Artinya QAOA boleh mengusulkan portfolio, tetapi risk engine memiliki hak veto.

AF.63. Risk veto

Misalnya QAOA memberikan:

Tetapi:

5

Maka:

t

Tidak peduli seberapa rendah QUBO energy-nya.

AF.64. Ini sangat penting secara finansial

Kita jangan membuat:

=

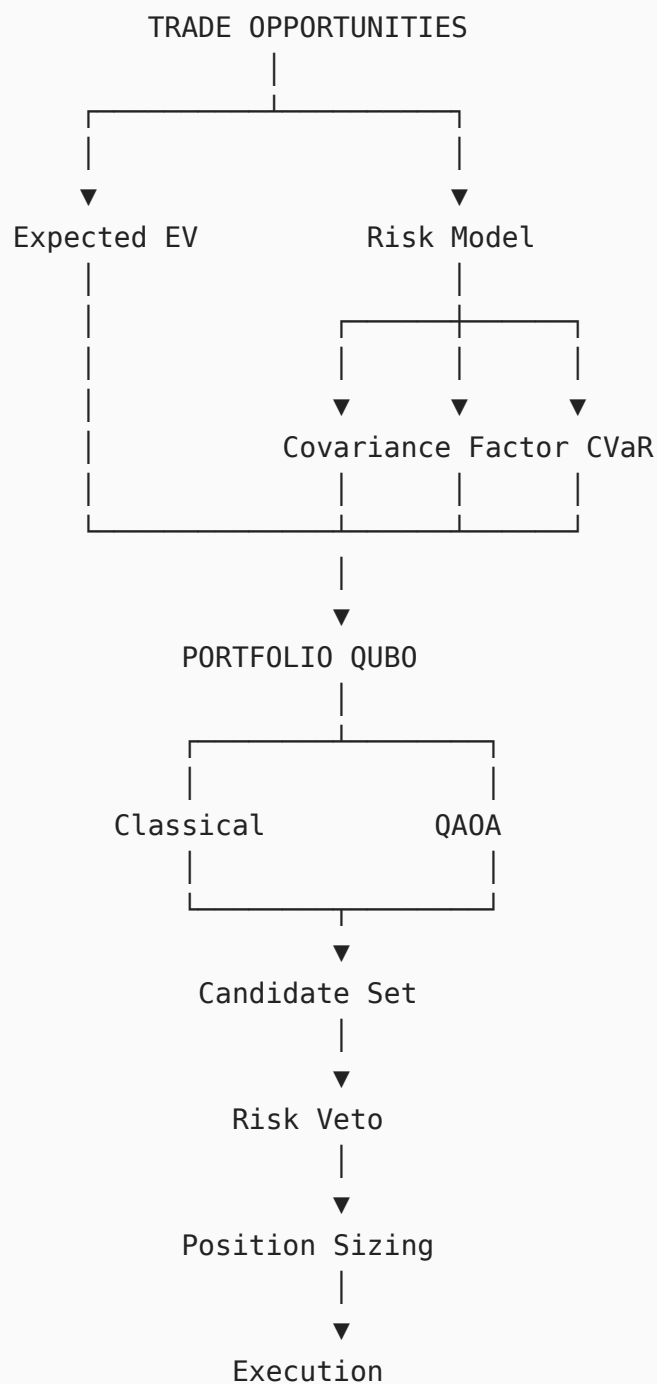
Karena objective yang salah akan menghasilkan solusi matematis yang benar tetapi keputusan finansial yang buruk.

Maka:

$tt \ t$

Risk management tetap menjadi layer independen.

AF.65. Arsitektur AF lengkap



AF.66. Contoh sederhana

Misalkan kita mempunyai lima opportunity:

’,’,’,’

Expected edge:

$$= 0000504$$

Secara individual:

Jika kita hanya melihat EV, kita akan memilih:

’,’

Tetapi ternyata:

$$, = 05,$$

$$, = 00$$

Sedangkan:

$$, = 01$$

Maka:

’,’

mungkin lebih optimal daripada:

’,’

AF.67. Inilah alasan optimizer quadratic diperlukan

Linear selection:

tidak dapat melihat:

Quadratic objective:

$$\boxed{-,}$$

dapat melihat interaction.

AF.68. Dan sekarang kita kembali ke tujuan awal kita

Tujuanmu bukan sekadar:

| membuat bot crypto.

Kita sedang membangun:

t

Dengan kemampuan menangani:

t

t

dan instrumen lain selama data dan execution model tersedia.

AF.69. Struktur universal versus asset-specific

Kita harus menjaga:

Universal

t

Asset-specific

tt

t

t

Dengan demikian arsitektur tetap universal tanpa berpura-pura bahwa crypto dan saham memiliki microstructure identik.

AF.70. Dan sekarang hubungan dengan skripsimu menjadi sangat jelas

Dalam penelitianmu, kita mempunyai:

$$t$$

Pada sistem trading ini kita memperluasnya menjadi:

$$tt \quad tt \quad t \quad Pt$$

Jadi quantum optimization bukan tempelan.

Ia muncul **secara alami dari struktur masalah**.

AF.71. Persamaan inti Tahap AF

Kalau seluruh tahap ini harus diringkas menjadi satu persamaan:

$$t =_{\in\{0,1\}} - t \, t \, t^2 - t^{-11} P^{-2}$$

Kemudian:

$$t$$

Ini adalah **jantung matematis Tahap AF**.

AF.72. Namun ada satu hal yang sengaja belum kita lakukan

Kita belum memasukkan:



secara penuh ke dalam optimizer.

Saat ini:

hanya menjawab:

| **ambil / jangan ambil.**

Sedangkan:

akan menjawab:

| **seberapa besar posisi?**

Dan ketika kita menggabungkan:

dan:

,

kita mendapatkan problem **mixed discrete-continuous optimization** yang jauh lebih menarik:

$$\in \{0, 1\}, \in$$

Di situlah kita dapat mulai membangun **AF+ / Tahap AG** secara lebih dalam: bagaimana menggabungkan **binary QUBO selection + continuous Markowitz sizing + CVaR + factor neutrality + \$1 stop-loss risk + leverage + liquidity constraints** menjadi satu hierarchical optimizer.

Dan setelah itu, baru kita dapat benar-benar menjawab pertanyaan paling penting dalam sistem kita:

???

Itulah titik di mana sistem kita mulai berubah dari sekadar **trade signal generator** menjadi **portfolio-level quantitative trading engine**.

Prompt:

8/17/2026, 10:22:31 PM

lanjut Tahap AG

Response:

8/17/2026, 10:22:32 PM

Tahap AG — Mixed-Integer Portfolio Optimization: dari “Trade mana?” menjadi “Trade mana + seberapa besar?”

Tahap AG adalah kelanjutan langsung dari AF. Di AF kita sudah mempunyai variabel biner

$$\in \{0, 1\},$$

yang menjawab:

| Apakah opportunity dipilih?

Sekarang kita tambahkan variabel kontinu

$$\in ,$$

yang menjawab:

Jika dipilih, seberapa besar modal/risk yang dialokasikan?

Maka sistem kita berubah dari **binary portfolio selection** menjadi:

□

dengan struktur fundamental:

$$\in \{0, 1\}, \in$$

AG.1. Dua variabel yang memiliki tugas berbeda

Untuk setiap opportunity :

$$= \begin{cases} 1, & 0, \end{cases}$$

sedangkan:

menyatakan exposure atau position weight.

Keduanya harus dihubungkan.

Jika:

$$= 0,$$

maka kita ingin:

$$= 0$$

Jika:

$$= 1,$$

maka:

boleh mengambil nilai tertentu.

Secara matematis:

□

dengan sebagai maximum position size.

AG.2. Mengapa constraint ini sangat penting?

Tanpa constraint:

,

optimizer bisa menghasilkan:

$$= 0, \quad 0$$

Artinya portfolio mengatakan:

"Saya tidak memilih trade ini, tetapi tetap memiliki posisi di dalamnya."

Tidak masuk akal.

Constraint tersebut membuat:

$$= 0 \quad = 0$$

Sedangkan:

$$= 1$$

AG.3. Portfolio return

Sekarang expected portfolio return menjadi:

$$=$$

atau:

$$=$$

Berbeda dari AF.

Di AF:

menganggap setiap trade mempunyai kontribusi binary.

Sekarang:

memperhitungkan **berapa besar exposure** yang kita berikan.

AG.4. Portfolio variance

Sekarang:

$$^2 =$$

Sehingga objective Markowitz:

$$=$$

Tetapi sekarang kita juga mempunyai .

Maka:

$$\begin{matrix} & - \\ , & \end{matrix}$$

subject to:

AG.5. Ini adalah inti mathematical architecture kita

Kita sekarang mempunyai:

$$\square$$

sehingga:

$$\square$$

dapat dioptimalkan secara simultan.

AG.6. Tetapi ada masalah besar

Bayangkan:

$$= 1000$$

Kita memiliki:

$$1000$$

variabel binary dan:

$$1000$$

variabel continuous.

Maka problem menjadi jauh lebih sulit daripada QUBO biasa.

Secara umum:

$$P = ttP$$

AG.7. Apakah kita harus langsung memasukkan semuanya ke QAOA?

Menurut saya:

□

Ini keputusan desain yang penting.

Kita jangan memaksakan:

Justru kita gunakan paradigma hybrid:

t

sesuai struktur masalah.

AG.8. Hierarchical decomposition

Kita dapat memecahkan:

,

menjadi dua tahap:

Tahap 1

Cari:

Tahap 2

Diberikan , cari:

Secara matematis:

$=$

kemudian:

$=$

AG.9. Mengapa pendekatan ini sangat masuk akal?

Karena discrete problem menjawab:

| "Apa yang kita percaya layak diperdagangkan?"

Sedangkan continuous problem menjawab:

| "Bagaimana kita mengalokasikan risk?"

Dua pertanyaan tersebut memang berbeda secara konseptual.

AG.10. Position sizing berbasis stop-loss

Sekarang kita hubungkan dengan sistem \$ \$1 \$ risk yang kita desain sebelumnya.

Misalkan entry:

$$P_t$$

dan stop-loss:

$$P$$

Distance:

$$= P_t - P$$

Untuk satu unit posisi, maximum loss:

$$= ,$$

dengan adalah nilai moneter per unit perubahan harga.

Jika risk budget:

$$= 1,$$

maka:

$$\boxed{= 1}$$

AG.11. Jika portfolio menggunakan risk unit

Lebih fleksibel jika kita mendefinisikan:

sebagai risk budget trade dalam satuan .

Misalnya:

$$= 1$$

Kemudian:

$$= t$$

Jadi:

$$\boxed{Pt = ttt \ t}$$

Ini adalah fondasi risk-based position sizing.

AG.12. Sekarang dapat kita definisikan dalam risk-space

Daripada:

$$= t,$$

kita bisa mendefinisikan:

$$=$$

Misalnya:

$$= 0.15$$

berarti BTC menerima:

$$15$$

dari risk budget tertentu.

Ini jauh lebih konsisten dengan sistem trading kita daripada sekadar equal capital weighting.

AG.13. Risk contribution

Untuk portfolio:

$$= ,$$

marginal contribution to risk dari asset :

$$=$$

Karena:

$$= ,$$

maka:

$$=$$

Sehingga:

$$=$$

AG.14. Total risk contribution

Risk contribution:

=

Maka:

$$=$$

Dan:

=

Ini sangat berguna untuk mengetahui:

Trade mana yang sebenarnya menyumbang risiko terbesar?

Karena position terbesar belum tentu mempunyai risk contribution terbesar.

AG.15. Contoh

Misalkan:

$$= 02$$

dan:

$$= 01$$

Tetapi A dan B sangat berkorelasi.

Mungkin:

$$= 015$$

dan:

$$= 012$$

Jadi B yang hanya setengah posisi A ternyata menyumbang risiko hampir sama.

Inilah alasan:

$$P_{tt} \quad t$$

AG.16. Risk parity sebagai alternatif

Kita dapat meminta:

Untuk seluruh trade:

$$\square$$

Ini disebut **risk parity**.

Tetapi untuk sistem kita, saya tidak menyarankan risk parity sebagai objective utama.

Lebih baik menjadikannya:

$$\boxed{t}$$

Karena kita tetap ingin mempertahankan expected edge.

AG.17. Objective sizing yang lebih lengkap

Setelah ditentukan:

$$\boxed{-}$$

subject to:

$$\begin{aligned} & , \\ & , \end{aligned}$$

dan:

AG.18. Leverage constraint

Jika portfolio mempunyai leverage maksimum:

$$,$$

maka:

$$\boxed{}$$

Misalnya:

$$= 2$$

Maka total gross exposure tidak boleh melebihi:

$$2$$

AG.19. Long-short portfolio

Untuk sistem yang mendukung short:

$$\in -,$$

Kita dapat memecah:

$$= - _,$$

dengan:

$$\begin{matrix} 0, \\ - 0 \end{matrix}$$

Dan:

$$= -$$

AG.20. Constraint selection-sizing

Kita dapat menggunakan:

$$-$$

Sehingga jika:

$$\begin{matrix} = 0 \\ = - = 0 \end{matrix}$$

Jika:

$$= 1$$

posisi dapat long atau short.

AG.21. Factor neutrality

Misalkan:

adalah factor loading matrix.

Portfolio exposure:

$$\boxed{=}$$

Jika kita ingin market-neutral:

$$\boxed{_t = 0}$$

Jika kita ingin USD-neutral:

$$\boxed{= 0}$$

Atau hanya membatasi:

AG.22. Ini sangat relevan dengan desain kita

Misalnya model menemukan:

10

saham yang semuanya bullish.

Tetapi ternyata semuanya mempunyai:

0

Artinya portfolio sebenarnya memiliki satu hidden macro bet:

t

Kita bisa mengontrolnya secara eksplisit.

AG.23. Gold factor

Hal yang sama berlaku untuk gold.

Jika:

terlalu besar, berarti portfolio mempunyai exposure besar terhadap factor gold.

Kita bisa:

Atau bahkan:

0

AG.24. Faktor bukan hanya untuk risk

Ini penting.

Factor exposure juga bisa menjadi **signal**.

Misalnya:

dan model kita memperkirakan aset tertentu akan underperform.

Maka:

dapat dibuat conditional:

$$= t$$

Dengan demikian factor model masuk ke:

$$Pt$$

dan:

$$\square$$

AG.25. Conditional covariance

Lebih lanjut:

$$t$$

juga tidak harus konstan.

Kita dapat memiliki:

$$t = t, t t$$

Jadi ketika volatility regime berubah:

$$,t$$

ikut berubah.

AG.26. Regime-dependent optimization

Misalkan terdapat regime:

$$t \in \{, , , , \}$$

Maka:

$$t = t$$

dan:

$$t = t$$

Optimizer menjadi:

$$\boxed{t = t}$$

AG.27. Ini membuat sistem adaptif

Contohnya:

Risk-on

$$t \ 0$$

$$t \ 0$$

volatility rendah.

Optimizer mungkin memperbolehkan:

Risk-off

Volatility meningkat.

Optimizer mengurangi:

$$1$$

AG.28. Dynamic risk budget

Bahkan:

dapat dibuat fungsi regime:

$$\boxed{,t = 0t}$$

Misalnya:

$$t \ 1$$

ketika:

$$tt$$

sangat tinggi.

Ini berarti sistem otomatis melakukan **risk compression**.

AG.29. Sekarang kita kembali ke \$1 risk

Ada dua level:

Trade level

$$1$$

Portfolio level

$$t$$

Keduanya harus dipertahankan.

Misalnya:

$$= 1$$

untuk setiap trade.

Jika:

$$t = 5,$$

maka sistem tidak boleh membuka 20 trade independen sekaligus jika aggregate risk melebihi budget.

AG.30. Correlation-adjusted risk

Worst-case:

$$t =$$

Tetapi expected aggregate risk lebih kecil jika trade independen.

Secara sederhana:

$$= \sigma^2$$

Inilah alasan correlation matters.

AG.31. Bahkan risk \$1 dapat dibuat adaptive

Tidak harus selalu:

$$1$$

Kita dapat mendefinisikan:

$$t = 0, t, t_{t,t}$$

Misalnya:

$$t = 1$$

dalam kondisi normal.

Tetapi ketika drawdown meningkat:

$$t > 1$$

AG.32. Drawdown-aware risk scaling

Misalkan equity:

$$E_t$$

High-water mark:

$$H_t = \max_{\tau \leq t} E_\tau$$

Drawdown:

$$D_t = 1 - \frac{E_t}{H_t}$$

Kemudian:

$$f_t = f_0 (1 - D_t)^\alpha$$

Contoh:

$$\alpha = 1, \quad f_t = f_0 (1 - D_t)$$

Jadi semakin besar drawdown:

$$f_t \text{ semakin kecil}$$

AG.33. Ini merupakan feedback control

Kita sekarang memiliki:

$$\dot{P}_t = P_t - P_{t-1}$$

Dengan demikian sistem menjadi:

$$\dot{P}_t = -\alpha P_t$$

AG.34. Transaction cost

Sizing tidak boleh mengabaikan biaya.

Misalkan:

adalah expected transaction cost per unit exposure.

Maka:

$$t =$$

Objective:

$$\boxed{\quad \quad}$$

AG.35. Market impact

Untuk trade yang lebih besar, cost tidak lagi linear.

Model sederhana:

$$t = _2$$

Maka:

$$t = _2$$

Ini membuat optimizer memperhitungkan liquidity.

AG.36. Mengapa ini penting untuk short-term trading?

Karena pada horizon pendek:

$$t$$

bisa sangat kecil.

Misalnya:

$$= 015$$

Tetapi:

$$tt = 00$$

Maka net edge:

$$00$$

Jika slippage:

$$010,$$

trade tersebut sebenarnya:

$$t \ 0$$

AG.37. Maka expected value kita harus diperbaiki

Jangan menggunakan:

Gunakan:

$$t = -t -$$

Untuk crypto:

bisa penting.

Untuk saham:

t

dapat relevan jika short.

AG.38. Objective final sizing

Kita sekarang dapat menulis:

$$- - -$$

subject to:

,

,

$\in$,

AG.39. Interpretasi setiap term

= expected profit.

= portfolio risk.

= transaction cost.

= market impact.

Constraint:

= selection control.

Constraint:

$\in$

= factor exposure control.

AG.40. Sekarang kita mempunyai dua optimizer

Optimizer A

tt

memilih:

Optimizer B

tt

memilih:

AG.41. Tetapi kita bisa melakukan sesuatu yang lebih menarik

Setelah mendapatkan:

,

kita dapat melakukan **iterative refinement**.

Pertama:

$0 \ 0$

Kemudian hitung:

0

Jika terlalu tinggi, kita kembali ke selection:

1

Lalu:

1 1

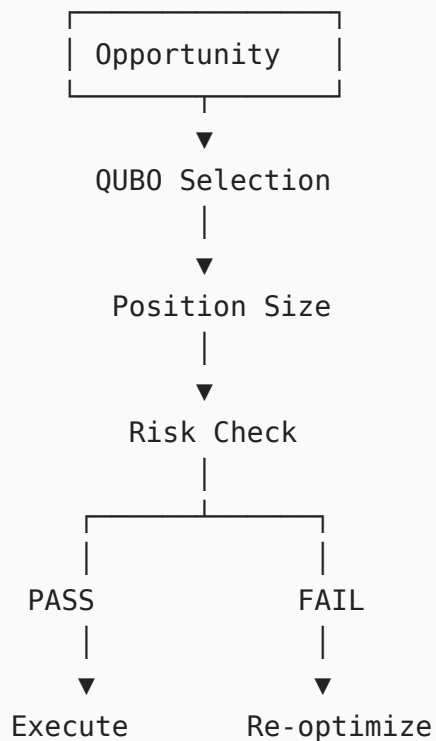
Dan seterusnya.

AG.42. Jadi bukan hanya pipeline

Bukan:

QUBO → Sizing → selesai

tetapi:



AG.43. Ini merupakan decomposition algorithm

Secara konseptual:

$t \ t \ t$

Kita bisa menggunakan pendekatan seperti:

- alternating optimization,
- Benders-style decomposition,
- coordinate refinement,

- heuristic local search.

Tidak semuanya harus quantum.

AG.44. Peran QAOA menjadi semakin jelas

QAOA tidak perlu mengoptimalkan:

secara langsung.

Ia dapat mengoptimalkan:



Kemudian classical optimizer menangani:



Ini membuat penggunaan QAOA jauh lebih realistis.

AG.45. Pipeline matematis Tahap AG

Sekarang sistem kita dapat dirumuskan:



AG.46. Contoh numerik kecil

Misalkan ada tiga trade:

’, ’

Expected return:

$$= 0.1201000$$

Covariance:

$$= \begin{bmatrix} 0.04 & 0.00 & 0.0000 \\ 0.00 & 0.05 & 0.0100 \\ 0.0000 & 0.0100 & 0.02 \end{bmatrix}$$

Misalkan optimizer memilih:

$$= \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Kemudian sizing mencari:

Dengan constraint:

$$= 1$$

dan:

$$0$$

AG.47. Unconstrained Markowitz

Tanpa constraint lain, problem:

$$-$$

First-order condition:

$$- = 0$$

Karena:

$$=$$

dan:

$$= 2,$$

maka:

$$- 2 = 0$$

Sehingga:

$$\boxed{= 12^{-1}}$$

Ini adalah solusi dasar Markowitz.

AG.48. Tetapi constraint budget mengubahnya

Jika:

$$= 1,$$

kita gunakan Lagrange multiplier:

$$= -1 -$$

FOC:

$$-2 - = 0$$

Sehingga:

$$\boxed{= 12^{-1} -}$$

kemudian ditentukan dari:

$$= 1$$

AG.49. Kenapa derivasi ini penting?

Karena kita sekarang memiliki pembagian kerja:

Quantum/discrete

Menentukan:

Classical analytical/numerical

Menentukan:

Dengan demikian kita dapat menggunakan matematika yang sudah kamu pelajari dalam Markowitz secara langsung.

AG.50. Tetapi untuk trading jangka pendek, Markowitz saja belum cukup

Karena:

sangat noisy.

Maka kita tidak boleh memperlakukan:

seolah-olah exact.

Kita harus memperhitungkan uncertainty:

$$=$$

AG.51. Robust optimization

Salah satu pendekatan:

$$\in$$

Kita tidak mengoptimalkan:

saja, tetapi worst-case expected return:

$$\in -$$

Ini jauh lebih masuk akal untuk short-horizon prediction.

AG.52. Confidence-adjusted expected return

Alternatif sederhana:

$$= -$$

dengan:

sebagai uncertainty estimate.

Jadi:

AG.53. Ini sangat penting bagi sistem kita

Sebuah trade bisa memiliki:

$$= 0.20$$

tetapi uncertainty:

$$= 0.0$$

Trade tersebut sebenarnya kurang menarik dibanding trade:

$$= 0.15, = 0.05$$

Maka:

AG.54. Confidence masuk ke QUBO

Kita sekarang dapat mengganti:

dengan:

$$t$$

Sehingga QUBO memilih bukan:

trade dengan predicted return terbesar,

tetapi:

trade dengan risk-adjusted, uncertainty-adjusted edge terbesar.

AG.55. Ini menghasilkan objective yang lebih matang

Discrete stage:

$$- \sum_t \sum_{t-1} P^{-2}$$

Continuous stage:

$$t - t - t$$

subject to risk and exposure constraints.

AG.56. Satu lagi masalah: turnover

Misalkan:

$$t-1$$

adalah portfolio sebelumnya.

Maka perubahan:

$$\Delta_t = t - t-1$$

Transaction cost:

$$t_t = \Delta_t$$

Jadi objective sebenarnya harus menggunakan:

$$t_t - t_t - t - t-1$$

Ini membuat portfolio **sticky**.

AG.57. Ini sangat cocok dengan sistem evaluasi periodik kita

Kamu sebelumnya menginginkan:

| setiap jam atau periode tertentu posisi dievaluasi.

Sekarang kita punya formulasi matematisnya.

Pada waktu t :

$$t-1$$

adalah portfolio saat ini.

Model menghitung:

$$t$$

Kemudian:

$$\Delta_t = t - t-1$$

Jika:

$$\Delta_t$$

kecil, kita tidak perlu banyak melakukan perubahan.

AG.58. Bahkan kita bisa membuat minimum improvement threshold

Misalkan:

$$t - t-1$$

Maka:

$$\square$$

Ini sangat penting agar optimizer tidak overtrade.

AG.59. Decision rule

Secara sederhana:

$$\Delta = t - t-1$$

Jika:

$$\Delta ,$$

rebalancing.

Jika:

$$\Delta ,$$

pertahankan portfolio.

AG.60. Ini menghubungkan AG dengan sistem position management sebelumnya

Kita sebelumnya membicarakan:

- floating loss,
- floating profit,
- BEP,
- trailing SL,
- TP extension.

Sekarang kita dapat melihatnya sebagai:

$$P_{tt} = tt$$

sedangkan:

$$P_{tt} = tt$$

AG.61. Dua skala optimisasi

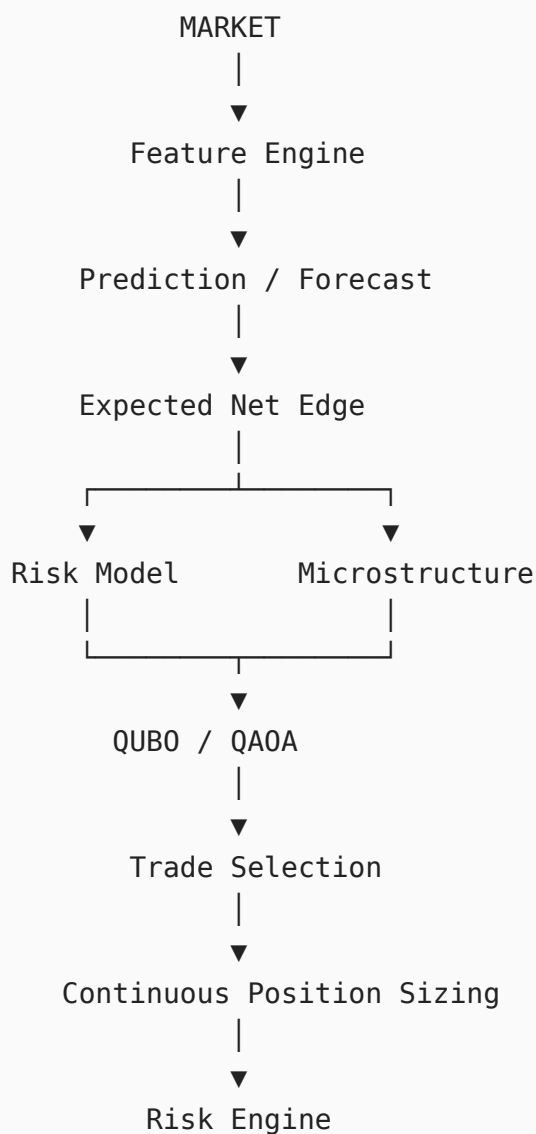
Global

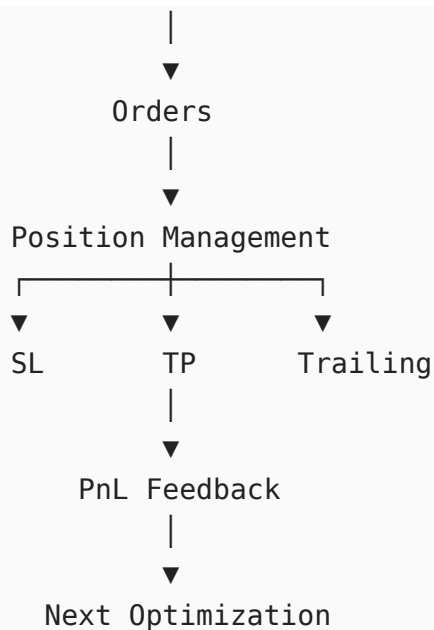
Local

P

Jangan dicampur menjadi satu optimizer raksasa.

AG.62. Arsitektur akhirnya mulai terlihat





AG.63. Poin terpenting Tahap AG

Kita sekarang mempunyai **tiga lapisan optimisasi**:

1	2
---	---

dan kemudian:

4

AG.64. Hubungan dengan TSP, Max-Cut, dan problem combinatorial lainnya

Sekarang menjadi lebih jelas mengapa kamu sejak awal tertarik pada:

- TSP,
- Max-Cut,
- portfolio optimization.

Mereka tidak harus menjadi tiga algoritma yang berdiri sendiri.

Mereka merupakan **primitive optimization structures**.

Misalnya:

P_{tt}

$tt \ t$

dan untuk masalah execution:

$$t \ PP$$

AG.65. Bahkan execution bisa menjadi optimization problem

Misalnya kita memiliki:

venue/order types.

Kita ingin menentukan:

dengan:

- latency,
- fee,
- liquidity,
- slippage.

Maka kita dapat memiliki:

$$ttt$$

Ini adalah layer yang berbeda dari portfolio selection.

AG.66. Maka sistem besar kita mulai menjadi

$$Pt \ Pt \ t \ Ptt$$

Dengan berbagai mathematical optimization primitive:

$$t \ tt \ t$$

AG.67. Dan ada prinsip desain yang sangat penting

Jangan menggunakan quantum algorithm hanya karena:

"masalahnya bisa dibuat QUBO."

Pertanyaan yang benar adalah:

□

Karena sistem kita harus mempunyai benchmark:

VS.

t

VS.

PP

VS.

VS.

t

AG.68. Ini nantinya akan menjadi eksperimen yang sangat kuat

Untuk setiap periode:

$t,$

kita dapat menjalankan:

t

dan:

t

Kemudian bandingkan:

,

,

,

,

,

tt

Dengan demikian kita tidak mengatakan:

"QAOA lebih bagus karena quantum."

Tetapi:

"Pada problem dan constraint tertentu, QAOA menghasilkan solusi dengan kualitas X dan computational characteristic Y dibanding baseline Z."

Ini jauh lebih ilmiah.

AG.69. Rumusan matematis AG yang paling lengkap

Untuk sekarang, saya akan menjadikan formulasi berikut sebagai **master formulation**:

$$x_t - x_{t-1} - t^{-2} \in \{0, 1\}, \quad x_t \in [0, 1]$$

Ini adalah problem yang secara konseptual sekarang ingin kita selesaikan.

AG.70. Tetapi jangan langsung coding formulasi sebesar ini

Ini penting.

Kita sebaiknya membangun secara bertahap:

AG-1

Binary selection:

AG-2

Selection + covariance:

AG-3

Selection + factor exposure.

AG-4

Continuous sizing:

AG-5

Transaction cost.

AG-6

CVaR.

AG-7

Dynamic rebalancing.

AG-8

QAOA benchmark.

Dengan demikian setiap komponen dapat diuji secara independen.

AG.71. Dan inilah keputusan desain yang saya sarankan

Untuk **versi pertama sistem nyata**, jangan langsung mencoba:

$1000t$

Mulai dengan:

$$= 10 - 0$$

opportunities.

Misalnya:

Crypto

BTC, ETH + 8–15 altcoins.

US equities

10–20 liquid stocks/ETFs.

Macro

Gold + USD proxy + equity index.

Kemudian baru:

50 100 500

AG.72. Mengapa?

Karena tujuan pertama bukan:

Tujuan pertama adalah:

□

Kita harus bisa membuktikan:

$Pt \quad t \quad t$

benar-benar bekerja tanpa leakage dan tanpa bug.

AG.73. Dan ada satu konsekuensi besar dari AG

Mulai tahap ini, **backtesting** kita tidak boleh lagi hanya mengevaluasi signal.

Kita harus melakukan:

$$t - ttt$$

Karena keputusan sekarang bergantung pada:

$$t, t, t-1, \dots, t, ttt$$

Jadi backtest harus mengetahui **state portfolio**, bukan hanya:

BUY
SELL
HOLD

AG.74. State sistem

Pada waktu t , state minimal:

$$t = t, t, P_{t, t, t}, P_{t, t, t}$$

Kemudian market memberikan:

$$t$$

Model menghasilkan:

$$t = t, t$$

Dan:

$$t1 = t, t, t$$

Ini sebenarnya sudah mulai menyerupai **stochastic control problem**.

AG.75. Dan itu akan menjadi jembatan kita ke Tahap berikutnya

Karena setelah kita mempunyai:

$$t$$

dan:

$t,$

pertanyaan berikutnya adalah:

Bagaimana optimizer memilih portfolio ketika kondisi pasar berubah secara terus-menerus dan posisi yang sudah ada memiliki state masing-masing?

Dengan kata lain:

$tttt\ tt$

Dan di sinilah kita akan mulai memasukkan secara matematis:

$tttt\ tt$

Itu akan menjadi fondasi yang sangat penting untuk **Tahap AH**, karena pada titik tersebut sistem kita tidak lagi sekadar mencari portfolio optimal pada satu timestamp, tetapi mulai memecahkan:

□

Prompt:

8/17/2026, 10:24:06 PM

aku kira kita sudah membahas terlalu banyak. menurutku tahap AH tidak perlu dilanjutkan lagi. bagaimana menurutmu?

Response:

8/17/2026, 10:24:06 PM

Ya. **Saya justru setuju bahwa kita sebaiknya berhenti di AG untuk sementara.** Kalau diteruskan ke AH, kita berisiko masuk ke wilayah *over-engineering* sebelum fondasi yang sudah kita bangun benar-benar diuji.

Yang sudah kita punya sekarang sebenarnya sudah sangat besar.

Posisi kita saat ini

Secara konseptual, arsitektur sistem sudah mencakup:

□

kemudian:

□

kemudian:

□

dan akhirnya:



Bahkan di AG kita sudah mencapai formulasi yang cukup serius:

$$, t - t - t - t - 1 - t , \in \{0, 1\}, \quad , \quad ,$$

Itu sudah cukup untuk menjadi **arsitektur penelitian sekaligus sistem trading yang nyata**.

Justru sekarang waktunya pindah dari "merancang" ke "membangun"

Menurut saya, kalau kita terus membuat:

| AG → AH → AI → AJ → AK → ...

kita akan memperoleh sistem yang secara matematis semakin indah, tetapi belum tentu semakin berguna.

Lebih baik sekarang kita melakukan:



Bukan:

Yang menurut saya sebaiknya kita lakukan sekarang

Saya akan membagi pekerjaan setelah AG menjadi **fase implementasi**, bukan lagi tahap AH.

Fase 1 — Mathematical specification

Kita bekukan dulu seluruh formulasi:

- universe aset,
- return,
- factor,
- expected return,
- covariance,
- QUBO,
- constraints,
- position sizing,
- transaction cost,

- slippage,
- risk budget,
- SL/TP,
- rebalancing.

Fase 2 — Classical baseline

Sebelum QAOA:

$$t \text{ } PP$$

sebagai pembanding.

Ini penting sekali.

Fase 3 — Quantum formulation

Baru kita implementasikan:

$$\square$$

dan bandingkan dengan classical solver.

Fase 4 — Event-driven backtest

Kemudian seluruh keputusan dimasukkan ke simulator:

$$t \text{ } t \text{ } t1$$

Termasuk:

- limit order,
- order tidak filled,
- partial fill,
- SL,
- TP,
- trailing,
- fee,
- slippage,
- funding,
- latency jika diperlukan.

Fase 5 — Walk-forward validation

Ini **sangat penting** untuk sistem kita.

Jangan:

sekali saja.

Lebih baik:

$t \quad t$

berulang mengikuti waktu.

Dan ada satu hal yang saya ingin kita pertahankan

Jangan sampai karena kita tertarik pada QAOA, kita akhirnya membuat sistem yang:

"quantum-first."

Menurut saya paradigma yang jauh lebih kuat adalah:

”

Artinya kita mulai dari pertanyaan:

Apa masalah finansialnya?

Kemudian:

Apa formulasi matematis terbaiknya?

Kemudian:

Classical algorithm apa yang paling kuat?

Baru:

Apakah QAOA/quantum optimization memberikan sesuatu yang menarik?

Dengan pendekatan ini, bahkan jika ternyata QAOA tidak mengalahkan solver klasik untuk ukuran problem tertentu, **penelitian kita tetap valid**.

Jadi keputusan saya

Stop di AG. Jangan lanjut AH sekarang.

Kita sudah mempunyai cukup banyak teori untuk mulai membangun sesuatu yang konkret.

Dan menurut saya langkah berikutnya bahkan **bukan menambah matematika lagi**, melainkan mengambil seluruh desain A–AG yang sudah kita bangun dan mengubahnya menjadi:

□

yang sangat konkret:

```
data/  
features/  
factors/  
signals/  
optimization/  
    qubo/  
    ising/  
    classical/  
    qaoa/  
portfolio/  
    sizing/  
    risk/  
execution/  
backtest/  
evaluation/
```

lalu menentukan **input** → **proses** → **output** → **persamaan** → **algoritma** → **unit test** untuk setiap modul.

Itu akan menjadi titik di mana proyek kita berubah dari "**ide quantitative trading yang sangat kompleks**" menjadi **sistem yang benar-benar bisa kita implementasikan dan diuji**.